AIClassDocuments

Giáo trình Python, Toán, Thống kê, Machine Learning và AI

46 chương · Song ngữ Anh - Việt

Bản xem trước để học tập và rà soát. Chưa phải chứng nhận học thuật độc lập.

Mục lục / Contents

PY-01 - PY-01 — Biến, đầu vào và đầu ra trong Python

PY-02 - PY-02 — Kiểu số, Boolean và chuỗi

PY-03 - PY-03 — Toán tử số học, so sánh và logic

PY-04 - PY-04 — Câu lệnh điều kiện if, elif, else

PY-05 - PY-05 — Vòng lặp for, while, break và continue

PY-06 - PY-06 — List, chỉ số, cắt danh sách và sắp xếp

PY-07 - PY-07 — Tuple, Set và các phép toán tập hợp

PY-08 - PY-08 — Dictionary: khóa, giá trị và thêm sửa xóa

PY-09 - PY-09 — Hàm, return, lambda và phạm vi biến

PY-10 - PY-10 — Class, Object, constructor và đóng gói

PY-11 - PY-11 — Kế thừa, đa hình và lớp trừu tượng ABC

PY-12 - PY-12 — File I/O, CSV và xử lý ngoại lệ

PY-13 - PY-13 — NumPy ndarray, shape, dtype và axis

PY-14 - PY-14 — NumPy indexing, slicing, boolean và fancy indexing

PY-15 - PY-15 — NumPy tổng hợp, axis và broadcasting

PY-16 - PY-16 — NumPy reshape, transpose, ma trận và lưu file

PY-17 - PY-17 — Pandas Series, DataFrame và đọc CSV

PY-18 - PY-18 — Pandas: missing values, duplicates và kiểu dữ liệu

PY-19 - PY-19 — Pandas: lọc, groupby, merge và sắp xếp

PY-20 - PY-20 — Logic mệnh đề, bảng chân trị và quy tắc đếm

PY-21 - PY-21 — Tập hợp, bao hàm–loại trừ và phân hoạch

PY-22 - PY-22 — Hoán vị, tổ hợp và tam giác Pascal

PY-23 - PY-23 — Đồ thị, danh sách kề và Jaccard similarity

PY-24 - PY-24 — Scalar, vector, norm, đơn vị và dot product

PY-25 - PY-25 — Ma trận, chuyển vị, nhân ma trận và định thức

PY-26 - PY-26 — Hệ phương trình tuyến tính, rank và nghiệm

PY-27 - PY-27 — Eigenvalue, Eigenvector và hướng biến thiên

PY-28 - PY-28 — Hàm tuyến tính, bậc hai, đa thức và hàm mũ

PY-29 - PY-29 — Activation: ReLU, sigmoid, tanh, softmax

PY-30 - PY-30 — Loss: MSE, MAE, Huber, cross-entropy và KL

PY-31 - PY-31 — Giới hạn, tính liên tục và đạo hàm

PY-32 - PY-32 — Đạo hàm riêng, gradient descent và learning rate

PY-33 - PY-33 — Jacobian, Hessian và điểm cực trị

PY-34 - PY-34 — Xác suất, không gian mẫu và biến cố

PY-35 - PY-35 — Xác suất có điều kiện, Bayes và xác suất toàn phần

PY-36 - PY-36 — Phân phối nhị thức và phân phối chuẩn

PY-37 - PY-37 — Thống kê mô tả: mean, median, mode, variance và std

PY-38 - PY-38 — Quantile, IQR, outlier, skewness và kurtosis

PY-39 - PY-39 — Lấy mẫu, CLT và sai số chuẩn

PY-40 - PY-40 — Kiểm định giả thuyết, p-value và khoảng tin cậy

PY-41 - PY-41 — Phân hoạch dữ liệu, classification và clustering

PY-42 - PY-42 — Tiền xử lý văn bản, token và Jaccard trong NLP

PY-43 - PY-43 — Git: quản lý phiên bản và hợp tác trên mã nguồn

PY-44 - PY-44 — Ứng dụng CLI: CRUD, kiểm tra dữ liệu và thống kê hồ sơ

PY-45 - PY-45 — Matplotlib: biểu đồ line, scatter và histogram

PY-46 - PY-46 — Mini project: dữ liệu → làm sạch → mô hình → đánh giá

PY-01 — Biến, đầu vào và đầu ra trong Python

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Variable — biến; **Input** — đầu vào.

`Chương trình → Biến → Kiểu dữ liệu → Điều kiện`

Học xong dùng để làm gì? Nhận tên, tuổi hoặc số lượng từ người dùng, lưu vào biến và in ra kết quả có thể kiểm tra.

Kết nối kiến thức tiếp theo: Sang PY-02, dùng kiểu số và chuỗi để dữ liệu trong biến được tính toán đúng thay vì chỉ hiển thị.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Variable	biến	Tên dùng để tham chiếu một giá trị
Input	đầu vào	Thông tin đưa vào chương trình
Output	đầu ra	Thông tin chương trình tạo ra
Type	kiểu dữ liệu	Nhóm giá trị như str, int, float

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`print()`	Hiển thị kết quả, nhận nhiều giá trị phân cách bằng dấu phẩy
`input()`	Nhận văn bản từ bàn phím; kết quả luôn là chuỗi
`int()/float()`	Đổi chuỗi hợp lệ sang số trước khi tính toán
`type()`	Kiểm tra kiểu dữ liệu khi chưa rõ đầu vào

4. Khái niệm liên quan

Một chương trình thường đi theo mạch **nhận thông tin → lưu vào biến → xử lý → đưa ra kết quả**. Bài này bắt đầu từ biến và cho thấy đầu vào, đầu ra và kiểu dữ liệu phối hợp ra sao trong mạch đó.

4.1. Variable — Biến

Từ vựng trong mục: Variable — biến; **Assignment** — phép gán.

Biến là tên để chương trình ghi nhớ một giá trị. Khi viết `total = 12`, Python gắn tên `total` với số `12`; dòng tiếp theo đọc lại giá trị qua tên đó. Biến giúp ta dùng lại dữ liệu và thay đổi dữ liệu ở một chỗ thay vì viết số trực tiếp khắp chương trình.

```
total = 12
print(total)
```

Kết quả khi chạy:

```
12
```

`total` là tên biến, còn `12` là giá trị được in ra.

4.2. Input — Đầu vào

Từ vựng trong mục: Input — đầu vào; **String** — chuỗi.

Đầu vào là dữ liệu chương trình nhận để xử lý: có thể từ bàn phím, biểu mẫu hoặc tệp. Ví dụ, biểu mẫu nhận tên khách `An`, ứng dụng lưu tên đó trong biến `name`. Để ví dụ tự chạy không cần gõ phím, ta mô phỏng dữ liệu đã nhận bằng một phép gán; khi nhận trực tiếp từ bàn phím, có thể dùng `input()`.

```
name = 'An'
print(name)
```

Kết quả khi chạy:

```
An
```

Chương trình in đúng tên đã nhận. Biến có thể đến từ nhiều nguồn, không chỉ bàn phím.

4.3. Output — Đầu ra

Từ vựng trong mục: Output — đầu ra; **Print** — hiển thị.

Đầu ra là thông tin chương trình gửi ra ngoài sau khi xử lý. Đầu ra có thể xuất hiện trên màn hình, được lưu vào file hoặc gửi đến hệ thống khác. Trong bài căn bản, `print()` giúp nhìn thấy giá trị đang được tạo ra; chuỗi trong dấu nháy là nội dung cần hiển thị.

```
print('Xin chào')
```

Kết quả khi chạy:

```
Xin chào
```

`print()` hiển thị dòng chữ; dấu nháy chỉ là cú pháp Python, không xuất hiện trong kết quả.

4.4. Type — Kiểu dữ liệu

Từ vựng trong mục: Type — kiểu dữ liệu; **Float** — số thực.

Cùng một biến có thể chứa chữ hoặc số, nhưng **kiểu dữ liệu** quyết định phép xử lý nào phù hợp. Chẳng hạn mã đơn `"0007"` là chuỗi (`str`) cần giữ số 0 đầu; giá trị `2.5` là số thực (`float`) có thể dùng để tính toán. Hàm `type()` cho biết kiểu của một giá trị.

```
print(type(2.5))
```

Kết quả khi chạy:

```
<class 'float'>
```

`float` báo đây là số thực, không phải chuỗi ký tự `"2.5"`.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Chào khách mới

Từ vựng trong mục: Variable — biến.

Tình huống thực tế: Một cửa hàng muốn gửi lời chào có tên khách. Khách mới tên Lan.

Yêu cầu: Viết chương trình in `Xin chào Lan!`.

Vì sao chọn cách giải này? Lưu tên trong biến `name` rồi truyền biến vào f-string để tạo lời chào. Khi đổi khách chỉ cần sửa tên.

Quy trình giải:

1. Lưu tên khách vào `name`.
2. Chèn tên vào lời chào bằng f-string.
3. In lời chào.

```
name = "Lan"
# Dùng tên đã lưu, không viết lại tên trong thông báo.
print(f"Xin chào {name}!")
```

Kết quả khi chạy:

```
Xin chào Lan!
```

Kết quả: `Xin chào Lan!` — lời chào chứa đúng tên khách.

Tình huống 2 — Tính tiền hóa đơn

Từ vựng trong mục: Input — đầu vào.

Tình huống thực tế: Một cửa hàng bán 3 sản phẩm, mỗi sản phẩm giá 120.000 đồng. Phần mềm đang lưu giá và số lượng dưới dạng chữ `"120000"` và `"3"`.

Yêu cầu: Tính tổng số tiền cần thanh toán.

Vì sao chọn cách giải này? `int()` đổi chữ số thành số nguyên để phép nhân giá với số lượng cho ra tổng tiền.

Quy trình giải:

1. Đổi chuỗi giá thành số bằng `int()`.
2. Đổi chuỗi số lượng thành số bằng `int()`.
3. Nhân giá với số lượng và in kết quả.

```
price = int("120000")
quantity = int("3")
print(price * quantity)
```

Kết quả khi chạy:

```
360000
```

Kết quả: `360000` — khách phải trả 360.000 đồng.

Tình huống 3 — Hiển thị điểm

Từ vựng trong mục: Output — đầu ra.

Tình huống thực tế: Trung tâm đào tạo muốn hiển thị điểm của học viên An, đạt 8,5 điểm trên thang 10.

Yêu cầu: In tên và điểm theo mẫu `An: 8.5/10`.

Vì sao chọn cách giải này? F-string ghép tên, điểm và thang điểm thành một dòng dễ đọc.

Quy trình giải:

1. Lưu tên `An` và điểm `8.5`.
2. Dùng f-string ghép thông tin.
3. In kết quả.

```
name, score = "An", 8.5
print(f"{name}: {score}/10")
```

Kết quả khi chạy:

```
An: 8.5/10
```

Kết quả: `An: 8.5/10` — dòng thông tin gồm tên và điểm học viên.

Tình huống 4 — Ghi nhiệt độ

Từ vựng trong mục: Type — kiểu dữ liệu.

Tình huống thực tế: Một cảm biến báo nhiệt độ 36,5 °C. Nhân viên cần biết số đo được lưu bằng kiểu dữ liệu gì.

Yêu cầu: In kiểu dữ liệu và số đo nhiệt độ.

Vì sao chọn cách giải này? Số có phần thập phân được lưu dưới dạng `float`; `type()` cho biết kiểu dữ liệu.

Quy trình giải:

1. Lưu `36.5` vào biến `temperature`.
2. Dùng `type()` xem kiểu dữ liệu.
3. In kiểu dữ liệu cùng số đo.

```
temperature = 36.5
print(type(temperature), temperature)
```

Kết quả khi chạy:

```
<class 'float'> 36.5
```

Kết quả: `<class 'float'> 36.5` — kiểu số thực và giá trị 36,5; code chưa in đơn vị °C.

Tình huống 5 — Giữ mã đơn

Từ vựng trong mục: Variable — biến.

Tình huống thực tế: Một cửa hàng dùng mã đơn bốn chữ số, trong đó hai số `0` đầu tiên phải được giữ lại. Mã đơn hiện tại là `0007`.

Yêu cầu: Lưu và in mã chính xác là `0007`, kiểm tra mã vẫn bắt đầu bằng `00`.

Vì sao chọn cách giải này? Mã đơn là thông tin nhận diện, không phải số để tính toán; dùng chuỗi `str` để không làm mất các số `0` ở đầu.

Quy trình giải:

1. Lưu `"0007"` thành chuỗi trong `order_id`.
2. In mã đơn.
3. Dùng `startswith("00")` kiểm tra hai ký tự đầu; `assert` báo lỗi nếu không đúng.

```
order_id = "0007"
print(order_id)
assert order_id.startswith("00")
```

Kết quả khi chạy:

```
0007
```

Kết quả: `0007` — mã được in nguyên vẹn; `assert` không in thêm khi kiểm tra đúng.

6. Những nhầm lẫn thường gặp

- `input()` nhận chữ; cần đổi sang `int()` hoặc `float()` khi muốn tính toán với số. Chuỗi không hợp lệ có thể gây `ValueError`.
- Mã đơn `0007` phải lưu dạng chuỗi nếu cần giữ hai số `0` ở đầu.
- `print()` in kết quả, còn `type()` xem kiểu dữ liệu; hai hàm có nhiệm vụ khác nhau.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a variable in Python? — Biến trong Python là gì?

A. A name referring to a value — Tên tham chiếu đến một giá trị

B. Information produced by a program — Thông tin do chương trình tạo ra

C. Data received for processing — Dữ liệu được nhận để xử lý

D. How a value is represented and used — Cách biểu diễn và sử dụng một giá trị

Đáp án: A

Giải thích: Biến Python là tên được liên kết với đối tượng, chẳng hạn `age = 20`. Biến khác dữ liệu đầu vào hay kết quả in ra; dùng `age` sau đó sẽ truy cập giá trị đang được gán.

Q2. What is program input? — Đầu vào chương trình là gì?

A. Information produced by a program — Thông tin do chương trình tạo ra

B. How a value is represented and used — Cách biểu diễn và sử dụng một giá trị

C. A name referring to a value — Tên tham chiếu đến một giá trị

D. Data received for processing — Dữ liệu được nhận để xử lý

Đáp án: D

Giải thích: Đầu vào là dữ liệu chương trình nhận, ví dụ văn bản người dùng nhập cho `input()`. Chương trình có thể xử lý văn bản rồi tạo đầu ra; đầu vào không phải kết quả chương trình.

Q3. What is program output? — Đầu ra chương trình là gì?

A. How a value is represented and used — Cách biểu diễn và sử dụng một giá trị

B. Information produced by a program — Thông tin do chương trình tạo ra

C. Data received for processing — Dữ liệu được nhận để xử lý

D. A name referring to a value — Tên tham chiếu đến một giá trị

Đáp án: B

Giải thích: Đầu ra là thông tin chương trình đưa ra ngoài, ví dụ dòng chữ do `print(5)` hiển thị. Nó khác đầu vào do người dùng cung cấp và khác biến dùng để tham chiếu giá trị trong chương trình.

Q4. What does a data type describe? — Kiểu dữ liệu mô tả điều gì?

A. Information produced by a program — Thông tin do chương trình tạo ra

B. How a value is represented and used — Cách biểu diễn và sử dụng một giá trị

C. A name referring to a value — Tên tham chiếu đến một giá trị

D. Data received for processing — Dữ liệu được nhận để xử lý

Đáp án: B

Giải thích: Kiểu dữ liệu quyết định ý nghĩa giá trị và các phép toán phù hợp: `3` thuộc kiểu `int`, còn `'3'` thuộc `str`. Vì vậy `3 + 2` là phép cộng số, còn `'3' + '2'` nối văn bản.

Q5. Which function prints a value? — Hàm nào in một giá trị?

A. `int()` — `int()`

B. `type()` — `type()`

C. `input()` — `input()`

D. `print()` — `print()`

Đáp án: D

Giải thích: `print()` ghi biểu diễn của giá trị ra đầu ra chuẩn; chẳng hạn `print(5)` hiển thị `5`. Hàm này không nhận câu trả lời người dùng như `input()`.

Q6. Which function reads keyboard input? — Hàm nào đọc dữ liệu bàn phím?

A. `input()` — `input()`

B. `int()` — `int()`

C. `type()` — `type()`

D. `print()` — `print()`

Đáp án: A

Giải thích: `input()` nhận một dòng người dùng gõ và trả về văn bản không kèm ký tự xuống dòng cuối. Dù người dùng gõ `12`, kết quả vẫn là chuỗi `'12'`, không tự đổi thành số nguyên `12`.

Q7. Which function finds a value's type? — Hàm nào kiểm tra kiểu giá trị?

A. `input()` — `input()`

B. `print()` — `print()`

C. `type()` — `type()`

D. `int()` — `int()`

Đáp án: C

Giải thích: `type(value)` trả về lớp/kiểu của đối tượng: `type(3)` là `int` còn `type('3')` là `str`. Hàm kiểm tra kiểu chứ không chuyển đổi giá trị.

Q8. Which function converts valid decimal digits to an integer? — Hàm nào đổi chuỗi chữ số hợp lệ thành số nguyên?

A. `int()` — `int()`

B. `print()` — `print()`

C. `type()` — `type()`

D. `input()` — `input()`

Đáp án: A

Giải thích: `int()` phân tích chuỗi biểu diễn số nguyên hợp lệ nên `int('012')` cho số nguyên `12`; số 0 đầu không được giữ lại. Chuỗi `'12.5'` không phải chuỗi số nguyên thập phân hợp lệ và gây `ValueError`.

Q9. Which type represents a count of whole items? — Kiểu nào biểu diễn số lượng món nguyên?

A. `str` — `str`

B. `float` — `float`

C. `int` — `int`

D. `bool` — `bool`

Đáp án: C

Giải thích: `int` biểu diễn số lượng nguyên như 4 vé hoặc 0 lỗi, không có phần lẻ. `float` phù hợp số đo có thể có phần thập phân, không cần thiết cho số đếm các đơn vị nguyên vẹn.

Q10. Which type suits a measurement with fractions? — Kiểu nào phù hợp với số đo có phần lẻ?

A. `int` — `int`

B. `float` — `float`

C. `bool` — `bool`

D. `str` — `str`

Đáp án: B

Giải thích: `float` biểu diễn giá trị số có thể chứa phần lẻ, ví dụ 2,5 mét. Khác số đếm nguyên, số đo có thể nằm giữa hai số nguyên; số thập phân dạng dấu phẩy động đôi khi chỉ là xấp xỉ.

Q11. Which type preserves a leading zero in an identifier? — Kiểu nào giữ số 0 đầu mã định danh?

A. `str` — `str`

B. `bool` — `bool`

C. `float` — `float`

D. `int` — `int`

Đáp án: A

Giải thích: Mã định danh như `'007'` nên lưu bằng `str` vì mọi ký tự, kể cả hai số 0 đầu, đều có ý nghĩa. Đổi sang `int` được `7` và làm mất định dạng mã.

Q12. Which type stores a true/false state? — Kiểu nào lưu trạng thái đúng/sai?

A. `bool` — `bool`

B. `int` — `int`

C. `str` — `str`

D. `float` — `float`

Đáp án: A

Giải thích: `bool` có hai giá trị logic `True` và `False`, thích hợp biểu diễn trạng thái như có được phép truy cập hay không. Đây không phải kiểu đo lường số hay nhãn văn bản.

Q13. What does `x = 5` do? — `x = 5` thực hiện điều gì?

A. Tests whether `x` equals `5` — Kiểm tra `x` có bằng `5` không

B. Adds `5` to the old value of `x` — Cộng `5` vào giá trị cũ của `x`

C. Displays `5` without storing it — Hiển thị `5` mà không gán

D. Assigns the integer `5` to `x` — Gán số nguyên `5` cho `x`

Đáp án: D

Giải thích: Dấu bằng đơn gán tên `x` cho số nguyên `5`, không kiểm tra bằng nhau và cũng không cộng thêm số. Muốn so sánh `x` có bằng `5` thì dùng `x == 5`.

Q14. What is the type of `input()`'s return value? — Giá trị trả về của `input()` có kiểu nào?

A. An integer when only digits are typed — Số nguyên nếu chỉ nhập chữ số

B. A floating-point number by default — Số thực theo mặc định

C. A Boolean indicating whether input exists — Boolean cho biết có đầu vào hay không

D. A string, even if it contains digits — Chuỗi, kể cả khi chỉ chứa chữ số

Đáp án: D

Giải thích: `input()` luôn trả về `str` chứa nội dung được nhập, ví dụ `'42'`. Nếu cần tính toán số, hãy dùng `int(input())` hoặc phép đổi kiểu phù hợp khi đầu vào hợp lệ.

Q15. What does `float('2.5')` produce? — `float('2.5')` tạo ra gì?

A. The string `'2.5'` — Chuỗi `'2.5'`

B. The integer `2` — Số nguyên `2`

C. The floating-point number `2.5` — Số thực `2.5`

D. `ValueError` for any decimal point — `ValueError` nếu có dấu thập phân

Đáp án: C

Giải thích: `float('2.5')` chuyển văn bản `'2.5'` thành giá trị số thực `2.5`. Kết quả không còn là chuỗi có dấu nháy và không cắt phần lẻ thành số nguyên `2`.

Q16. What does `int('abc')` raise? — `int('abc')` phát sinh gì?

A. `TypeError` — `TypeError`

B. `ValueError` — `ValueError`

C. The integer `0` — Số nguyên `0`

D. `NameError` — `NameError`

Đáp án: B

Giải thích: `int('abc')` phát sinh `ValueError` vì các chữ cái không tạo thành số nguyên hệ mười hợp lệ. Hàm vẫn tồn tại và nhận được chuỗi, nên đây là lỗi giá trị chuyển đổi chứ không phải `NameError`.

Q17. What does `print(2, 3)` display by default? — `print(2, 3)` mặc định hiển thị gì?

A. `2,3` — `2,3`

B. `2 3` — `2 3`

C. `(2, 3)` — `(2, 3)`

D. `23` — `23`

Đáp án: B

Giải thích: Mặc định `print` ngăn cách các đối số bằng một dấu cách: `print(2, 3)` hiển thị `2 3`. Nó không dính hai số thành `23` và không in dạng tuple `(2, 3)`.

Q18. If `x = 2` is followed by `x = 4` on the next line, what is the final value of `x`? — Nếu `x = 2` rồi `x = 4` ở dòng tiếp theo, giá trị cuối của `x` là gì?

A. `ValueError` — `ValueError`

B. `2` — `2`

C. `4` — `4`

D. `6` — `6`

Đáp án: C

Giải thích: Lệnh đầu gán `x` bằng `2`, lệnh tiếp theo gán lại chính tên đó bằng `4`. Vì vậy giá trị cuối là `4`, không phải tổng `6` hay giá trị cũ `2`.

Q19. Which Python type does `type(2.0)` identify? — `type(2.0)` xác định kiểu Python nào?

A. `int` — `int`

B. `bool` — `bool`

C. `float` — `float`

D. `str` — `str`

Đáp án: C

Giải thích: Literal `2.0` là đối tượng số thực, nên `type(2.0)` xác định kiểu `float` (hiển thị `<class 'float'>`). Trong khi đó `type(2)` là `int`; dấu thập phân làm thay đổi kiểu literal.

Q20. What does `print('Hi')` display? — `print('Hi')` hiển thị gì?

A. `hi` — `hi`

B. `None` — `None`

C. `'Hi'` including the quotes — `'Hi'` gồm cả dấu nháy

D. `Hi` without quotes — `Hi` không có dấu nháy

Đáp án: D

Giải thích: Dấu nháy dùng để bao chuỗi trong mã nguồn chứ không phải ký tự thuộc nội dung chuỗi. Vì vậy `print('Hi')` hiển thị `Hi` rồi xuống dòng, không in dấu nháy hoặc chữ thường `hi`.

8. Nối sang bài kế tiếp

`Chương trình → Biến → Kiểu dữ liệu → Điều kiện`.

PY-02 — Kiểu số, Boolean và chuỗi

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Integer — số nguyên; **Float** — số thực.

`Biến → int/float/bool/str → Slicing → Xử lý văn bản`

Học xong dùng để làm gì? Phân biệt số và chữ khi đọc dữ liệu, đổi kiểu để tính tiền, số lượng hoặc điểm trung bình mà không ghép nhầm chuỗi.

Kết nối kiến thức tiếp theo: Sang PY-03, dùng toán tử tính toán và so sánh các giá trị đã có kiểu phù hợp.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Integer	số nguyên	Giá trị không có phần thập phân
Float	số thực	Số có thể có phần lẻ
Boolean	giá trị logic	Giá trị True hoặc False
String	chuỗi	Dãy ký tự có thể cắt lát

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`str()/int()/float()`	Chuyển kiểu phù hợp trước khi xử lý
`strip()/lower()/upper()`	Dọn khoảng trắng và chuẩn hóa chữ
`split()/replace()`	Tách chuỗi hoặc thay thế đoạn văn bản
`s[start:stop]`	Cắt ký tự theo vị trí, không lấy chỉ số stop

4. Khái niệm liên quan

Kiểu dữ liệu (data type) cho Python biết một giá trị biểu diễn điều gì và có thể thực hiện những phép toán nào. Số lượng món hàng thường là số nguyên, nhiệt độ có thể là số thực, điều kiện đạt/chưa đạt là Boolean, còn mã sản phẩm phải được giữ như chuỗi.

4.1. Integer — Số nguyên

Từ vựng trong mục: Integer — số nguyên; **Type** — kiểu dữ liệu.

`int` biểu diễn những số không có phần thập phân, ví dụ 5 sản phẩm hoặc 12 học viên. Dùng `int` khi cần đếm hoặc tính toán số lượng nguyên; khác với chuỗi `"5"`, số `5` thực hiện được phép cộng số học. Ở đây biến `n` giữ số 5 và `type(n)` kiểm tra kiểu.

```
n = 5
print(type(n))
```

Kết quả khi chạy:

```
<class 'int'>
```

Kết quả cho biết `n` là số nguyên, không phải chuỗi.

4.2. Float — Số thực

Từ vựng trong mục: Float — số thực; **Decimal** — phần thập phân.

`float` lưu các số có phần thập phân, như 3,5 kg hàng hoặc nhiệt độ 36,5 °C. Nếu cần tính trên những số đo này, dùng số thực thay vì chữ. Trong ví dụ, `x` nhận `3.5` và `print()` hiển thị giá trị của nó.

```
x = 3.5
print(x)
```

Kết quả khi chạy:

```
3.5
```

Dấu chấm trong code biểu diễn phần thập phân; đơn vị đo phải được xác định theo bài toán.

4.3. Boolean — Giá trị đúng/sai

Từ vựng trong mục: Boolean — giá trị logic; **True/False** — đúng/sai.

`bool` chỉ có hai giá trị `True` và `False`, phù hợp với câu hỏi dạng "đã thanh toán chưa?" hoặc "đủ điều kiện chưa?". Biến `ok = True` ở đây tượng trưng cho trạng thái kiểm tra đã đạt. Đây không phải chữ `"True"`: Python hiểu là giá trị logic.

```
ok = True
print(ok)
```

Kết quả khi chạy:

```
True
```

Đầu ra là giá trị logic `True`, không phải số 1 hoặc một thông báo khác.

4.4. String — Chuỗi ký tự

Từ vựng trong mục: String — chuỗi; **Slice** — cắt chuỗi.

`str` lưu dãy ký tự theo đúng thứ tự, kể cả chữ số nằm trong mã như `AB12` hoặc số điện thoại bắt đầu bằng 0. Với `code = 'AB12'`, các vị trí lần lượt là 0:A, 1:B, 2:1, 3:2. Cú pháp `[:2]` lấy ký tự ở vị trí 0 và 1, không lấy vị trí 2.

```
code = 'AB12'
print(code[:2])
```

Kết quả khi chạy:

```
AB
```

Hai ký tự đầu được tách ra mà chuỗi ban đầu vẫn giữ nguyên.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Chuẩn hóa email

Từ vựng trong mục: String — chuỗi; **Method** — phương thức.

Tình huống thực tế: Một ứng dụng lưu email do khách nhập là `" Lan@Example.COM "`, trong đó có khoảng trắng thừa ở đầu và cuối, còn chữ hoa và chữ thường được nhập lẫn nhau.

Yêu cầu: In email không còn khoảng trắng ở hai đầu và được đổi thành chữ thường.

Vì sao chọn cách giải này? `strip()` bỏ khoảng trắng hai đầu; `lower()` đổi chữ thành chữ thường, giúp thống nhất cách hiển thị và hỗ trợ so sánh email. Cách này không kiểm tra email có hợp lệ hay không.

Quy trình giải:

1. Lưu chuỗi email ban đầu.
2. Dùng `strip()` bỏ khoảng trắng đầu và cuối.
3. Dùng `lower()` đổi chữ thành chữ thường rồi in kết quả.

```
email = "  Lan@Example.COM "
print(email.strip().lower())
```

Kết quả khi chạy:

```
lan@example.com
```

Kết quả: `lan@example.com` — email không còn khoảng trắng thừa và đã viết thường.

Tình huống 2 — Tách nhãn sản phẩm

Từ vựng trong mục: String — chuỗi; **Split** — tách.

Tình huống thực tế: Một cửa hàng nhận chuỗi nhãn sản phẩm `"AI,Python,Math"`. Ba nhãn được ngăn cách bằng dấu phẩy và cần tách riêng để xử lý từng nhãn.

Yêu cầu: Tạo danh sách gồm ba nhãn từ chuỗi này.

Vì sao chọn cách giải này? `split(",")` tách chuỗi ở mỗi dấu phẩy thành các phần tử riêng trong một danh sách.

Quy trình giải:

1. Lưu chuỗi nhãn vào `labels`.
2. Truyền dấu phẩy vào `split()` để tách các nhãn.
3. In danh sách nhận được.

```
labels = "AI,Python,Math"
print(labels.split(","))
```

Kết quả khi chạy:

```
['AI', 'Python', 'Math']
```

Kết quả: `['AI', 'Python', 'Math']` — ba nhãn được tách thành ba phần tử.

Tình huống 3 — Giữ mã đầu 0

Từ vựng trong mục: String — chuỗi; **Slice** — cắt chuỗi.

Tình huống thực tế: Hệ thống quản lý đơn hàng lưu mã `00045`. Các số `0` đầu mã phải được giữ nguyên vì đây là mã nhận diện, không phải số để tính toán.

Yêu cầu: Lưu mã đơn và in ba ký tự đầu của mã để kiểm tra chúng vẫn còn nguyên.

Vì sao chọn cách giải này? Chuỗi `str` giữ được các số `0` đứng đầu. Cú pháp `[:3]` lấy ba ký tự đầu, không chuyển mã sang số nguyên.

Quy trình giải:

1. Lưu `"00045"` vào `order_id`.
2. Lấy ba ký tự đầu bằng `order_id[:3]`.
3. In kết quả để xem phần đầu của mã.

```
order_id = "00045"
print(order_id[:3])
```

Kết quả khi chạy:

```
000
```

Kết quả: `000` — ba ký tự đầu của mã vẫn được giữ nguyên. Chương trình không in cả mã `00045`.

Tình huống 4 — Đổi số từ CSV

Từ vựng trong mục: Float — số thực; **Integer** — số nguyên.

Tình huống thực tế: Một cửa hàng nhập từ file CSV giá một sản phẩm là chuỗi `"12.5"` và số lượng là chuỗi `"3"`. Giá tính theo cùng đơn vị tiền của hệ thống.

Yêu cầu: Đổi hai chuỗi thành số, rồi tính số tiền cho ba sản phẩm.

Vì sao chọn cách giải này? Dùng `float()` cho giá có phần thập phân và `int()` cho số lượng nguyên, sau đó nhân hai số để tính tiền.

Quy trình giải:

1. Đổi `"12.5"` thành số thực bằng `float()`.
2. Đổi `"3"` thành số nguyên bằng `int()`.
3. Nhân giá và số lượng rồi in kết quả.

```
price = float("12.5")
qty = int("3")
print(price * qty)
```

Kết quả khi chạy:

```
37.5
```

Kết quả: `37.5` đơn vị tiền — tổng giá của ba sản phẩm.

Tình huống 5 — Che số liên lạc

Từ vựng trong mục: String — chuỗi; **Slice** — cắt chuỗi.

Tình huống thực tế: Ứng dụng chăm sóc khách hàng có số điện thoại mẫu `0900123456`. Khi hiển thị trên màn hình, ứng dụng chỉ được để lộ ba chữ số cuối.

Yêu cầu: In số điện thoại dưới dạng dấu `*` che toàn bộ ký tự trừ ba số cuối.

Vì sao chọn cách giải này? Lưu số điện thoại dạng chuỗi để giữ số `0` đầu. `phone[-3:]` lấy ba ký tự cuối; `len(phone)-3` tính số dấu `*` cần in.

Quy trình giải:

1. Lưu số điện thoại dưới dạng chuỗi.
2. Tạo số dấu `*` bằng độ dài chuỗi trừ ba.
3. Ghép phần che với ba ký tự cuối rồi in.

```
phone = "0900123456"
print("*" * (len(phone)-3) + phone[-3:])
```

Kết quả khi chạy:

```
*******456
```

Kết quả: `*******456` — bảy ký tự đầu được che, chỉ còn `456`.

6. Những nhầm lẫn thường gặp

- `int("12.5")` gây lỗi vì chuỗi này không viết theo dạng số nguyên; dùng `float("12.5")` nếu cần số có phần thập phân.
- Không chuyển mã đơn hoặc số điện thoại sang `int` nếu cần giữ số `0` ở đầu.
- `strip()` chỉ bỏ khoảng trắng ở hai đầu chuỗi; `lower()` không tự kiểm tra email hợp lệ.

- `s[:3]` lấy ba ký tự đầu (vị trí 0, 1, 2), không lấy ký tự ở vị trí 3.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Which description best defines an integer in Python? — Mô tả nào đúng nhất về số nguyên trong Python?

A. An ordered sequence of characters — Dãy ký tự có thứ tự

B. A negative, zero, or positive integer without a fractional part — Số nguyên âm, số 0 hoặc số nguyên dương không có phần lẻ

C. A true/false value — Một giá trị đúng/sai

D. A floating-point number — Số thực dấu phẩy động

Đáp án: B

Giải thích: Kiểu `int` của Python lưu chính xác các số nguyên âm, số 0 và số nguyên dương như `-3`, `0`, `12`. Không loại số âm khỏi kiểu này; khác với `int`, `float` biểu diễn số dấu phẩy động, kể cả `2.0`.

Q2. What is a float? — Float là kiểu dữ liệu nào?

A. A whole number without a fractional part — Số không có phần thập phân

B. A floating-point number — Số thực dấu phẩy động

C. A true/false value — Một giá trị đúng/sai

D. An ordered sequence of characters — Dãy ký tự có thứ tự

Đáp án: B

Giải thích: `float` là kiểu số dấu phẩy động trong Python, ví dụ `1.5` và `2.0`. Ngay cả `2.0` dù có giá trị toán học nguyên vẫn là `float` chứ không phải `int`.

Q3. What is a Boolean? — Boolean là gì?

A. A true/false value — Một giá trị đúng/sai

B. A floating-point number — Số thực dấu phẩy động

C. An ordered sequence of characters — Dãy ký tự có thứ tự

D. A whole number without a fractional part — Số không có phần thập phân

Đáp án: A

Giải thích: Boolean biểu diễn một trong hai trạng thái `True` hoặc `False`, thường dùng trong điều kiện như `3 > 2`. Vai trò của nó khác việc lưu số hoặc dãy ký tự có thứ tự.

Q4. What is a string? — Chuỗi là gì?

A. An ordered sequence of characters — Dãy ký tự có thứ tự

B. A whole number without a fractional part — Số không có phần thập phân

C. A floating-point number — Số thực dấu phẩy động

D. A true/false value — Một giá trị đúng/sai

Đáp án: A

Giải thích: Chuỗi (`str`) là dãy ký tự Unicode có thứ tự, chẳng hạn `'Python'`. Chuỗi chữ số như `'12'` vẫn là văn bản nên cộng với chuỗi khác sẽ nối chữ, không phải cộng số.

Q5. What does `strip()` normally remove? — `strip()` thường loại bỏ gì?

A. Whitespace at both ends — Khoảng trắng ở hai đầu

B. Text converted to lowercase — Văn bản chuyển sang chữ thường

C. Text converted to uppercase — Văn bản chuyển sang chữ hoa

D. Substitutes matching text — Thay các đoạn chữ khớp mẫu

Đáp án: A

Giải thích: Khi không truyền tham số, `strip()` bỏ khoảng trắng ở đầu và cuối, không xóa khoảng trắng giữa chuỗi. Ví dụ `' a b '.strip()` thành `'a b'` và giữ dấu cách giữa `a` và `b`.

Q6. What does `lower()` return? — `lower()` trả về gì?

A. Substitutes matching text — Thay các đoạn chữ khớp mẫu

B. Text converted to uppercase — Văn bản chuyển sang chữ hoa

C. Text converted to lowercase — Văn bản chuyển sang chữ thường

D. Whitespace at both ends — Khoảng trắng ở hai đầu

Đáp án: C

Giải thích: `lower()` trả về bản sao chữ thường, nên `'PyTh0n'.lower()` được `'python'`. Hàm không sửa chuỗi gốc tại chỗ vì chuỗi là bất biến.

Q7. What does `upper()` return? — `upper()` trả về gì?

A. Whitespace at both ends — Khoảng trắng ở hai đầu

B. Substitutes matching text — Thay các đoạn chữ khớp mẫu

C. Text converted to lowercase — Văn bản chuyển sang chữ thường

D. Text converted to uppercase — Văn bản chuyển sang chữ hoa

Đáp án: D

Giải thích: `upper()` trả về chữ hoa, ví dụ `'ab'.upper()` là `'AB'`. Khác `lower()`, nó đổi chữ cái về dạng hoa; ký tự như chữ số vẫn giữ nguyên.

Q8. What does `replace(old, new)` do? — `replace(old, new)` có tác dụng gì?

A. Whitespace at both ends — Khoảng trắng ở hai đầu

B. Text converted to uppercase — Văn bản chuyển sang chữ hoa

C. Text converted to lowercase — Văn bản chuyển sang chữ thường

D. Substitutes matching text — Thay các đoạn chữ khớp mẫu

Đáp án: D

Giải thích: `replace(old, new)` trả về chuỗi mới sau khi thay các đoạn khớp; `'aba'.replace('a', 'x')` được `'xbx'`. Hàm không sửa chuỗi gốc và không chỉ đơn thuần đổi hoa/thường.

Q9. What does `'a,b'.split(',')` return? — `'a,b'.split(',')` trả về gì?

A. `['a', 'b']` — `['a', 'b']`

B. `'ab'` — `'ab'`

C. `('a', 'b')` — `('a', 'b')`

D. `['a,b']` — `['a,b']`

Đáp án: A

Giải thích: `split(',')` tách văn bản tại mỗi dấu phẩy và trả về danh sách các chuỗi con: `'a,b'` thành `['a', 'b']`. Dấu phẩy là dấu phân cách nên bị bỏ đi; kết quả là list chứ không phải một chuỗi.

Q10. What does `'ABCD'[1:3]` yield? — `'ABCD'[1:3]` cho kết quả gì?

A. `'CD'` — `'CD'`

B. `'AB'` — `'AB'`

C. `'BC'` — `'BC'`

D. `'BCD'` — `'BCD'`

Đáp án: C

Giải thích: Chỉ số chuỗi bắt đầu từ 0, nên trong `'ABCD'`, vị trí 1 và 2 là `B` và `C`. Lát cắt `[1:3]` lấy vị trí 1 nhưng không lấy mốc dừng 3, cho `'BC'`.

Q11. What does `len('Python')` yield? — `len('Python')` trả về gì?

A. 5 — 5

B. 7 — 7

C. `'6'` (string) — `'6'` (chuỗi)

D. 6 (integer) — 6 (số nguyên)

Đáp án: D

Giải thích: `len()` đếm số ký tự trong chuỗi: `P`, `y`, `t`, `h`, `o`, `n` là sáu ký tự. Hàm trả về số nguyên `6`, không phải chuỗi `'6'`; chữ hoa/thường không đổi số lượng.

Q12. What does `'A'.join(['x','y'])` yield? — `'A'.join(['x','y'])` tạo gì?

A. `['x', 'A', 'y']` — `['x', 'A', 'y']`

B. `'xyA'` — `'xyA'`

C. `'xAy'` — `'xAy'`

D. `'Axy'` — `'Axy'`

Đáp án: C

Giải thích: `join()` chèn chuỗi đứng trước dấu chấm vào giữa các phần tử: `'A'.join(['x','y'])` cho `'xAy'`. Nó không thêm `A` ở đầu toàn danh sách hoặc sau phần tử cuối.

Q13. What does `int('12')` create? — `int('12')` tạo ra gì?

A. `ValueError` — `ValueError`

B. `12` (integer) — `12` (số nguyên)

C. `12.0` (float) — `12.0` (số thực)

D. `'12'` (string) — `'12'` (chuỗi)

Đáp án: B

Giải thích: `int('12')` đổi chuỗi số nguyên hợp lệ thành số nguyên `12` để có thể tính toán. Kết quả không còn là văn bản `'12'` và cũng không phải số thực `12.0`.

Q14. What does `float('1.5')` create? — `float('1.5')` tạo ra gì?

A. `ValueError` — `ValueError`

B. `1` (integer) — `1` (số nguyên)

C. `'1.5'` (string) — `'1.5'` (chuỗi)

D. `1.5` (float) — `1.5` (số thực)

Đáp án: D

Giải thích: `float('1.5')` chuyển văn bản số hợp lệ thành số thực `1.5`. Nó giữ phần lẻ thay vì cắt thành số nguyên `1`, và chuỗi hợp lệ này không gây lỗi.

Q15. What does `str(12)` create? — `str(12)` tạo gì?

A. `['1', '2']` — `['1', '2']`

B. `'12'` (string) — `'12'` (chuỗi)

C. `12.0` (float) — `12.0` (số thực)

D. `12` (integer) — `12` (số nguyên)

Đáp án: B

Giải thích: `str(12)` tạo biểu diễn văn bản `'12'`, phù hợp làm nhãn hoặc nối chuỗi. Kết quả không còn là số nguyên `12`: `'12' + '3'` tạo `'123'`, không phải tổng số `15`.

Q16. What does `bool('False')` return? — `bool('False')` trả về gì?

A. `True` — `True`

B. `False` — `False`

C. `ValueError` — `ValueError`

D. `'False'` (string) — `'False'` (chuỗi)

Đáp án: A

Giải thích: `bool()` kiểm tra tính đúng/sai của giá trị, không đọc nghĩa chữ `True` hay `False` trong văn bản. Do chuỗi `'False'` không rỗng nên `bool('False')` bằng `True`; với chuỗi, chỉ chuỗi rỗng được xem là sai.

Q17. What is true of Python strings? — Điều nào đúng về chuỗi Python?

A. Strings are unordered collections — Chuỗi là tập hợp không có thứ tự

B. Strings store only numbers — Chuỗi chỉ lưu số

C. Strings are immutable — Chuỗi không thể sửa trực tiếp từng ký tự

D. Strings can be changed in place with `s[0] = 'x'` — Có thể đổi chuỗi tại chỗ bằng `s[0] = 'x'`

Đáp án: C

Giải thích: Chuỗi là bất biến: gán `s[0] = 'x'` gây `TypeError`, kể cả khi `s` không rỗng. Các phép như `replace()` tạo chuỗi mới thay vì sửa từng ký tự của chuỗi cũ.

Q18. What does `s[-1]` select for a nonempty string? — `s[-1]` lấy gì khi chuỗi không rỗng?

A. The first character — Ký tự đầu

B. Always raises `IndexError` — Luôn gây `IndexError`

C. The final two characters — Hai ký tự cuối

D. The last character — Ký tự cuối

Đáp án: D

Giải thích: Chỉ số `-1` đếm từ cuối và lấy ký tự cuối của chuỗi không rỗng. Với `s = 'cat'`, `s[-1]` là `'t'`, còn `s[0]` lấy `'c'`.

Q19. What does `'Ab' == 'ab'` evaluate to? — `'Ab' == 'ab'` có giá trị gì?

A. `'False'` (string) — `'False'` (chuỗi)

B. `True` (Boolean) — `True` (Boolean)

C. `False` (Boolean) — `False` (Boolean)

D. `0` (integer) — `0` (số nguyên)

Đáp án: C

Giải thích: So sánh chuỗi xét từng ký tự và phân biệt chữ hoa/thường: `'A'` khác `'a'` nên `'Ab' == 'ab'` cho Boolean `False`. Nếu đổi cả hai chuỗi về chữ thường trước khi so sánh thì kết quả sẽ khác.

Q20. What is the result of `'4' + '2'`? — `'4' + '2'` cho kết quả gì?

A. `TypeError` — `TypeError`

B. `'42'` (string) — `'42'` (chuỗi)

C. `'6'` (string) — `'6'` (chuỗi)

D. `6` (integer) — `6` (số nguyên)

Đáp án: B

Giải thích: Cả hai toán hạng đều là chuỗi nên + nối theo thứ tự: `'4' + '2'` được `'42'`. Muốn cộng số, cần đổi kiểu trước, ví dụ `int('4') + int('2')` cho số nguyên 6.

8. Nối sang bài kế tiếp

`Biến → int/float/bool/str → Slicing → Xử lý văn bản.`

PY-03 — Toán tử số học, so sánh và logic

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Arithmetic — số học; **Comparison** — so sánh.

`Kiểu dữ liệu → Toán tử → Điều kiện → Quy tắc nghiệp vụ`

Học xong dùng để làm gì? Tính tổng tiền, so sánh ngưỡng điểm và kết hợp nhiều điều kiện thành kết quả đúng/sai.

Kết nối kiến thức tiếp theo: Sang PY-04, dùng chính các phép so sánh và logic đó để quyết định chương trình đi nhánh nào.

2. English vocabulary — Từ vựng

English	Nghĩa tiếng Việt	Giải thích
Arithmetic	số học	Phép cộng trừ nhân chia
Comparison	so sánh	So sánh hai giá trị
Logical	logic	Kết hợp các điều kiện
Bitwise	thao tác bit	Phép tính trên các bit của số nguyên

3. Các hàm và tùy chọn Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và ý nghĩa lựa chọn
`+, -, *, /`	Tính số học và tỷ lệ
`//, %, **`	Chia nguyên, lấy dư, lũy thừa
`==, !=, <=, >=, and, or`	So sánh và kết hợp điều kiện
`&, |, ^`	Chỉ dùng cho cờ bit hoặc toán bit được thiết kế rõ

4. Khái niệm liên quan

Toán tử (operator) là dấu hoặc từ khóa thực hiện một thao tác trên dữ liệu. Với một đơn hàng, ta dùng toán tử số học để tính tiền, so sánh để kiểm tra ngưỡng, rồi dùng logic để ghép điều kiện. Toán tử bit cũng dùng ký hiệu nhưng xử lý các chữ số nhị phân thay vì điều kiện thông thường.

4.1. Arithmetic — Toán tử số học

Từ vựng trong mục: Arithmetic — số học; **Multiplication** — phép nhân.

Số học giúp tính từ các dữ liệu có sẵn: `+` cộng, `-` trừ, `*` nhân, `/` chia. Ví dụ cửa hàng bán 5 món, mỗi món 3 nghìn đồng: lấy số lượng nhân đơn giá để được tổng 15 nghìn đồng. Python còn có `//` chia lấy phần nguyên và `%` lấy phần dư khi chia thùng hoặc chia nhóm.

```
print(5 * 3)
```

Kết quả khi chạy:

```
15
```

Giá trị 15 là tổng tiền theo đơn vị nghìn đồng của ví dụ.

4.2. Comparison — Toán tử so sánh

Từ vựng trong mục: Comparison — so sánh; **Boolean** — đúng/sai.

So sánh trả lời một câu hỏi bằng `True` hoặc `False`. Nếu kho có 8 món và đơn hàng cần 5 món, điều kiện `8 >= 5` kiểm tra đủ hàng để giao không. Các dấu `>`, `<`, `>=`, `<=`, `==`, `!=` biểu diễn lớn hơn, nhỏ hơn, bằng hoặc khác nhau. Phân biệt `==` để kiểm tra với `=` để gán.

```
print(8 >= 5)
```

Kết quả khi chạy:

```
True
```

Kho có ít nhất 5 món nên điều kiện đủ hàng là đúng.

4.3. Logical — Toán tử logic

Từ vựng trong mục: Logical — logic; **AND** — và; **OR** — hoặc.

Sau khi có các điều kiện đúng/sai, `and` yêu cầu mọi điều kiện cùng đúng; `or` chỉ cần một điều kiện đúng; `not` đảo trạng thái. Ví dụ người dùng đã đăng nhập (`True`) nhưng chưa xác minh (`False`): nếu hệ thống bắt buộc cả hai thì `and` trả về `False`.

```
print(True and False)
```

Kết quả khi chạy:

```
False
```

Một trong hai điều kiện chưa đạt, nên không thỏa yêu cầu `and`.

4.4. Bitwise — Toán tử theo bit

Từ vựng trong mục: Bitwise — theo bit; **Binary** — nhị phân.

Máy tính biểu diễn số nguyên bằng các bit `0` và `1`. Toán tử `&` so từng vị trí bit và chỉ giữ `1` khi hai phía cùng có `1`. Số 5 là `0101`, số 3 là `0011`; phần chung chỉ còn `0001`, tương ứng số 1. Đây là phép xử lý số nguyên theo bit, **khác** `and` dùng để nối điều kiện logic.

```
print(5 & 3)
```

Kết quả khi chạy:

```
1
```

Số `1` là kết quả phép AND theo bit giữa 5 và 3.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tính ưu đãi

Từ vựng trong mục: Arithmetic — số học; **Discount** — giảm giá.

Tình huống thực tế: Cửa hàng bán một sản phẩm giá 250.000 đồng và giảm 10% trên giá niêm yết.

Yêu cầu: Tính số tiền khách phải trả sau giảm giá.

Vì sao chọn cách giải này? Nhân giá gốc với `(1 - 0.10)` để lấy 90% giá ban đầu.

Quy trình giải:

1. Lưu giá gốc vào `price`.
2. Tính tỷ lệ còn lại là `1 - 0.10`.
3. Nhân giá với tỷ lệ còn lại và in kết quả.

```
price = 250000
print(price * (1 - 0.10))
```

Kết quả khi chạy:

```
225000.0
```

Kết quả: `225000.0` đồng — khách được giảm 25.000 đồng.

Tình huống 2 — Xét VIP

Từ vựng trong mục: Comparison — so sánh; **Logical AND** — phép và.

Tình huống thực tế: Một cửa hàng công nhận khách VIP khi đã là thành viên ít nhất 5 năm và chi tiêu từ 1.000.000 đồng. Một khách đã tham gia 5 năm và chi 1.200.000 đồng.

Yêu cầu: Kiểm tra khách này có đủ điều kiện VIP hay không.

Vì sao chọn cách giải này? Dùng >= để kiểm tra từng ngưỡng, rồi dùng `and` vì hai điều kiện phải đồng thời đúng.

Quy trình giải:

1. Lưu số năm thành viên và tổng chi tiêu.
2. Kiểm tra từng giá trị có đạt ngưỡng không.
3. Nối hai điều kiện bằng `and` và in kết quả.

```
years, spending = 5, 1200000
print(years >= 5 and spending >= 1000000)
```

Kết quả khi chạy:

```
True
```

Kết quả: `True` — khách đạt cả hai tiêu chí VIP.

Tình huống 3 — Đóng thùng hàng

Từ vựng trong mục: Integer division — chia lấy phần nguyên; **Remainder** — số dư.

Tình huống thực tế: Kho hàng có 23 sản phẩm và dùng thùng chứa đúng 5 sản phẩm khi đầy. Sản phẩm chưa đủ một thùng được giữ lại.

Yêu cầu: Tính số thùng đầy và số sản phẩm còn dư.

Vì sao chọn cách giải này? Chia nguyên `//` cho số thùng đầy; chia lấy dư `%` cho số sản phẩm còn lại.

Quy trình giải:

1. Lưu số sản phẩm và sức chứa mỗi thùng.
2. Dùng `//` đếm số thùng đầy.
3. Dùng `%` tính sản phẩm còn dư.

```
items, box_size = 23, 5
print(items // box_size, items % box_size)
```

Kết quả khi chạy:

```
4 3
```

Kết quả: `4 3` — có 4 thùng đầy và 3 sản phẩm dư.

Tình huống 4 — Đọc cờ thiết bị

Từ vựng trong mục: Bitwise AND — phép và theo bit; **Mask** — mặt nạ bit.

Tình huống thực tế: Một thiết bị lưu trạng thái bốn công tắc trong số nhị phân `0b1010`. Mỗi bit thể hiện một công tắc bật (`1`) hoặc tắt (`0`). Công tắc tương ứng với bit `0b0010` cần được kiểm tra.

Yêu cầu: In `True` nếu công tắc đó bật, ngược lại in `False`.

Vì sao chọn cách giải này? Dùng phép AND bit `&` cùng mặt nạ `0b0010` để lấy đúng bit cần kiểm tra; `bool()` đổi kết quả thành đúng/sai.

Quy trình giải:

1. Lưu trạng thái nhị phân.
2. Dùng `status & 0b0010` để lấy bit mục tiêu.
3. Dùng `bool()` và in kết quả.

```
status = 0b1010
print(bool(status & 0b0010))
```

Kết quả khi chạy:

```
True
```

Kết quả: `True` — bit `0b0010` đang bật.

Tình huống 5 — Ưu tiên đơn hàng

Từ vựng trong mục: Logical OR — phép hoặc; **Comparison** — so sánh.

Tình huống thực tế: Cửa hàng ưu tiên một đơn hàng nếu giá trị từ 2.000.000 đồng hoặc đơn bị trễ hơn 2 ngày. Một đơn có giá trị 3.000.000 đồng và chưa trễ ngày nào (`delay = 0`).

Yêu cầu: Kiểm tra đơn hàng có được ưu tiên hay không.

Vì sao chọn cách giải này? Dùng `or` vì chỉ cần một trong hai điều kiện đạt ngưỡng là đủ.

Quy trình giải:

1. Lưu giá trị đơn và số ngày trễ.
2. Kiểm tra giá trị đơn đạt từ 2.000.000 đồng.
3. Kiểm tra số ngày trễ lớn hơn 2, nối hai kết quả bằng `or`.

```
amount, delay = 3000000, 0
print(amount >= 2000000 or delay > 2)
```

Kết quả khi chạy:

```
True
```

Kết quả: `True` — đơn được ưu tiên nhờ giá trị đạt ngưỡng.

6. Những nhầm lẫn thường gặp

- `=` dùng để gán giá trị, còn `==` dùng để so sánh bằng nhau.
- `and` và `or` kiểm tra điều kiện; `&` và `|` thao tác trên các bit, không thay thế tùy tiện cho nhau.
- `/` cho kết quả chia thông thường, `//` lấy phần nguyên theo hướng làm tròn xuống, `%` lấy số dư.
- Với giá tiền cần tính chính xác đến từng đồng, nên cân nhắc dùng số nguyên đồng hoặc `Decimal` thay vì dựa vào số thực cho phép tính tiền phức tạp.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Which operator performs multiplication? — Toán tử nào thực hiện phép nhân?

A. * — *

B. & — &

C. == — ==

D. and — and

Đáp án: A

Giải thích: * nhân các số, ví dụ `3 * 4` cho 12. == so sánh giá trị, and kết hợp điều kiện logic, còn & thực hiện AND theo bit trên số nguyên.

Q2. Which operator tests equality? — Toán tử nào kiểm tra bằng nhau?

A. * — *

B. == — ==

C. & — &

D. and — and

Đáp án: B

Giải thích: == kiểm tra hai giá trị có bằng nhau và tạo Boolean, ví dụ `2 == 2` là `True`. Một dấu = dùng để gán giá trị cho tên, chứ không so sánh.

Q3. Which Python keyword performs short-circuit logical AND? — Từ khóa Python nào thực hiện phép VÀ logic có đánh giá ngắn mạch?

A. == — ==

B. & — &

C. and — and

D. * — *

Đáp án: C

Giải thích: Từ khóa and thực hiện phép VÀ logic ngắn mạch: nếu vế trái sai, Python không đánh giá vế phải. Ký hiệu & thực hiện AND theo bit trên số nguyên và kể cả dùng với Boolean cũng không đánh giá ngắn mạch.

Q4. Which operator performs bitwise AND on integers? — Toán tử nào thực hiện AND theo bit trên số nguyên?

A. and — and

B. == — ==

C. * — *

D. & — &

Đáp án: D

Giải thích: Với số nguyên, & thực hiện AND trên từng bit: `5 & 3` đối chiếu `0101` và `0011`, được `0001` (1). Đây không phải phép nhân thông thường hay từ khóa logic and.

Q5. What does `7 // 2` return? — `7 // 2` trả về gì?

A. `1` — `1`

B. `3.5` — `3.5`

C. `3` — `3`

D. `8` — `8`

Đáp án: C

Giải thích: `//` thực hiện chia lấy sàn: `7 / 2` bằng `3.5`, làm tròn xuống được `3`. Với thương âm, làm tròn xuống nghĩa là về âm vô cực nên `-7 // 2` bằng `-4`.

Q6. What does `7 % 2` return? — `7 % 2` trả về gì?

A. `1` — `1`

B. `3.5` — `3.5`

C. `3` — `3`

D. `8` — `8`

Đáp án: A

Giải thích: `%` trả phần dư theo phép chia lấy sàn: `7 = 2 * 3 + 1`, nên `7 % 2` bằng `1`. Nó không phải thương `3` hay kết quả chia thực `3.5`.

Q7. What does `2 ** 3` return? — `2 ** 3` trả về gì?

A. `3` — `3`

B. `1` — `1`

C. `3.5` — `3.5`

D. `8` — `8`

Đáp án: D

Giải thích: `**` là phép lũy thừa: `2 ** 3` nhân 2 với chính nó ba lần, cho `8`. Một dấu `*` là phép nhân, còn `^` trong Python là XOR theo bit, không phải lũy thừa.

Q8. What does `7 / 2` return? — `7 / 2` trả về gì?

A. `3.5` — `3.5`

B. `3` — `3`

C. `1` — `1`

D. `8` — `8`

Đáp án: A

Giải thích: Trong Python 3, `/` chia thực nên `7 / 2` trả về số thực `3.5`. Toán tử `//` lấy sàn thương được `3`, còn `%` trả phần dư `1`.

Q9. What is `True and False`? — `True and False` bằng gì?

A. `True` (Boolean) — `True` (Boolean)

B. `None` — `None`

C. `0` (integer) — `0` (số nguyên)

D. `False` (Boolean) — `False` (Boolean)

Đáp án: D

Giải thích: `and` Boolean yêu cầu cả hai vế đều đúng, nhưng vế thứ hai là `False`. Vì thế `True and False` cho Boolean `False`, không phải số nguyên `0` hay `None`.

Q10. What is `False or True`? — `False or True` bằng gì?

A. `1` (integer) — `1` (số nguyên)

B. `True` (Boolean) — `True` (Boolean)

C. `False` (Boolean) — `False` (Boolean)

D. `None` — `None`

Đáp án: B

Giải thích: `or` Boolean đúng nếu có ít nhất một vế đúng; vế phải của `False or True` là `True`. Kết quả ở đây là Boolean `True`, không phải số nguyên `1`, dù `True` có thể hoạt động như 1 trong một số phép số học.

Q11. What does `not True` produce? — `not True` cho kết quả gì?

A. `0` (integer) — `0` (số nguyên)

B. `None` — `None`

C. `False` (Boolean) — `False` (Boolean)

D. `True` (Boolean) — `True` (Boolean)

Đáp án: C

Giải thích: `not` đảo giá trị logic nên `not True` thành Boolean `False`. Ngược lại `not False` là `True`; số nguyên không là giá trị thuộc kiểu khác.

Q12. What does `5 & 3` produce? — `5 & 3` tạo ra gì?

A. `False` (Boolean) — `False` (Boolean)

B. `5` (integer) — `5` (số nguyên)

C. `True` (Boolean) — `True` (Boolean)

D. `1` (integer) — `1` (số nguyên)

Đáp án: D

Giải thích: Hai số nguyên là `5 = 0101` và `3 = 0011` ở hệ nhị phân bốn bit. Toán tử `&` chỉ giữ vị trí có cả hai bit bằng 1, cho `0001`, tức số nguyên `1`, không phải Boolean `True`.

Q13. Which operator tests that two values differ? — Toán tử nào kiểm tra hai giá trị khác nhau?

A. `!=` — `!=`

B. >= — >=

C. = — =

D. `in` — `in`

Đáp án: A

Giải thích: `!=` kiểm tra hai giá trị có khác nhau: `3 != 4` là `True`, còn `3 != 3` là `False`. Nó đối lập với == dùng để kiểm tra bằng nhau.

Q14. Which operator means 'at least'? — Toán tử nào biểu diễn 'lớn hơn hoặc bằng'?

A. `!=` — `!=`

B. `in` — `in`

C. >= — >=

D. = — =

Đáp án: C

Giải thích: >= nghĩa là lớn hơn hoặc bằng, nên `5 >= 5` và `6 >= 5` đều là `True`. Trường hợp bằng ngưỡng rất quan trọng: dùng > sẽ loại đúng giá trị 5.

Q15. Which operator checks membership in a collection? — Toán tử nào kiểm tra phần tử thuộc một tập dữ liệu?

A. `!=` — `!=`

B. `in` — `in`

C. >= — >=

D. = — =

Đáp án: B

Giải thích: Toán tử `in` kiểm tra phần tử có thuộc tập hợp: `2 in [1, 2, 3]` là `True`, còn `4 in [1, 2, 3]` là `False`. Nó không gán giá trị hay so sánh độ lớn.

Q16. Which operator assigns a new value to a variable? — Toán tử nào gán giá trị mới cho biến?

A. >= — >=

B. `!=` — `!=`

C. = — =

D. `in` — `in`

Đáp án: C

Giải thích: Toán tử gán = liên kết tên với giá trị, như `total = 10`. Muốn so sánh hai giá trị thì dùng ==; dùng = trong điều kiện so sánh thông thường sẽ sai cú pháp.

Q17. Which result comes from `3 > 5`? — `3 > 5` trả về gì?

A. `0` (integer) — `0` (số nguyên)

B. `False` (Boolean) — `False` (Boolean)

C. `True` (Boolean) — `True` (Boolean)

D. `None` — `None`

Đáp án: B

Giải thích: > kiểm tra lớn hơn nghiêm ngặt, mà 3 nhỏ hơn 5 nên `3 > 5` cho Boolean `False`. Kết quả là giá trị logic, không phải số nguyên `0`.

Q18. What result comes from `2 + 3 * 4`? — `2 + 3 * 4` bằng bao nhiêu?

A. `9` — `9`

B. `14` — `14`

C. `False` — `False`

D. `20` — `20`

Đáp án: B

Giải thích: Phép nhân có độ ưu tiên cao hơn cộng: tính `3 * 4 = 12` trước, rồi `2 + 12 = 14`. Thêm ngoặc như `(2 + 3) * 4` sẽ đổi kết quả thành `20`.

Q19. What does `True or False` return? — `True or False` trả về gì?

A. `True` (Boolean) — `True` (Boolean)

B. `1` (integer) — `1` (số nguyên)

C. `False` (Boolean) — `False` (Boolean)

D. `None` — `None`

Đáp án: A

Giải thích: `or` đúng khi ít nhất một toán hạng Boolean đúng; vì vế trái là `True` nên `True or False` trả `True`. Với đầu vào Boolean, đây là Boolean `True`, không phải số nguyên `1`.

Q20. What does `5 ^ 3` return for integers? — `5 ^ 3` trả về gì với số nguyên?

A. `5` — `5`

B. `2` — `2`

C. `15` — `15`

D. `6` — `6`

Đáp án: D

Giải thích: ^ là XOR theo bit, không phải lũy thừa: `5 = 0101` và `3 = 0011` khác nhau tại các vị trí bit `0100` và `0010`, cho `0110` = 6. Lũy thừa phải dùng `**`, không dùng ^.

8. Nối sang bài kế tiếp

`Kiểu dữ liệu → Toán tử → Điều kiện → Quy tắc nghiệp vụ.`

PY-04 — Câu lệnh điều kiện if, elif, else

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Condition — điều kiện; **Branch** — nhánh.

`Toán tử → Điều kiện → Phân loại theo quy tắc`

Học xong dùng để làm gì? Tạo quy tắc quyết định như phân loại điểm, xác định mức ưu đãi và xử lý trường hợp không đạt điều kiện.

Kết nối kiến thức tiếp theo: Sang PY-05, lặp lại quy tắc này cho nhiều bản ghi thay vì viết từng câu điều kiện thủ công.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Condition	điều kiện	Phép kiểm tra đúng hoặc sai
Branch	nhánh	Phần chương trình được chọn để chạy
Threshold	ngưỡng	Ranh giới để chuyển trạng thái
Nested if	điều kiện lồng	Kiểm tra bổ sung bên trong một nhánh

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`if expression:`	Chỉ chạy nhánh nếu điều kiện đúng
`elif expression:`	Thử điều kiện tiếp theo khi nhánh trước không đúng
`else:`	Trường hợp còn lại không cần điều kiện mới
`in, and, or`	Kiểm tra thành viên hoặc kết hợp nhiều điều kiện

4. Khái niệm liên quan

Rẽ nhánh giúp chương trình chọn hành động theo dữ liệu thay vì lúc nào cũng chạy mọi lệnh. Python xét điều kiện `if` trước; nếu sai mới xét các `elif` theo thứ tự, và dùng `else` khi không nhánh nào đúng. Nhờ vậy một giá trị đầu vào được xếp đúng một trạng thái trong cùng chuỗi nhánh.

4.1. If / elif / else — Cấu trúc rẽ nhánh

Từ vựng trong mục: Condition — điều kiện; **If / elif / else** — nếu / nếu khác / còn lại.

Giả sử một cảm biến có nhiệt độ 30 °C: từ 35 °C trở lên gọi là `Hot`, từ 20 đến dưới 35 °C gọi là `Warm`, còn thấp hơn gọi là `Cold`. Chương trình kiểm tra điều kiện từ trên xuống: `if` xét ngưỡng cao, `elif` xét ngưỡng tiếp theo khi nhánh trước sai, còn `else` nhận các trường hợp còn lại. Chỉ một nhánh trong chuỗi này được chạy.

```
temperature = 30

if temperature >= 35:
    state = 'Hot'
elif temperature >= 20:
    state = 'Warm'
else:
    state = 'Cold'

print(state)
```

Kết quả khi chạy:

```
Warm
```

Nhiệt độ 30 °C không đạt mức `Hot` nhưng đạt mức `Warm`, nên nhánh `elif` được chọn.

4.2. Branch — Nhánh lệnh

Từ vựng trong mục: Branch — nhánh; **Boolean** — đúng/sai.

Một nhánh là nhóm câu lệnh được thụt vào dưới `if` hoặc `else`; nó chỉ chạy khi chương trình chọn nhánh ấy. Ví dụ kho có `0` sản phẩm: nếu số lượng dương thì in `Còn hàng`, ngược lại in `Hết hàng`. Dấu `:` mở đầu nhánh; bốn dấu cách cho biết câu lệnh nào thuộc nhánh.

```
stock = 0

if stock > 0:
    print('Còn hàng')
else:
    print('Hết hàng')
```

Kết quả khi chạy:

```
Hết hàng
```

Vì `stock > 0` sai, chương trình bỏ nhánh `if` và chạy nhánh `else`.

4.3. Threshold — Ngưỡng

Từ vựng trong mục: Threshold — ngưỡng; **Comparison** — phép so sánh.

Ngưỡng là ranh giới dùng để chia dữ liệu thành các nhóm. Chẳng hạn trung tâm yêu cầu từ 80 điểm trở lên để đạt mức B. Với điểm 85, `85 >= 80` là đúng. Dấu >= có nghĩa **lớn hơn hoặc bằng**, nên đúng cả khi điểm bằng chính mốc 80; điều này quan trọng khi phân loại các giá trị ở ranh giới.

```
print(85 >= 80)
```

Kết quả khi chạy:

```
True
```

Điểm 85 đạt ngưỡng 80. Nếu dùng > thì người đạt đúng 80 sẽ bị loại ngoài ý muốn.

4.4. Nested if — Điều kiện lồng nhau

Từ vựng trong mục: Nested if — điều kiện lồng; **Modulo** — chia lấy dư.

Đôi khi chỉ nên kiểm tra điều kiện thứ hai khi điều kiện đầu tiên đúng. Ví dụ chỉ phân loại số chẵn sau khi xác định số đó dương. `x > 0` mở nhánh ngoài; bên trong, `x % 2 == 0` hỏi phép chia cho 2 có dư 0 không. Với `x = 2`, cả hai điều kiện đúng nên in `even`.

```
x = 2

if x > 0:
    if x % 2 == 0:
        print('even')
```

Kết quả khi chạy:

```
even
```

Số 2 vừa dương vừa chia hết cho 2, nên chương trình đi vào cả hai nhánh `if`.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Xếp loại điểm

Từ vựng trong mục: Condition — điều kiện.

Tình huống thực tế: Trung tâm đào tạo xếp điểm từ 90 trở lên là A, từ 80 đến dưới 90 là B, còn lại ghi `Other`. Một học viên đạt 85 điểm trên thang 100.

Yêu cầu: Viết chương trình in mức xếp loại của học viên.

Vì sao chọn cách giải này? Dùng `if` xét mốc cao nhất trước, `elif` xét mốc 80 khi chưa đạt 90; `else` xử lý các điểm còn lại.

Quy trình giải:

1. Lưu điểm `85`.
2. Kiểm tra điểm từ 90 trở lên.
3. Nếu không đạt, kiểm tra từ 80 trở lên.
4. In loại tương ứng.

```
score=85
if score>=90:
    grade="A"
elif score>=80:
    grade="B"
else:
```

```
    grade="Other"
print(grade)
```

Kết quả khi chạy:

```
B
```

Kết quả: `B` — 85 điểm đạt mức B theo quy tắc đã nêu.

Tình huống 2 — Cảnh báo hàng gần hết

Từ vựng trong mục: Branch — nhánh.

Tình huống thực tế: Cửa hàng chia tồn kho thành ba trạng thái: hết hàng nếu số lượng không lớn hơn 0, gần hết nếu còn từ 1 đến 5 món, bình thường nếu còn trên 5 món. Một sản phẩm hiện còn 4 món.

Yêu cầu: Viết chương trình in trạng thái tồn kho bằng `Out`, `Low` hoặc `OK`.

Vì sao chọn cách giải này? Dùng `if/elif/else` để chỉ chọn một trong ba trạng thái; kiểm tra hết hàng trước khi kiểm tra gần hết.

Quy trình giải:

1. Lưu số lượng `qty = 4`.
2. Kiểm tra hết hàng (`qty <= 0`).
3. Nếu chưa hết, kiểm tra gần hết (`qty <= 5`).
4. In trạng thái phù hợp.

```
qty=4
if qty<=0:
    state="Out"
elif qty<=5:
    state="Low"
else:
    state="OK"
print(state)
```

Kết quả khi chạy:

```
Low
```

Kết quả: `Low` — còn 4 món nên hàng đang gần hết.

Tình huống 3 — Xét VIP

Từ vựng trong mục: Threshold — ngưỡng.

Tình huống thực tế: Cửa hàng chỉ công nhận VIP khi khách là thành viên ít nhất 5 năm và đã chi tối thiểu 1.000.000 đồng. Một khách đã tham gia 6 năm và chi 1.500.000 đồng.

Yêu cầu: In `VIP` nếu khách đạt cả hai điều kiện, ngược lại in `Regular`.

Vì sao chọn cách giải này? `and` nối hai phép so sánh; `if` chọn nhánh VIP khi cả hai đúng và `else` xử lý trường hợp còn lại.

Quy trình giải:

1. Lưu thời gian tham gia và tổng tiền đã chi.
2. So sánh từng giá trị với ngưỡng.
3. Kết hợp hai điều kiện bằng `and`.
4. In loại khách theo nhánh phù hợp.

```
years,paid=6,1500000
if years>=5 and paid>=1000000:
    print("VIP")
else:
    print("Regular")
```

Kết quả khi chạy:

```
VIP
```

Kết quả: `VIP` — khách đạt cả hai ngưỡng quy định.

Tình huống 4 — Kiểm tra cảm biến

Từ vựng trong mục: Nested if — điều kiện lồng.

Tình huống thực tế: Hệ thống giám sát nhận trạng thái kết nối `online = True` và nhiệt độ `82` °C. Khi thiết bị mất kết nối, màn hình phải hiển thị `Offline`; nếu kết nối, nhiệt độ từ 80 °C trở lên báo `Hot`, còn thấp hơn báo `Normal`.

Yêu cầu: Viết chương trình hiển thị đúng trạng thái thiết bị.

Vì sao chọn cách giải này? Kiểm tra kết nối bằng `if` trước; chỉ khi đang online mới xét nhiệt độ bằng biểu thức điều kiện. Không kết luận nhiệt độ của thiết bị đã mất kết nối.

Quy trình giải:

1. Lưu trạng thái online và nhiệt độ.
2. Nếu online, so sánh nhiệt độ với mốc 80 °C.
3. In `Hot` hoặc `Normal`; nếu offline, in `Offline`.

```
online,temp=True,82
if online:
    print("Hot" if temp>=80 else "Normal")
else:
    print("Offline")
```

Kết quả khi chạy:

```
Hot
```

Kết quả: `Hot` — thiết bị đang kết nối và nhiệt độ 82 °C đạt ngưỡng cảnh báo.

Tình huống 5 — Tính quý của tháng

Từ vựng trong mục: Condition — điều kiện.

Tình huống thực tế: Một phần mềm báo cáo cần đổi số tháng thành quý trong năm: tháng 1–3 là quý 1, 4–6 là quý 2, 7–9 là quý 3, 10–12 là quý 4. Tháng cần kiểm tra là tháng 10.

Yêu cầu: Viết chương trình in số quý của tháng đã cho; nếu tháng không thuộc khoảng 1–12, in `Invalid`.

Vì sao chọn cách giải này? Dùng `if` kiểm tra tháng hợp lệ trước, rồi phép chia nguyên `//` để xác định quý; `else` bắt tháng ngoài khoảng.

Quy trình giải:

1. Lưu tháng bằng `month = 10`.
2. Kiểm tra `1 <= month <= 12`.
3. Nếu hợp lệ, tính `(month - 1) // 3 + 1`; ngược lại báo `Invalid`.

```
month=10
if 1<=month<=12:
    print((month-1)//3+1)
else:
    print("Invalid")
```

Kết quả khi chạy:

```
4
```

Kết quả: `4` — tháng 10 thuộc quý 4.

6. Nhầm lẫn dễ gặp

- Phải đặt dấu `:` sau `if`, `elif`, `else` và thụt các lệnh trong nhánh.
- Nếu một `if`/`elif` đã đúng, các nhánh sau trong cùng chuỗi sẽ không chạy.
- `else` không có điều kiện riêng. Không nhầm = (gán giá trị) với == (so sánh bằng nhau).
- Khi chia khoảng điểm hay nhiệt độ, phải chọn đúng >=, >, <= để không xếp nhầm giá trị tại ngưỡng.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does `if` do? — `if` dùng để làm gì?

A. Always runs every conditional branch — Luôn chạy mọi nhánh điều kiện

B. Runs its body only when the condition is false — Chỉ chạy thân khối khi điều kiện sai

C. Runs its body once regardless of the condition — Luôn chạy thân khối một lần bất kể điều kiện

D. Runs its indented body when the condition is true — Chạy thân khối thụt lề khi điều kiện đúng

Đáp án: D

Giải thích: `if` đánh giá điều kiện rồi chỉ chạy thân khối được thụt lề nếu điều kiện có giá trị đúng. Ví dụ với `x = 5`, `if x > 3:` sẽ chạy thân khối; không bắt buộc phải có `else`.

Q2. When is an `elif` condition checked? — Khi nào Python xét điều kiện `elif`?

A. Even after a preceding branch has matched — Kể cả khi nhánh trước đã đúng

B. Before the preceding `if` is checked — Trước khi kiểm tra `if` đứng trước

C. After `else` has executed — Sau khi `else` chạy

D. Only if earlier branches in the same chain did not match — Chỉ khi các nhánh trước trong cùng chuỗi không khớp

Đáp án: D

Giải thích: Python chỉ xét `elif` khi những điều kiện `if` hoặc `elif` đứng trước trong cùng chuỗi đều không khớp. Khi một nhánh trước đã đúng, các `elif` sau không được kiểm tra, nên không có nhiều nhánh cùng chạy.

Q3. When does an `else` block run? — Khi nào khối `else` chạy?

A. Every time the initial `if` is true — Mỗi khi `if` đầu tiên đúng

B. After any successful `elif` branch — Sau bất kỳ nhánh `elif` nào đúng

C. When no earlier condition in its chain was true — Khi mọi điều kiện trước trong cùng chuỗi đều sai

D. Only when a program raises an error — Chỉ khi chương trình phát sinh lỗi

Đáp án: C

Giải thích: `else` là nhánh dự phòng không có điều kiện riêng. Nó chỉ chạy khi mọi điều kiện `if` hoặc `elif` đứng trước trong cùng chuỗi đều sai, không chạy sau nhánh đã khớp.

Q4. What is a nested `if`? — `if` lồng nhau là gì?

A. An `if` statement inside another conditional block — Một lệnh `if` bên trong khối điều kiện khác

B. A condition joined by `and` — Điều kiện kết hợp bởi `and`

C. An `else` branch following `if` — Nhánh `else` theo sau `if`

D. Two `if` statements at the same indentation level — Hai lệnh `if` có cùng mức thụt lề

Đáp án: A

Giải thích: `if` lồng nhau là câu lệnh điều kiện nằm trong thân một câu lệnh điều kiện khác nên điều kiện trong chỉ được xét khi chương trình đi vào khối đó. Nó khác hai câu `if` độc lập, vốn có thể cùng chạy.

Q5. Which keyword introduces another conditional test? — Từ khóa nào bổ sung một điều kiện tiếp theo?

A. `if` — `if`

B. `else` — `else`

C. `elif` — `elif`

D. `in` — `in`

Đáp án: C

Giải thích: `elif` mở thêm một nhánh có điều kiện nối với `if` trước đó. Khác `else`, nhánh này có điều kiện riêng, ví dụ `elif score >= 80:`.

Q6. Which keyword gives an unconditional fallback branch? — Từ khóa nào tạo nhánh còn lại không cần điều kiện?

A. `in` — `in`

B. `elif` — `elif`

C. `if` — `if`

D. `else` — `else`

Đáp án: D

Giải thích: `else` là nhánh dự phòng không ghi điều kiện trong chuỗi `if/elif/else`; Python chỉ vào đó khi các điều kiện trước sai. Không thể viết `else condition:` vì `else` không nhận điều kiện.

Q7. Which keyword starts a conditional branch? — Từ khóa nào bắt đầu một nhánh điều kiện?

A. `else` — `else`

B. `if` — `if`

C. `elif` — `elif`

D. `in` — `in`

Đáp án: B

Giải thích: Nhánh điều kiện đầu tiên dùng `if`, tiếp theo là biểu thức và dấu hai chấm. `elif` chỉ có thể nối sau một `if` trước đó, không tự mở đầu chuỗi mới.

Q8. Which operator asks whether an item belongs to a list? — Toán tử nào kiểm tra phần tử có trong List?

A. `if` — `if`

B. `elif` — `elif`

C. `in` — `in`

D. `else` — `else`

Đáp án: C

Giải thích: `in` kiểm tra phần tử thuộc tập hợp: `3 in [1, 3]` là `True`. Trong khi đó `if`, `elif`, `else` là từ khóa điều khiển luồng, không phải toán tử kiểm tra phần tử.

Q9. What does `score >= 80` include? — Điều kiện `score >= 80` bao gồm gì?

A. Only the score exactly equal to 80 — Chỉ điểm đúng bằng 80

B. Every score of at least 80, including 80 — Mọi điểm từ 80 trở lên, kể cả 80

C. Only scores strictly above 80 — Chỉ điểm lớn hơn 80

D. Scores below 80 — Điểm thấp hơn 80

Đáp án: B

Giải thích: >= bao gồm chính giá trị ngưỡng, nên `score >= 80` đúng với 80, 81 và các giá trị lớn hơn. Dùng so sánh nghiêm ngặt `score > 80` sẽ loại đúng điểm 80.

Q10. What is the difference between = and ==? — Dấu = khác == ra sao?

A. = assigns; == compares equality — = để gán; == so sánh bằng nhau

B. = compares; == assigns — = để so sánh; == để gán

C. Both operators assign variables — Cả hai toán tử đều dùng gán biến

D. Both operators compare equality — Cả hai toán tử so sánh bằng nhau

Đáp án: A

Giải thích: = gán giá trị như `score = 80`, còn == so sánh và trả Boolean như `score == 80`. Không thể thay hai toán tử cho nhau trong điều kiện `if`.

Q11. What defines the body of an `if` in Python? — Điều gì xác định thân khối `if` trong Python?

A. The indentation of following statements — Mức thụt lề của các câu lệnh theo sau

B. The number of printed statements — Số câu lệnh in ra

C. The order of imported libraries — Thứ tự thư viện được nhập

D. The variable name in the condition — Tên biến trong điều kiện

Đáp án: A

Giải thích: Thụt lề xác định câu lệnh nào thuộc thân `if`, thông thường bốn dấu cách cho mỗi cấp. Dấu hai chấm kết thúc dòng điều kiện, còn các dòng thụt lề đúng sau đó mới nằm trong nhánh.

Q12. How are cases checked in an `if/elif` chain? — Các trường hợp được xét theo thứ tự nào trong chuỗi `if/elif`?

A. All branches run concurrently — Mọi nhánh chạy đồng thời

B. Top to bottom, stopping after the first match — Từ trên xuống, dừng tại nhánh đúng đầu tiên

C. From the last `elif` upward — Từ `elif` cuối lên trên

D. By whichever condition is cheapest — Theo điều kiện tốn ít tính toán nhất

Đáp án: B

Giải thích: Python xét nhánh từ trên xuống và chạy nhánh đầu tiên có điều kiện đúng. Nếu điểm 95 cũng thỏa `>= 80`, đặt `>= 80` trước `>= 90` sẽ chọn mức điểm thấp hơn quá sớm.

Q13. What does `True and False` evaluate to? — `True and False` bằng gì?

A. `0` (integer) — `0` (số nguyên)

B. `True` (Boolean) — `True` (Boolean)

C. `None` — `None`

D. `False` (Boolean) — `False` (Boolean)

Đáp án: D

Giải thích: `and` với hai toán hạng Boolean chỉ trả `True` khi cả hai đều `True`. Vì vế thứ hai là `False`, `True and False` cho Boolean `False`, không phải số nguyên `0`.

Q14. What does `True or False` evaluate to? — `True or False` bằng gì?

A. `None` — `None`

B. `1` (integer) — `1` (số nguyên)

C. `True` (Boolean) — `True` (Boolean)

D. `False` (Boolean) — `False` (Boolean)

Đáp án: C

Giải thích: `or` với toán hạng Boolean chỉ cần một vế đúng; vế trái là `True` nên `True or False` cho Boolean `True`. Kết quả không cùng kiểu với số nguyên `1`.

Q15. What does `not True` evaluate to? — `not True` bằng gì?

A. `0` (integer) — `0` (số nguyên)

B. `True` (Boolean) — `True` (Boolean)

C. `None` — `None`

D. `False` (Boolean) — `False` (Boolean)

Đáp án: D

Giải thích: `not` đảo giá trị logic của toán hạng, nên `not True` cho Boolean `False`. Số nguyên `0` cũng được xem là sai trong điều kiện nhưng không phải giá trị trả về ở đây.

Q16. What does `5 % 2 == 0` evaluate to? — `5 % 2 == 0` có kết quả gì?

A. `False` (Boolean) — `False` (Boolean)

B. `None` — `None`

C. `0` (integer) — `0` (số nguyên)

D. `True` (Boolean) — `True` (Boolean)

Đáp án: A

Giải thích: Phần dư `5 % 2` là `1`; vì vậy so sánh `1 == 0` là sai. Toàn bộ biểu thức trả về Boolean `False`, nghĩa là 5 không phải số chẵn.

Q17. How many branches run in one `if/elif/else` chain? — Có bao nhiêu nhánh chạy trong một chuỗi `if/elif/else`?

A. Every branch whose condition is true — Mọi nhánh có điều kiện đúng

B. Every `elif` plus `else` — Mọi nhánh `elif` và `else`

C. At most one branch, possibly none without `else` — Tối đa một nhánh, có thể không nhánh nào nếu thiếu `else`

D. Exactly two branches always — Luôn đúng hai nhánh

Đáp án: C

Giải thích: Trong một chuỗi `if/elif/else`, chương trình chạy tối đa một nhánh: nhánh đúng đầu tiên hoặc `else` khi không nhánh nào đúng. Nếu không có `else` và mọi điều kiện đều sai thì không nhánh nào chạy.

Q18. What happens if an `if` condition is false and there is no `else`? — Nếu `if` sai và không có `else` thì sao?

A. The body runs once regardless — Thân nhánh vẫn chạy một lần

B. The body is skipped and execution continues — Bỏ qua thân nhánh và tiếp tục chương trình

C. A `SyntaxError` is raised — Phát sinh `SyntaxError`

D. Python inserts and runs an implicit `else` — Python tự chèn và chạy `else`

Đáp án: B

Giải thích: Khi điều kiện `if` sai, thân nhánh đó được bỏ qua. Nếu không có `else`, chương trình tiếp tục sau khối điều kiện; đây là Python hợp lệ, không phải lỗi cú pháp.

Q19. How can two conditions both be required for access? — Cách nào yêu cầu đồng thời hai điều kiện để cho phép truy cập?

A. Use == alone — Chỉ dùng ==

B. Use `and` — Dùng `and`

C. Use `or` — Dùng `or`

D. Use `not` — Dùng `not`

Đáp án: B

Giải thích: Phép `and` bắt buộc đồng thời hai điều kiện như `has_ticket and age >= 18`. Dùng `or` sẽ chấp nhận người chỉ thỏa một điều kiện, nên không đảm bảo đủ cả hai yêu cầu.

Q20. In one `if/elif` grade chain, which order correctly tests `score >= 90`, `score >= 80`, and `score >= 70` to select the highest applicable grade? — Trong một chuỗi `if/elif` phân loại điểm, cần xét `score >= 90`, `score >= 80`, `score >= 70` theo thứ tự nào để chọn mức cao nhất phù hợp?

A. Test `>= 90`, then `>= 80`, then `>= 70` — Xét `>= 90`, rồi `>= 80`, rồi `>= 70`

B. Write independent `if` statements to select a single grade — Viết các `if` độc lập để chọn duy nhất một nhóm

C. Test `>= 70` before `>= 90` — Xét `>= 70` trước `>= 90`

D. Use any order because all matches run — Thứ tự tùy ý vì mọi nhánh đúng đều chạy

Đáp án: A

Giải thích: Chuỗi điều kiện chọn nhánh đúng đầu tiên; hãy xét ngưỡng điểm cao nhất trước, ví dụ `>= 90`, rồi `>= 80`, `>= 70`. Nếu bắt đầu từ `>= 70`, điểm 95 sẽ bị xếp nhóm quá sớm.

8. Nối sang bài tiếp theo

Toán tư → Điều kiện → Phân loại theo quy tắc.

PY-05 — Vòng lặp for, while, break và continue

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Iteration — lần lặp; **For loop** — vòng for.

`Điều kiện → Vòng lặp → Xử lý hàng loạt`

Học xong dùng để làm gì? Tự động tính toán hoặc kiểm tra nhiều dòng dữ liệu bằng for/while, tránh sao chép cùng thao tác.

Kết nối kiến thức tiếp theo: Sang PY-06, lưu nhiều giá trị trong List để vòng lặp có dữ liệu thực tế mà xử lý.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Iteration	lần lặp	Một lượt thực hiện trong quá trình lặp
For loop	vòng for	Lặp qua từng phần tử trong một nhóm
While loop	vòng while	Lặp khi một điều kiện còn đúng
Break/continue	dừng/bỏ qua	Dừng hẳn vòng lặp hoặc bỏ qua lượt hiện tại

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`range(start,stop)`	Tạo chỉ số nguyên, không bao gồm stop
`enumerate(iterable)`	Lấy đồng thời vị trí và giá trị
`break`	Dừng sớm khi đã tìm thấy
`continue`	Bỏ qua dữ liệu của một lượt lặp

4. Khái niệm liên quan

Vòng lặp cho phép chương trình thực hiện một công việc nhiều lần mà không phải sao chép cùng câu lệnh. Một lần thực hiện được gọi là **lần lặp**. Dùng `for` khi duyệt các phần tử đã có; dùng `while` khi còn phải lặp theo một điều kiện. Hai lệnh `break` và `continue` kiểm soát cách vòng lặp kết thúc hoặc bỏ qua một lượt.

4.1. Loop / Iteration — Vòng lặp và lần lặp

Từ vựng trong mục: Loop — vòng lặp; **Iteration** — lần lặp.

Ví dụ cần đánh số hai lượt kiểm tra liên tiếp. Thay vì viết hai lần `print()`, ta dùng `for` và `range(2)` để Python lần lượt cấp giá trị 0 và 1 cho biến `i`. Mỗi lần thân vòng lặp chạy là một **lần lặp**; ở ví dụ có hai lần lặp và hai dòng được in ra.

```
for i in range(2):
    print(i)
```

Kết quả khi chạy:

```
0
1
```

`range(2)` bắt đầu từ 0 và dừng trước 2, nên in 0 rồi 1.

4.2. For loop — Duyệt từng phần tử

Từ vựng trong mục: For loop — vòng `for`; **Iterable** — dữ liệu duyệt được.

Khi có sẵn danh sách `[1, 2]` đại diện hai mã số, `for` lấy từng phần tử theo thứ tự rồi chạy thân vòng lặp. Biến `x` lần lượt nhận 1 rồi 2; nếu danh sách có thêm phần tử, code không cần thêm một câu `print()` riêng cho phần tử mới.

```
for x in [1, 2]:
    print(x)
```

Kết quả khi chạy:

```
1
2
```

Mỗi phần tử tạo ra một dòng đầu ra, theo thứ tự đã lưu trong danh sách.

4.3. While loop — Lặp khi điều kiện còn đúng

Từ vựng trong mục: While loop — vòng `while`; **Condition** — điều kiện.

`while` kiểm tra một điều kiện **trước mỗi lượt**. Ví dụ số việc còn lại là `n = 2`; mỗi lượt hoàn thành một việc nên giảm `n` đi 1. Khi `n` bằng 0 thì `n > 0` sai, vòng lặp dừng. Điều kiện phải có cơ hội trở thành sai để tránh lặp vô hạn.

```
n = 2

while n > 0:
    n -= 1

print(n)
```

Kết quả khi chạy:

```
0
```

Hai lượt giảm từ 2 xuống 1 rồi 0; câu `print` bên ngoài vòng lặp cho thấy giá trị cuối cùng.

4.4. Break / continue — Dừng hoặc bỏ qua

Từ vựng trong mục: Break — dừng vòng lặp; **Continue** — bỏ qua lượt hiện tại.

`continue` bỏ phần còn lại của **lượt đang chạy** nhưng vẫn xét lượt tiếp theo. `break` thoát khỏi **toàn bộ vòng lặp hiện tại**. Ví dụ duyệt `[0, 1, 2, 3]`: bỏ qua 0, in 1 và 2, rồi dừng khi gặp 3. Hai lệnh có tác dụng khác nhau dù đều làm một số dòng không được thực hiện.

```
for x in [0, 1, 2, 3]:
    if x == 0:
        continue
    if x == 3:
        break
    print(x)
```

Kết quả khi chạy:

```
1
2
```

Số 0 bị bỏ qua bởi `continue`; khi đến 3, `break` dừng hẳn nên không in 3.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tổng hợp doanh thu

Từ vựng trong mục: Iteration — lần lặp.

Tình huống thực tế: Một cửa hàng ghi ba khoản tiền bán hàng trong ngày là 120, 80 và 200 nghìn đồng.

Yêu cầu: Dùng vòng lặp tính tổng doanh thu trong ngày (đơn vị: nghìn đồng).

Vì sao chọn cách giải này? Vòng `for` lấy từng khoản tiền và cộng dồn vào `total`, giúp dùng lại cách tính khi số đơn hàng tăng.

Quy trình giải:

1. Lưu ba khoản tiền trong `orders`.
2. Đặt `total = 0`.
3. Dùng `for` cộng từng khoản vào `total` và in tổng.

```
orders=[120,80,200]
total=0
for value in orders:
    total+=value
print(total)
```

Kết quả khi chạy:

```
400
```

Kết quả: `400` nghìn đồng, tương đương 400.000 đồng.

Tình huống 2 — Loại dòng trống

Từ vựng trong mục: For loop — vòng for.

Tình huống thực tế: Một trạm đo nhiệt độ nhận bốn lần đọc `[30, None, 35, 33]` °C. `None` nghĩa là lần đo không có số liệu, không phải 0 °C.

Yêu cầu: Dùng vòng lặp tạo danh sách chỉ chứa các nhiệt độ đo được.

Vì sao chọn cách giải này? Dùng `continue` để bỏ qua phần tử `None`, còn các giá trị số được đưa vào danh sách `valid`.

Quy trình giải:

1. Tạo danh sách dữ liệu và danh sách `valid` rỗng.
2. Duyệt từng giá trị bằng `for`.
3. Gặp `None` thì bỏ qua lượt này; còn lại thêm vào `valid`.
4. In danh sách kết quả.

```
data=[30,None,35,33]
valid=[]
for x in data:
    if x is None:
        continue
    valid.append(x)
print(valid)
```

Kết quả khi chạy:

```
[30, 35, 33]
```

Kết quả: `[30, 35, 33]` — chỉ còn ba lần đo có số liệu.

Tình huống 3 — Tìm hàng hết tồn

Từ vựng trong mục: While loop — vòng while.

Tình huống thực tế: Kho có bốn vị trí hàng, số lượng theo thứ tự là `[5, 3, 0, 4]`. Vị trí đầu tiên được đánh số `0`. Nhân viên muốn tìm vị trí đầu tiên đã hết hàng.

Yêu cầu: In vị trí đầu tiên có số lượng bằng `0`, rồi dừng tìm kiếm.

Vì sao chọn cách giải này? `enumerate()` cung cấp cả vị trí và số lượng; `break` dừng vòng lặp ngay khi tìm thấy vị trí hết hàng đầu tiên.

Quy trình giải:

1. Duyệt danh sách bằng `enumerate(stocks)`.
2. Kiểm tra số lượng tại từng vị trí.
3. Khi gặp `0`, in vị trí và dùng `break` để dừng.

```
stocks=[5,3,0,4]
for i,x in enumerate(stocks):
    if x==0:
        print("Hết hàng:",i)
        break
```

Kết quả khi chạy:

```
Hết hàng: 2
```

Kết quả: `Hết hàng: 2` — vị trí thứ ba trong danh sách (chỉ số 2) đã hết hàng.

Tình huống 4 — Giới hạn kết nối

Từ vựng trong mục: Break/continue — dừng/bỏ qua.

Tình huống thực tế: Ứng dụng thử kết nối máy chủ tối đa ba lần. Trong ví dụ mô phỏng, lần thử thứ nhất thất bại và lần thứ hai kết nối thành công.

Yêu cầu: Viết vòng lặp dừng khi kết nối thành công hoặc đã thử đủ ba lần, rồi in số lần thử và trạng thái kết nối.

Vì sao chọn cách giải này? `while` phù hợp vì chưa biết cần thử bao nhiêu lượt. Điều kiện `attempts < 3 and not connected` ngăn vòng lặp chạy mãi và dừng sớm khi kết nối được.

Quy trình giải:

1. Đặt số lần thử bằng 0, trạng thái kết nối là `False`.
2. Trong khi chưa đạt giới hạn và chưa kết nối, tăng số lần thử.
3. Mô phỏng kết nối thành công ở lần thứ hai.
4. In số lần đã thử và trạng thái.

```
attempts,connected=0,False
while attempts<3 and not connected:
    attempts+=1
    connected=(attempts==2)
print(attempts,connected)
```

Kết quả khi chạy:

```
2 True
```

Kết quả: `2 True` — chương trình thử hai lần rồi dừng vì đã kết nối. Đây là mô phỏng, không tạo kết nối mạng thật.

Tình huống 5 — Lập bảng lịch học

Từ vựng trong mục: Iteration — lần lặp.

Tình huống thực tế: Trung tâm đào tạo có hai học viên `An`, `Lan`; mỗi người cần lịch học cho ngày 1, 2 và 3.

Yêu cầu: In từng cặp tên học viên và ngày học, tổng cộng sáu dòng.

Vì sao chọn cách giải này? Dùng hai vòng `for` lồng nhau: vòng ngoài chọn học viên, vòng trong chọn từng ngày cho học viên đó.

Quy trình giải:

1. Tạo danh sách hai học viên.
2. Duyệt từng học viên bằng vòng `for` ngoài.
3. Với mỗi học viên, lặp `range(1, 4)` để lấy ngày 1, 2, 3.
4. In tên và ngày.

```
students=["An","Lan"]
for student in students:
    for day in range(1,4):
        print(student,day)
```

Kết quả khi chạy:

```
An 1
An 2
An 3
Lan 1
Lan 2
Lan 3
```

Kết quả: Sáu dòng: `An 1`, `An 2`, `An 3`, `Lan 1`, `Lan 2`, `Lan 3`.

6. Nhầm lẫn dễ gặp

- `range(1, 4)` tạo `1, 2, 3`, không chứa `4`.
- `break` dừng toàn bộ vòng lặp hiện tại, còn `continue` chỉ bỏ qua phần còn lại của một lượt.
- Với `while`, phải thay đổi biến hoặc trạng thái để vòng lặp có thể kết thúc.
- `enumerate()` mặc định đếm vị trí từ 0; dùng `start=1` nếu muốn đếm từ 1.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a `for` loop normally iterate over? — Vòng `for` thường duyệt qua gì?

A. Only dictionary keys — Chỉ khóa của dictionary

B. Successive items from an iterable — Các phần tử liên tiếp của đối tượng duyệt được

C. Only a Boolean condition — Chỉ một điều kiện Boolean

D. Always index-value pairs — Luôn là cặp chỉ số-giá trị

Đáp án: B

Giải thích: Vòng `for` lần lượt lấy phần tử từ đối tượng duyệt được, ví dụ các giá trị `0`, `1`, `2` của `range(3)`. Cách hoạt động này khác `while` kiểm tra lại điều kiện ở mỗi lượt.

Q2. When does a `while` loop repeat? — Khi nào vòng `while` tiếp tục lặp?

A. Always for a predetermined count — Luôn theo số lượt đã xác định

B. Until exactly one iteration has completed — Chỉ đến khi chạy xong đúng một lượt

C. As long as its condition remains true — Chừng nào điều kiện vẫn đúng

D. Only when iterating over a list — Chỉ khi duyệt một list

Đáp án: C

Giải thích: Vòng `while` kiểm tra điều kiện trước mỗi lượt và chỉ lặp khi điều kiện còn đúng. Ví dụ `while n < 3:` dừng khi `n` đạt 3 nếu thân vòng lặp tăng `n` sau mỗi lượt.

Q3. What is an iteration? — Một lần lặp là gì?

A. A Boolean test before the loop — Phép kiểm tra Boolean trước vòng lặp

B. Two loops inside each other — Hai vòng lặp lồng nhau

C. One pass through the loop body — Một lượt thực hiện thân vòng lặp

D. A collection that can be iterated — Một tập hợp có thể duyệt

Đáp án: C

Giải thích: Một lần lặp là một lượt chạy thân vòng lặp. Với `for x in range(3):`, thân vòng lặp chạy ba lần, lần lượt `x` bằng 0, 1, 2.

Q4. What are nested loops? — Vòng lặp lồng nhau là gì?

A. A loop that runs exactly once — Một vòng chỉ chạy đúng một lần

B. Two loops written one after another — Hai vòng lặp viết nối tiếp nhau

C. A `for` and `while` with the same variable — `for` và `while` dùng cùng một biến

D. A loop placed inside another loop's body — Một vòng đặt trong thân vòng khác

Đáp án: D

Giải thích: Vòng lặp lồng nhau đặt một vòng trong thân vòng khác. Nếu vòng ngoài có 2 phần tử và vòng trong có 3 phần tử, thân vòng trong có thể chạy 2 nhân 3 bằng 6 lần, ứng với từng cặp kết hợp.

Q5. What does `break` do inside a loop? — `break` làm gì trong vòng lặp?

A. Skips only the current iteration's remaining statements — Chỉ bỏ qua các lệnh còn lại của lượt hiện tại

B. Does nothing and continues with the next statement — Không làm gì rồi chạy lệnh tiếp

C. Immediately exits the entire Python process — Thoát toàn bộ tiến trình Python

D. Immediately exits the innermost enclosing loop — Thoát ngay vòng lặp bao gần nhất

Đáp án: D

Giải thích: `break` kết thúc ngay vòng lặp bao gần nhất đang chạy. Với vòng lặp lồng nhau, nó không tự kết thúc mọi vòng ngoài; `continue` thì vẫn để vòng lặp tiếp tục.

Q6. What does `continue` do? — `continue` có tác dụng gì?

A. Skips the remaining body and moves toward the next iteration — Bỏ phần thân còn lại và chuyển tới lượt kế tiếp

B. Restarts the complete program — Khởi động lại toàn bộ chương trình

C. Exits the current loop completely — Kết thúc hẳn vòng lặp hiện tại

D. Acts as a no-op placeholder — Là câu lệnh giữ chỗ không thao tác

Đáp án: A

Giải thích: `continue` bỏ qua những câu lệnh còn lại trong lượt hiện tại rồi tiếp tục theo loại vòng lặp. Với `for`, chương trình lấy phần tử kế tiếp; với `while`, nó kiểm tra lại điều kiện.

Q7. What does `pass` do? — `pass` làm gì?

A. Skips the remainder of the iteration — Bỏ phần còn lại của lượt lặp

B. Rechecks the condition without running later statements — Kiểm tra lại điều kiện mà bỏ qua các lệnh sau

C. Performs no operation and then continues normally — Không thao tác gì rồi tiếp tục bình thường

D. Leaves the loop — Rời khỏi vòng lặp

Đáp án: C

Giải thích: `pass` là câu lệnh hợp lệ nhưng không làm gì, hữu ích để giữ chỗ cho thân khối bắt buộc. Khác `continue`, nó không bỏ qua những câu lệnh phía sau trong cùng lượt lặp.

Q8. When does a loop's `else` execute? — `else` của vòng lặp chạy khi nào?

A. Every time a loop body executes — Mỗi khi thân vòng lặp chạy

B. When the loop finishes normally without `break` — Khi vòng lặp kết thúc bình thường mà không gặp `break`

C. Only when `break` was executed — Chỉ khi đã chạy `break`

D. At the start of the first iteration — Khi bắt đầu lượt lặp đầu

Đáp án: B

Giải thích: `else` của vòng lặp chạy sau khi vòng lặp kết thúc bình thường: `for` duyệt hết phần tử hoặc điều kiện `while` thành sai. Nó bị bỏ qua nếu `break` kết thúc vòng lặp, khác `else` dự phòng của `if`.

Q9. What does `range(1, 4)` represent? — `range(1, 4)` biểu diễn gì?

A. Values 1, 2, 3 — Các giá trị 1, 2, 3

B. Values 1, 2, 3, 4 — Các giá trị 1, 2, 3, 4

C. Values 1, 3 — Các giá trị 1, 3

D. Values 0, 1, 2, 3 — Các giá trị 0, 1, 2, 3

Đáp án: A

Giải thích: `range(1, 4)` biểu diễn các số 1, 2, 3 vì mốc dừng 4 không được lấy. Nếu muốn bao gồm 4, cần đặt mốc dừng là 5.

Q10. What does `enumerate(items)` provide? — `enumerate(items)` cung cấp gì?

A. Pairs sorted by item value — Các cặp được sắp theo giá trị

B. Pairs of index and item — Các cặp chỉ số và phần tử

C. Only the items, without their indexes — Chỉ các phần tử, không kèm chỉ số

D. Only the indexes, without items — Chỉ chỉ số, không kèm phần tử

Đáp án: B

Giải thích: `enumerate(items)` tạo các cặp chỉ số và phần tử, mặc định bắt đầu từ chỉ số 0. Với `['a', 'b']`, khi duyệt sẽ nhận `(0, 'a')` rồi `(1, 'b')`; hàm không sắp xếp danh sách.

Q11. What does `zip(a, b)` do? — `zip(a, b)` làm gì?

A. Pairs elements at corresponding positions — Ghép phần tử ở cùng vị trí

B. Concatenates both inputs into one sequence — Nối hai đầu vào thành một dãy

C. Sorts the inputs together — Sắp xếp hai đầu vào cùng nhau

D. Generates every possible pair — Tạo mọi cặp kết hợp

Đáp án: A

Giải thích: `zip(a, b)` ghép phần tử ở cùng vị trí, ví dụ `list(zip([1, 2], ['x', 'y']))` cho `[(1, 'x'), (2, 'y')]`. Mặc định hàm dừng khi dãy ngắn nhất hết phần tử, chứ không tạo mọi cặp kết hợp.

Q12. What does `reversed(items)` provide for a sequence? — `reversed(items)` cung cấp gì cho một dãy?

A. Always returns a fully materialized list — Luôn trả về list đã tạo sẵn

B. Provides an iterator in reverse positional order — Cung cấp iterator theo thứ tự vị trí đảo ngược

C. Sorts elements from largest to smallest — Sắp phần tử từ lớn đến nhỏ

D. Preserves the original order — Giữ nguyên thứ tự ban đầu

Đáp án: B

Giải thích: `reversed()` cung cấp bộ duyệt ngược thứ tự vị trí của dãy phù hợp. Với `[3, 1, 2]`, nó cho `2, 1, 3`, không phải sắp xếp số giảm dần `3, 2, 1`.

Q13. How many iterations does a complete `for` loop over `range(3)` execute? — Vòng `for` duyệt hết `range(3)` thực hiện bao nhiêu lượt lặp?

A. 3 iterations — 3 lượt lặp

B. 4 iterations — 4 lượt lặp

C. 0 iterations — 0 lượt lặp

D. 2 iterations — 2 lượt lặp

Đáp án: A

Giải thích: `range(3)` tạo các giá trị `0`, `1`, `2`: bắt đầu mặc định từ 0 và không lấy mốc dừng 3. Vòng `for` duyệt hết sẽ chạy thân đúng ba lượt; đây là số lượt lặp, không có nghĩa dãy chứa giá trị 3.

Q14. What does `range(2, 7, 2)` yield? — `range(2, 7, 2)` cho những số nào?

A. 3, 5 — 3, 5

B. 2, 4, 6 — 2, 4, 6

C. 2, 3, 4, 5, 6 — 2, 3, 4, 5, 6

D. 2, 4, 6, 8 — 2, 4, 6, 8

Đáp án: B

Giải thích: `range(start, stop, step)` bắt đầu tại 2, mỗi lượt cộng thêm 2 và không lấy giá trị đạt hoặc vượt mốc dừng 7. Vì vậy dãy là 2, 4, 6, không phải 2, 4, 6, 8.

Q15. What does `enumerate(['a','b'])` start indexing from? — `enumerate(['a','b'])` bắt đầu chỉ số từ đâu?

A. The number of items — Số phần tử

B. -1 — -1

C. 0 — 0

D. 1 — 1

Đáp án: C

Giải thích: `enumerate()` mặc định bắt đầu chỉ số bằng `0` nếu không truyền `start`. Với `['a', 'b']`, cặp đầu là `(0, 'a')`; `enumerate(items, start=1)` mới bắt đầu từ 1.

Q16. What does `list(zip([1,2], ['a']))` contain? — `list(zip([1,2], ['a']))` gồm gì?

A. `[(1, 'a'), (2, 'a')]` — `[(1, 'a'), (2, 'a')]`

B. `[(1, 2), ('a',)]` — `[(1, 2), ('a',)]`

C. `[(1, 'a'), (2, None)]` — `[(1, 'a'), (2, None)]`

D. `[(1, 'a')]` — `[(1, 'a')]`

Đáp án: D

Giải thích: `zip()` lấy một phần tử từ mỗi dãy ở một lượt và dừng khi dãy ngắn hơn hết. Vì vậy `list(zip([1,2], ['a']))` chỉ chứa `[(1, 'a')]`, không có cặp chứa 2.

Q17. What is a common cause of an infinite `while` loop? — Nguyên nhân thường gặp khiến `while` lặp vô hạn là gì?

A. A function containing `pass` is never called — Một hàm có `pass` không được gọi

B. A `break` is reached in the first iteration — Gặp `break` ở lượt đầu

C. A `for` loop exhausts a finite iterable — `for` duyệt hết một dãy hữu hạn

D. The `while` condition stays true and no exit is reached — Điều kiện `while` luôn đúng và không gặp lối thoát

Đáp án: D

Giải thích: Vòng `while` có thể lặp vô hạn khi điều kiện luôn đúng, ví dụ `while n < 3:` không bao giờ thay đổi `n` và không gặp `break`. Cần cập nhật biến hướng tới điều kiện sai hoặc chủ động có lối thoát.

Q18. How can a loop ignore invalid data and continue processing? — Làm sao bỏ qua dữ liệu lỗi nhưng vẫn xử lý phần tử sau?

A. Use `continue` for that invalid item — Dùng `continue` với phần tử lỗi

B. Use `return` to exit the whole function — Dùng `return` thoát toàn bộ hàm

C. Use only `pass` in the invalid-data branch — Chỉ dùng `pass` trong nhánh dữ liệu lỗi

D. Use `break` when the invalid item appears — Dùng `break` khi gặp phần tử lỗi

Đáp án: A

Giải thích: Dùng `continue` khi cần bỏ qua một phần tử không hợp lệ nhưng vẫn xử lý các phần tử sau. `break` sẽ kết thúc vòng lặp, còn chỉ dùng `pass` không ngăn những lệnh phía sau xử lý phần tử lỗi.

Q19. How can a search stop immediately after a match? — Làm sao dừng tìm kiếm ngay khi đã thấy phần tử phù hợp?

A. Use `pass` after the match — Dùng `pass` sau khi khớp

B. Use `continue` after the match — Dùng `continue` sau khi khớp

C. Use `else` instead of checking — Dùng `else` thay kiểm tra

D. Use `break` after the match — Dùng `break` sau khi khớp

Đáp án: D

Giải thích: Dùng `break` ngay khi tìm thấy phần tử cần tìm vì không cần kiểm tra những phần tử còn lại. `continue` chỉ chuyển tới phần tử tiếp theo, còn `pass` không dừng việc tìm kiếm.

Q20. How do you traverse every combination of two small collections? — Làm sao duyệt mọi kết hợp của hai nhóm nhỏ?

A. Use `enumerate()` on one collection — Dùng `enumerate()` trên một nhóm

B. Use two separate, nonnested loops — Dùng hai vòng độc lập không lồng nhau

C. Use one loop nested inside the other — Đặt một vòng lồng bên trong vòng còn lại

D. Use `zip()` on the two collections — Dùng `zip()` trên hai nhóm

Đáp án: C

Giải thích: Muốn duyệt mọi cặp kết hợp của hai nhóm, đặt vòng duyệt nhóm thứ hai bên trong vòng duyệt nhóm thứ nhất. Với 2 phần tử ngoài và 3 phần tử trong, ta được 6 cặp; `zip()` chỉ ghép phần tử cùng vị trí.

8. Nối sang bài tiếp theo

`Điều kiện → Vòng lặp → Xử lý hàng loạt.`

PY-06 — List, chỉ số, cắt danh sách và sắp xếp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: List — danh sách; **Index** — chỉ số.

`Vòng lặp → List → Vector/Array`

Học xong dùng để làm gì? Lưu và xử lý danh sách điểm, đơn hàng hoặc sản phẩm theo thứ tự; thêm, xóa, sửa và duyệt từng phần tử.

Kết nối kiến thức tiếp theo: PY-07 dùng Tuple cho dữ liệu cố định và Set để bỏ giá trị lặp.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
List	danh sách	Tập hợp có thứ tự, có thể thay đổi
Index	chỉ số	Vị trí tính từ 0 để lấy phần tử
Slice	cắt lát	Lấy đoạn phần tử theo start:stop
Mutability	khả năng sửa	Có thể cập nhật phần tử của danh sách

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`append(), extend(), insert()`	Thêm từng phần tử hoặc nhiều phần tử vào danh sách
`pop(), remove(), del`	Xóa theo vị trí hoặc giá trị
`sort(), sorted()`	Sắp xếp tại chỗ hoặc tạo danh sách mới
`items[start:stop]`	Lấy vùng con không bao gồm stop

4. Khái niệm liên quan

Một cửa hàng cần ghi các món trong giỏ theo đúng thứ tự khách thêm vào. **List** là cấu trúc Python lưu nhiều giá trị trong một biến, được viết bằng dấu ngoặc vuông `[]`. Từ giỏ hàng này, ta lần lượt học cách tạo danh sách, tìm theo vị trí, lấy một đoạn và thay đổi một món; đó cũng là nền tảng để duyệt dữ liệu bằng vòng `for`.

4.1. List — Danh sách

Từ vựng trong mục: List — danh sách; **Element** — phần tử.

List giữ các phần tử theo thứ tự, và mỗi phần tử có thể là tên món hàng, số hoặc một kiểu dữ liệu khác. Giỏ hàng ban đầu gồm `milk` (sữa) và `bread` (bánh mì); ta đặt cả hai vào biến `basket` để xử lý cùng nhau. Dùng dấu `[]` để tạo danh sách, dấu phẩy ngăn cách các phần tử.

```
basket = ['milk', 'bread']
print(basket)
```

Kết quả khi chạy:

```
['milk', 'bread']
```

Danh sách có hai phần tử theo thứ tự sữa trước, bánh mì sau. Khác với một chuỗi dài, từng món có thể được truy cập và thay đổi riêng.

4.2. Index — Vị trí phần tử

Từ vựng trong mục: Index — chỉ số; **Element** — phần tử.

Khi biết món cần lấy nằm ở vị trí nào, ta dùng **chỉ số**. Python bắt đầu đếm từ `0`: với `['milk', 'bread']`, `basket[0]` là `milk`, còn `basket[1]` là `bread`; `basket[-1]` cũng chỉ phần tử cuối. Chỉ số nằm ngoài giới hạn danh sách sẽ gây `IndexError`.

```
basket = ['milk', 'bread']
print(basket[0])
```

Kết quả khi chạy:

```
milk
```

Chỉ số `0` lấy đúng món sữa ở vị trí đầu tiên, không phải phần tử thứ hai.

4.3. Slice — Lấy một đoạn danh sách

Từ vựng trong mục: Slice — cắt lát; **Start/stop** — vị trí bắt đầu/kết thúc.

Nếu giỏ có ba món `milk`, `bread`, `rice` và chỉ muốn lấy hai món đầu, ta dùng `basket[:2]`. Với lát cắt `[start:stop]`, vị trí `start` được lấy, còn `stop` **không được lấy**. Bỏ trống `start` nghĩa là bắt đầu từ đầu danh sách. Lát cắt tạo list mới, không xóa phần tử khỏi giỏ gốc.

```
basket = ['milk', 'bread', 'rice']
print(basket[:2])
```

Kết quả khi chạy:

```
['milk', 'bread']
```

Kết quả gồm sữa và bánh mì; `rice` ở chỉ số `2` nằm ngoài đoạn được lấy.

4.4. Mutability — Thay đổi nội dung danh sách

Từ vựng trong mục: Mutability — khả năng thay đổi; **Assignment** — phép gán.

Khác với tuple ở bài kế tiếp, **List là kiểu có thể sửa** sau khi tạo. Nếu cửa hàng đổi món thứ hai từ `bread` thành `rice`, ta gán trực tiếp `basket[1] = 'rice'`. Danh sách vẫn có hai phần tử, chỉ giá trị tại vị trí 1 được thay. Muốn thêm món mới vào cuối thì dùng `append()`, không phải phép gán này.

```
basket = ['milk', 'bread']
basket[1] = 'rice'
print(basket)
```

Kết quả khi chạy:

```
['milk', 'rice']
```

Món thứ hai đổi thành gạo; món sữa đầu tiên vẫn giữ nguyên.

4.5. Sorting — Sắp xếp List

Từ vựng trong mục: Sorting — sắp xếp; **Ascending/descending** — tăng dần/giảm dần.

Khi có điểm ưu tiên `[7, 9, 4]`, ta thường muốn xem điểm cao trước. Hàm `sorted(scores, reverse=True)` tạo **List mới** theo thứ tự giảm dần; `reverse=False` (mặc định) cho thứ tự tăng dần. List ban đầu không bị sửa bởi `sorted()`, khác với phương thức `scores.sort()` vốn sắp xếp trực tiếp List gốc. Với ví dụ này, điểm 9 đứng đầu, rồi 7 và 4.

```
scores = [7, 9, 4]
ranked = sorted(scores, reverse=True)
print(ranked)
```

Kết quả khi chạy:

```
[9, 7, 4]
```

Danh sách kết quả được sắp theo điểm giảm dần. Các điểm số không bị thay đổi, chỉ thứ tự của chúng khác đi.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Giỏ hàng

Từ vựng trong mục: List — danh sách.

Tình huống thực tế: Một cửa hàng đang có giỏ hàng gồm `milk` (sữa) và `bread` (bánh mì). Khách muốn thêm `rice` (gạo) vào cuối giỏ.

Yêu cầu: In danh sách hàng sau khi thêm gạo.

Vì sao chọn cách giải này? `append()` thêm một phần tử vào cuối list mà vẫn giữ các món trước đó.

Quy trình giải:

1. Tạo list `basket` gồm sữa và bánh mì.
2. Gọi `basket.append("rice")`.
3. In toàn bộ giỏ hàng.

```
basket=["milk","bread"]
basket.append("rice")
print(basket)
```

Kết quả khi chạy:

```
['milk', 'bread', 'rice']
```

Kết quả: `['milk', 'bread', 'rice']` — giỏ có ba món theo đúng thứ tự.

Tình huống 2 — Lấy hai ưu tiên cao

Từ vựng trong mục: Index — chỉ số.

Tình huống thực tế: Một đội hỗ trợ nhận điểm ưu tiên của bốn yêu cầu là `[2, 9, 4, 7]`; điểm càng lớn thì yêu cầu càng cần xử lý trước.

Yêu cầu: In hai điểm ưu tiên cao nhất mà không thay đổi list gốc.

Vì sao chọn cách giải này? `sorted(..., reverse=True)` tạo danh sách mới từ cao xuống thấp, rồi `[:2]` lấy hai giá trị đầu.

Quy trình giải:

1. Lưu bốn điểm vào `scores`.
2. Sắp xếp giảm dần bằng `sorted(scores, reverse=True)`.
3. Cắt `[:2]` và in kết quả.

```
scores=[2,9,4,7]
print(sorted(scores,reverse=True)[:2])
```

Kết quả khi chạy:

```
[9, 7]
```

Kết quả: `[9, 7]` — hai mức ưu tiên cao nhất. List `scores` gốc không bị thay đổi.

Tình huống 3 — Lọc điểm đạt

Từ vựng trong mục: Slice — cắt lát.

Tình huống thực tế: Trung tâm đào tạo có điểm của bốn học viên là `[4, 7, 9, 3]` trên thang 10. Điểm đạt yêu cầu là từ 5 trở lên.

Yêu cầu: Tạo list mới chỉ chứa điểm đạt.

Vì sao chọn cách giải này? List comprehension `[x for x in scores if x >= 5]` duyệt từng điểm và chỉ giữ giá trị đạt ngưỡng.

Quy trình giải:

1. Lưu điểm vào `scores`.
2. Với mỗi `x`, kiểm tra `x >= 5`.
3. Thêm các điểm đạt vào `passed` và in list mới.

```
scores=[4,7,9,3]
passed=[x for x in scores if x>=5]
print(passed)
```

Kết quả khi chạy:

```
[7, 9]
```

Kết quả: `[7, 9]` — hai điểm đạt, vẫn giữ thứ tự xuất hiện ban đầu.

Tình huống 4 — Sửa tồn kho

Từ vựng trong mục: Mutability — khả năng sửa.

Tình huống thực tế: Kho lưu số lượng ba mặt hàng trong list `[8, 6, 9]`. Sau kiểm kê, mặt hàng thứ hai thực tế còn 4 sản phẩm, không phải 6.

Yêu cầu: Sửa lại số lượng mặt hàng thứ hai rồi in danh sách tồn kho mới.

Vì sao chọn cách giải này? List có thể chỉnh trực tiếp theo chỉ số; mặt hàng thứ hai nằm ở chỉ số `1`.

Quy trình giải:

1. Lưu `[8, 6, 9]` vào `stock`.
2. Gán `stock[1] = 4`.
3. In danh sách sau khi sửa.

```
stock=[8,6,9]
stock[1]=4
print(stock)
```

Kết quả khi chạy:

```
[8, 4, 9]
```

Kết quả: `[8, 4, 9]` — chỉ số lượng mặt hàng thứ hai được cập nhật.

Tình huống 5 — Hàng đợi yêu cầu

Từ vựng trong mục: List — danh sách.

Tình huống thực tế: Bộ phận hỗ trợ đang xử lý ba yêu cầu theo thứ tự `REQ01`, `REQ02`, `REQ03`. Yêu cầu đến trước phải được xử lý trước.

Yêu cầu: Lấy yêu cầu đầu hàng đợi ra để xử lý và in danh sách còn lại.

Vì sao chọn cách giải này? `pop(0)` lấy và xóa phần tử đầu list; phù hợp để minh họa hàng đợi nhỏ. Với hàng đợi lớn hoặc thường xuyên lấy đầu, `collections.deque` phù hợp hơn.

Quy trình giải:

1. Tạo list `queue` theo thứ tự yêu cầu đến.
2. Dùng `pop(0)` lấy yêu cầu đầu vào biến `done`.
3. In mã đã xử lý và list còn lại.

```
queue=["REQ01","REQ02","REQ03"]
done=queue.pop(0)
print(done,queue)
```

Kết quả khi chạy:

```
REQ01 ['REQ02', 'REQ03']
```

Kết quả: `REQ01 ['REQ02', 'REQ03']` — yêu cầu đầu đã được lấy ra, hai yêu cầu còn lại giữ nguyên thứ tự.

6. Nhầm lẫn dễ gặp

- Chỉ số list bắt đầu từ `0`; truy cập vượt ngoài list gây `IndexError`.
- `append([3,4])` thêm cả list như một phần tử, còn `extend([3,4])` thêm lần lượt `3` và `4`.
- `list.sort()` sửa list gốc và trả về `None`; `sorted(list)` tạo list mới.
- `pop(0)` sẽ lỗi nếu list rỗng. Với hàng đợi lớn, nên dùng `collections.deque`.
- Cắt list bằng `[:2]` không lấy phần tử có chỉ số `2`.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Python List? — List trong Python là gì?

A. A mapping of keys to values — Ánh xạ từ khóa sang giá trị

B. An ordered, mutable collection — Tập hợp có thứ tự và có thể sửa

C. An unordered collection of unique items — Tập hợp không thứ tự gồm phần tử duy nhất

D. An immutable sequence of items — Dãy phần tử bất biến

Đáp án: B

Giải thích: List trong Python là tập hợp có thứ tự và có thể thay đổi, nên mỗi phần tử có vị trí và có thể được sửa sau khi tạo. Ví dụ `[10, 20]` chứa hai phần tử theo thứ tự xác định; set không hỗ trợ chỉ số vị trí.

Q2. What is the index of the first list element? — Chỉ số của phần tử List đầu tiên là gì?

A. `0` — `0`

B. `-1` — `-1`

C. `len(a)` — `len(a)`

D. `1` — `1`

Đáp án: A

Giải thích: Chỉ số List bắt đầu từ `0`, vì vậy `a[0]` lấy phần tử đầu và `a[1]` lấy phần tử thứ hai. Chỉ số âm `-1` mang nghĩa khác: phần tử cuối.

Q3. For a List `a`, what does `a[:2]` return? — Với `a` là một List, `a[:2]` trả về gì?

A. Items from index 2 to the end — Các phần tử từ chỉ số 2 đến cuối

B. A new list of the first two items — List mới gồm hai phần tử đầu

C. A new list of the last two items — List mới gồm hai phần tử cuối

D. Only item at index 2 — Chỉ phần tử tại chỉ số 2

Đáp án: B

Giải thích: Lát cắt `a[:2]` mặc định bắt đầu tại chỉ số 0 và dừng trước chỉ số 2, tạo List mới gồm hai phần tử đầu. Với `[4, 5, 6]`, kết quả là `[4, 5]`, không phải phần tử tại chỉ số 2.

Q4. What does 'mutable' mean for a list? — List có tính mutable nghĩa là gì?

A. Its contents can be changed after creation — Có thể sửa nội dung sau khi tạo

B. Its elements cannot be changed — Không thể sửa các phần tử

C. Every element must have the same type — Mọi phần tử phải cùng kiểu

D. Its elements sort automatically — Các phần tử được tự động sắp xếp

Đáp án: A

Giải thích: Mutable nghĩa là có thể thay, thêm hoặc xóa phần tử trên List hiện có. Ví dụ gán `a = [1, 2]` rồi thực hiện `a[0] = 9` làm `a` thành `[9, 2]`; tuple không cho phép gán phần tử như vậy.

Q5. What does `append(x)` do? — `append(x)` làm gì?

A. Inserts `x` before the first item — Chèn `x` trước phần tử đầu

B. Removes the final element and returns it — Xóa phần tử cuối và trả về

C. Adds `x` as a single final item — Thêm `x` thành một phần tử cuối

D. Adds every element of `x` separately — Thêm riêng từng phần tử của `x`

Đáp án: C

Giải thích: `append(x)` thêm toàn bộ `x` thành đúng một phần tử cuối List và sửa List tại chỗ. Vì vậy thêm `[2, 3]` bằng `append` vào `[1]` tạo phần tử List lồng nhau, không thêm từng số; muốn thêm từng số thì dùng `extend()`.

Q6. What does `extend(items)` do? — `extend(items)` làm gì?

A. Deletes items matching `items` — Xóa phần tử khớp `items`

B. Appends the iterable as one nested item — Thêm cả iterable thành một phần tử lồng

C. Adds the individual elements from `items` — Thêm từng phần tử từ `items`

D. Replaces the first element with `items` — Thay phần tử đầu bằng `items`

Đáp án: C

Giải thích: `extend(items)` duyệt đối số và thêm từng phần tử riêng vào List hiện có. Với `a = [1]`, `a.extend([2, 3])` tạo `[1, 2, 3]`, còn `append([2, 3])` sẽ tạo `[1, [2, 3]]`.

Q7. What does `insert(0, x)` do? — `insert(0, x)` làm gì?

A. Replaces the first element with `x` — Thay phần tử đầu bằng `x`

B. Inserts `x` at the front, shifting the others — Chèn `x` vào đầu, đẩy các phần tử khác sang phải

C. Removes the first element — Xóa phần tử đầu

D. Adds `x` at the end — Thêm `x` vào cuối

Đáp án: B

Giải thích: `insert(0, x)` chèn `x` trước phần tử đầu tiên và đẩy các phần tử cũ sang phải. Ví dụ chèn `9` vào đầu `[1, 2]` cho `[9, 1, 2]`, không ghi đè số `1`.

Q8. What does `pop(0)` do on a nonempty list? — `pop(0)` làm gì với List không rỗng?

A. Removes and returns the last item — Xóa và trả phần tử cuối

B. Returns the first item without removing it — Trả phần tử đầu nhưng không xóa

C. Deletes the first occurrence of the value zero — Xóa lần xuất hiện đầu của giá trị không

D. Removes and returns the first item — Xóa và trả phần tử đầu

Đáp án: D

Giải thích: `pop(0)` xóa phần tử đầu của List không rỗng và trả chính phần tử đã xóa. Với `[7, 8]`, kết quả trả về là `7` và List còn `[8]`; chỉ đọc `a[0]` thì không xóa phần tử nào.

Q9. What does `remove(x)` do? — `remove(x)` dùng để làm gì?

A. Deletes every matching value — Xóa tất cả giá trị khớp

B. Returns a sorted copy — Trả bản sao đã sắp xếp

C. Deletes the first matching value — Xóa giá trị khớp đầu tiên

D. Deletes the item at index `x` — Xóa phần tử tại chỉ số `x`

Đáp án: C

Giải thích: `remove(x)` xóa lần xuất hiện đầu tiên có giá trị bằng `x`; nó tìm theo giá trị chứ không xem `x` là chỉ số. Với `[2, 3, 2]`, `remove(2)` còn `[3, 2]`, và giá trị không tồn tại sẽ gây `ValueError`.

Q10. What does `del a[1]` do? — `del a[1]` có tác dụng gì?

A. Returns the removed item — Trả về phần tử đã xóa

B. Deletes the item whose value is `1` — Xóa phần tử có giá trị bằng `1`

C. Deletes the item at index one — Xóa phần tử có chỉ số 1

D. Replaces the item at index one with `None` — Thay phần tử chỉ số 1 bằng `None`

Đáp án: C

Giải thích: `del a[1]` xóa phần tử tại chỉ số 1, tức phần tử thứ hai vì chỉ số bắt đầu từ 0. Khác `remove(1)`, lệnh này dùng vị trí; khác `pop(1)`, câu lệnh `del` không trả về giá trị bị xóa.

Q11. What does `a.sort()` return? — `a.sort()` trả về gì?

A. The sorted indexes — Các chỉ số đã sắp xếp

B. A new sorted list — List mới đã sắp xếp

C. The same list as a return value — Trả về chính List đó

D. `None` — `None`

Đáp án: D

Giải thích: `a.sort()` sắp List tại chỗ và trả về `None`, nên chính List cũ nhận thứ tự mới. Gán `result = a.sort()` sẽ cho `None`, không phải List mới đã sắp; muốn List riêng thì dùng `sorted(a)`.

Q12. What does `sorted(a)` do? — `sorted(a)` làm gì?

A. Reverses the original list in place — Đảo List gốc tại chỗ

B. Produces a set of unique items — Tạo Set các phần tử duy nhất

C. Creates a new sorted list — Tạo List mới đã sắp xếp

D. Sorts `a` in place and returns `None` — Sắp `a` tại chỗ và trả `None`

Đáp án: C

Giải thích: `sorted(a)` trả về List mới đã sắp xếp và không đổi thứ tự List gốc. Nếu `a = [3, 1]`, `sorted(a)` là `[1, 3]` còn `a` vẫn `[3, 1]`, khác `a.sort()`.

Q13. What does `a[-1]` read from a nonempty list? — `a[-1]` lấy gì từ List không rỗng?

A. The last element — Phần tử cuối

B. The first element — Phần tử đầu

C. The number of elements — Số phần tử

D. An `IndexError` for every nonempty list — Luôn có `IndexError` với List không rỗng

Đáp án: A

Giải thích: Chỉ số `-1` chỉ phần tử cuối của List không rỗng, bất kể List dài bao nhiêu. Với `[4, 7, 9]`, `a[-1]` là `9`; còn `len(a)` đếm đủ ba phần tử.

Q14. What does `len(a)` report? — `len(a)` cho biết gì?

A. The number of distinct values — Số giá trị khác nhau

B. The list's memory size in bytes — Kích thước List tính theo byte

C. The final valid index — Chỉ số hợp lệ cuối

D. The number of elements — Số phần tử

Đáp án: D

Giải thích: `len(a)` trả về số phần tử ở cấp ngoài của List; một List lồng trong được tính là một phần tử. Với `[1, [2, 3]]`, độ dài là `2`, không phải `3` và không phải chỉ số cuối `1`.

Q15. What happens for `a[10]` on a two-item list? — `a[10]` sẽ thế nào nếu List có hai phần tử?

A. The last list element — Phần tử cuối List

B. `None` — `None`

C. The integer `0` — Số nguyên `0`

D. `IndexError` — `IndexError`

Đáp án: D

Giải thích: List gồm hai phần tử chỉ có chỉ số dương hợp lệ `0` và `1`; đọc `a[10]` vượt giới hạn nên gây `IndexError`. Python không tự trả `None` hoặc mở rộng List khi đọc chỉ số không có.

Q16. What does `a[1] = 5` do? — `a[1] = 5` làm gì?

A. Changes the first item to `5` — Thay phần tử đầu thành `5`

B. Replaces the second element with `5` — Thay phần tử thứ hai bằng `5`

C. Inserts a new element at index 1 — Chèn phần tử mới tại chỉ số 1

D. Adds `5` to the end — Thêm `5` vào cuối

Đáp án: B

Giải thích: `a[1] = 5` thay phần tử hiện có tại chỉ số 1 thành `5`, miễn chỉ số đó tồn tại. Với `[8, 9]`, kết quả là `[8, 5]`; lệnh không chèn thêm phần tử hay sửa chỉ số 0.

Q17. Which operation sorts a list in place? — Thao tác nào sắp xếp ngay List gốc?

A. `a.sort()` — `a.sort()`

B. `sorted(a)` — `sorted(a)`

C. `a.reverse()` — `a.reverse()`

D. `reversed(a)` — `reversed(a)`

Đáp án: A

Giải thích: `a.sort()` sắp xếp ngay trên List gốc nên đọc `a` sau đó thấy thứ tự mới. `sorted(a)` trả List khác, còn `reverse()` chỉ đảo vị trí hiện có, không tính thứ tự sắp xếp.

Q18. Which expression sorts without mutating the original? — Biểu thức nào sắp xếp mà không sửa List gốc?

A. `sorted(a)` — `sorted(a)`

B. `a.reverse()` — `a.reverse()`

C. `a.extend(sorted(a))` — `a.extend(sorted(a))`

D. `a.sort()` — `a.sort()`

Đáp án: A

Giải thích: `sorted(a)` tạo List mới theo thứ tự sắp xếp và giữ nguyên `a`. Ngược lại `a.sort()` sửa trực tiếp `a` và trả `None`, nên không thể dùng thay nhau khi cần giữ dữ liệu gốc.

Q19. What does `reverse=True` mean in a numeric sort? — `reverse=True` có ý nghĩa gì khi sắp xếp số?

A. Reverse each number's digits — Đảo chữ số trong từng số

B. Ascending order — Thứ tự tăng dần

C. Random element order — Thứ tự phần tử ngẫu nhiên

D. Descending numeric order — Thứ tự số giảm dần

Đáp án: D

Giải thích: Trong phép sắp số mặc định tăng dần, `reverse=True` đảo thứ tự thành giảm dần. Với `[3, 1, 2]`, `sorted(a, reverse=True)` cho `[3, 2, 1]`, không đơn thuần đảo thứ tự vị trí gốc.

Q20. How do `append([3,4])` and `extend([3,4])` differ? — `append([3,4])` và `extend([3,4])` khác gì?

A. Both operations sort the existing list — Cả hai đều sắp xếp List

B. `append` adds one nested list; `extend` adds two elements — `append` thêm một List con; `extend` thêm hai phần tử

C. Both operations create the same flat list — Cả hai tạo cùng List phẳng

D. `append` adds both separately and `extend` nests them — `append` thêm riêng còn `extend` lồng List

Đáp án: B

Giải thích: `append([3, 4])` thêm nguyên List con thành một phần tử, còn `extend([3, 4])` thêm riêng hai số nguyên. Bắt đầu từ `[1, 2]`, hai kết quả lần lượt là `[1, 2, [3, 4]]` và `[1, 2, 3, 4]`.

8. Nối sang bài tiếp theo

`Vòng lặp → List → Vector/Array`.

PY-07 — Tuple, Set và các phép toán tập hợp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Tuple — bộ giá trị; **Set** — tập hợp.

`List → Tuple và Set → Tập hợp/Jaccard`

Học xong dùng để làm gì? Giữ cặp tọa độ cố định và lấy các nhãn không trùng nhau, so sánh phần tử chung/riêng của hai nhóm.

Kết nối kiến thức tiếp theo: PY-08 dùng Dictionary để liên kết mã hoặc tên với thông tin chi tiết.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Tuple	bộ giá trị	Dãy có thứ tự không sửa phần tử được
Set	tập hợp	Nhóm phần tử không trùng lặp
Union	phép hợp	Mọi phần tử có trong ít nhất một tập
Intersection	phép giao	Phần tử chung của hai tập

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`tuple(items)`	Lưu một bộ dữ liệu có thứ tự và cố định
`set(items)`	Khử giá trị trùng, không giữ thứ tự đáng tin
`a.union(b)`, `a.intersection(b)`	Lấy toàn bộ phần tử của hai tập (hợp), hoặc chỉ phần tử chung (giao); tương đương `a | b` và `a & b`
`a - b`, `a ^ b`	Tìm phần tử còn thiếu hoặc khác nhau

4. Khái niệm liên quan

Sau List, có hai nhu cầu khác nhau: **giữ một bộ dữ liệu có thứ tự nhưng không sửa trực tiếp** (Tuple), hoặc **chỉ lưu các giá trị không trùng và so sánh hai nhóm** (Set). Hai cấu trúc không thay thế lẫn nhau: tọa độ `(3, 4)` phù hợp với Tuple, còn danh sách kỹ năng không trùng phù hợp với Set. Từ Set, bài học mở rộng sang phép hợp và phép giao.

4.1. Tuple — Bộ giá trị cố định

Từ vựng trong mục: Tuple — bộ giá trị; **Coordinate** — tọa độ.

Tuple chứa các phần tử **có thứ tự**, thường dùng cho một cặp thông tin gắn với nhau như tọa độ `(x, y)`. Với điểm ở vị trí `x = 3`, `y = 4` mét, ta viết `point = (3, 4)`; `point[0]` là x và `point[1]` là y. Không thể gán `point[0] = 5` như List; nếu cần vị trí mới, tạo Tuple mới.

```
point = (3, 4)
print(point)
```

Kết quả khi chạy:

```
(3, 4)
```

Dấu ngoặc tròn biểu thị Tuple gồm hai tọa độ, khác dấu ngoặc vuông của List.

4.2. Set — Tập hợp không trùng

Từ vựng trong mục: Set — tập hợp; **Duplicate** — giá trị lặp.

Set giúp lưu **những giá trị khác nhau** mà không cần ghi cùng một giá trị nhiều lần. Một nhóm học ghi ba nhãn `Python`, `AI`, `Python`; khi chuyển sang Set, nhãn `Python` bị trùng chỉ còn một lần. Set không có chỉ số vị trí như `set[0]` và không đảm bảo thứ tự in. `set()` tạo tập rỗng, còn `{}` tạo Dictionary rỗng.

```
skills = set(['Python', 'AI', 'Python'])
print(len(skills))
```

Kết quả khi chạy:

```
2
```

Có hai kỹ năng khác nhau: Python và AI. Ta in **số lượng** để không phụ thuộc thứ tự hiển thị của Set.

4.3. Union — Gộp hai nhóm

Từ vựng trong mục: Union — phép hợp; **Set** — tập hợp.

Giả sử nhóm A học `{Python, AI}`, nhóm B học `{AI, SQL}`. Cửa hàng khóa học muốn biết **tất cả kỹ năng có trong ít nhất một nhóm**. Phép hợp `a | b` giữ Python, AI và SQL nhưng chỉ giữ AI một lần. Với Set, ký hiệu `|` là phép hợp tập hợp, không phải toán tử `or` nối điều kiện Boolean.

```
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
print(len(a | b))
```

Kết quả khi chạy:

```
3
```

Phép hợp gồm ba kỹ năng khác nhau; ký hiệu `|` gộp hai Set và tự loại phần trùng.

4.4. Intersection — Tìm phần chung

Từ vựng trong mục: Intersection — phép giao; **Set** — tập hợp.

Vẫn với hai nhóm A có `{Python, AI}` và B có `{AI, SQL}`, ta muốn tìm **kỹ năng cả hai nhóm đều học**. Phép giao `a & b` chỉ giữ AI, vì Python không có ở B và SQL không có ở A. Phép giao khác phép hợp: hợp trả ba kỹ năng, giao chỉ một kỹ năng.

```
a = {'Python', 'AI'}
b = {'AI', 'SQL'}
print(a & b)
```

Kết quả khi chạy:

```
{'AI'}
```

Chỉ có AI xuất hiện trong cả hai nhóm; tập chỉ có một phần tử nên thứ tự hiển thị không gây nhầm lẫn.

4.5. Difference — Phần tử còn thiếu trong Set

Từ vựng trong mục: Difference — hiệu tập hợp; **Set** — tập hợp.

Ngoài phép hợp và phép giao, ta còn cần biết **phần tử chỉ có ở nhóm A nhưng chưa có ở nhóm B**. Ví dụ khóa học yêu cầu `Python`, `SQL`, `AI`, còn học viên đã hoàn thành `Python`; hiệu `required - done` giữ `SQL` và `AI`. Dùng `sorted()` nếu muốn in các kỹ năng theo một thứ tự rõ ràng. Hiệu `a - b` khác **hiệu đối xứng** `a ^ b`, vốn lấy các phần tử chỉ thuộc một trong hai nhóm.

```
required = {'Python', 'SQL', 'AI'}
done = {'Python'}
print(sorted(required - done))
```

Kết quả khi chạy:

```
['AI', 'SQL']
```

Kết quả là một **List đã sắp xếp**, gồm hai kỹ năng cần học tiếp. Set gốc không bị thay đổi.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Khử email trùng

Từ vựng trong mục: Set — tập hợp; **Duplicate** — trùng lặp.

Tình huống thực tế: Ứng dụng gửi thông báo đang có bốn email `a@x.vn`, `b@x.vn`, `a@x.vn`, `c@x.vn`. Địa chỉ `a@x.vn` bị nhập hai lần.

Yêu cầu: Đếm có bao nhiêu địa chỉ email khác nhau.

Vì sao chọn cách giải này? `set()` bỏ bản ghi giống hệt nhau, `len()` đếm số email còn lại. Cách này chưa xử lý khác biệt chữ hoa/chữ thường.

Quy trình giải:

1. Lưu bốn email vào list.
2. Chuyển list thành set để bỏ email bị lặp.
3. Dùng `len()` đếm số email khác nhau.

```
emails=["a@x.vn","b@x.vn","a@x.vn","c@x.vn"]
print(len(set(emails)))
```

Kết quả khi chạy:

```
3
```

Kết quả: 3 — có ba địa chỉ email khác nhau.

Tình huống 2 — Lưu tọa độ

Từ vựng trong mục: Tuple — bộ giá trị; **Coordinate** — tọa độ.

Tình huống thực tế: Một phần mềm bản đồ lưu điểm có tọa độ `(3, 4)` mét, tính từ gốc `(0, 0)`. Hai giá trị lần lượt là vị trí x và y.

Yêu cầu: Lấy x, y từ tuple và tính bình phương khoảng cách đến gốc tọa độ.

Vì sao chọn cách giải này? Tuple lưu gọn cặp tọa độ có thứ tự. Công thức `x*x + y*y` tính bình phương khoảng cách, không phải khoảng cách thực.

Quy trình giải:

1. Lưu tọa độ vào tuple `point`.
2. Tách tuple thành `x` và `y`.
3. Tính `x*x + y*y` rồi in kết quả.

```
point=(3,4)
x,y=point
print(x*x+y*y)
```

Kết quả khi chạy:

```
25
```

Kết quả: 25 m^2 — bình phương khoảng cách; khoảng cách thật bằng 5 mét.

Tình huống 3 — Tìm kỹ năng chung

Từ vựng trong mục: Intersection — phép giao; **Set** — tập hợp.

Tình huống thực tế: Hai nhóm học viên có kỹ năng lần lượt là `{'Python', 'AI', 'SQL'}` và `{'Python', 'Math', 'AI'}`.

Yêu cầu: Tìm những kỹ năng có ở cả hai nhóm.

Vì sao chọn cách giải này? Phép giao `&` chỉ giữ các phần tử chung giữa hai set.

Quy trình giải:

1. Lưu kỹ năng của mỗi nhóm thành một set.
2. Dùng `a & b` để tìm phần tử chung.
3. In tập kỹ năng tìm được.

```
a={"Python","AI","SQL"}
b={"Python","Math","AI"}
print(a&b)
```

Kết quả khi chạy:

```
{'Python', 'AI'}
```

Thứ tự các phần tử trong Set khi in có thể khác nhau.

Kết quả: Tập chứa `Python` và `AI`; thứ tự hiển thị của set có thể thay đổi.

Tình huống 4 — Gộp nhóm chủ đề

Từ vựng trong mục: Union — phép hợp; **Set** — tập hợp.

Tình huống thực tế: Hai nhóm tài liệu được gắn nhãn `{'Python', 'Math'}` và `{'Math', 'AI'}`. Nhân viên cần danh sách chủ đề của cả hai nhóm mà không lặp lại `Math`.

Yêu cầu: Gộp các nhãn thành một set chỉ chứa các chủ đề khác nhau.

Vì sao chọn cách giải này? Phép hợp `|` lấy tất cả phần tử thuộc một trong hai set và bỏ phần trùng.

Quy trình giải:

1. Lưu nhãn của từng nhóm thành set.
2. Dùng `a | b` để gộp.
3. In các nhãn sau khi gộp.

```
a={"Python","Math"}
b={"Math","AI"}
print(a|b)
```

Kết quả khi chạy:

```
{'Python', 'Math', 'AI'}
```

Thứ tự các phần tử trong Set khi in có thể khác nhau.

Kết quả: Tập chứa `Python`, `Math`, `AI`; thứ tự in không cố định.

Tình huống 5 — Liệt kê bài còn thiếu

Từ vựng trong mục: Difference — hiệu tập hợp; **Set** — tập hợp.

Tình huống thực tế: Khóa học yêu cầu ba kỹ năng `Python`, `SQL`, `AI`. Một học viên đã học xong `Python`.

Yêu cầu: Liệt kê các kỹ năng học viên còn phải học.

Vì sao chọn cách giải này? Hiệu tập hợp `required - done` giữ những kỹ năng bắt buộc chưa có trong tập đã hoàn thành.

Quy trình giải:

1. Lưu tập kỹ năng bắt buộc.
2. Lưu tập kỹ năng đã học.
3. Lấy hiệu hai tập và in kết quả.

```
required={"Python","SQL","AI"}
done={"Python"}
print(required-done)
```

Kết quả khi chạy:

```
{'SQL', 'AI'}
```

Thứ tự các phần tử trong Set khi in có thể khác nhau.

Kết quả: Tập chứa `SQL` và `AI`; thứ tự in không cố định.

6. Nhầm lẫn dễ gặp

- Tuple không cho gán lại từng phần tử như `point[0] = 5`; nếu cần sửa tọa độ, tạo tuple mới.
- Set không có thứ tự cố định, không dùng `my_set[0]` và không dựa vào thứ tự khi in.
- Set chỉ giữ phần tử có thể băm được (hashable); không thể đặt list trực tiếp làm phần tử của set.
- Phép hợp `|` lấy tất cả phần tử; phép giao `&` chỉ lấy phần tử chung; phép trừ `-` tìm phần tử còn thiếu.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a tuple? — Tuple là gì?

A. An ordered mutable sequence — Dãy có thứ tự và có thể sửa

B. An unordered collection of unique items — Tập hợp không thứ tự của phần tử duy nhất

C. A key-to-value mapping — Ánh xạ khóa–giá trị

D. An ordered immutable collection — Tập hợp có thứ tự và bất biến

Đáp án: D

Giải thích: Tuple là dãy có thứ tự và bất biến: `(2, 3)` giữ tọa độ đầu trước tọa độ sau, nhưng không cho phép gán lại từng phần tử. List cũng có thứ tự nhưng có thể sửa các phần tử.

Q2. What is a set? — Set là gì?

A. An ordered collection of key-value entries — Tập hợp có thứ tự các cặp khóa–giá trị

B. A collection that always keeps duplicates — Tập hợp luôn giữ giá trị trùng

C. A collection of unique hashable members — Tập hợp gồm phần tử có thể băm và không trùng

D. A sequence accessed by positional indexes — Dãy truy cập bằng chỉ số vị trí

Đáp án: C

Giải thích: Set chứa các phần tử có thể băm và không trùng nhau, đồng thời không hỗ trợ truy cập theo chỉ số vị trí. Tạo `set([1, 1, 2])` chỉ còn hai phần tử `1` và `2`, khác List vốn giữ phần tử lặp.

Q3. What does set union collect? — Phép hợp Set lấy gì?

A. Only members of the first set not in the second — Chỉ phần tử tập đầu mà không thuộc tập sau

B. Distinct members appearing in either set — Phần tử khác nhau thuộc ít nhất một tập

C. Only members appearing in exactly one set — Chỉ phần tử thuộc đúng một tập

D. Only members common to both sets — Chỉ phần tử chung của hai tập

Đáp án: B

Giải thích: Phép hợp lấy mỗi phần tử khác nhau xuất hiện trong ít nhất một tập. Với `{1, 2}` và `{2, 3}`, hợp chứa `{1, 2, 3}`; phép giao chỉ giữ phần tử chung `2`.

Q4. What does set intersection collect? — Phép giao Set lấy gì?

A. Only members of the first set — Chỉ phần tử tập đầu

B. Members occurring in exactly one set — Phần tử xuất hiện đúng một tập

C. Only members shared by both sets — Chỉ phần tử chung của cả hai tập

D. All distinct members of either set — Mọi phần tử khác nhau thuộc ít nhất một tập

Đáp án: C

Giải thích: Phép giao chỉ giữ phần tử có mặt ở cả hai tập. Với `{1, 2}` và `{2, 3}`, giao là `{2}`; tập này nhỏ hơn phép hợp `{1, 2, 3}`.

Q5. Which expression constructs a two-element tuple? — Biểu thức nào tạo Tuple có hai phần tử?

A. `(2, 3)` — `(2, 3)`

B. `[2, 3]` — `[2, 3]`

C. `{2, 3}` — `{2, 3}`

D. `{2: 3}` — `{2: 3}`

Đáp án: A

Giải thích: `(2, 3)` là Tuple gồm hai phần tử có thứ tự với các giá trị phân cách bằng dấu phẩy. Ngược lại, `{2, 3}` là Set và `[2, 3]` là List có thể sửa.

Q6. Which expression constructs a nonempty set? — Biểu thức nào tạo Set không rỗng?

A. `[2, 3]` — `[2, 3]`

B. `{2: 3}` — `{2: 3}`

C. `(2, 3)` — `(2, 3)`

D. `{2, 3}` — `{2, 3}`

Đáp án: D

Giải thích: `{2, 3}` tạo Set không rỗng gồm hai số nguyên khác nhau. Dấu ngoặc tròn tạo Tuple và ngoặc vuông tạo List, còn ngoặc nhọn rỗng `{}` tạo Dictionary.

Q7. Which expression constructs an empty set? — Biểu thức nào tạo Set rỗng?

A. `[]` — `[]`

B. `set()` — `set()`

C. `()` — `()`

D. `{}` — `{}`

Đáp án: B

Giải thích: `set()` tạo Set rỗng. Ký hiệu gần giống `{}` lại là Dictionary rỗng, vì vậy không thể dùng cặp ngoặc nhọn trống để tạo Set không có phần tử.

Q8. Which expression constructs an empty dictionary? — Biểu thức nào tạo Dictionary rỗng?

A. `set()` — `set()`

B. `[]` — `[]`

C. `()` — `()`

D. `{}` — `{}`

Đáp án: D

Giải thích: `{}` biểu diễn Dictionary rỗng, tức ánh xạ khóa–giá trị chưa có mục nào. Muốn tạo Set rỗng phải dùng `set()`; List và Tuple rỗng lần lượt là `[]` và `()`.

Q9. What does `a | b` do for sets? — `a | b` làm gì với hai Set?

A. Set difference from `a` to `b` — Hiệu Set từ `a` trừ `b`

B. Symmetric difference — Hiệu đối xứng

C. Set union — Phép hợp Set

D. Set intersection — Phép giao Set

Đáp án: C

Giải thích: Với Set, `a | b` thực hiện phép hợp, giữ phần tử khác nhau thuộc `a`, `b` hoặc cả hai. Nếu `a={1,2}` và `b={2,3}`, kết quả chứa 1, 2, 3, khác giao vốn chỉ giữ 2.

Q10. What does `a & b` do for sets? — `a & b` làm gì với hai Set?

A. Set intersection — Phép giao Set

B. Set difference — Phép hiệu Set

C. Symmetric difference — Hiệu đối xứng

D. Set union — Phép hợp Set

Đáp án: A

Giải thích: `a & b` là phép giao Set, chỉ giữ giá trị xuất hiện ở cả hai tập. Với `{1,2}` và `{2,3}`, kết quả là `{2}`; nó không lấy mọi phần tử khác nhau như phép hợp.

Q11. What does `a - b` do for sets? — `a - b` làm gì với hai Set?

A. Members in `a` but not in `b` — Phần tử thuộc `a` nhưng không thuộc `b`

B. Members in exactly one input — Phần tử thuộc đúng một tập

C. Set union — Phép hợp Set

D. Set intersection — Phép giao Set

Đáp án: A

Giải thích: `a - b` chỉ giữ phần tử thuộc `a` mà không thuộc `b`, nên thứ tự hai vế quan trọng: `{1,2} - {2,3}` cho `{1}`. Đảo hai vế lại được `{3}`, khác hiệu đối xứng giữ cả 1 và 3.

Q12. What does `a ^ b` do for sets? — `a ^ b` làm gì với hai Set?

A. Set union — Phép hợp Set

B. Set intersection — Phép giao Set

C. Members of `a` only — Chỉ phần tử của `a`

D. Members in exactly one of the sets — Phần tử thuộc đúng một trong hai tập

Đáp án: D

Giải thích: `a ^ b` tính hiệu đối xứng: lấy phần tử có mặt đúng ở một trong hai tập. Với `{1,2}` và `{2,3}`, kết quả chứa 1 và 3 nhưng loại phần tử chung 2.

Q13. What is the length of `set([1,1,2])`? — `set([1,1,2])` có bao nhiêu phần tử?

A. 2 — 2

B. 3 — 3

C. 1 — 1

D. `ValueError` — `ValueError`

Đáp án: A

Giải thích: Tạo Set sẽ loại giá trị trùng, vì vậy `set([1,1,2])` chỉ còn phần tử 1 và 2. Độ dài do đó bằng số nguyên `2`, không phải ba vị trí của List gốc.

Q14. What happens when assigning `t[0] = 9` on a tuple? — Gán `t[0] = 9` trên Tuple sẽ thế nào?

A. A `TypeError` is raised — Phát sinh `TypeError`

B. A new tuple is created automatically — Tự động tạo Tuple mới

C. An `IndexError` is always raised — Luôn phát sinh `IndexError`

D. The tuple's first item changes to 9 — Phần tử đầu Tuple đổi thành 9

Đáp án: A

Giải thích: Tuple không cho phép gán lại từng phần tử nên `t[0] = 9` trên Tuple gây `TypeError`. Khác với List, việc gán vào một chỉ số hợp lệ được cho phép.

Q15. How do you test membership in a set? — Làm sao kiểm tra một giá trị thuộc Set?

A. `s[x]` — `s[x]`

B. `x in s` — `x in s`

C. `x == s` — `x == s`

D. `s.index(x)` — `s.index(x)`

Đáp án: B

Giải thích: Dùng `x in s` để kiểm tra phần tử, kết quả là Boolean. Với `s={1,3}`, `3 in s` là `True` và `2 in s` là `False`; Set không có phương thức `.index()` theo vị trí.

Q16. Is order of printed set elements guaranteed? — Thứ tự phần tử in từ Set có được bảo đảm không?

A. Always sorted in ascending order — Luôn theo thứ tự tăng dần

B. Always insertion order — Luôn theo thứ tự thêm vào

C. Always a new random permutation — Luôn tạo hoán vị ngẫu nhiên mới

D. No fixed display order is guaranteed — Không bảo đảm thứ tự hiển thị cố định

Đáp án: D

Giải thích: Set không bảo đảm thứ tự khi hiển thị hay duyệt các phần tử. Dù một lần chạy in `{1, 2, 3}`, không nên phụ thuộc thứ tự đó ở các lần sau; sắp xếp Set là thao tác riêng.

Q17. Which built-in immutable ordered sequence best represents a fixed coordinate pair `(x, y)`? — Kiểu dãy có thứ tự và bất biến nào phù hợp nhất biểu diễn cặp tọa độ cố định `(x, y)`?

A. Dictionary — Dictionary

B. Tuple — Tuple

C. List — List

D. Set — Set

Đáp án: B

Giải thích: Một cặp có thứ tự và bất biến như `(x, y)` phù hợp tọa độ cố định vì vị trí thứ nhất và thứ hai có ý nghĩa riêng. Set không truy cập theo vị trí, còn List cho phép gán lại phần tử.

Q18. Which built-in collection directly keeps only distinct tags? — Kiểu tập hợp nào trực tiếp chỉ giữ những nhãn khác nhau?

A. String, which holds one sequence of characters — String lưu dãy ký tự

B. Tuple, which retains duplicates — Tuple giữ nguyên các phần tử trùng

C. Set, which retains distinct tags — Set chỉ giữ nhãn khác nhau

D. List, which retains duplicates — List giữ nguyên các phần tử trùng

Đáp án: C

Giải thích: Set trực tiếp loại nhãn trùng vì mỗi giá trị chỉ xuất hiện một lần. Với nhãn `['red', 'red', 'blue']`, tạo Set giữ hai nhãn khác nhau `red` và `blue`, nhưng không bảo đảm thứ tự.

Q19. Which method checks whether sets share no elements? — Phương thức nào kiểm tra hai Set không có phần tử chung?

A. `union()` — `union()`

B. `isdisjoint()` — `isdisjoint()`

C. `intersection()` — `intersection()`

D. `issubset()` — `issubset()`

Đáp án: B

Giải thích: `a.isdisjoint(b)` trả `True` khi và chỉ khi hai Set không có phần tử chung. Với `{1,2}` và `{3}`, kết quả là `True`, còn `{1,2}` và `{2,4}` cho `False`.

Q20. Which function creates a sorted list from a set of comparable numbers? — Hàm nào tạo List đã sắp từ một Set chứa các số có thể so sánh?

A. `list(s)` without ordering — `list(s)` không sắp xếp

B. `reversed(s)` — `reversed(s)`

C. `sorted(s)` — `sorted(s)`

D. `s.sort()` — `s.sort()`

Đáp án: C

Giải thích: `sorted(s)` trả về List mới đã sắp xếp, ví dụ `sorted({3,1,2})` cho `[1,2,3]`. Nó không tạo thứ tự cố định cho Set gốc, vốn vẫn là tập không có thứ tự vị trí.

8. Nối sang bài tiếp theo

`List → Tuple và Set → Tập hợp/Jaccard`.

PY-08 — Dictionary: khóa, giá trị và thêm sửa xóa

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Dictionary — từ điển; **Key** — khóa.

`List/Set → Dictionary → JSON/CSV → DataFrame`

Học xong dùng để làm gì? Tra cứu hồ sơ học viên theo mã và biểu diễn bản ghi có nhiều trường bằng khóa–giá trị.

Kết nối kiến thức tiếp theo: PY-09 viết hàm tái sử dụng các bước xử lý những bản ghi đó.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích dễ nhớ
Dictionary	từ điển	Ánh xạ khóa sang giá trị
Key	khóa	Tên hoặc mã để truy tìm giá trị
Value	giá trị	Thông tin được lưu theo một khóa
Nested dict	từ điển lồng	Một từ điển có từ điển con

3. Hàm và cú pháp Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cú pháp	Mục đích và lựa chọn liên quan
`obj[key], obj.get(key)`	Tra cứu khóa hoặc nhận giá trị mặc định
`obj.items(), obj.values()`	Duyệt cặp khóa/giá trị hay riêng giá trị
`obj[key] = value`	Thêm hoặc cập nhật dữ liệu
`del, pop(key)`	Xóa thông tin bằng khóa

4. Khái niệm liên quan

List lấy dữ liệu theo **vị trí**, còn **Dictionary** lấy theo **khóa có ý nghĩa** như mã đơn hàng hoặc tên trường. Mỗi khóa gắn với một giá trị để tra cứu, thêm, sửa hoặc xóa thông tin. Bài dùng hồ sơ đơn hàng làm ví dụ xuyên suốt, từ một mã đơn đơn giản tới hồ sơ có nhiều thuộc tính.

4.1. Dictionary — Dữ liệu dạng khóa và giá trị

Từ vựng trong mục: Dictionary — từ điển; **Key–value pair** — cặp khóa–giá trị.

Một cửa hàng lưu đơn hàng mã `O1` có trạng thái `paid` (đã thanh toán). Trong Dictionary `order = {'id': 'O1', 'status': 'paid'}`, **`id` và `status` là hai khóa**; `O1` và `paid` là các giá trị tương ứng. Dấu `{}` bao các cặp `khóa: giá trị`. Dùng Dictionary khi cần đọc thông tin theo tên trường thay vì phải nhớ chỉ số trong List.

```
order = {'id': 'O1', 'status': 'paid'}
print(order)
```

Kết quả khi chạy:

```
{'id': 'O1', 'status': 'paid'}
```

Một đơn hàng có hai trường. Mỗi cặp khóa–giá trị mô tả một thông tin riêng.

4.2. Key — Khóa để tra cứu

Từ vựng trong mục: Key — khóa; **Lookup** — tra cứu.

Khóa là tên hoặc mã duy nhất **trong cùng một Dictionary**, được dùng để lấy giá trị đúng trường. Với hồ sơ `order`, biểu thức `order['id']` lấy mã O1; còn `order['status']` lấy trạng thái. Nếu dùng dấu `[]` với khóa không tồn tại, Python báo `KeyError`; có thể dùng `get()` khi cần giá trị mặc định.

```
order = {'id': 'O1', 'status': 'paid'}
print(order['id'])
```

Kết quả khi chạy:

```
O1
```

Python tra theo khóa `id`; không cần duyệt qua các trường bằng vòng lặp.

4.3. Value và thao tác thêm, sửa, xóa

Từ vựng trong mục: Value — giá trị; **Update** — cập nhật.

Giá trị là dữ liệu gắn với từng khóa; có thể là chuỗi, số, List hoặc Dictionary khác. Ví dụ đơn `O1` đang có trạng thái `pending` (đang chờ). Lệnh `order['status'] = 'paid'` **sửa** giá trị của khóa đã có. Lệnh `order['amount'] = 100` **thêm** trường số tiền mới; `order.pop('amount')` **xóa** trường ấy. Trong bài này, ta in Dictionary trước và sau khi xóa để nhìn thấy thay đổi; nếu xóa khóa không tồn tại mà không truyền giá trị mặc định cho `pop`, Python báo `KeyError`.

```
order = {'id': 'O1', 'status': 'pending'}
order['status'] = 'paid'
order['amount'] = 100
print(order)
order.pop('amount')
print(order)
```

Kết quả khi chạy:

```
{'id': 'O1', 'status': 'paid', 'amount': 100}
{'id': 'O1', 'status': 'paid'}
```

Dòng đầu có thêm trường `amount` giá trị 100; dòng thứ hai không còn trường này sau `pop()`. Mã đơn O1 và trạng thái `paid` vẫn giữ nguyên.

4.4. Nested dictionary — Hồ sơ có thông tin con

Từ vựng trong mục: Nested dictionary — từ điển lồng; **Field** — trường dữ liệu.

Nếu quản lý nhiều đơn hàng, ta có thể dùng **Dictionary ngoài** với khóa là mã đơn (`O1`, `O2`), và **Dictionary trong** lưu thông tin riêng như `status`, `amount`. Khi cần lấy số tiền đơn `O2`, tra trước `orders['O2']` để lấy hồ sơ con, sau đó tra `['amount']`. Cách này giữ các thuộc tính của từng đơn bên nhau.

```
orders = {
    'O1': {'status': 'paid', 'amount': 100},
    'O2': {'status': 'pending', 'amount': 200},
}
print(orders['O2']['amount'])
```

Kết quả khi chạy:

```
200
```

Kết quả `200` là số tiền của đơn O2 (đơn vị tiền theo quy định của cửa hàng), không phải mã đơn hay chỉ số dòng.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tra cứu đơn theo mã

Từ vựng trong mục: Dictionary — từ điển; **Key** — khóa.

Tình huống thực tế: Cửa hàng đang lưu trạng thái hai đơn hàng: `O1` đã thanh toán (`paid`) và `O2` đang chờ (`pending`). Khách nhập mã `O3`, chưa có trong hệ thống.

Yêu cầu: Tra cứu mã `O3` và in thông báo nếu không tìm thấy.

Vì sao chọn cách giải này? `get(key, default)` tìm theo mã đơn và trả về `Not found` khi khóa chưa tồn tại, thay vì phát sinh `KeyError`.

Quy trình giải:

1. Lưu trạng thái đơn theo từng mã.
2. Tra `O3` bằng `get()` và truyền vào câu báo `Not found`.
3. In kết quả.

```
orders={"O1":"paid","O2":"pending"}
print(orders.get("O3","Not found"))
```

Kết quả khi chạy:

```
Not found
```

Kết quả: `Not found` — không có mã đơn `03` trong dữ liệu.

Tình huống 2 — Cập nhật tồn kho

Từ vựng trong mục: Key — khóa; **Value** — giá trị.

Tình huống thực tế: Kho có sản phẩm mã `SKU01` với 5 món. Sau khi bán 2 món, nhân viên cần cập nhật số lượng còn lại.

Yêu cầu: Trừ 2 khỏi tồn kho của `SKU01` rồi in Dictionary.

Vì sao chọn cách giải này? Mã SKU là khóa của sản phẩm; `stock['SKU01'] -= 2` thay đổi số lượng ngay tại khóa tương ứng.

Quy trình giải:

1. Lưu số lượng ban đầu theo mã SKU.
2. Lấy số lượng ở khóa `SKU01` và trừ 2.
3. In tồn kho đã cập nhật.

```
stock={"SKU01":5}
stock["SKU01"]-=2
print(stock)
```

Kết quả khi chạy:

```
{'SKU01': 3}
```

Kết quả: `{'SKU01': 3}` — sản phẩm còn 3 món.

Tình huống 3 — Đếm nhãn

Từ vựng trong mục: Dictionary — từ điển; **Count** — số lần.

Tình huống thực tế: Một hệ thống phân loại ghi ba nhãn theo thứ tự `AI`, `Math`, `AI`. Người quản lý muốn biết mỗi nhãn xuất hiện bao nhiêu lần.

Yêu cầu: Tạo Dictionary đếm số lượt của từng nhãn.

Vì sao chọn cách giải này? `get(x, 0)` lấy số đếm cũ hoặc 0 nếu nhãn mới xuất hiện, sau đó cộng thêm 1.

Quy trình giải:

1. Tạo list nhãn và Dictionary `count` rỗng.
2. Duyệt từng nhãn bằng `for`.
3. Tăng bộ đếm của nhãn và in kết quả.

```
labels=["AI","Math","AI"]
count={}
for x in labels:
```

```
    count[x]=count.get(x,0)+1
print(count)
```

Kết quả khi chạy:

```
{'AI': 2, 'Math': 1}
```

Kết quả: `{'AI': 2, 'Math': 1}` — AI xuất hiện hai lần, Math một lần.

Tình huống 4 — Hồ sơ lồng nhau

Từ vựng trong mục: Nested dict — từ điển lồng; **Key** — khóa.

Tình huống thực tế: Trung tâm có hồ sơ `SV1` tên An, điểm 8 và `SV2` tên Lan, điểm 9. Hồ sơ lưu theo mã sinh viên, bên trong chứa tên và điểm.

Yêu cầu: Lấy điểm của `SV2` và in kết quả.

Vì sao chọn cách giải này? Dictionary lồng nhau giúp tìm sinh viên bằng mã rồi lấy đúng trường `score` của người đó.

Quy trình giải:

1. Lưu hai hồ sơ trong `students`.
2. Chọn sinh viên bằng khóa `SV2`.
3. Lấy khóa `score` trong hồ sơ con và in điểm.

```
students={"SV1":{"name":"An","score":8},"SV2":{"name":"Lan","score":9}}
print(students["SV2"]["score"])
```

Kết quả khi chạy:

```
9
```

Kết quả: `9` — điểm của học viên Lan (`SV2`).

Tình huống 5 — Xóa SKU không có

Từ vựng trong mục: Dictionary — từ điển; **Pop** — lấy và xóa.

Tình huống thực tế: Kho đang có sản phẩm `P01` với 10 món. Nhân viên muốn xóa mã `P02`, nhưng mã này không có trong Dictionary.

Yêu cầu: Thử xóa `P02` và in kết quả thao tác, không làm chương trình lỗi.

Vì sao chọn cách giải này? `pop('P02', None)` trả về `None` nếu không có mã, thay vì gây `KeyError`. Không có phần tử nào bị xóa.

Quy trình giải:

1. Lưu mã hàng đang có.
2. Gọi `pop()` với mã `P02` và giá trị mặc định `None`.
3. In biến `removed`.

```
items={"P01":10}
removed=items.pop("P02",None)
print(removed)
```

Kết quả khi chạy:

```
None
```

Kết quả: `None` — không có mã `P02` để xóa; `P01` vẫn còn trong Dictionary.

6. Nhầm lẫn dễ gặp

- `obj[key]` gây `KeyError` nếu khóa chưa tồn tại; `obj.get(key)` trả về `None` hoặc giá trị mặc định đã truyền vào.
- Gán `obj[key] = value` sẽ thêm khóa mới hoặc ghi đè giá trị đang có.
- `pop(key)` lấy và xóa giá trị; nếu khóa không tồn tại, nên truyền giá trị mặc định khi phù hợp.
- Khóa Dictionary phải là kiểu có thể băm (hashable), như chuỗi hoặc số; không dùng List làm khóa.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a dictionary? — Dictionary là gì?

A. An unordered collection of unique values only — Tập không thứ tự chỉ gồm giá trị duy nhất

B. An immutable pair of numbers — Cặp số bất biến

C. A mapping from hashable keys to values — Ánh xạ từ khóa có thể băm sang giá trị

D. A sequence with numeric positions only — Dãy chỉ truy cập bằng vị trí số

Đáp án: C

Giải thích: Dictionary ánh xạ các khóa khác nhau, có thể băm, tới giá trị tương ứng, ví dụ `{'name':'An', 'score':9}`. Bạn truy cập giá trị qua khóa thay vì chỉ số vị trí như List.

Q2. What does a dictionary key do? — Khóa Dictionary có vai trò gì?

A. Specifies the dictionary's length — Xác định độ dài Dictionary

B. Stores all keys as values — Lưu mọi khóa thành giá trị

C. Sorts the stored values — Sắp xếp các giá trị đã lưu

D. Identifies an entry for lookup — Xác định mục để tra cứu

Đáp án: D

Giải thích: Khóa xác định một mục để tra cứu, cập nhật hoặc xóa giá trị. Trong `{'age': 20}`, khóa `'age'` dẫn đến giá trị `20`; khóa không đồng nghĩa với giá trị đi kèm.

Q3. What is a dictionary value? — Giá trị Dictionary là gì?

A. The key's index position — Vị trí chỉ số của khóa

B. Data associated with a key — Dữ liệu gắn với khóa

C. Always a second dictionary — Luôn là Dictionary thứ hai

D. A name used only in a `for` loop — Tên chỉ dùng trong vòng `for`

Đáp án: B

Giải thích: Giá trị Dictionary là dữ liệu lưu dưới một khóa, chẳng hạn số `20` dưới khóa `'age'`. Giá trị có thể thuộc nhiều kiểu kể cả List hoặc Dictionary, trong khi khóa phải là đối tượng có thể băm.

Q4. What is a nested dictionary? — Dictionary lồng nhau là gì?

A. A dictionary stored inside another dictionary — Dictionary được lưu trong Dictionary khác

B. A list nested inside a set — List lồng trong Set

C. A set of duplicate keys — Set gồm khóa trùng

D. A dictionary with only numeric keys — Dictionary chỉ có khóa số

Đáp án: A

Giải thích: Dictionary lồng nhau là Dictionary được lưu làm giá trị trong Dictionary khác. Ví dụ `{'A': {'score': 9}}` gom trường `score` dưới khóa `'A'`, và hai lần truy cập ngoặc vuông sẽ lấy được con số.

Q5. What does `d['x']` do when x exists? — `d['x']` làm gì khi khóa x tồn tại?

A. Always returns `None` — Luôn trả `None`

B. Retrieves the value stored under `'x'` — Lấy giá trị lưu dưới khóa `'x'`

C. Returns `'x'` as a string — Trả chuỗi `'x'`

D. Deletes the entry for `'x'` — Xóa mục có khóa `'x'`

Đáp án: B

Giải thích: Khi khóa `'x'` tồn tại, `d['x']` lấy giá trị gắn với khóa đó. Khác `d.get('x')`, truy cập ngoặc vuông sẽ gây `KeyError` nếu khóa không tồn tại.

Q6. What does `d.get('x')` return if x is absent? — `d.get('x')` trả về gì nếu thiếu khóa x?

A. `KeyError` — `KeyError`

B. `None` by default — Mặc định là `None`

C. The empty dictionary `{}` — Dictionary rỗng `{}`

D. The string `'x'` — Chuỗi `'x'`

Đáp án: B

Giải thích: Khi không cung cấp mặc định, `d.get('x')` trả `None` nếu thiếu khóa thay vì gây `KeyError`. Cách này giúp xử lý dữ liệu thiếu, nhưng bản thân `None` cũng có thể là giá trị được lưu thật.

Q7. What does `d.get('x', 0)` return if x is absent? — `d.get('x', 0)` trả về gì nếu thiếu x?

A. The integer `0` — Số nguyên `0`

B. A newly inserted entry — Một mục mới được thêm

C. The string `'0'` — Chuỗi `'0'`

D. `None` — `None`

Đáp án: A

Giải thích: Đối số thứ hai của `get` là giá trị dự phòng khi không tìm thấy khóa, nên `d.get('x', 0)` trả số nguyên `0` nếu thiếu `'x'`. Lệnh không tạo khóa đó và không ghi đè giá trị đang có.

Q8. What does `d['missing']` raise? — `d['missing']` phát sinh gì khi thiếu khóa?

A. `0` — `0`

B. An empty list — List rỗng

C. `KeyError` — `KeyError`

D. `None` — `None`

Đáp án: C

Giải thích: Truy cập trực tiếp `d['missing']` gây `KeyError` khi không có khóa. Với dữ liệu tùy chọn, `get` có thể trả giá trị dự phòng; sự khác biệt này quyết định có xem việc thiếu khóa là lỗi hay không.

Q9. What does `d.items()` provide? — `d.items()` cung cấp gì?

A. A sorted list of keys — List khóa đã sắp

B. A view of key-value pairs — View của các cặp khóa–giá trị

C. Only a view of values — Chỉ view các giá trị

D. Only a view of keys — Chỉ view các khóa

Đáp án: B

Giải thích: `d.items()` trả một view động của các cặp khóa–giá trị, thường duyệt bằng `for key, value in d.items()`. Nó khác `keys()` chỉ đưa ra khóa và `values()` chỉ đưa ra giá trị.

Q10. What does `d.keys()` provide? — `d.keys()` cung cấp gì?

A. A view of values — View các giá trị

B. A list of keys sorted automatically — List khóa tự sắp

C. A view of pairs — View các cặp khóa–giá trị

D. A view of dictionary keys — View của các khóa Dictionary

Đáp án: D

Giải thích: `d.keys()` cung cấp view động của các khóa Dictionary. Với `{'a': 1}`, khi duyệt view này nhận `'a'`, không phải giá trị `1` hay cả cặp khóa–giá trị.

Q11. What does `d.values()` provide? — `d.values()` cung cấp gì?

A. A view of dictionary values — View các giá trị Dictionary

B. A dictionary copy — Bản sao Dictionary

C. A view of keys — View các khóa

D. A view of key-value pairs — View các cặp khóa–giá trị

Đáp án: A

Giải thích: `d.values()` trả view động của các giá trị đang lưu. Với `{'a': 1, 'b': 2}`, khi duyệt thu được 1 và 2; muốn kèm khóa tương ứng thì dùng `items()`.

Q12. What does `d.pop('x', None)` do if x is present? — `d.pop('x', None)` làm gì nếu x tồn tại?

A. Returns the value without deletion — Trả giá trị nhưng không xóa

B. Removes the key but always returns `None` — Xóa khóa nhưng luôn trả `None`

C. Deletes every dictionary entry — Xóa mọi mục Dictionary

D. Removes `'x'` and returns its former value — Xóa `'x'` và trả giá trị cũ

Đáp án: D

Giải thích: Khi `'x'` tồn tại, `d.pop('x', None)` xóa mục tương ứng và trả về giá trị cũ. Đối số `None` chỉ là giá trị dự phòng nếu không có khóa; nó không ép kết quả luôn thành `None`.

Q13. What does `d['x'] = 5` do if x already exists? — `d['x'] = 5` làm gì khi x đã tồn tại?

A. Replaces the value associated with `'x'` — Thay giá trị gắn với `'x'`

B. Adds a second duplicate `'x'` key — Thêm khóa `'x'` trùng thứ hai

C. Deletes the key `'x'` — Xóa khóa `'x'`

D. Tests whether `'x'` exists — Kiểm tra `'x'` tồn tại không

Đáp án: A

Giải thích: Gán `d['x'] = 5` khi khóa `'x'` đã tồn tại sẽ thay giá trị của mục đó thành `5`. Lệnh không tạo thêm một khóa `'x'` thứ hai vì mỗi khóa Dictionary là duy nhất.

Q14. What does `d['new'] = 5` do if new is absent? — `d['new'] = 5` làm gì khi khóa new chưa có?

A. Deletes an absent key — Xóa một khóa không tồn tại

B. Returns the total dictionary length — Trả tổng độ dài Dictionary

C. Replaces an unrelated existing key — Thay một khóa cũ không liên quan

D. Inserts a new key-value entry — Thêm một mục khóa–giá trị mới

Đáp án: D

Giải thích: Gán `d['new'] = 5` khi chưa có khóa sẽ thêm cặp `'new'` ánh xạ tới 5. Cùng cú pháp gán này lại cập nhật giá trị nếu khóa đã tồn tại.

Q15. What does `del d['x']` do for an existing key? — `del d['x']` làm gì với khóa đang tồn tại?

A. Deletes that key and its value — Xóa khóa đó cùng giá trị

B. Deletes the entire dictionary — Xóa toàn bộ Dictionary

C. Leaves the entry intact and returns its value — Giữ nguyên mục và trả giá trị

D. Replaces its value with `None` — Thay giá trị bằng `None`

Đáp án: A

Giải thích: `del d['x']` xóa khóa có sẵn cùng giá trị tương ứng khỏi Dictionary. Khác `pop`, câu lệnh `del` không trả về giá trị đã xóa.

Q16. What does `'x' in d` test? — `'x' in d` kiểm tra gì?

A. Whether `'x'` is a numeric index — `'x'` có phải chỉ số số hay không

B. Whether any key maps to `'x'` — Có khóa nào ánh xạ tới `'x'` không

C. Whether `'x'` is a dictionary key — `'x'` có là khóa Dictionary không

D. Whether `'x'` appears in values only — `'x'` có nằm trong các giá trị hay không

Đáp án: C

Giải thích: Với Dictionary, `in` mặc định kiểm tra khóa: `'x' in {'x': 3}` là `True`, còn `3 in {'x': 3}` là `False`. Muốn kiểm tra giá trị thay vì khóa, hãy xét thành viên trong `d.values()`.

Q17. May a List be a normal dictionary key? — Có thể dùng List làm khóa Dictionary thông thường không?

A. Yes, if the list has two elements — Có, nếu List có hai phần tử

B. Yes, if list values are distinct — Có, nếu các giá trị List khác nhau

C. No, lists are unhashable — Không, List không thể băm

D. Yes, every list is hashable — Có, mọi List đều có thể băm

Đáp án: C

Giải thích: List có thể thay đổi nên không thể băm và không dùng làm khóa Dictionary thông thường. Thử `{[1,2]: 'x'}` gây `TypeError`; một Tuple chứa các số có thể băm thì dùng được.

Q18. Can a tuple of integers be a dictionary key? — Tuple chứa số nguyên có thể làm khóa Dictionary không?

A. No, all tuples are unhashable — Không, mọi Tuple đều không thể băm

B. Yes, a tuple of integers is hashable — Có, Tuple chứa số nguyên có thể băm

C. Only if the tuple has one element — Chỉ nếu Tuple có một phần tử

D. Only if the tuple is empty — Chỉ nếu Tuple rỗng

Đáp án: B

Giải thích: Tuple chỉ chứa số nguyên có thể băm nên dùng làm khóa Dictionary, ví dụ `{(1,2): 'point'}`. Không phải mọi Tuple đều dùng được: Tuple chứa List cũng không thể băm vì một phần tử bên trong không thể băm.

Q19. How do you access the score in `d['A']` when it is a nested dictionary? — Lấy điểm trong Dictionary con `d['A']` thế nào?

A. `d['A', 'score']` — `d['A', 'score']`

B. `d['score']['A']` — `d['score']['A']`

C. `d.get('A', 'score')` — `d.get('A', 'score')`

D. `d['A']['score']` — `d['A']['score']`

Đáp án: D

Giải thích: `d['A']['score']` trước tiên lấy Dictionary con dưới khóa `'A'`, rồi tra mục `'score'` trong đó. Với `d={'A':{'score':9}}`, biểu thức trả 9; nếu thiếu một trong các khóa thì cách truy cập trực tiếp gây `KeyError`.

Q20. What must be unique within one dictionary? — Điều gì phải duy nhất trong một Dictionary?

A. Dictionary values — Các giá trị Dictionary

B. Both all keys and all values must be unique — Cả khóa và giá trị đều phải duy nhất

C. Dictionary keys — Các khóa Dictionary

D. Neither keys nor values must be unique — Cả khóa lẫn giá trị đều không cần duy nhất

Đáp án: C

Giải thích: Các khóa là duy nhất trong một Dictionary; gán lại cùng một khóa sẽ cập nhật mục cũ thay vì tạo thêm bản sao. Giá trị có thể lặp ở nhiều khóa nên tính duy nhất áp dụng cho khóa chứ không phải giá trị.

8. Nối sang bài tiếp theo

`List/Set → Dictionary → JSON/CSV → DataFrame`.

PY-09 — Hàm, return, lambda và phạm vi biến

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

`Vòng lặp → Hàm → OOP → Pipeline`

Học xong dùng để làm gì? Đóng gói phép tính thành hàm nhận dữ liệu, trả kết quả; dùng lambda làm khóa sắp xếp.

Kết nối kiến thức tiếp theo: PY-10 tổ chức dữ liệu và thao tác của một đối tượng thành class.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Function	hàm	Khối code tái sử dụng và nhận tham số
Parameter	tham số	Tên đầu vào dùng trong định nghĩa hàm
Return	trả về	Kết quả gửi từ hàm về nơi gọi
Lambda	hàm ẩn danh	Biểu thức hàm ngắn không cần đặt tên

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`def func(...):`	Định nghĩa một hàm, sử dụng return để trả kết quả
`default / keyword arguments`	Dùng tham số mặc định hoặc gọi theo tên cho dễ đọc
`lambda x: expr`	Dùng biểu thức ngắn khi hàm không cần nhiều câu lệnh
`sorted(..., key=...)`	Chọn tiêu chí để sắp xếp bản ghi
Biến cục bộ trong `def`	Tên tạo trong hàm thường chỉ dùng được trong hàm, không truy cập trực tiếp từ bên ngoài

4. Khái niệm liên quan

Một **hàm (function)** gom nhiều bước xử lý thành một công việc có tên, có thể gọi lại bằng dữ liệu khác nhau. Khi gọi hàm, giá trị đi vào qua **tham số**, kết quả có thể quay lại bằng **`return`**. `lambda` phù hợp cho một biểu thức ngắn, còn **phạm vi biến (scope)** giúp phân biệt biến chỉ dùng bên trong hàm với biến dùng bên ngoài. Ta dùng cùng ý tưởng tính tiền để lần lượt hiểu năm phần này.

4.1. Function — Hàm tái sử dụng

Từ vựng trong mục: Function — hàm; **Call** — lời gọi hàm.

Giả sử cửa hàng cần tính giá của món hàng nhân đôi khi mua hai phần giống nhau. Nếu viết phép nhân tại nhiều chỗ, thay đổi quy tắc sẽ khó. Ta định nghĩa `twice(x)` bằng từ khóa `def`: `x` là đầu vào, phép `x * 2` tạo kết quả. Khi viết `twice(3)`, Python chạy thân hàm với `x = 3`, rồi trả số 6. Dòng `print()` bên ngoài mới đưa số này lên màn hình.

```
def twice(x):
    return x * 2

print(twice(3))
```

Kết quả khi chạy:

```
6
```

Một định nghĩa hàm có thể dùng lại cho nhiều giá trị như 3, 5 hoặc 10 mà không viết lại phép nhân.

4.2. Parameter và Argument — Tham số và đối số

Từ vựng trong mục: Parameter — tham số; **Argument** — đối số.

Tham số là tên xuất hiện trong dòng định nghĩa, ví dụ `name` của `hello(name)`. **Đối số** là giá trị thật được truyền lúc gọi hàm, ví dụ `'Lan'` trong `hello('Lan')`. Trong thân hàm, `name` tạm nhận `'Lan'`, tạo chuỗi lời chào rồi `return` gửi nó về nơi gọi.

```
def hello(name):
    return f'Xin chào {name}!'

print(hello('Lan'))
```

Kết quả khi chạy:

```
Xin chào Lan!
```

`name` là tham số, còn `'Lan'` là đối số của lời gọi cụ thể. Truyền `'An'` sẽ tạo lời chào khác.

4.3. Return — Kết quả trả về

Từ vựng trong mục: Return — trả về; **Print** — hiển thị.

`return` khác `print()`: `return` **gửi giá trị về nơi gọi hàm** để chương trình có thể lưu hoặc tính tiếp; `print()` chỉ hiển thị. Ví dụ `add(a, b)` tính tổng hai số. Khi gán `total = add(2, 3)`, biến `total` nhận 5; lệnh `print(total)` mới làm số 5 xuất hiện trên màn hình.

```
def add(a, b):
    return a + b

total = add(2, 3)
print(total)
```

Kết quả khi chạy:

```
5
```

Giá trị trả về có thể lưu trong biến `total`; nếu hàm chỉ `print()` mà không `return`, nơi gọi không nhận được tổng để tính tiếp.

4.4. Lambda — Hàm ngắn dùng một biểu thức

Từ vựng trong mục: Lambda — hàm ẩn danh; **Expression** — biểu thức.

Khi chỉ cần phép biến đổi đơn giản, `lambda x: x + 1` là cách viết ngắn của một hàm nhận `x` và trả về `x + 1`. Chẳng hạn tăng số lượng hiện tại 5 lên thêm một món sẽ ra 6. `lambda` thích hợp làm tiêu chí sắp xếp hay lọc; công việc có nhiều bước nên dùng `def` để dễ đọc.

```
increase = lambda x: x + 1
print(increase(5))
```

Kết quả khi chạy:

```
6
```

Hàm được tạo bằng `lambda` nhận 5 và cho 6. Đây là một biểu thức, không có thân nhiều câu lệnh như `def`.

4.5. Scope — Phạm vi biến

Từ vựng trong mục: Scope — phạm vi biến; **Local variable** — biến cục bộ.

Phạm vi biến xác định nơi một tên biến có thể được dùng. Trong hàm `with_fee(price)`, biến `fee = 5` chỉ được tạo khi hàm chạy và dùng trong hàm; đó là **biến cục bộ (local)**. Bên ngoài hàm, chương trình chỉ dùng giá trị `return`, không truy cập trực tiếp `fee`. Ví dụ đơn hàng giá 100 (nghìn đồng) có phí xử lý 5 (nghìn đồng).

```
def with_fee(price):
    fee = 5
    return price + fee

final_price = with_fee(100)
print(final_price)
```

Kết quả khi chạy:

```
105
```

Số 105 là tổng giá và phí, đơn vị nghìn đồng. Biến `fee` ở trong hàm; nếu gọi `print(fee)` ở ngoài, Python sẽ báo `NameError`.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tính thuế giá trị gia tăng

Từ vựng trong mục: Function — hàm.

Tình huống thực tế: Một cửa hàng bán sản phẩm giá 100.000 đồng chưa có thuế. Để minh họa cách tính, cửa hàng dùng mức VAT giả định 10% cho sản phẩm này.

Yêu cầu: Viết hàm tính giá sau thuế và in số tiền cần thanh toán.

Vì sao chọn cách giải này? Hàm `with_vat` nhận giá và mức thuế, với `rate=0.10` là mức mặc định. `return` trả tổng tiền đã làm tròn.

Quy trình giải:

1. Định nghĩa hàm với tham số giá và thuế mặc định.
2. Nhân giá với `1 + rate`.
3. Gọi hàm bằng giá 100.000 và in kết quả.

```
def with_vat(price, rate=0.10):
    return round(price*(1+rate))
print(with_vat(100000))
```

Kết quả khi chạy:

```
110000
```

Kết quả: `110000` đồng — giá 100.000 đồng cộng 10% thuế.

Tình huống 2 — Sắp điểm theo thứ hạng

Từ vựng trong mục: Parameter — tham số.

Tình huống thực tế: Trung tâm có hai hồ sơ học viên: A đạt 8 điểm và B đạt 10 điểm trên thang 10. Cần ưu tiên hiển thị người có điểm cao hơn.

Yêu cầu: In danh sách hồ sơ theo điểm giảm dần.

Vì sao chọn cách giải này? `sorted()` tạo danh sách đã sắp xếp; `lambda r: r['score']` cho biết phải dùng trường điểm để so sánh.

Quy trình giải:

1. Tạo hai Dictionary học viên.
2. Dùng `sorted` với `key` lấy điểm và `reverse=True`.
3. In danh sách sau khi sắp xếp.

```
people=[{"name":"A","score":8},{"name":"B","score":10}]
print(sorted(people,key=lambda r:r["score"],reverse=True))
```

Kết quả khi chạy:

```
[{'name': 'B', 'score': 10}, {'name': 'A', 'score': 8}]
```

Kết quả: `[{'name': 'B', 'score': 10}, {'name': 'A', 'score': 8}]` — B đứng trước A.

Tình huống 3 — Kiểm tra điểm hợp lệ

Từ vựng trong mục: Return — trả về.

Tình huống thực tế: Một trung tâm chỉ chấp nhận điểm kiểm tra từ 0 đến 10. Nhân viên cần kiểm tra điểm nhập vào là 11.

Yêu cầu: Viết hàm trả về `True` khi điểm nằm trong khoảng hợp lệ, ngược lại trả về `False`.

Vì sao chọn cách giải này? Hàm `valid_score()` kết hợp hai phép so sánh trong một biểu thức, nên có thể gọi lại cho nhiều học viên.

Quy trình giải:

1. Định nghĩa hàm nhận điểm `x`.
2. Kiểm tra `0 <= x <= 10`.
3. Truyền điểm 11 vào hàm và in kết quả.

```
def valid_score(x):
    return 0<=x<=10
print(valid_score(11))
```

Kết quả khi chạy:

```
False
```

Kết quả: `False` — điểm 11 vượt quá thang điểm 10.

Tình huống 4 — Trả thống kê đơn giản

Từ vựng trong mục: Lambda — hàm ẩn danh.

Tình huống thực tế: Một cửa hàng ghi số lượng sản phẩm bán được trong ba ngày lần lượt là 7, 8 và 9 món.

Yêu cầu: Viết hàm trả về cả tổng số món lẫn số món trung bình mỗi ngày.

Vì sao chọn cách giải này? `return` có thể gửi về hai kết quả trong một tuple, sau đó dùng hai biến để nhận tổng và trung bình.

Quy trình giải:

1. Truyền list `[7, 8, 9]` vào hàm.
2. Dùng `sum()` tính tổng và chia cho số ngày để tính trung bình.
3. Nhận hai kết quả và in ra.

```
def summary(values):
    return sum(values), sum(values)/len(values)
```

```
total,average=summary([7,8,9])
print(total,average)
```

Kết quả khi chạy:

```
24 8.0
```

Kết quả: `24 8.0` — bán tổng cộng 24 món, trung bình 8 món/ngày.

Tình huống 5 — Lọc yêu cầu ưu tiên

Từ vựng trong mục: Function — hàm.

Tình huống thực tế: Bộ phận vận hành có bốn yêu cầu với điểm ưu tiên `[1, 5, 2, 4]`. Chỉ các yêu cầu có điểm từ 4 trở lên cần được xử lý trước.

Yêu cầu: In danh sách các điểm ưu tiên đạt ngưỡng.

Vì sao chọn cách giải này? `filter()` áp dụng điều kiện do `lambda` định nghĩa cho từng điểm; `list()` chuyển các kết quả được giữ lại thành danh sách.

Quy trình giải:

1. Lưu bốn điểm ưu tiên.
2. Lọc từng điểm bằng điều kiện `x >= 4`.
3. Chuyển kết quả lọc thành list rồi in ra.

```
priorities=[1,5,2,4]
print(list(filter(lambda x:x>=4,priorities)))
```

Kết quả khi chạy:

```
[5, 4]
```

Kết quả: `[5, 4]` — chỉ hai yêu cầu đạt ngưỡng ưu tiên.

6. Sai lầm thường gặp

- `return` trả giá trị về chỗ gọi hàm; nếu muốn thấy giá trị trên màn hình thì gọi `print()`.
- Tham số mặc định như `rate=0.10` được dùng khi không truyền giá trị khác vào.
- `lambda` phù hợp biểu thức ngắn, còn xử lý nhiều bước nên dùng `def` cho dễ đọc.
- Hàm tính trung bình cần xử lý danh sách rỗng trước khi chia cho `len(values)`.
- Khi sắp xếp bản ghi, phải chọn đúng trường dữ liệu trong `key`.
- Không dùng trực tiếp biến cục bộ của hàm từ bên ngoài; đưa dữ liệu cần dùng ra ngoài bằng `return`.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Python function? — Hàm Python là gì?

A. Only a variable declared at module level — Chỉ biến khai báo ở cấp module

B. Only code that prints text — Chỉ mã in văn bản

C. Only a loop executed once — Chỉ vòng lặp chạy một lần

D. Reusable callable code that can accept inputs — Phần mã có thể gọi lại, có thể nhận đầu vào

Đáp án: D

Giải thích: Hàm Python là phần mã có thể gọi lại nhiều lần, có thể nhận đầu vào và trả kết quả. Hàm thường có thể định nghĩa bằng `def`, còn `lambda` cũng tạo hàm từ một biểu thức; không phải mọi hàm đều bắt buộc có tên.

Q2. What is a parameter? — Tham số là gì?

A. A value supplied by the caller — Giá trị truyền từ nơi gọi

B. A name declared in a function's definition — Tên khai báo trong định nghĩa hàm

C. The value printed by a function — Giá trị được hàm in ra

D. A variable always declared global — Biến luôn được khai báo toàn cục

Đáp án: B

Giải thích: Tham số là tên khai báo trong định nghĩa hàm để nhận đầu vào. Trong `def double(x):`, `x` là tham số, còn số `3` trong `double(3)` là đối số được truyền khi gọi.

Q3. What is an argument? — Đối số là gì?

A. The name in the function definition — Tên trong định nghĩa hàm

B. A keyword beginning a function body — Từ khóa bắt đầu thân hàm

C. A value supplied in a function call — Giá trị truyền khi gọi hàm

D. The result returned by a function — Kết quả hàm trả về

Đáp án: C

Giải thích: Đối số là giá trị thực tế được truyền vào khi gọi hàm. Với `double(3)`, số nguyên `3` là đối số gắn vào tham số `x`; tên tham số xuất hiện trong phần định nghĩa.

Q4. What is a return value? — Giá trị trả về là gì?

A. A result sent back to the caller — Kết quả gửi về nơi gọi

B. A parameter name — Tên tham số

C. The function's source-code text — Văn bản mã nguồn hàm

D. A displayed message from `print` — Thông báo hiển thị bởi `print`

Đáp án: A

Giải thích: Giá trị trả về là thông tin hàm gửi về nơi gọi qua câu lệnh `return`. Khác `print` dùng để hiển thị, `return` giúp mã gọi hàm tiếp tục sử dụng kết quả tính toán.

Q5. Which keyword defines a normal function? — Từ khóa nào định nghĩa hàm thông thường?

A. `return` — `return`

B. `lambda` — `lambda`

C. `if` — `if`

D. `def` — `def`

Đáp án: D

Giải thích: `def` mở đầu định nghĩa hàm có tên, tiếp theo là tham số, dấu hai chấm và thân hàm thụt lề. `lambda` tạo biểu thức hàm, còn `return` dùng bên trong thân để trả kết quả.

Q6. Which keyword sends a value back to the caller? — Từ khóa nào trả giá trị về nơi gọi?

A. `lambda` — `lambda`

B. `if` — `if`

C. `def` — `def`

D. `return` — `return`

Đáp án: D

Giải thích: `return` kết thúc lần gọi hàm hiện tại và chuyển giá trị đã chỉ ra về nơi gọi. Với `return x * 2`, nơi gọi nhận giá trị gấp đôi; `print(x * 2)` chỉ hiển thị rồi trả `None`.

Q7. Which keyword defines a short anonymous function expression? — Từ khóa nào tạo biểu thức hàm ngắn không tên?

A. `def` — `def`

B. `return` — `return`

C. `lambda` — `lambda`

D. `if` — `if`

Đáp án: C

Giải thích: `lambda` tạo hàm từ một biểu thức, chẳng hạn `lambda x: x * 2`. Hàm có thể gán cho tên hoặc truyền làm đối số, khác `def` mở định nghĩa hàm theo dạng câu lệnh.

Q8. Which keyword supplies a conditional branch inside a function? — Từ khóa nào tạo nhánh điều kiện bên trong hàm?

A. `lambda` — `lambda`

B. `if` — `if`

C. `def` — `def`

D. `return` — `return`

Đáp án: B

Giải thích: `if` mở nhánh điều kiện bên trong hàm, giúp hàm chọn đường thực thi theo đầu vào. Bản thân `if` không tạo hàm và cũng không trả giá trị về nơi gọi.

Q9. What does a default parameter provide? — Tham số mặc định có tác dụng gì?

A. A value calculated only after the function returns — Giá trị chỉ tính sau khi hàm trả về

B. A value that must be supplied for every call — Giá trị bắt buộc phải truyền ở mọi lần gọi

C. A fallback value used when the caller omits an argument — Giá trị dự phòng khi nơi gọi không truyền đối số

D. A value that overrides every explicit argument — Giá trị ghi đè mọi đối số truyền vào

Đáp án: C

Giải thích: Giá trị mặc định của tham số được dùng khi nơi gọi bỏ qua đối số đó; với `def greet(name='An')`, `greet()` dùng `'An'`. Nếu gọi `greet('Binh')` thì giá trị truyền vào thay mặc định, vì vậy đây là dự phòng chứ không phải giá trị bắt buộc.

Q10. What is a keyword argument? — Đối số theo tên là gì?

A. An argument supplied using its parameter's name — Đối số truyền bằng tên tham số

B. A local variable defined outside the function — Biến cục bộ định nghĩa ngoài hàm

C. A function name passed without calling — Tên hàm được truyền mà không gọi

D. An argument always matched only by position — Đối số luôn ghép bằng vị trí

Đáp án: A

Giải thích: Đối số theo tên nêu rõ tên tham số lúc gọi, ví dụ `greet(name='An')`. Nó khác đối số theo vị trí như `greet('An')`, vốn ghép với tham số dựa trên vị trí truyền vào.

Q11. What does `sorted(rows, key=func)` use `func` for? — Trong `sorted(rows, key=func)`, func dùng để làm gì?

A. Filtering out all false-valued items before sorting — Loại mọi phần tử có giá trị sai trước khi sắp

B. Computing a sort key for each item — Tính khóa sắp xếp cho từng phần tử

C. Replacing every row with `None` — Thay mỗi hàng bằng `None`

D. Printing each row during sorting — In từng hàng trong lúc sắp

Đáp án: B

Giải thích: Trong `sorted(rows, key=func)`, `func` tính giá trị khóa so sánh cho từng phần tử; phép sắp xếp dùng các khóa đó. Ví dụ `key=len` sẽ ưu tiên chuỗi ngắn trước theo độ dài thay vì thứ tự chữ cái.

Q12. What does `filter(test, items)` keep? — `filter(test, items)` giữ gì?

A. Only items whose test result is false — Chỉ phần tử có kết quả kiểm tra sai

B. Items in descending order — Phần tử theo thứ tự giảm dần

C. Items for which `test(item)` is truthy — Phần tử mà `test(item)` có giá trị đúng

D. All items, regardless of the test — Mọi phần tử, bất kể phép kiểm tra

Đáp án: C

Giải thích: `filter(test, items)` chọn phần tử mà `test(item)` có giá trị đúng, trả iterator trong Python 3. Với `filter(lambda x: x > 0, [-1,2])`, chuyển sang `list` được `[2]`, không phải mọi phần tử ban đầu.

Q13. What is a local variable? — Biến cục bộ là gì?

A. Any name defined at module level — Bất kỳ tên nào khai báo cấp module

B. A name automatically shared by every function — Tên tự chia sẻ cho mọi hàm

C. A name that can never change value — Tên không bao giờ đổi giá trị

D. A name bound within a function's local scope — Tên được gán trong phạm vi cục bộ hàm

Đáp án: D

Giải thích: Biến cục bộ là tên được gán trong phạm vi cục bộ của hàm, như `total` gán trong `def add():`. Nó không tự tồn tại ngoài hàm, khác tên toàn cục khai báo ở cấp module.

Q14. What is a global variable? — Biến toàn cục là gì?

A. A name bound at module level — Tên được gán ở cấp module

B. A name that exists only inside a class — Tên chỉ tồn tại trong class

C. A name inaccessible to all functions — Tên mọi hàm đều không truy cập được

D. A name created only by `lambda` — Tên chỉ do `lambda` tạo

Đáp án: A

Giải thích: Biến toàn cục là tên được gán ở cấp module và thường có thể đọc trong hàm nếu không bị che khuất. Muốn gán lại chính biến toàn cục từ trong hàm thường phải khai báo `global`, khác việc chỉ đọc giá trị.

Q15. What is a lambda expression limited to? — Nội dung của lambda bị giới hạn như thế nào?

A. A complete `try/except` statement block — Khối câu lệnh `try/except` đầy đủ

B. A single expression — Một biểu thức

C. Only a numeric literal — Chỉ một literal số

D. A series of ordinary statements — Chuỗi các câu lệnh thông thường

Đáp án: B

Giải thích: Thân `lambda` trong Python chỉ chứa một biểu thức và giá trị biểu thức trở thành kết quả hàm. Nó không chứa khối nhiều câu lệnh thông thường như chuỗi lệnh `for` và gán; cần dùng `def` trong trường hợp đó.

Q16. What is a closure? — Closure là gì?

A. A function forced to return `None` — Hàm bị buộc trả `None`

B. A function that prints its enclosing scope — Hàm in phạm vi bao ngoài

C. A function that discards all free variables — Hàm loại bỏ mọi biến tự do

D. A function whose enclosing names remain accessible — Hàm vẫn truy cập được tên của phạm vi bao ngoài

Đáp án: D

Giải thích: Closure là hàm vẫn truy cập được các tên thuộc phạm vi bao ngoài, kể cả sau khi hàm ngoài đã trả về. Nó giữ liên kết tham chiếu tới các tên đó chứ không nhất thiết chụp cố định giá trị tại một thời điểm.

Q17. When a function reaches its end without executing `return`, what value does it return? — Khi hàm chạy đến cuối mà không thực hiện `return`, nó trả giá trị nào?

A. `None` — `None`

B. An empty string `''` — Chuỗi rỗng `''`

C. The last local variable — Biến cục bộ cuối

D. `NameError` — `NameError`

Đáp án: A

Giải thích: Nếu hàm chạy đến cuối mà không có `return` tường minh, Python ngầm trả `None`. Hàm vẫn có thể in thông báo nhưng việc in không biến thông báo đó thành giá trị trả về.

Q18. What does `print()` do instead of returning a useful result? — `print()` làm gì thay vì trả một kết quả để tính tiếp?

A. Displays output while returning `None` — Hiển thị đầu ra và trả `None`

B. Returns a tuple with the printed text — Trả Tuple chứa văn bản đã in

C. Raises `TypeError` whenever called — Mỗi khi gọi đều gây `TypeError`

D. Always returns the printed string — Luôn trả chuỗi đã in

Đáp án: A

Giải thích: `print()` đưa văn bản ra đầu ra chuẩn và giá trị trả về là `None`. Ví dụ `result = print('Hi')` hiển thị `Hi` nhưng lưu `None` vào `result`; dùng `return` trong hàm nếu mã phía sau cần kết quả.

Q19. What does a function's `return a, b` actually return? — `return a, b` thực chất trả gì?

A. The result of adding `a` and `b` — Kết quả cộng `a` với `b`

B. A tuple containing both values — Tuple chứa cả hai giá trị

C. A list containing both values — List chứa cả hai giá trị

D. The two values as separate function return objects — Hai giá trị thành hai đối tượng trả về riêng

Đáp án: B

Giải thích: Biểu thức trả về có các giá trị cách nhau bằng dấu phẩy đóng gói chúng thành Tuple, nên `return a, b` trả `(a, b)`. Nơi gọi có thể giải nén ra hai tên, nhưng hàm thực tế trả một đối tượng Tuple.

Q20. What happens when code outside a function reads an undefined local name? — Đọc biến cục bộ không tồn tại ở ngoài hàm gây gì?

A. The most recent printed value is used — Dùng giá trị vừa được in

B. The local name is exported automatically — Tên cục bộ tự xuất ra ngoài

C. `NameError` for the undefined name — `NameError` vì tên chưa được định nghĩa

D. `None` is silently substituted — Tự thay bằng `None`

Đáp án: C

Giải thích: Tên cục bộ chỉ được gán bên trong hàm không tự tồn tại ở phạm vi module. Đọc tên chưa được định nghĩa ở ngoài gây `NameError`; cần trả về tường minh hoặc lưu vào liên kết có thể truy cập.

8. Nối sang bài tiếp theo

`Vòng lặp → Hàm → OOP → Pipeline`.

PY-10 — Class, Object, constructor và đóng gói

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Class — lớp; **Object** — đối tượng.

`Hàm / Dictionary → OOP → Kế thừa`

Học xong dùng để làm gì? Thiết kế đối tượng học viên hoặc tài khoản với thuộc tính, phương thức và kiểm tra đầu vào.

Kết nối kiến thức tiếp theo: PY-11 mở rộng các loại đối tượng bằng kế thừa và ghi đè phương thức.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Class	lớp	Khuôn mô tả dữ liệu và cách xử lý
Object	đối tượng	Một thể hiện cụ thể của lớp
Constructor	hàm khởi tạo	Thiết lập thuộc tính lúc tạo instance
Encapsulation	đóng gói	Đặt dữ liệu và hành vi bên trong đối tượng

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`class Name:`	Định nghĩa kiểu đối tượng và các phương thức
`__init__(self, ...)`	Khởi tạo trạng thái của từng instance
`self.attribute`	Truy cập thuộc tính thuộc về chính đối tượng
`__str__/set_*/get_*`	Hiển thị và kiểm tra thao tác thay đổi giá trị

4. Khái niệm liên quan

Lập trình hướng đối tượng (OOP) giúp gom **dữ liệu của một đối tượng** và **những thao tác dùng dữ liệu ấy** vào cùng một cấu trúc. Ví dụ một học viên có tên, điểm và cách hiển thị hồ sơ; thay vì đặt các biến rời rạc, ta tạo một lớp `Student`. Từ lớp này, bài học lần lượt giải thích class, object, `__init__`, `self` và cách đóng gói thao tác thay đổi dữ liệu.

4.1. Class — Khuôn mẫu đối tượng

Từ vựng trong mục: Class — lớp; **Method** — phương thức.

Class giống bản thiết kế: định nghĩa những việc các đối tượng cùng loại có thể thực hiện. Ví dụ lớp `Student` có phương thức `greet()` trả một lời chào. Dòng `class Student:` tạo bản thiết kế, `def greet(self)` định nghĩa hành động; `self` đại diện cho đối tượng cụ thể đang gọi phương thức. Chỉ viết class chưa tạo hồ sơ học viên nào; `Student()` ở cuối mới tạo một đối tượng để gọi `greet()`.

```
class Student:
    def greet(self):
        return 'Xin chào'

student = Student()
print(student.greet())
```

Kết quả khi chạy:

```
Xin chào
```

Đối tượng `student` gọi phương thức được định nghĩa bên trong lớp `Student`.

4.2. Object — Mỗi học viên là một đối tượng

Từ vựng trong mục: Object — đối tượng; **Attribute** — thuộc tính.

Object là một thể hiện được tạo từ class. Cùng một mẫu `Student`, ta có thể tạo hai học viên An và Lan, mỗi người có tên riêng. Thuộc tính `self.name` lưu tên **trên từng đối tượng**, vì vậy thay tên của An không tự thay tên của Lan. Ta dùng dấu chấm `student.name` để lấy một thuộc tính.

```
class Student:
    def __init__(self, name):
        self.name = name

a = Student('An')
b = Student('Lan')
print(a.name, b.name)
```

Kết quả khi chạy:

```
An Lan
```

Cả `a` và `b` đều thuộc lớp Student, nhưng chứa dữ liệu tên riêng.

4.3. Constructor và self — Khởi tạo trạng thái

Từ vựng trong mục: Constructor — bước khởi tạo; **init** — phương thức khởi tạo; **self** — chính đối tượng.

Khi tạo `Student('An', 8)`, Python tạo đối tượng mới rồi tự gọi `__init__` để thiết lập tên và điểm ban đầu. Tham số `self` là đối tượng vừa được tạo; `name` và `score` nhận đối số truyền khi gọi. Lệnh `self.score = score` lưu điểm bên trong học viên, không chỉ trong biến tạm của hàm. Điểm ở đây tính trên thang 10.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

student = Student('An', 8)
print(student.name, student.score)
```

Kết quả khi chạy:

```
An 8
```

Phương thức `__init__` đưa dữ liệu đầu vào thành hai thuộc tính của cùng hồ sơ học viên.

4.4. Encapsulation — Đóng gói dữ liệu và quy tắc xử lý

Từ vựng trong mục: Encapsulation — đóng gói; **Method** — phương thức; **Validation** — kiểm tra hợp lệ.

Đóng gói nghĩa là quản lý dữ liệu cùng các thao tác tác động lên nó trong lớp. Ví dụ điểm chỉ được phép trong khoảng 0–10: lớp `Score` lưu số hiện tại tại `self._value` và có phương thức `set_value()` kiểm tra trước khi cập nhật. Dấu `_` là **quy ước** nhắc người lập trình không nên sửa trực tiếp, không phải cơ chế cấm truy cập tuyệt đối. Đọc điểm qua `get_value()` giúp giữ cùng một cách sử dụng.

```
class Score:
    def __init__(self):
        self._value = 0

    def set_value(self, x):
        if not 0 <= x <= 10:
            raise ValueError('Điểm ngoài khoảng 0–10')
        self._value = x

    def get_value(self):
        return self._value

score = Score()
score.set_value(9)
print(score.get_value())
```

Kết quả khi chạy:

```
9
```

Điểm 9 hợp lệ nên được lưu và đọc lại. Với điểm 11, `set_value` sẽ báo `ValueError` thay vì ghi dữ liệu sai.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tạo hồ sơ học viên

Từ vựng trong mục: Class — lớp.

Tình huống thực tế: Trung tâm tạo hồ sơ học viên An, có điểm 8 trên thang 10. In tên và điểm của học viên.

Vì sao chọn cách giải này? Dùng class `Student` để gom tên và điểm của một học viên.

Quy trình giải:

1. Tạo lớp `Student` với phương thức `__init__` nhận `name` và `score`.
2. Lưu hai đối số vào thuộc tính của đối tượng bằng `self`.
3. Tạo hồ sơ An với điểm 8 rồi in tên và điểm.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

s = Student('An', 8)
print(s.name, s.score)
```

Kết quả khi chạy:

```
An 8
```

Tình huống 2 — Giỏ hàng có phương thức

Từ vựng trong mục: Object — đối tượng.

Tình huống thực tế: Một khách đưa hai món giá 100 và 200 đơn vị tiền vào giỏ hàng. Tính tổng tiền trong giỏ.

Vì sao chọn cách giải này? Đối tượng `Cart` giữ list giá và có phương thức `add`, `total`.

Quy trình giải:

1. Tạo giỏ hàng `Cart` có danh sách `prices` ban đầu rỗng.
2. Dùng phương thức `add` thêm hai giá 100 và 200.
3. Dùng phương thức `total` cộng các giá và in 300.

```
class Cart:
    def __init__(self):
        self.prices = []

    def add(self, price):
        self.prices.append(price)

    def total(self):
        return sum(self.prices)

c = Cart()
c.add(100)
c.add(200)
print(c.total())
```

Kết quả khi chạy:

```
300
```

Tình huống 3 — Ràng buộc điểm nhập

Từ vựng trong mục: Constructor — hàm khởi tạo.

Tình huống thực tế: Hệ thống chỉ cho phép lưu điểm trong khoảng 0 đến 10. Một học viên có điểm 9. Lưu và in điểm hợp lệ.

Vì sao chọn cách giải này? Phương thức `set` kiểm tra giới hạn trước khi cập nhật.

Quy trình giải:

1. Khởi tạo đối tượng `Score` với giá trị ban đầu bằng 0.
2. Gọi `set(9)` để kiểm tra điểm thuộc khoảng 0–10 rồi lưu giá trị.
3. In điểm đã được lưu; nếu truyền 11, phương thức sẽ báo lỗi.

```
class Score:
    def __init__(self):
        self.value = 0

    def set(self, x):
        if not 0 <= x <= 10:
            raise ValueError('score')
        self.value = x

s = Score()
s.set(9)
print(s.value)
```

Kết quả khi chạy:

```
9
```

Tình huống 4 — In hóa đơn dễ đọc

Từ vựng trong mục: Encapsulation — đóng gói.

Tình huống thực tế: Một hóa đơn có mã O01 và số tiền 150.000 VND. In hóa đơn ở dạng dễ đọc.

Vì sao chọn cách giải này? `__str__` quy định cách đối tượng hiển thị khi truyền vào `print`.

Quy trình giải:

1. Lưu mã `001` và số tiền 150000 trong đối tượng `Order`.
2. Định nghĩa `__str__` để tạo chuỗi hiển thị mã đơn và số tiền.
3. Gọi `print(Order(...))` để Python tự dùng `__str__`.

```
class Order:
    def __init__(self, code, total):
        self.code = code
        self.total = total

    def __str__(self):
        return f'{self.code}: {self.total} VND'

print(Order('O01', 150000))
```

Kết quả khi chạy:

```
001: 150000 VND
```

Tình huống 5 — Hai cảm biến độc lập

Từ vựng trong mục: Class — lớp.

Tình huống thực tế: Hai cảm biến có giá trị ban đầu lần lượt 20 và 40. Cảm biến đầu được cập nhật thành 25. Kiểm tra hai cảm biến có lưu giá trị độc lập hay không.

Vì sao chọn cách giải này? Mỗi lần gọi `Sensor()` tạo một đối tượng có thuộc tính `value` riêng.

Quy trình giải:

1. Tạo hai đối tượng `Sensor(20)` và `Sensor(40)` độc lập.
2. Cập nhật `a.value` thành 25, không sửa `b.value`.
3. In giá trị của cả hai để thấy `25 40`.

```
class Sensor:
    def __init__(self, value):
        self.value = value

a, b = Sensor(20), Sensor(40)
a.value = 25
print(a.value, b.value)
```

Kết quả khi chạy:

```
25 40
```

6. Sai lầm thường gặp

- `class` là khuôn mẫu; đối tượng chỉ xuất hiện khi gọi lớp như `Student('An', 8)`.
- `self.name` là thuộc tính của đối tượng, không phải biến dùng chung giữa mọi học viên. Mỗi đối tượng có trạng thái riêng.
- `__init__` chuẩn bị dữ liệu ban đầu, còn `__str__` quy định cách hiển thị; hai phương thức có mục đích khác nhau.
- `self._value` dùng dấu gạch dưới theo quy ước, không tự tạo thuộc tính hoàn toàn riêng tư. Muốn bảo vệ dữ liệu cần kiểm soát cách cập nhật và kiểm tra hợp lệ.
- Điểm vượt 0–10 sẽ phát sinh `ValueError` trong ví dụ `Score`; không bỏ qua xử lý lỗi ở chương trình thực tế.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a class in Python OOP? — Class trong OOP Python là gì?

A. Only a built-in Python function — Chỉ một hàm Python có sẵn

B. A template defining object state and behavior — Khuôn mẫu định nghĩa trạng thái và hành vi đối tượng

C. A concrete instance created from a class — Instance cụ thể được tạo từ class

D. A value stored in one object — Giá trị lưu trong một đối tượng

Đáp án: B

Giải thích: Class định nghĩa cấu trúc dùng chung cho các đối tượng, gồm thuộc tính và phương thức. Ví dụ class `Student` có thể định nghĩa trường tên và phương thức `display()`; từng sinh viên là một instance của class đó.

Q2. What is an object or instance? — Object hay instance là gì?

A. A statement defining a new class — Câu lệnh định nghĩa class mới

B. A concrete instance of a class — Instance cụ thể của class

C. An attribute name only — Chỉ tên thuộc tính

D. A block of import statements — Khối lệnh import

Đáp án: B

Giải thích: Object hay instance là giá trị cụ thể được tạo từ class. Hai instance `Student` có thể chạy cùng các phương thức nhưng có giá trị `name` khác nhau, vì thế class không đồng nghĩa với một đối tượng.

Q3. What is an attribute? — Attribute là gì?

A. A method invocation — Lời gọi phương thức

B. Data or a value associated with an object — Dữ liệu hoặc giá trị gắn với đối tượng

C. A syntax token beginning every class — Token cú pháp bắt đầu mọi class

D. A class declaration — Câu lệnh khai báo class

Đáp án: B

Giải thích: Attribute là dữ liệu hoặc giá trị gắn với đối tượng được truy cập qua tên, như `student.name`. Nó biểu diễn trạng thái hay đặc điểm, còn phương thức được gọi để thực hiện hành vi.

Q4. What is an instance method? — Phương thức đối tượng là gì?

A. An object stored in a list — Đối tượng được lưu trong List

B. A value that cannot be called — Giá trị không thể gọi

C. A class-defined function operating on an instance — Hàm định nghĩa trong class thao tác trên instance

D. A class with no methods — Class không có phương thức

Đáp án: C

Giải thích: Phương thức instance là hàm định nghĩa trong class và nhận đối tượng làm đối số đầu khi gọi qua instance. Với `student.show()`, phương thức có thể đọc `self.name`; thuộc tính như `student.name` là dữ liệu, không phải lời gọi phương thức.

Q5. Which syntax begins a class definition? — Cú pháp nào mở đầu định nghĩa class?

A. `__init__` — `__init__`

B. `self` — `self`

C. `class Student:` — `class Student:`

D. `Student()` — `Student()`

Đáp án: C

Giải thích: `class Student:` mở đầu định nghĩa class; các lệnh thụt lề bên dưới tạo thân class. Gọi `Student()` là thao tác khác: tạo instance sau khi class phù hợp đã được định nghĩa.

Q6. If `Student` requires no constructor arguments, which expression creates a Student instance? — Nếu `Student` không yêu cầu đối số khởi tạo, biểu thức nào tạo một instance Student?

A. `Student()` — `Student()`

B. `def Student:` — `def Student:`

C. `self.Student` — `self.Student`

D. `class Student:` — `class Student:`

Đáp án: A

Giải thích: Gọi `Student()` tạo instance nếu class có thể khởi tạo mà không cần đối số bắt buộc. Nếu `__init__` yêu cầu đầu vào khác thì phải truyền chúng khi gọi, không được cho rằng ngoặc trống luôn hợp lệ.

Q7. Which special method initializes instance attributes? — Phương thức đặc biệt nào khởi tạo thuộc tính đối tượng?

A. `__repr__` — `__repr__`

B. `__newstate__` — `__newstate__`

C. `__str__` — `__str__`

D. `__init__` — `__init__`

Đáp án: D

Giải thích: `__init__` khởi tạo instance vừa được tạo, thường gán thuộc tính như `self.name = name`. Python tạo đối tượng trước rồi mới gọi hàm khởi tạo; bản thân `__init__` không thực hiện cấp phát đối tượng.

Q8. What does `self` refer to in an instance method? — `self` trong phương thức đối tượng chỉ gì?

A. The class object, never an instance — Đối tượng class, không bao giờ là instance

B. The first attribute value — Giá trị thuộc tính đầu tiên

C. A newly created unrelated object — Đối tượng khác vừa được tạo

D. The instance receiving the method call — Instance nhận lời gọi phương thức

Đáp án: D

Giải thích: Trong phương thức instance thông thường, `self` chỉ đối tượng cụ thể nhận lời gọi. Vì vậy `a.show()` và `b.show()` có thể đọc các thuộc tính khác nhau qua `self` dù cùng định nghĩa class.

Q9. What does `self.name = name` do? — `self.name = name` làm gì?

A. Stores `name` on the current instance — Lưu `name` trên instance hiện tại

B. Sets the same name on all instances — Đặt cùng tên trên mọi instance

C. Changes the class's name — Đổi tên class

D. Calls a method named `name` — Gọi phương thức tên `name`

Đáp án: A

Giải thích: `self.name = name` gán giá trị tham số `name` cho thuộc tính của instance hiện tại. Vì vậy mỗi đối tượng có thể mang tên khác nhau; lệnh không tạo một thuộc tính cấp class được dùng chung.

Q10. What does `__str__` normally customize? — `__str__` thường tùy chỉnh điều gì?

A. The readable string representation of an object — Biểu diễn chuỗi dễ đọc của đối tượng

B. The list of all class attributes — Danh sách mọi thuộc tính class

C. The class inheritance hierarchy — Hệ phân cấp kế thừa class

D. The instance's constructor arguments — Các đối số khởi tạo instance

Đáp án: A

Giải thích: `__str__` điều khiển chuỗi dễ đọc trả về từ `str(obj)` và thường được `print(obj)` hiển thị. Phương thức này phải trả một chuỗi thay vì tự in trực tiếp; nó khác `__init__` dùng để khởi tạo trạng thái.

Q11. What does `raise ValueError(...)` do? — `raise ValueError(...)` làm gì?

A. Raises an exception for an invalid value — Phát sinh ngoại lệ cho giá trị không hợp lệ

B. Marks an attribute private — Đánh dấu thuộc tính riêng tư

C. Converts invalid values to `None` — Chuyển giá trị sai thành `None`

D. Silently ignores invalid data — Lặng lẽ bỏ qua dữ liệu sai

Đáp án: A

Giải thích: `raise ValueError(...)` báo giá trị đầu vào không hợp lệ và ngắt luồng bình thường bằng một ngoại lệ. Ví dụ setter có thể từ chối tuổi âm thay vì lặng lẽ lưu trạng thái không hợp lý.

Q12. What does the prefix `_value` conventionally mean? — Tiền tố `_value` theo quy ước có nghĩa gì?

A. A constant that cannot be changed — Hằng không thể thay đổi

B. An attribute invisible to external code — Thuộc tính mã ngoài không thể thấy

C. An internal-use detail by convention — Chi tiết sử dụng nội bộ theo quy ước

D. An automatically encrypted value — Giá trị được tự động mã hóa

Đáp án: C

Giải thích: Một dấu gạch dưới đầu tên như `_value` theo quy ước đánh dấu chi tiết nội bộ. Python không thực thi tính riêng tư chỉ nhờ cách đặt tên này, nên mã bên ngoài vẫn có thể đọc thuộc tính nếu nó tồn tại.

Q13. What is encapsulation? — Đóng gói trong OOP là gì?

A. Copying every attribute into a global variable — Sao chép mọi thuộc tính sang biến toàn cục

B. Converting methods into strings — Chuyển phương thức thành chuỗi

C. Creating a new instance on every access — Tạo instance mới mỗi lần truy cập

D. Grouping related data and methods — Gom dữ liệu và phương thức liên quan

Đáp án: D

Giải thích: Đóng gói gom dữ liệu và các thao tác liên quan vào cùng class hoặc giao diện đối tượng. Ví dụ phương thức của `BankAccount` có thể cập nhật số dư kèm kiểm tra thay vì để mọi nơi gọi tự sửa các biến rời rạc.

Q14. Why validate values in a setter method? — Vì sao kiểm tra dữ liệu trong setter?

A. To make every instance share its private data — Buộc mọi instance chia sẻ dữ liệu riêng

B. To prevent all future method calls — Ngăn mọi lời gọi phương thức sau đó

C. To bypass all input validation — Bỏ qua toàn bộ kiểm tra dữ liệu

D. To validate inputs and preserve valid object state — Kiểm tra đầu vào và giữ trạng thái hợp lệ

Đáp án: D

Giải thích: Setter kiểm tra dữ liệu trước khi thay đổi trạng thái đối tượng, giúp giữ các ràng buộc như tuổi không được âm. Nếu thiếu kiểm tra, nơi gọi có thể lưu giá trị sai khiến phương thức khác hoạt động không đúng.

Q15. How do two instances of the same class relate? — Hai đối tượng cùng class có quan hệ gì?

A. They always share all instance attribute values — Chúng luôn dùng chung mọi giá trị thuộc tính instance

B. They cannot invoke the same methods — Chúng không thể gọi cùng phương thức

C. They share class behavior but can have different states — Chúng chia sẻ hành vi class nhưng có trạng thái khác nhau

D. They are the same object in memory — Chúng là cùng một đối tượng trong bộ nhớ

Đáp án: C

Giải thích: Các instance cùng class dùng chung định nghĩa phương thức nhưng có thể chứa các thuộc tính instance riêng. Nếu `a.name='An'` và `b.name='Binh'`, sửa `a.name` không tự thay đổi `b.name`.

Q16. What is a class attribute? — Thuộc tính cấp class là gì?

A. A method's local variable — Biến cục bộ của phương thức

B. An attribute defined on the class and found by lookup — Thuộc tính định nghĩa trên class và được tìm bằng tra cứu

C. An attribute defined on an object instance only — Thuộc tính chỉ định nghĩa trên instance

D. A required parameter of `__init__` — Tham số bắt buộc của `__init__`

Đáp án: B

Giải thích: Thuộc tính cấp class được khai báo trên class thay vì riêng từng instance và được tìm thấy khi tra cứu qua class hoặc instance. Instance có thể che khuất thuộc tính đó bằng giá trị riêng, nên đây không phải trường luôn đồng bộ vĩnh viễn.

Q17. What does `obj.method()` do? — `obj.method()` làm gì?

A. Invokes a method on `obj` — Gọi phương thức trên `obj`

B. Returns the object's class without invoking anything — Trả class của đối tượng mà không gọi gì

C. Always creates a new instance — Luôn tạo instance mới

D. Always deletes an attribute — Luôn xóa thuộc tính

Đáp án: A

Giải thích: `obj.method()` tìm và gọi phương thức có tên trên `obj`, thông thường truyền instance đó làm `self`. Chỉ đọc `obj.attribute` thì lấy giá trị, không tự gọi phương thức.

Q18. When the attribute exists, what does `obj.attribute` read? — Khi thuộc tính tồn tại, `obj.attribute` đọc gì?

A. Creates a new method automatically — Tự tạo phương thức mới

B. Reads the named attribute on `obj` — Đọc thuộc tính có tên trên `obj`

C. Invokes every method of the object — Gọi mọi phương thức của đối tượng

D. Returns `True` only for class attributes — Chỉ trả `True` với thuộc tính cấp class

Đáp án: B

Giải thích: `obj.attribute` truy cập giá trị của thuộc tính có tên nếu nó tồn tại. Khác `obj.method()`, biểu thức không gọi hàm; nếu thiếu thuộc tính, Python thông thường phát sinh `AttributeError`.

Q19. What does `isinstance(obj, Student)` check? — `isinstance(obj, Student)` kiểm tra gì?

A. The object's exact class only, excluding subclasses — Chỉ class chính xác, loại class con

B. Whether the object has a method named `Student` — Đối tượng có phương thức tên `Student` không

C. Whether the object is a `Student` or its subclass — Đối tượng thuộc `Student` hay class con không

D. The class name as a string — Tên class ở dạng chuỗi

Đáp án: C

Giải thích: `isinstance(obj, Student)` đúng khi `obj` thuộc class `Student` hoặc một class con của nó. Ngược lại, `type(obj) is Student` chỉ kiểm tra class thực tế chính xác và loại các instance của class con.

Q20. What does `type(obj)` return? — `type(obj)` trả về gì?

A. Whether a class membership test is true — Phép kiểm tra thuộc class có đúng không

B. The object itself with unchanged type — Chính đối tượng và kiểu không đổi

C. A Boolean checking only its attributes — Boolean chỉ kiểm tra thuộc tính

D. The object's actual runtime class — Class thực tế của đối tượng khi chạy

Đáp án: D

Giải thích: `type(obj)` trả về class thực tế của đối tượng lúc chạy, chẳng hạn `Student`. `isinstance(obj, Student)` thì trả Boolean về quan hệ thuộc class và có thể chấp nhận instance của class con.

8. Nối sang bài tiếp theo

`Hàm / Dictionary → OOP → Kế thừa.`

PY-11 — Kế thừa, đa hình và lớp trừu tượng ABC

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Inheritance — kế thừa; **Polymorphism** — đa hình.

`Class/Object → Inheritance → Abstract API → Kiến trúc ứng dụng`

Học xong dùng để làm gì? Dùng chung thao tác cho nhiều loại đối tượng, ví dụ các loại phí có công thức khác nhau.

Kết nối kiến thức tiếp theo: PY-12 đọc và lưu dữ liệu từ file/CSV, có xử lý lỗi.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Inheritance	kế thừa	Lớp con dùng lại các đặc điểm lớp cha
Polymorphism	đa hình	Cùng lời gọi nhưng các lớp có cách đáp khác nhau
Abstract class	lớp trừu tượng	Khung lớp ràng buộc phương thức cần cài đặt
Override	ghi đè	Lớp con cài lại phương thức của lớp cha

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`class Child(Parent)`	Kế thừa thông tin chung
`super().__init__(...)`	Gọi khởi tạo lớp cha để không lặp code
`ABC/@abstractmethod`	Bắt lớp con định nghĩa giao diện bắt buộc
`isinstance(obj,Class)`	Xác minh đối tượng thuộc lớp khi cần

4. Khái niệm liên quan

Kế thừa, đa hình và lớp trừu tượng giải quyết ba nhu cầu khác nhau trong OOP: dùng lại đặc điểm chung, xử lý cùng thao tác theo cách riêng và đặt ra chức năng bắt buộc cho lớp con. Từ ví dụ sản phẩm đến dịch vụ thông báo, ta xem từng khái niệm tác động lên lớp, đối tượng và phương thức thế nào.

4.1. Inheritance — Kế thừa từ lớp cha

Từ vựng trong mục: Inheritance — kế thừa; **Parent/child class** — lớp cha/lớp con.

Cửa hàng có nhiều loại sản phẩm, nhưng mỗi món đều có tên. Lớp cha `Product` giữ tên chung, còn lớp con `Electronic(Product)` nhận lại cách lưu tên và có thêm thuộc tính riêng `warranty_months` (tháng bảo hành). `super().__init__(name)` gọi phương thức khởi tạo của lớp cha; nhờ vậy không phải viết lại cách lưu tên ở mọi loại sản phẩm. Đây là **kế thừa**: lớp con dùng lại và mở rộng hành vi, thuộc tính từ lớp cha.

```
class Product:
    def __init__(self, name):
        self.name = name

class Electronic(Product):
    def __init__(self, name, warranty_months):
        super().__init__(name)
        self.warranty_months = warranty_months

phone = Electronic('Phone', 12)
print(phone.name, phone.warranty_months)
```

Kết quả khi chạy:

```
Phone 12
```

`Phone` là tên được lớp cha khởi tạo; `12` là số tháng bảo hành được lớp con bổ sung.

4.2. Polymorphism — Đa hình

Từ vựng trong mục: Polymorphism — đa hình; **Method** — phương thức.

Giả sử ứng dụng có thông báo bằng email và tin nhắn. Cả hai đều có phương thức `send(message)`, nhưng kết quả khác nhau tùy loại. **Đa hình** nghĩa là code gọi cùng một tên phương thức trên các đối tượng khác nhau mà mỗi đối tượng tự quyết định cách xử lý. Vòng `for` phía dưới không cần viết hai nhánh kiểm tra loại thông báo.

```
class Email:
    def send(self, message):
        return 'Email: ' + message

class SMS:
    def send(self, message):
        return 'SMS: ' + message

for sender in [Email(), SMS()]:
    print(sender.send('Hello'))
```

Kết quả khi chạy:

```
Email: Hello
SMS: Hello
```

Cùng lời gọi `send('Hello')` nhưng hai loại đối tượng trả về hai kết quả khác nhau.

4.3. Abstract class — Lớp trừu tượng

Từ vựng trong mục: Abstract class — lớp trừu tượng; **Abstract method** — phương thức trừu tượng.

Khi mọi bộ gửi thông báo **bắt buộc** phải có `send()`, ta tạo lớp trừu tượng `Notifier(ABC)` và đánh dấu `@abstractmethod`. Lớp này quy định hợp đồng nhưng chưa đưa ra cách gửi cụ thể. Lớp `Console` phải cài đặt `send()` mới tạo đối tượng được. Chỉ kế thừa `ABC` mà **không có phương thức trừu tượng** thì chưa áp đặt yêu cầu đó.

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
    def send(self, message):
        pass

class Console(Notifier):
    def send(self, message):
        return 'Console: ' + message

print(Console().send('Hello'))
```

Kết quả khi chạy:

```
Console: Hello
```

`Console` đáp ứng hợp đồng `send()`; nếu bỏ phương thức ấy khỏi `Console`, Python sẽ không cho khởi tạo đối tượng.

4.4. Override — Ghi đè phương thức

Từ vựng trong mục: Override — ghi đè; **Inheritance** — kế thừa.

Lớp cha `Shipping` có phí mặc định 0 đồng, nhưng giao hàng nhanh cần phí 35.000 đồng. Lớp con `Express` **ghi đè (override)** phương thức `fee()` bằng định nghĩa mới cùng tên. Khi gọi `Express().fee()`, Python dùng bản của `Express`, không dùng bản của `Shipping`. Đây là một cách để đa hình xuất hiện thông qua quan hệ kế thừa.

```
class Shipping:
    def fee(self):
        return 0

class Express(Shipping):
    def fee(self):
        return 35000

print(Shipping().fee(), Express().fee())
```

Kết quả khi chạy:

```
0 35000
```

Hai đối tượng gọi cùng `fee()` nhưng phí khác nhau: gói thường 0 đồng, gói nhanh 35.000 đồng.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Các loại sản phẩm

Từ vựng trong mục: Inheritance — kế thừa; **Method** — phương thức.

Tình huống thực tế: Cửa hàng có sản phẩm điện tử `Phone`. Hệ thống dùng lớp `Product` để lưu tên mọi sản phẩm và lớp `Electronic` để xác định nhóm hàng. Hãy in nhóm sản phẩm của chiếc điện thoại.

Vì sao chọn cách giải này? Lớp con tái sử dụng phương thức khởi tạo tên từ lớp cha và thêm phương thức phân nhóm.

Quy trình giải:

1. Định nghĩa `Product` lưu tên trong `self.name`.
2. Tạo `Electronic(Product)` với phương thức `category()`.
3. Tạo sản phẩm `Phone`, gọi phương thức phân nhóm và in kết quả.

```
class Product:
    def __init__(self, name):
        self.name = name

class Electronic(Product):
    def category(self):
        return 'electronic'

print(Electronic('Phone').category())
```

Kết quả khi chạy:

```
electronic
```

Điện thoại được xếp vào nhóm `electronic`; tên `Phone` được giữ qua phần khởi tạo của lớp cha.

Tình huống 2 — Giao diện gửi thông báo

Từ vựng trong mục: Abstract class — lớp trừu tượng; **Abstract method** — phương thức trừu tượng.

Tình huống thực tế: Ứng dụng yêu cầu mọi bộ gửi thông báo phải có phương thức `send()`. Bộ gửi ra màn hình (`Console`) nhận nội dung `Hello`. Hãy dùng lớp trừu tượng để quy định giao diện và in thông điệp đã gửi.

Vì sao chọn cách giải này? `ABC` cùng `@abstractmethod` buộc lớp con triển khai phương thức, giúp tránh lớp gửi thông báo thiếu chức năng.

Quy trình giải:

1. Định nghĩa `Notifier(ABC)` với phương thức trừu tượng `send()`.
2. Cài đặt `send()` trong `Console` để tạo nội dung trả về.
3. Gọi `Console().send('Hello')` và in kết quả.

```
from abc import ABC, abstractmethod

class Notifier(ABC):
    @abstractmethod
```

```
    def send(self, message):
        pass

class Console(Notifier):
    def send(self, message):
        return 'Sent:' + message

print(Console().send('Hello'))
```

Kết quả khi chạy:

```
Sent:Hello
```

Phương thức `send()` của `Console` đã được định nghĩa; chương trình trả về chuỗi `Sent:Hello`.

Tình huống 3 — Tính phí vận chuyển

Từ vựng trong mục: Override — ghi đè; **Inheritance** — kế thừa.

Tình huống thực tế: Một cửa hàng có dịch vụ vận chuyển thường miễn phí và giao nhanh tính 35.000 đồng mỗi đơn. Hãy in phí giao hàng nhanh bằng phương thức của lớp con.

Vì sao chọn cách giải này? Ghi đè `fee()` cho loại giao hàng nhanh mà vẫn giữ giao diện chung `fee()` của lớp cha.

Quy trình giải:

1. Lớp `Shipping` định nghĩa phí mặc định bằng 0.
2. Lớp `Express` kế thừa và ghi đè phí bằng 35000.
3. Tạo `Express`, gọi `fee()` và in số tiền.

```
class Shipping:
    def fee(self):
        return 0

class Express(Shipping):
    def fee(self):
        return 35000

print(Express().fee())
```

Kết quả khi chạy:

```
35000
```

Giá trị 35000 tương ứng 35.000 đồng phí giao nhanh; không dùng giá trị 0 của lớp cha.

Tình huống 4 — Đọc nhiều loại cảm biến

Từ vựng trong mục: Polymorphism — đa hình; **Override** — ghi đè.

Tình huống thực tế: Hệ thống có một cảm biến nhiệt độ đo 26,5 °C và một cảm biến độ ẩm đo 62%. Cả hai phải có phương thức `read()`. Hãy đọc lần lượt hai cảm biến mà không cần kiểm tra loại trong vòng lặp.

Vì sao chọn cách giải này? Hai lớp con ghi đè `read()`, cho phép vòng lặp gọi cùng phương thức trên cả hai đối tượng.

Quy trình giải:

1. Tạo lớp chung `Sensor` có `read()`.
2. Hai lớp `Temp` và `Humidity` cài đặt kết quả đọc riêng.
3. Duyệt hai đối tượng và in kết quả từng lời gọi `read()`.

```
class Sensor:
    def read(self):
        return 0

class Temp(Sensor):
    def read(self):
        return 26.5

class Humidity(Sensor):
    def read(self):
        return 62

for sensor in [Temp(), Humidity()]:
    print(sensor.read())
```

Kết quả khi chạy:

```
26.5
62
```

Dòng đầu là 26,5 °C; dòng thứ hai là 62% độ ẩm. Chương trình chỉ in số, đơn vị được giải thích trong bài.

Tình huống 5 — Chuyển định dạng báo cáo

Từ vựng trong mục: Override — ghi đè; **Method** — phương thức.

Tình huống thực tế: Hệ thống có lớp báo cáo thường trả chữ `plain`. Báo cáo HTML cần tạo chuỗi `<p>report</p>`. Hãy in nội dung báo cáo HTML bằng cùng phương thức `render()`.

Vì sao chọn cách giải này? Lớp `HtmlReport` ghi đè `render()` của lớp cha để trả định dạng HTML.

Quy trình giải:

1. Lớp cha `Report` định nghĩa `render()` cho văn bản thường.
2. Lớp con `HtmlReport` định nghĩa lại `render()` cho HTML.
3. Tạo `HtmlReport` và in kết quả.

```
class Report:
    def render(self):
        return 'plain'

class HtmlReport(Report):
    def render(self):
        return '<p>report</p>'

print(HtmlReport().render())
```

Kết quả khi chạy:

```
<p>report</p>
```

Đầu ra là chuỗi thẻ HTML; `print()` không biến chuỗi thành trang web.

6. Sai lầm thường gặp

- **Kế thừa** không tự biến mọi phương thức thành đa hình; lớp con có thể dùng lại hoặc ghi đè tùy nhu cầu.
- Kế thừa `ABC` **chưa đủ** để buộc lớp con triển khai phương thức; cần thêm `@abstractmethod`.
- `super().__init__(...)` dùng logic khởi tạo của lớp cha; nếu bỏ khi cần, thuộc tính chung như `name` có thể chưa được tạo.
- Ghi đè phương thức phải dùng đúng tên và cách nhận tham số phù hợp; khác tên sẽ là phương thức mới.
- Ví dụ cảm biến in hai giá trị có **đơn vị khác nhau**: không được cộng trực tiếp 26,5 °C và 62%.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does class inheritance primarily enable? — Kế thừa lớp chủ yếu giúp gì?

A. Convert a class into a file — Chuyển lớp thành tệp

B. Reuse and extend behavior of a parent class — Dùng lại và mở rộng hành vi của lớp cha

C. Copy every object in memory — Sao chép mọi đối tượng trong bộ nhớ

D. Prevent all method calls — Ngăn mọi lời gọi phương thức

Đáp án: B

Giải thích: Kế thừa giúp lớp con dùng lại phương thức của lớp cha và bổ sung hành vi riêng. Ví dụ `Dog(Animal)` có thể kế thừa `eat()` rồi thêm `bark()`; kế thừa không sao chép mọi đối tượng đã tạo.

Q2. What does `class Dog(Animal):` express? — `class Dog(Animal):` thể hiện điều gì?

A. Dog inherits from Animal — Dog kế thừa Animal

B. Dog is a function inside Animal — Dog là hàm trong Animal

C. Animal inherits from Dog — Animal kế thừa Dog

D. Dog is an instance of Animal — Dog là đối tượng tạo từ Animal

Đáp án: A

Giải thích: Trong `class Dog(Animal):`, `Dog` là lớp con còn `Animal` là lớp cơ sở. Cú pháp khai báo quan hệ kế thừa, không tạo đối tượng `Dog` mới hay một hàm bên trong `Animal`.

Q3. Why call `super().__init__(name)` in a subclass? — Vì sao lớp con gọi `super().__init__(name)`?

A. To delete the parent class — Để xóa lớp cha

B. To replace every attribute with None — Để thay mọi thuộc tính bằng None

C. To make the child an abstract class — Để biến lớp con thành lớp trừu tượng

D. To initialize inherited parent state — Để khởi tạo dữ liệu chung từ lớp cha

Đáp án: D

Giải thích: `super().__init__(name)` chuyển việc khởi tạo cho lớp kế tiếp trong thứ tự phân giải phương thức, thường là lớp cha. Nếu hàm đó gán `self.name`, lớp con dùng được thuộc tính ấy mà không phải viết lặp phần khởi tạo.

Q4. Which statement best describes polymorphism? — Câu nào mô tả đúng đa hình?

A. A child class cannot override methods — Lớp con không thể ghi đè phương thức

B. All methods must have different names — Mọi phương thức phải có tên khác nhau

C. Different objects respond to the same method call — Các đối tượng khác nhau xử lý cùng một lời gọi phương thức

D. All objects must store identical values — Mọi đối tượng phải có giá trị giống nhau

Đáp án: C

Giải thích: Đa hình cho phép mã gọi cùng một thao tác trên nhiều đối tượng tương thích. Chẳng hạn `dog.sound()` và `cat.sound()` có thể cho âm thanh khác nhau dù đều gọi `sound()`.

Q5. What is method overriding? — Ghi đè phương thức là gì?

A. Removing every inherited attribute — Xóa mọi thuộc tính kế thừa

B. Calling the parent method unchanged — Gọi nguyên phương thức lớp cha

C. Defining a same-named method in a subclass — Định nghĩa lại phương thức cùng tên ở lớp con

D. Changing a local variable only — Chỉ thay đổi một biến cục bộ

Đáp án: C

Giải thích: Ghi đè xảy ra khi lớp con định nghĩa phương thức trùng tên với phương thức kế thừa. Gọi trên đối tượng lớp con thường chọn bản của lớp con, nhưng phương thức gốc của lớp cha vẫn tồn tại.

Q6. What does `@abstractmethod` mark? — `@abstractmethod` đánh dấu điều gì?

A. A property unique to integers — Thuộc tính riêng của số nguyên

B. A method that can never be called — Phương thức không bao giờ được gọi

C. A method that prints automatically — Phương thức tự động in

D. A method subclasses must implement to become concrete — Phương thức lớp con phải triển khai để có thể khởi tạo

Đáp án: D

Giải thích: Khi lớp dùng cơ chế ABC của Python, `@abstractmethod` đánh dấu yêu cầu mà lớp con cụ thể phải đáp ứng. Phương thức trừu tượng vẫn có thể có phần thân; hạn chế nằm ở việc tạo đối tượng từ lớp còn yêu cầu trừu tượng chưa hoàn thành.

Q7. Is inheriting from `ABC` alone enough to require an override? — Chỉ kế thừa `ABC` có buộc ghi đè phương thức không?

A. Yes, every method becomes abstract — Có, mọi phương thức đều trừu tượng

B. No, an abstract method must be declared — Không, cần khai báo phương thức trừu tượng

C. Yes, all subclass methods are deleted — Có, mọi phương thức lớp con bị xóa

D. No, ABC forbids inheritance — Không, ABC cấm kế thừa

Đáp án: B

Giải thích: Kế thừa `ABC` bật cơ chế lớp trừu tượng nhưng không tự khiến mọi phương thức thành trừu tượng. Khai báo `@abstractmethod` phía trên `def area(self):` mới đặt ra yêu cầu trừu tượng cụ thể.

Q8. What happens if an abstract method is left unimplemented? — Điều gì xảy ra nếu phương thức trừu tượng chưa được cài đặt?

A. The concrete instance cannot be created — Không thể tạo đối tượng từ lớp còn phương thức trừu tượng

B. The method automatically sends an email — Phương thức tự gửi email

C. The base class is erased — Lớp cha bị xóa

D. Python silently returns zero — Python âm thầm trả về 0

Đáp án: A

Giải thích: Nếu lớp con vẫn còn phương thức trừu tượng, Python từ chối tạo instance và báo `TypeError`. Cài đặt đầy đủ các phương thức bắt buộc mới giúp lớp có thể khởi tạo; chỉ kế thừa tên phương thức là chưa đủ.

Q9. What does `isinstance(obj, Parent)` check? — `isinstance(obj, Parent)` kiểm tra gì?

A. Whether obj is an instance of Parent or its subclasses — obj có thuộc Parent hoặc lớp con hay không

B. Whether obj is a string — obj có phải chuỗi không

C. Whether Parent has no methods — Parent có rỗng phương thức không

D. Whether obj has been printed — obj đã được in hay chưa

Đáp án: A

Giải thích: `isinstance(obj, Parent)` kiểm tra đối tượng thuộc `Parent` hoặc lớp con tương thích và trả Boolean. Với `Child(Parent)` thông thường, `isinstance(Child(), Parent)` là `True`, khác phép kiểm tra chính xác `type(obj) is Parent`.

Q10. If a child overrides `fee()`, which version runs on `Child().fee()`? — Nếu lớp con ghi đè `fee()`, `Child().fee()` chạy bản nào?

A. The parent's version in every case — Luôn bản của lớp cha

B. Neither version — Không chạy bản nào

C. The child's version — Bản của lớp con

D. Both versions automatically — Tự động chạy cả hai bản

Đáp án: C

Giải thích: Tra cứu phương thức trên instance ưu tiên bản ghi đè ở `Child` trước phương thức cùng tên được kế thừa. Vì vậy `Child().fee()` chạy bản lớp con; bản lớp cha chỉ chạy thêm nếu phương thức ghi đè chủ động gọi nó.

Q11. Can a child access an inherited attribute set in the parent initializer? — Lớp con có thể dùng thuộc tính khởi tạo ở lớp cha không?

A. Only if the attribute is named `class` — Chỉ khi tên thuộc tính là class

B. Yes, if the parent initialization is performed — Có, nếu đã thực hiện khởi tạo lớp cha

C. No, inherited fields are always hidden — Không, thuộc tính kế thừa luôn bị ẩn

D. Only for integer attributes — Chỉ với thuộc tính số nguyên

Đáp án: B

Giải thích: Lớp con đọc được thuộc tính instance đã được phần khởi tạo thích hợp tạo ra. Nếu `Parent.__init__` gán `self.name`, lớp con gọi `super().__init__(name)` sẽ dùng được `self.name`; bỏ bước cần thiết có thể khiến thuộc tính chưa tồn tại.

Q12. Must polymorphic objects share exactly the same class? — Đối tượng đa hình có bắt buộc cùng một class không?

A. Yes, they must be identical objects — Có, phải là cùng một đối tượng

B. Yes, inheritance is forbidden — Có, không được kế thừa

C. No, they can provide the same expected method — Không, có thể thuộc các lớp khác nhau nhưng cùng phương thức

D. No, methods must all be absent — Không, không được có phương thức

Đáp án: C

Giải thích: Đa hình dựa vào giao diện thao tác chung, không yêu cầu các đối tượng cùng lớp cụ thể. Hàm gọi `obj.sound()` có thể làm việc với `Dog` và `Cat` nếu cả hai cung cấp phương thức đó, dù cách triển khai khác nhau.

Q13. What does `self` refer to inside an instance method? — `self` trong phương thức đối tượng chỉ gì?

A. Every object in the program — Toàn bộ đối tượng của chương trình

B. The current object receiving the call — Đối tượng hiện tại đang nhận lời gọi

C. Only the parent class definition — Chỉ phần định nghĩa lớp cha

D. The name of the Python module — Tên module Python

Đáp án: B

Giải thích: Trong phương thức instance thông thường, `self` là đối tượng cụ thể đang nhận lời gọi. Vì vậy `self.name` của `dog1` có thể khác `self.name` của `dog2`; `self` không phải lớp cha hay module.

Q14. Does overriding a method erase it from the parent class? — Ghi đè phương thức có xóa bản lớp cha không?

A. Yes, it deletes all parent code — Có, xóa toàn bộ code lớp cha

B. Yes, it deletes the class name — Có, xóa tên lớp

C. Only when `print()` is used — Chỉ khi dùng print

D. No, the parent method still exists — Không, phương thức của lớp cha vẫn tồn tại

Đáp án: D

Giải thích: Ghi đè ở lớp con thay đổi phương thức được dùng bởi đối tượng lớp con chứ không xóa định nghĩa trên `Parent`. `Parent().fee()` vẫn dùng bản của cha, còn `Child().fee()` có thể dùng bản ghi đè.

Q15. How can an override call the parent's implementation? — Phương thức ghi đè gọi bản của lớp cha bằng gì?

A. `super().method()` — `super().method()`

B. `None.method()` — `None.method()`

C. `print(super)` only — Chỉ `print(super)`

D. `del Parent` — `del Parent`

Đáp án: A

Giải thích: Trong phương thức ghi đè, `super().method()` chuyển lời gọi tới bản kế tiếp trong thứ tự phân giải phương thức, thường là lớp cha. Nó không gọi tùy ý một lớp bất kỳ; với đa kế thừa, lớp kế tiếp có thể khác lớp cha trực tiếp.

Q16. Can an abstract class define ordinary implemented methods? — Lớp trừu tượng có thể có phương thức thông thường đã triển khai không?

A. No, it cannot define attributes — Không, không thể có thuộc tính

B. No, it must only contain pass — Không, chỉ được chứa pass

C. Only when it has no subclasses — Chỉ khi không có lớp con

D. Yes, it can provide reusable behavior too — Có, nó vẫn có thể cung cấp hành vi dùng chung

Đáp án: D

Giải thích: Lớp trừu tượng có thể vừa có hàm hỗ trợ đã cài đặt vừa có yêu cầu trừu tượng. Chẳng hạn lớp cung cấp `describe()` nhưng buộc lớp con triển khai `area()`; trừu tượng không có nghĩa mọi phương thức phải rỗng.

Q17. What is the relation between overriding and polymorphism? — Ghi đè liên quan gì đến đa hình?

A. They both mean deleting attributes — Cả hai đều là xóa thuộc tính

B. Overriding can provide distinct behavior for a common call — Ghi đè tạo hành vi khác nhau cho cùng lời gọi

C. They only apply to CSV files — Chúng chỉ áp dụng cho CSV

D. Overriding prevents polymorphism — Ghi đè ngăn đa hình

Đáp án: B

Giải thích: Ghi đè là một cách thực hiện đa hình: các lớp con cung cấp thân hàm khác nhau cho cùng tên phương thức. Mã có thể gọi `shape.area()` mà không cần tự chọn hàm riêng cho từng loại hình.

Q18. Which part of `class Child(Parent):` names the superclass? — Trong `class Child(Parent):`, đâu là tên lớp cha?

A. `class` — `class`

B. `Child` — `Child`

C. `Parent` — `Parent`

D. `:` — `:`

Đáp án: C

Giải thích: Trong dòng `class Child(Parent):`, tên bên trong ngoặc là lớp cơ sở `Parent`. Lớp mới đang được khai báo là `Child`; dấu hai chấm mở đầu phần thân lớp được thụt lề.

Q19. What is a good reason to create a shared base class? — Khi nào nên tạo lớp cha chung?

A. To avoid using any methods — Để không cần phương thức

B. Every object is unrelated — Mọi đối tượng hoàn toàn không liên quan

C. To make all values constant — Để mọi giá trị là hằng số

D. Several subclasses share data or behavior — Nhiều lớp con có chung dữ liệu hoặc hành vi

Đáp án: D

Giải thích: Lớp cha chung hữu ích khi nhiều lớp con có trạng thái hoặc hành vi giống nhau. Ví dụ `Car` và `Bike` có thể dùng chung giao diện `Vehicle` thay vì lặp định nghĩa cùng hợp đồng phương thức ở từng lớp.

Q20. Which error may follow if a child skips necessary parent initialization? — Nếu lớp con bỏ khởi tạo lớp cha cần thiết, điều gì có thể xảy ra?

A. An expected inherited instance attribute may be missing — Một thuộc tính đối tượng cần thiết có thể chưa được tạo

B. All integers turn into strings — Mọi số nguyên biến thành chuỗi

C. Python automatically runs CSV conversion — Python tự chuyển CSV

D. Every method prints twice — Mọi phương thức in hai lần

Đáp án: A

Giải thích: Nếu lớp con ghi đè `__init__` mà bỏ phần khởi tạo cần thiết của cha, các trường dự kiến có thể chưa được tạo. Đọc `self.name` sau đó có thể gây `AttributeError`; Python không tự gọi hàm khởi tạo cha đã bị ghi đè.

8. Nối sang bài tiếp theo

Class/Object → Inheritance → Abstract API → Kiến trúc ứng dụng.

PY-12 — File I/O, CSV và xử lý ngoại lệ

DRAFT · Tài liệu tự học · Các mã và dữ liệu mẫu cần được kiểm tra trước khi công bố chính thức

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: File I/O — đọc ghi file; **CSV** — dữ liệu phân cách.

`Dictionary → File/CSV → DataFrame → Dataset AI`

Học xong dùng để làm gì? Đọc CSV, ghi kết quả, xử lý lỗi chuyển kiểu và đóng file an toàn.

Kết nối kiến thức tiếp theo: PY-13 đưa bảng số liệu vào mảng NumPy để tính theo hàng/cột.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
File I/O	đọc ghi file	Lấy hoặc lưu dữ liệu trên đĩa
CSV	dữ liệu phân cách	Định dạng bảng dạng văn bản
Exception	ngoại lệ	Lỗi xuất hiện khi chạy chương trình
Context manager	quản lý ngữ cảnh	Tự đóng tài nguyên sau khối with

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`open(...,encoding='utf-8')`	Đọc/ghi Unicode và tự đóng bằng with
`csv.DictReader/DictWriter`	Đọc và ghi CSV theo tên cột thay vì vị trí
`try/except ValueError`	Bắt riêng lỗi dữ liệu nhập không hợp lệ
`Path.read_text()/write_text()`	Đọc/ghi file với đường dẫn kiểu Path

4. Khái niệm liên quan

Một chương trình thực tế cần **lưu và đọc lại dữ liệu**, hiểu cấu trúc bảng CSV và xử lý đúng lỗi đầu vào. Bốn khái niệm File I/O, CSV, Exception và Context manager liên kết thành một quy trình: nhận dữ liệu → đọc/ghi an toàn → xử lý trường hợp bất thường.

4.1. File I/O — Đọc và ghi dữ liệu

Từ vựng trong mục: File I/O — đọc/ghi tệp; **Path** — đường dẫn.

File I/O (input/output) là việc đưa dữ liệu **từ tệp vào chương trình** hoặc **từ chương trình ra tệp**. Ví dụ hệ thống cần lưu trạng thái `Started` để lần chạy sau có thể đọc lại. Trong đoạn minh họa, `TemporaryDirectory()` tạo thư mục tạm, `Path` xác định đường dẫn `log.txt`, `write_text()` ghi và `read_text()` đọc. Dữ liệu chỉ tồn tại trong thư mục tạm trong suốt khối `with`, tránh để lại file rác.

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    file_path = Path(folder) / 'log.txt'
    file_path.write_text('Started', encoding='utf-8')
    print(file_path.read_text(encoding='utf-8'))
```

Kết quả khi chạy:

```
Started
```

Đầu ra đúng chuỗi đã ghi. `encoding='utf-8'` hỗ trợ văn bản tiếng Việt; tệp tạm được xóa sau khi ra khỏi khối.

4.2. CSV — Bảng dữ liệu phân cách bằng dấu phẩy

Từ vựng trong mục: CSV — dữ liệu phân cách; **Row/column** — hàng/cột.

CSV lưu dữ liệu dạng văn bản: **mỗi dòng là một bản ghi**, các giá trị trên một dòng được tách bằng dấu phẩy. Ví dụ bảng sản phẩm có **hai cột** `name` (tên hàng) và `price` (giá) cùng **hai hàng** Pen/10 và Book/35. `csv.DictReader` coi hàng đầu là tên cột và đọc mỗi hàng sau thành Dictionary, giúp tra `row['name']` thay vì nhớ vị trí cột. Giá đọc từ CSV ban đầu là **chuỗi**, muốn tính toán phải chuyển kiểu sau.

```
import csv
from io import StringIO

csv_text = 'name,price\nPen,10\nBook,35\n'
rows = list(csv.DictReader(StringIO(csv_text)))
print(rows[1]['name'], rows[1]['price'])
```

Kết quả khi chạy:

```
Book 35
```

Hàng dữ liệu thứ hai là Book, giá `35` đang ở dạng chuỗi; mã không tự đổi nó thành số nguyên.

4.3. Exception — Lỗi có thể xử lý

Từ vựng trong mục: Exception — ngoại lệ; **ValueError** — lỗi giá trị.

Khi chương trình nhận dữ liệu sai kiểu, không phải toàn bộ ứng dụng đều cần dừng. Chẳng hạn biểu mẫu yêu cầu số nguyên nhưng khách nhập `'abc'`; `int('abc')` sinh **`ValueError`**. Khối `try` chứa thao tác có thể lỗi, còn `except ValueError` chỉ bắt loại lỗi dự kiến và in thông báo. Không dùng `except:` quá rộng vì sẽ che mất những lỗi khác cần được sửa.

```
try:
    number = int('abc')
```

```
    print(number)
except ValueError:
    print('Invalid integer')
```

Kết quả khi chạy:

```
Invalid integer
```

Chỉ nhánh `except` chạy, vì chuỗi `abc` không thể chuyển thành số nguyên.

4.4. Context manager — Tự quản lý tài nguyên

Từ vựng trong mục: Context manager — quản lý ngữ cảnh; **with** — khối sử dụng tài nguyên.

Khi mở file hoặc bộ đệm, ta cần **đóng tài nguyên** sau khi dùng, kể cả khi có lỗi. Cấu trúc `with ... as ...:` nhận một tài nguyên rồi tự đóng khi thoát khối. Ví dụ `StringIO` là bộ đệm văn bản trong bộ nhớ, có thao tác đọc giống file, không cần tạo file thật. Ta có thể kiểm tra `stream.closed` sau khối `with` để thấy nó đã đóng.

```
from io import StringIO

with StringIO('abc') as stream:
    print(stream.read())

print(stream.closed)
```

Kết quả khi chạy:

```
abc
True
```

Dòng đầu là nội dung đã đọc; `True` xác nhận bộ đệm được đóng sau khi ra khỏi `with`.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Đọc CSV sản phẩm

Từ vựng trong mục: CSV — dữ liệu CSV; **DictReader** — đọc theo tên cột.

Tình huống thực tế: Cửa hàng có bảng `name,price` gồm Pen giá 10 và Book giá 35 theo cùng đơn vị tiền. Hãy đọc CSV và in tên mặt hàng nằm ở **hàng dữ liệu thứ hai**.

Vì sao chọn cách giải này? `csv.DictReader` đọc theo tên cột và `rows[1]` lấy hàng dữ liệu thứ hai (vì chỉ số bắt đầu từ 0).

Quy trình giải:

1. Viết dữ liệu CSV với hàng tên cột và hai sản phẩm.
2. Đọc bằng `DictReader`, chuyển các hàng thành List.
3. Lấy `rows[1]['name']` và in ra.

```
import csv
from io import StringIO
```

```
csv_text = 'name,price\nPen,10\nBook,35\n'
rows = list(csv.DictReader(StringIO(csv_text)))
print(rows[1]['name'])
```

Kết quả khi chạy:

```
Book
```

`Book` thuộc hàng dữ liệu thứ hai; hàng tiêu đề không được tính là sản phẩm.

Tình huống 2 — Lưu nhật ký ứng dụng

Từ vựng trong mục: File I/O — đọc ghi file; **UTF-8** — mã hóa văn bản.

Tình huống thực tế: Ứng dụng ghi trạng thái `Started` vào file nhật ký `log.txt` trong một thư mục tạm. Hãy đọc lại dòng trạng thái để xác minh đã lưu thành công.

Vì sao chọn cách giải này? `Path.write_text()` ghi văn bản UTF-8; `Path.read_text()` đọc lại, còn thư mục tạm bảo đảm ví dụ không để lại file.

Quy trình giải:

1. Tạo thư mục tạm và đường dẫn `log.txt`.
2. Ghi chữ `Started` vào file với UTF-8.
3. Đọc file và in trạng thái đã lưu.

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    file_path = Path(folder) / 'log.txt'
    file_path.write_text('Started', encoding='utf-8')
    print(file_path.read_text(encoding='utf-8'))
```

Kết quả khi chạy:

```
Started
```

Nội dung đọc trùng dữ liệu đã ghi; file ví dụ được xóa khi thư mục tạm đóng.

Tình huống 3 — Chặn dữ liệu nhập sai kiểu

Từ vựng trong mục: Exception — ngoại lệ; **ValueError** — lỗi giá trị.

Tình huống thực tế: Biểu mẫu chỉ nhận số nguyên nhưng khách nhập chuỗi `abc`. Hãy thử chuyển kiểu; nếu chuyển thất bại, in `None` để biểu thị chưa có số hợp lệ mà không làm chương trình dừng.

Vì sao chọn cách giải này? `int()` báo `ValueError` với chữ không phải số nguyên. `except ValueError` bắt lỗi này để gán `None` đúng trường hợp.

Quy trình giải:

1. Đặt `raw = 'abc'`.

2. Trong `try`, gọi `int(raw)` để chuyển số.
3. Nếu có `ValueError`, đặt số bằng `None`, rồi in kết quả.

```
raw = 'abc'

try:
    number = int(raw)
except ValueError:
    number = None

print(number)
```

Kết quả khi chạy:

```
None
```

`None` biểu thị chưa có giá trị số hợp lệ; nó không phải số 0.

Tình huống 4 — Xuất báo cáo CSV

Từ vựng trong mục: DictWriter — ghi CSV theo tên cột; **Fieldnames** — tên cột.

Tình huống thực tế: Báo cáo bán hàng có ngày `2026-10-24` và doanh thu `1200` đơn vị tiền. Hãy tạo dữ liệu CSV có hai cột `date`, `revenue` và một dòng dữ liệu, rồi in nội dung file dạng văn bản.

Vì sao chọn cách giải này? `csv.DictWriter` bảo đảm giá trị được ghi đúng cột. `lineterminator='\n'` tạo dấu xuống dòng thống nhất trong ví dụ; `end=''` tránh `print()` thêm dòng trống.

Quy trình giải:

1. Tạo bộ đệm văn bản `StringIO`.
2. Định nghĩa tên hai cột và dùng `writeheader()` ghi tiêu đề.
3. Ghi bản ghi doanh thu rồi in nguyên nội dung bộ đệm.

```
import csv
from io import StringIO

buffer = StringIO()
writer = csv.DictWriter(
    buffer, fieldnames=['date', 'revenue'], lineterminator='\n'
)
writer.writeheader()
writer.writerow({'date': '2026-10-24', 'revenue': 1200})
print(buffer.getvalue(), end='')
```

Kết quả khi chạy:

```
date,revenue
2026-10-24,1200
```

Dòng đầu chứa tên cột, dòng sau chứa giá trị. `revenue` ghi trong CSV là chữ biểu diễn số, không còn kiểu `int` khi đọc lại.

Tình huống 5 — Báo lỗi khi thiếu file

Từ vựng trong mục: FileNotFoundError — lỗi không tìm thấy file; **Path** — đường dẫn.

Tình huống thực tế: Một dịch vụ cần đọc `missing.csv` nhưng tệp này chưa được tạo. Hãy in thông báo `File unavailable` thay vì để chương trình dừng vì lỗi thiếu file.

Vì sao chọn cách giải này? Dùng `FileNotFoundError` để xử lý riêng tình huống file không tồn tại; thư mục tạm đảm bảo tên tệp này chưa được tạo trong ví dụ.

Quy trình giải:

1. Tạo thư mục tạm và đường dẫn tới `missing.csv` chưa tồn tại.
2. Thử gọi `read_text()` trong `try`.
3. Khi phát sinh `FileNotFoundError`, in thông báo.

```
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    missing = Path(folder) / 'missing.csv'
    try:
        text = missing.read_text(encoding='utf-8')
    except FileNotFoundError:
        text = 'File unavailable'
    print(text)
```

Kết quả khi chạy:

```
File unavailable
```

Tệp chưa tồn tại nên khối `except` xử lý lỗi một cách có kiểm soát.

6. Sai lầm thường gặp

- Khi đọc CSV bằng `DictReader`, dữ liệu **mặc định là chuỗi**; muốn cộng tiền cần chuyển kiểu số hợp lệ.
- Không nhầm **hàng tiêu đề CSV** với hàng dữ liệu đầu tiên. `rows[0]` là sản phẩm đầu trong ví dụ, không phải dòng `name,price`.
- Dùng `encoding='utf-8'` với dữ liệu tiếng Việt để tránh lỗi hiển thị do mã hóa.
- Nên dùng `with` khi làm việc với file để đóng tài nguyên ngay cả khi có ngoại lệ.
- Chỉ bắt lỗi dự kiến như `ValueError` hay `FileNotFoundError`; `except:` chung chung có thể che lỗi lập trình.
- `print(text)` sẽ tự thêm ký tự xuống dòng; nếu `text` đã kết thúc bằng `\n`, màn hình có thể thêm một dòng trống.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does File I/O mean? — File I/O có nghĩa gì?

A. Reading data from or writing data to files — Đọc dữ liệu từ hoặc ghi dữ liệu vào tệp

B. Drawing a plot — Vẽ một biểu đồ

C. Sorting a list in memory — Sắp xếp List trong bộ nhớ

D. Training a model — Huấn luyện mô hình

Đáp án: A

Giải thích: Nhập/xuất tệp là đọc dữ liệu từ tệp đã lưu hoặc ghi dữ liệu vào tệp. Ví dụ đọc CSV cung cấp các hàng đầu vào, còn lưu báo cáo tạo đầu ra tệp; chỉ sắp xếp trong bộ nhớ chưa phải thao tác I/O tệp.

Q2. What is the purpose of a file path? — Đường dẫn file dùng để làm gì?

A. Compute a matrix rank — Tính hạng ma trận

B. Sort rows automatically — Tự sắp xếp các hàng

C. Store a Boolean value — Lưu giá trị Boolean

D. Identify a file location — Xác định vị trí một tệp

Đáp án: D

Giải thích: Đường dẫn chỉ vị trí tệp hay thư mục, ví dụ `data/scores.csv`. Đường dẫn tương đối được hiểu theo thư mục làm việc, còn đường dẫn tuyệt đối xác định vị trí tính từ gốc hệ thống tệp.

Q3. What does `Path.read_text()` do? — `Path.read_text()` làm gì?

A. Convert the path into a function — Chuyển đường dẫn thành hàm

B. Sort CSV by price — Sắp CSV theo giá

C. Read text content from a file — Đọc nội dung văn bản từ tệp

D. Create a pandas index — Tạo chỉ số Pandas

Đáp án: C

Giải thích: `Path.read_text(encoding='utf-8')` đọc tệp văn bản và trả nội dung dưới dạng chuỗi Python. Nó không tự phân tách cột CSV hay biến đường dẫn thành hàm; giải mã có thể lỗi nếu chọn sai encoding.

Q4. What does `Path.write_text()` do? — `Path.write_text()` dùng để làm gì?

A. Read and sort all CSV rows — Đọc và sắp mọi hàng CSV

B. Only check whether a file exists — Chỉ kiểm tra tệp tồn tại

C. Count class methods — Đếm phương thức lớp

D. Write text to a file — Ghi văn bản vào tệp

Đáp án: D

Giải thích: `Path.write_text(text, encoding='utf-8')` ghi chuỗi vào tệp và thông thường thay nội dung trước đó. Muốn nối thêm, cần chế độ mở tệp phù hợp như `'a'`; `write_text` không phải thao tác chỉ kiểm tra tệp tồn tại.

Q5. Why specify `encoding='utf-8'` for text? — Vì sao dùng `encoding='utf-8'` với văn bản?

A. To encode and decode Unicode text consistently — Để mã hóa và giải mã chữ Unicode nhất quán

B. To guarantee a file is secret — Để bảo đảm tệp bí mật

C. To make all values integers — Để mọi giá trị thành số nguyên

D. To create three CSV columns — Để tạo ba cột CSV

Đáp án: A

Giải thích: UTF-8 quy định cách mã hóa ký tự Unicode thành byte khi ghi và giải mã byte thành ký tự khi đọc. Chỉ rõ UTF-8 nhất quán giúp giữ đúng chữ `Tiếng Việt`; nó không mã hóa bảo mật hoặc bảo đảm tệp bí mật.

Q6. How does a CSV file normally organize data? — CSV thông thường tổ chức dữ liệu ra sao?

A. Rows of text with separated fields — Các dòng văn bản chứa trường dữ liệu được phân cách

B. Binary images with pixels — Ảnh nhị phân gồm điểm ảnh

C. A database server connection — Kết nối máy chủ cơ sở dữ liệu

D. Python classes only — Chỉ chứa lớp Python

Đáp án: A

Giải thích: CSV lưu dữ liệu dạng bảng bằng các bản ghi có trường phân cách, thường có hàng đầu ghi tên cột. Mỗi hàng dữ liệu là một bản ghi, mỗi cột là một trường; văn bản trong dấu nháy có thể chứa dấu phẩy mà không tạo cột mới.

Q7. What does `csv.DictReader` use as dictionary keys? — `csv.DictReader` dùng gì làm khóa Dictionary?

A. CSV column names — Tên cột trong CSV

B. Names of Python packages — Tên gói Python

C. The current file size — Kích thước tệp hiện tại

D. Always numbers 0, 1, 2 — Luôn là số 0, 1, 2

Đáp án: A

Giải thích: Mặc định `csv.DictReader` lấy tên trường ở hàng CSV đầu tiên làm khóa từ điển. Với tiêu đề `name,age`, các hàng sau có khóa `'name'` và `'age'`; truyền `fieldnames=` tường minh sẽ thay cách lấy tiêu đề mặc định.

Q8. For a well-formed CSV row with all fields present, what type does `csv.DictReader` return for field values by default? — Với hàng CSV đủ các trường đúng cấu trúc, mặc định `csv.DictReader` trả giá trị ô thuộc kiểu gì?

A. String — Chuỗi

B. Float — Số thực

C. Integer — Số nguyên

D. Boolean — Giá trị logic

Đáp án: A

Giải thích: Với hàng CSV đúng cấu trúc, `csv.DictReader` trả dữ liệu ô dưới dạng chuỗi, nên ô `12` thành `'12'` chứ không phải số nguyên `12`. Cột bị thiếu có thể tạo `None` theo mặc định; muốn tính toán số vẫn phải đổi kiểu rõ ràng.

Q9. What does `csv.DictWriter.writeheader()` write? — `csv.DictWriter.writeheader()` ghi gì?

A. A Python class definition — Định nghĩa lớp Python

B. The column-name header row — Hàng tiêu đề chứa tên các cột

C. An exception message — Thông báo ngoại lệ

D. The final record twice — Nhân đôi bản ghi cuối

Đáp án: B

Giải thích: `csv.DictWriter.writeheader()` ghi một hàng chứa các tên trường đã cấu hình theo thứ tự. Nó không ghi toàn bộ bản ghi dữ liệu; các bản ghi cần dùng riêng `writerow()` hoặc `writerows()`.

Q10. Why prefer the CSV module over simply splitting every line by comma? — Vì sao nên dùng module CSV thay vì luôn tách chuỗi theo dấu phẩy?

A. It encrypts each file — Nó mã hóa bảo mật mọi tệp

B. It converts every field to a float — Nó đổi mọi trường thành số thực

C. It removes all empty rows automatically — Nó tự bỏ mọi dòng rỗng

D. It handles quoted fields and delimiters correctly — Nó xử lý đúng trường được đặt trong dấu nháy và dấu phân cách

Đáp án: D

Giải thích: Bộ đọc CSV hiểu trường có dấu nháy và quy tắc escape nên dấu phẩy trong ``"An, Binh"vẫn thuộc một trường. Tách máy móc bằngline.split(',')` sẽ chia sai giá trị có dấu nháy thành hai trường.

Q11. What is an exception in Python? — Ngoại lệ trong Python là gì?

A. A CSV row header — Hàng tiêu đề CSV

B. A guaranteed successful result — Một kết quả chắc chắn thành công

C. An error or abnormal condition raised during execution — Lỗi hoặc tình huống bất thường phát sinh khi chạy

D. A list slicing operator — Toán tử cắt List

Đáp án: C

Giải thích: Ngoại lệ báo tình huống bất thường khi chương trình chạy, như đổi kiểu thất bại hoặc thiếu tệp. Mã có thể bắt ngoại lệ dự kiến để xử lý; ngoại lệ không được bắt thường làm gián đoạn thao tác hiện tại.

Q12. What does `int('abc')` raise? — `int('abc')` phát sinh lỗi gì?

A. No error; returns zero — Không lỗi và trả về 0

B. `ValueError` — `ValueError`

C. `FileNotFoundError` — `FileNotFoundError`

D. `ZeroDivisionError` — `ZeroDivisionError`

Đáp án: B

Giải thích: `int('abc')` phát sinh `ValueError` vì các ký tự không biểu diễn số nguyên thập phân hợp lệ. Hàm `int` có tồn tại và nhận chuỗi, nên đây không phải lỗi thiếu tệp hoặc chia cho không.

Q13. Which exceptions does `except ValueError:` handle? — `except ValueError:` xử lý những ngoại lệ nào?

A. Every possible error — Mọi lỗi có thể xảy ra

B. `ValueError` exceptions, including invalid conversions — Ngoại lệ `ValueError`, kể cả lỗi đổi kiểu không hợp lệ

C. Only successful conversions — Chỉ khi chuyển kiểu thành công

D. All file permission failures — Mọi lỗi quyền truy cập tệp

Đáp án: B

Giải thích: `except ValueError:` bắt ngoại lệ kiểu `ValueError` và các kiểu con, trong đó có lỗi đổi số như `int('abc')`. Nó không bắt `FileNotFoundError` không liên quan hay mọi lỗi khác; vì vậy cần chọn nhánh bắt phù hợp với rủi ro của mã.

Q14. Which exception commonly indicates a missing file? — Ngoại lệ nào thường báo tệp không tồn tại?

A. `TypeError` always — Luôn `TypeError`

B. `ValueError` only — Chỉ `ValueError`

C. `KeyError` only — Chỉ `KeyError`

D. `FileNotFoundError` — `FileNotFoundError`

Đáp án: D

Giải thích: Mở một đường dẫn không tồn tại để đọc thường gây `FileNotFoundError`. Lỗi này khác `PermissionError`, khi tệp có thể tồn tại nhưng quyền truy cập bị từ chối; hai trường hợp cần cách xử lý khác nhau.

Q15. What does a `with` block provide for a file? — Khối `with` hỗ trợ việc gì khi dùng file?

A. Automatic numeric conversion — Tự chuyển toàn bộ số

B. Automatic sorting — Tự sắp xếp

C. Automatic resource cleanup on exit — Tự dọn dẹp và đóng tài nguyên khi ra khỏi khối

D. Automatic internet download — Tự tải từ Internet

Đáp án: C

Giải thích: Trình quản lý ngữ cảnh tệp như `with open(path) as f:` đóng tệp khi thoát khỏi khối, kể cả khi có ngoại lệ. Nó quản lý dọn dẹp tài nguyên, không tự sắp xếp dữ liệu hoặc đổi các ô thành số.

Q16. What is `StringIO` useful for? — `StringIO` hữu ích cho việc gì?

A. Reading only images — Chỉ đọc ảnh

B. Operating a camera sensor — Điều khiển cảm biến camera

C. File-like text operations in memory — Đọc ghi văn bản như file ngay trong bộ nhớ

D. Turning a set into a tuple — Biến Set thành Tuple

Đáp án: C

Giải thích: `io.StringIO` cung cấp luồng văn bản trong bộ nhớ với thao tác tương tự tệp như `read()` và `write()`. Nó hữu ích khi thử bộ đọc CSV với chuỗi mẫu nhỏ mà không phải tạo tệp vật lý.

Q17. What is the difference between `write` mode and append mode? — Chế độ ghi `w` và nối thêm `a` khác nhau thế nào?

A. Both only read existing content — Cả hai chỉ đọc nội dung

B. `w` replaces contents; `a` adds to the end — `w` thay nội dung; `a` nối thêm cuối tệp

C. `a` always deletes files — `a` luôn xóa tệp

D. `w` only opens network sockets — `w` chỉ mở socket mạng

Đáp án: B

Giải thích: Mở tệp có sẵn bằng chế độ `'w'` sẽ xóa nội dung cũ trước khi ghi, còn `'a'` nối dữ liệu mới vào cuối. Ví dụ tệp đang chứa `A`, ghi `B` bằng `'w'` được `B`, còn bằng `'a'` được `AB`.

Q18. Why is a bare `except:` often a poor choice? — Vì sao thường không nên dùng `except:` không chỉ rõ lỗi?

A. It prevents file creation — Nó ngăn tạo file

B. It never catches any error — Nó không bắt được lỗi nào

C. It can hide unexpected programming errors — Nó có thể che các lỗi lập trình không lường trước

D. It converts errors into CSV — Nó chuyển lỗi thành CSV

Đáp án: C

Giải thích: `except:` không ghi loại sẽ bắt phạm vi ngoại lệ quá rộng, kể cả các tín hiệu ngắt thường nên được chuyển tiếp. Nên dùng `except ValueError:` hoặc loại lỗi cụ thể để những bug không dự kiến vẫn lộ ra.

Q19. What does `print(text, end='')` change? — `print(text, end='')` thay đổi điều gì?

A. It closes every open file — Nó đóng mọi file đang mở

B. It removes existing newlines from text — Nó xóa mọi dấu xuống dòng có sẵn trong text

C. It converts text to numbers — Nó chuyển chữ thành số

D. It avoids adding the default newline — Nó không tự thêm dấu xuống dòng mặc định

Đáp án: D

Giải thích: Tham số `end` quy định chuỗi mà `print()` thêm vào sau các đối số; mặc định là xuống dòng `\n`. Với `end=''`, `print('A', end='')` không thêm dòng mới, nhưng cũng không xóa ký tự xuống dòng vốn có trong chuỗi.

Q20. Which distinction between `csv.reader` and `csv.DictReader` is correct? — `csv.reader` khác `csv.DictReader` ở điểm nào?

A. reader always validates all values — reader luôn kiểm tra mọi giá trị

B. reader yields field lists; DictReader uses column keys — reader trả danh sách trường; DictReader dùng khóa theo tên cột

C. Both always return float arrays — Cả hai luôn trả mảng số thực

D. DictReader writes files but cannot read — DictReader chỉ ghi mà không đọc

Đáp án: B

Giải thích: `csv.reader` trả mỗi bản ghi thành List chuỗi trường, ví dụ `['An', '9']`. `csv.DictReader` gắn trường với tên cột như `{'name': 'An', 'score': '9'}`, giúp tra theo tên thay vì chỉ số vị trí.

8. Nối sang bài tiếp theo

`Dictionary → File/CSV → DataFrame → Dataset AI.`

PY-13 — NumPy ndarray, shape, dtype và axis

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: ndarray — mảng NumPy; **Shape** — kích thước mảng.

`List → NumPy Array → Matrix → PCA`

Học xong dùng để làm gì? Biểu diễn dữ liệu số trong mảng có shape và dtype, tính tổng/trung bình theo chiều dữ liệu.

Kết nối kiến thức tiếp theo: PY-14 chọn hàng, cột hoặc phần tử thỏa ngưỡng trong mảng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
ndarray	mảng NumPy	Một mảng đồng nhất về kiểu phần tử
Shape	kích thước mảng	Số phần tử trên từng chiều
dtype	kiểu phần tử	Kiểu dữ liệu của giá trị trong mảng
Axis	trục tính toán	Chiều dọc theo phép tổng hợp

3. Thư viện, hàm và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm/cú pháp	Giải thích và lý do dùng
`np.array(data,dtype=float)`	Tạo mảng số thực, dtype=float cho phép lưu phần thập phân
`X.shape/X.ndim/X.size`	Xem hàng/cột, số chiều, số phần tử
`X.mean(axis=0)`	Tính trung bình của từng CỘT từ các hàng
`np.zeros()/np.ones()`	Tạo dữ liệu số được khởi tạo rõ hình dạng

4. Khái niệm liên quan

NumPy ndarray là công cụ lưu số liệu có cấu trúc. Một bảng có 3 học viên × 2 môn sẽ xuyên suốt phần khái niệm: từ tạo bảng, xem kích thước, hiểu kiểu phần tử cho tới tính toán theo cột. Nhờ giữ nguyên ý nghĩa hàng và cột, kết quả thống kê sẽ không bị hiểu nhầm.

4.1. ndarray — Mảng số cho nhiều học viên

Từ vựng trong mục: ndarray — mảng NumPy; **Element** — phần tử.

NumPy là thư viện hỗ trợ tính toán trên nhiều số cùng lúc. Kiểu dữ liệu chính là `ndarray` — một **mảng nhiều chiều**. Ví dụ trung tâm có ba học viên An, Bình, Lan (theo thứ tự ba hàng) và hai môn Toán, Python (theo thứ tự hai cột). Các điểm `[8,9]`, `[6,7]`, `[9,8]` nằm cùng một bảng. **Mỗi hàng** là một người, **mỗi cột** là một môn, **mỗi ô** là điểm của người đó ở môn đó trên thang 10.

```
import numpy as np

scores = np.array([
    [8, 9],
    [6, 7],
    [9, 8],
])
print(scores)
```

Kết quả khi chạy:

```
[[8 9]
 [6 7]
 [9 8]]
```

Hàng đầu là điểm của An, hàng thứ hai của Bình, hàng thứ ba của Lan; hai số trên một hàng lần lượt thuộc hai môn.

4.2. Shape — Kích thước của mảng

Từ vựng trong mục: Shape — kích thước; **Row/column** — hàng/cột.

Thuộc tính `shape` cho biết **số phần tử ở từng chiều** của mảng. Với bảng điểm ba học viên, hai môn, `shape == (3, 2)`: con số 3 là **số hàng/học viên**, con số 2 là **số cột/môn**. Thuộc tính `size` là số ô dữ liệu (3 × 2 = 6), còn `ndim` là số chiều (2). Không nhầm mảng `(3,2)` với ba môn của hai học viên.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]])
print(scores.shape, scores.size)
```

Kết quả khi chạy:

```
(3, 2) 6
```

Bảng có 3 người × 2 môn = 6 giá trị điểm; đây là mảng hai chiều.

4.3. dtype — Kiểu của các phần tử

Từ vựng trong mục: dtype — kiểu dữ liệu phần tử; **Float** — số thực.

Một `ndarray` thông thường lưu các phần tử bằng một kiểu dữ liệu chung (`dtype`). Với điểm có phần thập phân như 8,5, ta dùng `dtype=float` khi tạo mảng để giữ phần lẻ và thuận tiện tính trung bình. NumPy trên môi trường thông dụng dùng `float64` cho lựa chọn `float`. Ngược lại, nếu ép điểm thập phân thành số nguyên, phần lẻ có thể mất.

```
import numpy as np

```

```
scores = np.array([8.5, 7.0], dtype=float)
print(scores.dtype, scores.size)
```

Kết quả khi chạy:

```
float64 2
```

Mảng có hai điểm dạng số thực; `8.5` được giữ chứ không bị cắt thành `8`.

4.4. Axis — Tính theo hàng hoặc cột

Từ vựng trong mục: Axis — trục tính toán; **Mean** — trung bình.

Khi tổng hợp dữ liệu hai chiều, **`axis=0` gom các hàng để cho ra một kết quả trên mỗi cột**; **`axis=1` gom các cột để cho ra một kết quả trên mỗi hàng**. Trong bảng ba học viên × hai môn, `scores.mean(axis=0)` tính điểm trung bình **từng môn**: môn Toán `(8+6+9)/3 ≈ 7,67`, môn Python `(9+7+8)/3 = 8`. Nếu đổi sang `axis=1`, sẽ ra ba điểm trung bình tương ứng ba học viên.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
print(scores.mean(axis=0))
```

Kết quả khi chạy:

```
[7.66666667 8.        ]
```

Mảng đầu ra có hai số vì bảng có hai **cột/môn**; kết quả lần lượt là trung bình Toán và Python.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Gom bảng điểm học viên

Từ vựng trong mục: ndarray — mảng NumPy; **Shape** — kích thước.

Tình huống thực tế: Trung tâm ghi điểm hai môn Toán và Python của ba học viên theo thứ tự An (8,9), Bình (6,7), Lan (9,8), thang điểm 10. Hãy tạo mảng và in số hàng, số cột.

Vì sao chọn cách giải này? `np.array` gom điểm thành bảng, `shape` cho biết đúng 3 học viên và 2 môn.

Quy trình giải:

1. Tạo mảng hai chiều, mỗi hàng ứng với một học viên.
2. Giữ thứ tự cột Toán rồi Python.
3. In `scores.shape` để xác minh số người và môn.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
print(scores.shape)
```

Kết quả khi chạy:

```
(3, 2)
```

Kết quả cho biết 3 hàng học viên và 2 cột môn học.

Tình huống 2 — Trung bình theo từng môn

Từ vựng trong mục: Axis — trục tính toán; **Mean** — trung bình.

Tình huống thực tế: Với cùng bảng ba học viên, hai môn Toán và Python, trung tâm cần điểm trung bình của **mỗi môn** trên thang 10. Hãy in hai số trung bình theo thứ tự cột.

Vì sao chọn cách giải này? `mean(axis=0)` gom các học viên cùng cột môn và tính trung bình.

Quy trình giải:

1. Tạo mảng 3 hàng × 2 cột theo thứ tự Toán, Python.
2. Tính `scores.mean(axis=0)`.
3. Đối chiếu hai số với phép tính trung bình ở từng cột.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
print(scores.mean(axis=0))
```

Kết quả khi chạy:

```
[7.66666667 8.        ]
```

Toán trung bình khoảng 7,67; Python trung bình 8,00 điểm.

Tình huống 3 — Tính tổng doanh thu mỗi cửa hàng

Từ vựng trong mục: Axis — trục tính toán; **Sum** — phép cộng tổng.

Tình huống thực tế: Cửa hàng A có doanh thu ba ngày `[10,20,30]`, cửa hàng B có `[8,12,16]`, đơn vị triệu đồng. Bảng có **hai hàng cửa hàng** và **ba cột ngày**. Hãy tính tổng doanh thu riêng của từng cửa hàng.

Vì sao chọn cách giải này? Vì cửa hàng nằm trên hàng, `sum(axis=1)` cộng ba cột ngày cho mỗi hàng.

Quy trình giải:

1. Tạo `sales` với hàng đầu là cửa hàng A và hàng sau là B.
2. Cộng theo `axis=1`, tức theo từng hàng.
3. In hai tổng theo thứ tự A rồi B.

```
import numpy as np

sales = np.array([[10, 20, 30], [8, 12, 16]])
print(sales.sum(axis=1))
```

Kết quả khi chạy:

```
[60 36]
```

Cửa hàng A thu 60 triệu đồng; cửa hàng B thu 36 triệu đồng.

Tình huống 4 — Tạo bảng dự báo khởi tạo

Từ vựng trong mục: Shape — kích thước; **Zero array** — mảng số 0.

Tình huống thực tế: Đội phân tích chuẩn bị một bảng dự báo cho 2 cửa hàng và 3 ngày, chưa tính giá trị. Hãy tạo mảng toàn số 0 và in hình dạng bảng.

Vì sao chọn cách giải này? `np.zeros((2, 3))` khởi tạo đúng số hàng cửa hàng và số cột ngày; số 0 mới là giá trị ban đầu, không phải doanh thu dự báo đã xác minh.

Quy trình giải:

1. Gọi `np.zeros` với hình dạng `(2, 3)`.
2. Mỗi hàng dành cho một cửa hàng, mỗi cột dành cho một ngày.
3. In `forecast.shape` để kiểm tra cấu trúc.

```
import numpy as np

forecast = np.zeros((2, 3))
print(forecast.shape)
```

Kết quả khi chạy:

```
(2, 3)
```

Bảng dành cho hai cửa hàng, ba ngày, tổng cộng sáu vị trí chưa có dự báo thực.

Tình huống 5 — Kiểm tra kiểu và số lượng điểm

Từ vựng trong mục: dtype — kiểu phần tử; **size** — số phần tử.

Tình huống thực tế: Trung tâm nhập hai điểm kiểm tra 7,5 và 8,0 của một học viên. Hãy dùng mảng số thực để giữ phần thập phân rồi in kiểu và số lượng điểm.

Vì sao chọn cách giải này? `dtype=float` tránh mất phần thập phân; `size` đếm đúng hai số được lưu.

Quy trình giải:

1. Tạo mảng chứa `7.5` và `8.0` với `dtype=float`.
2. Truy cập thuộc tính `dtype` và `size`.
3. In tên kiểu và số phần tử.

```
import numpy as np

scores = np.array([7.5, 8.0], dtype=float)
print(scores.dtype, scores.size)
```

Kết quả khi chạy:

```
float64 2
```

Dữ liệu gồm hai điểm kiểu `float64`, không phải chuỗi.

6. Sai lầm thường gặp

- `shape=(3,2)` là **3 hàng và 2 cột**, không phải ngược lại; phải ghi rõ mỗi hàng/cột đại diện điều gì.
- `axis=0` tổng hợp theo từng cột trong mảng hai chiều; `axis=1` tổng hợp theo từng hàng. Chọn nhầm trục khiến ý nghĩa kết quả khác đi.
- `dtype=float` giữ phần thập phân, còn kiểu số nguyên có thể cắt phần lẻ nếu ép dữ liệu không phù hợp.
- Kết quả trung bình là số thực; các chữ số cuối có thể phụ thuộc biểu diễn dấu phẩy động, nên giải thích bằng giá trị gần đúng khi cần.
- `np.zeros()` chỉ tạo giá trị khởi tạo 0; không được diễn giải là dự báo thực tế đã được mô hình tính.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a NumPy `ndarray`? — `ndarray` của NumPy là gì?

A. A class inheritance rule — Quy tắc kế thừa lớp

B. A Python exception — Một ngoại lệ Python

C. A single text file — Một tệp văn bản đơn

D. An array for data with a common dtype — Mảng dữ liệu có kiểu phần tử chung

Đáp án: D

Giải thích: `ndarray` lưu phần tử theo một hay nhiều chiều có chỉ số và một `dtype` chung. Ví dụ `np.array([1, 2, 3])` là mảng số một chiều, không phải tệp hay ngoại lệ Python.

Q2. What does `shape=(3, 2)` mean for a 2-D array? — `shape=(3, 2)` nghĩa là gì với mảng hai chiều?

A. Only three total elements — Chỉ có ba phần tử

B. Three rows and two columns — Ba hàng và hai cột

C. Two rows and three columns — Hai hàng và ba cột

D. Three dimensions and two values — Ba chiều và hai giá trị

Đáp án: B

Giải thích: Với mảng hai chiều, `shape` ghi `(số_hàng, số_cột)`. Vì vậy `(3, 2)` là ba quan sát ở các hàng và hai đặc trưng ở các cột, tổng cộng sáu ô dữ liệu.

Q3. What is the size of an array with shape `(3, 2)`? — Mảng có shape `(3, 2)` chứa bao nhiêu phần tử?

A. 5 — 5

B. 6 — 6

C. 3 — 3

D. 2 — 2

Đáp án: B

Giải thích: `size` của ndarray đếm toàn bộ phần tử và bằng tích các kích thước trong `shape`. Với `(3, 2)`, tính `3 × 2 = 6`; cộng hai kích thước lại sẽ cho số phần tử sai.

Q4. What does `ndim` report? — Thuộc tính `ndim` cho biết gì?

A. The data type of an element — Kiểu phần tử

B. The number of rows only — Chỉ số hàng

C. The number of array dimensions — Số chiều của mảng

D. The maximum array value — Giá trị lớn nhất

Đáp án: C

Giải thích: `ndim` cho số trục/chiều, không phải số giá trị được lưu. Ma trận có shape `(3, 2)` có `ndim == 2` dù `size == 6`.

Q5. What does `dtype` describe? — `dtype` mô tả gì?

A. The file's extension — Phần mở rộng của file

B. The data type of array elements — Kiểu dữ liệu của các phần tử

C. The mean of all entries — Trung bình các phần tử

D. The array's row count — Số hàng

Đáp án: B

Giải thích: `dtype` mô tả kiểu và cách biểu diễn chung của các phần tử mảng, như `int64` hoặc `float64`. Nó khác `shape`, vốn cho biết số phần tử trên từng trục.

Q6. Why specify `dtype=float` when storing decimal scores? — Vì sao dùng `dtype=float` khi lưu điểm có phần lẻ?

A. To preserve fractional values — Để giữ giá trị có phần thập phân

B. To force integer truncation — Để ép cắt về số nguyên

C. To delete missing records — Để xóa bản ghi thiếu

D. To change rows into columns — Để đổi hàng thành cột

Đáp án: A

Giải thích: `dtype=float` yêu cầu NumPy lưu giá trị dưới dạng dấu phẩy động để điểm như `8.5` giữ được phần lẻ. Nếu đổi sang kiểu nguyên, phần lẻ có thể bị bỏ trong quá trình chuyển kiểu.

Q7. For a 2-D array, what does `mean(axis=0)` return? — Với mảng hai chiều, `mean(axis=0)` trả về gì?

A. Only the first cell — Chỉ ô đầu tiên

B. One mean per column — Một giá trị trung bình cho mỗi cột

C. The original array unchanged — Mảng gốc không đổi

D. One mean per row — Một giá trị trung bình cho mỗi hàng

Đáp án: B

Giải thích: Với mảng 2D, `axis=0` gộp theo chiều hàng và để lại một kết quả cho từng cột. Với `[[1, 3], [5, 7]]`, `mean(axis=0)` được `[3., 5.]`, không phải các trung bình theo hàng.

Q8. For a 2-D array, what does `sum(axis=1)` produce? — Với mảng hai chiều, `sum(axis=1)` tạo gì?

A. A list of column names — List tên cột

B. The shape tuple only — Chỉ tuple shape

C. One sum per column — Một tổng cho mỗi cột

D. One sum per row — Một tổng cho mỗi hàng

Đáp án: D

Giải thích: `sum(axis=1)` cộng các cột trong từng hàng. Với `[[1, 3], [5, 7]]`, tổng theo hàng là `4` và `12`, tạo kết quả một chiều gồm hai giá trị.

Q9. Which statement correctly distinguishes `shape` and `size`? — Đâu là điểm khác nhau giữa `shape` và `size`?

A. shape lists dimensions; size counts elements — shape ghi kích thước từng chiều; size đếm phần tử

B. size gives dtype; shape gives values — size cho kiểu dữ liệu; shape cho giá trị

C. They are always identical numbers — Chúng luôn là cùng một số

D. Both change elements automatically — Cả hai tự đổi phần tử

Đáp án: A

Giải thích: `shape` là tuple chỉ độ dài từng chiều, còn `size` là một số đếm toàn bộ phần tử. Mảng có shape `(3, 2)` gồm hai chiều và `size == 6`; hai thuộc tính này không tự sắp xếp lại dữ liệu.

Q10. What does `np.array([1, 2, 3])` create? — `np.array([1, 2, 3])` tạo ra gì?

A. A one-dimensional NumPy array — Một mảng NumPy một chiều

B. A class named Array — Lớp tên Array

C. A CSV file on disk — File CSV trên đĩa

D. A two-column table — Bảng hai cột

Đáp án: A

Giải thích: `np.array([1, 2, 3])` tạo ndarray có shape `(3,)`, nghĩa là một chiều chứa ba phần tử. Đây không phải bảng hai cột, vì bảng đó cần cấu trúc hai chiều.

Q11. What does `np.zeros((2, 3))` create? — `np.zeros((2, 3))` tạo gì?

A. A 2×3 table of missing strings — Bảng 2×3 gồm chuỗi thiếu

B. A 3×2 array of ones — Mảng 3×2 toàn số 1

C. A 2×3 array initialized to zeros — Mảng 2×3 được khởi tạo toàn số 0

D. An empty array with no elements — Mảng rỗng không có phần tử

Đáp án: C

Giải thích: `np.zeros((2, 3))` tạo hai hàng và ba cột, mọi ô khởi tạo bằng không. Mặc định các ô thuộc kiểu số thực, không phải chuỗi thiếu dữ liệu hay mảng không có phần tử.

Q12. How is `np.ones((2, 2))` different from `np.zeros((2, 2))`? — `np.ones((2, 2))` khác `np.zeros((2, 2))` thế nào?

A. It has four dimensions — Nó có bốn chiều

B. It cannot be used in arithmetic — Nó không thể tính toán

C. It always creates a CSV file — Nó luôn tạo file CSV

D. It initializes entries to one instead of zero — Nó khởi tạo giá trị 1 thay cho 0

Đáp án: D

Giải thích: `np.ones((2, 2))` và `np.zeros((2, 2))` cùng shape và có bốn ô số. Hàm thứ nhất điền `1`, hàm thứ hai điền `0`; không hàm nào tạo bốn chiều.

Q13. How do NumPy array and Python List differ in numeric operations? — Mảng NumPy khác List Python về phép toán số như thế nào?

A. A NumPy array stores only strings — Mảng NumPy chỉ lưu chuỗi

B. A List always multiplies elements element-wise — List luôn nhân từng phần tử

C. A List cannot hold numbers — List không chứa được số

D. NumPy enables element-wise numerical operations — NumPy hỗ trợ phép toán số theo từng phần tử

Đáp án: D

Giải thích: Với ndarray số, `a * 2` nhân từng phần tử lên hai: `[1, 2]` thành `[2, 4]`. Ngược lại, `[1, 2] * 2` của List Python lặp dãy thành `[1, 2, 1, 2]`.

Q14. For an array with rows as learners and columns as subjects, what does axis=0 average? — Nếu hàng là học viên và cột là môn, `axis=0` tính trung bình theo gì?

A. Only the last subject — Chỉ môn cuối

B. Each learner across subjects — Từng học viên qua các môn

C. Each subject across learners — Từng môn qua các học viên

D. Only the first learner — Chỉ học viên đầu

Đáp án: C

Giải thích: Nếu mỗi hàng là một học viên và mỗi cột là một môn, `mean(axis=0)` tính trung bình qua các học viên cho từng môn riêng. Có hai môn thì được hai trung bình; `axis=1` lại tính trung bình các môn của từng học viên.

Q15. How many dimensions does a two-dimensional score table have? — Bảng điểm hai chiều có `ndim` bằng bao nhiêu?

A. The total number of scores — Tổng số điểm

B. 3 — 3

C. 2 — 2

D. 1 — 1

Đáp án: C

Giải thích: Bảng điểm sắp theo hàng và cột có hai trục, dù có bao nhiêu hàng. Vì vậy `ndim` bằng `2`; tổng số ô điểm cần đọc từ `size`.

Q16. Which code accesses array shape without computing new values? — Cú pháp nào lấy shape mà không tính thêm dữ liệu?

A. `X.sort()` — `X.sort()`

B. `X.shape` — `X.shape`

C. `X.delete()` — `X.delete()`

D. `X.write()` — `X.write()`

Đáp án: B

Giải thích: `X.shape` đọc thuộc tính ndarray, trả tuple kích thước các chiều mà không sửa giá trị. Với bảng ba hàng hai cột, kết quả là `(3, 2)`; `X.sort()` thì thay đổi thứ tự phần tử.

Q17. Which code counts all array entries? — Cú pháp nào đếm tổng số phần tử mảng?

A. `X.shape[0]` in every case — Luôn `X.shape[0]`

B. `X.dtype` — `X.dtype`

C. `X.size` — `X.size`

D. `X.mean()` — `X.mean()`

Đáp án: C

Giải thích: `X.size` đếm phần tử qua toàn bộ các chiều. Với mảng `(3, 2)`, kết quả là `6`, còn `X.shape[0]` chỉ trả số hàng là `3`.

Q18. For finite floating-point values within the target integer range, what does NumPy `.astype(int)` do to their fractional parts? — Với giá trị dấu phẩy động hữu hạn nằm trong phạm vi kiểu số nguyên đích, NumPy `.astype(int)` xử lý phần lẻ thế nào?

A. It discards fractional parts toward zero — Bỏ phần lẻ theo hướng về số 0

B. It rounds every value to the nearest integer — Làm tròn mọi giá trị về số nguyên gần nhất

C. It preserves fractional parts unchanged — Giữ nguyên toàn bộ phần lẻ

D. It forces the shape to become `(1, 1)` — Buộc shape chuyển thành `(1, 1)`

Đáp án: A

Giải thích: Với số thực hữu hạn thuộc phạm vi kiểu nguyên đích, `.astype(int)` bỏ phần lẻ theo hướng về không chứ không làm tròn gần nhất. Vì vậy `[2.9, -2.9]` đổi thành số nguyên `[2, -2]` và vẫn giữ shape một chiều ban đầu.

Q19. What is the output shape of `X.mean(axis=0)` for `X.shape == (3,2)`? — Nếu `X.shape == (3,2)`, `X.mean(axis=0)` có shape nào?

A. `(2,)` — `(2,)`

B. `(3,)` — `(3,)`

C. `(3,2)` — `(3,2)`

D. `(1,3)` — `(1,3)`

Đáp án: A

Giải thích: Ma trận `(3, 2)` gồm ba hàng và hai cột. `mean(axis=0)` gộp ba hàng, để lại một trung bình cho mỗi cột nên shape mặc định là `(2,)`, không phải `(3,)`.

Q20. Why label the meaning of each row and column? — Vì sao phải nêu rõ ý nghĩa từng hàng, cột?

A. To convert a matrix into text — Để chuyển ma trận thành chữ

B. To change dtype automatically — Để tự đổi dtype

C. To avoid all numeric calculations — Để tránh mọi phép tính

D. To interpret aggregated results correctly — Để hiểu đúng kết quả tổng hợp

Đáp án: D

Giải thích: Nhãn hàng và cột cho biết giá trị tổng hợp đang đo cái gì chứ không chỉ cách tính. Nếu hàng là học viên và cột là môn, `mean(axis=0)` trả điểm trung bình từng môn; hiểu ngược cấu trúc sẽ trả lời câu hỏi khác.

8. Nối sang bài tiếp theo

`List → NumPy Array → Matrix → PCA`.

PY-14 — NumPy indexing, slicing, boolean và fancy indexing

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Index — chỉ số; **Slice** — cắt lát.

`NumPy ndarray → Chọn hàng/cột → DataFrame/PCA`

Học xong dùng để làm gì? Lọc quan sát, chọn cột đặc trưng, hiểu lúc nào thao tác sửa dữ liệu gốc qua view.

Kết nối kiến thức tiếp theo: PY-15 thực hiện phép tổng hợp và broadcasting trên dữ liệu được chọn.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Index	chỉ số	Địa chỉ phần tử của mảng
Slice	cắt lát	Lấy một đoạn theo start:stop
Boolean mask	mặt nạ logic	Mảng True/False để lọc phần tử
Fancy indexing	chỉ số nâng cao	Chọn nhiều vị trí theo danh sách chỉ số

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`X[row,col]`	Chọn một ô theo hàng/cột, đếm từ 0
`X[:,j] / X[i,:]`	Lấy cả cột j hoặc hàng i
`X[condition]`	Giữ các phần tử thỏa điều kiện logic
`X[:,[0,2]]`	Chọn các cột không liền nhau

4. Khái niệm liên quan

Trong bảng điểm NumPy, **hàng là học viên**, **cột là môn học**, **ô là điểm một học viên ở một môn**. Bài này đi từ chọn một ô bằng chỉ số đến chọn nhiều ô/hàng/cột bằng slice, mặt nạ Boolean và fancy indexing. Mỗi cách trả lời một câu hỏi khác nhau khi làm việc với dữ liệu.

4.1. Indexing — Đọc một vị trí

Từ vựng trong mục: Index — chỉ số; **Row/column** — hàng/cột.

Mảng hai chiều giống một bảng có hàng và cột. Giả sử `scores` có **hai hàng học viên** An, Bình và **hai cột môn** Toán, Python, lần lượt là `[[8,9],[6,7]]`. **Chỉ số bắt đầu từ 0**; `scores[1,0]` lấy hàng thứ hai (Bình), cột đầu (Toán), cho điểm 6. Dấu phẩy trong `[row, col]` phân biệt vị trí hàng và cột.

```
import numpy as np

scores = np.array([[8, 9], [6, 7]])
print(scores[1, 0])
```

Kết quả khi chạy:

```
6
```

Chỉ số hàng 1 và cột 0 lấy điểm Toán của học viên thứ hai, không phải điểm học viên đầu.

4.2. Slicing — Cắt một đoạn liên tiếp

Từ vựng trong mục: Slice — lát cắt; **Start/stop** — đầu/cuối.

Slicing chọn một đoạn theo chỉ số, cú pháp `[start:stop]` lấy từ start **đến trước stop**. Với điểm `[5,6,7,8]`, `scores[1:3]` lấy vị trí 1 và 2, nghĩa là hai điểm 6, 7. Trên bảng hai chiều, `scores[:,0]` lấy tất cả hàng nhưng chỉ cột đầu; dấu `:` đứng riêng nghĩa là lấy toàn bộ chiều đó. Lát cắt NumPy cơ bản có thể là **view** dùng chung bộ nhớ với mảng gốc.

```
import numpy as np

scores = np.array([5, 6, 7, 8])
print(scores[1:3])
```

Kết quả khi chạy:

```
[6 7]
```

Giá trị 8 ở vị trí 3 không được lấy vì điểm stop bị loại khỏi lát cắt.

4.3. Boolean mask — Lọc theo điều kiện

Từ vựng trong mục: Boolean mask — mặt nạ logic; **Condition** — điều kiện.

Nếu các điểm là `[4,7,9]` và yêu cầu giữ điểm từ 7 trở lên, phép `scores >= 7` tạo mặt nạ `[False, True, True]`. Dùng `scores[mask]` chỉ lấy phần tử ứng với `True`. Với bảng hai chiều, `X[X[:,0] >= 8]` giữ **các hàng có điểm ở cột đầu đạt 8**, chứ không chỉ lấy các ô điểm cao.

```
import numpy as np

scores = np.array([4, 7, 9])
mask = scores >= 7
print(scores[mask])
```

Kết quả khi chạy:

```
[7 9]
```

Hai giá trị 7 và 9 vượt hoặc bằng ngưỡng, còn 4 bị loại.

4.4. Fancy indexing — Chọn vị trí rời rạc

Từ vựng trong mục: Fancy indexing — chỉ số nâng cao; **Index array** — mảng chỉ số.

Khác với **Slice** lấy đoạn liên tiếp, **Fancy indexing** cho phép chỉ định nhiều vị trí không liền nhau. Với `[3,5,7]`, `scores[[0,2]]` lấy phần tử đầu và cuối, bỏ phần tử giữa. Trên bảng hai chiều, `X[:, [0,2]]` lấy cột thứ nhất và thứ ba. Fancy indexing bằng mảng chỉ số thường tạo **bản sao** thay vì view của dữ liệu gốc.

```
import numpy as np

scores = np.array([3, 5, 7])
print(scores[[0, 2]])
```

Kết quả khi chạy:

```
[3 7]
```

Chương trình lấy vị trí 0 và 2 mà không phải lấy cả đoạn 0–2.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tra điểm một học viên

Từ vựng trong mục: Index — chỉ số; **Shape** — kích thước.

Tình huống thực tế: Trung tâm có điểm Toán và Python của ba học viên An (8,9), Bình (6,7), Lan (9,8), thang 10. Hãy in **điểm Toán của Bình** bằng chỉ số hàng và cột.

Vì sao chọn cách giải này? `scores[1,0]` định vị chính xác hàng Bình và cột Toán, vì Python đếm từ 0.

Quy trình giải:

1. Lưu ba học viên theo hàng An, Bình, Lan.
2. Lưu môn Toán ở cột 0, Python ở cột 1.
3. Lấy ô `[1,0]` và in giá trị.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]])
print(scores[1, 0])
```

Kết quả khi chạy:

```
6
```

Học viên Bình có điểm Toán 6 trên thang 10.

Tình huống 2 — Lọc học viên có điểm cao

Từ vựng trong mục: Boolean mask — mặt nạ Boolean; **Threshold** — ngưỡng.

Tình huống thực tế: Vẫn bảng ba học viên, hai môn Toán và Python như trên. Trung tâm chỉ muốn giữ những học viên có **điểm Toán từ 8 trở lên**. Hãy in nguyên hàng điểm của những học viên đạt ngưỡng.

Vì sao chọn cách giải này? Mặt nạ `scores[:,0] >= 8` được tạo từ cột Toán, sau đó áp vào toàn bộ hàng.

Quy trình giải:

1. Lấy cột Toán của cả ba học viên bằng `scores[:, 0]`.
2. So sánh với 8 để tạo mặt nạ Boolean.
3. Lấy các hàng thỏa ngưỡng và in bảng.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]])
print(scores[scores[:, 0] >= 8])
```

Kết quả khi chạy:

```
[[8 9]
 [9 8]]
```

An và Lan có điểm Toán từ 8 trở lên; Bình không đạt.

Tình huống 3 — Chọn hai môn không liền nhau

Từ vựng trong mục: Fancy indexing — chỉ số nâng cao; **Column** — cột.

Tình huống thực tế: Hai học viên có điểm của ba môn Toán, Văn, Python lần lượt là `[8,7,9]` và `[6,5,8]`. Giáo viên chỉ muốn xem **Toán và Python**, bỏ cột Văn ở giữa.

Vì sao chọn cách giải này? Fancy indexing `[:, [0, 2]]` chọn hai cột rời nhau mà vẫn giữ mọi hàng học viên.

Quy trình giải:

1. Lưu hai hàng học viên và ba cột theo thứ tự Toán, Văn, Python.
2. Chọn tất cả hàng với `:` và cột 0, 2 bằng danh sách chỉ số.
3. In bảng hai cột kết quả.

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores[:, [0, 2]])
```

Kết quả khi chạy:

```
[[8 9]
 [6 8]]
```

Bảng còn lại hai môn Toán và Python, giữ nguyên hai hàng học viên.

Tình huống 4 — Lấy ba khách hàng đầu

Từ vựng trong mục: Slice — lát cắt; **Array** — mảng.

Tình huống thực tế: Cửa hàng có năm khoản chi tiêu khách hàng là `[100,200,50,300,120]` nghìn đồng, lưu theo thứ tự thời gian. Hãy lấy **ba khoản đầu tiên** để kiểm tra nhanh.

Vì sao chọn cách giải này? `[:3]` lấy các chỉ số 0, 1, 2 mà không lấy chỉ số 3.

Quy trình giải:

1. Tạo mảng gồm năm khoản chi tiêu theo thứ tự.
2. Dùng lát cắt `[:3]` lấy ba vị trí đầu.
3. In ba số tiền (nghìn đồng).

```
import numpy as np

spending = np.array([100, 200, 50, 300, 120])
print(spending[:3])
```

Kết quả khi chạy:

```
[100 200  50]
```

Ba khoản đầu là 100, 200 và 50 nghìn đồng.

Tình huống 5 — Thay mã số đo lỗi trên bản sao

Từ vựng trong mục: Boolean mask — mặt nạ Boolean; **Copy** — bản sao.

Tình huống thực tế: Cảm biến lưu nhiệt độ `[20,-1,23,24]` °C; `-1` là mã lỗi, **không phải nhiệt độ thật**. Hãy tạo bản sao, thay mã lỗi bằng 0 để minh họa phép gán có điều kiện, đồng thời giữ nguyên mảng gốc.

Vì sao chọn cách giải này? `copy()` tránh sửa đầu vào; mặt nạ `<0` chọn mã âm cần thay. Giá trị 0 ở bản xử lý chỉ là **giá trị tạm**, không được suy diễn là số đo thật.

Quy trình giải:

1. Lưu mảng gốc và tạo `clean = values.copy()`.
2. Chọn phần tử âm bằng `clean < 0`.
3. Gán 0 ở vị trí đó, in cả hai mảng để so sánh.

```
import numpy as np

values = np.array([20, -1, 23, 24])
clean = values.copy()
clean[clean < 0] = 0
print(values, clean)
```

Kết quả khi chạy:

```
[20 -1 23 24] [20  0 23 24]
```

Bản gốc giữ `-1`; bản sao đổi mã lỗi thành 0. Trong dữ liệu thực tế cần đánh dấu thiếu thay vì coi 0 °C là số đo hợp lệ.

6. Sai lầm thường gặp

- Chỉ số bắt đầu từ **0**. Mảng có ba hàng thì hàng thứ ba là chỉ số 2, không phải 3.
- `a[1:3]` lấy phần tử 1, 2 và **không** lấy 3. Lát cắt NumPy cơ bản thường là view: sửa view có thể sửa mảng gốc.
- Mặt nạ Boolean chọn phần tử/hàng thỏa điều kiện, còn Fancy indexing chọn vị trí theo danh sách chỉ số và thường tạo bản sao.
- Trong mảng 2-D, `X[:, 0]` là cột đầu, `X[0, :]` là hàng đầu: không đảo ý nghĩa hàng/cột.
- Không coi mã lỗi cảm biến `-1` hay giá trị tạm `0` là số đo hợp lệ khi phân tích dữ liệu thật.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. Where does Python/NumPy indexing begin? — Chỉ số Python/NumPy bắt đầu từ đâu?

A. -2 — -2

B. 0 — 0

C. The number of dimensions — Số chiều của mảng

D. 1 — 1

Đáp án: B

Giải thích: Chỉ số bắt đầu từ không: `a[0]` là phần tử đầu, còn `a[1]` là phần tử thứ hai. Chỉ số âm đếm ngược từ cuối.

Q2. For a 2-D array, what does `X[1, 0]` select? — Trong mảng hai chiều, `X[1, 0]` chọn gì?

A. Every row, first column — Mọi hàng, cột đầu

B. Row 2, column 1 — Hàng thứ hai, cột đầu tiên

C. All columns of the first row — Mọi cột của hàng đầu

D. Row 1, column 2 — Hàng đầu, cột thứ hai

Đáp án: B

Giải thích: Mảng 2D được truy cập theo `X[hàng, cột]`, với chỉ số bắt đầu từ không. Vì thế `X[1, 0]` là hàng hai, cột một.

Q3. What does `a[-1]` mean for a nonempty 1-D array? — `a[-1]` nghĩa là gì với mảng một chiều không rỗng?

A. An invalid index in all cases — Chỉ số luôn không hợp lệ

B. The last element — Phần tử cuối

C. All negative values — Mọi giá trị âm

D. The first element — Phần tử đầu

Đáp án: B

Giải thích: Với mảng không rỗng, chỉ số `-1` lấy phần tử cuối; ở `[4, 6, 8]` đó là `8`. Ngược lại, chỉ số `0` lấy phần tử đầu.

Q4. Which endpoint does a slice `a[1:4]` exclude? — Slice `a[1:4]` không lấy vị trí nào?

A. Index 3 — Chỉ số 3

B. Index 1 — Chỉ số 1

C. Index 4 — Chỉ số 4

D. Index 2 — Chỉ số 2

Đáp án: C

Giải thích: Chỉ số dừng của slice không được lấy: `a[1:4]` chọn vị trí 1, 2, 3 nếu có. Vị trí 4 không thuộc lát cắt.

Q5. What does `X[:, 0]` select? — `X[:, 0]` lấy gì?

A. Only the top-left value always — Luôn chỉ ô góc trên trái

B. All columns in the first row — Mọi cột ở hàng đầu

C. Only rows with value zero — Chỉ hàng có giá trị 0

D. All rows in the first column — Mọi hàng ở cột đầu

Đáp án: D

Giải thích: Trong `X[:, 0]`, dấu `:` chọn mọi hàng và `0` chọn cột đầu. Kết quả là dữ liệu theo cột, khác `X[0, :]` lấy một hàng.

Q6. What does `X[0, :]` select? — `X[0, :]` lấy gì?

A. All columns of the first row — Mọi cột của hàng đầu

B. The last row — Hàng cuối

C. All rows of the first column — Mọi hàng của cột đầu

D. The diagonal — Đường chéo

Đáp án: A

Giải thích: `X[0, :]` cố định hàng số không và giữ mọi cột. Với `[[1,2],[3,4]]`, kết quả là hàng đầu `[1,2]`, không phải cột đầu.

Q7. What does a Boolean mask contain? — Mặt nạ Boolean chứa gì?

A. Only positive integers — Chỉ số nguyên dương

B. Random floating-point numbers — Số thực ngẫu nhiên

C. Only column names — Chỉ tên cột

D. True/False values indicating which entries to keep — Giá trị True/False cho biết phần tử nào được giữ

Đáp án: D

Giải thích: Mặt nạ Boolean chứa các vị trí `True` và `False`, thường sinh từ phép so sánh. Với `[4, 8]`, điều kiện `a >= 7` chỉ chọn phần tử thứ hai.

Q8. What is selected by `a[a >= 7]`? — `a[a >= 7]` chọn gì?

A. All entries smaller than 7 — Mọi phần tử nhỏ hơn 7

B. Only the element at index 7 — Chỉ phần tử chỉ số 7

C. Elements at least 7 — Các phần tử từ 7 trở lên

D. The first seven entries — Bảy phần tử đầu

Đáp án: C

Giải thích: Biểu thức so giá trị với ngưỡng 7 rồi giữ phần tử thỏa điều kiện. Với `[4, 7, 9]`, kết quả là `[7, 9]`, không phải phần tử có chỉ số 7.

Q9. What does `a[[0, 2]]` illustrate? — `a[[0, 2]]` minh họa điều gì?

A. A logical AND operation — Phép AND logic

B. Slicing one continuous range — Cắt một đoạn liên tục

C. Fancy indexing with selected positions — Fancy indexing chọn các vị trí cụ thể

D. A matrix transpose — Chuyển vị ma trận

Đáp án: C

Giải thích: `a[[0, 2]]` dùng mảng chỉ số nguyên để chọn các vị trí cụ thể. Nó có thể lấy phần tử đầu và thứ ba không liền nhau, khác với lát cắt liên tục.

Q10. What is the difference between slicing and fancy indexing? — Slicing khác fancy indexing ở điểm nào?

A. Both always select entire matrices — Cả hai luôn lấy cả ma trận

B. Slicing requires Boolean values only — Slicing chỉ dùng Boolean

C. Slices describe ranges; fancy indexing specifies index arrays — Slice chỉ đoạn, fancy indexing chỉ mảng vị trí

D. Fancy indexing never uses integers — Fancy indexing không dùng số nguyên

Đáp án: C

Giải thích: Slice xác định dãy chỉ số có quy luật như `a[1:6:2]`; fancy indexing nguyên chỉ vị trí tùy ý như `a[[5, 1, 5]]`. Fancy indexing có thể lặp hoặc đổi thứ tự vị trí.

Q11. In NumPy, basic slicing generally returns what? — Basic slicing của NumPy thường trả về gì?

A. A view of the original data — View chia sẻ dữ liệu với mảng gốc

B. A guaranteed independent copy — Luôn là bản sao độc lập

C. A plain Python set — Một Set Python

D. A new CSV file — File CSV mới

Đáp án: A

Giải thích: Basic slicing thường trả view chia sẻ bộ nhớ nền. Sửa `part = a[1:3]` có thể ảnh hưởng giá trị gốc tương ứng; hãy dùng `.copy()` khi cần dữ liệu độc lập.

Q12. Does advanced integer-array indexing generally return a view? — Indexing bằng mảng số nguyên nâng cao thường tạo view không?

A. Yes, it always shares memory — Có, luôn chia sẻ bộ nhớ

B. No, it generally returns a copy — Không, thường trả bản sao

C. It converts the source to strings — Nó chuyển nguồn thành chuỗi

D. It only returns None — Nó chỉ trả None

Đáp án: B

Giải thích: Indexing nâng cao bằng số nguyên thường sao chép dữ liệu được chọn thay vì tạo view. Sửa `b = a[[0,2]]` thường không đổi `a`; gán trực tiếp `a[[0,2]] = ...` là thao tác khác.

Q13. How do you keep source data unchanged before assignment? — Làm sao giữ mảng gốc không đổi trước khi gán?

A. Make an explicit `copy()` — Tạo bản sao bằng `copy()`

B. Use `del X` first — Xóa X trước

C. Always modify a slice directly — Luôn sửa trực tiếp slice

D. Replace every value with zero — Thay mọi giá trị bằng 0

Đáp án: A

Giải thích: Lệnh `b = a.copy()` tạo bộ nhớ riêng trước khi sửa. Sau đó gán `b[0] = 9` không đổi `a[0]`, khác với sửa view dùng chung dữ liệu.

Q14. For shape (3,2), what does `X[:, 1]` return? — Nếu shape là (3,2), `X[:, 1]` trả về gì?

A. Three values from the second column — Ba giá trị ở cột thứ hai

B. Two values from the first row — Hai giá trị của hàng đầu

C. An array with five values — Mảng năm giá trị

D. One value in the center — Một giá trị giữa bảng

Đáp án: A

Giải thích: Mảng `(3,2)` có ba hàng hai cột. `X[:, 1]` lấy cột thứ hai qua cả ba hàng, cho kết quả có shape `(3,)`.

Q15. What does `X[:, [0, 2]]` mean when X has three columns? — `X[:, [0, 2]]` nghĩa là gì nếu X có ba cột?

A. First two rows, all columns — Hai hàng đầu, mọi cột

B. The sum of columns zero and two — Tổng hai cột 0 và 2

C. All rows and the first and third columns — Mọi hàng, cột thứ nhất và thứ ba

D. Only the second column — Chỉ cột thứ hai

Đáp án: C

Giải thích: Với ba cột, chỉ số `[0,2]` xác định cột thứ nhất và thứ ba. Dấu hai chấm giữ mọi hàng; biểu thức chọn hai cột chứ không cộng giá trị của chúng.

Q16. What is the effect of `a[:3]`? — `a[:3]` có tác dụng gì?

A. Take up to the first three entries — Lấy tối đa ba phần tử đầu

B. Take index 3 alone — Chỉ lấy phần tử số 3

C. Sort entries descending — Sắp phần tử giảm dần

D. Remove the first three entries — Xóa ba phần tử đầu

Đáp án: A

Giải thích: Bỏ start nghĩa là mặc định từ không, còn chỉ số dừng 3 không được lấy. Vì vậy `a[:3]` lấy tối đa ba phần tử đầu ngay cả khi mảng ngắn hơn; nó không xóa chúng.

Q17. Which operation filters entire rows based on the first column? — Lệnh nào lọc cả hàng theo điều kiện ở cột đầu?

A. `X.shape` — `X.shape`

B. `X[:, 0]` alone — Chỉ `X[:, 0]`

C. `X[0, 0]` — `X[0, 0]`

D. `X[X[:, 0] >= 8]` — `X[X[:, 0] >= 8]`

Đáp án: D

Giải thích: `X[:,0] >= 8` tạo một Boolean cho mỗi hàng bằng cách xét giá trị cột đầu. Dùng mask này để lấy từ `X` sẽ giữ nguyên các hàng thỏa điều kiện, không chỉ phần tử cột đầu.

Q18. What happens when a scalar index is beyond array bounds? — Khi chỉ số phần tử vượt giới hạn mảng thì sao?

A. It silently creates a new row — Nó âm thầm tạo hàng mới

B. NumPy raises IndexError — NumPy báo IndexError

C. It converts the array to a dictionary — Nó chuyển mảng thành Dictionary

D. It always returns zero — Nó luôn trả 0

Đáp án: B

Giải thích: Chỉ số đơn vượt phạm vi một trục mảng gây `IndexError`. Với mảng dài ba phần tử, chỉ số 3 không hợp lệ vì vị trí dương cuối là 2; truy cập không tự mở rộng mảng.

Q19. How does `X[mask]` differ from `X[[0,2]]`? — `X[mask]` khác `X[[0,2]]` ra sao?

A. Both necessarily select the same rows — Cả hai luôn lấy cùng hàng

B. A mask automatically sorts values — Mask tự sắp xếp giá trị

C. Indices always compare values to a threshold — Chỉ số luôn so giá trị với ngưỡng

D. A mask selects by condition; indices select by position — Mask chọn theo điều kiện; chỉ số chọn theo vị trí

Đáp án: D

Giải thích: Mask Boolean đánh dấu vị trí vượt qua phép kiểm tra, còn mảng chỉ số nguyên nêu thẳng các vị trí. Mask dựa trên điều kiện và `X[[0,2]]` không nhất thiết chọn cùng hàng.

Q20. Why state what rows and columns represent? — Vì sao cần nêu rõ ý nghĩa hàng và cột?

A. To make all arrays immutable — Để mọi mảng bất biến

B. To force arrays into 1-D — Để buộc mảng thành một chiều

C. To avoid using dtype — Để tránh dtype

D. To select the intended data without swapping dimensions — Để chọn đúng dữ liệu và không đảo chiều

Đáp án: D

Giải thích: Ý nghĩa từng trục quyết định lát cắt nào trả lời đúng vấn đề. Nếu hàng là học viên và cột là môn, `X[:,0]` lấy điểm môn đầu của mọi học viên, còn `X[0,:]` lấy điểm các môn của học viên đầu.

8. Nối bài tiếp theo

`NumPy ndarray → Chọn hàng/cột → DataFrame/PCA.`

PY-15 — NumPy tổng hợp, axis và broadcasting

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Aggregate — phép tổng hợp; **Axis** — trục tính.

`Index/Slice → Aggregate/Broadcast → Vectorization → AI`

Học xong dùng để làm gì? Tính trung bình, tổng và biến đổi hàng loạt các cột số có shape tương thích.

Kết nối kiến thức tiếp theo: PY-16 đổi shape, chuyển vị và nhân ma trận để xử lý dữ liệu dạng khối.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Aggregate	phép tổng hợp	Gom nhiều giá trị thành chỉ số thống kê
Axis	trục tính	Chiều dọc theo phép gom giá trị
Broadcasting	quảng bá mảng	Mở rộng kích thước tương thích khi tính
Vectorization	vector hóa	Tính cả mảng không cần viết vòng for từng phần tử

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`np.sum(X,axis=0)`	Tổng theo từng cột, axis=1 tổng theo hàng
`np.mean()/np.max()/np.min()`	Tổng hợp trung bình, cực đại và cực tiểu
`X + scalar / X + vector`	Broadcast nếu shape tương thích
`np.sqrt()/np.exp()`	Biến đổi từng phần tử, chú ý miền giá trị

4. Khái niệm liên quan

Sau khi biết tạo và chọn ô trong `ndarray`, ta cần **tổng hợp và biến đổi nhiều giá trị cùng lúc**. Phần này tiếp nối bảng doanh thu và điểm số: tổng hợp để rút ra thống kê, chọn `axis` để biết tổng hợp theo hàng hay cột, rồi dùng broadcasting và vectorization để xử lý đồng loạt mà không viết từng phép tính.

4.1. Aggregation — Tổng hợp từ nhiều giá trị

Từ vựng trong mục: Aggregation — tổng hợp; **Sum/mean** — tổng/trung bình.

Khi quản lý doanh thu, ta thường không cần xem từng giao dịch mà muốn biết **tổng**, **trung bình** hoặc **cao nhất**. Đó là **phép tổng hợp (aggregation)**: kết hợp nhiều giá trị thành một giá trị thống kê. Với doanh thu ba ngày `[10,20,30]` triệu đồng, `sum()` cho 60 triệu đồng; `mean()` cho 20 triệu đồng/ngày; `max()` cho 30 triệu đồng. Không nhầm tổng với trung bình.

```
import numpy as np

sales = np.array([10, 20, 30])
print(sales.sum(), sales.mean(), sales.max())
```

Kết quả khi chạy:

```
60 20.0 30
```

Tổng 60 triệu đồng, trung bình 20 triệu đồng/ngày và ngày cao nhất 30 triệu đồng.

4.2. Axis — Tổng hợp theo hàng hoặc cột

Từ vựng trong mục: Axis — trục tính; **Row/column** — hàng/cột.

Trong bảng doanh thu có **hai hàng cửa hàng** A, B và **ba cột ngày**, `sales.sum(axis=1)` tính tổng ba ngày trên từng **hàng cửa hàng**; còn `sales.sum(axis=0)` tính tổng hai cửa hàng theo từng **cột ngày**. `axis=0` gom **qua các hàng**, `axis=1` gom **qua các cột**. Vì vậy trước khi chọn axis, phải biết mỗi chiều đang mô tả gì.

```
import numpy as np

sales = np.array([[10, 20, 30], [8, 12, 16]])
print(sales.sum(axis=1))
```

Kết quả khi chạy:

```
[60 36]
```

Kết quả có hai phần tử vì mảng có hai hàng: cửa hàng A tổng 60 và B tổng 36 triệu đồng.

4.3. Broadcasting — Áp dụng phép toán theo chiều tương thích

Từ vựng trong mục: Broadcasting — quảng bá mảng; **Shape** — kích thước.

Broadcasting cho phép NumPy thực hiện phép toán giữa các đối tượng khác kích thước **nếu các chiều tương thích**. Ví dụ một trường cộng 1 điểm chuyên cần cho từng điểm trong bảng hai học viên × ba môn: phép `scores + 1` tự áp dụng 1 vào mọi ô, không cần vòng `for`. Khi một chiều có kích thước 1 hoặc bằng chiều tương ứng, NumPy thường mở rộng được; các chiều không tương thích sẽ báo `ValueError`.

```
import numpy as np

scores = np.array([[7, 8, 9], [6, 7, 8]])
print(scores + 1)
```

Kết quả khi chạy:

```
[[ 8  9 10]
 [ 7  8  9]]
```

Số 1 được cộng vào cả sáu ô, nhưng shape vẫn là 2 hàng × 3 cột.

4.4. Vectorization — Tính trực tiếp trên mảng

Từ vựng trong mục: Vectorization — vector hóa; **Element-wise** — theo từng phần tử.

Vectorization là viết phép tính trực tiếp trên toàn mảng thay vì tự lặp từng ô bằng Python. Khi ba kênh cảm biến có giá trị `[10,20,30]` và hệ số hiệu chỉnh `[1.0,0.5,2.0]`, phép `signals * factors` nhân các phần tử **cùng vị trí**: 10×1, 20×0,5 và 30×2. Đây là phép **nhân từng phần tử**, khác phép nhân ma trận `@`.

```
import numpy as np

signals = np.array([10, 20, 30])
factors = np.array([1.0, 0.5, 2.0])
print(signals * factors)
```

Kết quả khi chạy:

```
[10. 10. 60.]
```

Ba số là tín hiệu sau khi áp dụng hệ số riêng cho từng kênh; NumPy không cộng các kênh lại.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Tổng doanh thu từng cửa hàng

Từ vựng trong mục: Aggregation — tổng hợp; **Axis** — trục tính.

Tình huống thực tế: Hai cửa hàng A và B có doanh thu ba ngày lần lượt là `[10,20,30]` và `[8,12,16]`, đơn vị triệu đồng. Hãy tính tổng cho **mỗi cửa hàng** để so sánh.

Vì sao chọn cách giải này? Hai cửa hàng ở hai hàng, còn ba ngày ở các cột; `sum(axis=1)` tổng hợp ba ngày của mỗi hàng.

Quy trình giải:

1. Tạo mảng 2 hàng × 3 cột, hàng đầu là A, hàng sau là B.
2. Dùng `sum(axis=1)` cộng qua ba cột ngày.
3. In hai tổng theo thứ tự A rồi B.

```
import numpy as np

sales = np.array([[10, 20, 30], [8, 12, 16]])
print(sales.sum(axis=1))
```

Kết quả khi chạy:

```
[60 36]
```

Doanh thu ba ngày của A là 60 triệu, B là 36 triệu đồng.

Tình huống 2 — Trung bình theo từng môn

Từ vựng trong mục: Mean — trung bình; **Axis** — trục.

Tình huống thực tế: Ba học viên An, Bình, Lan có điểm Toán/Python lần lượt (8,9), (6,7), (9,8) trên thang 10. Hãy tính trung bình **theo từng môn**, không phải từng học viên.

Vì sao chọn cách giải này? Môn học ở các cột nên `mean(axis=0)` tổng hợp các hàng học viên cho từng cột môn.

Quy trình giải:

1. Tạo mảng 3 hàng học viên × 2 cột môn Toán, Python.
2. Chọn `axis=0` để lấy trung bình từng cột.
3. In hai điểm trung bình.

```
import numpy as np

scores = np.array([[8, 9], [6, 7], [9, 8]], dtype=float)
print(scores.mean(axis=0))
```

Kết quả khi chạy:

```
[7.66666667 8.        ]
```

Điểm trung bình Toán ≈ 7,67; Python là 8,0.

Tình huống 3 — Cộng điểm chuyên cần có giới hạn

Từ vựng trong mục: Broadcasting — quảng bá; **Clip** — giới hạn giá trị.

Tình huống thực tế: Trường có hai học viên, ba môn; điểm hiện tại lần lượt `[7,8,9]`, `[6,7,8]`. Mỗi điểm được cộng 1 điểm chuyên cần nhưng không được vượt 10. Hãy in bảng sau cộng.

Vì sao chọn cách giải này? Broadcasting cộng 1 vào mọi ô; `np.minimum(...,10)` bảo đảm mỗi ô không vượt điểm tối đa.

Quy trình giải:

1. Lưu điểm trong mảng 2 hàng × 3 cột.
2. Cộng 1 cho toàn bộ mảng theo broadcasting.
3. Áp dụng `np.minimum` giới hạn điểm 10 rồi in kết quả.

```
import numpy as np

scores = np.array([[7, 8, 9], [6, 7, 8]])
print(np.minimum(scores + 1, 10))
```

Kết quả khi chạy:

```
[[ 8  9 10]
 [ 7  8  9]]
```

Mỗi ô tăng một điểm; điểm 9 chỉ lên 10, không vượt giới hạn.

Tình huống 4 — Hiệu chỉnh ba kênh cảm biến

Từ vựng trong mục: Vectorization — vector hóa; **Element-wise** — theo phần tử.

Tình huống thực tế: Ba kênh cảm biến có giá trị `[10,20,30]` và hệ số hiệu chỉnh tương ứng `[1.0,0.5,2.0]`. Hãy tính tín hiệu sau hiệu chỉnh của từng kênh.

Vì sao chọn cách giải này? Nhân hai mảng cùng shape sẽ nhân từng vị trí tương ứng, đơn giản hơn viết ba phép nhân riêng.

Quy trình giải:

1. Tạo mảng tín hiệu và mảng hệ số theo cùng thứ tự kênh.
2. Nhân hai mảng trực tiếp bằng `*`.
3. In ba giá trị sau hiệu chỉnh.

```
import numpy as np

signals = np.array([10, 20, 30])
factors = np.array([1.0, 0.5, 2.0])
print(signals * factors)
```

Kết quả khi chạy:

```
[10. 10. 60.]
```

Tín hiệu lần lượt là 10, 10 và 60 theo đơn vị của từng kênh.

Tình huống 5 — Tìm giá trị cao nhất theo cột

Từ vựng trong mục: Aggregation — tổng hợp; **Max** — cực đại.

Tình huống thực tế: Bảng có ba ngày đo và hai kênh cảm biến: các hàng là `(2,4)`, `(5,1)`, `(3,6)` theo cùng đơn vị đo trên từng kênh. Hãy tìm mức lớn nhất của **mỗi kênh**, không phải mỗi ngày.

Vì sao chọn cách giải này? Kênh nằm theo cột, nên `max(axis=0)` lấy giá trị lớn nhất trong các hàng cho mỗi cột.

Quy trình giải:

1. Tạo bảng ba hàng ngày × hai cột kênh.
2. Dùng `max(axis=0)` lấy cực đại từng cột.
3. In hai số cực đại theo thứ tự kênh.

```
import numpy as np

values = np.array([[2, 4], [5, 1], [3, 6]])
print(values.max(axis=0))
```

Kết quả khi chạy:

```
[5 6]
```

Kênh thứ nhất có giá trị tối đa 5, kênh thứ hai tối đa 6.

6. Sai lầm thường gặp

- Với mảng hai chiều, `axis=0` tổng hợp **theo cột** (gom các hàng), còn `axis=1` tổng hợp **theo hàng** (gom các cột). Không đoán axis chỉ từ tên hàm.
- `sum`, `mean`, `max` có ý nghĩa khác nhau: tổng, trung bình và cực đại, không thể thay thế tùy tiện.
- Broadcasting chỉ hoạt động khi **shape tương thích**. Hai mảng có chiều không tương thích có thể phát sinh `ValueError`.
- Dấu `*` của NumPy nhân **từng phần tử**, không phải phép nhân ma trận `@`.
- Khi cộng điểm, phải áp dụng giới hạn tối đa nếu quy định yêu cầu, ví dụ `np.minimum(scores + 1, 10)`.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is aggregation in NumPy? — Phép tổng hợp trong NumPy là gì?

A. Reading every file in a folder — Đọc mọi file trong thư mục

B. Converting all arrays to strings — Đổi mọi mảng thành chuỗi

C. Creating a new class by inheritance — Tạo lớp con nhờ kế thừa

D. Combining multiple values into summary statistics — Kết hợp nhiều giá trị thành thống kê tổng hợp

Đáp án: D

Giải thích: Tổng hợp gộp nhiều giá trị số thành thống kê như `sum`, `mean` hoặc `max`. Với `[2, 4, 6]`, trung bình `4` tóm tắt ba giá trị thay vì giữ từng ô.

Q2. How does `sum()` differ from `mean()`? — `sum()` khác `mean()` ra sao?

A. Both always return the same result — Cả hai luôn bằng nhau

B. sum totals values; mean computes their average — sum cộng tổng; mean tính trung bình

C. sum sorts; mean deletes values — sum sắp xếp; mean xóa giá trị

D. mean returns the largest number — mean trả số lớn nhất

Đáp án: B

Giải thích: `sum` cộng tất cả giá trị được chọn, còn `mean` lấy tổng chia cho số giá trị. Với `[2, 4, 6]`, tổng bằng `12` nhưng trung bình bằng `4`; hai phép không thường bằng nhau.

Q3. What does `max()` return? — `max()` trả về gì?

A. The arithmetic average — Trung bình cộng

B. The sum of every negative value — Tổng các số âm

C. The largest value overall or along a specified axis — Giá trị lớn nhất toàn mảng hoặc theo trục được chỉ định

D. The number of columns only — Chỉ số cột

Đáp án: C

Giải thích: `max` chọn giá trị lớn nhất của mảng hoặc riêng theo trục được chỉ định. Với `[[1, 7], [5, 3]]`, cực đại toàn mảng là `7`, khác `mean` dùng để tính trung bình.

Q4. In a 2-D array, what does `sum(axis=0)` return? — Trong mảng hai chiều, `sum(axis=0)` trả về gì?

A. One sum per row — Một tổng cho mỗi hàng

B. One sum per column — Một tổng cho mỗi cột

C. Only the first row — Chỉ hàng đầu

D. A boolean mask — Mặt nạ Boolean

Đáp án: B

Giải thích: Với mảng 2D, `axis=0` gộp theo chiều hàng và để lại mỗi cột một kết quả. Với `[[1, 2], [3, 4]]`, tổng cột là `[4, 6]`, không phải tổng hàng `[3, 7]`.

Q5. In a 2-D array, what does `sum(axis=1)` return? — Trong mảng hai chiều, `sum(axis=1)` trả về gì?

A. The data type of values — Kiểu dữ liệu

B. One sum per column — Một tổng cho mỗi cột

C. Only a scalar index — Chỉ một chỉ số

D. One sum per row — Một tổng cho mỗi hàng

Đáp án: D

Giải thích: `axis=1` gộp các cột trong từng hàng. Với `[[1, 2], [3, 4]]`, `sum(axis=1)` cho tổng hàng `[3, 7]`; `axis=0` lại cho tổng theo cột.

Q6. What does broadcasting allow? — Broadcasting cho phép làm gì?

A. Automatically converting all arrays to CSV — Tự chuyển mọi mảng thành CSV

B. Operating on arrays with compatible shapes — Thực hiện phép toán trên các mảng có shape tương thích

C. Replacing a function with a class — Thay hàm bằng lớp

D. Using any shapes without restrictions — Dùng mọi shape không giới hạn

Đáp án: B

Giải thích: Broadcasting ghép các chiều có kích thước tương thích để tính theo phần tử mà không phải viết vòng lặp Python. Ví dụ ma trận `(2,3)` cộng được vector `(3,)` trên các hàng, nhưng vector `(2,)` không khớp chiều cuối bằng 3.

Q7. Why does `a + 1` work on a numeric ndarray? — Vì sao `a + 1` hoạt động với ndarray số?

A. The array is converted to a string — Mảng bị chuyển thành chuỗi

B. Only the first element is changed — Chỉ phần tử đầu thay đổi

C. The scalar broadcasts to every array element — Số 1 được broadcast tới từng phần tử

D. It performs matrix multiplication — Nó nhân ma trận

Đáp án: C

Giải thích: Số đơn không có trục mảng gây xung đột nên NumPy broadcast nó tới mọi phần tử. Với `a=[2,4]`, `a + 1` cho `[3,5]`, không phải nhân ma trận hay chỉ sửa phần tử đầu.

Q8. Which pair of trailing dimensions is broadcast-compatible? — Hai chiều cuối nào tương thích broadcasting?

A. Only both zero — Chỉ khi cùng bằng 0

B. Equal sizes or one size equal to 1 — Bằng nhau hoặc một chiều có kích thước 1

C. Always 2 and 3 — Luôn là 2 và 3

D. Any unrelated positive sizes — Hai số dương bất kỳ

Đáp án: B

Giải thích: NumPy đối chiếu các chiều từ phải sang trái: mỗi cặp phải bằng nhau hoặc một bên có kích thước `1`. Vì vậy chiều `3` và `1` tương thích, còn `3` và `2` trên cùng trục thường không tương thích.

Q9. What happens for incompatible broadcast shapes? — Điều gì xảy ra khi hai shape không tương thích broadcasting?

A. A ValueError is generally raised — Thường phát sinh ValueError

B. NumPy always returns None — NumPy luôn trả None

C. NumPy silently discards extra rows — NumPy tự bỏ hàng thừa

D. A CSV file is created — Một file CSV được tạo

Đáp án: A

Giải thích: Shape không tương thích không thể ghép cho phép toán theo phần tử, nên NumPy thường gây `ValueError`. Ví dụ cộng `(2,3)` với `(2,)` lỗi vì hai chiều cuối 3 và 2 không khớp.

Q10. What is vectorization? — Vectorization là gì?

A. Removing all calculations — Loại bỏ mọi phép tính

B. Applying numerical operations to arrays directly — Áp dụng phép toán số trực tiếp lên mảng

C. Turning every number into a set — Biến mọi số thành Set

D. Manually printing each item — Tự in từng phần tử

Đáp án: B

Giải thích: Vector hóa áp dụng phép toán trực tiếp trên mảng NumPy thay vì viết vòng lặp Python qua từng phần tử. Ví dụ `a * 2` nhân mọi ô số, còn tổng hợp như `a.sum()` gộp chúng thành một thống kê.

Q11. What does NumPy `a * b` do for equal-shaped numeric arrays? — NumPy `a * b` làm gì với hai mảng số cùng shape?

A. Multiply corresponding elements — Nhân các phần tử cùng vị trí

B. Concatenate the arrays — Nối hai mảng

C. Always compute matrix multiplication — Luôn nhân ma trận

D. Compare the shapes only — Chỉ so sánh shape

Đáp án: A

Giải thích: Với mảng số NumPy, `*` nhân các ô tương ứng và có thể dùng broadcasting. Vì vậy hai mảng `[2,3]` và `[4,5]` nhân theo phần tử cho `[8,15]`; phép nhân ma trận dùng `@`.

Q12. Which operator is used for matrix multiplication? — Toán tử nào dùng để nhân ma trận?

A. `and` — `and`

B. `%` — `%`

C. `@` — `@`

D. `*` alone — Chỉ `*`

Đáp án: C

Giải thích: Toán tử `@` thực hiện nhân ma trận theo quy tắc đại số tuyến tính, kết hợp hàng với cột. Khác `*`, nó không chỉ nhân các phần tử ở cùng vị trí.

Q13. For finite numeric array values, what does `np.minimum(a, 10)` do? — Với mảng chứa các giá trị số hữu hạn, `np.minimum(a, 10)` làm gì?

A. Sum all values and multiply by 10 — Cộng tổng rồi nhân 10

B. Select the largest value only — Chỉ chọn số lớn nhất

C. Remove every element below 10 — Xóa mọi phần tử dưới 10

D. Cap each value at 10 — Giới hạn mỗi giá trị tối đa 10

Đáp án: D

Giải thích: Với giá trị số hữu hạn, `np.minimum(a, 10)` so từng ô với 10 rồi lấy số nhỏ hơn. Vì vậy `[8,12]` thành `[8,10]`; nó không xóa phần tử nhỏ hơn 10.

Q14. How does `np.min()` differ from `np.minimum()`? — `np.min()` khác `np.minimum()` thế nào?

A. min performs sorting descending — min sắp xếp giảm dần

B. Both always return the same shape — Cả hai luôn trả cùng shape

C. min reduces values; minimum compares values element-wise — min tổng hợp lấy cực tiểu; minimum so từng phần tử

D. minimum reads files; min writes files — minimum đọc file, min ghi file

Đáp án: C

Giải thích: `np.min(a)` mặc định gộp nhiều giá trị thành một cực tiểu, hoặc lấy cực tiểu theo trục được chọn. `np.minimum(a,b)` so từng ô tương ứng có thể dùng broadcasting và thường trả mảng kết quả.

Q15. For shape `(2,3)`, what is the shape of `sum(axis=1)`? — Với shape `(2,3)`, `sum(axis=1)` có shape nào?

A. `(2,3)` — `(2,3)`

B. `(1,3)` — `(1,3)`

C. `(3,)` — `(3,)`

D. `(2,)` — `(2,)`

Đáp án: D

Giải thích: Ma trận shape `(2,3)` có hai hàng, mỗi hàng gồm ba số. `sum(axis=1)` cộng ba cột trong từng hàng, để lại hai tổng với shape `(2,)`.

Q16. For shape `(2,3)`, what is the shape of `mean(axis=0)`? — Với shape `(2,3)`, `mean(axis=0)` có shape nào?

A. `(3,2)` — `(3,2)`

B. `(2,3)` — `(2,3)`

C. `(2,)` — `(2,)`

D. `(3,)` — `(3,)`

Đáp án: D

Giải thích: Với mảng `(2,3)`, `mean(axis=0)` lấy trung bình qua hai hàng cho từng cột. Còn lại ba trung bình cột, nên shape kết quả mặc định là `(3,)`, không phải `(2,)`.

Q17. Which expression adds a distinct offset to each column of a `(2,3)` matrix? — Biểu thức nào cộng một giá trị riêng cho mỗi cột của ma trận (2,3)?

A. `X + np.array([1,2,3])` — `X + np.array([1,2,3])`

B. `X.shape + 1` — `X.shape + 1`

C. `X @ np.array([1,2,3])` — `X @ np.array([1,2,3])`

D. `X + np.array([1,2])` — `X + np.array([1,2])`

Đáp án: A

Giải thích: Vector bù dài ba khớp chiều cuối gồm ba cột của ma trận `(2,3)`. Broadcasting cộng `[1,2,3]` vào cả hai hàng, mỗi cột một mức riêng; vector dài hai không khớp chiều ba.

Q18. Which statement about broadcasting is correct? — Phát biểu nào đúng về broadcasting?

A. It may apply a smaller operand without physically copying it — Nó có thể áp toán hạng nhỏ mà không cần sao chép vật lý

B. It permanently changes the array shape in every case — Nó luôn thay shape vĩnh viễn

C. It always duplicates the entire array in memory first — Nó luôn sao chép toàn mảng trước

D. It requires writing explicit for loops — Nó bắt buộc viết vòng for

Đáp án: A

Giải thích: Broadcasting có thể tái sử dụng toán hạng nhỏ trên các chiều tương thích mà không cần nhân bản vật lý toàn bộ dữ liệu trước. Nó khác việc phải viết vòng lặp hoặc đổi shape của đầu vào vĩnh viễn.

Q19. How do aggregation and vectorization differ? — Aggregation khác vectorization thế nào?

A. Aggregation summarizes values; vectorization operates over arrays — Aggregation tổng hợp giá trị; vectorization tính trực tiếp trên mảng

B. Both mean sorting only — Cả hai chỉ là sắp xếp

C. Aggregation never uses NumPy — Aggregation không bao giờ dùng NumPy

D. Vectorization must reduce to one number — Vectorization luôn phải trả một số

Đáp án: A

Giải thích: Tổng hợp gộp các giá trị thành thống kê, thường giảm một trục, còn phép toán vector hóa tính trực tiếp trên nhiều ô. Ví dụ `a.mean()` tóm tắt dữ liệu, còn `a * 2` biến đổi từng phần tử.

Q20. Why should data meaning be known before choosing `axis`? — Vì sao phải hiểu ý nghĩa dữ liệu trước khi chọn `axis`?

A. It avoids specifying array shape — Nó giúp khỏi nêu shape

B. It ensures every value is positive — Nó bảo đảm mọi giá trị dương

C. It determines whether rows or columns answer the question — Nó quyết định cần tổng hợp theo hàng hay cột

D. It changes Python syntax rules — Nó đổi quy tắc cú pháp Python

Đáp án: C

Giải thích: Chọn trục đúng phụ thuộc ý nghĩa hàng và cột. Nếu hàng là học viên, cột là môn, `mean(axis=0)` tính trung bình từng môn qua học viên, còn `mean(axis=1)` tính trung bình các môn của mỗi học viên.

8. Nối bài tiếp theo

`Index/Slice → Aggregate/Broadcast → Vectorization → AI.`

PY-16 — NumPy reshape, transpose, ma trận và lưu file

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Reshape — đổi hình dạng; **Transpose** — chuyển vị.

`ndarray → Reshape/Transpose → Matrix → Model Input`

Học xong dùng để làm gì? Chuyển mảng thành ma trận, nhân ma trận và lưu dữ liệu NPY để dùng lại mà không mất shape/dtype.

Kết nối kiến thức tiếp theo: PY-17 đưa dữ liệu vào DataFrame có nhãn hàng và tên cột.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Reshape	đổi hình dạng	Đổi shape nhưng giữ số phần tử
Transpose	chuyển vị	Đổi trục hàng và cột
Matrix multiply	nhân ma trận	Tạo kết quả bằng phép nhân tuyến tính
Serialization	lưu dữ liệu	Ghi mảng vào tệp để đọc lại

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`reshape(rows,cols)`	Số phần tử trước/sau phải bằng nhau
`X.T / np.transpose(X)`	Đổi trục từ hàng thành cột
`A @ B / np.dot(A,B)`	Nhân tuyến tính yêu cầu kích thước tương thích
`np.save()/np.load()`	Lưu và đọc định dạng NPY, chỉ tải từ nguồn tin cậy

4. Khái niệm liên quan

Từ một dãy nhiệt độ hoặc bảng điểm, ta cần biết **bố trí lại dữ liệu**, **đổi chiều quan sát**, **tính điểm tổng hợp** và **lưu để sử dụng lại**. Bốn khái niệm này nối nhau: `reshape` bố trí hàng/cột, `.T` chuyển vị, `@` kết hợp các chiều đúng quy tắc, còn `np.save`/`np.load` lưu và phục hồi mảng.

4.1. Reshape — Đổi hình dạng mảng

Từ vựng trong mục: Reshape — đổi hình dạng; **Shape** — kích thước.

Reshape đổi cách sắp xếp một dãy phần tử thành các hàng/cột mới **mà không đổi số phần tử**. Ví dụ máy đo có 6 nhiệt độ theo thứ tự thời gian `[20,21,22,23,24,25]` °C. Nếu mỗi hàng đại diện một ngày và mỗi cột là một giờ đo, `reshape(2,3)` tạo bảng **2 ngày × 3 giờ**. Ngày đầu chứa ba số đầu, ngày hai chứa ba số sau. Không thể reshape 6 số thành `(4,2)` vì cần 8 số.

```
import numpy as np

temperatures = np.array([20, 21, 22, 23, 24, 25])
print(temperatures.reshape(2, 3))
```

Kết quả khi chạy:

```
[[20 21 22]
 [23 24 25]]
```

Mỗi hàng là một ngày, mỗi cột là một giờ đo theo thứ tự. Shape đổi từ `(6,)` thành `(2,3)`, còn thứ tự giá trị không đổi.

4.2. Transpose — Hoán đổi ý nghĩa hàng và cột

Từ vựng trong mục: Transpose — chuyển vị; **Row/column** — hàng/cột.

Transpose (`.T`) tráo hai trục của bảng hai chiều. Ví dụ bảng `scores` có **2 hàng học viên** An, Bình và **3 cột môn** Toán, Văn, Python: `[8,7,9]` và `[6,5,8]`. Sau `.T`, bảng thành **3 hàng môn** và **2 cột học viên**. Các điểm giữ nguyên nhưng cách trình bày đổi, giúp thống kê theo môn dễ nhìn hơn. Khác `reshape`, chuyển vị đổi vị trí của các ô theo trục.

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.T)
```

Kết quả khi chạy:

```
[[8 6]
 [7 5]
 [9 8]]
```

Hàng đầu chứa điểm Toán của An và Bình, hàng hai là Văn, hàng ba là Python. Shape chuyển từ `(2,3)` thành `(3,2)`.

4.3. Matrix multiplication — Nhân ma trận

Từ vựng trong mục: Matrix multiplication — nhân ma trận; **Dot product** — tích vô hướng.

Toán tử `@` thực hiện **phép nhân ma trận**, không phải nhân từng ô như `*`. Với một học viên có điểm `[8,7,9]` cho ba môn và trọng số `[0.2,0.3,0.5]`, `scores @ weights` tính `8×0.2 + 7×0.3 + 9×0.5 = 8.2`. Với bảng nhiều học viên, các hàng là người và các cột là môn; ma trận trọng số phải có số hàng bằng số cột của bảng điểm, ví dụ `(2,3) @ (3,1) → (2,1)`.

```
import numpy as np

scores = np.array([8, 7, 9])
```

```
weights = np.array([0.2, 0.3, 0.5])
print(scores @ weights)
```

Kết quả khi chạy:

```
8.2
```

Điểm có trọng số là 8,2 trên thang 10. `@` nhân các thành phần tương ứng rồi cộng thành một điểm tổng kết.

4.4. Serialization — Lưu và đọc lại mảng

Từ vựng trong mục: Serialization — tuần tự hóa; **NPY** — định dạng lưu mảng NumPy.

Tuần tự hóa (serialization) là lưu dữ liệu trong bộ nhớ thành định dạng có thể đọc lại. NumPy dùng `np.save` ghi mảng thành tệp `.npy`, chứa cả **giá trị, shape và dtype**; `np.load` phục hồi dữ liệu. Ví dụ lưu ba điểm `[3,5,7]` vào thư mục tạm rồi đọc lại để kiểm tra. `allow_pickle=False` phù hợp khi chỉ cần dữ liệu số, tránh đọc đối tượng Python được pickle hóa từ nguồn không tin cậy. `tolist()` chỉ đổi mảng thành List, **không lưu file**.

```
import numpy as np
from pathlib import Path
from tempfile import TemporaryDirectory

scores = np.array([3, 5, 7])
with TemporaryDirectory() as folder:
    file_path = Path(folder) / 'scores.npy'
    np.save(file_path, scores)
    restored = np.load(file_path, allow_pickle=False)
    print(restored)
```

Kết quả khi chạy:

```
[3 5 7]
```

Mảng được ghi ra tệp `.npy` rồi đọc lại đủ ba phần tử. Thư mục tạm tự được dọn sau khi thoát khỏi `with`.

5. Năm tình huống thực tế

Tình huống 1 — Tổ chức dữ liệu đo theo giờ

Từ vựng trong mục: Reshape — đổi hình dạng.

Tình huống thực tế: Thiết bị ghi sáu mức nhiệt độ 20, 21, 22, 23, 24, 25 °C theo thứ tự. Sắp dữ liệu thành bảng hai hàng, ba cột.

Vì sao chọn cách giải này? `reshape(2,3)` giữ nguyên 6 số nhưng đổi hình dạng.

Quy trình giải:

1. Ghi sáu giá trị nhiệt độ theo thứ tự giờ đo.
2. Đổi thành bảng hai ngày × ba giờ bằng `reshape(2, 3)`.
3. In bảng và kiểm tra ngày đầu chứa ba số đầu, ngày sau chứa ba số cuối.

```
import numpy as np
values = np.array([20, 21, 22, 23, 24, 25])
print(values.reshape(2, 3))
```

Kết quả khi chạy:

```
[[20 21 22]
 [23 24 25]]
```

Tình huống 2 — Đổi hàng cột báo cáo

Từ vựng trong mục: Transpose — chuyển vị.

Tình huống thực tế: Trung tâm có bảng điểm hai học viên, ba môn: `(8,7,9)` và `(6,5,8)`. Chuyển hàng thành cột và in kích thước kết quả.

Vì sao chọn cách giải này? `.T` chuyển từ bảng 2×3 thành 3×2.

Quy trình giải:

1. Tạo bảng điểm gồm hai hàng học viên và ba cột môn.
2. Dùng `.T` đổi thành ba hàng môn × hai cột học viên.
3. In `scores.T.shape` để xác minh `(3, 2)`.

```
import numpy as np
scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.T.shape)
```

Kết quả khi chạy:

```
(3, 2)
```

Tình huống 3 — Điểm tổng kết có trọng số

Từ vựng trong mục: Matrix multiply — nhân ma trận.

Tình huống thực tế: Một học viên có điểm ba thành phần `[8,7,9]`, trọng số tương ứng `[0.2,0.3,0.5]`. Tính điểm tổng kết theo trọng số.

Vì sao chọn cách giải này? Tích vô hướng `@` nhân từng điểm với trọng số rồi cộng.

Quy trình giải:

1. Lưu ba điểm môn và ba trọng số theo đúng thứ tự.
2. Nhân tích vô hướng bằng toán tử `@`.
3. In điểm tổng kết, tính bằng `8×0.2 + 7×0.3 + 9×0.5`.

```
import numpy as np
scores = np.array([8, 7, 9])
w = np.array([.2, .3, .5])
print(scores@w)
```

Kết quả khi chạy:

```
8.2
```

Tình huống 4 — Tạo đặc trưng tổng hợp

Từ vựng trong mục: Matrix multiplication — nhân ma trận.

Tình huống thực tế: Hai học viên có ba điểm `(8,7,9)` và `(6,5,8)`, cần tạo hai điểm tổng hợp theo ma trận trọng số trong code. In bảng điểm tổng hợp của hai học viên.

Vì sao chọn cách giải này? Phép `X @ W` nhân mảng dữ liệu 2×3 với trọng số 3×2, cho bảng 2×2.

Quy trình giải:

1. Tạo bảng điểm 2×3 và ma trận trọng số 3×2.
2. Nhân `X @ W` để tạo hai đặc trưng tổng hợp trên mỗi học viên.
3. In bảng 2×2 và xác định từng hàng vẫn tương ứng một học viên.

```
import numpy as np
X = np.array([[8, 7, 9], [6, 5, 8]])
W = np.array([[.5, 0], [.5, .5], [0, .5]])
print(X@W)
```

Kết quả khi chạy:

```
[[7.5 8. ]
 [5.5 6.5]]
```

Tình huống 5 — Lưu và đọc mảng an toàn

Từ vựng trong mục: Serialization — lưu dữ liệu.

Tình huống thực tế: Nhà phân tích có dãy số `[3,5,7]` cần lưu để đọc lại. Ghi vào file tạm `.npy`, tải lại và in đúng các số.

Vì sao chọn cách giải này? `np.save` và `np.load(..., allow_pickle=False)` dùng định dạng mảng NumPy.

Quy trình giải:

1. Tạo mảng số và thư mục tạm làm nơi lưu.
2. Dùng `np.save` ghi `.npy`, `np.load(..., allow_pickle=False)` đọc lại.
3. In mảng đã khôi phục, đối chiếu với dữ liệu gốc.

```
import numpy as np
from pathlib import Path
from tempfile import TemporaryDirectory
with TemporaryDirectory() as folder:
    p = Path(folder)/"data.npy"
    np.save(p, np.array([3, 5, 7]))
    print(np.load(p, allow_pickle=False))
```

Kết quả khi chạy:

```
[3 5 7]
```

6. Sai lầm thường gặp

- `reshape` chỉ thành công khi **tích các kích thước mới bằng số phần tử ban đầu**. Sáu giá trị tạo được bảng 2×3, không phải 4×2.
- `reshape` và `.T` không tương đương: reshape nhóm lại các phần tử theo thứ tự, transpose tráo vị trí theo trục.
- `A * B` nhân từng ô khi broadcast được; `A @ B` nhân ma trận theo chiều tương thích. Với ma trận 2×3, bên phải cần có 3 hàng.
- Phải xác định rõ hàng là học viên/ngày, cột là môn/giờ đo trước khi đọc kết quả.
- `np.save` tạo file thực; `array.tolist()` chỉ chuyển sang List, không ghi file. Không bật `allow_pickle=True` với dữ liệu không đáng tin.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does `reshape()` change? — `reshape()` thay đổi điều gì?

A. The array's shape without changing its total element count — Hình dạng mảng mà không đổi số phần tử

B. Writes an array to an NPY file — Ghi mảng vào tệp NPY

C. Swaps rows and columns — Hoán đổi hàng và cột

D. Matrix multiplication — Phép nhân ma trận

Đáp án: A

Giải thích: reshape bố trí lại cùng các phần tử theo kích thước mới. Với sáu phần tử, đổi `(2,3)` thành `(3,2)` hợp lệ, nhưng đổi thành `(4,2)` thì không.

Q2. What does `.T` do to a 2-D array? — `.T` làm gì với mảng 2-D?

A. The array's shape without changing its total element count — Hình dạng mảng mà không đổi số phần tử

B. Writes an array to an NPY file — Ghi mảng vào tệp NPY

C. Matrix multiplication — Phép nhân ma trận

D. Swaps rows and columns — Hoán đổi hàng và cột

Đáp án: D

Giải thích: Chuyển vị tráo hai trục của ma trận. Với `[[1,2,3],[4,5,6]]`, chuyển vị tạo ba hàng lần lượt là `[1,4]`, `[2,5]`, `[3,6]`.

Q3. What does `A @ B` mean for compatible matrices? — `A @ B` nghĩa là gì với các ma trận tương thích?

A. Swaps rows and columns — Hoán đổi hàng và cột

B. Matrix multiplication — Phép nhân ma trận

C. The array's shape without changing its total element count — Hình dạng mảng mà không đổi số phần tử

D. Writes an array to an NPY file — Ghi mảng vào tệp NPY

Đáp án: B

Giải thích: Toán tử @ tuân theo quy tắc nhân ma trận chứ không nhân riêng từng ô. Nếu `A` có shape `(2,3)` và `B` là `(3,4)`, thì `A @ B` có shape `(2,4)`; dấu `*` lại nhân từng phần tử.

Q4. What does `np.save()` do? — `np.save()` làm gì?

A. Swaps rows and columns — Hoán đổi hàng và cột

B. Matrix multiplication — Phép nhân ma trận

C. Writes an array to an NPY file — Ghi mảng vào tệp NPY

D. The array's shape without changing its total element count — Hình dạng mảng mà không đổi số phần tử

Đáp án: C

Giải thích: NumPy lưu giá trị, shape và dtype ở định dạng NPY. Khác CSV văn bản, tệp NPY giữ thông tin riêng của NumPy để khôi phục đúng mảng.

Q5. Which shape specifically has two rows and three columns? — Shape nào có đúng hai hàng và ba cột?

A. `(3, 2)` — `(3, 2)`

B. `(2, 3)` — `(2, 3)`

C. `(6,)` — `(6,)`

D. `(2, 4)` — `(2, 4)`

Đáp án: B

Giải thích: Chiều thứ nhất là số hàng và chiều thứ hai là số cột: (2,3). Shape `(2,3)` có tổng sáu ô; `(3,2)` cũng sáu ô nhưng số hàng và cột đảo vị trí.

Q6. What is the transpose shape of a `(2, 3)` matrix? — Chuyển vị ma trận `(2, 3)` có shape nào?

A. `(2, 3)` — `(2, 3)`

B. `(2, 4)` — `(2, 4)`

C. `(3, 2)` — `(3, 2)`

D. `(6,)` — `(6,)`

Đáp án: C

Giải thích: Chuyển vị đảo kích thước hàng và cột. Ma trận `(2,3)` trở thành `(3,2)` sau `.T` vì mỗi cột cũ trở thành một hàng.

Q7. What is the shape of `(2, 3) @ (3, 4)`? — Phép `(2, 3) @ (3, 4)` cho shape gì?

A. `(3, 2)` — `(3, 2)`

B. `(2, 3)` — `(2, 3)`

C. `(2, 4)` — `(2, 4)`

D. `(6,)` — `(6,)`

Đáp án: C

Giải thích: Hai chiều trong cùng bằng 3, còn hai chiều ngoài là 2 và 4. Chiều chung bằng 3 được kết hợp khi nhân, còn hai chiều ngoài 2 và 4 được giữ trong kết quả.

Q8. What shape has a one-dimensional array of six values? — Mảng một chiều có sáu giá trị có shape nào?

A. `(3, 2)` — `(3, 2)`

B. `(2, 4)` — `(2, 4)`

C. `(2, 3)` — `(2, 3)`

D. `(6,)` — `(6,)`

Đáp án: D

Giải thích: Mảng phẳng sáu phần tử có một chiều độ dài sáu. Dấu phẩy trong `(6,)` biểu diễn shape một chiều; `(1,6)` mới có hai chiều.

Q9. How does `reshape` differ from transpose? — reshape khác transpose ở đâu?

A. Reshape regroups positions; transpose exchanges axes — Reshape nhóm lại vị trí; transpose đổi trục

B. Converts the array into Python lists or scalars — Chuyển mảng thành List hoặc số Python

C. `*` multiplies element-wise; `@` performs matrix multiplication — `*` nhân từng phần tử; `@` nhân ma trận

D. Reads the saved array — Đọc lại mảng đã lưu

Đáp án: A

Giải thích: reshape và transpose có thể cho cùng shape nhưng giá trị ở vị trí khác. Với `[[1,2,3],[4,5,6]]`, `reshape(3,2)` tạo các hàng `[1,2]`, `[3,4]`, `[5,6]`, còn `.T` cho `[1,4]`, `[2,5]`, `[3,6]`.

Q10. How does `*` differ from `@` for numeric ndarrays? — `*` khác `@` thế nào với ndarray số?

A. Reads the saved array — Đọc lại mảng đã lưu

B. `*` multiplies element-wise; `@` performs matrix multiplication — `*` nhân từng phần tử; `@` nhân ma trận

C. Reshape regroups positions; transpose exchanges axes — Reshape nhóm lại vị trí; transpose đổi trục

D. Converts the array into Python lists or scalars — Chuyển mảng thành List hoặc số Python

Đáp án: B

Giải thích: Dấu sao nhân theo phần tử, @ tuân quy tắc nhân ma trận. Với hai ma trận 2×2 toàn số một, `A * B` vẫn toàn số một, còn `A @ B` toàn số hai do cộng các tích theo hàng–cột.

Q11. What does `np.load()` normally do for an NPY file? — `np.load()` làm gì với tệp NPY?

A. Reads the saved array — Đọc lại mảng đã lưu

B. Reshape regroups positions; transpose exchanges axes — Reshape nhóm lại vị trí; transpose đổi trục

C. `*` multiplies element-wise; `@` performs matrix multiplication — `*` nhân từng phần tử; `@` nhân ma trận

D. Converts the array into Python lists or scalars — Chuyển mảng thành List hoặc số Python

Đáp án: A

Giải thích: load phục hồi mảng NumPy đã ghi. Với tệp NPY đáng tin cậy, `np.load(path)` khôi phục mảng; tránh bật pickle cho dữ liệu đối tượng chưa tin cậy.

Q12. What does `array.tolist()` do? — `array.tolist()` làm gì?

A. Converts the array into Python lists or scalars — Chuyển mảng thành List hoặc số Python

B. Reshape regroups positions; transpose exchanges axes — Reshape nhóm lại vị trí; transpose đổi trục

C. `*` multiplies element-wise; `@` performs matrix multiplication — `*` nhân từng phần tử; `@` nhân ma trận

D. Reads the saved array — Đọc lại mảng đã lưu

Đáp án: A

Giải thích: tolist chuyển biểu diễn dữ liệu trong bộ nhớ chứ không ghi tệp. Mảng hai chiều chuyển thành List lồng nhau, còn mảng không chiều chuyển thành giá trị vô hướng Python.

Q13. What happens if you reshape six elements to `(4, 2)`? — Điều gì xảy ra nếu reshape sáu phần tử thành `(4, 2)`?

A. `allow_pickle=False` — `allow_pickle=False`

B. Left columns and right rows — Số cột bên trái và số hàng bên phải

C. A ValueError due to incompatible element counts — Phát sinh ValueError vì số phần tử không khớp

D. Shape and dtype — Shape và dtype

Đáp án: C

Giải thích: Bảng 4×2 cần tám giá trị, không phải sáu. Shape `(4,2)` cần `4 × 2 = 8` ô, nên NumPy không thể phân bố đúng sáu phần tử hiện có vào shape này.

Q14. What must match to multiply two matrices with `@`? — Muốn nhân hai ma trận bằng `@`, chiều nào phải khớp?

A. `allow_pickle=False` — `allow_pickle=False`

B. Left columns and right rows — Số cột bên trái và số hàng bên phải

C. A ValueError due to incompatible element counts — Phát sinh ValueError vì số phần tử không khớp

D. Shape and dtype — Shape và dtype

Đáp án: B

Giải thích: Nhân ma trận yêu cầu hai chiều trong bằng nhau. Nếu `A` là `(m,n)` và `B` là `(n,p)`, phép nhân hợp lệ và trả shape `(m,p)`.

Q15. Which option avoids loading pickled objects from an NPY file? — Tùy chọn nào tránh đọc đối tượng pickle trong file NPY?

A. A ValueError due to incompatible element counts — Phát sinh ValueError vì số phần tử không khớp

B. Left columns and right rows — Số cột bên trái và số hàng bên phải

C. Shape and dtype — Shape và dtype

D. `allow_pickle=False` — `allow_pickle=False`

Đáp án: D

Giải thích: Tắt pickle an toàn hơn khi đọc dữ liệu số chưa tin cậy. Với `np.load(path, allow_pickle=False)`, mảng object cần pickle sẽ không được đọc, giúp giảm rủi ro chạy mã từ tệp không tin cậy.

Q16. What does NPY preserve in addition to array values? — NPY lưu thêm gì ngoài các giá trị?

A. `allow_pickle=False` — `allow_pickle=False`

B. A ValueError due to incompatible element counts — Phát sinh ValueError vì số phần tử không khớp

C. Shape and dtype — Shape và dtype

D. Left columns and right rows — Số cột bên trái và số hàng bên phải

Đáp án: C

Giải thích: NPY lưu thông tin để khôi phục cấu trúc và kiểu phần tử. Khác các dòng văn bản thường, shape và dtype được lưu để mảng số `(2,3)` khôi phục đúng cấu trúc.

Q17. What is a 1-D dot product of two equal-length numeric vectors? — Tích vô hướng của hai vector một chiều bằng nhau độ dài là gì?

A. The sum of pairwise products — Tổng các tích của từng cặp phần tử

B. A 3-by-3 identity matrix — Ma trận đơn vị 3×3

C. `dtype` — `dtype`

D. `shape` — `shape`

Đáp án: A

Giải thích: Tích vô hướng nhân các vị trí tương ứng rồi cộng kết quả. Với `[1,2]` và `[3,4]`, tích vô hướng bằng `1×3 + 2×4 = 11`, là một số đơn chứ không phải List.

Q18. What does `np.eye(3)` create? — `np.eye(3)` tạo gì?

A. `shape` — `shape`

B. The sum of pairwise products — Tổng các tích của từng cặp phần tử

C. `dtype` — `dtype`

D. A 3-by-3 identity matrix — Ma trận đơn vị 3×3

Đáp án: D

Giải thích: Ma trận đơn vị có 1 ở đường chéo và 0 ở vị trí khác. Đường chéo chính là `[1,1,1]`, còn sáu ô ngoài chéo đều bằng không; nó không phải ma trận toàn số một.

Q19. Which property reports the array dimensions? — Thuộc tính nào cho biết kích thước các chiều?

A. A 3-by-3 identity matrix — Ma trận đơn vị 3×3

B. The sum of pairwise products — Tổng các tích của từng cặp phần tử

C. `dtype` — `dtype`

D. `shape` — `shape`

Đáp án: D

Giải thích: shape liệt kê độ dài của từng trục mảng. Với mảng 2D ba hàng hai cột, `X.shape` là `(3,2)` chứ không phải kiểu dữ liệu.

Q20. Which property reports element data type? — Thuộc tính nào cho biết kiểu phần tử?

A. `shape` — `shape`

B. `dtype` — `dtype`

C. A 3-by-3 identity matrix — Ma trận đơn vị 3×3

D. The sum of pairwise products — Tổng các tích của từng cặp phần tử

Đáp án: B

Giải thích: dtype ghi kiểu số hoặc kiểu dữ liệu khác của phần tử. Ví dụ mảng số nguyên có dtype nguyên, mảng số thập phân có dtype thực; `shape` mới cho kích thước các chiều.

8. Nối bài tiếp theo

`ndarray → Reshape/Transpose → Matrix → Model Input.`

PY-17 — Pandas Series, DataFrame và đọc CSV

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Series — chuỗi Pandas; **DataFrame** — bảng dữ liệu.

`NumPy ndarray → Pandas DataFrame → Làm sạch → EDA/AI`

Học xong dùng để làm gì? Đọc CSV thành bảng, kiểm tra kiểu và tên cột, tính kết quả theo từng bản ghi.

Kết nối kiến thức tiếp theo: PY-18 làm sạch giá trị thiếu, trùng và sai định dạng trước khi phân tích.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Series	chuỗi Pandas	Dữ liệu một chiều có nhãn chỉ mục
DataFrame	bảng dữ liệu	Bảng có hàng và cột đặt tên
Column	cột	Một feature mang ý nghĩa xác định
CSV	tệp giá trị phân cách	Định dạng văn bản cho bảng có header

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`pd.Series(values)`	Tạo dãy có chỉ mục
`pd.DataFrame(data)`	Tạo bảng từ dictionary/list record
`pd.read_csv(path)`	Đọc CSV có tiêu đề, quy định đường dẫn/encoding
`df.head()/df.info()/df.describe()`	Xem nhanh bản ghi, schema, thống kê

4. Khái niệm liên quan

Pandas giúp quản lý dữ liệu dạng bảng trước khi phân tích và huấn luyện AI. Ta bắt đầu từ một dãy có nhãn (**Series**), mở rộng thành bảng nhiều thuộc tính (**DataFrame**), chọn và tính trên từng cột, rồi đọc dữ liệu thật từ **CSV**. Trong mọi ví dụ, phải nêu rõ một hàng đại diện cho ai/cái gì và một cột là thuộc tính nào.

4.1. Series — Dữ liệu một chiều có nhãn

Từ vựng trong mục: Series — dãy có nhãn; **Index** — nhãn hàng.

Series là dãy một chiều của Pandas, trong đó mỗi giá trị gắn với một **index** (nhãn). Ví dụ theo dõi điểm hai học viên An, Bình: nhãn `An`, `Binh` xác định ai sở hữu giá trị 8 hay 6. Khác List thuần túy thường truy cập

bằng vị trí, Series còn truy cập theo **nhãn** bằng `.loc`. Khi không tự đặt nhãn, Pandas thường tạo chỉ số 0, 1, 2...

```
import pandas as pd

scores = pd.Series([8, 6], index=['An', 'Binh'])
print(scores.loc['Binh'])
```

Kết quả khi chạy:

```
6
```

Nhãn `Binh` trỏ tới giá trị điểm 6; nhãn là chỉ số hàng, không phải nội dung một cột riêng.

4.2. DataFrame — Bảng hàng và cột

Từ vựng trong mục: DataFrame — bảng dữ liệu; **Row/column** — hàng/cột.

DataFrame là bảng hai chiều. Cửa hàng có hai sản phẩm: Pen giá 10.000 đồng và Book giá 35.000 đồng. **Mỗi hàng** là một sản phẩm, **mỗi cột** là một thuộc tính (`Product` tên hàng; `Price` giá đơn vị đồng). Chỉ số 0, 1 bên trái là **index**, không phải một cột dữ liệu mặc định. Mỗi cột tương tự một Series có chung nhãn hàng.

```
import pandas as pd

products = pd.DataFrame({
    'Product': ['Pen', 'Book'],
    'Price': [10000, 35000],
})
print(products)
```

Kết quả khi chạy:

```
  Product  Price
0     Pen  10000
1    Book  35000
```

Bảng có 2 hàng sản phẩm × 2 cột thuộc tính; số 0 và 1 là nhãn index mặc định.

4.3. Column và dtype — Đọc và tính trên thuộc tính

Từ vựng trong mục: Column — cột; **dtype** — kiểu dữ liệu cột.

Cột là tập các giá trị của **một thuộc tính** trên nhiều hàng. Với đơn giá `[10,20]` nghìn đồng và số lượng `[2,3]` cái, ta tạo hai cột `price` và `qty` rồi thêm `revenue = price * qty`. Pandas thực hiện phép nhân **theo hàng có cùng index**. Đơn giá nhân số lượng tạo doanh thu 20 và 60 nghìn đồng. `df['price']` chọn một cột, còn `df[['price','qty']]` chọn bảng hai cột.

```
import pandas as pd

sales = pd.DataFrame({'price': [10, 20], 'qty': [2, 3]})
sales['revenue'] = sales['price'] * sales['qty']
print(sales['revenue'].tolist())
```

Kết quả khi chạy:

```
[20, 60]
```

Hai số là doanh thu của hàng thứ nhất và thứ hai. Nhân cột có ý nghĩa khi các giá trị đã căn đúng theo cùng hàng.

4.4. read_csv — Đọc bảng từ văn bản CSV

Từ vựng trong mục: CSV — bảng văn bản phân cách; **read_csv** — đọc CSV.

CSV lưu dữ liệu theo các dòng văn bản, thường dòng đầu chứa tên cột. `pd.read_csv()` chuyển nội dung thành DataFrame, có thể từ đường dẫn file hoặc bộ đệm `StringIO`. Ví dụ hai đơn hàng O1, O2 với `amount` 100 và 200 đồng; **mỗi hàng là một đơn**, cột `id` là mã đơn, cột `amount` là số tiền. Pandas thường tự nhận dạng cột số, nhưng dữ liệu phức tạp có thể cần chỉ định kiểu hoặc làm sạch trước khi tính.

```
import pandas as pd
from io import StringIO

csv_text = 'id,amount\nO1,100\nO2,200\n'
orders = pd.read_csv(StringIO(csv_text))
print(orders['amount'].sum())
```

Kết quả khi chạy:

```
300
```

Tổng số tiền hai đơn là 300 đồng. `read_csv` đọc bảng từ văn bản chứ không cần nhập lại thủ công từng dòng.

5. Năm tình huống thực tế

Tình huống 1 — Tạo bảng giá sản phẩm

Từ vựng trong mục: Series — chuỗi Pandas.

Tình huống thực tế: Cửa hàng quản lý hai mặt hàng: Pen giá 10.000 đồng, số lượng 2; Book giá 35.000 đồng, số lượng 1. Tạo và in bảng có cột Product, Price, Qty.

Vì sao chọn cách giải này? DataFrame đặt tên các cột, mỗi hàng là một sản phẩm.

Quy trình giải:

1. Tạo mỗi hàng là một sản phẩm với cột `Product`, `Price`, `Qty`.
2. Giữ đơn giá theo đồng và số lượng theo cái.
3. In DataFrame, kiểm tra Pen và Book nằm ở hai hàng.

```
import pandas as pd
df = pd.DataFrame({"Product":["Pen", "Book"], "Price":[10000, 35000], "Qty":[2, 1]})
print(df)
```

Kết quả khi chạy:

```
  Product  Price  Qty
0     Pen  10000    2
1    Book  35000    1
```

Tình huống 2 — Đọc đơn từ CSV văn bản

Từ vựng trong mục: DataFrame — bảng dữ liệu.

Tình huống thực tế: Ứng dụng nhận CSV có hai đơn O1 trị giá 10.000 và O2 trị giá 30.000 đồng. Đọc và in bảng hai đơn hàng.

Vì sao chọn cách giải này? `read_csv` chuyển CSV thành DataFrame; `head()` xem những hàng đầu.

Quy trình giải:

1. Lưu CSV có tiêu đề `id,amount` và hai đơn O1, O2.
2. Đưa văn bản vào `StringIO`, đọc bằng `pd.read_csv`.
3. In `head()` để kiểm tra cột mã đơn và số tiền.

```
import pandas as pd
import io
csv_text = "id,amount\nO1,10000\nO2,30000\n"
df = pd.read_csv(io.StringIO(csv_text))
print(df.head())
```

Kết quả khi chạy:

```
   id  amount
0  O1   10000
1  O2   30000
```

Tình huống 3 — Tạo doanh thu mỗi dòng

Từ vựng trong mục: Column — cột.

Tình huống thực tế: Hai sản phẩm có đơn giá `[10,20]` và số lượng `[2,3]` theo cùng đơn vị tiền. Thêm cột doanh thu mỗi dòng bằng giá nhân số lượng.

Vì sao chọn cách giải này? Pandas nhân từng hàng của hai cột Price và Qty.

Quy trình giải:

1. Tạo bảng hai hàng với cột `Price` và `Qty` cùng thứ tự sản phẩm.
2. Nhân hai cột để thêm `Revenue` theo từng hàng.
3. In bảng, kiểm tra doanh thu 20 và 60 theo đơn vị tiền đã chọn.

```
import pandas as pd
df = pd.DataFrame({"Price":[10, 20], "Qty":[2, 3]})
df["Revenue"] = df.Price*df.Qty
print(df)
```

Kết quả khi chạy:

```
   Price  Qty  Revenue
0     10    2       20
1     20    3       60
```

Tình huống 4 — Xem kiểu dữ liệu đầu vào

Từ vựng trong mục: CSV — tệp giá trị phân cách.

Tình huống thực tế: Bảng có tên A, B và số lượng 2, 3 sản phẩm. In kiểu dữ liệu của hai cột để kiểm tra.

Vì sao chọn cách giải này? `dtypes` cho biết cột chữ và cột số đang được lưu dưới kiểu nào.

Quy trình giải:

1. Tạo cột `name` gồm chữ và `qty` gồm số nguyên.
2. Truy cập `df.dtypes` để kiểm tra kiểu của từng cột.
3. Đối chiếu `name` dạng chuỗi và `qty` dạng số nguyên.

```
import pandas as pd
df = pd.DataFrame({"name":["A", "B"], "qty":[2, 3]})
print(df.dtypes)
```

Kết quả khi chạy:

```
name    object
qty      int64
dtype: object
```

Tình huống 5 — Chọn dữ liệu tập huấn luyện

Từ vựng trong mục: Series — chuỗi Pandas.

Tình huống thực tế: Bảng hai khách có `visits` là 2, 5; `spent` là 100, 200; `city` là A, B. Chọn hai cột visits và spent làm dữ liệu mô hình rồi in kích thước.

Vì sao chọn cách giải này? `df[[...]]` chọn hai cột, bỏ city khỏi đầu vào mẫu.

Quy trình giải:

1. Tạo bảng hai khách gồm `visits`, `spent`, `city`.
2. Chọn đúng hai cột số `visits`, `spent` làm đầu vào.
3. In `shape` của bảng chọn để thấy hai hàng × hai đặc trưng.

```
import pandas as pd
df = pd.DataFrame({"visits":[2, 5], "spent":[100, 200], "city":["A", "B"]})
X = df[["visits", "spent"]]
print(X.shape)
```

Kết quả khi chạy:

```
(2, 2)
```

6. Sai lầm thường gặp

- **Series** một chiều, **DataFrame** hai chiều. `df['x']` chọn Series; `df[['x']]` giữ dạng DataFrame một cột.
- Phải hiểu **index** là nhãn hàng, không nhất thiết là cột dữ liệu. Pandas căn dữ liệu theo index khi kết hợp các Series.
- Đơn giá, số lượng và doanh thu có **đơn vị khác nhau**; nhân `price × qty` mới có ý nghĩa là thành tiền.
- CSV vốn là văn bản; cần xem lại `dtype` vì dữ liệu bẩn có thể khiến cột số bị đọc thành chuỗi.
- Không đưa mã định danh hoặc nhãn `city` vào mô hình dự đoán chỉ vì chúng có mặt trong bảng; phải chọn đặc trưng theo mục tiêu.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a Pandas Series? — Pandas Series là gì?

A. Parse tabular CSV data into a DataFrame — Đọc dữ liệu CSV thành DataFrame

B. One field across multiple records — Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table — Bảng dữ liệu hai chiều có nhãn

D. A one-dimensional labeled data structure — Cấu trúc dữ liệu một chiều có nhãn

Đáp án: D

Giải thích: Series liên kết từng giá trị với một nhãn index. Ví dụ `pd.Series([5,6], index=['a','b'])` gắn nhãn `'a', 'b'` cho các giá trị, khác với List không có hệ nhãn riêng.

Q2. What is a Pandas DataFrame? — Pandas DataFrame là gì?

A. A one-dimensional labeled data structure — Cấu trúc dữ liệu một chiều có nhãn

B. A two-dimensional labeled table — Bảng dữ liệu hai chiều có nhãn

C. Parse tabular CSV data into a DataFrame — Đọc dữ liệu CSV thành DataFrame

D. One field across multiple records — Một thuộc tính qua nhiều bản ghi

Đáp án: B

Giải thích: DataFrame tổ chức bản ghi thành hàng và trường thành cột. Bảng gồm ba quan sát và hai trường có shape `(3,2)`; Series lại là cấu trúc một chiều có nhãn.

Q3. What does a column represent? — Một cột trong bảng thể hiện gì?

A. A one-dimensional labeled data structure — Cấu trúc dữ liệu một chiều có nhãn

B. One field across multiple records — Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table — Bảng dữ liệu hai chiều có nhãn

D. Parse tabular CSV data into a DataFrame — Đọc dữ liệu CSV thành DataFrame

Đáp án: B

Giải thích: Cột lưu cùng một thuộc tính ở các hàng. Nếu các hàng là đơn hàng, cột `price` ghi giá của từng đơn và cần xác định rõ đơn vị tiền tệ.

Q4. What is the purpose of `pd.read_csv()`? — `pd.read_csv()` dùng để làm gì?

A. A one-dimensional labeled data structure — Cấu trúc dữ liệu một chiều có nhãn

B. One field across multiple records — Một thuộc tính qua nhiều bản ghi

C. A two-dimensional labeled table — Bảng dữ liệu hai chiều có nhãn

D. Parse tabular CSV data into a DataFrame — Đọc dữ liệu CSV thành DataFrame

Đáp án: D

Giải thích: read_csv đọc các trường được phân cách cùng hàng tên cột. Ví dụ hàng tiêu đề CSV `name,score` tạo tên cột tương ứng, nhưng các ô số sai định dạng vẫn có thể cần làm sạch sau khi đọc.

Q5. What does `pd.Series([1,2])` return? — `pd.Series([1,2])` trả về gì?

A. A Series containing column x — Series chứa cột x

B. A Series with default index 0,1 — Series có index mặc định 0,1

C. A DataFrame containing one column — DataFrame có một cột

D. A tuple with row and column counts — Tuple gồm số hàng và số cột

Đáp án: B

Giải thích: Không chỉ định nhãn thì Pandas tự gán chỉ số nguyên. Không truyền index, hai giá trị nhận nhãn `0` và `1`; kết quả là Series chứ không phải DataFrame một cột.

Q6. What does `df[['x']]` return? — `df[['x']]` trả về gì?

A. A DataFrame containing one column — DataFrame có một cột

B. A tuple with row and column counts — Tuple gồm số hàng và số cột

C. A Series containing column x — Series chứa cột x

D. A Series with default index 0,1 — Series có index mặc định 0,1

Đáp án: A

Giải thích: Hai lớp ngoặc vuông chọn danh sách cột và giữ dạng bảng. Nếu `df` có năm hàng, `df[['x']]` giữ kết quả hai chiều với shape `(5,1)`, khác `df['x']`.

Q7. When DataFrame column labels are unique, what does `df['x']` return? — Khi tên cột DataFrame không trùng nhau, `df['x']` trả về gì?

A. A tuple with row and column counts — Tuple gồm số hàng và số cột

B. A Series containing column x — Series chứa cột x

C. A DataFrame containing one column — DataFrame có một cột

D. A Series with default index 0,1 — Series có index mặc định 0,1

Đáp án: B

Giải thích: Tra cứu một cột bằng ngoặc đơn trả Series. Với tên cột không trùng, `df['x']` là Series một chiều; hai lớp ngoặc vuông thì trả DataFrame.

Q8. What does `df.shape` return? — `df.shape` trả về gì?

A. A Series with default index 0,1 — Series có index mặc định 0,1

B. A DataFrame containing one column — DataFrame có một cột

C. A Series containing column x — Series chứa cột x

D. A tuple with row and column counts — Tuple gồm số hàng và số cột

Đáp án: D

Giải thích: Shape cho biết hai kích thước của DataFrame. Với DataFrame bốn bản ghi và ba trường, `df.shape` là `(4,3)`, không phải số ô không thiếu.

Q9. What does `df.head(3)` show? — `df.head(3)` hiển thị gì?

A. Columns, non-null counts and data types — Các cột, số ô không thiếu và kiểu dữ liệu

B. Data types of columns — Kiểu dữ liệu của các cột

C. The first three rows — Ba hàng đầu

D. Descriptive statistics such as mean and quartiles — Thống kê mô tả như trung bình và các tứ phân vị

Đáp án: C

Giải thích: head lấy số hàng đầu theo đối số. Nếu DataFrame chỉ có hai hàng, `head(3)` trả hai hàng đó chứ không tạo thêm hàng giả.

Q10. What does `df.dtypes` describe? — `df.dtypes` mô tả gì?

A. Descriptive statistics such as mean and quartiles — Thống kê mô tả như trung bình và các tứ phân vị

B. The first three rows — Ba hàng đầu

C. Data types of columns — Kiểu dữ liệu của các cột

D. Columns, non-null counts and data types — Các cột, số ô không thiếu và kiểu dữ liệu

Đáp án: C

Giải thích: Mỗi cột có thể mang dtype riêng. Cột `price` có thể có dtype thực, còn `name` mang kiểu văn bản; `dtypes` không tính thống kê mô tả.

Q11. What does `df.info()` primarily summarize? — `df.info()` chủ yếu tóm tắt gì?

A. The first three rows — Ba hàng đầu

B. Data types of columns — Kiểu dữ liệu của các cột

C. Descriptive statistics such as mean and quartiles — Thống kê mô tả như trung bình và các tứ phân vị

D. Columns, non-null counts and data types — Các cột, số ô không thiếu và kiểu dữ liệu

Đáp án: D

Giải thích: info cho biết cấu trúc và mức độ đầy đủ của bảng. Nó hiển thị tên cột, số giá trị không thiếu, dtype và bộ nhớ; khác thao tác xem trước hàng dữ liệu bằng `head()`.

Q12. What does `df.describe()` commonly provide for numeric columns? — `df.describe()` thường cung cấp gì cho cột số?

A. Descriptive statistics such as mean and quartiles — Thống kê mô tả như trung bình và các tứ phân vị

B. The first three rows — Ba hàng đầu

C. Columns, non-null counts and data types — Các cột, số ô không thiếu và kiểu dữ liệu

D. Data types of columns — Kiểu dữ liệu của các cột

Đáp án: A

Giải thích: describe tính những thống kê cơ bản cho cột số. Với số `[2,4,6]`, trung bình bằng 4 và các tứ phân vị cho biết mức phân tán; `head()` chỉ hiển thị hàng dữ liệu gốc.

Q13. What is an index in a DataFrame? — Index của DataFrame là gì?

A. Column labels — Các tên cột

B. One observation or record — Một quan sát hoặc bản ghi

C. Labels identifying rows — Các nhãn xác định hàng

D. Index labels — Nhãn index

Đáp án: C

Giải thích: Index tách với các cột thông thường trừ khi được chuyển đổi. Index có thể là nhãn như mã học viên và tách khỏi các cột dữ liệu thường, trừ khi được đưa trở lại thành cột.

Q14. How are Series values aligned during arithmetic? — Pandas căn các giá trị Series khi tính toán bằng gì?

A. Labels identifying rows — Các nhãn xác định hàng

B. One observation or record — Một quan sát hoặc bản ghi

C. Index labels — Nhãn index

D. Column labels — Các tên cột

Đáp án: C

Giải thích: Pandas ghép giá trị theo các nhãn index tương ứng. Nếu hai Series sắp nhãn theo thứ tự khác nhau, Pandas ghép giá trị cùng nhãn trước khi cộng chứ không cộng máy móc theo vị trí.

Q15. What does `df.columns` contain? — `df.columns` chứa gì?

A. One observation or record — Một quan sát hoặc bản ghi

B. Index labels — Nhãn index

C. Column labels — Các tên cột

D. Labels identifying rows — Các nhãn xác định hàng

Đáp án: C

Giải thích: Thuộc tính columns liệt kê tên trường. Với các cột `name` và `age`, `list(df.columns)` trả danh sách tên cột, không phải nhãn hàng.

Q16. What is a DataFrame row usually intended to represent? — Mỗi hàng DataFrame thường nên biểu diễn gì?

A. One observation or record — Một quan sát hoặc bản ghi

B. Column labels — Các tên cột

C. Index labels — Nhãn index

D. Labels identifying rows — Các nhãn xác định hàng

Đáp án: A

Giải thích: Ý nghĩa hàng nhất quán giúp hiểu đúng các phép tổng hợp. Một hàng có thể là một học viên, giao dịch hoặc lần đo, nhưng cần định nghĩa nhất quán để hiểu đúng thống kê.

Q17. What does `df['price'] * df['qty']` calculate? — `df['price'] * df['qty']` tính gì?

A. An element-wise product aligned by index — Tích từng phần tử được căn theo index

B. To interpret the data and calculations correctly — Để hiểu đúng dữ liệu và phép tính

C. A numeric column may be parsed as text — Cột số có thể bị đọc thành chữ

D. Two named columns — Hai cột được nêu tên

Đáp án: A

Giải thích: Giá và lượng tương ứng được nhân ở từng bản ghi. Nếu cùng một hàng có giá 10 đô la và số lượng 3 món, tích là 30 đô la, không phải tổng của nhiều hàng.

Q18. What does `df[['a','b']]` select? — `df[['a','b']]` chọn gì?

A. A numeric column may be parsed as text — Cột số có thể bị đọc thành chữ

B. An element-wise product aligned by index — Tích từng phần tử được căn theo index

C. To interpret the data and calculations correctly — Để hiểu đúng dữ liệu và phép tính

D. Two named columns — Hai cột được nêu tên

Đáp án: D

Giải thích: Danh sách tên cho phép lấy nhiều cột. Kết quả vẫn là DataFrame chứa `a` và `b` theo thứ tự đã nêu, còn `df['a']` thường trả Series.

Q19. What is a frequent risk when loading CSV numeric data? — Nguy cơ thường gặp khi đọc số từ CSV là gì?

A. A numeric column may be parsed as text — Cột số có thể bị đọc thành chữ

B. Two named columns — Hai cột được nêu tên

C. An element-wise product aligned by index — Tích từng phần tử được căn theo index

D. To interpret the data and calculations correctly — Để hiểu đúng dữ liệu và phép tính

Đáp án: A

Giải thích: Giá trị không hợp lệ có thể khiến Pandas không nhận kiểu số. Ô như `'unknown'` nằm trong cột số có thể khiến kiểu bị suy ra thành văn bản/object, nên phải kiểm tra và đổi kiểu trước khi tính.

Q20. Why define what each row and column means? — Vì sao cần nêu ý nghĩa từng hàng và cột?

A. A numeric column may be parsed as text — Cột số có thể bị đọc thành chữ

B. To interpret the data and calculations correctly — Để hiểu đúng dữ liệu và phép tính

C. An element-wise product aligned by index — Tích từng phần tử được căn theo index

D. Two named columns — Hai cột được nêu tên

Đáp án: B

Giải thích: Ý nghĩa hàng và cột quyết định cách diễn giải kết quả. Nếu hàng là giao dịch và cột gồm giá cùng số lượng, nêu rõ đơn vị giúp tránh nhân nhầm trường và hiểu sai doanh thu.

8. Nối bài tiếp theo

`NumPy ndarray → Pandas DataFrame → Làm sạch → EDA/AI.`

PY-18 — Pandas: missing values, duplicates và kiểu dữ liệu

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Missing value — giá trị thiếu; **Duplicate** — bản ghi trùng.

`DataFrame → Data Cleaning → EDA → ML Pipeline`

Học xong dùng để làm gì? Phát hiện và xử lý dữ liệu thiếu, trùng; tránh dùng thông tin test khi điền khuyết.

Kết nối kiến thức tiếp theo: PY-19 ghép những bảng dữ liệu sạch và tổng hợp theo nhóm.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Dùng khi nào
Missing value	giá trị thiếu	Ô không chứa số liệu hợp lệ
Duplicate	bản ghi trùng	Hai hàng dữ liệu giống nhau
Imputation	điền khuyết	Thay ô trống bằng giá trị phù hợp
Data validation	kiểm tra dữ liệu	Xác nhận dữ liệu đúng kiểu và giới hạn

3. Thư viện, hàm và lựa chọn

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / tham số	Giải thích ngay theo hàm
`df.isna().sum()`	Đếm số ô khuyết theo cột
`df.fillna()/dropna()`	Điền hoặc bỏ dữ liệu thiếu; cân nhắc mất mẫu
`df.duplicated()/drop_duplicates()`	Phát hiện và loại bỏ hàng trùng
`pd.to_numeric(errors='coerce')`	Chuyển chuỗi lỗi thành NaN để xử lý có kiểm soát

4. Khái niệm liên quan

Data cleaning (làm sạch dữ liệu) là quá trình tìm và xử lý các giá trị thiếu, dòng lặp và giá trị sai kiểu trước khi phân tích. Với dữ liệu khách hàng và đơn hàng, ta đi từ **phát hiện** thiếu/trùng → **quyết định** xử lý → **kiểm tra** kết quả; không xóa hoặc điền giá trị chỉ vì hàm có sẵn.

4.1. Missing values — Phát hiện dữ liệu thiếu

Từ vựng trong mục: Missing value — giá trị thiếu; **NaN** — giá trị số thiếu.

Giá trị thiếu là ô chưa có dữ liệu hợp lệ, chẳng hạn ba khách hàng có tuổi `[20, None, 25]` và khách thứ hai chưa khai báo. Trong DataFrame, **mỗi hàng là một khách**, cột `age` là tuổi (năm). Pandas thường biểu diễn số thiếu là `NaN`; `isna()` tạo giá trị Boolean đánh dấu ô thiếu, rồi `sum()` đếm số trường hợp `True`. Thiếu khác với tuổi 0: không được tùy tiện coi khách chưa nhập tuổi là trẻ sơ sinh.

```
import pandas as pd

customers = pd.DataFrame({'age': [20, None, 25]})
print(customers['age'].isna().sum())
```

Kết quả khi chạy:

```
1
```

Có một khách chưa có tuổi. Kết quả là **số ô thiếu**, không phải giá trị tuổi của khách.

4.2. Duplicates — Nhận diện dòng trùng

Từ vựng trong mục: Duplicate — bản ghi trùng; **Row** — hàng.

Dữ liệu có thể chứa một giao dịch được ghi hai lần. Ví dụ bảng gồm các dòng `(O1, 10)`, `(O1, 10)`, `(O2, 20)`: **mỗi hàng là một giao dịch**, cột `id` là mã đơn, `amount` là tiền. `duplicated()` đánh dấu dòng lặp giống hệt **sau dòng đầu tiên**; `drop_duplicates()` giữ một bản. Tuy nhiên hai giao dịch cùng mã chưa chắc trùng hoàn toàn nếu số tiền hoặc chi tiết khác nhau — phải xác định quy tắc nhận diện trùng trước khi xóa.

```
import pandas as pd

orders = pd.DataFrame({'id': ['O1', 'O1', 'O2'], 'amount': [10, 10, 20]})
print(orders.duplicated().tolist())
```

Kết quả khi chạy:

```
[False, True, False]
```

Dòng thứ hai lặp đúng mọi cột của dòng đầu, nên chỉ dòng ấy được đánh dấu trùng.

4.3. Imputation — Điền dữ liệu thiếu hợp lý

Từ vựng trong mục: Imputation — điền khuyết; **Median** — trung vị.

Điền khuyết thay giá trị thiếu bằng một giá trị được lựa chọn theo ý nghĩa dữ liệu. Giả sử ba mức chi tiêu là `[100, None, 200]` nghìn đồng. Trung vị của hai số có thật là `(100+200)/2 = 150`; `fillna(median)` điền 150 vào ô thiếu. Không có một cách điền đúng cho mọi bài toán: với mã đơn thiếu nên xác minh nguồn, không tự thay bằng số trung vị. Nếu có tập train/test, thống kê dùng để điền phải học từ **train** để tránh rò rỉ dữ liệu.

```
import pandas as pd

spent = pd.Series([100, None, 200], dtype=float)
median = spent.median()
print(spent.fillna(median).tolist())
```

Kết quả khi chạy:

```
[100.0, 150.0, 200.0]
```

Ô thứ hai được điền 150 nghìn đồng; hai số đo hợp lệ 100 và 200 được giữ nguyên.

4.4. Data validation — Kiểm tra giá trị và kiểu

Từ vựng trong mục: Validation — kiểm tra tính hợp lệ; **Coerce** — chuyển giá trị sai thành thiếu.

Kiểm tra dữ liệu bảo đảm kiểu và phạm vi phù hợp trước khi tính. Một bảng giá nhận ba chuỗi `'100'`, `'bad'`, `'250'`. `pd.to_numeric(errors='coerce')` chuyển chuỗi số thành số thực, còn chữ `bad` không đổi được thành `NaN`. Sau đó mới quyết định bỏ hàng, điền khuyết hay yêu cầu sửa ở nguồn. Nếu dùng `errors='raise'`, trường hợp không hợp lệ sẽ phát sinh lỗi để chương trình xử lý.

```
import pandas as pd

raw = pd.Series(['100', 'bad', '250'])
prices = pd.to_numeric(raw, errors='coerce')
print(prices.isna().sum())
```

Kết quả khi chạy:

```
1
```

Có đúng một giá trị giá không hợp lệ. Chuyển thành NaN giúp phát hiện, không có nghĩa đã sửa được giá trị thật.

5. Năm tình huống thực tế

Tình huống 1 — Báo số khách thiếu tuổi

Từ vựng trong mục: Missing value — giá trị thiếu.

Tình huống thực tế: Một bảng tuổi khách có ba hàng `[20, None, 25]`, trong đó khách thứ hai chưa nhập tuổi. Đếm số tuổi bị thiếu.

Vì sao chọn cách giải này? `isna()` đánh dấu các ô thiếu; `sum()` đếm các ô đó theo cột.

Quy trình giải:

1. Tạo bảng `age` gồm ba khách, người thứ hai có giá trị thiếu.
2. Dùng `isna()` đánh dấu từng ô thiếu.
3. Dùng `sum()` đếm và in số trường hợp cần rà soát.

```
import pandas as pd
df = pd.DataFrame({"age":[20, None, 25]})
print(df.isna().sum())
```

Kết quả khi chạy:

```
age    1
dtype: int64
```

Tình huống 2 — Điền giá trị trung vị

Từ vựng trong mục: Imputation — điền khuyết; **Median** — trung vị.

Tình huống thực tế: Cửa hàng có mức chi tiêu `[100, None, 200]` đơn vị tiền; một dòng chưa có số liệu. Điền chỗ thiếu bằng trung vị của số liệu hiện có.

Vì sao chọn cách giải này? Trung vị của 100 và 200 là 150; `fillna` chỉ thay giá trị thiếu.

Quy trình giải:

1. Tạo cột `spent` có hai số 100, 200 và một ô thiếu.
2. Tính trung vị từ các số có thật, được 150.
3. Gọi `fillna` trên cột chi tiêu và in bảng đã điền.

```
import pandas as pd
df = pd.DataFrame({"spent":[100, None, 200]})
df["spent"] = df["spent"].fillna(df["spent"].median())
print(df)
```

Kết quả khi chạy:

```
   spent
0  100.0
1  150.0
2  200.0
```

Tình huống 3 — Loại dòng trùng hoàn toàn

Từ vựng trong mục: Duplicate — bản ghi trùng.

Tình huống thực tế: Đơn hàng có ba dòng O1:10, O1:10 và O2:20, trong đó hai dòng O1 giống hệt nhau. Loại dòng trùng hoàn toàn và in bảng còn lại.

Vì sao chọn cách giải này? `drop_duplicates()` giữ một bản của những dòng giống nhau.

Quy trình giải:

1. Tạo bảng ba dòng với hai bản ghi O1 giống hệt và một O2.
2. Gọi `drop_duplicates()` để loại dòng trùng hoàn toàn.
3. In bảng còn hai bản ghi; giữ nguyên index cũ nếu không reset.

```
import pandas as pd
df = pd.DataFrame({"id":["O1", "O1", "O2"], "amount":[10, 10, 20]})
print(df.drop_duplicates())
```

Kết quả khi chạy:

```
   id  amount
0  O1      10
2  O2      20
```

Tình huống 4 — Đổi chuỗi giá sang số

Từ vựng trong mục: Data validation — kiểm tra dữ liệu.

Tình huống thực tế: Dữ liệu giá nhận ba chuỗi `100`, `bad`, `250`. Giá trị `bad` không thể đổi thành số. Đổi các chuỗi số thành số, còn giá trị không hợp lệ thành NaN.

Vì sao chọn cách giải này? `to_numeric(errors='coerce')` cho phép xử lý giá trị sai kiểu mà không dừng chương trình.

Quy trình giải:

1. Đọc ba chuỗi giá, trong đó `'bad'` không phải số.
2. Dùng `pd.to_numeric(..., errors='coerce')` chuyển lỗi thành NaN.
3. In Series và xác định vị trí giá chưa hợp lệ.

```
import pandas as pd
s = pd.Series(["100", "bad", "250"])
print(pd.to_numeric(s, errors="coerce"))
```

Kết quả khi chạy:

```
0    100.0
1      NaN
2    250.0
dtype: float64
```

Tình huống 5 — Chuẩn bị train/test tránh leakage

Từ vựng trong mục: Missing value — giá trị thiếu.

Tình huống thực tế: Dữ liệu train là `[10, None, 20]` và test là `[None,30]`. Lấy trung vị từ train rồi điền số thiếu của cả train và test.

Vì sao chọn cách giải này? Tính thống kê chỉ từ tập train giúp tránh rò rỉ thông tin từ test.

Quy trình giải:

1. Tạo riêng Series train `[10, None, 20]` và test `[None, 30]`.
2. Tính trung vị **chỉ trên train**, thu được 15.
3. Dùng 15 điền vào ô thiếu của cả train và test, không học từ test.

```
import pandas as pd
train = pd.Series([10, None, 20])
test = pd.Series([None, 30])
median = train.median()
print(train.fillna(median), test.fillna(median))
```

Kết quả khi chạy:

```
0    10.0
1    15.0
2    20.0
dtype: float64 0    15.0
1    30.0
dtype: float64
```

6. Sai lầm thường gặp

- **Thiếu dữ liệu** không đồng nghĩa với 0; điền 0 vào tuổi, giá hay lượng hàng thiếu có thể làm sai thống kê.
- `duplicated()` mặc định xét toàn bộ các cột; xóa trùng phải hiểu **khóa định danh** và ý nghĩa từng bản ghi.
- `drop_duplicates()` thường giữ **index cũ**; cần `reset_index(drop=True)` nếu muốn đánh lại nhãn hàng.
- `to_numeric(errors='coerce')` đổi dữ liệu sai thành NaN, **không tự đoán** giá trị thật.
- Với Machine Learning, chỉ tính trung vị/trung bình để điền từ **tập train**, sau đó áp dụng cho test để tránh data leakage.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a missing value? — Giá trị thiếu là gì?

A. Replacing missing entries with chosen estimates — Thay ô thiếu bằng giá trị ước lượng đã chọn

B. Checking types and rules before analysis — Kiểm tra kiểu và quy tắc trước khi phân tích

C. An absent or unknown data entry — Ô dữ liệu chưa có giá trị xác định

D. A record repeated under a defined matching rule — Bản ghi lặp theo tiêu chí đối chiếu xác định

Đáp án: C

Giải thích: Thiếu nghĩa là chưa có giá trị quan sát hợp lệ ở ô đó. Trong Pandas, `NaN` và `None` có thể biểu thị ô thiếu; giá trị thiếu khác một số 0 đã được đo thật.

Q2. What is a duplicate record? — Bản ghi trùng là gì?

A. A record repeated under a defined matching rule — Bản ghi lặp theo tiêu chí đối chiếu xác định

B. Checking types and rules before analysis — Kiểm tra kiểu và quy tắc trước khi phân tích

C. An absent or unknown data entry — Ô dữ liệu chưa có giá trị xác định

D. Replacing missing entries with chosen estimates — Thay ô thiếu bằng giá trị ước lượng đã chọn

Đáp án: A

Giải thích: Nhận diện trùng phụ thuộc các trường dùng làm tiêu chí. Hai hàng cùng mã khách có thể là hai đơn riêng, nên phải xác định tiêu chí trùng trước khi xóa dữ liệu.

Q3. What is imputation? — Điền khuyết là gì?

A. An absent or unknown data entry — Ô dữ liệu chưa có giá trị xác định

B. Checking types and rules before analysis — Kiểm tra kiểu và quy tắc trước khi phân tích

C. Replacing missing entries with chosen estimates — Thay ô thiếu bằng giá trị ước lượng đã chọn

D. A record repeated under a defined matching rule — Bản ghi lặp theo tiêu chí đối chiếu xác định

Đáp án: C

Giải thích: Điền khuyết dùng giá trị lựa chọn chứ không coi đó là dữ liệu đã đo thật. Điền tuổi thiếu bằng trung vị của tập train là phép ước lượng, không chứng minh tuổi thật bằng trung vị đó.

Q4. What is data validation? — Kiểm tra dữ liệu hợp lệ là gì?

A. A record repeated under a defined matching rule — Bản ghi lặp theo tiêu chí đối chiếu xác định

B. An absent or unknown data entry — Ô dữ liệu chưa có giá trị xác định

C. Checking types and rules before analysis — Kiểm tra kiểu và quy tắc trước khi phân tích

D. Replacing missing entries with chosen estimates — Thay ô thiếu bằng giá trị ước lượng đã chọn

Đáp án: C

Giải thích: Kiểm tra hợp lệ phát hiện sai kiểu và giá trị vượt phạm vi. Với trường tuổi, bước kiểm tra có thể yêu cầu giá trị số không âm và loại chuỗi không hợp lệ.

Q5. What does `isna()` identify? — `isna()` phát hiện gì?

A. Missing entries — Các ô thiếu

B. Turns invalid numeric text into NaN — Chuyển chữ không đổi được sang số thành NaN

C. Repeated rows after the first occurrence — Các hàng lặp sau lần xuất hiện đầu

D. Replaces missing entries with a value — Thay ô thiếu bằng giá trị cho trước

Đáp án: A

Giải thích: isna trả mặt nạ Boolean đánh dấu giá trị thiếu. Với `[2, NaN]`, `isna()` đánh dấu `[False, True]`; hàm xác định chỗ thiếu mà không xóa dữ liệu.

Q6. What does `duplicated()` mark by default? — `duplicated()` mặc định đánh dấu gì?

A. Turns invalid numeric text into NaN — Chuyển chữ không đổi được sang số thành NaN

B. Repeated rows after the first occurrence — Các hàng lặp sau lần xuất hiện đầu

C. Replaces missing entries with a value — Thay ô thiếu bằng giá trị cho trước

D. Missing entries — Các ô thiếu

Đáp án: B

Giải thích: duplicated xét toàn hàng nếu không chỉ định subset. Với hai hàng giống nhau, hàng đầu thường nhận `False`, hàng sau nhận `True`; `subset=` thay đổi các cột dùng để xét trùng.

Q7. What does `fillna(value)` do? — `fillna(value)` làm gì?

A. Turns invalid numeric text into NaN — Chuyển chữ không đổi được sang số thành NaN

B. Missing entries — Các ô thiếu

C. Repeated rows after the first occurrence — Các hàng lặp sau lần xuất hiện đầu

D. Replaces missing entries with a value — Thay ô thiếu bằng giá trị cho trước

Đáp án: D

Giải thích: fillna thay ô thiếu mà giữ các giá trị hợp lệ. Với điểm thiếu, `df.fillna(0)` mặc định trả DataFrame đã điền mà không đổi `df` nếu chưa gán kết quả hoặc yêu cầu sửa tại chỗ.

Q8. What does `pd.to_numeric(..., errors='coerce')` do with invalid text? — `to_numeric(..., errors='coerce')` xử lý chuỗi không phải số thế nào?

A. Replaces missing entries with a value — Thay ô thiếu bằng giá trị cho trước

B. Turns invalid numeric text into NaN — Chuyển chữ không đổi được sang số thành NaN

C. Repeated rows after the first occurrence — Các hàng lặp sau lần xuất hiện đầu

D. Missing entries — Các ô thiếu

Đáp án: B

Giải thích: Cơ chế coerce đánh dấu lỗi chuyển kiểu là thiếu chứ không đoán số. Khi đổi kiểu số, chuỗi `'bad'` trở thành dấu thiếu, còn chuỗi hợp lệ như `'12'` được chuyển thành số 12.

Q9. What does `dropna()` normally remove? — `dropna()` mặc định xóa gì?

A. Rows containing missing data — Các hàng chứa dữ liệu thiếu

B. The middle value after sorting, or mean of the two middle values — Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

C. Raise an exception on invalid input — Báo ngoại lệ khi đầu vào không hợp lệ

D. The first occurrence of each distinct row — Lần xuất hiện đầu của từng hàng khác nhau

Đáp án: A

Giải thích: DataFrame.dropna mặc định bỏ hàng có ít nhất một ô thiếu. Với DataFrame có ô thiếu, `dropna()` mặc định bỏ hàng chứa ít nhất một ô thiếu; nó không tự điền các ô đó.

Q10. What does `drop_duplicates()` normally keep? — `drop_duplicates()` thường giữ gì?

A. The middle value after sorting, or mean of the two middle values — Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

B. The first occurrence of each distinct row — Lần xuất hiện đầu của từng hàng khác nhau

C. Rows containing missing data — Các hàng chứa dữ liệu thiếu

D. Raise an exception on invalid input — Báo ngoại lệ khi đầu vào không hợp lệ

Đáp án: B

Giải thích: Mặc định các lần lặp sau của cùng hàng sẽ bị bỏ. Với các hàng A, A, B, mặc định giữ A đầu và B, không giữ A cuối trừ khi đặt `keep='last'`.

Q11. What is the median? — Trung vị là gì?

A. Raise an exception on invalid input — Báo ngoại lệ khi đầu vào không hợp lệ

B. The first occurrence of each distinct row — Lần xuất hiện đầu của từng hàng khác nhau

C. Rows containing missing data — Các hàng chứa dữ liệu thiếu

D. The middle value after sorting, or mean of the two middle values — Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

Đáp án: D

Giải thích: Trung vị ít nhạy với giá trị cực đoan hơn trung bình cộng. Với dãy đã sắp `[1,2,100]`, trung vị là 2 còn trung bình lớn hơn 34; nếu số phần tử chẵn thì lấy trung bình hai số ở giữa.

Q12. What is the meaning of `errors='raise'` in numeric conversion? — `errors='raise'` khi đổi sang số có nghĩa gì?

A. Rows containing missing data — Các hàng chứa dữ liệu thiếu

B. The middle value after sorting, or mean of the two middle values — Giá trị giữa khi sắp, hoặc trung bình hai giá trị giữa

C. The first occurrence of each distinct row — Lần xuất hiện đầu của từng hàng khác nhau

D. Raise an exception on invalid input — Báo ngoại lệ khi đầu vào không hợp lệ

Đáp án: D

Giải thích: Raise làm lỗi lộ rõ thay vì chuyển âm thầm thành thiếu. Dùng `pd.to_numeric(['bad'], errors='raise')` báo lỗi, còn `errors='coerce'` trả dấu giá trị số bị thiếu.

Q13. What is data leakage in train/test preprocessing? — Data leakage khi chuẩn bị train/test là gì?

A. Zero may have a different real-world meaning — Số 0 có thể mang ý nghĩa thực tế khác

B. To audit and reverse cleaning decisions — Để đối chiếu và phục hồi quyết định xử lý

C. Records with shared IDs may still represent distinct events — Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

D. Using information from test data while fitting preprocessing — Dùng thông tin của test khi học tham số tiền xử lý

Đáp án: D

Giải thích: Thống kê tiền xử lý phải học chỉ từ tập train. Tính trung vị điền khuyết trên cả train và test làm thông tin của tập đánh giá lọt vào giai đoạn chuẩn bị mô hình.

Q14. Why is replacing every missing value with zero risky? — Vì sao thay mọi ô thiếu bằng 0 là rủi ro?

A. To audit and reverse cleaning decisions — Để đối chiếu và phục hồi quyết định xử lý

B. Using information from test data while fitting preprocessing — Dùng thông tin của test khi học tham số tiền xử lý

C. Zero may have a different real-world meaning — Số 0 có thể mang ý nghĩa thực tế khác

D. Records with shared IDs may still represent distinct events — Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

Đáp án: C

Giải thích: Tuổi chưa nhập không tự đồng nghĩa tuổi bằng 0. Số lượng hàng thiếu nghĩa là chưa biết, còn số lượng 0 nói rõ không có món nào; thay hai trường hợp cho nhau sẽ đổi ý nghĩa.

Q15. Why check key columns before removing duplicates? — Vì sao phải xem cột khóa trước khi xóa trùng?

A. Records with shared IDs may still represent distinct events — Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

B. Using information from test data while fitting preprocessing — Dùng thông tin của test khi học tham số tiền xử lý

C. Zero may have a different real-world meaning — Số 0 có thể mang ý nghĩa thực tế khác

D. To audit and reverse cleaning decisions — Để đối chiếu và phục hồi quyết định xử lý

Đáp án: A

Giải thích: Quy tắc nghiệp vụ xác định hai dòng có thực sự trùng không. Hai giao dịch của cùng khách hàng có thể là hai sự kiện hợp lệ khác nhau dù trùng một mã định danh.

Q16. Why preserve the original raw data while cleaning? — Vì sao nên giữ dữ liệu gốc khi làm sạch?

A. Zero may have a different real-world meaning — Số 0 có thể mang ý nghĩa thực tế khác

B. To audit and reverse cleaning decisions — Để đối chiếu và phục hồi quyết định xử lý

C. Records with shared IDs may still represent distinct events — Hai dòng cùng ID vẫn có thể là sự kiện khác nhau

D. Using information from test data while fitting preprocessing — Dùng thông tin của test khi học tham số tiền xử lý

Đáp án: B

Giải thích: Bản gốc giúp kiểm tra từng phép biến đổi. Giữ bản nguồn không đổi giúp so sánh trước/sau và xem xét lại các quyết định làm sạch chưa đúng.

Q17. What does a Boolean missing-value mask contain? — Mặt nạ dữ liệu thiếu chứa gì?

A. True and False values — Các giá trị True và False

B. Original retained-row labels remain — Nhãn index của hàng được giữ vẫn còn

C. Choose a justified missing-data treatment — Chọn cách xử lý dữ liệu thiếu có cơ sở

D. Rebuilds sequential indices without retaining old index as a column — Đánh lại index tuần tự, không giữ index cũ thành cột

Đáp án: A

Giải thích: Mỗi vị trí cho biết ô dữ liệu có thiếu hay không. Một ô mask là `True` nếu đầu vào tương ứng bị thiếu và `False` nếu không; mask không phải danh sách giá trị thay thế.

Q18. What happens to index labels after `drop_duplicates()` by default? — Sau `drop_duplicates()`, index mặc định thay đổi thế nào?

A. True and False values — Các giá trị True và False

B. Choose a justified missing-data treatment — Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Rebuilds sequential indices without retaining old index as a column — Đánh lại index tuần tự, không giữ index cũ thành cột

D. Original retained-row labels remain — Nhãn index của hàng được giữ vẫn còn

Đáp án: D

Giải thích: Xóa hàng trùng không tự đánh lại index cho hàng còn lại. Xóa các hàng mang nhãn 1 và 3 không tự đánh lại nhãn còn lại; phải reset index riêng nếu muốn nhãn liên tiếp.

Q19. What does `reset_index(drop=True)` do? — `reset_index(drop=True)` làm gì?

A. Original retained-row labels remain — Nhãn index của hàng được giữ vẫn còn

B. Choose a justified missing-data treatment — Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Rebuilds sequential indices without retaining old index as a column — Đánh lại index tuần tự, không giữ index cũ thành cột

D. True and False values — Các giá trị True và False

Đáp án: C

Giải thích: Đánh lại nhãn hàng giúp bảng đã xử lý dễ đọc hơn. Sau khi bỏ hàng, `reset_index(drop=True)` tạo nhãn `0,1,2,...` mà không đưa index cũ thành cột mới.

Q20. What should happen after coercing invalid numeric text to NaN? — Sau khi chuyển chuỗi số sai thành NaN cần làm gì?

A. Rebuilds sequential indices without retaining old index as a column — Đánh lại index tuần tự, không giữ index cũ thành cột

B. Choose a justified missing-data treatment — Chọn cách xử lý dữ liệu thiếu có cơ sở

C. Original retained-row labels remain — Nhãn index của hàng được giữ vẫn còn

D. True and False values — Các giá trị True và False

Đáp án: B

Giải thích: Coerce phát hiện lỗi, không khôi phục được giá trị thực chưa biết. Sau khi coerce, hãy đếm số ô bị thiếu rồi quyết định loại bỏ, từ chối hay điền khuyết theo yêu cầu bài toán.

8. Nối bài tiếp theo

`DataFrame → Data Cleaning → EDA → ML Pipeline.`

PY-19 — Pandas: lọc, groupby, merge và sắp xếp

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: loc — chọn theo nhãn; **iloc** — chọn theo vị trí.

`DataFrame → lọc → nhóm → ghép bảng → AI`

Học xong dùng để làm gì? Gộp doanh thu theo thành phố và ghép thông tin khách hàng với đơn hàng theo khóa.

Kết nối kiến thức tiếp theo: PY-20 dùng logic và nguyên lý đếm để diễn đạt điều kiện và số trường hợp.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
loc	chọn theo nhãn	Chọn hàng/cột theo tên
iloc	chọn theo vị trí	Chọn hàng/cột theo số nguyên tính từ 0
groupby	nhóm dữ liệu	Gom các hàng cùng giá trị để tổng hợp
merge	ghép bảng	Nối bảng qua khóa chung

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`df.loc[label]`	Chọn theo nhãn dòng/cột; slicing theo nhãn có thể gồm điểm cuối
`df.iloc[position]`	Chọn theo vị trí; stop của slice không bao gồm
`df.groupby(key).agg()`	Gộp nhóm rồi tính tổng/trung bình/số dòng
`pd.merge(left,right,on=,how=)`	Ghép theo khóa, cách inner/left/right/outer khác nhau

4. Khái niệm liên quan

Phân tích bảng với Pandas thường gồm ba thao tác nối tiếp: **lọc** để giữ dòng đúng điều kiện, **nhóm** để tính thống kê theo loại, rồi **ghép** để bổ sung thông tin từ bảng khác; muốn chọn hàng chính xác cần phân biệt `.loc` theo nhãn và `.iloc` theo vị trí. Ví dụ hàng sản phẩm, giao dịch và khách hàng sẽ cho thấy vì sao cách tổ chức bảng quyết định phép xử lý.

4.1. Filtering, loc — Lọc hàng theo điều kiện hoặc nhãn

Từ vựng trong mục: Filter — lọc; **loc** — chọn theo nhãn.

Lọc dữ liệu là giữ các hàng thỏa một điều kiện. Ví dụ bảng có ba mặt hàng A, B, C, giá lần lượt 300, 900, 1200 (nghìn đồng). **Mỗi hàng** là một sản phẩm, **cột price** là giá. `df['price'] >= 800` tạo một Series Boolean; `df.loc[mask, ['name','price']]` chỉ giữ các hàng có giá từ 800 và hai cột muốn xem. `.loc` cũng chọn trực tiếp theo **nhãn index**, khác chọn vị trí số.

```
import pandas as pd

products = pd.DataFrame({'name': ['A', 'B', 'C'], 'price': [300, 900, 1200]})
selected = products.loc[products['price'] >= 800, 'name']
print(selected.tolist())
```

Kết quả khi chạy:

```
['B', 'C']
```

Sản phẩm B, C thỏa mức giá; A giá 300 thấp hơn 800 nên bị loại. Kết quả chuyển sang List để tập trung vào tên.

4.2. iloc — Chọn theo vị trí hàng và cột

Từ vựng trong mục: iloc — chọn theo vị trí; **Index** — chỉ số.

`.iloc` chọn **vị trí số nguyên**, tính từ 0, bất kể nhãn index của bảng. Với ba khách A, B, C theo thứ tự hàng, `df.iloc[1]` là khách ở **hàng thứ hai**. Cú pháp `df.iloc[row, col]` lấy một ô, ví dụ `[1, 0]` là cột đầu của hàng hai. Trái lại `.loc['B']` chỉ hoạt động khi **nhãn index thực sự là B**, không phải chỉ vì cột `id` có chữ B.

```
import pandas as pd

people = pd.DataFrame({'id': ['A', 'B', 'C']})
print(people.iloc[1, 0])
```

Kết quả khi chạy:

```
B
```

Chỉ số 1 lấy hàng thứ hai và chỉ số cột 0 lấy trường `id`, nên in B.

4.3. GroupBy — Tổng hợp trong từng nhóm

Từ vựng trong mục: GroupBy — nhóm theo khóa; **Aggregation** — phép tổng hợp.

`groupby` gom những hàng có **cùng giá trị ở một cột khóa**, rồi tính thống kê riêng cho từng nhóm. Giả sử ba giao dịch có `city = A,A,B` và `sale = 10,20,8` nghìn đồng. **Mỗi hàng** là một giao dịch; **cột city** là vùng, **sale** là tiền. `groupby('city')['sale'].sum()` cộng doanh thu của A thành 30 và B thành 8. Không nhầm GroupBy với lọc: lọc loại hàng, còn groupby kết hợp các hàng thành thống kê nhóm.

```
import pandas as pd

```

```
sales = pd.DataFrame({'city': ['A', 'A', 'B'], 'sale': [10, 20, 8]})
print(sales.groupby('city')['sale'].sum().to_dict())
```

Kết quả khi chạy:

```
{'A': 30, 'B': 8}
```

Vùng A có hai giao dịch nên tổng 30 nghìn đồng; vùng B chỉ có một giao dịch 8 nghìn đồng.

4.4. Merge — Ghép bảng theo khóa

Từ vựng trong mục: Merge — ghép bảng; **Join key** — khóa ghép.

Khi thông tin nằm trong hai bảng, `merge` dùng cột **khóa chung** để ghép các hàng liên quan. Bảng `orders` có đơn của khách `id=1` và `id=2`, cột `sale` là giá trị đơn; bảng `customers` có tên An, Lan theo cùng id. `merge(on='id', how='left')` giữ mọi dòng bên trái và bổ sung tên nếu tìm thấy. Nếu bảng bên phải có nhiều dòng trùng khóa, một dòng đơn hàng có thể bị nhân bản; dùng `validate='many_to_one'` để kiểm tra mỗi id bên phải là duy nhất.

```
import pandas as pd

orders = pd.DataFrame({'id': [1, 2], 'sale': [10, 20]})
customers = pd.DataFrame({'id': [1, 2], 'name': ['An', 'Lan']})
merged = orders.merge(customers, on='id', how='left', validate='many_to_one')
print(merged['name'].tolist())
```

Kết quả khi chạy:

```
['An', 'Lan']
```

Hai tên khách được ghép đúng theo id, không phải dựa vào vị trí hàng ngẫu nhiên.

5. Năm vấn đề thực tế

Tình huống 1 — Đơn hàng giá cao

Từ vựng trong mục: loc — chọn theo nhãn.

Tình huống thực tế: Cửa hàng có ba sản phẩm A, B, C với giá 300, 900 và 1.200 đơn vị tiền. In các sản phẩm có giá từ 800 trở lên.

Vì sao chọn cách giải này? Điều kiện `df.price >= 800` chọn hai hàng phù hợp.

Quy trình giải:

1. Tạo bảng gồm tên và giá của ba sản phẩm.
2. Tạo mặt nạ `price >= 800` để giữ sản phẩm đạt ngưỡng.
3. In bảng đã lọc, giữ B và C cùng giá tương ứng.

```
import pandas as pd
df = pd.DataFrame({"name":["A", "B", "C"], "price":[300, 900, 1200]})
print(df[df.price>=800])
```

Kết quả khi chạy:

```
  name  price
1    B    900
2    C   1200
```

Tình huống 2 — Tra dòng thứ hai

Từ vựng trong mục: iloc — chọn theo vị trí.

Tình huống thực tế: Bảng mã khách theo thứ tự A, B, C. Lấy dòng thứ hai của bảng.

Vì sao chọn cách giải này? `iloc[1]` lấy vị trí thứ hai do chỉ số bắt đầu từ 0.

Quy trình giải:

1. Tạo bảng mã khách A, B, C theo đúng thứ tự hàng.
2. Gọi `.iloc[1]` để lấy hàng ở chỉ số 1.
3. In dữ liệu của khách thứ hai, không nhầm nhãn với vị trí.

```
import pandas as pd
df = pd.DataFrame({"id":["A", "B", "C"]})
print(df.iloc[1])
```

Kết quả khi chạy:

```
id    B
Name: 1, dtype: object
```

Tình huống 3 — Doanh thu từng vùng

Từ vựng trong mục: groupby — nhóm dữ liệu.

Tình huống thực tế: Bảng doanh thu có ba giao dịch: vùng A được 10 và 20, vùng B được 8 đơn vị tiền. Tính tổng doanh thu theo từng vùng.

Vì sao chọn cách giải này? `groupby('city')['sale'].sum()` cộng các dòng có cùng vùng.

Quy trình giải:

1. Tạo ba dòng doanh thu và gắn khóa vùng A, A, B.
2. Nhóm theo cột `city` và cộng `sale` cho từng nhóm.
3. In tổng của vùng A và B, đơn vị tiền giữ nguyên.

```
import pandas as pd
df = pd.DataFrame({"city":["A", "A", "B"], "sale":[10, 20, 8]})
print(df.groupby("city")["sale"].sum())
```

Kết quả khi chạy:

```
city
A    30
```

```
B     8
Name: sale, dtype: int64
```

Tình huống 4 — Đơn hàng và khách

Từ vựng trong mục: merge — ghép bảng.

Tình huống thực tế: Bảng bán hàng có ID 1,2 với doanh thu 10,20. Bảng khách có ID 1 là An, ID 2 là Lan. Ghép tên khách vào từng dòng doanh thu theo ID.

Vì sao chọn cách giải này? `merge(..., on='id', how='left')` giữ các dòng bán hàng bên trái và lấy tên tương ứng.

Quy trình giải:

1. Tạo bảng đơn hàng với khóa `id` và bảng khách cũng có `id`.
2. Ghép từ bảng đơn bằng `merge(on='id', how='left')`.
3. In tên sau khi ghép và kiểm tra số hàng vẫn là hai.

```
import pandas as pd
a = pd.DataFrame({"id":[1, 2], "sale":[10, 20]})
b = pd.DataFrame({"id":[1, 2], "name":["An", "Lan"]})
print(a.merge(b, on="id", how="left"))
```

Kết quả khi chạy:

```
   id  sale name
0   1    10   An
1   2    20  Lan
```

Tình huống 5 — Top doanh số

Từ vựng trong mục: sort_values — sắp xếp theo giá trị.

Tình huống thực tế: Bốn nhân viên A, B, C, D có doanh số 30,80,20,60 đơn vị tiền. In hai người có doanh số cao nhất.

Vì sao chọn cách giải này? `sort_values(ascending=False).head(2)` sắp giảm dần và lấy hai dòng đầu.

Quy trình giải:

1. Tạo bảng bốn nhân viên với doanh số tương ứng.
2. Sắp xếp theo `sale` giảm dần.
3. Dùng `head(2)` để lấy hai nhân viên đứng đầu và in bảng.

```
import pandas as pd
df = pd.DataFrame({"id":["A", "B", "C", "D"], "sale":[30, 80, 20, 60]})
print(df.sort_values("sale", ascending=False).head(2))
```

Kết quả khi chạy:

```
  id  sale
1  B    80
3  D    60
```

6. Sai lầm thường gặp

- `.loc` dùng **nhãn**, `.iloc` dùng **vị trí số**; không nhầm nhãn cột `id` với index của DataFrame.
- Với lọc Boolean, phải dùng điều kiện trên đúng cột; `df.loc[mask, columns]` khác `.iloc`.
- `groupby('city').sum()` chỉ có ý nghĩa khi hiểu **mỗi hàng là giao dịch**, cột `city` là khóa phân nhóm và `sale` là lượng cần tổng hợp.
- `merge` dùng khóa; nếu bảng phụ có **khóa trùng**, số hàng sau ghép có thể tăng. Có thể dùng `validate='many_to_one'` khi bảng phải phải duy nhất.
- `sort_values()` mặc định tăng dần; để lấy nhóm doanh thu cao nhất cần `ascending=False` trước `head()`.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a Boolean filter do? — Lọc Boolean làm gì?

A. Integer positions — Vị trí số nguyên

B. Groups records sharing a key — Gom bản ghi có chung khóa

C. Keeps rows matching a condition — Giữ hàng thỏa điều kiện

D. Index labels — Nhãn index

Đáp án: C

Giải thích: Mặt nạ Boolean xác định hàng cần giữ. Với `df[df['score'] >= 8]`, chỉ những bản ghi đạt ngưỡng điểm được giữ lại; các hàng khác bị lọc, không phải được gộp thống kê.

Q2. What does `loc` use to select rows? — `loc` chọn hàng bằng gì?

A. Keeps rows matching a condition — Giữ hàng thỏa điều kiện

B. Integer positions — Vị trí số nguyên

C. Groups records sharing a key — Gom bản ghi có chung khóa

D. Index labels — Nhãn index

Đáp án: D

Giải thích: loc chọn theo nhãn hàng. Nếu DataFrame có nhãn hàng `'A'`, `df.loc['A']` chọn theo nhãn đó dù hàng không nằm ở vị trí đầu.

Q3. What does `iloc` use to select rows? — `iloc` chọn hàng bằng gì?

A. Integer positions — Vị trí số nguyên

B. Keeps rows matching a condition — Giữ hàng thỏa điều kiện

C. Groups records sharing a key — Gom bản ghi có chung khóa

D. Index labels — Nhãn index

Đáp án: A

Giải thích: iloc chọn theo vị trí đếm từ 0. `df.iloc[0]` chọn hàng đầu theo vị trí; hàng có nhãn `0` không nhất thiết nằm đầu sau khi sắp xếp dữ liệu.

Q4. What does `groupby` do? — `groupby` làm gì?

A. Index labels — Nhãn index

B. Keeps rows matching a condition — Giữ hàng thỏa điều kiện

C. Groups records sharing a key — Gom bản ghi có chung khóa

D. Integer positions — Vị trí số nguyên

Đáp án: C

Giải thích: Phân nhóm gom các hàng có chung một trường. Với các thành phố `HN`, `HN`, `HCM`, groupby theo thành phố tạo hai nhóm để tổng hợp mà không tự loại các thành phố còn lại.

Q5. What is the join key in `merge`? — Khóa ghép trong `merge` là gì?

A. A sale total per city — Tổng doanh thu theo từng vùng

B. Descending order — Thứ tự giảm dần

C. A shared field used to match rows — Trường chung để ghép các hàng

D. All rows from the left table — Mọi hàng bảng trái

Đáp án: C

Giải thích: Các hàng được ghép khi khóa phù hợp. Nếu hai bảng đều có `customer_id`, các hàng có mã khớp có thể ghép với nhau; vẫn cần xem kiểu dữ liệu và khóa trùng.

Q6. What does `how='left'` retain? — `how='left'` giữ gì?

A. Descending order — Thứ tự giảm dần

B. A shared field used to match rows — Trường chung để ghép các hàng

C. A sale total per city — Tổng doanh thu theo từng vùng

D. All rows from the left table — Mọi hàng bảng trái

Đáp án: D

Giải thích: Ghép trái không bỏ hàng bên trái chưa khớp. Left join giữ mọi bản ghi bảng trái kể cả khi bảng phải không khớp và điền giá trị thiếu phía phải; nhiều hàng phải cùng khớp có thể làm tăng số hàng kết quả.

Q7. What does `groupby('city')['sale'].sum()` return? — `groupby('city')['sale'].sum()` trả gì?

A. A shared field used to match rows — Trường chung để ghép các hàng

B. A sale total per city — Tổng doanh thu theo từng vùng

C. Descending order — Thứ tự giảm dần

D. All rows from the left table — Mọi hàng bảng trái

Đáp án: B

Giải thích: sum cộng giá trị sale theo từng vùng. Nếu `HN` có doanh thu 10, 15 và `HCM` có 8, tổng từng thành phố lần lượt là 25 và 8, không chỉ một tổng toàn bảng.

Q8. What does `ascending=False` mean in sorting? — `ascending=False` khi sắp xếp nghĩa là gì?

A. All rows from the left table — Mọi hàng bảng trái

B. A shared field used to match rows — Trường chung để ghép các hàng

C. Descending order — Thứ tự giảm dần

D. A sale total per city — Tổng doanh thu theo từng vùng

Đáp án: C

Giải thích: False chuyển sang thứ tự giảm dần. Sắp điểm `[3,1,2]` với `ascending=False` cho `[3,2,1]`, không giữ thứ tự gốc.

Q9. What distinguishes `loc` from `iloc`? — `loc` khác `iloc` ở đâu?

A. Retains selected rows versus grouping by key — Giữ hàng được chọn so với gom theo khóa

B. First two rows versus last two — Hai hàng đầu so với hai hàng cuối

C. Labels versus positions — Nhãn so với vị trí

D. Matching rows only versus all left rows — Chỉ hàng khớp so với mọi hàng trái

Đáp án: C

Giải thích: Nhãn 7 không nhất thiết nằm ở vị trí 7. Nhãn `'7'` có thể nằm tại vị trí đầu, nên `loc['7']` và `iloc[7]` có thể chọn các hàng khác nhau hoặc gây lỗi.

Q10. How does filtering differ from grouping? — Lọc khác phân nhóm như thế nào?

A. Matching rows only versus all left rows — Chỉ hàng khớp so với mọi hàng trái

B. Labels versus positions — Nhãn so với vị trí

C. First two rows versus last two — Hai hàng đầu so với hai hàng cuối

D. Retains selected rows versus grouping by key — Giữ hàng được chọn so với gom theo khóa

Đáp án: D

Giải thích: Lọc bỏ hàng không khớp; nhóm tổ chức bản ghi. Lọc chỉ giữ bản ghi thỏa điều kiện, còn phân nhóm tổ chức bản ghi theo khóa để có thể tính tổng từng nhóm bằng `sum()`.

Q11. How does `head(2)` differ from `tail(2)`? — `head(2)` khác `tail(2)` thế nào?

A. Retains selected rows versus grouping by key — Giữ hàng được chọn so với gom theo khóa

B. First two rows versus last two — Hai hàng đầu so với hai hàng cuối

C. Labels versus positions — Nhãn so với vị trí

D. Matching rows only versus all left rows — Chỉ hàng khớp so với mọi hàng trái

Đáp án: B

Giải thích: Hai hàm lấy dữ liệu ở hai đầu bảng. Với năm hàng, `head(2)` lấy hai hàng đầu, còn `tail(2)` lấy hàng thứ tư và thứ năm theo thứ tự hiện tại.

Q12. How does an inner join differ from a left join? — Inner join khác left join thế nào?

A. First two rows versus last two — Hai hàng đầu so với hai hàng cuối

B. Matching rows only versus all left rows — Chỉ hàng khớp so với mọi hàng trái

C. Retains selected rows versus grouping by key — Giữ hàng được chọn so với gom theo khóa

D. Labels versus positions — Nhãn so với vị trí

Đáp án: B

Giải thích: Inner join bỏ hàng trái không ghép được. Nếu một khách trong bảng trái không có hàng ghép bên phải, inner join bỏ khách đó, còn left join giữ lại với các trường phải bị thiếu.

Q13. What risk comes from duplicate right-side join keys? — Khóa ghép bảng phải bị trùng gây nguy cơ gì?

A. Unexpected row multiplication — Nhân số hàng ngoài ý muốn

B. Ascending order — Thứ tự tăng dần

C. Unique right-side join keys — Khóa ghép bảng phải duy nhất

D. Rebuilds sequential row labels — Đánh lại nhãn hàng liên tiếp

Đáp án: A

Giải thích: Mỗi hàng phải khớp thêm có thể nhân dữ liệu trái. Một khóa trái khớp hai hàng phải có thể sinh hai hàng ghép, làm tổng bị phóng đại nếu không dự kiến khóa trùng.

Q14. What does `validate='many_to_one'` check? — `validate='many_to_one'` kiểm tra gì?

A. Rebuilds sequential row labels — Đánh lại nhãn hàng liên tiếp

B. Unique right-side join keys — Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication — Nhân số hàng ngoài ý muốn

D. Ascending order — Thứ tự tăng dần

Đáp án: B

Giải thích: Nhiều hàng trái được phép ghép một hàng phải. Kiểm tra này cho phép nhiều hàng trái có cùng khóa nhưng yêu cầu phía phải tối đa một hàng cho mỗi khóa, nếu không Pandas báo lỗi kiểm chứng merge.

Q15. What does `reset_index(drop=True)` change? — `reset_index(drop=True)` thay đổi gì?

A. Ascending order — Thứ tự tăng dần

B. Unique right-side join keys — Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication — Nhân số hàng ngoài ý muốn

D. Rebuilds sequential row labels — Đánh lại nhãn hàng liên tiếp

Đáp án: D

Giải thích: Đánh lại index loại bỏ nhãn hàng cũ. Sau khi lọc còn nhãn `[2,5]`, `reset_index(drop=True)` đánh nhãn lại `[0,1]` mà không giữ nhãn cũ thành cột.

Q16. What does `ascending=True` request? — `ascending=True` yêu cầu gì?

A. Ascending order — Thứ tự tăng dần

B. Unique right-side join keys — Khóa ghép bảng phải duy nhất

C. Unexpected row multiplication — Nhân số hàng ngoài ý muốn

D. Rebuilds sequential row labels — Đánh lại nhãn hàng liên tiếp

Đáp án: A

Giải thích: Tăng dần là hướng sắp xếp mặc định. Khi sắp tăng dần, các số 1, 2, 3 theo đúng thứ tự đó; `ascending=False` chuyển sang giảm dần.

Q17. What does `df.iloc[0]` select? — `df.iloc[0]` chọn gì?

A. The first row by position — Hàng đầu theo vị trí

B. Keys must have compatible representations — Khóa cần biểu diễn tương thích

C. Rows within each group — Số hàng trong từng nhóm

D. Matching rows in named columns a and b — Hàng phù hợp ở hai cột a và b

Đáp án: A

Giải thích: Vị trí đầu tiên mang số 0. `iloc[0]` chọn vị trí không bất kể nhãn hàng là gì; thao tác thường trả một Series đại diện cho hàng đầu.

Q18. What does `df.loc[mask, ['a','b']]` select? — `df.loc[mask, ['a','b']]` chọn gì?

A. Rows within each group — Số hàng trong từng nhóm

B. The first row by position — Hàng đầu theo vị trí

C. Keys must have compatible representations — Khóa cần biểu diễn tương thích

D. Matching rows in named columns a and b — Hàng phù hợp ở hai cột a và b

Đáp án: D

Giải thích: Mask lọc hàng, danh sách tên lọc cột. Mask Boolean chọn các hàng đạt điều kiện, còn `['a','b']` giới hạn kết quả ở đúng hai cột có tên đó.

Q19. What does `groupby('type').size()` count? — `groupby('type').size()` đếm gì?

A. Rows within each group — Số hàng trong từng nhóm

B. Matching rows in named columns a and b — Hàng phù hợp ở hai cột a và b

C. The first row by position — Hàng đầu theo vị trí

D. Keys must have compatible representations — Khóa cần biểu diễn tương thích

Đáp án: A

Giải thích: size đếm số hàng ở mỗi nhóm. Khác `count()` trên một cột có giá trị thiếu, `size()` của nhóm vẫn đếm hàng ngay cả khi ô ở cột khác bị thiếu.

Q20. Why check types of columns used as merge keys? — Vì sao cần kiểm tra kiểu cột khóa khi merge?

A. Rows within each group — Số hàng trong từng nhóm

B. Keys must have compatible representations — Khóa cần biểu diễn tương thích

C. Matching rows in named columns a and b — Hàng phù hợp ở hai cột a và b

D. The first row by position — Hàng đầu theo vị trí

Đáp án: B

Giải thích: Kiểu khóa khác nhau hoặc sai có thể cản trở ghép đúng. Khóa chuỗi `'001'` không có biểu diễn giống số `1`, nên cần chuẩn hóa kiểu và định dạng trước khi ghép dữ liệu.

8. Nối sang bài tiếp

`DataFrame → lọc → nhóm → ghép bảng → AI`.

PY-20 — Logic mệnh đề, bảng chân trị và quy tắc đếm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Proposition — mệnh đề; **Truth table** — bảng chân trị.

`Boolean → logic mệnh đề → tổ hợp → xác suất`

Học xong dùng để làm gì? Lập bảng đúng/sai của quy tắc và đếm tổ hợp lựa chọn, ví dụ điều kiện xét duyệt.

Kết nối kiến thức tiếp theo: PY-21 dùng tập hợp và bao hàm–loại trừ để xử lý nhóm bị chồng lấn.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Proposition	mệnh đề	Câu có giá trị True hoặc False
Truth table	bảng chân trị	Liệt kê kết quả ứng với mọi tổ hợp logic
Implication	kéo theo	Quan hệ P⇒Q tương đương (not P) or Q
Counting	nguyên lý đếm	Đếm số khả năng khi ghép lựa chọn

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`and/or/not`	Ghép điều kiện, chọn ít nhất một hoặc phủ định
`(not p) or q`	Biểu diễn phép kéo theo P⇒Q
`itertools.product(A,B)`	Tạo tích Descartes của hai tập tùy chọn
`math.prod(counts)`	Nhân số lựa chọn từng bước độc lập

4. Khái niệm liên quan

Logic mệnh đề giúp chương trình quyết định đúng/sai theo quy tắc rõ ràng; **bảng chân trị** giúp kiểm tra tất cả trường hợp của nhiều điều kiện; **phép kéo theo** biểu diễn quy định "nếu... thì...". Khi phải đếm các lựa chọn có thể xảy ra, **nguyên lý cộng và nhân** cho kết quả mà không cần thử bằng tay từng cách.

4.1. Proposition — Phát biểu đúng hoặc sai

Từ vựng trong mục: Proposition — mệnh đề; **Boolean** — đúng/sai.

Mệnh đề là một phát biểu có giá trị đúng hoặc sai xác định. Chẳng hạn `3 > 1` là đúng, còn `3 < 1` là sai; trong Python, hai kết quả được biểu diễn bằng `True` và `False`. Không phải mọi câu đều là mệnh đề: câu hỏi "Bạn khỏe không?" chưa mang một giá trị đúng/sai để tính toán. Ta dùng mệnh đề khi viết điều kiện kiểm tra truy cập hoặc quy tắc vận hành.

```
print(3 > 1)
```

Kết quả khi chạy:

```
True
```

Mệnh đề `3 > 1` đúng vì số 3 lớn hơn 1; `print` hiển thị Boolean `True`.

4.2. Truth table — Bảng chân trị

Từ vựng trong mục: Truth table — bảng chân trị; **AND/OR/NOT** — và/hoặc/phủ định.

Bảng chân trị liệt kê mọi kết hợp đúng/sai có thể có của các điều kiện. Với hai biến p, q, mỗi biến có hai trạng thái nên có `2×2 = 4` cặp. `p and q` chỉ đúng khi cả hai đúng; `p or q` đúng nếu **ít nhất một** đúng; `not p` đảo đúng thành sai và ngược lại. Bảng giúp kiểm tra quy tắc ngay cả ở trường hợp khó nhận ra bằng trực giác.

```
for p in [True, False]:
    for q in [True, False]:
        print(p, q, p and q, p or q)
```

Kết quả khi chạy:

```
True True True True
True False False True
False True False True
False False False False
```

Hai cột đầu là đầu vào p, q; cột thứ ba là AND; cột cuối là OR. Bốn dòng bao phủ mọi trường hợp.

4.3. Implication — Nếu p thì q

Từ vựng trong mục: Implication — kéo theo; **Premise/conclusion** — tiền đề/kết luận.

Mệnh đề kéo theo `p ⇒ q` diễn đạt "nếu p thì q", trong đó p là tiền đề và q là kết luận. Nó **chỉ sai** khi p đúng mà q sai. Ví dụ quy định "nếu đơn được phê duyệt (p), đơn phải có chữ ký (q)": phê duyệt `True` nhưng có chữ ký `False` là vi phạm. Trong Python ta viết `(not p) or q`. Lưu ý đây là cách kiểm tra logic, không tự chứng minh quan hệ nguyên nhân.

```
approved = True
signed = False
print((not approved) or signed)
```

Kết quả khi chạy:

```
False
```

Đã phê duyệt nhưng chưa ký nên quy tắc kéo theo không được thỏa mãn.

4.4. Counting — Nguyên lý cộng và nhân

Từ vựng trong mục: Counting — quy tắc đếm; **Product** — tích Descartes.

Nguyên lý nhân: nếu có 3 cách chọn màu và mỗi màu đều có 4 cỡ, số cặp (màu, cỡ) là `3×4 = 12`. **Nguyên lý cộng** áp dụng cho lựa chọn thuộc **một trong các nhóm không trùng nhau**: có 3 mẫu áo hoặc 4 mẫu quần để chọn **một món**, tổng cộng 7 cách. Nếu các nhóm có phần giao, phải trừ phần bị đếm lặp; bài tiếp theo sẽ học kỹ phép bao hàm–loại trừ. `itertools.product` liệt kê từng cặp thay vì chỉ đếm.

```
colors = 3
sizes = 4
print(colors * sizes)
```

Kết quả khi chạy:

```
12
```

Có 12 cách chọn một màu và một cỡ khi mọi màu đều có đủ bốn cỡ.

5. Năm tình huống lập trình thực tế

Tình huống 1 — Điều kiện đăng nhập

Từ vựng trong mục: Proposition — mệnh đề; **AND** — và.

Tình huống thực tế: Một cổng học tập chỉ cho mở nội dung khi người dùng đã đăng nhập và xác minh tài khoản. Trong lần kiểm tra này `logged_in=True`, `verified=False`. In kết quả người dùng có được quyền truy cập hay không.

Vì sao chọn cách giải này? Toán tử `and` chỉ cho `True` khi cả hai điều kiện đều đúng.

Quy trình giải: Đặt hai trạng thái Boolean → nối chúng bằng `and` → in kết quả.

```
logged_in, verified = True, False
print(logged_in and verified)
```

Kết quả khi chạy:

```
False
```

Giải thích kết quả: `False` — tài khoản đã đăng nhập nhưng chưa xác minh.

Tình huống 2 — Bảng chân trị

Từ vựng trong mục: Truth table — bảng chân trị; **OR** — hoặc.

Tình huống thực tế: Nhân viên kiểm thử muốn lập bảng chân trị cho `p and q` và `p or q`, trong đó `p`, `q` đều có thể là `True` hoặc `False`. In tất cả bốn tổ hợp của p và q cùng kết quả của hai phép logic.

Vì sao chọn cách giải này? Hai vòng `for` lồng nhau tạo đủ 2 × 2 tổ hợp.

Quy trình giải: Duyệt p theo hai trạng thái → với mỗi p duyệt q → in p, q, p and q, p or q.

```
for p in [True, False]:
    for q in [True, False]:
        print(p, q, p and q, p or q)
```

Kết quả khi chạy:

```
True True True True
True False False True
False True False True
False False False False
```

Giải thích kết quả: Bốn dòng cho đủ các trường hợp; `and` chỉ đúng khi cả hai đúng, `or` đúng khi ít nhất một đúng.

Tình huống 3 — Kiểm quy tắc kéo theo

Từ vựng trong mục: Implication — kéo theo; **Premise** — tiền đề.

Tình huống thực tế: Một quy định nội bộ phát biểu: "Nếu đã được phê duyệt (p) thì phải có chứng nhận (q)". Có trường hợp `p=True` nhưng `q=False`. Kiểm tra quy tắc kéo theo có được đáp ứng không.

Vì sao chọn cách giải này? `(not p) or q` tương đương phép kéo theo và nhận `False` khi tiền đề đúng nhưng kết luận sai.

Quy trình giải: Gán p=True, q=False → tính `(not p) or q` → in kết quả.

```
p, q = True, False
print((not p) or q)
```

Kết quả khi chạy:

```
False
```

Giải thích kết quả: `False` — trường hợp này vi phạm quy tắc.

Tình huống 4 — Cấu hình áo

Từ vựng trong mục: Counting — nguyên lý đếm; **Combination** — cách kết hợp.

Tình huống thực tế: Cửa hàng cho khách chọn 1 trong 3 màu áo và 1 trong 4 cỡ; mọi màu đều có đủ bốn cỡ. Tính số cách chọn một màu và một cỡ.

Vì sao chọn cách giải này? Nguyên lý nhân phù hợp khi mỗi lựa chọn màu kết hợp được với mọi lựa chọn cỡ.

Quy trình giải: Lấy số màu nhân số cỡ: 3 × 4 → in tổng số kết hợp.

```
print(3 * 4)
```

Kết quả khi chạy:

```
12
```

Giải thích kết quả: 12 cách chọn.

Tình huống 5 — Ghép các tùy chọn

Từ vựng trong mục: Cartesian product — tích Descartes; **Option** — lựa chọn.

Tình huống thực tế: Cửa hàng bán áo có hai màu `red`, `blue` và hai cỡ `S`, `M`, mọi màu đều có đủ hai cỡ. In danh sách cụ thể tất cả cặp (màu, cỡ).

Vì sao chọn cách giải này? `itertools.product` liệt kê tích Descartes, khác với chỉ đếm bằng phép nhân.

Quy trình giải: Tạo list hai màu và hai cỡ → dùng `product` → chuyển sang list và in.

```
from itertools import product
print(list(product(["red", "blue"], ["S", "M"])))
```

Kết quả khi chạy:

```
[('red', 'S'), ('red', 'M'), ('blue', 'S'), ('blue', 'M')]
```

Giải thích kết quả: Bốn cặp: `(red,S)`, `(red,M)`, `(blue,S)`, `(blue,M)`.

6. Sai lầm thường gặp

- Không nhầm `and` (cả hai đúng) với `or` (ít nhất một đúng); `not` đảo giá trị Boolean.
- Phép kéo theo `p ⇒ q` chỉ sai khi `p=True`, `q=False`; tiền đề sai không tự chứng minh kết luận đúng.
- Bảng chân trị hai biến có bốn tổ hợp; nếu thiếu một tổ hợp, kiểm tra logic có thể bỏ sót lỗi.
- Nguyên lý **nhân** đếm lựa chọn theo nhiều bước kết hợp được với nhau; nguyên lý **cộng** đếm các phương án thay thế không trùng. Nếu trùng phải điều chỉnh số đếm.
- Set không bảo đảm thứ tự in, còn kết quả `itertools.product` gồm các tuple theo thứ tự nguồn đã đưa vào.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a proposition? — Mệnh đề là gì?

A. A table of all input truth-value combinations — Bảng các tổ hợp giá trị đúng/sai đầu vào

B. The logical conditional 'if p then q' — Điều kiện logic 'nếu p thì q'

C. A statement that is either true or false — Phát biểu có thể xác định đúng hoặc sai

D. Multiply choice counts across sequential stages when each earlier choice permits the stated number of later choices — Nhân số cách chọn qua các bước khi mỗi lựa chọn trước cho phép đúng số cách ở bước sau

Đáp án: C

Giải thích: Mệnh đề có một giá trị đúng/sai xác định. Ví dụ `2 + 2 = 4` là mệnh đề đúng, còn câu hỏi như `Mấy giờ rồi?` không tự khẳng định một giá trị đúng hay sai.

Q2. What is a truth table? — Bảng chân trị là gì?

A. Multiply the numbers of sequential independent choices — Nhân số lựa chọn qua các bước kết hợp đầy đủ

B. A statement that is either true or false — Phát biểu có thể xác định đúng hoặc sai

C. A table of all input truth-value combinations — Bảng các tổ hợp giá trị đúng/sai đầu vào

D. The logical conditional 'if p then q' — Điều kiện logic 'nếu p thì q'

Đáp án: C

Giải thích: Bảng chân trị liệt kê các khả năng Boolean. Với hai mệnh đề p và q, bốn hàng là (T,T), (T,F), (F,T), (F,F); bảng xét mọi cách gán giá trị.

Q3. What is logical implication `p ⇒ q`? — Phép kéo theo `p ⇒ q` là gì?

A. The logical conditional 'if p then q' — Điều kiện logic 'nếu p thì q'

B. A statement that is either true or false — Phát biểu có thể xác định đúng hoặc sai

C. Multiply the numbers of sequential independent choices — Nhân số lựa chọn qua các bước kết hợp đầy đủ

D. A table of all input truth-value combinations — Bảng các tổ hợp giá trị đúng/sai đầu vào

Đáp án: A

Giải thích: Kéo theo chỉ sai khi tiền đề đúng, kết luận sai. Phép kéo theo đòi khi tiền đề p đúng thì kết luận q không được sai; khi p sai, kéo theo vật chất vẫn đúng bất kể q.

Q4. What is the multiplication counting principle? — Nguyên lý nhân trong đếm là gì?

A. A statement that is either true or false — Phát biểu có thể xác định đúng hoặc sai

B. A table of all input truth-value combinations — Bảng các tổ hợp giá trị đúng/sai đầu vào

C. The logical conditional 'if p then q' — Điều kiện logic 'nếu p thì q'

D. Multiply stage-choice counts when each earlier choice permits the same stated number of valid continuations — Nhân số cách chọn mỗi bước khi mọi lựa chọn trước đều cho phép cùng số cách đi tiếp hợp lệ

Đáp án: D

Giải thích: Nếu bước đầu có m cách và mỗi cách có đúng n lựa chọn nối tiếp thì có m×n kết quả hoàn chỉnh. Quy tắc đếm theo từng bước này không đòi các biến cố độc lập về xác suất. Nếu có hai cách chọn áo và mỗi áo đều ghép được với ba quần, tổng cộng `2 × 3 = 6` bộ; không cần giả thiết độc lập xác suất.

Q5. When is `p and q` true? — Khi nào `p and q` đúng?

A. Reverses the truth value of p — Đảo giá trị đúng/sai của p

B. When p is true and q is false — Khi p đúng và q sai

C. When at least one of p or q is true — Khi có ít nhất một trong p hoặc q đúng

D. When both p and q are true — Khi p và q đều đúng

Đáp án: D

Giải thích: AND cần cả hai điều kiện đúng. Nếu p đúng nhưng q sai thì `p and q` sai; một điều kiện đúng chưa đủ cho phép AND.

Q6. When is `p or q` true? — Khi nào `p or q` đúng?

A. Reverses the truth value of p — Đảo giá trị đúng/sai của p

B. When p is true and q is false — Khi p đúng và q sai

C. When at least one of p or q is true — Khi có ít nhất một trong p hoặc q đúng

D. When both p and q are true — Khi p và q đều đúng

Đáp án: C

Giải thích: OR bao hàm chấp nhận một hoặc cả hai điều kiện đúng. Đây là OR bao hàm: p và q có thể cùng đúng, khác XOR chỉ đúng khi đúng một vế.

Q7. What does `not p` do? — `not p` làm gì?

A. When p is true and q is false — Khi p đúng và q sai

B. Reverses the truth value of p — Đảo giá trị đúng/sai của p

C. When both p and q are true — Khi p và q đều đúng

D. When at least one of p or q is true — Khi có ít nhất một trong p hoặc q đúng

Đáp án: B

Giải thích: NOT phủ định giá trị Boolean. Nếu p=True thì `not p=False`; nếu p=False thì phủ định trở thành True.

Q8. When is `p ⇒ q` false? — Khi nào `p ⇒ q` sai?

A. When both p and q are true — Khi p và q đều đúng

B. When at least one of p or q is true — Khi có ít nhất một trong p hoặc q đúng

C. When p is true and q is false — Khi p đúng và q sai

D. Reverses the truth value of p — Đảo giá trị đúng/sai của p

Đáp án: C

Giải thích: Tiền đề đúng nhưng kết luận sai làm phép kéo theo sai. Với p=True và q=False, `(not p) or q` thành `(False or False)`, nên phép kéo theo sai.

Q9. How many rows does a truth table with two variables have? — Bảng chân trị hai biến có mấy hàng?

A. 8 — 8

B. 16 — 16

C. 2 — 2

D. 4 — 4

Đáp án: D

Giải thích: Hai biến có hai trạng thái mỗi biến, tạo 2^2 trường hợp. Mỗi biến Boolean có hai giá trị, tạo $2 \times 2 = 4$ tổ hợp chứ không chỉ hai tổ hợp.

Q10. How many rows does a truth table with three variables have? — Bảng chân trị ba biến có mấy hàng?

A. 2 — 2

B. 4 — 4

C. 16 — 16

D. 8 — 8

Đáp án: D

Giải thích: Ba lựa chọn nhị phân tạo 2^3 tổ hợp. Ba biến Boolean tạo $2 \times 2 \times 2 = 8$ cách gán đầu vào cho bảng chân trị.

Q11. How many rows does a truth table with one variable have? — Bảng chân trị một biến có mấy hàng?

A. 4 — 4

B. 8 — 8

C. 2 — 2

D. 16 — 16

Đáp án: C

Giải thích: Một biến Boolean có True và False. Hai hàng ứng với biến duy nhất nhận True và False; bảng có `2^1 = 2` hàng.

Q12. How many rows does a truth table with four variables have? — Bảng chân trị bốn biến có mấy hàng?

A. 16 — 16

B. 4 — 4

C. 2 — 2

D. 8 — 8

Đáp án: A

Giải thích: Bốn biến Boolean tạo 2^4 khả năng. Mỗi biến Boolean thêm vào nhân đôi số hàng, nên bốn biến cần `2^4 = 16` tổ hợp.

Q13. Which expression is equivalent to implication `p ⇒ q`? — Biểu thức nào tương đương `p ⇒ q`?

A. `False` — `False`

B. `(not p) or q` — `(not p) or q`

C. `(True, False)` — `(True, False)`

D. `True` — `True`

Đáp án: B

Giải thích: Biểu thức chỉ sai khi p=True, q=False. Cả `p ⇒ q` và `(not p) or q` chỉ sai khi p=True, q=False nên bảng chân trị của chúng trùng nhau.

Q14. Which input pair makes `p ⇒ q` false? — Cặp p, q nào làm `p ⇒ q` sai?

A. `(True, False)` — `(True, False)`

B. `(not p) or q` — `(not p) or q`

C. `True` — `True`

D. `False` — `False`

Đáp án: A

Giải thích: Tiền đề đúng và kết luận sai là trường hợp duy nhất làm phép kéo theo sai. Thứ tự cặp là `(p, q)`, nên `(True, False)` có tiền đề đúng và kết luận sai như yêu cầu.

Q15. What is `True and False`? — `True and False` bằng gì?

A. `True` — `True`

B. `False` — `False`

C. `(not p) or q` — `(not p) or q`

D. `(True, False)` — `(True, False)`

Đáp án: B

Giải thích: AND sai nếu có một vế sai. Kết quả là Boolean `False`, không phải số nguyên 0; vế thứ hai sai trong khi AND cần cả hai đúng.

Q16. What is `False or True`? — `False or True` bằng gì?

A. `True` — `True`

B. `(True, False)` — `(True, False)`

C. `False` — `False`

D. `(not p) or q` — `(not p) or q`

Đáp án: A

Giải thích: OR đúng khi có một vế đúng. Vì có một vế đúng, OR bao hàm trả Boolean `True`; `False and True` thì lại sai.

Q17. When does the addition counting principle apply directly? — Khi nào có thể áp dụng trực tiếp nguyên lý cộng?

A. All ordered pairs taking one element from each input — Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

B. When alternatives are disjoint — Khi các phương án thay thế không trùng nhau

C. When some outcomes belong to more than one counted group — Khi một kết quả nằm trong nhiều nhóm được đếm

D. When each step has the stated choices for every earlier selection — Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

Đáp án: B

Giải thích: Các nhóm không trùng có thể cộng mà không đếm lặp. Nếu năm kết quả chỉ thuộc nhóm A và ba kết quả chỉ thuộc nhóm B, tổng có `5 + 3 = 8` phương án không đếm trùng.

Q18. When can choices be counted by multiplying options? — Khi nào có thể đếm bằng phép nhân?

A. When each step has the stated choices for every earlier selection — Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

B. When alternatives are disjoint — Khi các phương án thay thế không trùng nhau

C. All ordered pairs taking one element from each input — Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

D. When some outcomes belong to more than one counted group — Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: A

Giải thích: Nguyên lý nhân yêu cầu số nhánh lựa chọn phù hợp ở từng bước. Nếu mỗi lựa chọn bước đầu đều có đúng ba cách đi tiếp thì nhân số cách mỗi bước; khi số nhánh thay đổi phải cộng số cách theo từng nhánh cẩn thận.

Q19. What does `itertools.product(A, B)` generate? — `itertools.product(A, B)` tạo gì?

A. When each step has the stated choices for every earlier selection — Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

B. All ordered pairs taking one element from each input — Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

C. When alternatives are disjoint — Khi các phương án thay thế không trùng nhau

D. When some outcomes belong to more than one counted group — Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: B

Giải thích: product liệt kê tích Descartes. Với `A=[1,2]` và `B=['x','y']`, product tạo `(1,'x')`, `(1,'y')`, `(2,'x')`, `(2,'y')`, không chỉ ghép vị trí tương ứng.

Q20. When should overlapping alternatives be adjusted during counting? — Khi nào cần điều chỉnh phần giao khi đếm?

A. All ordered pairs taking one element from each input — Mọi cặp có thứ tự gồm một phần tử từ mỗi nguồn

B. When each step has the stated choices for every earlier selection — Khi mỗi bước có đủ số lựa chọn với mọi lựa chọn trước đó

C. When alternatives are disjoint — Khi các phương án thay thế không trùng nhau

D. When some outcomes belong to more than one counted group — Khi một kết quả nằm trong nhiều nhóm được đếm

Đáp án: D

Giải thích: Nhóm có giao nhau không được đếm trùng. Với hai nhóm có giao, công thức |A ∪ B| = |A| + |B| − |A ∩ B| trừ phần được tính hai lần.

8. Nối sang bài tiếp

Boolean → logic mệnh đề → tổ hợp → xác suất.

PY-21 — Tập hợp, bao hàm–loại trừ và phân hoạch

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Union — phép hợp; **Intersection** — phép giao.

`Set → hợp/giao/hiệu → bao hàm–loại trừ → train/test`

Học xong dùng để làm gì? Đếm số người thuộc nhiều nhóm mà không tính trùng, phân hoạch đối tượng thành các tập không giao nhau.

Kết nối kiến thức tiếp theo: PY-22 đếm số cách chọn nhóm và xếp thứ tự từ tập phần tử.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Union	phép hợp	Lấy tất cả phần tử thuộc ít nhất một tập
Intersection	phép giao	Lấy phần tử có trong cả hai tập
Inclusion–exclusion	bao hàm–loại trừ	Cộng từng tập rồi trừ phần giao bị đếm hai lần
Partition	phân hoạch	Chia thành các nhóm rời nhau phủ đủ tập gốc

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`a | b, a & b`	Hợp và giao các tập hợp
`a - b, a ^ b`	Hiệu và hiệu đối xứng
`len(a | b)`	Đếm phần tử khác nhau
`a.isdisjoint(b)`	Kiểm tra hai nhóm có trùng phần tử không

4. Khái niệm liên quan

Tập hợp giúp lưu các đối tượng **khác nhau** rồi so sánh thành viên giữa các nhóm. Ta dùng hai lớp học Python và AI làm ví dụ xuyên suốt: phép hợp tìm tất cả học viên, phép giao tìm người học cả hai, quy tắc bao hàm–loại trừ giúp đếm không trùng, còn phân hoạch giúp chia bộ dữ liệu thành các phần rời nhau.

4.1. Set và phép hợp — Ai tham gia ít nhất một nhóm?

Từ vựng trong mục: Set — tập hợp; **Union** — phép hợp.

Set không giữ phần tử trùng. Lớp Python gồm SV1, SV2; lớp AI gồm SV2, SV3. Phép **hợp** `python | ai` chứa tất cả người có mặt trong **ít nhất một** lớp. SV2 nằm ở cả hai nhưng chỉ xuất hiện một lần. Khi cần đầu ra ổn định, dùng `sorted()` vì Set không bảo đảm thứ tự hiển thị.

```
python = {'SV1', 'SV2'}
ai = {'SV2', 'SV3'}
print(sorted(python | ai))
```

Kết quả khi chạy:

```
['SV1', 'SV2', 'SV3']
```

Ba mã học viên khác nhau thuộc ít nhất một lớp; `sorted` tạo List để hiển thị theo thứ tự rõ ràng.

4.2. Intersection và Difference — Phần chung và phần riêng

Từ vựng trong mục: Intersection — phép giao; **Difference** — hiệu tập hợp.

Giao `a & b` chỉ lấy phần tử thuộc cả hai nhóm; với lớp Python {SV1,SV2} và AI {SV2,SV3}, phần chung là SV2. **Hiệu** `a - b` chỉ lấy phần tử thuộc a nhưng không thuộc b, tức SV1 trong ví dụ. Hiệu khác hiệu đối xứng `a ^ b`, vốn chứa các phần tử chỉ có ở đúng một nhóm.

```
a = {'SV1', 'SV2'}
b = {'SV2', 'SV3'}
print(sorted(a & b), sorted(a - b))
```

Kết quả khi chạy:

```
['SV2'] ['SV1']
```

SV2 học cả hai lớp, còn SV1 chỉ học Python. Hai phép toán trả lời hai câu hỏi khác nhau.

4.3. Inclusion–exclusion — Đếm không lặp người

Từ vựng trong mục: Inclusion–exclusion — bao hàm–loại trừ; **Cardinality** — số phần tử.

Nếu cộng số học viên hai lớp trực tiếp, người học cả hai bị đếm **hai lần**. Ví dụ lớp Python có 2 người, lớp AI có 2 người, một người chung: số người khác nhau bằng `2 + 2 - 1 = 3`. Công thức `|A ∪ B| = |A| + |B| - |A ∩ B|` trừ phần bị đếm lặp. Với hơn hai tập, cần tính thêm các giao nhiều tập.

```
a = {'SV1', 'SV2'}
b = {'SV2', 'SV3'}
print(len(a) + len(b) - len(a & b))
```

Kết quả khi chạy:

```
3
```

Kết quả đếm ba học viên duy nhất, không coi SV2 là hai người khác nhau.

4.4. Partition — Chia đủ dữ liệu thành phần rời

Từ vựng trong mục: Partition — phân hoạch; **Disjoint** — không giao nhau.

Phân hoạch tập dữ liệu gốc thành các nhóm **không rỗng**, **không giao nhau**, và **hợp lại bằng đúng tập gốc**. Ví dụ toàn bộ mã `{1,2,3,4,5}`, train `{1,2,3}`, test `{4,5}`. `isdisjoint` chỉ kiểm tra không trùng; để chứng minh phân hoạch còn phải kiểm tra hợp của chúng có phủ đủ dữ liệu gốc. Với Machine Learning, không trùng ID **chưa chắc** loại bỏ mọi hình thức rò rỉ dữ liệu.

```
all_ids = {1, 2, 3, 4, 5}
train = {1, 2, 3}
test = {4, 5}
print(bool(train) and bool(test) and train.isdisjoint(test) and train | test == all_ids)
```

Kết quả khi chạy:

```
True
```

Hai tập đều có dữ liệu, không trùng và phủ đủ toàn bộ năm mã, nên tạo một phân hoạch hợp lệ.

5. Năm vấn đề thực tế

Tình huống 1 — Đếm người học ít nhất một môn

Từ vựng trong mục: Union — phép hợp.

Tình huống thực tế: Một lớp có học viên SV1, SV2, SV3 đăng ký môn Python và SV2, SV4 đăng ký AI. Đếm số học viên học ít nhất một trong hai môn.

Vì sao chọn cách giải này? Phép hợp chỉ giữ mỗi mã học viên một lần, đúng yêu cầu đếm người học ít nhất một môn.

Quy trình giải:

1. Tạo Set mã người đăng ký Python và Set mã người đăng ký AI.
2. Dùng phép hợp để loại mã xuất hiện trong cả hai nhóm.
3. Dùng `len` đếm số học viên duy nhất.

```
a = {"SV1", "SV2", "SV3"}
b = {"SV2", "SV4"}
print(len(a|b))
```

Kết quả khi chạy:

```
4
```

Tình huống 2 — Người học cả hai môn

Từ vựng trong mục: Intersection — phép giao.

Tình huống thực tế: Hai nhóm học viên A gồm SV1, SV2; B gồm SV2, SV3. Tìm mã học viên có mặt ở cả hai nhóm.

Vì sao chọn cách giải này? Phép giao trả đúng thành viên có ở cả hai nhóm, không cần duyệt từng mã bằng vòng lặp.

Quy trình giải:

1. Lưu danh sách mã của hai nhóm thành Set.
2. Dùng phép giao `&` tìm người có mặt ở cả hai.
3. In kết quả dưới dạng danh sách có thứ tự để dễ đối chiếu.

```
a = {"SV1", "SV2"}
b = {"SV2", "SV3"}
print(sorted(a & b))
```

Kết quả khi chạy:

```
['SV2']
```

Tình huống 3 — Kiểm công thức đếm

Từ vựng trong mục: Inclusion–exclusion — bao hàm–loại trừ.

Tình huống thực tế: Hai nhóm mã ID {1,2,3} và {3,4} trùng mã 3. Tính tổng số mã khác nhau và đối chiếu với phép hợp.

Vì sao chọn cách giải này? Bao hàm–loại trừ trừ đi phần đã bị tính hai lần và đối chiếu được với phép hợp.

Quy trình giải:

1. Xác định số mã trong A, B và phần giao.
2. Tính `len(a) + len(b) - len(a & b)`.
3. So sánh với `len(a | b)` để kiểm chứng hai cách tính.

```
a = {1, 2, 3}
b = {3, 4}
print(len(a)+len(b)-len(a&b), len(a|b))
```

Kết quả khi chạy:

```
4 4
```

Tình huống 4 — Chia tập train/test

Từ vựng trong mục: Partition — phân hoạch.

Tình huống thực tế: Đội AI dùng mã dữ liệu {1,2,3} làm tập huấn luyện, {4,5} làm tập kiểm tra. Kiểm tra hai tập có trùng ID không.

Vì sao chọn cách giải này? Phân hoạch không chỉ cần hai nhóm không trùng mà còn phải phủ đúng toàn bộ ID nguồn.

Quy trình giải:

1. Lưu ID train, ID test và tập ID toàn bộ.
2. Kiểm tra hai nhóm không giao nhau bằng `isdisjoint`.

3. Kiểm tra các ID của train và test gộp lại có đúng tập gốc và không rỗng.

```
train = {1, 2, 3}
test = {4, 5}
assert train.isdisjoint(test)
print(train.isdisjoint(test) and train | test == True)
```

Kết quả khi chạy:

```
{1, 2, 3, 4, 5}
```

Tình huống 5 — Tìm sở thích riêng

Từ vựng trong mục: Union — phép hợp.

Tình huống thực tế: Nhóm học Web gồm An, Bình; nhóm học AI gồm Bình, Lan. Tìm học viên chỉ tham gia đúng một trong hai nhóm.

Vì sao chọn cách giải này? Hiệu đối xứng trả thành viên chỉ nằm ở một trong hai nhóm; người có mặt cả hai bị loại.

Quy trình giải:

1. Lưu danh sách học viên của lớp Web và lớp AI thành hai Set.
2. Lấy hiệu đối xứng `web ^ ai` để bỏ người tham gia cả hai.
3. Sắp xếp và in những người chỉ ở một lớp.

```
web = {"An", "Binh"}
ai = {"Binh", "Lan"}
print(sorted(web ^ ai))
```

Kết quả khi chạy:

```
['An', 'Lan']
```

6. Sai lầm thường gặp

- Set không giữ thứ tự; dùng `sorted()` khi muốn kết quả mẫu có thứ tự ổn định.
- Phép hợp `|` khác phép giao `&`: hợp lấy ít nhất một nhóm, giao lấy cả hai nhóm.
- Hiệu `A - B` khác hiệu đối xứng `A ^ B`: một chiều so với chỉ xuất hiện ở đúng một trong hai tập.
- Bao hàm–loại trừ tránh đếm trùng. Nếu có ba tập trở lên, công thức cần cộng/trừ thêm các giao nhiều tập.
- `isdisjoint()` chỉ chứng minh hai tập không trùng, chưa chứng minh chúng phủ toàn bộ tập gốc.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What does a union contain? — Phép hợp chứa gì?

A. Elements in at least one set — Phần tử thuộc ít nhất một tập

B. Elements common to both sets — Phần tử có ở cả hai tập

C. Elements in A but not in B — Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set — Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: A

Giải thích: Hợp chứa các phần tử khác nhau của hai tập.

Q2. What does an intersection contain? — Phép giao chứa gì?

A. Elements in at least one set — Phần tử thuộc ít nhất một tập

B. Elements common to both sets — Phần tử có ở cả hai tập

C. Elements in A but not in B — Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set — Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: B

Giải thích: Giao chỉ giữ phần tử chung của hai tập.

Q3. What is set difference A−B? — Hiệu tập hợp A−B là gì?

A. Elements in at least one set — Phần tử thuộc ít nhất một tập

B. Elements common to both sets — Phần tử có ở cả hai tập

C. Elements in A but not in B — Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set — Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: C

Giải thích: Hiệu A−B loại phần tử của B ra khỏi A.

Q4. What defines a partition of a set? — Điều gì xác định một phân hoạch?

A. Elements in at least one set — Phần tử thuộc ít nhất một tập

B. Elements common to both sets — Phần tử có ở cả hai tập

C. Elements in A but not in B — Phần tử thuộc A nhưng không thuộc B

D. Nonempty disjoint parts covering the original set — Các phần không rỗng, rời nhau và phủ đủ tập gốc

Đáp án: D

Giải thích: Mọi phần không giao nhau và hợp bằng tập gốc.

Q5. Which operator computes union for sets? — Toán tử nào tính phép hợp Set?

A. `a | b` — `a | b`

B. `a & b` — `a & b`

C. `a ^ b` — `a ^ b`

D. `a.isdisjoint(b)` — `a.isdisjoint(b)`

Đáp án: A

Giải thích: Dấu | biểu diễn phép hợp tập hợp.

Q6. Which operator computes intersection? — Toán tử nào tính phép giao?

A. `a | b` — `a | b`

B. `a & b` — `a & b`

C. `a ^ b` — `a ^ b`

D. `a.isdisjoint(b)` — `a.isdisjoint(b)`

Đáp án: B

Giải thích: Dấu & giữ các phần tử chung.

Q7. Which operator computes symmetric difference? — Toán tử nào tính hiệu đối xứng?

A. `a | b` — `a | b`

B. `a & b` — `a & b`

C. `a ^ b` — `a ^ b`

D. `a.isdisjoint(b)` — `a.isdisjoint(b)`

Đáp án: C

Giải thích: Hiệu đối xứng giữ phần tử chỉ ở một bên.

Q8. Which method checks that two sets do not overlap? — Phương thức nào kiểm tra hai Set không trùng nhau?

A. `a | b` — `a | b`

B. `a & b` — `a & b`

C. `a ^ b` — `a ^ b`

D. `a.isdisjoint(b)` — `a.isdisjoint(b)`

Đáp án: D

Giải thích: isdisjoint đúng khi phần giao rỗng.

Q9. Why does inclusion–exclusion subtract an intersection? — Vì sao bao hàm–loại trừ phải trừ phần giao?

A. To correct double counting — Để sửa việc đếm trùng

B. Distinct elements in either set — Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members — Khi hai tập không có phần tử chung

D. An ordered List of its elements — List chứa các phần tử được sắp thứ tự

Đáp án: A

Giải thích: Phần tử giao đã được tính trong cả hai số đếm.

Q10. What does `len(a | b)` count? — `len(a | b)` đếm gì?

A. To correct double counting — Để sửa việc đếm trùng

B. Distinct elements in either set — Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members — Khi hai tập không có phần tử chung

D. An ordered List of its elements — List chứa các phần tử được sắp thứ tự

Đáp án: B

Giải thích: Hợp tự loại phần trùng của các nhóm.

Q11. When is `a.isdisjoint(b)` True? — Khi nào `a.isdisjoint(b)` bằng True?

A. To correct double counting — Để sửa việc đếm trùng

B. Distinct elements in either set — Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members — Khi hai tập không có phần tử chung

D. An ordered List of its elements — List chứa các phần tử được sắp thứ tự

Đáp án: C

Giải thích: Hai tập rời có phần giao rỗng.

Q12. What does `sorted(my_set)` produce? — `sorted(my_set)` tạo gì?

A. To correct double counting — Để sửa việc đếm trùng

B. Distinct elements in either set — Số phần tử khác nhau ở ít nhất một tập

C. When the sets share no members — Khi hai tập không có phần tử chung

D. An ordered List of its elements — List chứa các phần tử được sắp thứ tự

Đáp án: D

Giải thích: sorted tạo List và không sửa Set gốc.

Q13. How many distinct values are in `{1,1,2}`? — `{1,1,2}` có mấy phần tử khác nhau?

A. 2 — 2

B. 4 — 4

C. Every part is nonempty and the union covers the original set — Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared — Không có phần tử chung

Đáp án: A

Giải thích: Giá trị trùng chỉ xuất hiện một lần trong Set.

Q14. If |A|=3, |B|=2, |A∩B|=1, what is |A∪B|? — Nếu |A|=3, |B|=2, |A∩B|=1 thì |A∪B| bằng bao nhiêu?

A. 2 — 2

B. 4 — 4

C. Every part is nonempty and the union covers the original set — Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared — Không có phần tử chung

Đáp án: B

Giải thích: Dùng công thức 3+2−1=4.

Q15. Which two conditions besides disjointness are needed to verify a nonempty partition? — Ngoài rời nhau, cần kiểm tra gì để xác nhận các phần không rỗng là phân hoạch?

A. 2 — 2

B. 4 — 4

C. Every part is nonempty and the union covers the original set — Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared — Không có phần tử chung

Đáp án: C

Giải thích: Phân hoạch phải phủ đủ phần tử gốc và không có phần rỗng.

Q16. What does the empty intersection mean? — Phần giao rỗng nghĩa là gì?

A. 2 — 2

B. 4 — 4

C. Every part is nonempty and the union covers the original set — Mỗi phần không rỗng và hợp phủ đủ tập gốc

D. No element is shared — Không có phần tử chung

Đáp án: D

Giải thích: Giao rỗng biểu thị hai tập rời nhau.

Q17. What does A∪A equal? — A∪A bằng gì?

A. A — A

B. Empty set — Tập rỗng

C. No fixed order is promised — Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits — Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: A

Giải thích: Hợp một tập với chính nó không thêm phần tử.

Q18. What does A∩∅ equal? — A∩∅ bằng gì?

A. A — A

B. Empty set — Tập rỗng

C. No fixed order is promised — Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits — Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: B

Giải thích: Tập rỗng không có phần tử nào chung với A nên phép giao rỗng.

Q19. Is set iteration order guaranteed? — Thứ tự duyệt Set có được đảm bảo không?

A. A — A

B. Empty set — Tập rỗng

C. No fixed order is promised — Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits — Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: C

Giải thích: Set Python không phải dãy có thứ tự.

Q20. Why can two data splits with disjoint IDs still leak information? — Vì sao hai tập dữ liệu không trùng ID vẫn có thể rò rỉ thông tin?

A. A — A

B. A itself — Chính tập A

C. No fixed order is promised — Không bảo đảm thứ tự cố định

D. Related records or future information may cross splits — Bản ghi liên quan hoặc thông tin tương lai có thể lọt qua hai tập

Đáp án: D

Giải thích: ID không trùng chưa loại được mọi nguồn rò rỉ.

8. Nối sang bài tiếp

`Set → hợp/giao/hiệu → bao hàm–loại trừ → train/test.`

PY-22 — Hoán vị, tổ hợp và tam giác Pascal

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Permutation — hoán vị; **Combination** — tổ hợp.

`Nguyên lý đếm → hoán vị/tổ hợp → phân phối nhị thức`

Học xong dùng để làm gì? Tính số cách chọn người, chia vai hoặc sắp lịch, phân biệt có và không xét thứ tự.

Kết nối kiến thức tiếp theo: PY-23 mô hình hóa quan hệ và độ tương đồng giữa đối tượng bằng đồ thị/ Jaccard.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Permutation	hoán vị	Chọn có phân biệt thứ tự
Combination	tổ hợp	Chọn nhóm không xét thứ tự
Factorial	giai thừa	Tích các số từ 1 đến n
Binomial coefficient	hệ số nhị thức	Số cách chọn k từ n

3. Hàm Python và tham số

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Vì sao dùng
`math.factorial(n)`	Tính n!, n không âm
`math.perm(n,k)`	Đếm cách chọn có thứ tự k từ n
`math.comb(n,k)`	Đếm cách chọn không thứ tự
`itertools.combinations/permutations`	Liệt kê kết quả khi n nhỏ

4. Khái niệm liên quan

Khi chọn người hoặc sắp xếp công việc, câu hỏi đầu tiên là **đổi thứ tự có tạo ra kết quả khác không?** Nếu có, ta đếm các cách sắp xếp bằng **hoán vị**; nếu không, ta đếm **tổ hợp**. **Giai thừa** là phép nhân nền tảng để tính những số cách đó, còn **hệ số nhị thức** tạo nên các hàng của tam giác Pascal.

4.1. Permutation — Chọn và phân vai có thứ tự

Từ vựng trong mục: Permutation — chỉnh hợp/hoán vị có thứ tự; **Order** — thứ tự.

Có năm ứng viên và hai vị trí **khác nhau** là trưởng nhóm, phó nhóm. Chọn trưởng có 5 cách; sau đó chọn phó còn 4 cách, nên có `5 × 4 = 20` phương án. Đảo vai trò hai người tạo phương án khác. `math.perm(n,k)` đếm cách lấy **k người khác nhau từ n người** và phân thứ tự, với `0 ≤ k ≤ n`. Nếu dùng hết n người, kết quả là `n!`.

```
import math

print(math.perm(5, 2))
```

Kết quả khi chạy:

```
20
```

Có 20 cách phân vai trưởng/phó; đổi thứ tự hai người được tính là cách khác.

4.2. Combination — Chọn nhóm không phân vai

Từ vựng trong mục: Combination — tổ hợp; **Order-independent** — không xét thứ tự.

Nếu năm ứng viên chỉ cần chọn **hai người cùng một đội**, cặp An–Bình và Bình–An là **một** đội, không phải hai. Công thức `C(n,k) = n! / (k!(n-k)!)`; với 5 chọn 2 có 10 nhóm. `math.comb(n,k)` tính trực tiếp số nhóm mà không cần liệt kê từng cặp. Trong thực tế cần xác nhận có được chọn một người nhiều lần hay không; ở đây mỗi ứng viên chỉ chọn tối đa một lần.

```
import math

print(math.comb(5, 2))
```

Kết quả khi chạy:

```
10
```

Số nhóm bằng nửa số cách phân vai hai người vì hoán đổi hai vai không tạo đội mới.

4.3. Factorial — Sắp xếp tất cả phần tử

Từ vựng trong mục: Factorial — giai thừa; **Arrangement** — thứ tự sắp xếp.

Với bốn nhiệm vụ **khác nhau**, có 4 cách chọn việc đầu, 3 cách chọn việc thứ hai, rồi 2 và 1. Tổng số cách sắp xếp là `4! = 4×3×2×1 = 24`. Đây là **giai thừa**. Quy ước `0! = 1` vì có đúng một cách sắp xếp dãy rỗng. `math.factorial` áp dụng cho số nguyên không âm.

```
import math

print(math.factorial(4))
```

Kết quả khi chạy:

```
24
```

Bốn nhiệm vụ khác nhau có 24 thứ tự thực hiện nếu mỗi nhiệm vụ xuất hiện đúng một lần.

4.4. Binomial coefficient và tam giác Pascal

Từ vựng trong mục: Binomial coefficient — hệ số nhị thức; **Pascal triangle** — tam giác Pascal.

Hệ số nhị thức `C(n,k)` là số cách chọn k phần tử không xét thứ tự, đồng thời là hệ số của `a^(n-k)b^k` trong khai triển `(a+b)^n`. Các giá trị `C(n,0)` đến `C(n,n)` tạo thành **hàng n của tam giác Pascal** (đếm từ hàng 0). Với n = 4, hàng có năm hệ số `1,4,6,4,1`; hai cạnh bằng 1 và mỗi ô trong bằng tổng hai ô ngay phía trên.

```
import math

print([math.comb(4, k) for k in range(5)])
```

Kết quả khi chạy:

```
[1, 4, 6, 4, 1]
```

Hàng thứ tư tính từ 0 có năm hệ số, ứng với k từ 0 đến 4.

5. Năm vấn đề thực tế

Tình huống 1 — Chọn trưởng và phó nhóm

Từ vựng trong mục: Permutation — hoán vị.

Tình huống thực tế: Ban tổ chức có 5 ứng viên và cần chọn một trưởng nhóm, một phó nhóm, hai vị trí khác nhau. Có bao nhiêu cách chọn?

Vì sao chọn cách giải này? Hai vai trò khác nhau nên phải tính số cách có thứ tự, dùng `perm`.

Quy trình giải:

1. Xác định trưởng nhóm và phó nhóm là hai vị trí khác nhau.
2. Chọn trưởng trong năm người rồi phó trong bốn người còn lại.
3. Nhân 5×4 hoặc gọi `math.perm(5,2)` để đếm.

```
import math
print(math.perm(5, 2))
```

Kết quả khi chạy:

```
20
```

Tình huống 2 — Chọn đội thi

Từ vựng trong mục: Combination — tổ hợp.

Tình huống thực tế: Một đội thi cần chọn 5 người từ 12 ứng viên; vai trò của người được chọn không phân biệt. Có bao nhiêu đội?

Vì sao chọn cách giải này? Không phân vai nên mỗi nhóm năm người chỉ được đếm một lần, dùng `comb`.

Quy trình giải:

1. Ghi nhận đội có năm thành viên và có 12 ứng viên.
2. Xác định thứ tự thành viên không quan trọng.
3. Gọi `math.comb(12,5)` để đếm các đội khác nhau.

```
import math
print(math.comb(12, 5))
```

Kết quả khi chạy:

```
792
```

Tình huống 3 — Sắp thứ tự kiểm thử

Từ vựng trong mục: Factorial — giai thừa.

Tình huống thực tế: Bộ kiểm thử có ba tác vụ A, B, C; cần thử mọi thứ tự thực hiện. Liệt kê các thứ tự.

Vì sao chọn cách giải này? Đề yêu cầu toàn bộ thứ tự của ba tác vụ, nên cần hoán vị độ dài ba và nhận `3!` kết quả.

Quy trình giải:

1. Tạo ba tác vụ A, B, C, mỗi tác vụ phải xuất hiện một lần.
2. Dùng `itertools.permutations(tasks)` không truyền độ dài để lấy đủ ba tác vụ.
3. In sáu thứ tự; đối chiếu `3! = 6`.

```
from itertools import permutations
print(list(permutations(['A', 'B', 'C'])))
```

Kết quả khi chạy:

```
[('A', 'B', 'C'), ('A', 'C', 'B'), ('B', 'A', 'C'), ('B', 'C', 'A'), ('C', 'A', 'B'), ('C', 'B',
'A')]
```

Tình huống 4 — Đề xuất cặp môn học

Từ vựng trong mục: Binomial coefficient — hệ số nhị thức.

Tình huống thực tế: Nhóm học chọn hai môn trong ba môn Python, AI, SQL; thứ tự không quan trọng. Liệt kê các cặp môn.

Vì sao chọn cách giải này? Cặp môn không có thứ tự, vì vậy `combinations` không nhân đôi những cặp đảo chiều.

Quy trình giải:

1. Lưu ba môn Python, AI, SQL vào List.
2. Dùng `combinations(subjects,2)` vì không phân biệt hai thứ tự trong cùng cặp.
3. In ba cặp môn có thể chọn.

```
from itertools import combinations
print(list(combinations(["Python", "AI", "SQL"], 2)))
```

Kết quả khi chạy:

```
[('Python', 'AI'), ('Python', 'SQL'), ('AI', 'SQL')]
```

Tình huống 5 — Tính hàng Pascal

Từ vựng trong mục: Permutation — hoán vị.

Tình huống thực tế: Giáo viên cần viết hàng n=4 của tam giác Pascal bằng C(4,k), k từ 0 đến 4.

Vì sao chọn cách giải này? Các giá trị của tam giác Pascal chính là các hệ số nhị thức `C(n,k)`.

Quy trình giải:

1. Chọn hàng n=4 của tam giác Pascal.
2. Tính các hệ số `math.comb(4,k)` với k=0..4.
3. In năm giá trị và kiểm tra tính đối xứng.

```
import math
print([math.comb(4, k) for k in range(5)])
```

Kết quả khi chạy:

```
[1, 4, 6, 4, 1]
```

6. Sai lầm thường gặp

- Phân biệt bài toán **có thứ tự** (`perm`) với **không xét thứ tự** (`comb`).
- `itertools.permutations(items,2)` chỉ tạo các cặp có thứ tự, không phải sắp xếp toàn bộ ba phần tử; không truyền k để lấy toàn bộ.
- Cả `math.perm` và `math.comb` giả định các phần tử phân biệt và không chọn trùng một phần tử.
- `math.factorial` cần số nguyên không âm; `0! = 1`.
- `itertools` để **liệt kê** các phương án; `math` giúp **đếm** khi số phương án quá lớn không nên liệt kê.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. When should permutations be used? — Khi nào dùng hoán vị/chỉnh hợp?

A. When order matters — Khi thứ tự quan trọng

B. When order does not matter — Khi thứ tự không quan trọng

C. The product 1×2×…×n — Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items — Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: A

Giải thích: Đổi thứ tự người được chọn tạo cách sắp khác.

Q2. When should combinations be used? — Khi nào dùng tổ hợp?

A. When order matters — Khi thứ tự quan trọng

B. When order does not matter — Khi thứ tự không quan trọng

C. The product 1×2×…×n — Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items — Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: B

Giải thích: Nhóm không đổi khi thành viên đổi chỗ.

Q3. What is factorial n!? — Giai thừa n! là gì?

A. When order matters — Khi thứ tự quan trọng

B. When order does not matter — Khi thứ tự không quan trọng

C. The product 1×2×…×n — Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items — Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: C

Giải thích: Giai thừa đếm cách sắp xếp n phần tử phân biệt.

Q4. What is a binomial coefficient C(n,k)? — Hệ số nhị thức C(n,k) là gì?

A. When order matters — Khi thứ tự quan trọng

B. When order does not matter — Khi thứ tự không quan trọng

C. The product 1×2×…×n — Tích 1×2×…×n

D. The number of unordered k-subsets from n distinct items — Số nhóm k phần tử không xét thứ tự từ n phần tử khác nhau

Đáp án: D

Giải thích: C(n,k) còn là hệ số trong khai triển (a+b)^n.

Q5. Which function counts ordered selections? — Hàm nào đếm cách chọn có thứ tự?

A. `math.perm(n, k)` — `math.perm(n, k)`

B. `math.comb(n, k)` — `math.comb(n, k)`

C. `math.factorial(n)` — `math.factorial(n)`

D. `itertools.permutations(items)` — `itertools.permutations(items)`

Đáp án: A

Giải thích: perm đếm cách chọn có thứ tự không lặp.

Q6. Which function counts unordered selections? — Hàm nào đếm cách chọn không thứ tự?

A. `math.perm(n, k)` — `math.perm(n, k)`

B. `math.comb(n, k)` — `math.comb(n, k)`

C. `math.factorial(n)` — `math.factorial(n)`

D. `itertools.permutations(items)` — `itertools.permutations(items)`

Đáp án: B

Giải thích: comb đếm các nhóm khác nhau, không xét sắp xếp.

Q7. Which function calculates n!? — Hàm nào tính n!?

A. `math.perm(n, k)` — `math.perm(n, k)`

B. `math.comb(n, k)` — `math.comb(n, k)`

C. `math.factorial(n)` — `math.factorial(n)`

D. `itertools.permutations(items)` — `itertools.permutations(items)`

Đáp án: C

Giải thích: factorial nhân các số nguyên từ một tới n.

Q8. Which function lists all ordered arrangements? — Hàm nào liệt kê các cách sắp xếp có thứ tự?

A. `math.perm(n, k)` — `math.perm(n, k)`

B. `math.comb(n, k)` — `math.comb(n, k)`

C. `math.factorial(n)` — `math.factorial(n)`

D. `itertools.permutations(items)` — `itertools.permutations(items)`

Đáp án: D

Giải thích: Không truyền r, permutations liệt kê toàn bộ thứ tự.

Q9. What is the value of 0!? — 0! bằng bao nhiêu?

A. 1 — 1

B. 6 — 6

C. 10 pairs — 10 cặp

D. 12 pairs — 12 cặp

Đáp án: A

Giải thích: Dãy rỗng có một cách sắp xếp.

Q10. How many full orderings of three distinct items exist? — Ba phần tử phân biệt có bao nhiêu thứ tự đầy đủ?

A. 1 — 1

B. 6 — 6

C. 10 pairs — 10 cặp

D. 12 pairs — 12 cặp

Đáp án: B

Giải thích: 3! = 3×2×1 = 6.

Q11. How many unordered pairs can be chosen from five distinct items? — Chọn cặp không thứ tự từ năm phần tử có bao nhiêu cách?

A. 1 — 1

B. 6 — 6

C. 10 pairs — 10 cặp

D. 12 pairs — 12 cặp

Đáp án: C

Giải thích: Chọn hai từ năm không xét thứ tự có C(5,2)=10 cặp.

Q12. How many ordered pairs can be formed from four distinct items without repetition? — Từ bốn phần tử phân biệt có bao nhiêu cặp có thứ tự không lặp?

A. 1 — 1

B. 6 — 6

C. 10 pairs — 10 cặp

D. 12 pairs — 12 cặp

Đáp án: D

Giải thích: 4×3 = 12.

Q13. What identity relates combinations and permutations? — Công thức nào liên hệ tổ hợp và chỉnh hợp?

A. P(n,k)=C(n,k)×k! — P(n,k)=C(n,k)×k!

B. 1 — 1

C. n — n

D. Each inner entry sums two entries directly above — Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: A

Giải thích: Mỗi nhóm k người có k! cách sắp.

Q14. What is C(n,0)? — C(n,0) bằng gì?

A. P(n,k)=C(n,k)×k! — P(n,k)=C(n,k)×k!

B. 1 — 1

C. n — n

D. Each inner entry sums two entries directly above — Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: B

Giải thích: Có một cách chọn tập rỗng.

Q15. What is C(n,n−1)? — C(n,n−1) bằng gì?

A. P(n,k)=C(n,k)×k! — P(n,k)=C(n,k)×k!

B. 1 — 1

C. n — n

D. Each inner entry sums two entries directly above — Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: C

Giải thích: Chọn n−1 từ n tương đương chọn một phần tử bỏ lại nên có n cách.

Q16. How are entries in Pascal's triangle obtained? — Các ô trong tam giác Pascal được tạo ra sao?

A. P(n,k)=C(n,k)×k! — P(n,k)=C(n,k)×k!

B. 1 — 1

C. n — n

D. Each inner entry sums two entries directly above — Mỗi ô bên trong bằng tổng hai ô ngay phía trên

Đáp án: D

Giải thích: Hệ thức Pascal cộng hai hệ số ở hàng trước.

Q17. What does `itertools.combinations(items,2)` list? — `itertools.combinations(items,2)` liệt kê gì?

A. Unordered two-item groups — Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements — Cặp hai phần tử có thứ tự

C. It returns zero — Hàm trả về 0

D. The number grows factorially — Số cách tăng theo giai thừa

Đáp án: A

Giải thích: Hàm tạo mỗi cặp khác nhau đúng một lần.

Q18. What does `itertools.permutations(items,2)` list? — `itertools.permutations(items,2)` liệt kê gì?

A. Unordered two-item groups — Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements — Cặp hai phần tử có thứ tự

C. It returns zero — Hàm trả về 0

D. The number grows factorially — Số cách tăng theo giai thừa

Đáp án: B

Giải thích: Đổi thứ tự hai phần tử tạo kết quả khác.

Q19. What happens if k>n in `math.comb(n,k)` for nonnegative integers? — Nếu k>n trong math.comb với số nguyên không âm thì sao?

A. Unordered two-item groups — Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements — Cặp hai phần tử có thứ tự

C. It returns zero — Hàm trả về 0

D. The number grows factorially — Số cách tăng theo giai thừa

Đáp án: C

Giải thích: Không thể chọn k phần tử nếu tổng phần tử ít hơn k.

Q20. Why avoid materializing all permutations for large n? — Vì sao tránh liệt kê toàn bộ hoán vị khi n lớn?

A. Unordered two-item groups — Nhóm hai phần tử không xét thứ tự

B. Ordered two-item arrangements — Cặp hai phần tử có thứ tự

C. It returns zero — Hàm trả về 0

D. The number grows factorially — Số cách tăng theo giai thừa

Đáp án: D

Giải thích: n! tăng nhanh khiến không thể lưu hoặc in hết.

8. Nối sang bài tiếp

`Nguyên lý đếm → hoán vị/tổ hợp → phân phối nhị thức.`

PY-23 — Đồ thị, danh sách kề và Jaccard similarity

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Graph — đồ thị **Vertex** — đỉnh.

`Set → Graph → Similarity → Gợi ý bạn học`

Học xong dùng để làm gì? Dùng cạnh và nút mô tả liên kết; so nhãn/sở thích bằng hệ số tương đồng Jaccard.

Kết nối kiến thức tiếp theo: PY-24 chuyển đặc trưng đối tượng thành vector có thể đo độ dài và hướng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Graph	đồ thị	Các đỉnh và cạnh mô tả quan hệ giữa đối tượng
Vertex	đỉnh	Một đối tượng được biểu diễn trong đồ thị
Edge	cạnh	Liên hệ hoặc đường nối giữa hai đỉnh
Jaccard	độ tương đồng	Tỷ lệ số phần tử giao trên hợp giữa hai tập

3. Hàm và cấu trúc dữ liệu liên quan

Từ vựng trong mục: Function — hàm; **Adjacency list** — danh sách kề.

Hàm / tham số	Dùng làm gì
`dict[str,set[str]]`	Danh sách kề đơn giản: key là đỉnh, value là các đỉnh nối
`networkx.Graph()/DiGraph()`	Đồ thị vô hướng/có hướng, cần cài networkx
`nx.shortest_path(G,a,b)`	Tìm đường nối ngắn nhất nếu có kết nối
`len(a & b) / len(a | b)`	Tương đồng Jaccard khi hợp khác rỗng

4. Khái niệm liên quan

Đồ thị (graph) mô tả mạng quan hệ: **đỉnh (vertex)** là đối tượng, **cạnh (edge)** là mối nối giữa đối tượng. Trong mạng học viên, mỗi người là một đỉnh và một cạnh biểu thị quan hệ bạn bè. Ta biểu diễn mạng bằng **danh sách kề** rồi liên hệ với **Jaccard**, thước đo phần sở thích chung so với toàn bộ sở thích của hai người.

4.1. Graph và adjacency list — Mạng và danh sách kề

Từ vựng trong mục: Graph — đồ thị; **Adjacency list** — danh sách kề.

Một **đồ thị** gồm tập **đỉnh** và tập **cạnh** nối chúng. Để biểu diễn mạng bạn bè An–Bình–Lan mà không cần vẽ, dùng Dictionary: **khóa** là tên người, **Set giá trị** là những người kết nối trực tiếp với họ. Nếu quan hệ hai chiều, cả hai đầu phải ghi tên nhau. Ví dụ An kết bạn với Bình và Lan, còn Bình/Lan đều kết nối ngược lại An.

```
graph = {
    'An': {'Binh', 'Lan'},
    'Binh': {'An'},
    'Lan': {'An'},
}
print(sorted(graph['An']))
```

Kết quả khi chạy:

```
['Binh', 'Lan']
```

An có hai kết nối trực tiếp là Bình và Lan. Dictionary giúp tra nhanh những người kề với một đỉnh.

4.2. Vertex — Đỉnh đại diện một đối tượng

Từ vựng trong mục: Vertex — đỉnh; **Node** — nút.

Đỉnh là một đối tượng trong mạng: một học viên, trạm xe hoặc trang web. Tên đỉnh chỉ là định danh, không nói lên nó đã kết nối với ai. Trong ví dụ có hai học viên An và Bình, ta dùng Set chứa hai tên làm tập đỉnh; `len(vertices)` đếm đối tượng khác nhau, không phải số liên kết.

```
vertices = {'An', 'Binh'}
print(len(vertices))
```

Kết quả khi chạy:

```
2
```

Mạng có hai đỉnh khác nhau; chưa suy ra có cạnh nào giữa họ.

4.3. Edge — Cạnh có hướng và vô hướng

Từ vựng trong mục: Edge — cạnh; **Directed** — có hướng.

Cạnh biểu diễn mối quan hệ giữa hai đỉnh. Với bạn bè hai chiều, cạnh An–Bình là **vô hướng** và chỉ cần tính một lần; với hành động A gửi thông báo tới B, cạnh A→B **có hướng**, không tự suy ra B→A. Danh sách kề có hướng `{'A':{'B'},'B':set()}` diễn tả A gửi B, nhưng B không gửi lại A.

```
graph = {'A': {'B'}, 'B': set()}
print('B' in graph['A'], 'A' in graph['B'])
```

Kết quả khi chạy:

```
True False
```

Quan hệ A→B tồn tại; chiều ngược lại không tồn tại. Cách lưu cạnh phải phù hợp loại đồ thị.

4.4. Jaccard similarity — Độ giống nhau của hai tập

Từ vựng trong mục: Jaccard — tương đồng Jaccard; **Union/intersection** — hợp/giao.

Jaccard đo mức trùng nhau của hai tập: `J(A,B) = |A ∩ B| / |A ∪ B|`, trong khoảng **0–1** khi hợp không rỗng. Người A biết `{Python, AI}`, người B biết `{Python, SQL}`: giao có 1 kỹ năng `Python`, hợp có 3 kỹ năng, nên độ tương đồng là `1/3 ≈ 0.333`. Không dùng số lượng phần tử trùng trực tiếp làm tỷ lệ. Với cả hai tập rỗng phải định nghĩa riêng quy ước để tránh chia 0.

```
a = {'Python', 'AI'}
b = {'Python', 'SQL'}
print(round(len(a & b) / len(a | b), 3))
```

Kết quả khi chạy:

```
0.333
```

Hai hồ sơ giống nhau khoảng 33,3% theo Jaccard vì chỉ chung một trên ba kỹ năng khác nhau.

5. Năm tình huống ứng dụng

Tình huống 1 — Bạn bè trực tiếp

Từ vựng trong mục: Graph — đồ thị.

Tình huống thực tế: Mạng bạn bè cho biết An kết nối với Bình và Lan, còn Bình kết nối lại An. Liệt kê các bạn trực tiếp của An.

Vì sao chọn cách giải này? Danh sách kề cho biết những người nối trực tiếp với An; không cần phép nhân ma trận.

Quy trình giải:

1. Tạo Dictionary danh sách kề, trong đó mỗi người là một khóa và tập bạn bè là giá trị.
2. Lấy tập bạn kề của An bằng `graph['An']`.
3. Sắp xếp tên để đầu ra không phụ thuộc thứ tự Set.

```
graph = {"An":{"Binh", "Lan"}, "Binh":{"An"}, "Lan":{"An"}}
print(sorted(graph['An']))
```

Kết quả khi chạy:

```
['Binh', 'Lan']
```

Tình huống 2 — So trùng sở thích

Từ vựng trong mục: Jaccard similarity — độ tương đồng tập hợp.

Tình huống thực tế: Hai hồ sơ sở thích A={Python,AI}, B={Python,SQL}. Tính mức tương đồng Jaccard bằng số mục chung chia số mục khác nhau.

Vì sao chọn cách giải này? Jaccard tính tỷ lệ giao/hợp của hai Set và bảo đảm kết quả trong khoảng 0–1.

Quy trình giải:

1. Tạo hai Set kỹ năng `a`, `b` từ hai hồ sơ.
2. Đếm phần giao và phần hợp; mẫu số phải khác 0.
3. Chia số kỹ năng chung cho số kỹ năng khác nhau và in độ tương đồng.

```
a = {"Python", "AI"}
b = {"Python", "SQL"}
print(len(a&b)/len(a|b))
```

Kết quả khi chạy:

```
0.3333333333333333
```

Tình huống 3 — Gợi ý đối tác học

Từ vựng trong mục: Jaccard similarity — độ tương đồng tập hợp.

Tình huống thực tế: Một học viên thích Python và AI; bạn B thích AI, bạn C thích cả Python lẫn AI. Chọn người có điểm tương đồng cao nhất.

Vì sao chọn cách giải này? Tính Jaccard với từng ứng viên rồi chọn điểm lớn nhất sẽ cho người có sở thích trùng nhiều nhất.

Quy trình giải:

1. Lưu tập sở thích mục tiêu và Dictionary các ứng viên B, C.
2. Tính Jaccard giữa mục tiêu với từng ứng viên.
3. Dùng `max(scores, key=scores.get)` chọn ứng viên có tỷ lệ cao nhất.

```
target = {"AI", "Python"}
people = {"B":{"AI"}, "C":{"AI", "Python"}}
scores = {n:len(target&s)/len(target|s) for n, s in people.items()}
print(max(scores, key=scores.get))
```

Kết quả khi chạy:

```
C
```

Tình huống 4 — Quan hệ có hướng

Từ vựng trong mục: Directed edge — cạnh có hướng.

Tình huống thực tế: Mạng thông báo có hướng A→B và B→C. Kiểm tra đường A→B có đồng nghĩa với B→A không.

Vì sao chọn cách giải này? Đồ thị có hướng phân biệt A→B và B→A; kiểm tra hai chiều phải thực hiện riêng.

Quy trình giải:

1. Lưu đồ thị có hướng với A dẫn đến B và B dẫn đến C.
2. Kiểm tra B có thuộc danh sách kề của A không.
3. Kiểm tra riêng A có thuộc danh sách kề của B không và in hai kết quả.

```
graph = {"A":{"B"}, "B":{"C"}, "C":set()}
print("B" in graph["A"], "A" in graph["B"])
```

Kết quả khi chạy:

```
True False
```

Tình huống 5 — Tìm đường trong mạng

Từ vựng trong mục: Graph — đồ thị.

Tình huống thực tế: Ba trạm A, B, C nối bằng hai đoạn A-B và B-C. Tìm đường đi ngắn nhất giữa A và C.

Vì sao chọn cách giải này? `networkx.shortest_path` tìm chuỗi đỉnh ngắn nhất theo số cạnh khi các cạnh không có trọng số.

Quy trình giải:

1. Tạo đồ thị không hướng và thêm cạnh A–B, B–C bằng NetworkX (cần cài thư viện trước).
2. Gọi `nx.shortest_path(G, 'A', 'C')`.
3. In danh sách trạm theo thứ tự đường đi.

```
import networkx as nx
G = nx.Graph()
G.add_edges_from([("A", "B"), ("B", "C")])
print(nx.shortest_path(G, "A", "C"))
```

Kết quả khi chạy:

```
['A', 'B', 'C']
```

6. Sai lầm thường gặp

- **Đỉnh** là đối tượng, **cạnh** là quan hệ; không nhầm số đỉnh với số cạnh.
- Đồ thị có hướng không mặc nhiên có cạnh ngược; đồ thị vô hướng cần diễn giải kết nối hai chiều.
- Set không bảo đảm thứ tự in; dùng `sorted()` khi liệt kê tên trong tài liệu.
- Jaccard bằng `|AnB| / |AuB|`; nếu cả hai tập rỗng thì phải xử lý trước khi chia.
- `networkx` là thư viện ngoài cần cài đặt; `shortest_path` có thể báo lỗi nếu hai đỉnh không nối với nhau.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a graph? — Đồ thị là gì?

A. A collection of vertices and edges — Tập các đỉnh và cạnh

B. An entity or node in the network — Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices — Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors — Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: A

Giải thích: Đồ thị mô tả quan hệ giữa các đối tượng.

Q2. What is a vertex? — Đỉnh trong đồ thị là gì?

A. A collection of vertices and edges — Tập các đỉnh và cạnh

B. An entity or node in the network — Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices — Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors — Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: B

Giải thích: Mỗi đỉnh đại diện một đối tượng.

Q3. What is an edge? — Cạnh của đồ thị là gì?

A. A collection of vertices and edges — Tập các đỉnh và cạnh

B. An entity or node in the network — Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices — Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors — Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: C

Giải thích: Cạnh biểu diễn kết nối giữa các đỉnh.

Q4. What is an adjacency list? — Danh sách kề là gì?

A. A collection of vertices and edges — Tập các đỉnh và cạnh

B. An entity or node in the network — Một đối tượng hoặc nút trong mạng

C. A relationship connecting two vertices — Quan hệ nối hai đỉnh

D. A mapping from each vertex to its neighbors — Ánh xạ mỗi đỉnh tới các đỉnh kề

Đáp án: D

Giải thích: Danh sách kề lưu các kết nối của từng đỉnh.

Q5. What does a directed edge A→B assert? — Cạnh có hướng A→B cho biết gì?

A. A connection from A to B — Kết nối đi từ A đến B

B. A connection without direction — Kết nối không phân biệt chiều

C. The neighbors stored for An — Những đỉnh kề được lưu cho An

D. A sequence of connected vertices — Chuỗi các đỉnh nối tiếp nhau

Đáp án: A

Giải thích: Chiều kết nối quan trọng; B→A không tự tồn tại.

Q6. What does an undirected edge mean? — Cạnh vô hướng có ý nghĩa gì?

A. A connection from A to B — Kết nối đi từ A đến B

B. A connection without direction — Kết nối không phân biệt chiều

C. The neighbors stored for An — Những đỉnh kề được lưu cho An

D. A sequence of connected vertices — Chuỗi các đỉnh nối tiếp nhau

Đáp án: B

Giải thích: Quan hệ vô hướng đi được theo cả hai chiều.

Q7. What does `graph['An']` return in an adjacency dictionary? — `graph['An']` trong Dictionary kề trả gì?

A. A connection from A to B — Kết nối đi từ A đến B

B. A connection without direction — Kết nối không phân biệt chiều

C. The neighbors stored for An — Những đỉnh kề được lưu cho An

D. A sequence of connected vertices — Chuỗi các đỉnh nối tiếp nhau

Đáp án: C

Giải thích: Giá trị của khóa đỉnh chứa các nút lân cận.

Q8. What does a path in a graph describe? — Đường đi trong đồ thị biểu diễn gì?

A. A connection from A to B — Kết nối đi từ A đến B

B. A connection without direction — Kết nối không phân biệt chiều

C. The neighbors stored for An — Những đỉnh kề được lưu cho An

D. A sequence of connected vertices — Chuỗi các đỉnh nối tiếp nhau

Đáp án: D

Giải thích: Hai đỉnh liên tiếp trên đường đi phải có cạnh nối.

Q9. What is Jaccard similarity for sets A and B? — Jaccard giữa hai Set A, B bằng gì?

A. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

B. 1 — 1

C. 0 — 0

D. A zero union denominator — Mẫu số hợp bằng 0

Đáp án: A

Giải thích: Jaccard so phần chung với toàn bộ phần tử khác nhau.

Q10. What is Jaccard for two identical nonempty sets? — Jaccard của hai Set giống nhau và không rỗng là gì?

A. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

B. 1 — 1

C. 0 — 0

D. A zero union denominator — Mẫu số hợp bằng 0

Đáp án: B

Giải thích: Phép giao bằng phép hợp.

Q11. What is Jaccard for two disjoint nonempty sets? — Jaccard của hai Set không giao nhau, không rỗng là gì?

A. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

B. 1 — 1

C. 0 — 0

D. A zero union denominator — Mẫu số hợp bằng 0

Đáp án: C

Giải thích: Phép giao không có phần tử nào.

Q12. What must be handled before computing Jaccard on two empty sets? — Cần xử lý gì trước khi tính Jaccard cho hai Set rỗng?

A. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

B. 1 — 1

C. 0 — 0

D. A zero union denominator — Mẫu số hợp bằng 0

Đáp án: D

Giải thích: Nếu không xử lý sẽ chia cho 0.

Q13. What does `nx.Graph()` create? — `nx.Graph()` tạo gì?

A. An undirected NetworkX graph — Đồ thị vô hướng NetworkX

B. A directed NetworkX graph — Đồ thị có hướng NetworkX

C. A path with fewest edges — Đường đi có ít cạnh nhất

D. Install and import the package — Cài và import thư viện

Đáp án: A

Giải thích: Graph tạo cạnh vô hướng theo mặc định.

Q14. What does `nx.DiGraph()` create? — `nx.DiGraph()` tạo gì?

A. An undirected NetworkX graph — Đồ thị vô hướng NetworkX

B. A directed NetworkX graph — Đồ thị có hướng NetworkX

C. A path with fewest edges — Đường đi có ít cạnh nhất

D. Install and import the package — Cài và import thư viện

Đáp án: B

Giải thích: DiGraph phân biệt chiều cạnh.

Q15. What does `nx.shortest_path()` find on an unweighted graph? — `nx.shortest_path()` tìm gì trên đồ thị không trọng số?

A. An undirected NetworkX graph — Đồ thị vô hướng NetworkX

B. A directed NetworkX graph — Đồ thị có hướng NetworkX

C. A path with fewest edges — Đường đi có ít cạnh nhất

D. Install and import the package — Cài và import thư viện

Đáp án: C

Giải thích: Đường ngắn nhất không trọng số có ít cạnh nhất.

Q16. What is required to use NetworkX in Python? — Để dùng NetworkX trong Python cần gì?

A. An undirected NetworkX graph — Đồ thị vô hướng NetworkX

B. A directed NetworkX graph — Đồ thị có hướng NetworkX

C. A path with fewest edges — Đường đi có ít cạnh nhất

D. Install and import the package — Cài và import thư viện

Đáp án: D

Giải thích: NetworkX không thuộc thư viện chuẩn Python.

Q17. How can neighbor display order be made consistent? — Làm sao in danh sách các đỉnh kề theo thứ tự ổn định?

A. Call `sorted()` on neighbor sets — Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned — Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously — Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges — Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: A

Giải thích: Set không bảo đảm thứ tự khi in.

Q18. What happens if no path exists between two vertices? — Nếu không có đường nối hai đỉnh thì sao?

A. Call `sorted()` on neighbor sets — Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned — Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously — Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges — Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: B

Giải thích: Thuật toán tìm đường cần xử lý hai đỉnh không liên thông.

Q19. Why should node names be unique? — Vì sao tên đỉnh nên duy nhất?

A. Call `sorted()` on neighbor sets — Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned — Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously — Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges — Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: C

Giải thích: Mã đỉnh trùng khiến các đối tượng bị nhập làm một.

Q20. What distinguishes Jaccard from graph shortest path? — Jaccard khác đường đi ngắn nhất ở đâu?

A. Call `sorted()` on neighbor sets — Gọi `sorted()` trên Set đỉnh kề

B. No valid route can be returned — Không có tuyến đường hợp lệ để trả về

C. To identify each graph entity unambiguously — Để xác định từng đối tượng không mơ hồ

D. Jaccard compares sets; shortest path follows graph edges — Jaccard so hai tập; đường ngắn nhất đi theo cạnh

Đáp án: D

Giải thích: Hai công cụ trả lời hai loại câu hỏi khác nhau.

8. Nối sang bài kế tiếp

`Set → Graph → Similarity → Gợi ý bạn học.`

PY-24 — Scalar, vector, norm, đơn vị và dot product

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Scalar — đại lượng vô hướng; **Vector** — véc-tơ.

`NumPy → Scalar/Vector → Matrix → PCA`

Học xong dùng để làm gì? Biểu diễn học viên hay sản phẩm dưới dạng vector đặc trưng số và thực hiện phép cộng/trừ vector.

Kết nối kiến thức tiếp theo: PY-25 xếp nhiều vector thành ma trận để tính cùng lúc nhiều quan sát.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Scalar	đại lượng vô hướng	Một số đơn lẻ có thể đo độ lớn
Vector	véc-tơ	Dãy số mô tả nhiều thành phần của một đối tượng
Norm	độ lớn véc-tơ	Độ dài theo chuẩn đã chọn
Dot product	tích vô hướng	Nhân từng cặp tọa độ tương ứng rồi cộng

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.array([...],dtype=float)`	Tạo véc-tơ kiểu thực khi phép tính tạo phần thập phân
`np.linalg.norm(v)`	Độ lớn Euclidean của véc-tơ
`v/np.linalg.norm(v)`	Chuẩn hóa về độ dài 1, phải tránh norm bằng 0
`a @ b / np.dot(a,b)`	Tích vô hướng nếu hai vector một chiều cùng độ dài

4. Khái niệm liên quan

Bài học nối bốn ý: **scalar** là số đơn lẻ, **vector** chứa tọa độ, **norm** đo độ dài, **dot product** kết hợp các thành phần. Robot dịch chuyển `(3,4)` mét có vector hai chiều nhưng quãng đường thẳng chỉ là một số bằng 5 mét.

4.1. Scalar — Số đơn lẻ và đơn vị

Scalar là một giá trị số như vận tốc 2,5 mét/giây. Python lưu số, còn ý nghĩa và đơn vị phải do người lập trình nêu. Scalar khác vector vì không chứa nhiều thành phần tọa độ.

```
speed = 2.5
print(speed)
```

Kết quả khi chạy:

```
2.5
```

Số 2.5 đại diện vận tốc m/s, không phải tọa độ.

4.2. Vector — Các tọa độ có thứ tự

Vector là dãy tọa độ theo một hệ trục. Robot đi 3 m theo x và 4 m theo y nên dùng vector `[3,4]`. Phần tử đầu là x, phần tử sau là y. Đổi `[3,4]` thành `[4,3]` sẽ đổi ý nghĩa hai trục.

```
import numpy as np

movement = np.array([3, 4])
print(movement)
```

Kết quả khi chạy:

```
[3 4]
```

Hai phần tử lần lượt là độ dịch chuyển theo x và y (mét).

4.3. Norm — Độ dài vector

Chuẩn Euclid là độ dài `sqrt(x²+y²)`. Với vector `(3,4)`, độ dài bằng `sqrt(9+16)=5` m. Vector đơn vị giữ hướng và có norm bằng 1; muốn chuẩn hóa phải chia các thành phần cho norm, trừ trường hợp norm bằng 0.

```
import numpy as np

movement = np.array([3.0, 4.0])
print(np.linalg.norm(movement))
```

Kết quả khi chạy:

```
5.0
```

Robot cách gốc tọa độ 5 mét theo đường thẳng.

4.4. Dot product — Nhân và cộng cùng vị trí

Tích vô hướng của hai vector cùng số chiều bằng tổng các tích từng thành phần: `(1,2)·(3,4)=1×3+2×4=11`. Dùng để tính điểm có trọng số hoặc kiểm tra hai vector khác không vuông góc (khi tích vô hướng bằng 0).

```
import numpy as np

a = np.array([1, 2])
b = np.array([3, 4])
print(a @ b)
```

Kết quả khi chạy:

```
11
```

Tích vô hướng trả một scalar bằng 11.

5. Năm tình huống ứng dụng

Tình huống 1 — Tính khoảng cách 2D

Từ vựng trong mục: Scalar — đại lượng vô hướng.

Tình huống thực tế: Robot di chuyển từ (0,0) tới (3,4) mét. Tính khoảng cách theo đường thẳng.

Vì sao chọn cách giải này? Norm của vector dịch chuyển là khoảng cách tới gốc.

Quy trình giải:

1. Tạo vector từ (0,0) đến (3,4) mét.
2. Tính norm Euclid.
3. In khoảng cách 5 mét.

```
import numpy as np
v = np.array([3, 4], dtype=float)
print(np.linalg.norm(v))
```

Kết quả khi chạy:

```
5.0
```

Tình huống 2 — Chuẩn hóa hướng di chuyển

Từ vựng trong mục: Vector — véc-tơ.

Tình huống thực tế: Robot nhận vector hướng (3,4). Chuẩn hóa thành vector đơn vị để chỉ biểu thị hướng.

Vì sao chọn cách giải này? Chia vector cho norm tạo hướng có độ dài bằng 1.

Quy trình giải:

1. Tạo vector hướng (3,4) dạng float.
2. Tính norm và chia nếu norm khác 0.
3. In vector đơn vị [0.6,0.8].

```
import numpy as np
v = np.array([3., 4.])
norm = np.linalg.norm(v)
u = v/norm if norm else v
print(u)
```

Kết quả khi chạy:

```
[0.6 0.8]
```

Tình huống 3 — Điểm tổng kết có trọng số

Từ vựng trong mục: Dot product — tích vô hướng.

Tình huống thực tế: Điểm ba môn của học viên là [8,7,9] theo trọng số [20%,30%,50%]. Tính điểm tổng kết.

Vì sao chọn cách giải này? Tích vô hướng kết hợp điểm số và trọng số từng môn.

Quy trình giải:

1. Lưu ba điểm và ba trọng số theo cùng thứ tự môn.
2. Nhân bằng @ để tính tổng có trọng số.
3. In 8.2 điểm trên thang 10.

```
import numpy as np
scores = np.array([8, 7, 9])
weights = np.array([.2, .3, .5])
print(scores@weights)
```

Kết quả khi chạy:

```
8.2
```

Tình huống 4 — Tìm hai hướng vuông góc

Từ vựng trong mục: Dot product — tích vô hướng.

Tình huống thực tế: Hai hướng [1,0] và [0,1] cần kiểm tra vuông góc bằng tích vô hướng.

Vì sao chọn cách giải này? Tích vô hướng bằng 0 với hai vector khác không chứng tỏ vuông góc.

Quy trình giải:

1. Tạo hai vector khác không [1,0] và [0,1].
2. Tính tích vô hướng.
3. In 0 và xác định hai vector vuông góc.

```
import numpy as np
a = np.array([1, 0])
b = np.array([0, 1])
print(a@b)
```

Kết quả khi chạy:

```
0
```

Tình huống 5 — Tính sai khác cảm biến

Từ vựng trong mục: Scalar — đại lượng vô hướng.

Tình huống thực tế: Hai lần đo cảm biến theo cặp cùng đơn vị là (20,30) và (23,34). Tính độ lệch Euclid.

Vì sao chọn cách giải này? Norm của hiệu vector đo khoảng cách giữa hai lần đo.

Quy trình giải:

1. Lưu hai cặp đo cùng đơn vị.
2. Tính hiệu vector và norm của hiệu.
3. In chênh lệch Euclid là 5.

```
import numpy as np
a = np.array([20, 30])
b = np.array([23, 34])
print(np.linalg.norm(a-b))
```

Kết quả khi chạy:

```
5.0
```

6. Sai lầm thường gặp

- Scalar là số đơn; vector là dãy các thành phần có thứ tự.
- Khi tính khoảng cách, các thành phần phải cùng hệ trục và đơn vị phù hợp.
- Không chuẩn hóa vector không vì norm bằng 0.
- Dấu * NumPy nhân từng phần tử; @ với hai vector 1-D tính tích vô hướng.
- Khi kiểm tra vuông góc với dữ liệu float, nên so với một ngưỡng sai số nhỏ.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a scalar? — Scalar là gì?

A. One numeric value — Một giá trị số

B. An ordered list of components — Dãy thành phần có thứ tự

C. Square root of the sum of squared components — Căn của tổng bình phương các thành phần

D. Sum of pairwise component products — Tổng tích các cặp thành phần tương ứng

Đáp án: A

Giải thích: Scalar không gồm nhiều thành phần tọa độ.

Q2. What is a vector? — Vector là gì?

A. One numeric value — Một giá trị số

B. An ordered list of components — Dãy thành phần có thứ tự

C. Square root of the sum of squared components — Căn của tổng bình phương các thành phần

D. Sum of pairwise component products — Tổng tích các cặp thành phần tương ứng

Đáp án: B

Giải thích: Vị trí từng thành phần biểu thị tọa độ hoặc đặc trưng.

Q3. What is Euclidean norm? — Norm Euclid là gì?

A. One numeric value — Một giá trị số

B. An ordered list of components — Dãy thành phần có thứ tự

C. Square root of the sum of squared components — Căn của tổng bình phương các thành phần

D. Sum of pairwise component products — Tổng tích các cặp thành phần tương ứng

Đáp án: C

Giải thích: Norm L2 đo độ dài theo Euclid.

Q4. What is the dot product? — Tích vô hướng là gì?

A. One numeric value — Một giá trị số

B. An ordered list of components — Dãy thành phần có thứ tự

C. Square root of the sum of squared components — Căn của tổng bình phương các thành phần

D. Sum of pairwise component products — Tổng tích các cặp thành phần tương ứng

Đáp án: D

Giải thích: Tích vô hướng cho một số từ hai vector cùng số thành phần.

Q5. Which function creates a NumPy vector? — Hàm nào tạo vector NumPy?

A. `np.array([1, 2])` — `np.array([1, 2])`

B. `np.linalg.norm(v)` — `np.linalg.norm(v)`

C. `v / np.linalg.norm(v)` — `v / np.linalg.norm(v)`

D. `a @ b` — `a @ b`

Đáp án: A

Giải thích: np.array lưu các thành phần số theo thứ tự.

Q6. Which function computes a vector's L2 length? — Hàm nào tính độ dài L2 của vector?

A. `np.array([1, 2])` — `np.array([1, 2])`

B. `np.linalg.norm(v)` — `np.linalg.norm(v)`

C. `v / np.linalg.norm(v)` — `v / np.linalg.norm(v)`

D. `a @ b` — `a @ b`

Đáp án: B

Giải thích: Norm mặc định trên vector số một chiều là Euclid.

Q7. Which expression normalizes a nonzero vector? — Biểu thức nào chuẩn hóa vector khác không?

A. `np.array([1, 2])` — `np.array([1, 2])`

B. `np.linalg.norm(v)` — `np.linalg.norm(v)`

C. `v / np.linalg.norm(v)` — `v / np.linalg.norm(v)`

D. `a @ b` — `a @ b`

Đáp án: C

Giải thích: Chia cho norm đưa độ dài vector về một.

Q8. Which expression computes the dot product of two 1-D vectors? — Biểu thức nào tính tích vô hướng hai vector 1-D?

A. `np.array([1, 2])` — `np.array([1, 2])`

B. `np.linalg.norm(v)` — `np.linalg.norm(v)`

C. `v / np.linalg.norm(v)` — `v / np.linalg.norm(v)`

D. `a @ b` — `a @ b`

Đáp án: D

Giải thích: Toán tử @ tính tích vô hướng của hai vector.

Q9. What is the norm of vector (3,4)? — Norm của vector (3,4) bằng bao nhiêu?

A. 5 — 5

B. (0.6,0.8) — (0.6,0.8)

C. 11 — 11

D. 0 — 0

Đáp án: A

Giải thích: Độ dài Pythagoras là căn của 9+16.

Q10. What is the unit vector in the direction (3,4)? — Vector đơn vị theo hướng (3,4) là gì?

A. 5 — 5

B. (0.6,0.8) — (0.6,0.8)

C. 11 — 11

D. 0 — 0

Đáp án: B

Giải thích: Chia mỗi thành phần cho độ dài năm.

Q11. What is the dot product (1,2)·(3,4)? — Tích vô hướng (1,2)·(3,4) bằng bao nhiêu?

A. 5 — 5

B. (0.6,0.8) — (0.6,0.8)

C. 11 — 11

D. 0 — 0

Đáp án: C

Giải thích: Tính 1×3 + 2×4.

Q12. What is the dot product of (1,0) and (0,1)? — Tích vô hướng của (1,0) và (0,1) là bao nhiêu?

A. 5 — 5

B. (0.6,0.8) — (0.6,0.8)

C. 11 — 11

D. 0 — 0

Đáp án: D

Giải thích: Cả hai tích thành phần đều bằng không.

Q13. What is the dot product of two nonzero perpendicular vectors? — Tích vô hướng của hai vector khác không vuông góc là gì?

A. Zero — Bằng không

B. That its norm is nonzero — Norm khác không

C. The result can be physically misleading — Kết quả có thể sai ý nghĩa vật lý

D. They must have equal lengths — Chúng phải có cùng số phần tử

Đáp án: A

Giải thích: Hai hướng trực giao có tích vô hướng bằng không.

Q14. What must be checked before normalizing a vector? — Cần kiểm tra gì trước khi chuẩn hóa vector?

A. Zero — Bằng không

B. That its norm is nonzero — Norm khác không

C. The result can be physically misleading — Kết quả có thể sai ý nghĩa vật lý

D. They must have equal lengths — Chúng phải có cùng số phần tử

Đáp án: B

Giải thích: Chia cho norm bằng không không xác định.

Q15. What happens if coordinate units are mixed in a distance calculation? — Nếu đơn vị tọa độ không thống nhất khi tính khoảng cách thì sao?

A. Zero — Bằng không

B. That its norm is nonzero — Norm khác không

C. The result can be physically misleading — Kết quả có thể sai ý nghĩa vật lý

D. They must have equal lengths — Chúng phải có cùng số phần tử

Đáp án: C

Giải thích: Tính norm có ý nghĩa khi đơn vị phù hợp.

Q16. What must be true to compute a 1-D dot product? — Muốn tính tích vô hướng của hai vector 1-D cần điều kiện gì?

A. Zero — Bằng không

B. That its norm is nonzero — Norm khác không

C. The result can be physically misleading — Kết quả có thể sai ý nghĩa vật lý

D. They must have equal lengths — Chúng phải có cùng số phần tử

Đáp án: D

Giải thích: Từng thành phần được ghép theo vị trí tương ứng.

Q17. What does multiplying a vector by a scalar do? — Nhân vector với scalar làm gì?

A. Scales every component — Nhân từng thành phần với scalar

B. Subtract corresponding components — Trừ các thành phần cùng vị trí

C. Euclidean distance between a and b — Khoảng cách Euclid giữa a và b

D. To multiply each weight by its intended feature — Để trọng số nhân đúng đặc trưng tương ứng

Đáp án: A

Giải thích: Nhân scalar làm đổi độ lớn từng thành phần.

Q18. How is vector subtraction performed? — Phép trừ vector được thực hiện thế nào?

A. Scales every component — Nhân từng thành phần với scalar

B. Subtract corresponding components — Trừ các thành phần cùng vị trí

C. Euclidean distance between a and b — Khoảng cách Euclid giữa a và b

D. To multiply each weight by its intended feature — Để trọng số nhân đúng đặc trưng tương ứng

Đáp án: B

Giải thích: Phép trừ vector diễn ra theo từng thành phần.

Q19. What does `np.linalg.norm(a-b)` measure? — `np.linalg.norm(a-b)` đo điều gì?

A. Scales every component — Nhân từng thành phần với scalar

B. Subtract corresponding components — Trừ các thành phần cùng vị trí

C. Euclidean distance between a and b — Khoảng cách Euclid giữa a và b

D. To multiply each weight by its intended feature — Để trọng số nhân đúng đặc trưng tương ứng

Đáp án: C

Giải thích: Norm của độ dịch chuyển cho khoảng cách.

Q20. Why are feature weights paired with values in the same order? — Vì sao trọng số cần cùng thứ tự với các giá trị?

A. Scales every component — Nhân từng thành phần với scalar

B. Subtract corresponding components — Trừ các thành phần cùng vị trí

C. Euclidean distance between a and b — Khoảng cách Euclid giữa a và b

D. To multiply each weight by its intended feature — Để trọng số nhân đúng đặc trưng tương ứng

Đáp án: D

Giải thích: Sai thứ tự tạo ra tích vô hướng khác.

8. Nối sang bài kế tiếp

`NumPy → Scalar/Vector → Matrix → PCA.`

PY-25 — Ma trận, chuyển vị, nhân ma trận và định thức

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Matrix — ma trận; **Transpose** — chuyển vị.

`Vector → Matrix → Hệ tuyến tính → Eigenvalue/PCA`

Học xong dùng để làm gì? Tổ chức bảng đặc trưng thành ma trận và nhân ma trận để biến đổi dữ liệu nhiều chiều.

Kết nối kiến thức tiếp theo: PY-26 giải các ràng buộc dưới dạng hệ tuyến tính và kiểm tra rank.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Matrix	ma trận	Bảng hai chiều chứa giá trị theo hàng và cột
Transpose	chuyển vị	Đổi vai trò hàng và cột của ma trận
Determinant	định thức	Số của ma trận vuông dùng xét tính khả nghịch
Inverse	ma trận nghịch đảo	Ma trận B làm AB=BA=I khi tồn tại

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.eye(n)`	Ma trận đơn vị vuông, nhân vào không đổi vector
`A.T / np.transpose(A)`	Chuyển vị ma trận
`A @ B`	Nhân ma trận khi số cột A bằng số hàng B
`np.linalg.det()/inv()`	Định thức và nghịch đảo khi khả nghịch; giải hệ nên ưu tiên solve

4. Khái niệm liên quan

Ma trận (matrix) là bảng có hàng và cột được xác định rõ theo dữ liệu. Từ bảng điểm hai học viên và ba môn, bài học giải thích chuyển vị để đổi cách xem, nhân ma trận để tổng hợp trọng số, rồi định thức và nghịch đảo khi làm việc với ma trận vuông.

4.1. Matrix — Bảng số có hàng và cột

Ma trận là bảng số hai chiều. Với điểm học viên An `[8,7,9]` và Bình `[6,5,8]` ở ba môn Toán, Văn, Python, bảng có **hai hàng học viên** và **ba cột môn**. Giá trị `[1,0]` nghĩa là điểm Toán của Bình bằng 6. `shape` cho kích thước, theo thứ tự (số hàng, số cột).

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.shape)
```

Kết quả khi chạy:

```
(2, 3)
```

Bảng có hai học viên và ba môn học.

4.2. Transpose — Hoán đổi hai trục

Chuyển vị `A.T` biến hàng thành cột và cột thành hàng. Với bảng điểm hai học viên × ba môn, ma trận sau chuyển vị có ba hàng môn và hai cột học viên. Ví dụ hàng đầu là điểm Toán của An và Bình, `[8,6]`; không có điểm nào bị thay đổi.

```
import numpy as np

scores = np.array([[8, 7, 9], [6, 5, 8]])
print(scores.T)
```

Kết quả khi chạy:

```
[[8 6]
 [7 5]
 [9 8]]
```

Kết quả gồm ba hàng môn × hai cột học viên.

4.3. Matrix multiplication — Nhân ma trận

Nhân `X @ W` yêu cầu số cột của X bằng số hàng của W. Bảng điểm X có shape `(2,3)`, trọng số ba môn W có shape `(3,1)`, tích có shape `(2,1)` cho điểm tổng của từng học viên. Dòng An là `8×0.2 + 7×0.3 + 9×0.5 = 8.2`. `@` khác phép nhân từng phần tử `*`.

```
import numpy as np

X = np.array([[8, 7, 9], [6, 5, 8]])
W = np.array([[0.2], [0.3], [0.5]])
print(X @ W)
```

Kết quả khi chạy:

```
[[8.2]
 [6.7]]
```

Hai hàng là hai học viên; một cột là điểm tổng kết của mỗi người.

4.4. Determinant và inverse — Khả nghịch

Định thức `det(A)` là số xác định cho **ma trận vuông**. Với A=`[[2,1],[1,2]]`, định thức bằng `2×2 - 1×1 = 3`, khác 0, nên tồn tại ma trận nghịch đảo A^{-1} thỏa `A @ A⁻¹ = I`. Nếu định thức bằng 0 thì không có nghịch đảo. Khi giải hệ phương trình, ưu tiên `np.linalg.solve` hơn việc tính nghịch đảo trực tiếp.

```
import numpy as np

A = np.array([[2.0, 1.0], [1.0, 2.0]])
print(round(float(np.linalg.det(A)), 6))
```

Kết quả khi chạy:

```
3.0
```

Định thức 3 khác 0, nên ma trận vuông này khả nghịch.

5. Năm tình huống ứng dụng

Tình huống 1 — Đổi bố cục dữ liệu báo cáo

Từ vựng trong mục: Matrix — ma trận.

Tình huống thực tế: Báo cáo bán hàng có 2 cửa hàng và 3 ngày, mỗi hàng là một cửa hàng. Đổi thành 3 hàng ngày và 2 cột cửa hàng.

Vì sao chọn cách giải này? Chuyển vị đổi bảng hai hàng cửa hàng × ba cột ngày thành ba hàng ngày × hai cột cửa hàng.

Quy trình giải:

1. Lưu bảng hai cửa hàng × ba ngày.
2. Chuyển vị bằng `.T`.
3. In shape `(3,2)` tương ứng ngày × cửa hàng.

```
import numpy as np
X = np.array([[10, 20, 30], [12, 8, 5]])
print(X.T.shape)
```

Kết quả khi chạy:

```
(3, 2)
```

Tình huống 2 — Nhân bảng điểm và trọng số

Từ vựng trong mục: Matrix multiplication — nhân ma trận.

Tình huống thực tế: Hai học viên có điểm theo ba môn lần lượt (8,7,9) và (6,5,8); hệ số là (0.2,0.3,0.5). Tính điểm có trọng số.

Vì sao chọn cách giải này? Nhân bảng điểm 2×3 với trọng số 3×1 tạo mỗi học viên một điểm tổng.

Quy trình giải:

1. Tạo bảng hai học viên × ba môn.
2. Đặt trọng số ba môn theo một cột (3×1).
3. Nhân `X @ W` và in hai điểm tổng.

```
import numpy as np
X = np.array([[8, 7, 9], [6, 5, 8]])
W = np.array([[.2], [.3], [.5]])
print(X@W)
```

Kết quả khi chạy:

```
[[8.2]
 [6.7]]
```

Tình huống 3 — Kiểm tra ma trận khả nghịch

Từ vựng trong mục: Determinant — định thức.

Tình huống thực tế: Ma trận hệ số [[2,1],[1,2]] có thể khả nghịch nếu định thức khác 0. Tính định thức để kiểm tra.

Vì sao chọn cách giải này? Định thức khác 0 xác định ma trận vuông khả nghịch; làm tròn hạn chế nhiễu số cuối.

Quy trình giải:

1. Tạo ma trận vuông hệ số `[[2,1],[1,2]]`.
2. Tính định thức và làm tròn sáu chữ số.
3. Kiểm tra giá trị 3 khác 0 để kết luận khả nghịch.

```
import numpy as np
A = np.array([[2, 1], [1, 2]], dtype=float)
print(round(np.linalg.det(A), 6))
```

Kết quả khi chạy:

```
3.0
```

Tình huống 4 — Kiểm tra nghịch đảo

Từ vựng trong mục: Inverse — ma trận nghịch đảo.

Tình huống thực tế: Với ma trận đường chéo diag(2,4), kiểm tra tích ma trận và nghịch đảo có gần ma trận đơn vị không.

Vì sao chọn cách giải này? Nhân ma trận với nghịch đảo gần ma trận đơn vị; `allclose` chấp nhận sai số dấu phẩy động.

Quy trình giải:

1. Tạo ma trận đường chéo `diag(2,4)`.
2. Tính nghịch đảo rồi nhân `A @ inv(A)`.
3. So với ma trận đơn vị bằng `np.allclose`.

```
import numpy as np
A = np.diag([2., 4.])
print(np.allclose(A@np.linalg.inv(A), np.eye(2)))
```

Kết quả khi chạy:

```
True
```

Tình huống 5 — Kiểm tra shape trước nhân

Từ vựng trong mục: Matrix — ma trận.

Tình huống thực tế: Một ma trận 2×3 nhân một ma trận 3×2. Kiểm tra kích thước tích kết quả.

Vì sao chọn cách giải này? Kiểm tra chiều trong bằng nhau để xác định phép nhân hợp lệ và shape kết quả.

Quy trình giải:

1. Tạo A shape 2×3 và B shape 3×2.
2. So số cột A với số hàng B.
3. Nhân A@B và in shape `(2,2)`.

```
import numpy as np
A = np.ones((2, 3))
B = np.ones((3, 2))
assert A.shape[1]==B.shape[0]
print((A@B).shape)
```

Kết quả khi chạy:

```
(2, 2)
```

6. Sai lầm thường gặp

- `shape=(2,3)` nghĩa là hai hàng và ba cột, không phải ba hàng hai cột.
- `.T` hoán đổi trục; phép nhân `@` yêu cầu số cột bên trái bằng số hàng bên phải.
- Định thức chỉ tính trên ma trận vuông; ma trận có định thức 0 không có nghịch đảo.
- Dữ liệu số thực có sai số làm tròn; dùng `np.allclose` khi kiểm tra gần đúng với ma trận đơn vị.
- Khi giải hệ tuyến tính, thường ưu tiên `np.linalg.solve` thay vì tính nghịch đảo trực tiếp.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Từ vựng trong mục: Question — câu hỏi; **Answer** — đáp án.

Q1. What is a matrix? — Ma trận là gì?

A. A two-dimensional array of values — Bảng dữ liệu hai chiều

B. A matrix with rows and columns exchanged — Ma trận đổi vai trò hàng và cột

C. Rows of the left matrix with columns of the right — Hàng ma trận trái với cột ma trận phải

D. A scalar associated with a square matrix — Một số gắn với ma trận vuông

Đáp án: A

Giải thích: Ma trận sắp phần tử theo hàng và cột.

Q2. What is the transpose of a matrix? — Chuyển vị của ma trận là gì?

A. A two-dimensional array of values — Bảng dữ liệu hai chiều

B. A matrix with rows and columns exchanged — Ma trận đổi vai trò hàng và cột

C. Rows of the left matrix with columns of the right — Hàng ma trận trái với cột ma trận phải

D. A scalar associated with a square matrix — Một số gắn với ma trận vuông

Đáp án: B

Giải thích: Chuyển vị hoán đổi các trục ma trận.

Q3. What does matrix multiplication combine? — Phép nhân ma trận kết hợp gì?

A. A two-dimensional array of values — Bảng dữ liệu hai chiều

B. A matrix with rows and columns exchanged — Ma trận đổi vai trò hàng và cột

C. Rows of the left matrix with columns of the right — Hàng ma trận trái với cột ma trận phải

D. A scalar associated with a square matrix — Một số gắn với ma trận vuông

Đáp án: C

Giải thích: Mỗi ô kết quả là tích vô hướng một hàng với một cột.

Q4. What is a determinant? — Định thức là gì?

A. A two-dimensional array of values — Bảng dữ liệu hai chiều

B. A matrix with rows and columns exchanged — Ma trận đổi vai trò hàng và cột

C. Rows of the left matrix with columns of the right — Hàng ma trận trái với cột ma trận phải

D. A scalar associated with a square matrix — Một số gắn với ma trận vuông

Đáp án: D

Giải thích: Định thức hỗ trợ kiểm tra tính khả nghịch.

Q5. What is the shape of a matrix with two rows and three columns? — Ma trận hai hàng ba cột có shape gì?

A. `(2, 3)` — `(2, 3)`

B. `(3, 2)` — `(3, 2)`

C. `(2, 1)` — `(2, 1)`

D. `(3, 3)` — `(3, 3)`

Đáp án: A

Giải thích: Shape ghi số hàng trước số cột.

Q6. What is the transpose shape of a (2,3) matrix? — Chuyển vị ma trận (2,3) có shape gì?

A. `(2, 3)` — `(2, 3)`

B. `(3, 2)` — `(3, 2)`

C. `(2, 1)` — `(2, 1)`

D. `(3, 3)` — `(3, 3)`

Đáp án: B

Giải thích: Hàng và cột đổi vai trò.

Q7. What shape results from (2,3) @ (3,1)? — Phép (2,3) @ (3,1) cho shape gì?

A. `(2, 3)` — `(2, 3)`

B. `(3, 2)` — `(3, 2)`

C. `(2, 1)` — `(2, 1)`

D. `(3, 3)` — `(3, 3)`

Đáp án: C

Giải thích: Hai chiều trong bằng 3; giữ hai chiều ngoài.

Q8. What is the shape of `np.eye(3)`? — `np.eye(3)` tạo ma trận shape nào?

A. `(2, 3)` — `(2, 3)`

B. `(3, 2)` — `(3, 2)`

C. `(2, 1)` — `(2, 1)`

D. `(3, 3)` — `(3, 3)`

Đáp án: D

Giải thích: Ma trận đơn vị bậc 3 là ma trận vuông.

Q9. Which operator performs matrix multiplication? — Toán tử nào thực hiện nhân ma trận?

A. `@` — `@`

B. `.T` — `.T`

C. `np.linalg.det(A)` — `np.linalg.det(A)`

D. `np.linalg.inv(A)` — `np.linalg.inv(A)`

Đáp án: A

Giải thích: @ thực hiện đúng quy tắc nhân ma trận.

Q10. Which attribute returns the transpose of a 2-D ndarray? — Thuộc tính nào cho chuyển vị mảng 2-D?

A. `@` — `@`

B. `.T` — `.T`

C. `np.linalg.det(A)` — `np.linalg.det(A)`

D. `np.linalg.inv(A)` — `np.linalg.inv(A)`

Đáp án: B

Giải thích: T là thuộc tính chuyển vị tiện dụng.

Q11. Which function computes a square matrix determinant? — Hàm nào tính định thức ma trận vuông?

A. `@` — `@`

B. `.T` — `.T`

C. `np.linalg.det(A)` — `np.linalg.det(A)`

D. `np.linalg.inv(A)` — `np.linalg.inv(A)`

Đáp án: C

Giải thích: det trả định thức dạng số.

Q12. Which function computes a nonsingular matrix inverse? — Hàm nào tính nghịch đảo ma trận khả nghịch?

A. `@` — `@`

B. `.T` — `.T`

C. `np.linalg.det(A)` — `np.linalg.det(A)`

D. `np.linalg.inv(A)` — `np.linalg.inv(A)`

Đáp án: D

Giải thích: inv tính nghịch đảo khi ma trận khả nghịch.

Q13. When is a square matrix invertible? — Khi nào ma trận vuông khả nghịch?

A. Its determinant is nonzero — Định thức khác không

B. 3 — 3

C. Identity matrix — Ma trận đơn vị

D. `np.allclose(A, B)` — `np.allclose(A, B)`

Đáp án: A

Giải thích: Định thức khác 0 tương đương khả nghịch.

Q14. What is determinant of [[2,1],[1,2]]? — Định thức [[2,1],[1,2]] bằng gì?

A. Its determinant is nonzero — Định thức khác không

B. 3 — 3

C. Identity matrix — Ma trận đơn vị

D. `np.allclose(A, B)` — `np.allclose(A, B)`

Đáp án: B

Giải thích: Định thức 2×2 bằng 2×2−1×1.

Q15. What does `A @ inv(A)` approximate? — `A @ inv(A)` gần bằng gì?

A. Its determinant is nonzero — Định thức khác không

B. 3 — 3

C. Identity matrix — Ma trận đơn vị

D. `np.allclose(A, B)` — `np.allclose(A, B)`

Đáp án: C

Giải thích: Nhân ma trận với nghịch đảo bằng I trong tính toán chính xác.

Q16. How do you test approximate equality of float arrays? — Kiểm tra hai mảng float gần bằng nhau bằng gì?

A. Its determinant is nonzero — Định thức khác không

B. 3 — 3

C. Identity matrix — Ma trận đơn vị

D. `np.allclose(A, B)` — `np.allclose(A, B)`

Đáp án: D

Giải thích: allclose chấp nhận sai số dấu phẩy động nhỏ.

Q17. Which dimensions must match for A @ B? — Nhân A @ B cần các chiều nào bằng nhau?

A. Columns of A and rows of B — Số cột A và số hàng B

B. Element-wise multiplication — Nhân theo từng phần tử

C. `np.linalg.solve(A, b)` — `np.linalg.solve(A, b)`

D. A linear algebra error — Lỗi đại số tuyến tính

Đáp án: A

Giải thích: Nhân ma trận yêu cầu hai chiều trong trùng.

Q18. What does `A * B` do for compatible NumPy arrays? — `A * B` trên mảng NumPy tương thích làm gì?

A. Columns of A and rows of B — Số cột A và số hàng B

B. Element-wise multiplication — Nhân theo từng phần tử

C. `np.linalg.solve(A, b)` — `np.linalg.solve(A, b)`

D. A linear algebra error — Lỗi đại số tuyến tính

Đáp án: B

Giải thích: Dấu sao nhân phần tử tương ứng có broadcasting.

Q19. What is preferred to solve Ax=b numerically? — Để giải hệ Ax=b bằng NumPy nên ưu tiên gì?

A. Columns of A and rows of B — Số cột A và số hàng B

B. Element-wise multiplication — Nhân theo từng phần tử

C. `np.linalg.solve(A, b)` — `np.linalg.solve(A, b)`

D. A linear algebra error — Lỗi đại số tuyến tính

Đáp án: C

Giải thích: Giải trực tiếp thường tốt hơn tính nghịch đảo rồi nhân.

Q20. What happens to a singular matrix when using `np.linalg.inv`? — Dùng `np.linalg.inv` với ma trận suy biến thường gây gì?

A. Columns of A and rows of B — Số cột A và số hàng B

B. Element-wise multiplication — Nhân theo từng phần tử

C. `np.linalg.solve(A, b)` — `np.linalg.solve(A, b)`

D. A linear algebra error — Lỗi đại số tuyến tính

Đáp án: D

Giải thích: Ma trận suy biến không có nghịch đảo.

8. Nối sang bài kế tiếp

`Vector → Matrix → Hệ tuyến tính → Eigenvalue/PCA`.

PY-26 — Hệ phương trình tuyến tính, rank và nghiệm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Linear system — hệ tuyến tính; **Rank** — hạng ma trận.

`Ma trận → Rank/Solve → Regression → AI`

Học xong dùng để làm gì? Kiểm tra hệ ràng buộc có đủ thông tin để giải hay có phương trình trùng, thiếu hoặc mâu thuẫn.

Kết nối kiến thức tiếp theo: PY-27 xác định hướng đặc biệt của phép biến đổi ma trận qua trị riêng/ vector riêng.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Linear system	hệ tuyến tính	Nhiều phương trình chứa cùng các ẩn số
Rank	hạng ma trận	Số hàng/cột độc lập tuyến tính
Coefficient matrix	ma trận hệ số	Các hệ số của ẩn trong Ax=b
Residual	sai số dư	Độ sai khác giữa dự đoán Ax và b

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.linalg.solve(A,b)`	Giải Ax=b khi ma trận A vuông không suy biến
`np.linalg.matrix_rank(A)`	Kiểm tra số chiều độc lập tuyến tính
`np.column_stack((A,b))`	Ghép ma trận hệ số và vế phải thành augmented matrix
`np.linalg.lstsq(A,b,rcond=None)`	Tìm nghiệm bình phương tối thiểu khi quá nhiều phương trình

4. Khái niệm liên quan

Nội dung chính: một hệ tuyến tính liên kết nhiều phương trình qua cùng các ẩn. Ta xác định đúng **hàng = phương trình**, **cột = ẩn**, **vế phải = giá trị đã biết**, sau đó kiểm tra tính độc lập và độ khớp của nghiệm.

4.1. Linear system — Một hệ phương trình chung các ẩn

Từ vựng trong mục: Linear system — hệ tuyến tính; **Equation** — phương trình.

Hệ tuyến tính là **nhiều phương trình cùng chứa các ẩn cần tìm**. Cửa hàng bán loại A giá x và loại B giá y (đơn vị nghìn đồng): mua một A và một B hết 5; mua hai A và một B hết 8. Ta có x+y=5 và 2x+y=8. Để viết Ax=b, mỗi **hàng của A là một phương trình**, mỗi **cột là hệ số của một ẩn**, b chứa tổng tiền từng hóa đơn. Giải hệ tìm được x=3, y=2.

```
import numpy as np

A = np.array([[1, 1], [2, 1]], dtype=float)
b = np.array([5, 8], dtype=float)
print(np.linalg.solve(A, b))
```

Kết quả khi chạy:

```
[3. 2.]
```

Loại A giá 3 nghìn và loại B giá 2 nghìn đồng; nghiệm đồng thời thỏa hai hóa đơn.

4.2. Rank — Số thông tin độc lập

Từ vựng trong mục: Rank — hạng; **Linear dependence** — phụ thuộc tuyến tính.

Rank cho biết có bao nhiêu hàng/cột độc lập tuyến tính trong một ma trận. Ví dụ hai hàng `[1,2]` và `[2,4]`: hàng sau chỉ bằng hai lần hàng đầu nên không đưa thêm thông tin độc lập. Rank bằng 1 dù bảng có hai hàng và hai cột. Với một hệ 2 ẩn, nếu rank(A)=2 và A vuông thì nghiệm duy nhất khi hệ có vế phải b phù hợp (ở đây ma trận khả nghịch); nếu rank thấp cần kiểm tra cả ma trận mở rộng để biết vô nghiệm hay vô số nghiệm.

```
import numpy as np

A = np.array([[1, 2], [2, 4]])
print(np.linalg.matrix_rank(A))
```

Kết quả khi chạy:

```
1
```

Hai hàng phụ thuộc nhau nên số chiều thông tin độc lập chỉ là một.

4.3. Coefficient matrix — Bảng hệ số và vế phải

Từ vựng trong mục: Coefficient matrix — ma trận hệ số; **Augmented matrix** — ma trận mở rộng.

Trong Ax=b, **A chỉ chứa hệ số của các ẩn**, còn b chứa **giá trị đích của từng phương trình**. Ví dụ hệ x+y=5, 2x+y=8 có A hai hàng × hai cột, b dài hai phần tử. Ghép b thành một cột cuối bằng `np.column_stack((A,b))` cho ma trận mở rộng 2×3. Cột cuối này **không phải một ẩn mới**. Khi giải, số hàng A phải bằng số phần tử b.

```
import numpy as np

A = np.array([[1, 1], [2, 1]])
b = np.array([5, 8])
print(np.column_stack((A, b)))
```

Kết quả khi chạy:

```
[[1 1 5]
 [2 1 8]]
```

Hai cột đầu là hệ số của x, y; cột cuối là tổng tiền hóa đơn.

4.4. Residual — Sai lệch và nghiệm bình phương tối thiểu

Từ vựng trong mục: Residual — sai số dư; **Least squares** — bình phương tối thiểu.

Khi thử nghiệm x cho hệ `Ax=b`, **sai số dư** `r=Ax-b` đo mỗi phương trình lệch bao nhiêu so với vế phải. Nếu `r=0` thì nghiệm thỏa chính xác; dữ liệu đo thực tế thường có nhiễu nên không tìm được nghiệm thỏa mọi hàng. Khi số phương trình nhiều hơn số ẩn, `np.linalg.lstsq` tìm x làm **tổng bình phương các sai số dư nhỏ nhất**. Ví dụ ba số đo có x=1,2,3 và giá trị y=2,4,6.1; fit đường `y=m*x+c` không khớp hoàn toàn và cho gần `m=2.05`, `c=-0.067`.

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([2, 4, 6.1])
A = np.column_stack((x, np.ones_like(x)))
coeff = np.linalg.lstsq(A, y, rcond=None)[0]
print(np.round(coeff, 3))
```

Kết quả khi chạy:

```
[ 2.05  -0.067]
```

Hệ số góc xấp xỉ 2,05 và hệ số chặn xấp xỉ −0,067; đây là nghiệm xấp xỉ do số đo có nhiễu.

5. Năm tình huống ứng dụng

Tình huống 1 — Tính giá hai loại hàng

Tình huống thực tế: Một hóa đơn gồm một món A và một món B giá tổng cộng 5 nghìn đồng; hóa đơn khác gồm hai A và một B hết 8 nghìn đồng. Hai đơn giá chưa biết, ký hiệu x,y. Tìm giá từng món.

Vì sao chọn cách giải này? Hệ hai phương trình độc lập và hai ẩn có ma trận hệ số vuông, khả nghịch; dùng `solve` tìm nghiệm chính xác.

Quy trình giải:

1. Đặt hai ẩn x, y là giá A, B, cùng đơn vị nghìn đồng.
2. Đưa hệ số từ hai hóa đơn vào hai hàng ma trận A và tổng tiền vào b.
3. Dùng `np.linalg.solve` rồi in hai đơn giá.

```
import numpy as np

A = np.array([[1, 1], [2, 1]], dtype=float)
b = np.array([5, 8], dtype=float)
print(np.linalg.solve(A, b))
```

Kết quả khi chạy:

```
[3. 2.]
```

Giá món A là 3 nghìn, món B là 2 nghìn đồng.

Tình huống 2 — Phát hiện hai cột dữ liệu phụ thuộc

Tình huống thực tế: Bảng đặc trưng của hai mẫu có hàng `[1,2]`, `[2,4]`. Cột hai luôn gấp đôi cột một, nên đội phân tích muốn biết bảng có hai hướng thông tin độc lập hay không.

Vì sao chọn cách giải này? Rank phản ánh số chiều độc lập; với hai cột tỷ lệ nhau, hạng chỉ bằng một.

Quy trình giải:

1. Tạo ma trận hai hàng × hai cột, mỗi hàng là một mẫu.
2. Dùng `matrix_rank` tính số chiều độc lập.
3. In rank để đối chiếu với hai cột đầu vào.

```
import numpy as np

A = np.array([[1, 2], [2, 4]])
print(np.linalg.matrix_rank(A))
```

Kết quả khi chạy:

```
1
```

Chỉ có một hướng thông tin độc lập, không phải hai.

Tình huống 3 — Kiểm tra nghiệm của hệ

Tình huống thực tế: Một hệ có A=`[[1,1],[2,1]]` và b=`[5,8]`. Người học đề xuất nghiệm x=`[3,2]`. Kiểm tra nghiệm này có tái tạo được cả hai vế phải không.

Vì sao chọn cách giải này? Tính `A@x` rồi dùng `np.allclose` so b, tránh lệ thuộc vào việc tự nhìn từng phương trình.

Quy trình giải:

1. Nhập ma trận hai phương trình và hai giá trị ở vế phải.
2. Nhập nghiệm thử `[3,2]`.
3. So `A @ x` với b và in kết quả Boolean.

```
import numpy as np

A = np.array([[1, 1], [2, 1]])
b = np.array([5, 8])
x = np.array([3, 2])
print(np.allclose(A @ x, b))
```

Kết quả khi chạy:

```
True
```

Nghiệm đã cho thỏa đồng thời cả hai phương trình.

Tình huống 4 — Ước lượng đường thẳng từ số đo có nhiễu

Tình huống thực tế: Thiết bị ghi x lần lượt 1,2,3 và y lần lượt 2,4,6.1, cùng đơn vị đo phù hợp. Cần ước lượng đường y=m*x+c dù ba điểm không nằm chính xác trên một đường.

Vì sao chọn cách giải này? `lstsq` tối thiểu hóa tổng bình phương sai số khi ba phương trình chỉ có hai ẩn m,c.

Quy trình giải:

1. Đặt mỗi hàng của A là `[x_i,1]`, tương ứng hệ số m và c.
2. Đưa ba giá trị y vào vế phải và giải bằng `lstsq`.
3. Làm tròn hệ số để đầu ra minh họa ổn định.

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([2, 4, 6.1])
A = np.column_stack((x, np.ones_like(x)))
coeff = np.linalg.lstsq(A, y, rcond=None)[0]
print(np.round(coeff, 3))
```

Kết quả khi chạy:

```
[ 2.05  -0.067]
```

Đường khớp xấp xỉ `y=2.05x−0.067`, không phải một đường đi qua chính xác cả ba số đo.

Tình huống 5 — Đo sai số dư của một dự báo

Tình huống thực tế: Một mô hình trả dự báo `[2,3]` cho hai giá trị mục tiêu `[2,4]` cùng đơn vị. Cần biết mô hình chênh bao nhiêu ở từng quan sát, quy ước dư = dự báo − thực tế.

Vì sao chọn cách giải này? Vector `Ax-b` mô tả độ lệch có dấu cho từng hàng; số âm nghĩa là dự báo thấp hơn thực tế.

Quy trình giải:

1. Dùng ma trận đơn vị để `A@x` chính là hai dự báo.
2. Trừ vector b của hai giá trị thực tế.
3. In vector sai số dư theo thứ tự hai quan sát.

```
import numpy as np

A = np.eye(2)
x = np.array([2, 3])
b = np.array([2, 4])
print(A @ x - b)
```

Kết quả khi chạy:

```
[ 0. -1.]
```

Quan sát đầu đúng hoàn toàn; quan sát thứ hai dự báo thấp hơn thực tế một đơn vị.

6. Sai lầm thường gặp

- Một hàng của A là một phương trình; một cột là hệ số của một ẩn. Không được đảo hai chiều khi đọc dữ liệu.
- b là giá trị đã biết ở vế phải, không phải cột hệ số của một ẩn mới.
- Hệ phụ thuộc hàng có thể vô nghiệm hoặc vô số nghiệm; xét cả rank(A) và rank([A, b]), không kết luận chỉ từ rank(A).
- `solve` dành cho ma trận vuông khả nghịch; `lstsq` giải gần đúng được cả hệ có nhiều số đo.
- Residual có dấu; bình phương tối thiểu tối ưu tổng **bình phương** dư chứ không tối ưu tổng dư có dấu.
- Chỉ dùng `np.allclose` để kiểm nghiệm trong sai số số học, không coi đó là bằng nhau tuyệt đối.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Các câu hỏi chỉ kiểm tra khái niệm và cách áp dụng kiến thức, không yêu cầu nhớ tình huống thực hành.

Q1. What is a linear system? — Hệ tuyến tính là gì?

A. Several linear equations sharing unknowns — Nhiều phương trình tuyến tính chung các ẩn

B. The number of linearly independent rows or columns — Số hàng hoặc cột độc lập tuyến tính

C. The coefficients multiplying the unknowns — Các hệ số nhân với từng ẩn

D. `A @ x - b` — `A @ x - b`

Đáp án: A

Giải thích: Các phương trình cùng chứa các ẩn cần giải.

Q2. What does the rank of a matrix measure? — Rank của ma trận đo gì?

A. Several linear equations sharing unknowns — Nhiều phương trình tuyến tính chung các ẩn

B. The number of linearly independent rows or columns — Số hàng hoặc cột độc lập tuyến tính

C. The coefficients multiplying the unknowns — Các hệ số nhân với từng ẩn

D. `A @ x - b` — `A @ x - b`

Đáp án: B

Giải thích: Rank đếm hướng độc lập, không chỉ số hàng.

Q3. What does a coefficient matrix contain? — Ma trận hệ số chứa gì?

A. Several linear equations sharing unknowns — Nhiều phương trình tuyến tính chung các ẩn

B. The number of linearly independent rows or columns — Số hàng hoặc cột độc lập tuyến tính

C. The coefficients multiplying the unknowns — Các hệ số nhân với từng ẩn

D. `A @ x - b` — `A @ x - b`

Đáp án: C

Giải thích: Ma trận chứa hệ số số học của từng ẩn.

Q4. What is the residual vector for Ax=b? — Vector sai số dư của Ax=b là gì?

A. Several linear equations sharing unknowns — Nhiều phương trình tuyến tính chung các ẩn

B. The number of linearly independent rows or columns — Số hàng hoặc cột độc lập tuyến tính

C. The coefficients multiplying the unknowns — Các hệ số nhân với từng ẩn

D. `A @ x - b` — `A @ x - b`

Đáp án: D

Giải thích: Lấy Ax trừ b cho độ lệch từng phương trình.

Q5. Which tool solves a nonsingular square system? — Hàm nào giải hệ vuông không suy biến?

A. `np.linalg.solve(A,b)` — `np.linalg.solve(A,b)`

B. `np.linalg.matrix_rank(A)` — `np.linalg.matrix_rank(A)`

C. `np.column_stack((A,b))` — `np.column_stack((A,b))`

D. `np.linalg.lstsq(A,b,rcond=None)` — `np.linalg.lstsq(A,b,rcond=None)`

Đáp án: A

Giải thích: solve dùng cho ma trận hệ số vuông, không suy biến.

Q6. Which tool determines matrix rank? — Hàm nào tính hạng ma trận?

A. `np.linalg.solve(A,b)` — `np.linalg.solve(A,b)`

B. `np.linalg.matrix_rank(A)` — `np.linalg.matrix_rank(A)`

C. `np.column_stack((A,b))` — `np.column_stack((A,b))`

D. `np.linalg.lstsq(A,b,rcond=None)` — `np.linalg.lstsq(A,b,rcond=None)`

Đáp án: B

Giải thích: matrix_rank đo số chiều độc lập.

Q7. Which tool builds an augmented matrix? — Hàm nào ghép ma trận mở rộng?

A. `np.linalg.solve(A,b)` — `np.linalg.solve(A,b)`

B. `np.linalg.matrix_rank(A)` — `np.linalg.matrix_rank(A)`

C. `np.column_stack((A,b))` — `np.column_stack((A,b))`

D. `np.linalg.lstsq(A,b,rcond=None)` — `np.linalg.lstsq(A,b,rcond=None)`

Đáp án: C

Giải thích: column_stack ghép b thành cột bên cạnh A.

Q8. Which tool computes a least-squares solution? — Hàm nào tìm nghiệm bình phương tối thiểu?

A. `np.linalg.solve(A,b)` — `np.linalg.solve(A,b)`

B. `np.linalg.matrix_rank(A)` — `np.linalg.matrix_rank(A)`

C. `np.column_stack((A,b))` — `np.column_stack((A,b))`

D. `np.linalg.lstsq(A,b,rcond=None)` — `np.linalg.lstsq(A,b,rcond=None)`

Đáp án: D

Giải thích: lstsq tối thiểu hóa bình phương độ dài Ax−b.

Q9. For Ax=b with A shape (3,2), what do the rows represent? — Với A shape (3,2), các hàng biểu diễn gì?

A. Three equations or observations — Ba phương trình hoặc quan sát

B. Coefficients of two unknowns — Hệ số của hai ẩn

C. The right-hand-side values — Các giá trị vế phải

D. It cannot have a unique solution for every b — Không thể có nghiệm duy nhất với mọi b

Đáp án: A

Giải thích: Ba hàng tương ứng ba quan sát hoặc phương trình.

Q10. For Ax=b with A shape (3,2), what do the columns represent? — Với A shape (3,2), các cột biểu diễn gì?

A. Three equations or observations — Ba phương trình hoặc quan sát

B. Coefficients of two unknowns — Hệ số của hai ẩn

C. The right-hand-side values — Các giá trị vế phải

D. It cannot have a unique solution for every b — Không thể có nghiệm duy nhất với mọi b

Đáp án: B

Giải thích: Hai cột là hệ số của hai ẩn ở từng phương trình.

Q11. What does the final column of an augmented matrix represent? — Cột cuối của ma trận mở rộng biểu diễn gì?

A. Three equations or observations — Ba phương trình hoặc quan sát

B. Coefficients of two unknowns — Hệ số của hai ẩn

C. The right-hand-side values — Các giá trị vế phải

D. It cannot have a unique solution for every b — Không thể có nghiệm duy nhất với mọi b

Đáp án: C

Giải thích: Ma trận mở rộng ghi các vế phải ở cột cuối.

Q12. What does a rank-deficient square matrix imply? — Ma trận vuông thiếu hạng gợi ý điều gì?

A. Three equations or observations — Ba phương trình hoặc quan sát

B. Coefficients of two unknowns — Hệ số của hai ẩn

C. The right-hand-side values — Các giá trị vế phải

D. It cannot have a unique solution for every b — Không thể có nghiệm duy nhất với mọi b

Đáp án: D

Giải thích: Thiếu hạng đầy đủ khiến không bảo đảm nghiệm duy nhất.

Q13. If the residual is exactly zero, what is true? — Nếu sai số dư bằng 0 thì điều gì đúng?

A. Every equation is satisfied — Mọi phương trình đều được thỏa

B. The sum of squared residuals — Tổng bình phương sai số dư

C. Whether a candidate solution approximately satisfies the system — Nghiệm thử có thỏa hệ gần đúng hay không

D. Ranks of A and its augmented matrix — Hạng của A và ma trận mở rộng

Đáp án: A

Giải thích: Sai số dư bằng 0 nghĩa là Ax=b.

Q14. What does least squares minimize? — Bình phương tối thiểu làm nhỏ nhất gì?

A. Every equation is satisfied — Mọi phương trình đều được thỏa

B. The sum of squared residuals — Tổng bình phương sai số dư

C. Whether a candidate solution approximately satisfies the system — Nghiệm thử có thỏa hệ gần đúng hay không

D. Ranks of A and its augmented matrix — Hạng của A và ma trận mở rộng

Đáp án: B

Giải thích: Bình phương sai lệch làm lỗi lớn bị phạt mạnh.

Q15. What does `np.allclose(A@x,b)` check? — `np.allclose(A@x,b)` kiểm tra gì?

A. Every equation is satisfied — Mọi phương trình đều được thỏa

B. The sum of squared residuals — Tổng bình phương sai số dư

C. Whether a candidate solution approximately satisfies the system — Nghiệm thử có thỏa hệ gần đúng hay không

D. Ranks of A and its augmented matrix — Hạng của A và ma trận mở rộng

Đáp án: C

Giải thích: allclose so kết quả trong ngưỡng sai số số học.

Q16. What should be compared to diagnose inconsistency of a system? — Để xét hệ vô nghiệm cần so sánh gì?

A. Every equation is satisfied — Mọi phương trình đều được thỏa

B. The sum of squared residuals — Tổng bình phương sai số dư

C. Whether a candidate solution approximately satisfies the system — Nghiệm thử có thỏa hệ gần đúng hay không

D. Ranks of A and its augmented matrix — Hạng của A và ma trận mở rộng

Đáp án: D

Giải thích: Hạng A khác hạng [A, b] cho biết hệ vô nghiệm.

Q17. Why isn't the right-hand side b another unknown? — Vì sao b không phải một ẩn mới?

A. It contains known target values — Nó chứa giá trị mục tiêu đã biết

B. They add no new independent direction — Chúng không bổ sung hướng độc lập mới

C. When observations outnumber unknowns or do not fit exactly — Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

D. Prediction is lower than actual — Dự báo thấp hơn thực tế

Đáp án: A

Giải thích: b là dữ liệu đã biết, x mới là biến chưa biết.

Q18. What happens when rows of A are multiples of each other? — Nếu các hàng A là bội nhau thì sao?

A. It contains known target values — Nó chứa giá trị mục tiêu đã biết

B. They add no new independent direction — Chúng không bổ sung hướng độc lập mới

C. When observations outnumber unknowns or do not fit exactly — Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

D. Prediction is lower than actual — Dự báo thấp hơn thực tế

Đáp án: B

Giải thích: Hàng tỷ lệ chỉ lặp thông tin tuyến tính.

Q19. When is `lstsq` useful? — Khi nào `lstsq` hữu ích?

A. It contains known target values — Nó chứa giá trị mục tiêu đã biết

B. They add no new independent direction — Chúng không bổ sung hướng độc lập mới

C. When observations outnumber unknowns or do not fit exactly — Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

D. Prediction is lower than actual — Dự báo thấp hơn thực tế

Đáp án: C

Giải thích: Bình phương tối thiểu tìm cách khớp tốt nhất khi hệ không thỏa hết.

Q20. What does a negative residual mean under r=prediction−actual? — Sai số dư âm theo r=dự báo−thực tế nghĩa là gì?

A. It contains known target values — Nó chứa giá trị mục tiêu đã biết

B. They add no new independent direction — Chúng không bổ sung hướng độc lập mới

C. When observations outnumber unknowns or do not fit exactly — Khi nhiều quan sát hơn ẩn hoặc không khớp chính xác

D. Prediction is lower than actual — Dự báo thấp hơn thực tế

Đáp án: D

Giải thích: Dự báo trừ thực tế là số âm khi dự báo thiếu.

8. Nối sang bài kế tiếp

`Ma trận → Rank/Solve → Regression → AI.`

PY-27 — Eigenvalue, Eigenvector và hướng biến thiên

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Eigenvector — vector riêng; **Eigenvalue** — giá trị riêng.

`Hệ tuyến tính → Eigenvalues/Vectors → Covariance → PCA`

Học xong dùng để làm gì? Tìm các hướng giữ nguyên phương khi ma trận tác động, làm nền để hiểu PCA và biến đổi dữ liệu.

Kết nối kiến thức tiếp theo: PY-28 mô tả quan hệ giữa vector đầu vào và dự đoán bằng hàm toán học.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Giải thích
Eigenvector	vector riêng	Hướng mà phép biến đổi tuyến tính chỉ co/dãn
Eigenvalue	giá trị riêng	Hệ số co/dãn theo một vector riêng
Covariance	hiệp phương sai	Cách hai đặc trưng biến thiên cùng nhau
Principal direction	hướng chính	Hướng chứa nhiều phương sai nhất trong PCA

3. NumPy và các hàm liên quan

Từ vựng trong mục: Function — hàm; **Shape** — kích thước mảng.

Hàm / tham số	Dùng làm gì
`np.linalg.eigh(C)`	Trả eigenvalues/eigenvectors cho ma trận đối xứng
`np.linalg.eigvalsh(C)`	Chỉ trả eigenvalues của ma trận đối xứng
`np.cov(X,rowvar=False)`	Cột là feature, hàng là mẫu khi tính covariance
`np.argsort(values)[::-1]`	Sắp eigenvalue giảm dần để chọn hướng lớn

4. Khái niệm liên quan

Nội dung chính: ma trận biến đổi có thể giữ nguyên phương của một số vector riêng. Khi ma trận là hiệp phương sai, trị riêng đo phương sai theo hướng đó, tạo cầu nối sang PCA.

4.1. Eigenvector — Hướng không bị xoay sau biến đổi

Từ vựng trong mục: Eigenvector — vector riêng; **Transformation** — phép biến đổi.

Một **eigenvector** là vector khác không mà khi nhân với ma trận vuông A, nó chỉ đổi **độ lớn hoặc đảo chiều**, không rời khỏi đường phương ban đầu: `A @ v = λ*v`. Ví dụ ma trận `diag(2,3)` nhân vector `[0,1]` cho `[0,3]`, vẫn nằm trên trục tung. Điều kiện là vector khác không và phù hợp số chiều của A; vector riêng không phải bất kỳ vector nào.

```
import numpy as np

A = np.diag([2.0, 3.0])
v = np.array([0.0, 1.0])
print(A @ v)
```

Kết quả khi chạy:

```
[0. 3.]
```

Đầu ra vẫn có hoành độ 0; phương trục tung được giữ, độ dài theo chiều đó nhân ba.

4.2. Eigenvalue — Hệ số biến đổi trên hướng riêng

Từ vựng trong mục: Eigenvalue — trị riêng; **Scale factor** — hệ số co giãn.

Trị riêng λ cho biết A phóng đại, thu nhỏ hoặc đảo chiều **vector riêng** bao nhiêu lần. Với `diag(2,3)`, hướng trục x có λ=2 và trục y có λ=3. Đó là tính chất của ma trận A; trị riêng **không** tự động mang nghĩa phương sai trừ khi A đang là **ma trận hiệp phương sai**. `np.linalg.eigvalsh` lấy các trị riêng của ma trận đối xứng theo thứ tự tăng dần.

```
import numpy as np

A = np.diag([2.0, 3.0])
print(np.linalg.eigvalsh(A))
```

Kết quả khi chạy:

```
[2. 3.]
```

Hai trị riêng là 2 và 3, tương ứng mức co giãn của hai trục tọa độ.

4.3. Covariance — Hai đặc trưng cùng thay đổi ra sao?

Từ vựng trong mục: Covariance — hiệp phương sai; **Feature** — đặc trưng.

Hiệp phương sai đo xu hướng hai đặc trưng tăng/giảm cùng nhau. Dữ liệu có ba **hàng mẫu** `(1,2)`, `(2,4)`, `(3,6)` và hai **cột đặc trưng** x, y, trong đó y=2x. `np.cov(X,rowvar=False)` lấy **cột là biến**, **hàng là quan sát**, trả ma trận 2×2. Các ô đường chéo là phương sai từng biến, hai ô còn lại là hiệp phương sai. Không nhầm covariance với tương quan chuẩn hóa từ −1 đến 1.

```
import numpy as np

```

```
X = np.array([[1, 2], [2, 4], [3, 6]])
print(np.cov(X, rowvar=False))
```

Kết quả khi chạy:

```
[[1. 2.]
 [2. 4.]]
```

Biến y có phương sai gấp bốn x và hiệp phương sai dương 2, vì y tăng theo x.

4.4. Principal direction — Hướng biến thiên chính trong PCA

Từ vựng trong mục: Principal direction — hướng chính; **PCA** — phân tích thành phần chính.

Với **ma trận hiệp phương sai đối xứng**, vector riêng của **trị riêng lớn nhất** biểu diễn hướng có phương sai lớn nhất. Đây là nguyên lý của PCA. Ví dụ hiệp phương sai `diag(2,3)` có phương sai theo x là 2 và theo y là 3, nên hướng chính là trục y. **Dấu của eigenvector không cố định**: `[0,1]` và `[0,-1]` đều mô tả cùng một trục. Lấy `abs` khi chỉ muốn minh họa độ lớn thành phần.

```
import numpy as np

C = np.diag([2.0, 3.0])
values, vectors = np.linalg.eigh(C)
print(np.abs(vectors[:, -1]))
```

Kết quả khi chạy:

```
[0. 1.]
```

Hướng có phương sai lớn nhất nằm trên trục y; đổi dấu vector riêng không đổi hướng trục.

5. Năm tình huống ứng dụng

Tình huống 1 — Chọn hướng biến thiên lớn nhất

Tình huống thực tế: Một bảng thống kê hai đặc trưng đã được chuẩn hóa ý nghĩa đơn vị và có hiệp phương sai `diag(2,3)`. Cần xác định hướng mang nhiều phương sai hơn.

Vì sao chọn cách giải này? `eigh` dành cho ma trận đối xứng và trả trị riêng tăng dần; trị riêng cuối lớn nhất cùng vector riêng tương ứng.

Quy trình giải:

1. Tạo ma trận hiệp phương sai 2×2.
2. Tìm trị riêng và các vector riêng bằng `eigh`.
3. In trị riêng lớn nhất và độ lớn các thành phần vector riêng để không phụ thuộc dấu.

```
import numpy as np

C = np.diag([2.0, 3.0])
values, vectors = np.linalg.eigh(C)
print(values[-1], np.abs(vectors[:, -1]))
```

Kết quả khi chạy:

```
3.0 [0. 1.]
```

Trục y là hướng chính với phương sai 3, lớn hơn trục x (2).

Tình huống 2 — Phát hiện dữ liệu phụ thuộc tuyến tính

Tình huống thực tế: Ba hàng quan sát có hai đặc trưng lần lượt `(1,2)`, `(2,4)`, `(3,6)`; cột hai luôn gấp đôi cột một. Cần xem ma trận hiệp phương sai có hướng hoàn toàn không biến thiên không.

Vì sao chọn cách giải này? Khi hai cột phụ thuộc tuyến tính, ma trận hiệp phương sai có ít nhất một trị riêng gần 0.

Quy trình giải:

1. Lưu ba hàng quan sát và hai cột đặc trưng.
2. Tính hiệp phương sai theo cột bằng `rowvar=False`.
3. Dùng `eigvalsh` lấy hai trị riêng.

```
import numpy as np

X = np.array([[1, 2], [2, 4], [3, 6]], dtype=float)
print(np.linalg.eigvalsh(np.cov(X, rowvar=False)))
```

Kết quả khi chạy:

```
[0. 5.]
```

Một hướng có phương sai 0 do cột hai phụ thuộc cột đầu; hướng còn lại có phương sai 5.

Tình huống 3 — Kiểm nghiệm phương trình vector riêng

Tình huống thực tế: Cho ma trận A=`diag(2,3)`. Chọn vector riêng ứng với trị riêng lớn nhất và kiểm tra quan hệ `A@v = λ*v`.

Vì sao chọn cách giải này? `np.allclose` đối chiếu hai vế trong sai số số thực, tránh so bằng tuyệt đối.

Quy trình giải:

1. Tìm cặp trị riêng/vector riêng từ A đối xứng.
2. Lấy trị riêng lớn nhất và cột vector tương ứng.
3. So hai vế `A@v` và `λ*v`.

```
import numpy as np

A = np.diag([2.0, 3.0])
values, vectors = np.linalg.eigh(A)
print(np.allclose(A @ vectors[:, -1], values[-1] * vectors[:, -1]))
```

Kết quả khi chạy:

```
True
```

Cặp trị riêng và vector riêng đáp ứng định nghĩa của eigenvector.

Tình huống 4 — Sắp hướng theo độ biến thiên

Tình huống thực tế: Hệ thống thống kê có ba phương sai trên các hướng độc lập lần lượt là `[4,1,3]`. Cần ưu tiên hướng có phương sai cao nhất.

Vì sao chọn cách giải này? `argsort` trả chỉ số theo giá trị, rồi `[::-1]` đảo sang thứ tự giảm dần.

Quy trình giải:

1. Tạo mảng trị riêng `[4,1,3]`.
2. Tính chỉ số sắp tăng, rồi đảo thứ tự.
3. In chỉ số của các hướng từ lớn tới nhỏ.

```
import numpy as np

values = np.array([4, 1, 3])
print(np.argsort(values)[::-1])
```

Kết quả khi chạy:

```
[0 2 1]
```

Ưu tiên hướng ở vị trí 0 (phương sai 4), tiếp đến vị trí 2 (3), cuối vị trí 1 (1).

Tình huống 5 — Giảm số cột đặc trưng bằng PCA

Tình huống thực tế: Ba quan sát hai chiều `(1,2)`, `(2,3)`, `(3,4)` được lưu thành bảng 3 hàng × 2 cột. Cần chiếu dữ liệu xuống một trục biến thiên chính để minh họa giảm chiều.

Vì sao chọn cách giải này? PCA tìm hướng có phương sai lớn; `n_components=1` giữ một trục, không giảm số hàng quan sát.

Quy trình giải:

1. Tạo bảng 3×2, hàng là quan sát và cột là đặc trưng.
2. Khởi tạo `PCA(n_components=1)` từ scikit-learn (thư viện cần cài).
3. Fit/transform và in shape dữ liệu sau chiếu.

```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[1, 2], [2, 3], [3, 4]], dtype=float)
projected = PCA(n_components=1).fit_transform(X)
print(projected.shape)
```

Kết quả khi chạy:

```
(3, 1)
```

Ba mẫu vẫn giữ nguyên; mỗi mẫu còn một giá trị tọa độ trên trục chính.

6. Sai lầm thường gặp

- Eigenvector là **hướng**, eigenvalue là **hệ số** nhân theo hướng ấy; không hoán đổi hai khái niệm.
- Chỉ diễn giải eigenvalue như phương sai khi A là ma trận hiệp phương sai.
- Với covariance, các hàng là mẫu và cột là đặc trưng khi `rowvar=False`; đảo trục làm thay đổi kết quả.
- Eigenvector và vector đối dấu biểu diễn cùng trục; không kiểm tra dấu từng phần tử cứng nhắc.
- `np.linalg.eigh` thích hợp cho ma trận đối xứng. PCA cần xem thang đo và đơn vị từng đặc trưng.
- scikit-learn là thư viện ngoài; cần cài trước khi chạy ví dụ PCA.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Chỉ kiểm tra kiến thức bài học; không kiểm tra ghi nhớ tình huống đánh số.

Q1. What defines an eigenvector v of A? — Điều kiện định nghĩa vector riêng v của A là gì?

A. `A @ v = λ*v` with v nonzero — `A @ v = λ*v` với v khác không

B. Scaling along a corresponding eigenvector — Hệ số co giãn theo vector riêng tương ứng

C. How two variables vary together — Hai biến cùng biến thiên ra sao

D. A covariance eigenvector with high eigenvalue — Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: A

Giải thích: Vector khác không giữ phương đến hệ số nhân.

Q2. What does an eigenvalue represent? — Trị riêng biểu thị gì?

A. `A @ v = λ*v` with v nonzero — `A @ v = λ*v` với v khác không

B. Scaling along a corresponding eigenvector — Hệ số co giãn theo vector riêng tương ứng

C. How two variables vary together — Hai biến cùng biến thiên ra sao

D. A covariance eigenvector with high eigenvalue — Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: B

Giải thích: Trị riêng là hệ số λ nhân vector riêng.

Q3. What does covariance describe? — Hiệp phương sai mô tả gì?

A. `A @ v = λ*v` with v nonzero — `A @ v = λ*v` với v khác không

B. Scaling along a corresponding eigenvector — Hệ số co giãn theo vector riêng tương ứng

C. How two variables vary together — Hai biến cùng biến thiên ra sao

D. A covariance eigenvector with high eigenvalue — Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: C

Giải thích: Hiệp phương sai đo sự biến thiên cùng nhau.

Q4. What is a principal direction in PCA? — Hướng chính trong PCA là gì?

A. `A @ v = λ*v` with v nonzero — `A @ v = λ*v` với v khác không

B. Scaling along a corresponding eigenvector — Hệ số co giãn theo vector riêng tương ứng

C. How two variables vary together — Hai biến cùng biến thiên ra sao

D. A covariance eigenvector with high eigenvalue — Vector riêng hiệp phương sai có trị riêng lớn

Đáp án: D

Giải thích: Vector riêng hiệp phương sai nổi bật mang phương sai lớn nhất.

Q5. Which function suits a symmetric matrix eigendecomposition? — Hàm nào phù hợp phân rã trị riêng ma trận đối xứng?

A. `np.linalg.eigh(C)` — `np.linalg.eigh(C)`

B. `np.linalg.eigvalsh(C)` — `np.linalg.eigvalsh(C)`

C. `np.cov(X,rowvar=False)` — `np.cov(X,rowvar=False)`

D. `np.argsort(values)[::-1]` — `np.argsort(values)[::-1]`

Đáp án: A

Giải thích: eigh được thiết kế cho ma trận đối xứng/Hermitian.

Q6. Which function returns only symmetric eigenvalues? — Hàm nào chỉ trả trị riêng ma trận đối xứng?

A. `np.linalg.eigh(C)` — `np.linalg.eigh(C)`

B. `np.linalg.eigvalsh(C)` — `np.linalg.eigvalsh(C)`

C. `np.cov(X,rowvar=False)` — `np.cov(X,rowvar=False)`

D. `np.argsort(values)[::-1]` — `np.argsort(values)[::-1]`

Đáp án: B

Giải thích: eigvalsh không trả vector riêng.

Q7. Which call treats DataFrame-like columns as variables? — Lệnh nào coi cột là biến khi tính covariance?

A. `np.linalg.eigh(C)` — `np.linalg.eigh(C)`

B. `np.linalg.eigvalsh(C)` — `np.linalg.eigvalsh(C)`

C. `np.cov(X,rowvar=False)` — `np.cov(X,rowvar=False)`

D. `np.argsort(values)[::-1]` — `np.argsort(values)[::-1]`

Đáp án: C

Giải thích: rowvar=False coi cột là biến.

Q8. Which expression sorts eigenvalue indices descending? — Biểu thức nào sắp chỉ số trị riêng giảm dần?

A. `np.linalg.eigh(C)` — `np.linalg.eigh(C)`

B. `np.linalg.eigvalsh(C)` — `np.linalg.eigvalsh(C)`

C. `np.cov(X,rowvar=False)` — `np.cov(X,rowvar=False)`

D. `np.argsort(values)[::-1]` — `np.argsort(values)[::-1]`

Đáp án: D

Giải thích: argsort cho chỉ số tăng; đảo để được giảm.

Q9. What is the shape of a covariance matrix for three features? — Ma trận covariance cho ba đặc trưng có shape gì?

A. `(3,3)` — `(3,3)`

B. Columns of the second returned array — Các cột của mảng trả về thứ hai

C. v and −v describe the same eigendirection — v và −v biểu diễn cùng trục riêng

D. No variance along that direction — Không biến thiên theo hướng đó

Đáp án: A

Giải thích: Mỗi đặc trưng ghép với từng đặc trưng.

Q10. Which object contains eigenvectors returned by `eigh`? — Trong kết quả của `eigh`, vector riêng nằm đâu?

A. `(3,3)` — `(3,3)`

B. Columns of the second returned array — Các cột của mảng trả về thứ hai

C. v and −v describe the same eigendirection — v và −v biểu diễn cùng trục riêng

D. No variance along that direction — Không biến thiên theo hướng đó

Đáp án: B

Giải thích: eigh lưu mỗi vector riêng ở một cột.

Q11. Why can an eigenvector's sign change? — Vì sao dấu vector riêng có thể đảo?

A. `(3,3)` — `(3,3)`

B. Columns of the second returned array — Các cột của mảng trả về thứ hai

C. v and −v describe the same eigendirection — v và −v biểu diễn cùng trục riêng

D. No variance along that direction — Không biến thiên theo hướng đó

Đáp án: C

Giải thích: Đảo dấu vector riêng vẫn thỏa phương trình.

Q12. What does a zero covariance eigenvalue suggest? — Trị riêng covariance bằng 0 gợi ý gì?

A. `(3,3) — (3,3)`

B. Columns of the second returned array — Các cột của mảng trả về thứ hai

C. v and −v describe the same eigendirection — v và −v biểu diễn cùng trục riêng

D. No variance along that direction — Không biến thiên theo hướng đó

Đáp án: D

Giải thích: Trị riêng 0 nghĩa là không có phương sai theo trục đó.

Q13. How does PCA choose its first component? — PCA chọn thành phần đầu ra sao?

A. By largest explained variance — Theo phương sai giải thích lớn nhất

B. Variance of one feature — Phương sai của một đặc trưng

C. Two — Hai

D. `A @ v` approximately equals `λ*v` — `A @ v` xấp xỉ `λ*v`

Đáp án: A

Giải thích: Thành phần đầu tương ứng phương sai cao nhất.

Q14. What does a covariance diagonal entry represent? — Ô đường chéo covariance biểu thị gì?

A. By largest explained variance — Theo phương sai giải thích lớn nhất

B. Variance of one feature — Phương sai của một đặc trưng

C. Two — Hai

D. `A @ v` approximately equals `λ*v` — `A @ v` xấp xỉ `λ*v`

Đáp án: B

Giải thích: Hiệp phương sai của biến với chính nó là phương sai.

Q15. How many eigenvalues does a 2×2 symmetric matrix have counting multiplicity? — Ma trận đối xứng 2×2 có bao nhiêu trị riêng tính cả bội?

A. By largest explained variance — Theo phương sai giải thích lớn nhất

B. Variance of one feature — Phương sai của một đặc trưng

C. Two — Hai

D. `A @ v` approximately equals `λ*v` — `A @ v` xấp xỉ `λ*v`

Đáp án: C

Giải thích: Ma trận 2×2 có hai trị riêng tính bội.

Q16. Which relation validates an eigenpair? — Quan hệ nào dùng kiểm tra cặp riêng?

A. By largest explained variance — Theo phương sai giải thích lớn nhất

B. Variance of one feature — Phương sai của một đặc trưng

C. Two — Hai

D. `A @ v` approximately equals λ*v — `A @ v` xấp xỉ λ*v

Đáp án: D

Giải thích: Cặp riêng thỏa Av=λv theo sai số số học.

Q17. Which data layout corresponds to `rowvar=False`? — `rowvar=False` tương ứng bố trí nào?

A. Rows are samples and columns are features — Hàng là mẫu và cột là đặc trưng

B. Only when the matrix is an appropriate covariance matrix — Chỉ khi ma trận là covariance phù hợp

C. One principal-component feature per sample — Một đặc trưng thành phần chính cho mỗi mẫu

D. Feature scales and units — Thang đo và đơn vị các đặc trưng

Đáp án: A

Giải thích: Quan sát nằm theo hàng khi rowvar=False.

Q18. Does an eigenvalue always mean variance? — Trị riêng có luôn mang nghĩa phương sai không?

A. Rows are samples and columns are features — Hàng là mẫu và cột là đặc trưng

B. Only when the matrix is an appropriate covariance matrix — Chỉ khi ma trận là covariance phù hợp

C. One principal-component feature per sample — Một đặc trưng thành phần chính cho mỗi mẫu

D. Feature scales and units — Thang đo và đơn vị các đặc trưng

Đáp án: B

Giải thích: Diễn giải phương sai cần ma trận hiệp phương sai.

Q19. What does `PCA(n_components=1)` preserve? — `PCA(n_components=1)` giữ lại gì?

A. Rows are samples and columns are features — Hàng là mẫu và cột là đặc trưng

B. Only when the matrix is an appropriate covariance matrix — Chỉ khi ma trận là covariance phù hợp

C. One principal-component feature per sample — Một đặc trưng thành phần chính cho mỗi mẫu

D. Feature scales and units — Thang đo và đơn vị các đặc trưng

Đáp án: C

Giải thích: Dữ liệu sau chiếu còn một cột, vẫn giữ số hàng.

Q20. What should be considered before comparing PCA variances across features? — Cần lưu ý gì trước khi so phương sai PCA giữa đặc trưng?

A. Rows are samples and columns are features — Hàng là mẫu và cột là đặc trưng

B. Only when the matrix is an appropriate covariance matrix — Chỉ khi ma trận là covariance phù hợp

C. One principal-component feature per sample — Một đặc trưng thành phần chính cho mỗi mẫu

D. Feature scales and units — Thang đo và đơn vị các đặc trưng

Đáp án: D

Giải thích: Đặc trưng có thang lớn có thể lấn át PCA nếu chưa chuẩn hóa.

8. Nối sang bài kế tiếp

Hệ tuyến tính → Eigenvalues/Vectors → Covariance → PCA.

PY-28 — Hàm tuyến tính, bậc hai, đa thức và hàm mũ

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Linear function — hàm tuyến tính; **Quadratic** — hàm bậc hai.

`Hàm Python → Hàm toán theo dữ liệu → Loss → Gradient`

Học xong dùng để làm gì? Mô tả dự đoán bằng hàm f(x), nhận biết đầu vào, đầu ra và ảnh hưởng của tham số.

Kết nối kiến thức tiếp theo: PY-29 bổ sung hàm kích hoạt để biểu diễn quan hệ phi tuyến.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Linear function	hàm tuyến tính	Đường thẳng có tốc độ thay đổi hằng số
Quadratic	hàm bậc hai	Hàm chứa x^2, đồ thị là parabol khi hệ số bậc hai khác 0
Polynomial	hàm đa thức	Tổng các lũy thừa nguyên không âm của x
Exponential	hàm mũ	Biến nằm trong số mũ, thường mô tả tăng/giảm theo tỷ lệ

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`y=m*x+b`	Quan hệ tuyến tính, m là độ dốc, b là hệ số chặn
`y=a*x**2+b*x+c`	Quan hệ bậc hai, dùng khi đường cong
`np.polyval(coefficients,x)`	Đánh giá đa thức theo hệ số cao xuống thấp
`np.exp(x)/base**x`	Hàm mũ; cẩn thận overflow và miền thực

4. Khái niệm liên quan

Nội dung chính: đầu vào x được chuyển thành đầu ra y theo một quy tắc. Hàm bậc nhất tạo đường thẳng với độ dốc không đổi; hàm bậc hai và đa thức tạo đường cong; hàm mũ nhân theo tỷ lệ sau mỗi kỳ. Mỗi ví dụ nói rõ biến, hệ số và đơn vị.

4.1. Linear function — Mỗi bước tăng một mức cố định

Từ vựng trong mục: Linear function — hàm bậc nhất; **Slope/intercept** — hệ số góc/hệ số chặn.

Trong bài, **hàm tuyến tính dạng affine** `y=m*x+b` mô tả đầu ra thay đổi đều theo đầu vào. Dịch vụ thu 10 nghìn đồng ban đầu và 5 nghìn mỗi km: x là số km, m=5 nghìn/km, b=10 nghìn. Đi 3 km hết 25 nghìn đồng. Đồ thị là đường thẳng; m nói **mỗi km thêm bao nhiêu**, b nói **giá ở x=0**. Trong toán tuyến tính thuần túy, `mx+b` với b khác 0 gọi chính xác là hàm affine.

```
def delivery_fee(km):
    return 5 * km + 10

print(delivery_fee(3))
```

Kết quả khi chạy:

```
25
```

Phí là 25 nghìn đồng, gồm phí gốc 10 và 3×5 nghìn theo khoảng cách.

4.2. Quadratic — Mức thay đổi không cố định

Từ vựng trong mục: Quadratic — bậc hai; **Coefficient** — hệ số.

Hàm bậc hai $y=ax^2+bx+c$ (a khác 0) có đồ thị **parabol**, khác đường thẳng vì tốc độ thay đổi không cố định. Ví dụ chi phí mẫu $2x^2 + 3x + 10$ với x là mức tải (đơn vị tải đã quy định), 2 có đơn vị chi phí/tải², 3 là chi phí/tải và 10 là chi phí cố định. Khi x=4, giá trị bằng 54. Không hiểu hệ số bình phương là chi phí thực nếu mô hình mới chỉ là giả định.

```
def cost(load):
    return 2 * load ** 2 + 3 * load + 10

print(cost(4))
```

Kết quả khi chạy:

```
54
```

Với tải 4, mô hình cho chi phí 54 đơn vị tiền; đây là kết quả theo hàm đã đặt.

4.3. Polynomial — Nhiều lũy thừa của một biến

Từ vựng trong mục: Polynomial — đa thức; **Degree** — bậc đa thức.

Đa thức cộng các hạng `a_n*x^n` theo lũy thừa nguyên không âm. Tuyến tính là bậc 1, bậc hai là bậc 2, và còn có đa thức bậc cao hơn. `np.polyval([1,0,2], x)` biểu diễn $x^2 + 0*x + 2$; **hệ số đặt từ bậc cao xuống bậc thấp**, rất dễ viết ngược. Với ba đầu vào 0,1,2, hàm cho kết quả 2,3,6.

```
import numpy as np

x = np.array([0, 1, 2])
print(np.polyval([1, 0, 2], x))
```

Kết quả khi chạy:

```
[2 3 6]
```

Hệ số `[1,0,2]` nghĩa là x^2+2, không phải $1+2x^2$.

4.4. Exponential — Nhân tỷ lệ lặp qua mỗi kỳ

Từ vựng trong mục: Exponential — hàm mũ; **Growth rate** — tỷ lệ tăng.

Hàm mũ có biến ở **số mũ**, ví dụ `y=y0*(1+r)^t`. Với một chỉ số ban đầu 1.000 đơn vị và tỷ lệ tăng 5% mỗi kỳ, sau 3 kỳ là $1000 \times 1.05^3 = 1157.625$. Làm tròn hai chữ số được 1157.63. Hàm mũ khác tuyến tính vì mỗi kỳ tăng 5% của **giá trị mới**, không phải luôn tăng 50 đơn vị. Mô hình này chỉ mô phỏng một giả định tăng trưởng đều.

```
initial = 1000
rate = 0.05
periods = 3
print(round(initial * (1 + rate) ** periods, 2))
```

Kết quả khi chạy:

```
1157.63
```

Sau ba kỳ tăng trưởng lũy kế, chỉ số khoảng 1157,63 đơn vị, không phải giá trị dự báo chắc chắn.

5. Năm tình huống ứng dụng

Tình huống 1 — Tính phí giao hàng theo km

Tình huống thực tế: Dịch vụ tính 10.000 đồng cố định và 5.000 đồng/km. Một khách đặt giao 3 km. Hãy tính tổng phí đồng với mô hình tăng đều theo km.

Vì sao chọn cách giải này? Phí tăng đúng một mức cố định theo mỗi km nên dùng hàm dạng `base+rate*km`.

Quy trình giải:

1. Đặt `km=3`, phí gốc 10.000 và 5.000 đồng/km.
2. Nhân số km với đơn giá rồi cộng phí gốc.
3. In tổng phí tính theo đồng.

```
km = 3
fee = 10000 + 5000 * km
print(fee)
```

Kết quả khi chạy:

```
25000
```

Tổng phí 25.000 đồng gồm 10.000 phí gốc và 15.000 theo km.

Tình huống 2 — Tính chi phí theo tải có thành phần bậc hai

Tình huống thực tế: Mô hình giả định chi phí là $2x^2+3x+10$, đơn vị tiền đã chọn. Thiết bị đang có tải x=4. Tính chi phí theo công thức.

Vì sao chọn cách giải này? Hàm bậc hai mô tả phần chi phí có độ tăng lớn dần khi mức tải tăng.

Quy trình giải:

1. Nhận mức tải 4.
2. Thay x=4 vào biểu thức gồm hạng bậc hai, tuyến tính và chi phí cố định.
3. In giá trị 54 đơn vị tiền.

```
load = 4
cost = 2 * load ** 2 + 3 * load + 10
print(cost)
```

Kết quả khi chạy:

```
54
```

Chi phí theo mô hình đã cho bằng 54 đơn vị tiền.

Tình huống 3 — Tính đầu ra đa thức trên nhiều điểm

Tình huống thực tế: Mô hình minh họa $y=x^2+2$ cần đánh giá ở đầu vào x=0,1,2. In các kết quả theo thứ tự đầu vào.

Vì sao chọn cách giải này? `np.polyval` áp hệ số một lần cho toàn mảng và không cần tự viết vòng lặp tính x^2+2.

Quy trình giải:

1. Tạo mảng `[0,1,2]`.
2. Truyền hệ số đa thức từ bậc cao tới thấp `[1,0,2]`.
3. In ba kết quả cùng thứ tự.

```
import numpy as np

x = np.array([0, 1, 2])
print(np.polyval([1, 0, 2], x))
```

Kết quả khi chạy:

```
[2 3 6]
```

Đầu ra ở x=0,1,2 lần lượt là 2, 3, 6.

Tình huống 4 — Mô phỏng tăng trưởng theo kỳ

Tình huống thực tế: Một chỉ số có giá trị ban đầu 1.000 đơn vị, được giả định tăng 5% mỗi kỳ, kéo dài ba kỳ liên tiếp. Hãy tính kết quả cuối.

Vì sao chọn cách giải này? Lũy thừa mô tả tăng dựa trên giá trị của kỳ trước, khác cộng một lượng cố định.

Quy trình giải:

1. Gán chỉ số ban đầu 1000, tỷ lệ 0.05 và số kỳ 3.
2. Tính `initial*(1+rate)**periods`.
3. Làm tròn hai chữ số và in kết quả.

```
initial = 1000
rate = 0.05
periods = 3
print(round(initial * (1 + rate) ** periods, 2))
```

Kết quả khi chạy:

```
1157.63
```

Giá trị sau ba kỳ là khoảng 1157.63 đơn vị theo giả định tăng trưởng.

Tình huống 5 — Tạo các điểm của đường dự đoán

Tình huống thực tế: Đội phân tích muốn kiểm tra đường y=2x+5 tại sáu giá trị x cách đều từ 0 đến 10. Cần liệt kê từng cặp (x,y) để dễ vẽ sau này.

Vì sao chọn cách giải này? `np.linspace` tạo mốc cách đều; NumPy tính y cho toàn bộ mốc và `zip` ghép đúng từng cặp.

Quy trình giải:

1. Tạo sáu mốc x từ 0 đến 10 bằng `linspace`.
2. Tính y=2x+5 cho từng mốc.
3. Ghép các cặp và in dạng số nguyên (ở đây mọi giá trị đều nguyên).

```
import numpy as np

x = np.linspace(0, 10, 6)
y = 2 * x + 5
print([(int(a), int(b)) for a, b in zip(x, y)])
```

Kết quả khi chạy:

```
[(0, 5), (2, 9), (4, 13), (6, 17), (8, 21), (10, 25)]
```

Mỗi cặp gồm đầu vào x và đầu ra y tương ứng; đường tăng đều theo x.

6. Sai lầm thường gặp

- Hàm affine `mx+b` có hệ số chặn; không phải mọi biểu thức có chữ x đều là hàm tuyến tính theo nghĩa toán học chặt chẽ.
- Không nhầm bậc hai với hàm mũ: bậc hai có x ở cơ số rồi bình phương, hàm mũ có x ở số mũ.
- `np.polyval` đọc hệ số từ **bậc cao xuống thấp**.

- Độ dốc và hệ số chặn cần đơn vị tương thích với x và y.
- Tăng trưởng mũ giả định tỷ lệ ổn định, không bảo đảm dự báo đúng trong thực tế.
- Đồ thị đẹp hoặc khớp vài điểm chưa chứng minh mô hình dự báo tốt trên dữ liệu mới.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Câu hỏi chỉ về nội dung lý thuyết, cú pháp và cách phân biệt hàm.

Q1. Which form is a first-degree affine function? — Dạng nào là hàm affine bậc nhất?

A. `y=m*x+b` — `y=m*x+b`

B. `y=a*x**2+b*x+c` with $a \neq 0$ — `y=a*x**2+b*x+c` với $a \neq 0$

C. Highest nonzero power of x — Lũy thừa cao nhất có hệ số khác 0

D. A variable appears in the exponent — Biến xuất hiện ở số mũ

Đáp án: A

Giải thích: Dạng affine gồm hệ số góc và hệ số chặn.

Q2. Which form contains a nonzero quadratic term? — Dạng nào có hạng bậc hai khác 0?

A. `y=m*x+b` — `y=m*x+b`

B. `y=a*x**2+b*x+c` with $a \neq 0$ — `y=a*x**2+b*x+c` với $a \neq 0$

C. Highest nonzero power of x — Lũy thừa cao nhất có hệ số khác 0

D. A variable appears in the exponent — Biến xuất hiện ở số mũ

Đáp án: B

Giải thích: Hàm bậc hai có hạng x bình phương.

Q3. What does polynomial degree measure? — Bậc đa thức cho biết gì?

A. `y=m*x+b` — `y=m*x+b`

B. `y=a*x**2+b*x+c` with $a \neq 0$ — `y=a*x**2+b*x+c` với $a \neq 0$

C. Highest nonzero power of x — Lũy thừa cao nhất có hệ số khác 0

D. A variable appears in the exponent — Biến xuất hiện ở số mũ

Đáp án: C

Giải thích: Bậc là số mũ cao nhất có hệ số khác 0.

Q4. What characterizes an exponential function? — Đặc điểm của hàm mũ là gì?

A. `y=m*x+b` — `y=m*x+b`

B. `y=a*x**2+b*x+c` with $a \neq 0$ — `y=a*x**2+b*x+c` với $a \neq 0$

C. Highest nonzero power of x — Lũy thừa cao nhất có hệ số khác 0

D. A variable appears in the exponent — Biến xuất hiện ở số mũ

Đáp án: D

Giải thích: Hàm mũ có số mũ thay đổi.

Q5. What does slope m represent in y=mx+b? — Hệ số góc m trong y=mx+b biểu thị gì?

A. Output change per one input unit — Mức thay đổi đầu ra mỗi đơn vị đầu vào

B. Predicted output at x=0 — Đầu ra dự tính khi x=0

C. Parabola — Parabol

D. It compounds relative changes — Nó cộng dồn mức tăng theo tỷ lệ

Đáp án: A

Giải thích: Hệ số góc là mức đổi y mỗi đơn vị x.

Q6. What does intercept b represent? — Hệ số chặn b biểu thị gì?

A. Output change per one input unit — Mức thay đổi đầu ra mỗi đơn vị đầu vào

B. Predicted output at x=0 — Đầu ra dự tính khi x=0

C. Parabola — Parabol

D. It compounds relative changes — Nó cộng dồn mức tăng theo tỷ lệ

Đáp án: B

Giải thích: Đặt x=0 còn hệ số chặn.

Q7. What shape does a quadratic graph typically have? — Đồ thị hàm bậc hai thường có dạng gì?

A. Output change per one input unit — Mức thay đổi đầu ra mỗi đơn vị đầu vào

B. Predicted output at x=0 — Đầu ra dự tính khi x=0

C. Parabola — Parabol

D. It compounds relative changes — Nó cộng dồn mức tăng theo tỷ lệ

Đáp án: C

Giải thích: Hạng bậc hai khác 0 tạo đồ thị parabol.

Q8. How does exponential growth differ from linear growth? — Tăng trưởng mũ khác tăng tuyến tính ở đâu?

A. Output change per one input unit — Mức thay đổi đầu ra mỗi đơn vị đầu vào

B. Predicted output at x=0 — Đầu ra dự tính khi x=0

C. Parabola — Parabol

D. It compounds relative changes — Nó cộng dồn mức tăng theo tỷ lệ

Đáp án: D

Giải thích: Lãi kép nhân theo tỷ lệ mỗi kỳ.

Q9. Which NumPy function evaluates a polynomial? — Hàm NumPy nào tính đa thức?

A. `np.polyval(coefficients,x)` — `np.polyval(coefficients,x)`

B. Highest degree first — Bậc cao nhất trước

C. `np.exp(x)` — `np.exp(x)`

D. `np.linspace(start,stop,num)` — `np.linspace(start,stop,num)`

Đáp án: A

Giải thích: polyval tính đa thức theo các hệ số có thứ tự.

Q10. In what order does `np.polyval` take coefficients? — `np.polyval` nhận hệ số theo thứ tự gì?

A. `np.polyval(coefficients,x)` — `np.polyval(coefficients,x)`

B. Highest degree first — Bậc cao nhất trước

C. `np.exp(x)` — `np.exp(x)`

D. `np.linspace(start,stop,num)` — `np.linspace(start,stop,num)`

Đáp án: B

Giải thích: Hệ số bậc cao nhất đứng đầu.

Q11. Which function computes e raised to x? — Hàm nào tính e mũ x?

A. `np.polyval(coefficients,x)` — `np.polyval(coefficients,x)`

B. Highest degree first — Bậc cao nhất trước

C. `np.exp(x)` — `np.exp(x)`

D. `np.linspace(start,stop,num)` — `np.linspace(start,stop,num)`

Đáp án: C

Giải thích: exp tính hàm mũ cơ số e.

Q12. Which function makes evenly spaced samples? — Hàm nào tạo các điểm cách đều?

A. `np.polyval(coefficients,x)` — `np.polyval(coefficients,x)`

B. Highest degree first — Bậc cao nhất trước

C. `np.exp(x)` — `np.exp(x)`

D. `np.linspace(start,stop,num)` — `np.linspace(start,stop,num)`

Đáp án: D

Giải thích: linspace tạo số lượng điểm cách đều yêu cầu.

Q13. What is the value of y=2x+5 at x=0? — Giá trị y=2x+5 khi x=0 là gì?

A. 5 — 5

B. 6 — 6

C. 16 — 16

D. 4 — 4

Đáp án: A

Giải thích: Khi x=0, ta có 2×0+5=5.

Q14. What is the value of x^2+2 at x=2? — Giá trị x^2+2 khi x=2 là gì?

A. 5 — 5

B. 6 — 6

C. 16 — 16

D. 4 — 4

Đáp án: B

Giải thích: Bình phương 2 rồi cộng 2 bằng 6.

Q15. What is 2 raised to the fourth power? — 2 mũ 4 bằng gì?

A. 5 — 5

B. 6 — 6

C. 16 — 16

D. 4 — 4

Đáp án: C

Giải thích: 2 nhân bốn lần bằng 16.

Q16. How many intervals are between five evenly spaced points? — Năm điểm cách đều tạo bao nhiêu khoảng?

A. 5 — 5

B. 6 — 6

C. 16 — 16

D. 4 — 4

Đáp án: D

Giải thích: n điểm tạo n−1 khoảng.

Q17. What does `np.exp(0)` equal? — `np.exp(0)` bằng bao nhiêu?

A. 1 — 1

B. Its slope changes with x — Độ dốc thay đổi theo x

C. A constant additive increment — Mỗi bước cộng cùng một mức

D. Accuracy on unseen real-world data — Độ chính xác trên dữ liệu thực chưa gặp

Đáp án: A

Giải thích: Cơ số khác 0 mũ 0 bằng 1.

Q18. How does a quadratic differ from a first-degree affine function? — Hàm bậc hai khác hàm affine bậc nhất ở đâu?

A. 1 — 1

B. Its slope changes with x — Độ dốc thay đổi theo x

C. A constant additive increment — Mỗi bước cộng cùng một mức

D. Accuracy on unseen real-world data — Độ chính xác trên dữ liệu thực chưa gặp

Đáp án: B

Giải thích: Mức tăng của hàm bậc hai thay đổi theo x.

Q19. Which constraint gives a constant-per-step growth model? — Mô hình tăng đều mỗi bước cần điều kiện gì?

A. 1 — 1

B. Its slope changes with x — Độ dốc thay đổi theo x

C. A constant additive increment — Mỗi bước cộng cùng một mức

D. Accuracy on unseen real-world data — Độ chính xác trên dữ liệu thực chưa gặp

Đáp án: C

Giải thích: Cộng hằng số tạo mức tăng cố định.

Q20. What does a fitted function alone not prove? — Một hàm được khớp dữ liệu chưa chứng minh được gì?

A. 1 — 1

B. Its slope changes with x — Độ dốc thay đổi theo x

C. A constant additive increment — Mỗi bước cộng cùng một mức

D. Accuracy on unseen real-world data — Độ chính xác trên dữ liệu thực chưa gặp

Đáp án: D

Giải thích: Khả năng dự đoán cần được kiểm chứng ngoài các ví dụ khớp.

8. Bài tiếp theo

`Hàm Python → Hàm toán theo dữ liệu → Loss → Gradient.`

PY-29 — Activation: ReLU, sigmoid, tanh, softmax

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: ReLU — hàm chỉnh lưu; **Sigmoid** — hàm sigmoid.

`Hàm số → Activation → Neural Network → Classification`

Học xong dùng để làm gì? Hiểu ReLU/sigmoid biến đổi tín hiệu và giúp mô hình vượt giới hạn của lớp tuyến tính thuần túy.

Kết nối kiến thức tiếp theo: PY-30 dùng loss để đo dự đoán sai thực tế bao nhiêu.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
ReLU	hàm chỉnh lưu	Giữ số dương, số âm thành 0
Sigmoid	hàm sigmoid	Ánh xạ số thực về khoảng (0,1)
Tanh	hàm tanh	Ánh xạ số thực về (-1,1)
Softmax	hàm softmax	Biến logits thành phân phối xác suất tổng bằng 1

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`np.maximum(0,x)`	ReLU, tạo phi tuyến cho mạng
`1/(1+np.exp(-x))`	Sigmoid, cẩn thận số quá lớn
`np.tanh(x)`	Kích hoạt có đầu ra âm/dương
`np.exp(z-max(z))/sum(... )`	Softmax ổn định hơn tính exp(z) trực tiếp

4. Khái niệm liên quan

Nội dung chính: một mạng nơ-ron áp hàm kích hoạt lên điểm thô để tạo quan hệ phi tuyến. ReLU chặn phần âm, sigmoid nén về (0,1), tanh giữ dấu trong (−1,1), còn softmax ràng buộc nhiều lớp thành phân phối có tổng 1.

4.1. ReLU — Giữ phần dương, chặn phần âm

Từ vựng trong mục: ReLU — Rectified Linear Unit; **Activation** — kích hoạt.

Activation function (hàm kích hoạt) biến đổi đầu ra của một nơ-ron trước khi chuyển sang bước kế tiếp. **ReLU** tính `max(0,x)`: đầu vào âm được chuyển về 0, đầu vào dương được giữ. Ví dụ các tín hiệu `[-2,0,3]` thành `[0,0,3]`. Đây là hàm gấp khúc tại 0, giúp mạng có quan hệ phi tuyến; song nơ-ron có thể ngừng cập nhật nếu liên tục nhận vùng âm.

```
import numpy as np

signals = np.array([-2, 0, 3])
print(np.maximum(0, signals).tolist())
```

Kết quả khi chạy:

```
[0, 0, 3]
```

ReLU không giữ phần âm; đầu vào dương bằng 3 đi qua không đổi.

4.2. Sigmoid — Ánh xạ một điểm thô sang (0,1)

Từ vựng trong mục: Sigmoid — hàm chữ S; **Logit** — điểm thô.

Sigmoid là `1/(1+exp(-x))`, đổi một **logit** (điểm thô có thể âm hoặc dương) thành số trong khoảng **(0,1)**. Ở x=0 đầu ra đúng 0.5; x âm dưới 0.5 và x dương trên 0.5. Trong phân loại nhị phân, thường dùng sigmoid cho lớp dương, nhưng bản thân giá trị đầu ra **không bảo đảm là xác suất đã hiệu chuẩn**. Khi x quá lớn về độ lớn, cần triển khai số học ổn định hơn công thức trực tiếp.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
scores = 1 / (1 + np.exp(-x))
print(np.round(scores, 3).tolist())
```

Kết quả khi chạy:

```
[0.269, 0.5, 0.731]
```

Điểm âm được ánh xạ dưới 0.5, điểm 0 thành 0.5 và điểm dương trên 0.5.

4.3. Tanh — Giữ dấu âm và dương quanh zero

Từ vựng trong mục: Tanh — tang hyperbolic; **Zero-centered** — cân quanh 0.

Tanh cũng cong chữ S nhưng đầu ra nằm trong **(−1,1)** thay vì (0,1). Đầu vào âm cho đầu ra âm, đầu vào 0 cho 0, đầu vào dương cho dương. Vì cân quanh 0, tanh có thể thích hợp khi cần giữ dấu tín hiệu. Với x=−1,0,1 kết quả khoảng −0.762, 0, 0.762. Khi |x| rất lớn, tanh bão hòa gần ±1 nên độ dốc nhỏ.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
print(np.round(np.tanh(x), 3).tolist())
```

Kết quả khi chạy:

```
[-0.762, 0.0, 0.762]
```

Hàm giữ dấu âm/dương và bằng 0 tại đầu vào 0.

4.4. Softmax — So sánh nhiều lớp cùng lúc

Từ vựng trong mục: Softmax — chuẩn hóa điểm nhiều lớp; **Class logit** — điểm thô theo lớp.

Khi mô hình phân loại **một nhãn trong ba lớp loại trừ nhau**, nó có ba logit (điểm thô) `z=[2,1,0.1]`. Softmax dùng `exp(z_i)/sum(exp(z_j))` để mỗi lớp nhận giá trị trong (0,1) và **tổng các lớp bằng 1**. Trừ `max(z)` trước khi tính exp giúp giảm nguy cơ tràn số mà không đổi kết quả. Khác sigmoid áp riêng từng giá trị, softmax ràng buộc các lớp cạnh tranh nhau.

```
import numpy as np

z = np.array([2.0, 1.0, 0.1])
e = np.exp(z - z.max())
p = e / e.sum()
print(np.round(p, 3).tolist())
```

Kết quả khi chạy:

```
[0.659, 0.242, 0.099]
```

Lớp đầu có điểm chuẩn hóa cao nhất; tổng ba điểm trước làm tròn là 1.

5. Năm tình huống ứng dụng

Tình huống 1 — Chặn tín hiệu âm bằng ReLU

Tình huống thực tế: Một lớp mạng nhận bốn tín hiệu đầu vào `[-2,0,3,5]`, không có đơn vị vì đây là logit nội bộ. Cần giữ giá trị dương và cho mọi giá trị âm về 0.

Vì sao chọn cách giải này? ReLU đúng với quy tắc xử lý `max(0,x)` mà không cần từng điều kiện `if`.

Quy trình giải:

1. Lưu bốn đầu vào trong ndarray.
2. Áp `np.maximum(0,x)` đồng thời lên mọi phần tử.
3. In các giá trị sau kích hoạt.

```
import numpy as np

x = np.array([-2, 0, 3, 5])
print(np.maximum(0, x).tolist())
```

Kết quả khi chạy:

```
[0, 0, 3, 5]
```

Hai giá trị âm/0 cho đầu ra 0, hai giá trị dương được giữ.

Tình huống 2 — Chuyển ba logit bằng sigmoid

Tình huống thực tế: Mô hình phân loại nhị phân có ba điểm thô `[-1,0,1]` của ba quan sát. Cần biến đổi điểm về khoảng (0,1) để so sánh.

Vì sao chọn cách giải này? Sigmoid tác động lên từng điểm riêng và cho 0.5 khi logit bằng 0.

Quy trình giải:

1. Lưu ba logit dạng float.
2. Áp công thức sigmoid lên mảng.
3. Làm tròn ba chữ số và in.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
print(np.round(1 / (1 + np.exp(-x)), 3).tolist())
```

Kết quả khi chạy:

```
[0.269, 0.5, 0.731]
```

Thứ tự ba quan sát không đổi; điểm sigmoid chưa mặc nhiên được hiệu chuẩn.

Tình huống 3 — Giữ tín hiệu âm dương bằng tanh

Tình huống thực tế: Một mạng dùng tín hiệu âm, 0, dương để biểu diễn hướng tác động; đầu vào lần lượt là −1,0,1. Cần đầu ra cân quanh 0.

Vì sao chọn cách giải này? Tanh giữ dấu trong khoảng (−1,1), phù hợp hơn sigmoid nếu cần tín hiệu âm.

Quy trình giải:

1. Tạo ba tín hiệu số thực.
2. Áp `np.tanh` và làm tròn.
3. In ba giá trị sau biến đổi.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
print(np.round(np.tanh(x), 3).tolist())
```

Kết quả khi chạy:

```
[-0.762, 0.0, 0.762]
```

Tín hiệu âm vẫn âm, dương vẫn dương và giá trị 0 giữ nguyên.

Tình huống 4 — Chuẩn hóa ba điểm phân loại

Tình huống thực tế: Một mô hình dự đoán một trong ba loại sản phẩm với logit `[2,1,0.1]`; cần tỷ trọng cho từng lớp sao cho tổng bằng 1.

Vì sao chọn cách giải này? Softmax biến toàn bộ bộ logit thành một phân phối, có bước trừ cực đại để tính ổn định.

Quy trình giải:

1. Lưu ba logit cho ba lớp loại trừ nhau.
2. Trừ logit lớn nhất rồi tính exp.
3. Chia cho tổng exp, in giá trị đã làm tròn.

```
import numpy as np

z = np.array([2.0, 1.0, 0.1])
e = np.exp(z - z.max())
prob = e / e.sum()
print(np.round(prob, 3).tolist())
```

Kết quả khi chạy:

```
[0.659, 0.242, 0.099]
```

Lớp đầu nhận điểm lớn nhất. Giá trị làm tròn không được dùng để kiểm tra tổng chính xác tuyệt đối.

Tình huống 5 — Ghép hai ReLU thành hàm có khoảng chết

Tình huống thực tế: Đầu vào x chạy qua −2, −1, 0, 1, 2. Đội nghiên cứu muốn hàm bằng 0 trong đoạn [−1,1] nhưng tăng bên ngoài hai mép.

Vì sao chọn cách giải này? Tổng `ReLU(x−1)+ReLU(−x−1)` tạo hàm gấp khúc đối xứng quanh 0.

Quy trình giải:

1. Tạo năm điểm x cách đều từ −2 tới 2.
2. Áp hai ReLU lên x−1 và −x−1.
3. Cộng hai kết quả và in dãy.

```
import numpy as np

x = np.arange(-2, 3)
y = np.maximum(0, x - 1) + np.maximum(0, -x - 1)
print(y.tolist())
```

Kết quả khi chạy:

```
[1, 0, 0, 0, 1]
```

Hàm có giá trị 0 ở ba điểm giữa và tăng đối xứng ở ngoài khoảng.

6. Sai lầm thường gặp

- ReLU không chặn **giá trị dương lớn**; nó chỉ chặn phần âm.
- Sigmoid nén mỗi giá trị về (0,1), nhưng các giá trị này không tự trở thành xác suất đã hiệu chuẩn.
- Tanh nằm trong (−1,1), còn sigmoid trong (0,1); không thay thế khi cần giữ tín hiệu âm.

- Softmax áp **cùng lúc** lên các logit của nhiều lớp; không áp độc lập từng logit.
- Khi tính softmax nên trừ logit lớn nhất trước exp để giảm tràn số.
- Softmax phù hợp các lớp loại trừ nhau; nhiều nhãn có thể đồng thời đúng thường dùng nhiều đầu ra sigmoid thay vì ép tổng một.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Chỉ hỏi kiến thức, phân biệt hàm và cách đọc đầu ra; không hỏi tình huống đánh số.

Q1. What is a ReLU activation? — Hàm kích hoạt ReLU là gì?

A. `max(0,x)` — `max(0,x)`

B. `1/(1+exp(-x))` — `1/(1+exp(-x))`

C. A smooth activation mapping into (−1,1) — Hàm trơn đưa giá trị vào (−1,1)

D. Normalization of several class logits to sum one — Chuẩn hóa nhiều logit lớp có tổng bằng một

Đáp án: A

Giải thích: ReLU chặn giá trị âm về không.

Q2. What is sigmoid? — Sigmoid là gì?

A. `max(0,x)` — `max(0,x)`

B. `1/(1+exp(-x))` — `1/(1+exp(-x))`

C. A smooth activation mapping into (−1,1) — Hàm trơn đưa giá trị vào (−1,1)

D. Normalization of several class logits to sum one — Chuẩn hóa nhiều logit lớp có tổng bằng một

Đáp án: B

Giải thích: Sigmoid là phép biến đổi logistic hình chữ S.

Q3. What is tanh? — Tanh là gì?

A. `max(0,x)` — `max(0,x)`

B. `1/(1+exp(-x))` — `1/(1+exp(-x))`

C. A smooth activation mapping into (−1,1) — Hàm trơn đưa giá trị vào (−1,1)

D. Normalization of several class logits to sum one — Chuẩn hóa nhiều logit lớp có tổng bằng một

Đáp án: C

Giải thích: Tanh cân quanh 0 và giới hạn giữa −1 và 1.

Q4. What is softmax? — Softmax là gì?

A. `max(0,x)` — `max(0,x)`

B. `1/(1+exp(-x))` — `1/(1+exp(-x))`

C. A smooth activation mapping into (−1,1) — Hàm trơn đưa giá trị vào (−1,1)

D. Normalization of several class logits to sum one — Chuẩn hóa nhiều logit lớp có tổng bằng một

Đáp án: D

Giải thích: Softmax ràng buộc điểm lớp thành một phân phối chuẩn hóa.

Q5. What is the output range of ReLU? — Đầu ra ReLU thuộc khoảng nào?

A. From zero upward without a fixed upper bound — Từ không trở lên, không có cận trên cố định

B. Strictly between 0 and 1 — Nằm giữa 0 và 1, không gồm hai đầu

C. Strictly between −1 and 1 — Nằm giữa −1 và 1, không gồm hai đầu

D. They are nonnegative and sum to one — Không âm và có tổng bằng một

Đáp án: A

Giải thích: Đầu vào dương được ReLU giữ nguyên.

Q6. What is the output range of sigmoid? — Đầu ra sigmoid thuộc khoảng nào?

A. From zero upward without a fixed upper bound — Từ không trở lên, không có cận trên cố định

B. Strictly between 0 and 1 — Nằm giữa 0 và 1, không gồm hai đầu

C. Strictly between −1 and 1 — Nằm giữa −1 và 1, không gồm hai đầu

D. They are nonnegative and sum to one — Không âm và có tổng bằng một

Đáp án: B

Giải thích: Sigmoid với đầu vào hữu hạn nằm trong (0,1).

Q7. What is the output range of tanh? — Đầu ra tanh thuộc khoảng nào?

A. From zero upward without a fixed upper bound — Từ không trở lên, không có cận trên cố định

B. Strictly between 0 and 1 — Nằm giữa 0 và 1, không gồm hai đầu

C. Strictly between −1 and 1 — Nằm giữa −1 và 1, không gồm hai đầu

D. They are nonnegative and sum to one — Không âm và có tổng bằng một

Đáp án: C

Giải thích: Tanh với đầu vào hữu hạn thuộc (−1,1).

Q8. What is true of softmax class scores? — Điều nào đúng về các điểm softmax?

A. From zero upward without a fixed upper bound — Từ không trở lên, không có cận trên cố định

B. Strictly between 0 and 1 — Nằm giữa 0 và 1, không gồm hai đầu

C. Strictly between −1 and 1 — Nằm giữa −1 và 1, không gồm hai đầu

D. They are nonnegative and sum to one — Không âm và có tổng bằng một

Đáp án: D

Giải thích: Softmax chia từng exp cho tổng các exp.

Q9. Which NumPy function applies ReLU? — Hàm NumPy nào thực hiện ReLU?

A. `np.maximum(0,x)` — `np.maximum(0,x)`

B. `np.tanh(x)` — `np.tanh(x)`

C. The largest logit — Logit lớn nhất

D. Tanh — Tanh

Đáp án: A

Giải thích: maximum chặn từng phần tử.

Q10. Which NumPy function directly computes tanh? — Hàm NumPy nào tính tanh trực tiếp?

A. `np.maximum(0,x)` — `np.maximum(0,x)`

B. `np.tanh(x)` — `np.tanh(x)`

C. The largest logit — Logit lớn nhất

D. Tanh — Tanh

Đáp án: B

Giải thích: NumPy có hàm tanh trực tiếp.

Q11. Which quantity should be subtracted from logits before stable softmax exponentiation? — Nên trừ gì khỏi logit trước khi tính exp softmax ổn định?

A. `np.maximum(0,x)` — `np.maximum(0,x)`

B. `np.tanh(x)` — `np.tanh(x)`

C. The largest logit — Logit lớn nhất

D. Tanh — Tanh

Đáp án: C

Giải thích: Trừ cực đại giữ các số exp không vượt một.

Q12. Which activation outputs exactly 0 at input 0? — Hàm nào trả đúng 0 với đầu vào 0?

A. `np.maximum(0,x)` — `np.maximum(0,x)`

B. `np.tanh(x)` — `np.tanh(x)`

C. The largest logit — Logit lớn nhất

D. Tanh — Tanh

Đáp án: D

Giải thích: Tanh(0)=0, còn sigmoid(0)=0.5.

Q13. What is sigmoid(0)? — sigmoid(0) bằng gì?

A. 0.5 — 0.5

B. 0 — 0

C. A negative output — Giá trị đầu ra âm

D. It reduces overflow risk without changing the result — Giảm nguy cơ tràn số mà không đổi kết quả

Đáp án: A

Giải thích: Sigmoid tại zero bằng một nửa.

Q14. What is ReLU(−2)? — ReLU(−2) bằng gì?

A. 0.5 — 0.5

B. 0 — 0

C. A negative output — Giá trị đầu ra âm

D. It reduces overflow risk without changing the result — Giảm nguy cơ tràn số mà không đổi kết quả

Đáp án: B

Giải thích: ReLU chặn đầu vào âm.

Q15. What does tanh produce for negative input? — Tanh tạo đầu ra thế nào khi đầu vào âm?

A. 0.5 — 0.5

B. 0 — 0

C. A negative output — Giá trị đầu ra âm

D. It reduces overflow risk without changing the result — Giảm nguy cơ tràn số mà không đổi kết quả

Đáp án: C

Giải thích: Tanh giữ dấu của đầu vào hữu hạn khác 0.

Q16. Why does subtracting max(logits) help softmax? — Vì sao trừ cực đại logit giúp softmax?

A. 0.5 — 0.5

B. 0 — 0

C. A negative output — Giá trị đầu ra âm

D. It reduces overflow risk without changing the result — Giảm nguy cơ tràn số mà không đổi kết quả

Đáp án: D

Giải thích: Dịch đều các logit không đổi softmax nhưng giảm tràn số.

Q17. Which output structure suits mutually exclusive multi-class prediction? — Dạng đầu ra nào phù hợp phân loại nhiều lớp loại trừ nhau?

A. A softmax distribution over classes — Phân phối softmax trên các lớp

B. Their inputs remain in the nonpositive flat region — Đầu vào luôn ở vùng phẳng không dương

C. No, calibration needs checking — Không, cần đánh giá độ hiệu chuẩn

D. Softmax couples scores so they sum to one — Softmax ràng buộc điểm các lớp có tổng một

Đáp án: A

Giải thích: Softmax thường dùng cho đầu ra các lớp loại trừ nhau.

Q18. Why might ReLU neurons stop updating? — Vì sao một số nơ-ron ReLU có thể ngừng cập nhật?

A. A softmax distribution over classes — Phân phối softmax trên các lớp

B. Their inputs remain in the nonpositive flat region — Đầu vào luôn ở vùng phẳng không dương

C. No, calibration needs checking — Không, cần đánh giá độ hiệu chuẩn

D. Softmax couples scores so they sum to one — Softmax ràng buộc điểm các lớp có tổng một

Đáp án: B

Giải thích: Độ dốc ReLU bằng không ở đầu vào âm.

Q19. Do sigmoid outputs automatically prove calibrated probabilities? — Đầu ra sigmoid có mặc nhiên là xác suất đã hiệu chuẩn không?

A. A softmax distribution over classes — Phân phối softmax trên các lớp

B. Their inputs remain in the nonpositive flat region — Đầu vào luôn ở vùng phẳng không dương

C. No, calibration needs checking — Không, cần đánh giá độ hiệu chuẩn

D. Softmax couples scores so they sum to one — Softmax ràng buộc điểm các lớp có tổng một

Đáp án: C

Giải thích: Điểm số cần kiểm tra với kết quả thực tế.

Q20. Which difference distinguishes softmax from independent sigmoid outputs? — Điểm nào phân biệt softmax với sigmoid độc lập?

A. A softmax distribution over classes — Phân phối softmax trên các lớp

B. Their inputs remain in the nonpositive flat region — Đầu vào luôn ở vùng phẳng không dương

C. No, calibration needs checking — Không, cần đánh giá độ hiệu chuẩn

D. Softmax couples scores so they sum to one — Softmax ràng buộc điểm các lớp có tổng một

Đáp án: D

Giải thích: Các sigmoid độc lập không bắt buộc có tổng bằng một.

8. Bài tiếp theo

`Hàm sô´→ Activation → Neural Network → Classification.`

PY-30 — Loss: MSE, MAE, Huber, cross-entropy và KL

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: MSE — sai số bình phương trung bình; **MAE** — sai số tuyệt đối trung bình.

`Dự đoán → Sai số → Loss → Tối ưu hóa`

Học xong dùng để làm gì? Đo sai số dự đoán bằng MSE hoặc loss thích hợp và đặt mục tiêu cần tối ưu.

Kết nối kiến thức tiếp theo: PY-31 tính đạo hàm để biết loss thay đổi ra sao khi chỉnh tham số.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
MSE	sai số bình phương trung bình	Trung bình bình phương hiệu dự đoán và giá trị thật
MAE	sai số tuyệt đối trung bình	Trung bình trị tuyệt đối sai số
Cross-entropy	entropy chéo	Phạt dự đoán xác suất sai nhãn mạnh hơn
KL divergence	độ lệch KL	Đo sai khác một chiều giữa hai phân phối xác suất

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`np.mean((y-pred)**2)`	MSE nhạy với sai số lớn do bình phương
`np.mean(np.abs(y-pred))`	MAE ít khuếch đại outlier hơn MSE
`np.where(abs_err<=delta,... )`	Huber phối hợp bình phương và tuyến tính
`-np.sum(y*np.log(probs))`	Cross-entropy trên xác suất có clip để tránh log(0)

4. Khái niệm liên quan

Nội dung chính: loss đo mức sai giữa dự báo và mục tiêu. Bài học đi từ hai cách tính sai lệch trên số (**MSE**, **MAE**), tới cách kết hợp hai mức phạt (**Huber**), rồi sang loss trên **xác suất và phân phối** (**cross-entropy**, **KL**). Chọn loss phải dựa vào ý nghĩa dữ liệu và mức phạt cần thiết.

4.1. MSE — Sai số bình phương trung bình

Từ vựng trong mục: MSE — Mean Squared Error; **Residual** — sai lệch.

Loss function là hàm cho biết dự báo sai lệch đến mức nào so với mục tiêu, để ta đánh giá hoặc tối ưu mô hình. **MSE** lấy sai số `(dự báo−thực tế)`, bình phương từng sai số rồi **trung bình**. Với thực tế `[2,3]` và dự báo `[1,4]`, sai số là `[-1,+1]`, bình phương thành `[1,1]` và MSE bằng 1. Đơn vị MSE là **bình phương đơn vị đầu ra**, không phải cùng đơn vị như giá nhà.

```
import numpy as np

y_true = np.array([2, 3])
y_pred = np.array([1, 4])
print(np.mean((y_true - y_pred) ** 2))
```

Kết quả khi chạy:

```
1.0
```

MSE bằng 1 đơn vị bình phương. Sai số lớn được phạt mạnh hơn vì bình phương.

4.2. MAE — Sai số tuyệt đối trung bình

Từ vựng trong mục: MAE — Mean Absolute Error; **Absolute error** — sai số tuyệt đối.

MAE lấy trị tuyệt đối của từng sai số rồi chia cho số quan sát. Ví dụ thực tế `[10,20,30]` và dự báo `[9,19,50]` có sai lệch tuyệt đối `[1,1,20]`, nên MAE bằng `22/3≈7.333`. Khác MSE, MAE có **cùng đơn vị** với đầu ra và không bình phương để khuếch đại lỗi lớn. Nó không có nghĩa là loại bỏ ảnh hưởng của ngoại lệ hoàn toàn.

```
import numpy as np

y_true = np.array([10, 20, 30])
y_pred = np.array([9, 19, 50])
print(round(float(np.mean(np.abs(y_true - y_pred))), 3))
```

Kết quả khi chạy:

```
7.333
```

Sai lệch trung bình tuyệt đối khoảng 7,333 đơn vị đo.

4.3. Huber — Trộn bình phương và tuyến tính theo ngưỡng

Từ vựng trong mục: Huber loss — tổn thất Huber; **Delta** — ngưỡng.

Huber dùng mức phạt giống **bình phương** khi $|e|\le\delta$ và chuyển sang **tuyến tính** khi lỗi vượt δ. Với δ=1: lỗi có độ lớn 1 mất 0.5, độ lớn 2 mất 1.5, độ lớn 5 mất 4.5. Trung bình ba lỗi `[-1,2,5]` là `6.5/3≈2.167`. Huber là cách cân bằng giữa MSE nhạy với lỗi lớn và MAE tăng tuyến tính; δ quyết định điểm chuyển và mang cùng đơn vị với sai số.

```
import numpy as np

e = np.array([-1.0, 2.0, 5.0])
delta = 1.0
loss = np.where(
    np.abs(e) <= delta,
```

```
    0.5 * e ** 2,
    delta * (np.abs(e) - 0.5 * delta),
)
print(round(float(loss.mean()), 3))
```

Kết quả khi chạy:

```
2.167
```

Tổn thất trung bình xấp xỉ 2,167 theo Huber δ=1; lỗi lớn không bị bình phương mãi.

4.4. Cross-entropy và KL — So nhãn hoặc phân phối

Từ vựng trong mục: Cross-entropy — entropy chéo; **KL divergence** — phân kỳ KL.

Cross-entropy đánh giá phân phối dự báo so với phân phối nhãn/đích. Với phân loại một nhãn đúng, dự báo xác suất cho nhãn đúng là 0.8 thì loss bằng $-\ln(0.8) \approx 0.2231$: xác suất cho nhãn đúng càng thấp, phạt càng lớn. **KL divergence** $D_{KL}(P||Q) = \Sigma\ P_i\ \ln(P_i/Q_i)$ so sự khác nhau giữa **hai phân phối** P và Q; với $P=[0.5,0.5]$, $Q=[0.6,0.4]$, KL≈0.0204. KL **không đối xứng** và không phải khoảng cách hình học. Các xác suất trong log cần dương; trong code thực tế có thể clip hoặc xử lý giá trị 0 đúng quy ước.

```
import numpy as np

p_true = 0.8
P = np.array([0.5, 0.5])
Q = np.array([0.6, 0.4])
print(round(float(-np.log(p_true)), 4))
print(round(float(np.sum(P * np.log(P / Q))), 4))
```

Kết quả khi chạy:

```
0.2231
0.0204
```

Dòng đầu là cross-entropy của nhãn đúng; dòng sau là KL từ phân phối P sang Q. Hai số mang ý nghĩa khác nhau.

5. Năm tình huống ứng dụng

Tình huống 1 — Đánh giá sai số giá nhà bằng MSE

Tình huống thực tế: Hai căn nhà có giá thực tế $[100,200]$ và dự báo $[110,170]$, tất cả cùng đơn vị triệu đồng. Cần đánh giá mô hình bằng mức phạt nhấn mạnh dự báo sai lớn.

Vì sao chọn cách giải này? Bình phương lỗi khiến dự báo sai 30 triệu bị phạt mạnh hơn lỗi 10 triệu.

Quy trình giải:

1. Tạo hai mảng giá thực tế và giá dự báo theo đúng thứ tự căn nhà.
2. Trừ, bình phương từng sai lệch, lấy trung bình.
3. In MSE; đơn vị là triệu đồng bình phương.

```
import numpy as np

y_true = np.array([100, 200])
y_pred = np.array([110, 170])
print(np.mean((y_true - y_pred) ** 2))
```

Kết quả khi chạy:

```
500.0
```

MSE bằng 500 (triệu đồng)², không phải sai số tiền trung bình 500 triệu.

Tình huống 2 — Đo lỗi trung bình dễ hiểu bằng MAE

Tình huống thực tế: Ba kết quả thực tế `[10,20,30]` và dự báo `[9,19,50]` cùng đơn vị có một dự báo sai lớn ở vị trí cuối. Cần đo sai lệch trung bình theo đơn vị gốc.

Vì sao chọn cách giải này? MAE đo trị tuyệt đối nên dễ diễn giải theo cùng đơn vị; không bình phương lỗi lớn.

Quy trình giải:

1. Tạo mảng thực tế và dự báo cùng thứ tự.
2. Lấy trị tuyệt đối sai lệch theo từng quan sát rồi trung bình.
3. In kết quả làm tròn ba chữ số.

```
import numpy as np

y_true = np.array([10, 20, 30])
y_pred = np.array([9, 19, 50])
print(round(float(np.mean(np.abs(y_true - y_pred))), 3))
```

Kết quả khi chạy:

```
7.333
```

Độ lệch trung bình tuyệt đối khoảng 7.333 đơn vị gốc.

Tình huống 3 — Áp dụng Huber cho số đo nhiễu

Tình huống thực tế: Ba sai số thiết bị là `[−1,2,5]` đơn vị đo. Dùng Huber với ngưỡng δ=1 để tăng phạt tuyến tính khi lỗi lớn.

Vì sao chọn cách giải này? Huber dùng phần bình phương ở lỗi nhỏ và phần tuyến tính ở lỗi lớn, tránh khuếch đại lỗi 5 quá mạnh.

Quy trình giải:

1. Lưu ba sai số và ngưỡng δ=1.
2. Dùng `np.where` tính hai nhánh của hàm Huber theo độ lớn lỗi.
3. Lấy trung bình và in sau làm tròn.

```
import numpy as np

e = np.array([-1.0, 2.0, 5.0])
delta = 1.0
loss = np.where(
    np.abs(e) <= delta,
    0.5 * e ** 2,
    delta * (np.abs(e) - 0.5 * delta),
)
print(round(float(loss.mean()), 3))
```

Kết quả khi chạy:

```
2.167
```

Loss Huber trung bình khoảng 2.167 cho ba sai số với δ=1.

Tình huống 4 — Tính cross-entropy của nhãn đúng

Tình huống thực tế: Bộ phân loại đưa xác suất 0.8 cho nhãn đúng của một mẫu. Hãy tính mức phạt logarit cho dự báo này.

Vì sao chọn cách giải này? Với nhãn one-hot, cross-entropy bằng âm log xác suất của nhãn đúng.

Quy trình giải:

1. Lưu xác suất nhãn đúng 0.8, giá trị nằm trong (0,1].
2. Tính `-math.log(prob_true)`.
3. Làm tròn bốn chữ số và in loss.

```
import math

prob_true = 0.8
print(round(-math.log(prob_true), 4))
```

Kết quả khi chạy:

```
0.2231
```

Nếu mô hình tăng xác suất cho nhãn đúng về gần 1, loss giảm về gần 0.

Tình huống 5 — So hai phân phối bằng KL divergence

Tình huống thực tế: Phân phối P là `[0.5,0.5]` và Q là `[0.6,0.4]` cho hai lớp; các tổng đều bằng 1. Cần đo P khác Q bao nhiêu theo chiều `D_KL(P||Q)`.

Vì sao chọn cách giải này? KL cân chênh lệch logarit theo xác suất P và phụ thuộc chiều so sánh, không phải khoảng cách đối xứng.

Quy trình giải:

1. Tạo hai phân phối dương, có tổng 1.
2. Tính `P * log(P/Q)` ở từng lớp rồi cộng.
3. In KL làm tròn bốn chữ số.

```
import numpy as np

P = np.array([0.5, 0.5])
Q = np.array([0.6, 0.4])
print(round(float(np.sum(P * np.log(P / Q))), 4))
```

Kết quả khi chạy:

```
0.0204
```

KL dương và nhỏ vì hai phân phối khá gần nhau, nhưng không đồng nhất.

6. Sai lầm thường gặp

- **MSE** có đơn vị đầu ra bình phương, **MAE** có đơn vị gốc; không đọc MSE như mức lệch tiền trung bình.
- Huber cần ngưỡng δ phù hợp đơn vị sai số; ngưỡng này quyết định khi nào chuyển từ phạt bình phương sang tuyến tính.
- Cross-entropy với nhãn đúng cần xác suất **của lớp đúng**, không phải điểm lớn nhất bất kỳ.
- `log(0)` không xác định; phải xử lý xác suất bằng công thức ổn định hoặc clip phù hợp khi dùng NumPy đơn giản.
- KL **bất đối xứng**, không phải khoảng cách Euclid và chỉ có nghĩa với các phân phối hợp lệ.
- Loss huấn luyện thấp không thay thế việc đánh giá trên tập validation/test độc lập.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Chỉ hỏi kiến thức lý thuyết, so sánh loss và đọc công thức, không hỏi dữ liệu tình huống.

Q1. What is MSE? — MSE là gì?

A. Mean squared prediction error — Trung bình sai số dự báo bình phương

B. Mean absolute prediction error — Trung bình trị tuyệt đối sai số

C. Quadratic for small errors and linear for large — Bình phương lỗi nhỏ, tuyến tính lỗi lớn

D. Compare two probability distributions in a directed way — So hai phân phối xác suất có phân biệt chiều

Đáp án: A

Giải thích: MSE lấy trung bình bình phương sai lệch.

Q2. What is MAE? — MAE là gì?

A. Mean squared prediction error — Trung bình sai số dự báo bình phương

B. Mean absolute prediction error — Trung bình trị tuyệt đối sai số

C. Quadratic for small errors and linear for large — Bình phương lỗi nhỏ, tuyến tính lỗi lớn

D. Compare two probability distributions in a directed way — So hai phân phối xác suất có phân biệt chiều

Đáp án: B

Giải thích: MAE lấy trung bình độ lệch tuyệt đối.

Q3. What characterizes Huber loss? — Đặc trưng của Huber loss là gì?

A. Mean squared prediction error — Trung bình sai số dự báo bình phương

B. Mean absolute prediction error — Trung bình trị tuyệt đối sai số

C. Quadratic for small errors and linear for large — Bình phương lỗi nhỏ, tuyến tính lỗi lớn

D. Compare two probability distributions in a directed way — So hai phân phối xác suất có phân biệt chiều

Đáp án: C

Giải thích: Huber đổi tốc độ tăng mức phạt sau một ngưỡng.

Q4. What is KL divergence used for? — KL divergence dùng để làm gì?

A. Mean squared prediction error — Trung bình sai số dự báo bình phương

B. Mean absolute prediction error — Trung bình trị tuyệt đối sai số

C. Quadratic for small errors and linear for large — Bình phương lỗi nhỏ, tuyến tính lỗi lớn

D. Compare two probability distributions in a directed way — So hai phân phối xác suất có phân biệt chiều

Đáp án: D

Giải thích: KL dùng tỷ số logarit theo một chiều giữa các phân phối.

Q5. What does categorical cross-entropy penalize? — Categorical cross-entropy phạt điều gì?

A. Low predicted probability for the correct label — Xác suất dự đoán thấp cho nhãn đúng

B. `mean((y-pred)**2)` — `mean((y-pred)**2)`

C. `mean(abs(y-pred))` — `mean(abs(y-pred))`

D. `delta` — `delta`

Đáp án: A

Giải thích: Lớp đúng cần được gán xác suất cao.

Q6. Which formula computes MSE? — Công thức nào tính MSE?

A. Low predicted probability for the correct label — Xác suất dự đoán thấp cho nhãn đúng

B. `mean((y-pred)**2)` — `mean((y-pred)**2)`

C. `mean(abs(y-pred))` — `mean(abs(y-pred))`

D. `delta` — `delta`

Đáp án: B

Giải thích: Bình phương từng sai lệch trước khi trung bình.

Q7. Which formula computes MAE? — Công thức nào tính MAE?

A. Low predicted probability for the correct label — Xác suất dự đoán thấp cho nhãn đúng

B. `mean((y-pred)**2)` — `mean((y-pred)**2)`

C. `mean(abs(y-pred))` — `mean(abs(y-pred))`

D. `delta` — `delta`

Đáp án: C

Giải thích: Lấy trị tuyệt đối từng sai lệch rồi trung bình.

Q8. Which parameter sets Huber's transition point? — Tham số nào đặt điểm chuyển Huber?

A. Low predicted probability for the correct label — Xác suất dự đoán thấp cho nhãn đúng

B. `mean((y-pred)**2)` — `mean((y-pred)**2)`

C. `mean(abs(y-pred))` — `mean(abs(y-pred))`

D. `delta` — `delta`

Đáp án: D

Giải thích: Delta phân chia hai miền của Huber.

Q9. What does `-log(p_true)` represent for one correct class? — `-log(p_true)` biểu thị gì cho một nhãn đúng?

A. Cross-entropy loss for that sample — Cross-entropy của mẫu đó

B. Nonnegative — Không âm

C. No, KL is asymmetric — Không, KL bất đối xứng

D. The distributions are identical — Hai phân phối giống nhau

Đáp án: A

Giải thích: Với one-hot, chỉ còn hạng log của lớp đúng.

Q10. What is the sign of KL divergence for valid distributions? — KL divergence có dấu gì với phân phối hợp lệ?

A. Cross-entropy loss for that sample — Cross-entropy của mẫu đó

B. Nonnegative — Không âm

C. No, KL is asymmetric — Không, KL bất đối xứng

D. The distributions are identical — Hai phân phối giống nhau

Đáp án: B

Giải thích: KL không âm theo bất đẳng thức Gibbs.

Q11. Does swapping P and Q generally preserve KL? — Đổi chỗ P và Q có giữ nguyên KL không?

A. Cross-entropy loss for that sample — Cross-entropy của mẫu đó

B. Nonnegative — Không âm

C. No, KL is asymmetric — Không, KL bất đối xứng

D. The distributions are identical — Hai phân phối giống nhau

Đáp án: C

Giải thích: D(P||Q) và D(Q||P) có thể khác.

Q12. What does a zero KL divergence mean for valid finite distributions? — KL bằng 0 nghĩa là gì với các phân phối hữu hạn hợp lệ?

A. Cross-entropy loss for that sample — Cross-entropy của mẫu đó

B. Nonnegative — Không âm

C. No, KL is asymmetric — Không, KL bất đối xứng

D. The distributions are identical — Hai phân phối giống nhau

Đáp án: D

Giải thích: KL bằng 0 khi các phân phối trùng nhau trong điều kiện phù hợp.

Q13. What is the unit of MSE when predictions are in meters? — Dự báo theo mét thì MSE có đơn vị gì?

A. Square meters — Mét bình phương

B. Meters — Mét

C. MSE — MSE

D. Huber loss — Huber loss

Đáp án: A

Giải thích: Bình phương sai lệch mét cho đơn vị m^2.

Q14. What is the unit of MAE when predictions are in meters? — Dự báo theo mét thì MAE có đơn vị gì?

A. Square meters — Mét bình phương

B. Meters — Mét

C. MSE — MSE

D. Huber loss — Huber loss

Đáp án: B

Giải thích: Sai lệch tuyệt đối giữ đơn vị gốc.

Q15. Which loss increases quadratically with error magnitude? — Loss nào tăng theo bình phương độ lớn lỗi?

A. Square meters — Mét bình phương

B. Meters — Mét

C. MSE — MSE

D. Huber loss — Huber loss

Đáp án: C

Giải thích: MSE bình phương độ lớn lỗi.

Q16. Which regression loss has a threshold between quadratic and linear penalties? — Loss hồi quy nào có ngưỡng chuyển bình phương sang tuyến tính?

A. Square meters — Mét bình phương

B. Meters — Mét

C. MSE — MSE

D. Huber loss — Huber loss

Đáp án: D

Giải thích: Huber chuyển miền tại delta.

Q17. Why clip predicted probabilities before a naive log implementation? — Vì sao clip xác suất trước khi lấy log bằng công thức đơn giản?

A. To avoid log(0) and numerical infinities — Để tránh log(0) và vô cực số học

B. MAE — MAE

C. No, evaluate on held-out data — Không, cần đánh giá trên dữ liệu độc lập

D. Predicted probability assigned to the true class — Xác suất dự báo dành cho lớp đúng

Đáp án: A

Giải thích: Xác suất 0 gây kỳ dị log trong công thức số học đơn giản.

Q18. Which loss is easier to interpret in the original output units? — Loss nào dễ đọc theo đơn vị đầu ra ban đầu?

A. To avoid log(0) and numerical infinities — Để tránh log(0) và vô cực số học

B. MAE — MAE

C. No, evaluate on held-out data — Không, cần đánh giá trên dữ liệu độc lập

D. Predicted probability assigned to the true class — Xác suất dự báo dành cho lớp đúng

Đáp án: B

Giải thích: MAE báo sai số trung bình cùng đơn vị đo.

Q19. Does a lower training loss guarantee better unseen-data performance? — Training loss thấp có bảo đảm dự đoán tốt trên dữ liệu mới?

A. To avoid log(0) and numerical infinities — Để tránh log(0) và vô cực số học

B. MAE — MAE

C. No, evaluate on held-out data — Không, cần đánh giá trên dữ liệu độc lập

D. Predicted probability assigned to the true class — Xác suất dự báo dành cho lớp đúng

Đáp án: C

Giải thích: Khả năng tổng quát cần kiểm chứng bằng dữ liệu độc lập.

Q20. For one-hot labels, what information matters in cross-entropy? — Với nhãn one-hot, điều gì quyết định cross-entropy?

A. To avoid log(0) and numerical infinities — Để tránh log(0) và vô cực số học

B. MAE — MAE

C. No, evaluate on held-out data — Không, cần đánh giá trên dữ liệu độc lập

D. Predicted probability assigned to the true class — Xác suất dự báo dành cho lớp đúng

Đáp án: D

Giải thích: Âm log-likelihood dùng xác suất lớp đúng.

8. Bài tiếp theo

`Dự đoán → Sai số → Loss → Tối ưu hóa.`

PY-31 — Giới hạn, tính liên tục và đạo hàm

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Limit — giới hạn; **Continuity** — tính liên tục.

`Hàm số → Limit → Derivative → Gradient`

Học xong dùng để làm gì? Đo tốc độ thay đổi của hàm theo biến và suy ra chiều tăng/giảm của sai số.

Kết nối kiến thức tiếp theo: PY-32 dùng gradient và tốc độ học để cập nhật trọng số mô hình.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Limit	giới hạn	Giá trị hàm tiến gần khi đầu vào tiến đến điểm
Continuity	tính liên tục	Giới hạn và giá trị hàm phù hợp tại điểm xét
Derivative	đạo hàm	Tốc độ thay đổi cục bộ của hàm theo đầu vào
Tangent slope	độ dốc tiếp tuyến	Giá trị đạo hàm tại một điểm

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`sympy.symbols()/limit()`	Tạo biến ký hiệu và tính giới hạn kể cả điểm không xác định
`sympy.diff(f,x)`	Đạo hàm theo biến x
`expr.subs(x,value)`	Thay giá trị vào công thức symbolic
`np.gradient(y,x)`	Ước lượng đạo hàm từ dãy mẫu, phụ thuộc bước lấy mẫu

4. Khái niệm chính và liên quan

Mạch kiến thức: Giới hạn mô tả xu hướng gần một điểm; liên tục đòi hỏi xu hướng ấy bằng giá trị tại điểm; đạo hàm dùng giới hạn để đo độ thay đổi; độ dốc tiếp tuyến là đạo hàm sau khi thay một tọa độ cụ thể.

4.1. Limit — Giá trị hàm tiến đến

Từ vựng: Limit — giới hạn; **Approach** — tiến gần.

Giới hạn hỏi **f(x) tiến gần giá trị nào khi x tiến gần a**, không bắt buộc f(a) phải có giá trị. Ví dụ $(x^2-4)/(x-2)$ không xác định tại x=2 vì mẫu số bằng 0. Với x khác 2, rút gọn thành $x+2$; khi $x \to 2$, kết quả tiến đến **4**. `sympy.limit` tính giá trị tiến đến của biểu thức thay vì thay trực tiếp x=2 gây chia 0.

```
import sympy as sp

x = sp.symbols('x')
f = (x ** 2 - 4) / (x - 2)
print(sp.limit(f, x, 2))
```

Kết quả khi chạy:

```
4
```

Giới hạn bằng 4 dù biểu thức gốc không xác định ngay tại x=2.

4.2. Continuity — Hàm liên tục tại một điểm

Từ vựng: Continuity — tính liên tục; **Function value** — giá trị hàm.

Hàm liên tục tại x=a nếu **f(a) tồn tại**, giới hạn từ hai phía tồn tại và bằng f(a). Ví dụ quãng đường mô phỏng $s(t)=t^2$ là hàm liên tục; tại t=2 giây, $s(2)=4$ mét và khi t tiến gần 2, quãng đường cũng tiến đến 4 mét. Đây là tính chất của **mô hình hàm**, không tự suy ra cảm biến vật lý hoạt động liên tục.

```
import sympy as sp

t = sp.symbols('t')
s = t ** 2
print(sp.limit(s, t, 2) == s.subs(t, 2))
```

Kết quả khi chạy:

```
True
```

Giới hạn bằng giá trị hàm ở t=2, nên mô hình liên tục tại điểm xét.

4.3. Derivative — Tốc độ thay đổi tức thời

Từ vựng: Derivative — đạo hàm; **Rate of change** — tốc độ thay đổi.

Đạo hàm `f'(x)` cho biết đầu ra thay đổi bao nhiêu **khi đầu vào thay đổi rất nhỏ**. Về công thức, đó là giới hạn $(f(x+h)-f(x))/h$ khi $h \to 0$ nếu giới hạn tồn tại. Với $f(x)=x^2$, ta có $f'(x)=2x$; độ dốc không cố định: tại x=1 là 2, tại x=3 là 6. Nếu f là quãng đường (mét) theo thời gian (giây), đơn vị đạo hàm là mét/giây.

```
import sympy as sp

x = sp.symbols('x')
print(sp.diff(x ** 2, x))
```

Kết quả khi chạy:

```
2*x
```

Đạo hàm là biểu thức 2x; tăng x thì độ dốc cục bộ tăng.

4.4. Tangent slope — Độ dốc tại một giá trị cụ thể

Từ vựng: Tangent — tiếp tuyến; **Slope** — độ dốc.

Nếu đạo hàm là **hàm mô tả độ dốc**, thì độ dốc tiếp tuyến tại x=a là **một số**, chính là `f'(a)`. Với đường `f(x)=x²`, đạo hàm 2x, nên tại x=3 độ dốc bằng 6. Phân biệt `sp.diff` tạo biểu thức đạo hàm và `.subs(x,3)` thay tọa độ để lấy độ dốc tại đúng điểm đó.

```
import sympy as sp

x = sp.symbols('x')
slope = sp.diff(x ** 2, x)
print(slope.subs(x, 3))
```

Kết quả khi chạy:

```
6
```

Đường cong có độ dốc tiếp tuyến bằng 6 tại x=3.

5. Năm tình huống ứng dụng

Tình huống 1 — Giới hạn tại điểm khuyết của biểu thức

Tình huống thực tế: Một công thức chuẩn hóa cho ra `f(x)=(x²−4)/(x−2)`; tại x=2 mẫu số bằng 0. Nhóm phân tích muốn biết **giá trị mà công thức tiến gần**, không cố tính f(2).

Vì sao chọn cách giải này? Dùng giới hạn vì phép thay thế trực tiếp bị chia cho 0, còn xu hướng hai phía vẫn có thể tồn tại.

Quy trình giải:

1. Đặt x là biến ký hiệu.
2. Tạo biểu thức với tử và mẫu, chọn điểm x=2.
3. Gọi `sp.limit`, in giới hạn.

```
import sympy as sp

x = sp.symbols('x')
f = (x ** 2 - 4) / (x - 2)
print(sp.limit(f, x, 2))
```

Kết quả khi chạy:

```
4
```

Giới hạn là 4; biểu thức gốc vẫn không xác định tại x=2.

Tình huống 2 — Tính vận tốc từ mô hình quãng đường

Tình huống thực tế: Một robot được mô phỏng theo $s(t)=t^2$ mét, t tính bằng giây. Cần tìm vận tốc tức thời tại t=3 giây, thay vì vận tốc trung bình trên cả hành trình.

Vì sao chọn cách giải này? Vận tốc là đạo hàm quãng đường theo thời gian, có đơn vị mét/giây.

Quy trình giải:

1. Khai báo t và hàm quãng đường.
2. Lấy đạo hàm s'(t)=2t.
3. Thay t=3 rồi in vận tốc.

```
import sympy as sp

t = sp.symbols('t')
s = t ** 2
v = sp.diff(s, t)
print(v.subs(t, 3))
```

Kết quả khi chạy:

```
6
```

Vận tốc tức thời theo mô hình là 6 m/s.

Tình huống 3 — Độ dốc của hàm doanh thu

Tình huống thực tế: Mô hình doanh thu giả định $R(x)=2x^2+3x$ nghìn đồng, x là số đơn vị hàng. Đội vận hành muốn biết mức doanh thu thay đổi cục bộ quanh x=4, chưa xét việc x chỉ nhận số nguyên trong giao dịch thực tế.

Vì sao chọn cách giải này? Đạo hàm chỉ ra độ tăng cục bộ của mô hình liên tục, không phải tổng doanh thu tại x=4.

Quy trình giải:

1. Tạo biểu thức $R(x)=2x^2+3x$.
2. Tính $R'(x)=4x+3$.
3. Thay x=4, in độ dốc.

```
import sympy as sp

x = sp.symbols('x')
revenue = 2 * x ** 2 + 3 * x
print(sp.diff(revenue, x).subs(x, 4))
```

Kết quả khi chạy:

```
19
```

Độ dốc là 19 nghìn đồng mỗi đơn vị hàng theo mô hình liên tục.

Tình huống 4 — Kiểm tra tính liên tục của hàm số

Tình huống thực tế: Mô hình tính vị trí $f(x)=x^2$ tại x=2 mét đầu vào. Cần xem giới hạn của hàm tại điểm này có bằng chính giá trị f(2) hay không.

Vì sao chọn cách giải này? So giới hạn với giá trị hàm là kiểm tra định nghĩa liên tục tại điểm đã chọn.

Quy trình giải:

1. Định nghĩa hàm và điểm x=2.
2. Tính giới hạn và giá trị tại điểm.
3. In kết quả so sánh Boolean.

```
import sympy as sp

x = sp.symbols('x')
f = x ** 2
print(sp.limit(f, x, 2) == f.subs(x, 2))
```

Kết quả khi chạy:

```
True
```

Mô hình liên tục tại x=2 vì giới hạn và giá trị thực của hàm đều là 4.

Tình huống 5 — Ước lượng vận tốc từ năm lần đo

Tình huống thực tế: Cảm biến ghi quãng đường theo mô hình $s=t^2$ mét ở thời điểm 0,1,2,3,4 giây. Chỉ có năm mẫu rời rạc, nên cần **ước lượng** đạo hàm thay vì dùng biểu thức ký hiệu.

Vì sao chọn cách giải này? `np.gradient` dùng chênh lệch hữu hạn; biên có thể kém chính xác hơn các điểm giữa.

Quy trình giải:

1. Tạo trục thời gian năm giây và quãng đường tương ứng.
2. Gọi `np.gradient(distance,time)`.
3. In năm ước lượng vận tốc theo m/s.

```
import numpy as np

time = np.arange(5, dtype=float)
distance = time ** 2
print(np.gradient(distance, time))
```

Kết quả khi chạy:

```
[1. 2. 4. 6. 7.]
```

Ở các điểm giữa, ước lượng bằng đạo hàm 2t; hai đầu dùng sai phân một phía nên khác giá trị tức thời chính xác.

6. Sai lầm thường gặp

- Giới hạn không nhất thiết bằng giá trị hàm tại điểm nếu hàm không liên tục hoặc không xác định ở đó.
- Muốn liên tục cần giới hạn trái, phải và giá trị hàm bằng nhau.
- Đạo hàm là tốc độ thay đổi tức thời; độ dốc trung bình trên một khoảng có thể khác.
- Với quãng đường theo giây, đạo hàm có đơn vị m/s; không diễn giải thành mét.
- `np.gradient` xấp xỉ từ mẫu; ở đầu và cuối dãy thường dùng sai phân một phía.
- SymPy là thư viện ngoài cần cài khi chạy các ví dụ ký hiệu.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What does a limit describe? — Giới hạn mô tả gì?

A. What f(x) approaches as x approaches a point — Giá trị f(x) tiến đến khi x tiến gần một điểm

B. The two-sided limit exists and equals f(a) — Giới hạn hai phía tồn tại và bằng f(a)

C. Instantaneous rate of change — Tốc độ thay đổi tức thời

D. The derivative evaluated at a — Giá trị đạo hàm tại a

Đáp án: A

Giải thích: Giới hạn xét đầu vào lân cận điểm.

Q2. What does continuity at x=a require? — Liên tục tại x=a cần gì?

A. What f(x) approaches as x approaches a point — Giá trị f(x) tiến đến khi x tiến gần một điểm

B. The two-sided limit exists and equals f(a) — Giới hạn hai phía tồn tại và bằng f(a)

C. Instantaneous rate of change — Tốc độ thay đổi tức thời

D. The derivative evaluated at a — Giá trị đạo hàm tại a

Đáp án: B

Giải thích: Liên tục đòi hỏi giới hạn bằng giá trị hàm.

Q3. What does a derivative describe? — Đạo hàm mô tả gì?

A. What f(x) approaches as x approaches a point — Giá trị f(x) tiến đến khi x tiến gần một điểm

B. The two-sided limit exists and equals f(a) — Giới hạn hai phía tồn tại và bằng f(a)

C. Instantaneous rate of change — Tốc độ thay đổi tức thời

D. The derivative evaluated at a — Giá trị đạo hàm tại a

Đáp án: C

Giải thích: Đạo hàm đo độ thay đổi tại điểm.

Q4. What is tangent slope at x=a? — Độ dốc tiếp tuyến tại x=a là gì?

A. What f(x) approaches as x approaches a point — Giá trị f(x) tiến đến khi x tiến gần một điểm

B. The two-sided limit exists and equals f(a) — Giới hạn hai phía tồn tại và bằng f(a)

C. Instantaneous rate of change — Tốc độ thay đổi tức thời

D. The derivative evaluated at a — Giá trị đạo hàm tại a

Đáp án: D

Giải thích: Độ dốc tiếp tuyến là f'(a).

Q5. What does `sp.limit(f,x,a)` calculate? — `sp.limit(f,x,a)` tính gì?

A. The limit of f as x approaches a — Giới hạn của f khi x tiến đến a

B. The derivative of f with respect to x — Đạo hàm f theo biến x

C. Substitutes a into symbolic expression — Thay a vào biểu thức ký hiệu

D. Derivatives from sampled y and x values — Đạo hàm từ các giá trị y, x lấy mẫu

Đáp án: A

Giải thích: limit tính giới hạn mà không cần thế tại điểm kỳ dị.

Q6. What does `sp.diff(f,x)` return? — `sp.diff(f,x)` trả gì?

A. The limit of f as x approaches a — Giới hạn của f khi x tiến đến a

B. The derivative of f with respect to x — Đạo hàm f theo biến x

C. Substitutes a into symbolic expression — Thay a vào biểu thức ký hiệu

D. Derivatives from sampled y and x values — Đạo hàm từ các giá trị y, x lấy mẫu

Đáp án: B

Giải thích: diff tính đạo hàm ký hiệu.

Q7. What does `expr.subs(x,a)` do? — `expr.subs(x,a)` làm gì?

A. The limit of f as x approaches a — Giới hạn của f khi x tiến đến a

B. The derivative of f with respect to x — Đạo hàm f theo biến x

C. Substitutes a into symbolic expression — Thay a vào biểu thức ký hiệu

D. Derivatives from sampled y and x values — Đạo hàm từ các giá trị y, x lấy mẫu

Đáp án: C

Giải thích: subs thay giá trị vào biểu thức ký hiệu.

Q8. What does `np.gradient(y,x)` approximate? — `np.gradient(y,x)` xấp xỉ gì?

A. The limit of f as x approaches a — Giới hạn của f khi x tiến đến a

B. The derivative of f with respect to x — Đạo hàm f theo biến x

C. Substitutes a into symbolic expression — Thay a vào biểu thức ký hiệu

D. Derivatives from sampled y and x values — Đạo hàm từ các giá trị y, x lấy mẫu

Đáp án: D

Giải thích: gradient xấp xỉ đạo hàm từ mẫu.

Q9. What is the derivative of x^2? — Đạo hàm của x^2 là gì?

A. 2*x — 2*x

B. Zero — Bằng 0

C. 6 — 6

D. 4 — 4

Đáp án: A

Giải thích: Quy tắc đạo hàm lũy thừa cho 2x.

Q10. What is the derivative of a constant? — Đạo hàm của một hằng số là gì?

A. 2*x — 2*x

B. Zero — Bằng 0

C. 6 — 6

D. 4 — 4

Đáp án: B

Giải thích: Hằng số không đổi theo biến.

Q11. What is the slope of x^2 at x=3? — Độ dốc của x^2 tại x=3 là gì?

A. 2*x — 2*x

B. Zero — Bằng 0

C. 6 — 6

D. 4 — 4

Đáp án: C

Giải thích: 2 nhân 3 bằng sáu.

Q12. What is the limit of $(x^2-4)/(x-2)$ at x=2? — Giới hạn của $(x^2-4)/(x-2)$ khi $x \to 2$ là gì?

A. 2*x — 2*x

B. Zero — Bằng 0

C. 6 — 6

D. 4 — 4

Đáp án: D

Giải thích: Rút gọn gần điểm còn x+2.

Q13. What may happen if the left and right limits differ? — Nếu giới hạn trái và phải khác nhau thì sao?

A. The two-sided limit does not exist — Giới hạn hai phía không tồn tại

B. Distance per unit time — Khoảng cách trên mỗi đơn vị thời gian

C. Sampling spacing and numerical scheme — Khoảng cách mẫu và cách lấy sai phân

D. Yes, if nearby values approach one number — Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: A

Giải thích: Giới hạn một phía khác nhau khiến giới hạn hai phía không có.

Q14. What does a derivative unit become for distance over time? — Đạo hàm quãng đường theo thời gian có đơn vị gì?

A. The two-sided limit does not exist — Giới hạn hai phía không tồn tại

B. Distance per unit time — Khoảng cách trên mỗi đơn vị thời gian

C. Sampling spacing and numerical scheme — Khoảng cách mẫu và cách lấy sai phân

D. Yes, if nearby values approach one number — Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: B

Giải thích: Quãng đường chia thời gian cho đơn vị vận tốc.

Q15. What determines finite-difference accuracy? — Độ chính xác sai phân hữu hạn phụ thuộc gì?

A. The two-sided limit does not exist — Giới hạn hai phía không tồn tại

B. Distance per unit time — Khoảng cách trên mỗi đơn vị thời gian

C. Sampling spacing and numerical scheme — Khoảng cách mẫu và cách lấy sai phân

D. Yes, if nearby values approach one number — Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: C

Giải thích: Khoảng lấy mẫu ảnh hưởng độ dốc ước lượng.

Q16. Can a function have a finite limit at an undefined point? — Hàm chưa xác định tại một điểm có thể có giới hạn hữu hạn không?

A. The two-sided limit does not exist — Giới hạn hai phía không tồn tại

B. Distance per unit time — Khoảng cách trên mỗi đơn vị thời gian

C. Sampling spacing and numerical scheme — Khoảng cách mẫu và cách lấy sai phân

D. Yes, if nearby values approach one number — Có, nếu giá trị lân cận tiến đến cùng một số

Đáp án: D

Giải thích: Giới hạn có thể có dù hàm không xác định ngay điểm đó.

Q17. How does a derivative differ from an average slope? — Đạo hàm khác độ dốc trung bình thế nào?

A. It describes a local infinitesimal change — Nó mô tả độ thay đổi cục bộ rất nhỏ

B. 2 — 2

C. `sympy.diff` — `sympy.diff`

D. It often uses one-sided differences — Nó thường dùng sai phân một phía

Đáp án: A

Giải thích: Độ dốc tức thời là đại lượng giới hạn cục bộ.

Q18. What is the derivative of 2x+3? — Đạo hàm của 2x+3 là gì?

A. It describes a local infinitesimal change — Nó mô tả độ thay đổi cục bộ rất nhỏ

B. 2 — 2

C. `sympy.diff` — `sympy.diff`

D. It often uses one-sided differences — Nó thường dùng sai phân một phía

Đáp án: B

Giải thích: Hệ số của x cho đạo hàm bằng 2.

Q19. Which tool calculates symbolic derivatives? — Công cụ nào tính đạo hàm ký hiệu?

A. It describes a local infinitesimal change — Nó mô tả độ thay đổi cục bộ rất nhỏ

B. 2 — 2

C. `sympy.diff` — `sympy.diff`

D. It often uses one-sided differences — Nó thường dùng sai phân một phía

Đáp án: C

Giải thích: SymPy hỗ trợ đạo hàm ký hiệu bằng diff.

Q20. Why is a numerical gradient at a boundary less accurate? — Vì sao gradient số ở biên có thể kém chính xác hơn?

A. It describes a local infinitesimal change — Nó mô tả độ thay đổi cục bộ rất nhỏ

B. 2 — 2

C. `sympy.diff` — `sympy.diff`

D. It often uses one-sided differences — Nó thường dùng sai phân một phía

Đáp án: D

Giải thích: Ước lượng ở đầu dãy thường dùng dữ liệu một phía.

8. Bài tiếp theo

`Hàm sô´→ Limit → Derivative → Gradient.`

PY-32 — Đạo hàm riêng, gradient descent và learning rate

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Partial derivative — đạo hàm riêng; **Gradient** — vector gradient.

`Derivative → Gradient → Optimization → Training`

Học xong dùng để làm gì? Cập nhật tham số từng bước để giảm loss, phát hiện hiện tượng học quá nhanh gây dao động.

Kết nối kiến thức tiếp theo: PY-33 dùng Jacobian/Hessian để xem độ nhạy và độ cong trong bài toán nhiều biến.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Partial derivative	đạo hàm riêng	Đạo hàm theo một biến khi các biến khác giữ cố định
Gradient	vector gradient	Vector gồm các đạo hàm riêng của hàm nhiều biến
Learning rate	tốc độ học	Bước nhân gradient khi cập nhật tham số
Epoch	vòng huấn luyện	Một lượt sử dụng hết dữ liệu train để cập nhật mô hình

3. Hàm và cách gọi bằng Python

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / tham số	Giải thích
`sympy.diff(f,x)`	Đạo hàm riêng của hàm f theo x
`grad=[df/dx,df/dy,...]`	Ghép các đạo hàm riêng thành vector
`w = w - lr * gradient`	Cập nhật đi ngược hướng tăng loss
`np.mean((pred-y)**2)`	Loss MSE mẫu cho tối ưu hồi quy

4. Khái niệm chính và liên quan

Mạch kiến thức: Đạo hàm riêng đo độ dốc theo từng tham số; gradient gom các độ dốc; gradient descent dùng hướng ngược gradient với learning rate thích hợp; epoch nói về số lượt đi qua tập train, không mặc định bằng số bước cập nhật.

4.1. Partial derivative — Độ dốc theo một tham số

Từ vựng: Partial derivative — đạo hàm riêng; **Fixed variable** — biến giữ cố định.

Một mô hình có hai tham số x,y và loss $f(x,y)=x^2+3xy+y^2$. **Đạo hàm riêng theo x** giả sử y giữ nguyên, được $2x+3y$, còn theo y là $3x+2y$. Ở điểm $(1,2)$, hai độ dốc lần lượt 8 và 7. Đạo hàm riêng khác đạo hàm hàm một biến vì ta phải chỉ rõ **biến đang thay đổi**.

```
import sympy as sp

x, y = sp.symbols('x y')
f = x ** 2 + 3 * x * y + y ** 2
print(sp.diff(f, x))
```

Kết quả khi chạy:

```
2*x + 3*y
```

Đạo hàm riêng theo x bằng 2x+3y, coi y là hằng số.

4.2. Gradient — Tập hợp độ dốc của tất cả tham số

Từ vựng: Gradient — vector gradient; **Parameter** — tham số mô hình.

Nếu loss phụ thuộc nhiều tham số, **gradient** gom các đạo hàm riêng **theo đúng thứ tự tham số**: $\nabla f=(\partial f/\partial x,\ \partial f/\partial y)$. Với $f=x^2+y^2$ tại $(3,4)$, gradient $[6,8]$ là vector hai chiều. Gradient chỉ **hướng tăng nhanh nhất cục bộ** của f; đi theo **gradient âm** giúp thử giảm loss khi bước đủ nhỏ.

```
import numpy as np

params = np.array([3.0, 4.0])
print((2 * params).tolist())
```

Kết quả khi chạy:

```
[6.0, 8.0]
```

Gradient tại (3,4) có hai thành phần: đạo hàm theo x bằng 6, theo y bằng 8.

4.3. Learning rate — Độ dài bước cập nhật

Từ vựng: Learning rate — tốc độ học; **Update** — cập nhật.

Gradient descent cập nhật tham số bằng $w_new = w - lr \times gradient$. Với $L(w)=w^2$, ta có đạo hàm $2w$. Bắt đầu w=3 và lr=0.1, gradient=6 nên w mới bằng $3-0.1\times6=2.4$. **Learning rate** không phải một đạo hàm; nó quyết định bước đi. Quá lớn có thể dao động/phân kỳ, quá nhỏ có thể rất chậm.

```
w = 3.0
learning_rate = 0.1
w = w - learning_rate * 2 * w
print(round(w, 2))
```

Kết quả khi chạy:

```
2.4
```

Một bước đi ngược gradient đưa tham số từ 3 xuống 2.4, gần cực tiểu w=0 hơn.

4.4. Epoch — Một lượt qua dữ liệu train

Từ vựng: Epoch — một vòng qua tập train; **Step** — một bước cập nhật.

Epoch thường là một lượt xử lý toàn bộ dữ liệu train. Nó **không bằng một bước gradient** trong mọi trường hợp: khi huấn luyện minibatch, một epoch có nhiều batch và nhiều lần cập nhật. Để minh họa vòng lặp, có thể in số thứ tự epoch 1, 2, 3. Với **gradient descent một batch toàn bộ dữ liệu**, mỗi epoch thường tương ứng một lần cập nhật; cách đếm và kiểm tra loss cần nhất quán.

```
for epoch in range(1, 4):
    print(epoch)
```

Kết quả khi chạy:

```
1
2
3
```

Có ba lượt epoch được đánh số 1 đến 3; đây mới là bộ đếm, không phải bản thân việc huấn luyện.

5. Năm tình huống ứng dụng

Tình huống 1 — Từng bước giảm một loss đơn giản

Tình huống thực tế: Một tham số điều khiển mô phỏng w được tối ưu với $L(w)=w^2$ (loss không đơn vị). Bắt đầu $w=4$, learning rate 0.1, thực hiện 10 lần cập nhật và xem w tiến gần 0 ra sao.

Vì sao chọn cách giải này? Với đạo hàm 2w, bước $w \leftarrow w-0.1\times 2w$ làm giảm độ lớn w mỗi lần trong ví dụ lồi này.

Quy trình giải:

1. Đặt w=4 và learning rate 0.1.
2. Lặp 10 lần, mỗi lần lấy gradient=2w rồi trừ bước cập nhật.
3. In w làm tròn bốn chữ số.

```
w = 4.0
lr = 0.1
for _ in range(10):
    w = w - lr * 2 * w
print(round(w, 4))
```

Kết quả khi chạy:

```
0.4295
```

Tham số đi từ 4 xuống khoảng 0.4295, tiến gần cực tiểu 0.

Tình huống 2 — Tìm đạo hàm riêng tại một điểm

Tình huống thực tế: Loss theo hai tham số là $f=x^2+3xy+y^2$, tại x=1,y=2. Đội kỹ thuật muốn biết chỉ thay đổi x làm loss nhạy thế nào nếu giữ y cố định.

Vì sao chọn cách giải này? Lấy đạo hàm riêng theo x rồi thay giá trị hai biến mới đúng ý nghĩa thay một tham số.

Quy trình giải:

1. Khai báo biến x,y và biểu thức f.
2. Tính ∂f/∂x bằng `sp.diff(f,x)`.
3. Thay x=1,y=2 và in độ dốc riêng.

```
import sympy as sp

x, y = sp.symbols('x y')
f = x ** 2 + 3 * x * y + y ** 2
print(sp.diff(f, x).subs({x: 1, y: 2}))
```

Kết quả khi chạy:

```
8
```

Nếu tăng x rất nhỏ quanh (1,2), loss tăng xấp xỉ 8 lần phần thay đổi x.

Tình huống 3 — Theo dõi lịch sử loss khi tối ưu

Tình huống thực tế: Một tham số mô phỏng khởi đầu w=3 với $L(w)=w^2$. Nhóm phát triển muốn ghi loss trước năm lần cập nhật để xem loss có giảm theo từng bước không.

Vì sao chọn cách giải này? Ghi loss trong List cho thấy xu hướng hội tụ thay vì chỉ biết giá trị cuối.

Quy trình giải:

1. Tạo List `history`, đặt w=3 và lr=0.1.
2. Mỗi vòng thêm `round(w*w,3)` vào lịch sử rồi cập nhật w.
3. In năm số loss theo thứ tự.

```
w = 3.0
history = []
for _ in range(5):
    history.append(round(w ** 2, 3))
    w = w - 0.1 * 2 * w
print(history)
```

Kết quả khi chạy:

```
[9.0, 5.76, 3.686, 2.359, 1.51]
```

Loss giảm dần qua năm giá trị, nhưng kết quả này chỉ phản ánh hàm lồi một biến đang mô phỏng.

Tình huống 4 — So bước học nhỏ và quá lớn

Tình huống thực tế: Một mô hình thử tối ưu $L(w)=w^2$ từ w=4, chạy 5 bước với hai learning rate 0.1 và 1.1. Cần kiểm tra cả hai có làm loss giảm như nhau không.

Vì sao chọn cách giải này? Learning rate 1.1 vượt khoảng ổn định của hàm này, làm tham số đổi dấu và độ lớn tăng.

Quy trình giải:

1. Với từng learning rate, đặt lại w=4 để so công bằng.
2. Cập nhật 5 bước bằng gradient 2w.
3. In loss cuối sau làm tròn ba chữ số.

```
for lr in [0.1, 1.1]:
    w = 4.0
    for _ in range(5):
        w = w - lr * 2 * w
    print(lr, round(w ** 2, 3))
```

Kết quả khi chạy:

```
0.1 1.718
1.1 99.068
```

Với lr 0.1, loss còn 1.718; với lr 1.1 loss tăng lên 99.068 dù cùng điểm khởi đầu.

Tình huống 5 — Fit một hệ số tuyến tính bằng gradient

Tình huống thực tế: Dữ liệu ba điểm gồm x=`[1,2,3]` và y=`[2,4,6]`. Mô hình không có hệ số chặn, dự báo $\hat{y}=w*x$. Bắt đầu w=0 và dùng 100 bước cập nhật để khớp hệ số w.

Vì sao chọn cách giải này? Đạo hàm MSE theo w là $2\times\text{mean}(x*(wx-y))$; cập nhật lặp cho hệ số gần 2.

Quy trình giải:

1. Tạo mảng x,y theo đúng cặp quan sát.
2. Đặt w=0, lr=0.1, lặp 100 lần để tính gradient MSE và cập nhật.
3. In w làm tròn ba chữ số.

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 6.0])
w = 0.0
for _ in range(100):
    gradient = 2 * np.mean(x * (w * x - y))
    w = w - 0.1 * gradient
print(round(w, 3))
```

Kết quả khi chạy:

```
2.0
```

Mô hình tìm được hệ số gần 2 cho quan hệ y=2x; chưa có bước đánh giá trên dữ liệu mới.

6. Sai lầm thường gặp

- Đạo hàm riêng chỉ thay một tham số; gradient chứa nhiều đạo hàm riêng theo đúng thứ tự tham số.
- Gradient chỉ hướng **tăng nhanh nhất**, nên gradient descent thường trừ gradient.
- Learning rate quá lớn có thể phân kỳ; quá nhỏ khiến học chậm.
- Một epoch không luôn tương ứng một lần cập nhật vì minibatch có nhiều bước.
- Ghi loss giúp theo dõi tiến triển nhưng loss train giảm chưa bảo đảm kết quả validation tốt.
- Khi tính MSE theo dữ liệu, phải ghép đúng từng mẫu x với nhãn y.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a partial derivative? — Đạo hàm riêng là gì?

A. Change rate in one variable while others are held fixed — Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

B. A vector of partial derivatives — Vector gồm các đạo hàm riêng

C. A factor controlling an optimization update step — Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. One pass through the training dataset — Một lượt đi qua tập dữ liệu train

Đáp án: A

Giải thích: Chỉ thay một biến và giữ biến khác cố định.

Q2. What is a gradient? — Gradient là gì?

A. Change rate in one variable while others are held fixed — Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

B. A vector of partial derivatives — Vector gồm các đạo hàm riêng

C. A factor controlling an optimization update step — Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. One pass through the training dataset — Một lượt đi qua tập dữ liệu train

Đáp án: B

Giải thích: Các đạo hàm riêng hợp thành một vector.

Q3. What is a learning rate? — Learning rate là gì?

A. Change rate in one variable while others are held fixed — Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

B. A vector of partial derivatives — Vector gồm các đạo hàm riêng

C. A factor controlling an optimization update step — Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. One pass through the training dataset — Một lượt đi qua tập dữ liệu train

Đáp án: C

Giải thích: Nó nhân mức độ cập nhật.

Q4. What is an epoch in training? — Epoch trong huấn luyện là gì?

A. Change rate in one variable while others are held fixed — Tốc độ thay đổi theo một biến khi giữ các biến khác cố định

B. A vector of partial derivatives — Vector gồm các đạo hàm riêng

C. A factor controlling an optimization update step — Hệ số điều khiển độ lớn bước cập nhật tối ưu

D. One pass through the training dataset — Một lượt đi qua tập dữ liệu train

Đáp án: D

Giải thích: Một epoch xử lý hết dữ liệu train một lượt.

Q5. Which tool differentiates f with respect to x symbolically? — Hàm nào đạo hàm ký hiệu f theo x?

A. `sp.diff(f,x)` — `sp.diff(f,x)`

B. `w = w - lr*gradient` — `w = w - lr*gradient`

C. `np.mean((pred-y)**2)` — `np.mean((pred-y)**2)`

D. A `for` loop — Vòng lặp `for`

Đáp án: A

Giải thích: diff tính đạo hàm ký hiệu.

Q6. Which expression is a gradient-descent update? — Biểu thức nào cập nhật theo gradient descent?

A. `sp.diff(f,x)` — `sp.diff(f,x)`

B. `w = w - lr*gradient` — `w = w - lr*gradient`

C. `np.mean((pred-y)**2)` — `np.mean((pred-y)**2)`

D. A `for` loop — Vòng lặp `for`

Đáp án: B

Giải thích: Trừ gradient nhằm giảm loss cục bộ.

Q7. Which formula is MSE? — Công thức nào là MSE?

A. `sp.diff(f,x)` — `sp.diff(f,x)`

B. `w = w - lr*gradient` — `w = w - lr*gradient`

C. `np.mean((pred-y)**2)` — `np.mean((pred-y)**2)`

D. A `for` loop — Vòng lặp `for`

Đáp án: C

Giải thích: MSE trung bình bình phương sai số.

Q8. Which Python construct repeats optimization steps? — Cấu trúc Python nào lặp các bước tối ưu?

A. `sp.diff(f,x)` — `sp.diff(f,x)`

B. `w = w - lr*gradient` — `w = w - lr*gradient`

C. `np.mean((pred-y)**2)` — `np.mean((pred-y)**2)`

D. A `for` loop — Vòng lặp `for`

Đáp án: D

Giải thích: for lặp các bước cập nhật.

Q9. What is $\partial(x^2+3xy)/\partial x$? — Đạo hàm riêng theo x của x^2+3xy là gì?

A. `2*x+3*y` — `2*x+3*y`

B. `[2*x,2*y]` — `[2*x,2*y]`

C. `2*w` — `2*w`

D. 2.4 — 2.4

Đáp án: A

Giải thích: Lấy đạo hàm x^2 và xy theo x.

Q10. What is the gradient of x^2+y^2? — Gradient của x^2+y^2 là gì?

A. `2*x+3*y` — `2*x+3*y`

B. `[2*x,2*y]` — `[2*x,2*y]`

C. `2*w` — `2*w`

D. 2.4 — 2.4

Đáp án: B

Giải thích: Mỗi đạo hàm bằng hai lần tọa độ.

Q11. For loss w^2, what is its derivative? — Đạo hàm loss w^2 là gì?

A. `2*x+3*y` — `2*x+3*y`

B. `[2*x,2*y]` — `[2*x,2*y]`

C. `2*w` — `2*w`

D. 2.4 — 2.4

Đáp án: C

Giải thích: Dùng quy tắc lũy thừa cho w^2.

Q12. If w=3, lr=0.1 and grad=6, what is next w? — Nếu w=3, lr=0.1 và gradient=6, w mới là gì?

A. `2*x+3*y` — `2*x+3*y`

B. `[2*x,2*y]` — `[2*x,2*y]`

C. 2*w — 2*w

D. 2.4 — 2.4

Đáp án: D

Giải thích: 3−0.1×6=2.4.

Q13. Why might a large learning rate fail? — Vì sao learning rate lớn dễ thất bại?

A. Updates can overshoot or diverge — Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

B. It can make convergence slow — Nó có thể làm hội tụ chậm

C. Locally reduce the objective — Giảm hàm mục tiêu về mặt cục bộ

D. No, minibatches can cause multiple updates — Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: A

Giải thích: Bước lớn có thể vượt qua cực tiểu liên tục.

Q14. Why might a very small learning rate be inefficient? — Vì sao learning rate quá nhỏ kém hiệu quả?

A. Updates can overshoot or diverge — Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

B. It can make convergence slow — Nó có thể làm hội tụ chậm

C. Locally reduce the objective — Giảm hàm mục tiêu về mặt cục bộ

D. No, minibatches can cause multiple updates — Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: B

Giải thích: Bước quá nhỏ khiến tối ưu chậm.

Q15. What does moving opposite the gradient aim to do? — Đi ngược gradient nhằm làm gì?

A. Updates can overshoot or diverge — Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

B. It can make convergence slow — Nó có thể làm hội tụ chậm

C. Locally reduce the objective — Giảm hàm mục tiêu về mặt cục bộ

D. No, minibatches can cause multiple updates — Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: C

Giải thích: Gradient âm là hướng giảm với bước đủ nhỏ.

Q16. Does one epoch always equal one update? — Một epoch luôn bằng một lần cập nhật không?

A. Updates can overshoot or diverge — Bước cập nhật có thể vượt cực tiểu hoặc phân kỳ

B. It can make convergence slow — Nó có thể làm hội tụ chậm

C. Locally reduce the objective — Giảm hàm mục tiêu về mặt cục bộ

D. No, minibatches can cause multiple updates — Không, nhiều minibatch có thể tạo nhiều lần cập nhật

Đáp án: D

Giải thích: Một epoch có thể gồm nhiều batch.

Q17. Why record loss history? — Vì sao lưu lịch sử loss?

A. To inspect optimization progress — Để xem tiến trình tối ưu

B. Each error must use its correct sample — Mỗi sai số cần đúng cặp quan sát

C. No, generalization needs validation — Không, cần kiểm chứng trên dữ liệu mới

D. Steepest increase — Hướng tăng nhanh nhất

Đáp án: A

Giải thích: Lịch sử cho thấy xu hướng giảm hoặc bất ổn.

Q18. Why pair features and targets in the same order? — Vì sao cần ghép x và y cùng thứ tự?

A. To inspect optimization progress — Để xem tiến trình tối ưu

B. Each error must use its correct sample — Mỗi sai số cần đúng cặp quan sát

C. No, generalization needs validation — Không, cần kiểm chứng trên dữ liệu mới

D. Steepest increase — Hướng tăng nhanh nhất

Đáp án: B

Giải thích: Hàng ghép sai làm sai số vô nghĩa.

Q19. Does falling training loss guarantee test accuracy? — Loss train giảm có đảm bảo độ chính xác test?

A. To inspect optimization progress — Để xem tiến trình tối ưu

B. Each error must use its correct sample — Mỗi sai số cần đúng cặp quan sát

C. No, generalization needs validation — Không, cần kiểm chứng trên dữ liệu mới

D. Steepest increase — Hướng tăng nhanh nhất

Đáp án: C

Giải thích: Cần dữ liệu độc lập để đo tổng quát hóa.

Q20. Which direction does a gradient point locally? — Gradient hướng theo phía nào về mặt cục bộ?

A. To inspect optimization progress — Để xem tiến trình tối ưu

B. Each error must use its correct sample — Mỗi sai số cần đúng cặp quan sát

C. No, generalization needs validation — Không, cần kiểm chứng trên dữ liệu mới

D. Steepest increase — Hướng tăng nhanh nhất

Đáp án: D

Giải thích: Gradient hướng tăng cục bộ nhanh nhất.

8. Bài tiếp theo

`Derivative → Gradient → Optimization → Training.`

PY-33 — Jacobian, Hessian và điểm cực trị

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Jacobian — ma trận Jacobian; **Hessian** — ma trận Hessian.

`Gradient → Jacobian → Hessian → Tối ưu mô hình`

Học xong dùng để làm gì? Phân tích nhiều đầu ra thay đổi theo nhiều đầu vào và độ cong của hàm mục tiêu.

Kết nối kiến thức tiếp theo: PY-34 mô hình hóa dữ liệu và sự kiện không chắc chắn bằng xác suất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Jacobian	ma trận Jacobian	Bảng đạo hàm riêng bậc 1 của hàm nhiều đầu ra
Hessian	ma trận Hessian	Bảng đạo hàm riêng bậc hai của hàm vô hướng
Critical point	điểm tới hạn	Điểm có gradient bằng 0 hoặc không xác định
Curvature	độ cong	Cách độ dốc biến thiên quanh vị trí đang xét

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`sympy.Matrix([...]).jacobian(vars)`	Đạo hàm riêng hàm vector theo biến
`sympy.hessian(f,vars)`	Đạo hàm bậc hai của hàm vô hướng
`sympy.solve(equations,vars)`	Tìm điểm thỏa hệ điều kiện
`np.linalg.eigvalsh(H)`	Xét dấu eigenvalue Hessian đối xứng tại điểm cực trị

4. Khái niệm chính và liên quan

Mạch kiến thức: Với hàm có nhiều đầu ra, Jacobian ghi độ nhạy từng đầu ra theo các biến đầu vào. Với hàm loss một đầu ra, Hessian ghi độ cong theo cặp tham số; kết hợp gradient bằng 0 và dấu trị riêng Hessian mới giúp phân biệt cực trị với yên ngựa.

4.1. Jacobian — Đầu ra thay đổi theo từng đầu vào

Từ vựng: Jacobian — ma trận Jacobian; **Partial derivative** — đạo hàm riêng.

Khi một hàm có **n đầu vào và m đầu ra**, Jacobian là bảng kích thước **m hàng đầu ra × n cột đầu vào**, tại mỗi ô là đạo hàm riêng của đầu ra hàng đó theo đầu vào cột đó. Với F1=x+y, F2=xy, các hàng lần lượt đại diện F1, F2; các cột theo x,y. Tại (x,y)=(3,2), Jacobian bằng [[1,1],[2,3]]. Đây là độ nhạy cục bộ, không phải ma trận số liệu gốc.

```
import sympy as sp

x, y = sp.symbols('x y')
F = sp.Matrix([x + y, x * y])
J = F.jacobian([x, y])
print(J.subs({x: 3, y: 2}).tolist())
```

Kết quả khi chạy:

```
[[1, 1], [2, 3]]
```

Hàng 1 là độ nhạy của F1 theo x,y; hàng 2 là độ nhạy của F2 theo x,y.

4.2. Hessian — Độ thay đổi của gradient

Từ vựng: Hessian — ma trận Hessian; **Second derivative** — đạo hàm bậc hai.

Hessian là ma trận các đạo hàm riêng **bậc hai** của một hàm **đầu ra vô hướng**. Hàm loss $L(x,y)=x^2+2y^2$ có Hessian [[2,0],[0,4]]. Hàng/cột đều tương ứng thứ tự **tham số x, y**; số 4 theo y lớn hơn 2 theo x nghĩa là độ cong theo y lớn hơn trong mô hình này. Khác Jacobian cho hàm nhiều đầu ra, Hessian giúp xem gradient đổi nhanh thế nào.

```
import sympy as sp

x, y = sp.symbols('x y')
loss = x ** 2 + 2 * y ** 2
print(sp.hessian(loss, [x, y]).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [0, 4]]
```

Hàm cong mạnh hơn theo y; hai đạo hàm chéo bằng 0.

4.3. Critical point — Điểm cần kiểm tra cực trị

Từ vựng: Critical point — điểm tới hạn; **Stationary point** — điểm dừng.

Điểm dừng của hàm khả vi có gradient bằng vector 0, tức không còn hướng tăng tuyến tính ngay tại điểm. Với $L(x,y)=(x-3)^2+(y-2)^2$, các đạo hàm $2(x-3)$, $2(y-2)$ đều bằng 0 khi (3,2). Đây là **ứng viên cực trị**, chưa đủ để nói là cực tiểu: hàm hình yên ngựa cũng có thể có gradient 0. Khái niệm critical point rộng hơn còn có điểm không khả vi ở miền trong.

```
import sympy as sp

x, y = sp.symbols('x y')
loss = (x - 3) ** 2 + (y - 2) ** 2
solution = sp.solve([sp.diff(loss, x), sp.diff(loss, y)], [x, y])
print(solution[x], solution[y])
```

Kết quả khi chạy:

```
3 2
```

Gradient bằng 0 tại (3,2); cần xét độ cong để phân loại điểm này.

4.4. Curvature — Phân biệt cực tiểu, cực đại, yên ngựa

Từ vựng: Curvature — độ cong; **Eigenvalue** — trị riêng.

Tại một **điểm dừng** của hàm đủ trơn, dấu trị riêng Hessian giúp phân loại: **tất cả dương** → cực tiểu địa phương nghiêm ngặt; **tất cả âm** → cực đại địa phương nghiêm ngặt; **vừa dương vừa âm** → điểm yên ngựa. Ví dụ x^2-y^2 có Hessian `diag(2,–2)`, nên tại (0,0) đi theo x hàm tăng nhưng đi theo y hàm giảm. Nếu có trị riêng bằng 0, phép thử Hessian đơn giản **chưa kết luận**.

```
import numpy as np

H = np.array([[2.0, 0.0], [0.0, -2.0]])
print(np.linalg.eigvalsh(H))
```

Kết quả khi chạy:

```
[-2.  2.]
```

Một trị riêng âm, một dương nên Hessian bất định; (0,0) là điểm yên ngựa của x^2-y^2.

5. Năm tình huống ứng dụng

Tình huống 1 — Độ nhạy của hai đầu ra cảm biến

Tình huống thực tế: Hai chỉ số đầu ra của mô hình cảm biến là `F1=x+y`, `F2=xy`; x và y là hai đầu vào đã được chuẩn hóa cùng thang. Tại x=3,y=2, cần biết mỗi đầu ra nhạy ra sao khi một đầu vào thay đổi nhỏ.

Vì sao chọn cách giải này? Jacobian đo đạo hàm riêng của từng đầu ra theo từng đầu vào ở điểm đang xét.

Quy trình giải:

1. Khai báo hai đầu vào và hai biểu thức đầu ra.
2. Tạo Jacobian với cột theo [x,y].
3. Thay x=3,y=2 và in các hệ số độ nhạy.

```
import sympy as sp

x, y = sp.symbols('x y')
```

```
F = sp.Matrix([x + y, x * y])
J = F.jacobian([x, y])
print(J.subs({x: 3, y: 2}).tolist())
```

Kết quả khi chạy:

```
[[1, 1], [2, 3]]
```

Đầu ra F2 nhạy 2 theo x và 3 theo y tại điểm xét, khác F1 luôn nhạy 1 theo mỗi biến.

Tình huống 2 — Độ cong của loss hai tham số

Tình huống thực tế: Loss mô hình là $L=x^2+2y^2$ với x,y không đơn vị; đội phát triển muốn so độ cong hai hướng thay đổi tham số để chọn cách tối ưu.

Vì sao chọn cách giải này? Hessian chứa đạo hàm riêng bậc hai, so được mức cong theo hai tham số.

Quy trình giải:

1. Định nghĩa hàm loss hai biến.
2. Lấy Hessian theo thứ tự [x,y].
3. In ma trận 2×2, so các ô đường chéo.

```
import sympy as sp

x, y = sp.symbols('x y')
loss = x ** 2 + 2 * y ** 2
print(sp.hessian(loss, [x, y]).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [0, 4]]
```

Mức cong theo y bằng 4, lớn hơn mức cong theo x bằng 2.

Tình huống 3 — Tìm bộ tham số làm loss nhỏ nhất

Tình huống thực tế: Một mô hình có hàm loss $L=(x-3)^2+(y-2)^2$. Cần tìm hai tham số tại đó loss không còn giảm theo đạo hàm riêng bậc nhất.

Vì sao chọn cách giải này? Giải hệ gradient bằng 0 cho ứng viên tối ưu, rồi dùng dạng tổng bình phương để xác nhận cực tiểu.

Quy trình giải:

1. Lấy đạo hàm L theo x và y.
2. Giải đồng thời hai phương trình bằng 0.
3. In tọa độ bộ tham số nghiệm.

```
import sympy as sp

x, y = sp.symbols('x y')
loss = (x - 3) ** 2 + (y - 2) ** 2
```

```
solution = sp.solve([sp.diff(loss, x), sp.diff(loss, y)], [x, y])
print(solution[x], solution[y])
```

Kết quả khi chạy:

```
3 2
```

Điểm (3,2) là cực tiểu toàn cục của tổng hai bình phương không âm.

Tình huống 4 — Phân biệt điểm yên ngựa với cực tiểu

Tình huống thực tế: Loss thử nghiệm $L=x^2-y^2$ có điểm dừng tại (0,0). Tuy nhiên theo trục x hàm tăng, theo trục y hàm giảm; nhóm kỹ thuật cần biết có phải cực tiểu không.

Vì sao chọn cách giải này? Trị riêng Hessian trái dấu là bằng chứng điểm dừng dạng yên ngựa.

Quy trình giải:

1. Tạo Hessian `[[2,0],[0,-2]]` tại điểm (0,0).
2. Tính trị riêng đối xứng bằng `eigvalsh`.
3. Kiểm tra có cả trị riêng âm và dương, in kết luận.

```
import numpy as np

H = np.array([[2.0, 0.0], [0.0, -2.0]])
values = np.linalg.eigvalsh(H)
print(bool((values < 0).any() and (values > 0).any()))
```

Kết quả khi chạy:

```
True
```

Hessian bất định; điểm dừng này là yên ngựa, không phải cực tiểu.

Tình huống 5 — Kiểm tra kích thước Jacobian

Tình huống thực tế: Mô hình có ba đầu vào x,y,z và hai đầu ra `F1=x+y+z`, `F2=x*y`. Cần biết ma trận độ nhạy của mô hình có bao nhiêu hàng và cột.

Vì sao chọn cách giải này? Jacobian có một hàng cho mỗi đầu ra và một cột cho mỗi đầu vào; với 2 đầu ra, 3 đầu vào có shape (2,3).

Quy trình giải:

1. Khai báo ba đầu vào x,y,z.
2. Tạo hai đầu ra thành SymPy Matrix.
3. Tính Jacobian theo [x,y,z], in shape.

```
import sympy as sp

x, y, z = sp.symbols('x y z')
F = sp.Matrix([x + y + z, x * y])
print(F.jacobian([x, y, z]).shape)
```

Kết quả khi chạy:

```
(2, 3)
```

Jacobian có hai hàng đầu ra và ba cột đầu vào, không nhất thiết là ma trận vuông.

6. Sai lầm thường gặp

- Jacobian có số hàng bằng số đầu ra và số cột bằng số đầu vào; không luôn là ma trận vuông.
- Hessian dùng cho hàm vô hướng đủ khả vi hai lần; hàng/cột là các tham số, không phải hai đầu ra.
- Gradient bằng 0 mới chỉ chỉ ra điểm dừng, chưa đủ nói là cực tiểu.
- Hessian có trị riêng vừa âm vừa dương ở điểm dừng chỉ ra yên ngựa; nếu có trị riêng 0, bài kiểm bậc hai đơn giản chưa đủ.
- Khi dùng `eigvalsh`, ma trận đầu vào phải đối xứng theo phép toán đang xét.
- SymPy là thư viện ngoài; các ví dụ ký hiệu cần cài đặt.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What does a Jacobian contain? — Jacobian chứa gì?

A. First partial derivatives of vector outputs — Các đạo hàm riêng bậc một của đầu ra vector

B. Second partial derivatives of a scalar output — Các đạo hàm riêng bậc hai của đầu ra vô hướng

C. A differentiable point where the gradient is zero — Điểm khả vi có gradient bằng 0

D. Classifying local behavior near stationary points — Phân loại hành vi cục bộ gần điểm dừng

Đáp án: A

Giải thích: Mỗi hàng chứa đạo hàm bậc một của một đầu ra.

Q2. What does a Hessian contain? — Hessian chứa gì?

A. First partial derivatives of vector outputs — Các đạo hàm riêng bậc một của đầu ra vector

B. Second partial derivatives of a scalar output — Các đạo hàm riêng bậc hai của đầu ra vô hướng

C. A differentiable point where the gradient is zero — Điểm khả vi có gradient bằng 0

D. Classifying local behavior near stationary points — Phân loại hành vi cục bộ gần điểm dừng

Đáp án: B

Giải thích: Hessian lấy đạo hàm tiếp của gradient.

Q3. What is a stationary point? — Điểm dừng là gì?

A. First partial derivatives of vector outputs — Các đạo hàm riêng bậc một của đầu ra vector

B. Second partial derivatives of a scalar output — Các đạo hàm riêng bậc hai của đầu ra vô hướng

C. A differentiable point where the gradient is zero — Điểm khả vi có gradient bằng 0

D. Classifying local behavior near stationary points — Phân loại hành vi cục bộ gần điểm dừng

Đáp án: C

Giải thích: Điểm dừng có các đạo hàm riêng bậc một bằng 0.

Q4. What is curvature information used for? — Thông tin độ cong dùng để làm gì?

A. First partial derivatives of vector outputs — Các đạo hàm riêng bậc một của đầu ra vector

B. Second partial derivatives of a scalar output — Các đạo hàm riêng bậc hai của đầu ra vô hướng

C. A differentiable point where the gradient is zero — Điểm khả vi có gradient bằng 0

D. Classifying local behavior near stationary points — Phân loại hành vi cục bộ gần điểm dừng

Đáp án: D

Giải thích: Độ cong giúp phân loại điểm dừng.

Q5. What is Jacobian shape for m outputs and n inputs? — Jacobian m đầu ra và n đầu vào có shape nào?

A. `(m,n)` — `(m,n)`

B. `.jacobian(variables)` — `.jacobian(variables)`

C. `sp.hessian(f, variables)` — `sp.hessian(f, variables)`

D. `np.linalg.eigvalsh(H)` — `np.linalg.eigvalsh(H)`

Đáp án: A

Giải thích: Mỗi đầu ra là một hàng, mỗi đầu vào là một cột.

Q6. Which SymPy method builds a Jacobian? — Hàm SymPy nào tạo Jacobian?

A. `(m,n)` — `(m,n)`

B. `.jacobian(variables)` — `.jacobian(variables)`

C. `sp.hessian(f, variables)` — `sp.hessian(f, variables)`

D. `np.linalg.eigvalsh(H)` — `np.linalg.eigvalsh(H)`

Đáp án: B

Giải thích: Phương thức lấy mọi đạo hàm riêng bậc một.

Q7. Which SymPy function builds a Hessian? — Hàm SymPy nào tạo Hessian?

A. `(m,n)` — `(m,n)`

B. `.jacobian(variables)` — `.jacobian(variables)`

C. `sp.hessian(f, variables)` — `sp.hessian(f, variables)`

D. `np.linalg.eigvalsh(H)` — `np.linalg.eigvalsh(H)`

Đáp án: C

Giải thích: hessian tính các đạo hàm riêng bậc hai.

Q8. Which tool computes symmetric Hessian eigenvalues? — Hàm nào tính trị riêng Hessian đối xứng?

A. `(m,n)` — `(m,n)`

B. `.jacobian(variables)` — `.jacobian(variables)`

C. `sp.hessian(f, variables)` — `sp.hessian(f, variables)`

D. `np.linalg.eigvalsh(H)` — `np.linalg.eigvalsh(H)`

Đáp án: D

Giải thích: eigvalsh tính trị riêng ma trận đối xứng.

Q9. What is Hessian of x^2+y^2? — Hessian của x^2+y^2 là gì?

A. `[[2,0],[0,2]]` — `[[2,0],[0,2]]`

B. `[[2,0],[0,-2]]` — `[[2,0],[0,-2]]`

C. `(3,2)` — `(3,2)`

D. `[[1,1],[y,x]]` — `[[1,1],[y,x]]`

Đáp án: A

Giải thích: Đạo hàm bậc hai của mỗi hạng bình phương bằng 2.

Q10. What is Hessian of x^2-y^2? — Hessian của x^2-y^2 là gì?

A. `[[2,0],[0,2]]` — `[[2,0],[0,2]]`

B. `[[2,0],[0,-2]]` — `[[2,0],[0,-2]]`

C. `(3,2)` — `(3,2)`

D. `[[1,1],[y,x]]` — `[[1,1],[y,x]]`

Đáp án: B

Giải thích: Độ cong theo y bằng âm 2.

Q11. What are stationary coordinates for $(x-3)^2+(y-2)^2$? — Tọa độ dừng của $(x-3)^2+(y-2)^2$ là gì?

A. `[[2,0],[0,2]]` — `[[2,0],[0,2]]`

B. `[[2,0],[0,-2]]` — `[[2,0],[0,-2]]`

C. `(3,2)` — `(3,2)`

D. `[[1,1],[y,x]]` — `[[1,1],[y,x]]`

Đáp án: C

Giải thích: Cả hai bình phương nhỏ nhất khi sai khác bằng 0.

Q12. What is Jacobian of outputs x+y and xy? — Jacobian của đầu ra x+y và xy là gì?

A. `[[2,0],[0,2]]` — `[[2,0],[0,2]]`

B. `[[2,0],[0,-2]]` — `[[2,0],[0,-2]]`

C. `(3,2)` — `(3,2)`

D. `[[1,1],[y,x]]` — `[[1,1],[y,x]]`

Đáp án: D

Giải thích: Lấy đạo hàm mỗi đầu ra theo x và y.

Q13. What does a positive-definite Hessian at a stationary point indicate? — Hessian xác định dương tại điểm dừng biểu thị gì?

A. A strict local minimum — Cực tiểu địa phương nghiêm ngặt

B. A strict local maximum — Cực đại địa phương nghiêm ngặt

C. A saddle point — Điểm yên ngựa

D. The simple second-derivative test is inconclusive — Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: A

Giải thích: Độ cong dương mọi hướng cho cực tiểu địa phương nghiêm ngặt.

Q14. What does a negative-definite Hessian at a stationary point indicate? — Hessian xác định âm tại điểm dừng biểu thị gì?

A. A strict local minimum — Cực tiểu địa phương nghiêm ngặt

B. A strict local maximum — Cực đại địa phương nghiêm ngặt

C. A saddle point — Điểm yên ngựa

D. The simple second-derivative test is inconclusive — Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: B

Giải thích: Độ cong âm mọi hướng cho cực đại địa phương nghiêm ngặt.

Q15. What do mixed-sign Hessian eigenvalues indicate at a stationary point? — Trị riêng Hessian trái dấu ở điểm dừng biểu thị gì?

A. A strict local minimum — Cực tiểu địa phương nghiêm ngặt

B. A strict local maximum — Cực đại địa phương nghiêm ngặt

C. A saddle point — Điểm yên ngựa

D. The simple second-derivative test is inconclusive — Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: C

Giải thích: Một số hướng tăng và hướng khác giảm.

Q16. What if a Hessian has a zero eigenvalue at a stationary point? — Nếu Hessian có trị riêng 0 tại điểm dừng thì sao?

A. A strict local minimum — Cực tiểu địa phương nghiêm ngặt

B. A strict local maximum — Cực đại địa phương nghiêm ngặt

C. A saddle point — Điểm yên ngựa

D. The simple second-derivative test is inconclusive — Phép kiểm đạo hàm bậc hai đơn giản chưa kết luận

Đáp án: D

Giải thích: Trị riêng 0 cần kiểm tra sâu hơn.

Q17. What is Hessian shape for a scalar function of three inputs? — Hessian hàm vô hướng ba biến có shape nào?

A. `(3,3)` — `(3,3)`

B. Sensitivities of one output to each input — Độ nhạy của một đầu ra theo từng đầu vào

C. How each output responds to one input — Các đầu ra đáp ứng với một đầu vào

D. It can also occur at a maximum or saddle — Nó còn xuất hiện ở cực đại hoặc yên ngựa

Đáp án: A

Giải thích: Các đạo hàm riêng bậc hai ghép từng biến với biến khác.

Q18. What does one Jacobian row represent? — Một hàng Jacobian biểu thị gì?

A. `(3,3)` — `(3,3)`

B. Sensitivities of one output to each input — Độ nhạy của một đầu ra theo từng đầu vào

C. How each output responds to one input — Các đầu ra đáp ứng với một đầu vào

D. It can also occur at a maximum or saddle — Nó còn xuất hiện ở cực đại hoặc yên ngựa

Đáp án: B

Giải thích: Hàng cố định đầu ra, cột đổi đầu vào.

Q19. What does one Jacobian column represent? — Một cột Jacobian biểu thị gì?

A. `(3,3)` — `(3,3)`

B. Sensitivities of one output to each input — Độ nhạy của một đầu ra theo từng đầu vào

C. How each output responds to one input — Các đầu ra đáp ứng với một đầu vào

D. It can also occur at a maximum or saddle — Nó còn xuất hiện ở cực đại hoặc yên ngựa

Đáp án: C

Giải thích: Cột cố định một đầu vào qua mọi đầu ra.

Q20. Why isn't zero gradient alone sufficient to prove a minimum? — Vì sao gradient 0 chưa đủ chứng minh cực tiểu?

A. `(3,3)` — `(3,3)`

B. Sensitivities of one output to each input — Độ nhạy của một đầu ra theo từng đầu vào

C. How each output responds to one input — Các đầu ra đáp ứng với một đầu vào

D. It can also occur at a maximum or saddle — Nó còn xuất hiện ở cực đại hoặc yên ngựa

Đáp án: D

Giải thích: Cực đại và yên ngựa cũng có thể có gradient 0.

8. Liên hệ bài tiếp theo

`Gradient → Jacobian → Hessian → Tối ưu mô hình.`

PY-34 — Xác suất, không gian mẫu và biến cố

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Experiment — phép thử; **Sample space** — không gian mẫu.

`Logic/Tập hợp → Xác suất → Bayes → Dự đoán bất định`

Học xong dùng để làm gì? Tính xác suất một sự kiện xảy ra và kiểm tra quy tắc của không gian mẫu.

Kết nối kiến thức tiếp theo: PY-35 tính xác suất khi đã có thông tin điều kiện và cập nhật bằng Bayes.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Experiment	phép thử	Thao tác có kết quả quan sát được
Sample space	không gian mẫu	Tập mọi kết quả khả dĩ
Event	biến cố	Nhóm kết quả được quan tâm
Probability	xác suất	Mức khả năng xảy ra một biến cố, từ 0 tới 1

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`len(event)/len(space)`	Xác suất cổ điển khi kết quả đồng khả năng
`random.choice(values)`	Rút một phần tử ngẫu nhiên từ danh sách
`np.random.default_rng(seed)`	Tạo bộ sinh ngẫu nhiên tái lập
`np.mean(condition)`	Ước lượng tỷ lệ đúng trong mô phỏng

4. Khái niệm chính và liên quan

Mạch kiến thức: Từ một phép thử ta xác định không gian mẫu chứa mọi kết quả, chọn một tập con là biến cố, rồi mới tính xác suất dựa trên giả định đồng khả năng hoặc dựa vào phân phối đã biết. Mô phỏng cho tần suất quan sát, không thay thế xác suất lý thuyết.

4.1. Experiment — Phép thử và kết quả ngẫu nhiên

Từ vựng: Experiment — phép thử; **Outcome** — kết quả.

Phép thử ngẫu nhiên là thao tác có những kết quả khả dĩ biết trước, nhưng không biết chắc kết quả của **một lần thử**. Tung một đồng xu công bằng là ví dụ: kết quả có thể là ngửa hoặc sấp; mỗi lần chỉ xuất hiện một mặt. Để minh họa mà không in kết quả thất thường, tạo số ngẫu nhiên bằng `np.random.default_rng(seed)` giúp **tái lập cùng chuỗi giả ngẫu nhiên**, nhưng seed không biến thử nghiệm thành không ngẫu nhiên trong lý thuyết.

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.integers(0, 2, size=5).tolist())
```

Kết quả khi chạy:

```
[0, 1, 1, 0, 0]
```

Năm kết quả mô phỏng 0/1 với seed 42 tái lập được trong phiên bản NumPy tương thích.

4.2. Sample space — Liệt kê toàn bộ kết quả có thể

Từ vựng: Sample space — không gian mẫu; **Outcome** — phần tử mẫu.

Không gian mẫu Ω chứa toàn bộ kết quả có thể của phép thử. Với xúc xắc sáu mặt chuẩn, `Ω={1,2,3,4,5,6}`; mỗi số chỉ là **một kết quả**, chưa phải xác suất. Đếm `len(Ω)=6` cho biết số trường hợp khả dĩ. Chỉ khi xúc xắc **công bằng** ta mới giả định cả sáu mặt đồng xác suất `1/6`. Set không bảo đảm thứ tự hiển thị nên in qua `sorted()`.

```
space = set(range(1, 7))
print(sorted(space))
```

Kết quả khi chạy:

```
[1, 2, 3, 4, 5, 6]
```

Không gian mẫu của một lần gieo có sáu mặt, được sắp thứ tự để dễ đọc.

4.3. Event — Một nhóm kết quả quan tâm

Từ vựng: Event — biến cố; **Subset** — tập con.

Biến cố là một tập con của không gian mẫu. Khi gieo xúc xắc, biến cố `E = ra số chẵn` gồm `{2,4,6}`. Nó không phải một kết quả duy nhất: trong một lần gieo, E xảy ra nếu số được gieo **thuộc** Set E. Gọi `sorted(E)` để hiển thị rõ ba trường hợp; phép kiểm tra `4 in E` cho True.

```
even = {2, 4, 6}
print(sorted(even))
```

Kết quả khi chạy:

```
[2, 4, 6]
```

Biến cố ra số chẵn có ba kết quả thuận lợi trong sáu mặt xúc xắc.

4.4. Probability — Tỷ lệ khả năng dưới giả định

Từ vựng: Probability — xác suất; **Equally likely** — đồng khả năng.

Xác suất P(E) là số trong `[0,1]` đo khả năng của biến cố E. Khi các phần tử không gian mẫu **đồng khả năng**, `P(E)=sô´kêt quả thuận lợi / sô´kêt quả có thê.̂` Với xúc xắc công bằng, E là các mặt chẵn `{2,4,6}` nên `P(E)=3/6=0.5`. Nếu xúc xắc lệch, không thể dùng số mặt thuận lợi chia số mặt một cách máy móc: cần xác suất của từng mặt.

```
space = set(range(1, 7))
even = {2, 4, 6}
print(len(even) / len(space))
```

Kết quả khi chạy:

```
0.5
```

Một lần gieo xúc xắc công bằng có xác suất 50% ra mặt chẵn.

5. Năm tình huống ứng dụng

Tình huống 1 — Xác suất ra mặt chẵn của xúc xắc

Tình huống thực tế: Trò chơi sử dụng xúc xắc công bằng có sáu mặt đánh số 1 tới 6; người chơi thắng nếu ra mặt chẵn. Tính xác suất thắng trong một lần gieo.

Vì sao chọn cách giải này? Sáu mặt đồng khả năng nên xác suất cổ điển là số mặt chẵn chia sáu.

Quy trình giải:

1. Liệt kê không gian mẫu sáu mặt và biến cố {2,4,6}.
2. Đếm số kết quả thuận lợi, số kết quả có thể.
3. Chia hai số và in xác suất.

```
space = set(range(1, 7))
even = {2, 4, 6}
print(len(even) / len(space))
```

Kết quả khi chạy:

```
0.5
```

Xác suất thắng bằng 0.5 (50%) theo giả định xúc xắc công bằng.

Tình huống 2 — Mô phỏng 1.000 lần tung đồng xu

Tình huống thực tế: Phòng thực hành mô phỏng 1.000 lần tung đồng xu công bằng, mã 1 là ngửa, mã 0 là sấp. Cần xem tỷ lệ ngửa quan sát được và hiểu vì sao nó có thể khác 0.5.

Vì sao chọn cách giải này? `default_rng(seed)` giúp kết quả tái lập và trung bình của dãy 0/1 bằng tỷ lệ mặt ngửa.

Quy trình giải:

1. Khởi tạo RNG seed 42.
2. Sinh 1.000 số 0 hoặc 1.
3. Tính tỷ lệ số 1 và làm tròn ba chữ số.

```
import numpy as np

rng = np.random.default_rng(42)
tosses = rng.integers(0, 2, size=1000)
print(round(float(tosses.mean()), 3))
```

Kết quả khi chạy:

```
0.509
```

Tỷ lệ mô phỏng khoảng 50.9%, không bắt buộc bằng xác suất lý thuyết 50%.

Tình huống 3 — Xác suất tổng hai xúc xắc bằng chín

Tình huống thực tế: Hai xúc xắc công bằng được gieo độc lập. Mỗi kết quả là **cặp có thứ tự** `(a,b)`, mỗi giá trị từ 1 đến 6. Người chơi thắng nếu tổng hai mặt bằng 9.

Vì sao chọn cách giải này? Hai xúc xắc cho 6×6=36 cặp đồng khả năng, trong đó có bốn cặp cộng thành 9.

Quy trình giải:

1. Liệt kê mọi cặp a,b bằng hai vòng for.
2. Đếm những cặp có tổng bằng 9.
3. Chia cho 36 và làm tròn ba chữ số.

```
pairs = [(a, b) for a in range(1, 7) for b in range(1, 7)]
valid = sum(a + b == 9 for a, b in pairs)
print(round(valid / len(pairs), 3))
```

Kết quả khi chạy:

```
0.111
```

Có bốn cặp thắng trên 36 cặp, xác suất bằng khoảng 11.1%.

Tình huống 4 — Ước lượng tỷ lệ sự kiện hiếm

Tình huống thực tế: Một ứng dụng giả lập 10.000 lượt, mỗi lượt có xác suất 0.2 phát sinh sự kiện và các lượt độc lập. Đội phân tích cần biết tỷ lệ thực tế trong một lần mô phỏng.

Vì sao chọn cách giải này? So sánh số ngẫu nhiên đều với 0.2 mô phỏng Bernoulli; trung bình Boolean là tỷ lệ sự kiện xảy ra.

Quy trình giải:

1. Khởi tạo RNG seed 1, sinh 10.000 số trong [0,1).

2. Đánh dấu sự kiện khi số nhỏ hơn 0.2.
3. Lấy trung bình và làm tròn ba chữ số.

```
import numpy as np

rng = np.random.default_rng(1)
events = rng.random(10000) < 0.2
print(round(float(events.mean()), 3))
```

Kết quả khi chạy:

```
0.2
```

Tỷ lệ mô phỏng gần 20% theo giả định; seed cho kết quả có thể tái lập.

Tình huống 5 — So mô phỏng ít lượt và nhiều lượt

Tình huống thực tế: Một đồng xu công bằng được mô phỏng hai nhóm liên tiếp: 100 lượt rồi 10.000 lượt, cùng bộ sinh seed 7. Cần so độ chênh lệch tỷ lệ mặt ngửa so với 0.5.

Vì sao chọn cách giải này? Số lượt lớn thường cho ước lượng ổn định hơn về mặt thống kê, nhưng không đảm bảo từng lần chạy đều gần hơn.

Quy trình giải:

1. Khởi tạo một RNG cố định seed 7.
2. Lần lượt sinh 100 và 10.000 lần, mỗi lần tính tỷ lệ mã 1.
3. In kích thước và tỷ lệ làm tròn ba chữ số.

```
import numpy as np

rng = np.random.default_rng(7)
for count in [100, 10000]:
    tosses = rng.integers(0, 2, size=count)
    print(count, round(float(tosses.mean()), 3))
```

Kết quả khi chạy:

```
100 0.56
10000 0.493
```

Mẫu lớn ở lần chạy này gần 0.5 hơn, nhưng đó là quan sát mô phỏng chứ không phải định luật đúng cho mọi lần.

6. Sai lầm thường gặp

- Phép thử không giống biến cố: phép thử tạo kết quả, biến cố là nhóm kết quả được quan tâm.
- Công thức số kết quả thuận lợi chia tổng kết quả chỉ đúng với các kết quả **đồng khả năng**.
- Trong mô phỏng, tỷ lệ quan sát có thể khác xác suất thật ngay cả khi seed cố định.
- `np.random.default_rng(seed)` giúp tái lập trong môi trường tương thích, không đảm bảo mẫu luôn đúng tỷ lệ lý thuyết.
- Set không có thứ tự cố định, nên in bằng `sorted` nếu cần đối chiếu đầu ra.

- Khi số mẫu tăng, độ dao động của ước lượng thường giảm, không bắt buộc mỗi lần chạy đều gần hơn.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a random experiment? — Phép thử ngẫu nhiên là gì?

A. A repeatable procedure with uncertain individual outcome — Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

B. Set of all possible outcomes — Tập tất cả kết quả có thể

C. A subset of the sample space — Một tập con của không gian mẫu

D. A measure between zero and one of an event's chance — Thước đo từ 0 đến 1 cho khả năng biến cố

Đáp án: A

Giải thích: Phép thử có thể sinh kết quả khác nhau.

Q2. What is a sample space? — Không gian mẫu là gì?

A. A repeatable procedure with uncertain individual outcome — Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

B. Set of all possible outcomes — Tập tất cả kết quả có thể

C. A subset of the sample space — Một tập con của không gian mẫu

D. A measure between zero and one of an event's chance — Thước đo từ 0 đến 1 cho khả năng biến cố

Đáp án: B

Giải thích: Không gian mẫu chứa mọi kết quả khả dĩ.

Q3. What is an event? — Biến cố là gì?

A. A repeatable procedure with uncertain individual outcome — Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

B. Set of all possible outcomes — Tập tất cả kết quả có thể

C. A subset of the sample space — Một tập con của không gian mẫu

D. A measure between zero and one of an event's chance — Thước đo từ 0 đến 1 cho khả năng biến cố

Đáp án: C

Giải thích: Biến cố là một nhóm kết quả.

Q4. What is probability? — Xác suất là gì?

A. A repeatable procedure with uncertain individual outcome — Thao tác lặp được nhưng kết quả mỗi lần không chắc chắn

B. Set of all possible outcomes — Tập tất cả kết quả có thể

C. A subset of the sample space — Một tập con của không gian mẫu

D. A measure between zero and one of an event's chance — Thước đo từ 0 đến 1 cho khả năng biến cố

Đáp án: D

Giải thích: Xác suất trong đoạn 0 đến 1.

Q5. When does counting favorable outcomes over total outcomes work? — Khi nào chia số trường hợp thuận lợi cho tổng số trường hợp là đúng?

A. When outcomes are equally likely — Khi các kết quả đồng khả năng

B. n values of zero or one — n giá trị 0 hoặc 1

C. Reproducible pseudorandom sequences — Chuỗi giả ngẫu nhiên tái lập

D. Fraction of True values — Tỷ lệ các phần tử True

Đáp án: A

Giải thích: Đếm cổ điển đòi hỏi các kết quả đồng khả năng.

Q6. What does `rng.integers(0,2,size=n)` generate? — `rng.integers(0,2,size=n)` tạo gì?

A. When outcomes are equally likely — Khi các kết quả đồng khả năng

B. n values of zero or one — n giá trị 0 hoặc 1

C. Reproducible pseudorandom sequences — Chuỗi giả ngẫu nhiên tái lập

D. Fraction of True values — Tỷ lệ các phần tử True

Đáp án: B

Giải thích: Cận trên 2 không lấy 2, chỉ còn 0 và 1.

Q7. What does `default_rng(seed)` provide? — `default_rng(seed)` cung cấp gì?

A. When outcomes are equally likely — Khi các kết quả đồng khả năng

B. n values of zero or one — n giá trị 0 hoặc 1

C. Reproducible pseudorandom sequences — Chuỗi giả ngẫu nhiên tái lập

D. Fraction of True values — Tỷ lệ các phần tử True

Đáp án: C

Giải thích: Seed điều khiển chuỗi giả ngẫu nhiên tái lập.

Q8. What does `np.mean(mask)` compute for Boolean mask? — `np.mean(mask)` tính gì với mặt nạ Boolean?

A. When outcomes are equally likely — Khi các kết quả đồng khả năng

B. n values of zero or one — n giá trị 0 hoặc 1

C. Reproducible pseudorandom sequences — Chuỗi giả ngẫu nhiên tái lập

D. Fraction of True values — Tỷ lệ các phần tử True

Đáp án: D

Giải thích: True tính như 1, False tính như 0.

Q9. What is P(even) for a fair six-sided die? — Xác suất mặt chẵn của xúc xắc công bằng sáu mặt?

A. 1/2 — 1/2

B. 36 — 36

C. 4 — 4

D. Zero — Bằng 0

Đáp án: A

Giải thích: Ba trong sáu mặt là số chẵn.

Q10. How many outcomes are possible with two ordered six-sided dice? — Hai xúc xắc sáu mặt theo cặp có thứ tự có bao nhiêu kết quả?

A. 1/2 — 1/2

B. 36 — 36

C. 4 — 4

D. Zero — Bằng 0

Đáp án: B

Giải thích: Hai xúc xắc có 6×6 cặp có thứ tự.

Q11. How many ordered pairs of dice sum to nine? — Bao nhiêu cặp có thứ tự của hai xúc xắc có tổng 9?

A. 1/2 — 1/2

B. 36 — 36

C. 4 — 4

D. Zero — Bằng 0

Đáp án: C

Giải thích: Bốn cặp là (3,6), (4,5), (5,4), (6,3).

Q12. What is the probability of the empty event? — Xác suất biến cố rỗng là gì?

A. 1/2 — 1/2

B. 36 — 36

C. 4 — 4

D. Zero — Bằng 0

Đáp án: D

Giải thích: Biến cố không thể có xác suất 0.

Q13. What is the probability of the entire sample space? — Xác suất của cả không gian mẫu là gì?

A. One — Bằng 1

B. `1-P(A)` — `1-P(A)`

C. No, sampling variation occurs — Không, có dao động do lấy mẫu

D. One die's result does not change probabilities of the other — Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: A

Giải thích: Biến cố chắc chắn có xác suất 1.

Q14. What is P(not A) in terms of P(A)? — P(không A) theo P(A) là gì?

A. One — Bằng 1

B. `1-P(A)` — `1-P(A)`

C. No, sampling variation occurs — Không, có dao động do lấy mẫu

D. One die's result does not change probabilities of the other — Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: B

Giải thích: Biến cố và biến cố đối có tổng xác suất 1.

Q15. Does a simulation always equal the true probability exactly? — Tỷ lệ mô phỏng có luôn bằng xác suất thật?

A. One — Bằng 1

B. `1-P(A)` — `1-P(A)`

C. No, sampling variation occurs — Không, có dao động do lấy mẫu

D. One die's result does not change probabilities of the other — Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: C

Giải thích: Mẫu hữu hạn dao động quanh xác suất nền.

Q16. What does independence of two dice mean? — Hai xúc xắc độc lập nghĩa là gì?

A. One — Bằng 1

B. `1-P(A)` — `1-P(A)`

C. No, sampling variation occurs — Không, có dao động do lấy mẫu

D. One die's result does not change probabilities of the other — Kết quả một xúc xắc không đổi xác suất của xúc xắc kia

Đáp án: D

Giải thích: Độc lập nghĩa là một kết quả không đổi xác suất của biến cố còn lại.

Q17. Why is printing a Set directly fragile for sample output? — Vì sao in trực tiếp Set khó đối chiếu đầu ra mẫu?

A. Display order is not promised — Thứ tự hiển thị không được đảm bảo

B. That empirical frequency equals theoretical probability — Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability — Thường làm giảm dao động lấy mẫu

D. Its faces do not have equal probabilities — Các mặt không đồng xác suất

Đáp án: A

Giải thích: Set không bảo đảm thứ tự in.

Q18. What does a fixed RNG seed not guarantee? — Seed RNG cố định không bảo đảm điều gì?

A. Display order is not promised — Thứ tự hiển thị không được đảm bảo

B. That empirical frequency equals theoretical probability — Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability — Thường làm giảm dao động lấy mẫu

D. Its faces do not have equal probabilities — Các mặt không đồng xác suất

Đáp án: B

Giải thích: Seed giúp lặp lại mô phỏng nhưng không loại dao động.

Q19. How does larger sample size generally affect estimate variability? — Tăng số mẫu thường ảnh hưởng độ dao động ước lượng thế nào?

A. Display order is not promised — Thứ tự hiển thị không được đảm bảo

B. That empirical frequency equals theoretical probability — Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability — Thường làm giảm dao động lấy mẫu

D. Its faces do not have equal probabilities — Các mặt không đồng xác suất

Đáp án: C

Giải thích: Mẫu lớn thường ổn định tần suất tương đối hơn.

Q20. Why can't a biased die use simple favorable-count ratios? — Vì sao xúc xắc lệch không dùng phép đếm mặt đơn giản?

A. Display order is not promised — Thứ tự hiển thị không được đảm bảo

B. That empirical frequency equals theoretical probability — Tần suất thực nghiệm đúng bằng xác suất lý thuyết

C. It tends to reduce sampling variability — Thường làm giảm dao động lấy mẫu

D. Its faces do not have equal probabilities — Các mặt không đồng xác suất

Đáp án: D

Giải thích: Xác suất không đều cần cộng theo trọng số.

8. Liên hệ bài tiếp theo

`Logic/Tập hợp → Xác suất → Bayes → Dự đoán bất định.`

PY-35 — Xác suất có điều kiện, Bayes và xác suất toàn phần

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Conditional probability — xác suất có điều kiện; **Independence** — độc lập.

`Biến cố´→ Điều kiện → Bayes → Đánh giá mô hình AI`

Học xong dùng để làm gì? Cập nhật xác suất dựa trên kết quả xét nghiệm hoặc tín hiệu mới, phân biệt xác suất tiên nghiệm/hậu nghiệm.

Kết nối kiến thức tiếp theo: PY-36 mô tả mọi kết quả có thể xảy ra bằng phân phối xác suất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Conditional probability	xác suất có điều kiện	Khả năng A xảy ra khi biết B đã xảy ra
Independence	độc lập	Hai biến cố không ảnh hưởng xác suất của nhau
Bayes theorem	định lý Bayes	Cập nhật xác suất giả thuyết khi có bằng chứng
Total probability	xác suất toàn phần	Cộng xác suất qua các nhóm loại trừ và đủ

3. Công thức và cách gọi bằng Python

Từ vựng: Event — biến cố; **Prior** — xác suất trước bằng chứng; **Posterior** — xác suất sau bằng chứng.

Công thức / cú pháp	Cách hiểu
`P(A given B) = P(A and B) / P(B)`	Xác suất có điều kiện khi `P(B)>0`
`P(A and B) = P(A given B) * P(B)`	Quy tắc nhân xác suất
`posterior = likelihood * prior / evidence`	Bayes với xác suất bằng chứng khác 0
`sum(group_share * group_rate)`	Xác suất toàn phần qua các nhóm rời và phủ đủ

4. Khái niệm chính và liên quan

Mạch kiến thức: Trước hết xác định nhóm điều kiện B để tính P(A khi B). Khi hai biến cố độc lập, biết B không đổi xác suất của A. Khi có bằng chứng mới, Bayes tính lại xác suất giả thuyết; xác suất toàn phần giúp tính tỷ lệ chung từ các nhóm có tỷ trọng khác nhau.

4.1. Conditional probability — Thu hẹp mẫu nền

Từ vựng: Conditional probability — xác suất có điều kiện; **Conditioning event** — biến cố điều kiện.

Xác suất có điều kiện `P(A given B)` chỉ xét những trường hợp **B đã xảy ra**, không còn xét toàn bộ quần thể ban đầu. Ví dụ trong 100 khách **đã xem quảng cáo** có 20 người mua. Khi đó P(mua **khi đã xem**) = 20/100 = 0.2, không phải tỷ lệ mua của tất cả khách. Công thức `P(A given B) = P(AnB)/P(B)` chỉ dùng khi `P(B)>0`. Tỷ lệ này **không tự chứng minh** xem quảng cáo đã gây ra hành vi mua.

```
seen = 100
bought = 20
print(bought / seen)
```

Kết quả khi chạy:

```
0.2
```

Trong nhóm 100 người đã xem quảng cáo, tỷ lệ mua quan sát được bằng 20%.

4.2. Independence — Không làm đổi xác suất của nhau

Từ vựng: Independence — độc lập; **Joint probability** — xác suất đồng thời.

Hai biến cố A và B **độc lập** nếu biết B xảy ra không đổi khả năng của A: `P(A given B)=P(A)` (khi P(B)>0). Tương đương `P(AnB)=P(A)×P(B)`. Giả sử P(A)=0.5 và P(B)=0.4 thì nếu độc lập, P(cả A và B)=0.2. Trong dữ liệu thực, việc hai tỷ lệ gần nhau chưa tự chứng minh độc lập; cần biết cách lấy mẫu và đo sai số.

```
p_a = 0.5
p_b = 0.4
print(round(p_a * p_b, 3))
```

Kết quả khi chạy:

```
0.2
```

Nếu hai biến cố độc lập thì giao của chúng có xác suất 0.2.

4.3. Bayes theorem — Cập nhật từ bằng chứng

Từ vựng: Prior — tỷ lệ nền trước bằng chứng; **Likelihood** — khả năng bằng chứng khi giả thuyết đúng; **Posterior** — xác suất sau bằng chứng.

Định lý **Bayes** trả lời "khi đã thấy **tín hiệu dương**, xác suất thực sự có lỗi là bao nhiêu?" Ví dụ chỉ 1% sản phẩm lỗi; máy báo dương với 99% hàng lỗi (**độ nhạy**) và 5% hàng không lỗi (**dương giả**). Trên 10.000 sản phẩm giả định có 100 lỗi: khoảng 99 dương thật, nhưng 495 dương giả từ 9.900 hàng tốt. Tổng dương là 594, nên xác suất lỗi khi máy báo dương là `99/594 ≈ 0.167`, **không phải 99%**. Tỷ lệ nền quyết định nhiều đến kết luận.

```
prior = 0.01
sensitivity = 0.99
false_positive = 0.05
```

```
positive = sensitivity * prior + false_positive * (1 - prior)
print(round(sensitivity * prior / positive, 3))
```

Kết quả khi chạy:

```
0.167
```

Sau tín hiệu dương, xác suất lỗi khoảng 16,7% theo các giả định đã nêu.

4.4. Total probability — Cộng qua các nhóm đầy đủ

Từ vựng: Total probability — xác suất toàn phần; **Partition** — phân hoạch.

Nếu quần thể được chia thành các **nhóm rời nhau và phủ đủ**, xác suất chung là trung bình có trọng số của xác suất trong nhóm. Ví dụ 60% khách ở nhóm X, trong đó 80% ủng hộ, còn 40% ở nhóm Y với tỷ lệ ủng hộ 20%. Tỷ lệ chung là `0.6×0.8 + 0.4×0.2 = 0.56`. Không cộng trực tiếp 80% và 20%, vì hai nhóm có quy mô khác nhau. Tổng xác suất các nhóm phải bằng 1.

```
shares = [0.6, 0.4]
support = [0.8, 0.2]
print(round(sum(a * b for a, b in zip(shares, support)), 3))
```

Kết quả khi chạy:

```
0.56
```

Xác suất chung là 56%, được tính theo tỷ trọng mỗi nhóm.

5. Năm tình huống ứng dụng

Tình huống 1 — Diễn giải cảnh báo của hệ kiểm hàng

Tình huống thực tế: Dây chuyền có tỷ lệ lỗi nền 1%, máy kiểm phát hiện 99% hàng lỗi và báo nhầm 5% hàng không lỗi. Khi một sản phẩm bị báo dương, đội kiểm tra muốn biết xác suất **thực sự lỗi**.

Vì sao chọn cách giải này? Bayes dùng cả tỷ lệ nền và đặc tính máy; độ nhạy 99% không phải tỷ lệ hàng thật lỗi trong tất cả cảnh báo.

Quy trình giải:

1. Gán tỷ lệ lỗi 0.01, độ nhạy 0.99 và dương giả 0.05.
2. Tính xác suất tất cả cảnh báo bằng tổng hai nhóm lỗi/không lỗi.
3. Lấy phần dương thật chia tổng dương, làm tròn ba chữ số.

```
prior = 0.01
sensitivity = 0.99
false_positive = 0.05
p_alert = sensitivity * prior + false_positive * (1 - prior)
print(round(sensitivity * prior / p_alert, 3))
```

Kết quả khi chạy:

```
0.167
```

Khoảng 16.7% cảnh báo dương là lỗi thật theo giả định; cần kiểm tra bổ sung trước quyết định xử lý.

Tình huống 2 — Tỷ lệ mua hàng trong nhóm đã xem quảng cáo

Tình huống thực tế: Một chiến dịch có 100 khách **đã xem** quảng cáo và trong nhóm đó 20 khách mua. Cần báo tỷ lệ mua có điều kiện trên lượt xem, không phải của mọi người dùng.

Vì sao chọn cách giải này? Điều kiện 'đã xem' giới hạn mẫu nền còn 100 khách, nên mẫu số phải là số đã xem.

Quy trình giải:

1. Lưu số khách đã xem và số khách vừa xem vừa mua.
2. Chia số mua trong nhóm cho số đã xem.
3. In tỷ lệ và nói rõ đây không phải ước lượng nhân quả.

```
seen = 100
bought_after_seen = 20
print(bought_after_seen / seen)
```

Kết quả khi chạy:

```
0.2
```

Có 20% người thuộc nhóm đã xem quảng cáo thực hiện mua trong quan sát này.

Tình huống 3 — Tính tỷ lệ ủng hộ chung theo nhóm

Tình huống thực tế: Khảo sát giả định có 60% đối tượng thuộc nhóm X với tỷ lệ ủng hộ 80%; 40% thuộc nhóm Y với tỷ lệ ủng hộ 20%. Hai nhóm rời nhau, bao phủ mọi đối tượng. Cần tính tỷ lệ chung.

Vì sao chọn cách giải này? Cộng xác suất toàn phần theo tỷ trọng nhóm; không thể lấy trung bình đơn giản hai tỷ lệ 80% và 20%.

Quy trình giải:

1. Ghi tỷ trọng hai nhóm và tỷ lệ ủng hộ trong từng nhóm.
2. Nhân tỷ trọng với tỷ lệ ủng hộ tương ứng.
3. Cộng rồi in xác suất chung.

```
shares = [0.6, 0.4]
rates = [0.8, 0.2]
print(round(sum(a * b for a, b in zip(shares, rates)), 3))
```

Kết quả khi chạy:

```
0.56
```

Tỷ lệ chung là 56% dưới giả định hai nhóm bao phủ toàn bộ quần thể.

Tình huống 4 — Kiểm tra giả định độc lập giữa hai sự kiện

Tình huống thực tế: Hai sự kiện A, B có xác suất P(A)=0.5, P(B)=0.4 và P(A∩B)=0.2 theo một mô hình xác suất cho sẵn. Cần kiểm tra bộ giá trị này có đáp ứng điều kiện độc lập không.

Vì sao chọn cách giải này? So xác suất đồng thời với tích xác suất riêng là tiêu chuẩn độc lập cho mô hình đã cho.

Quy trình giải:

1. Lưu ba xác suất đã biết.
2. Tính tích P(A)×P(B).
3. Dùng sai số nhỏ so với P(A∩B) và in Boolean.

```
p_a = 0.5
p_b = 0.4
p_joint = 0.2
print(abs(p_joint - p_a * p_b) < 1e-12)
```

Kết quả khi chạy:

```
True
```

Các xác suất đã cho thỏa quan hệ độc lập về mặt toán học; dữ liệu khảo sát thực cần xem độ bất định.

Tình huống 5 — Đánh giá một máy kiểm chất lượng khác

Tình huống thực tế: Lô hàng có tỷ lệ lỗi nền 2%, máy phát hiện 95% sản phẩm lỗi nhưng báo nhầm 10% sản phẩm tốt. Cần biết xác suất hàng lỗi sau một cảnh báo dương.

Vì sao chọn cách giải này? Bayes tách dương thật và dương giả trước khi xét trong toàn bộ các cảnh báo.

Quy trình giải:

1. Dùng prior=0.02, sensitivity=0.95, false_positive=0.10.
2. Tính tổng xác suất báo dương từ hai nhóm.
3. Chia phần dương thật cho tổng, in ba chữ số.

```
prior = 0.02
sensitivity = 0.95
false_positive = 0.10
p_alert = sensitivity * prior + false_positive * (1 - prior)
print(round(sensitivity * prior / p_alert, 3))
```

Kết quả khi chạy:

```
0.162
```

Chỉ khoảng 16.2% hàng bị báo dương là lỗi thật theo mô hình; không nhầm với độ nhạy 95%.

6. Sai lầm thường gặp

- P(A khi B) và P(B khi A) có **hai mẫu nền khác nhau**, không được tráo hai xác suất.
- Độ nhạy của máy kiểm không phải xác suất sản phẩm thực sự lỗi sau cảnh báo; phải tính cả tỷ lệ nền và dương giả.
- Xác suất có điều kiện cần mẫu số P(B) lớn hơn 0.
- Các nhóm trong xác suất toàn phần phải rời nhau và phủ toàn bộ quần thể.
- Quan sát khách mua sau khi thấy quảng cáo không tự chứng minh quảng cáo gây ra mua hàng.
- Kiểm tra độc lập trên dữ liệu khảo sát cần xét sai số và cách lấy mẫu, không chỉ so hai số xấp xỉ.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is conditional probability? — Xác suất có điều kiện là gì?

A. Probability of A given B has occurred — Xác suất A khi biết B đã xảy ra

B. One event does not change the probability of the other — Biến cố này không đổi xác suất của biến cố kia

C. A posterior from evidence, likelihood and prior — Xác suất sau bằng chứng từ likelihood và prior

D. Overall chance weighted over a partition — Xác suất chung theo trọng số một phân hoạch

Đáp án: A

Giải thích: Nó thu hẹp mẫu nền về nhóm B.

Q2. What is independence of events? — Độc lập giữa hai biến cố là gì?

A. Probability of A given B has occurred — Xác suất A khi biết B đã xảy ra

B. One event does not change the probability of the other — Biến cố này không đổi xác suất của biến cố kia

C. A posterior from evidence, likelihood and prior — Xác suất sau bằng chứng từ likelihood và prior

D. Overall chance weighted over a partition — Xác suất chung theo trọng số một phân hoạch

Đáp án: B

Giải thích: Độc lập nghĩa là biến cố này không đổi xác suất biến cố kia.

Q3. What does Bayes' theorem compute? — Định lý Bayes tính gì?

A. Probability of A given B has occurred — Xác suất A khi biết B đã xảy ra

B. One event does not change the probability of the other — Biến cố này không đổi xác suất của biến cố kia

C. A posterior from evidence, likelihood and prior — Xác suất sau bằng chứng từ likelihood và prior

D. Overall chance weighted over a partition — Xác suất chung theo trọng số một phân hoạch

Đáp án: C

Giải thích: Bayes cập nhật tỷ lệ nền từ bằng chứng.

Q4. What does total probability calculate? — Xác suất toàn phần tính gì?

A. Probability of A given B has occurred — Xác suất A khi biết B đã xảy ra

B. One event does not change the probability of the other — Biến cố này không đổi xác suất của biến cố kia

C. A posterior from evidence, likelihood and prior — Xác suất sau bằng chứng từ likelihood và prior

D. Overall chance weighted over a partition — Xác suất chung theo trọng số một phân hoạch

Đáp án: D

Giải thích: Xác suất toàn phần dùng tỷ trọng của từng nhóm.

Q5. Which formula gives P(A given B)? — Công thức P(A khi biết B) là gì?

A. `P(A and B)/P(B)` when P(B)>0 — `P(A và B)/P(B)` khi P(B)>0

B. `P(A and B)=P(A)*P(B)` — `P(A và B)=P(A)*P(B)`

C. `likelihood*prior/evidence` — `likelihood*prior/evidence`

D. Disjoint and exhaustive — Rời nhau và phủ đủ

Đáp án: A

Giải thích: Lấy xác suất giao chia xác suất biến cố điều kiện.

Q6. Which equation expresses event independence? — Đẳng thức nào mô tả biến cố độc lập?

A. `P(A and B)/P(B)` when P(B)>0 — `P(A và B)/P(B)` khi P(B)>0

B. `P(A and B)=P(A)*P(B)` — `P(A và B)=P(A)*P(B)`

C. `likelihood*prior/evidence` — `likelihood*prior/evidence`

D. Disjoint and exhaustive — Rời nhau và phủ đủ

Đáp án: B

Giải thích: Độc lập khi xác suất giao bằng tích hai xác suất.

Q7. Which rule gives posterior after a positive signal? — Quy tắc nào tính posterior khi tín hiệu dương?

A. `P(A and B)/P(B)` when P(B)>0 — `P(A và B)/P(B)` khi P(B)>0

B. `P(A and B)=P(A)*P(B)` — `P(A và B)=P(A)*P(B)`

C. `likelihood*prior/evidence` — `likelihood*prior/evidence`

D. Disjoint and exhaustive — Rời nhau và phủ đủ

Đáp án: C

Giải thích: Bayes chuẩn hóa likelihood nhân prior bằng xác suất bằng chứng.

Q8. What condition is required for total probability groups? — Các nhóm dùng xác suất toàn phần phải thế nào?

A. `P(A and B)/P(B)` when P(B)>0 — `P(A và B)/P(B)` khi P(B)>0

B. `P(A and B)=P(A)*P(B)` — `P(A và B)=P(A)*P(B)`

C. `likelihood*prior/evidence` — `likelihood*prior/evidence`

D. Disjoint and exhaustive — Rời nhau và phủ đủ

Đáp án: D

Giải thích: Nhóm phải không giao nhau và phủ toàn bộ.

Q9. What does prior mean? — Prior có nghĩa gì?

A. Probability before current evidence — Xác suất trước khi có bằng chứng hiện tại

B. Probability of evidence under a hypothesis — Xác suất bằng chứng khi giả thuyết đúng

C. Updated hypothesis probability after evidence — Xác suất giả thuyết được cập nhật sau bằng chứng

D. Probability of positive signal when no defect exists — Xác suất báo dương khi thực tế không có lỗi

Đáp án: A

Giải thích: Prior là xác suất nền.

Q10. What does likelihood mean? — Likelihood biểu thị gì?

A. Probability before current evidence — Xác suất trước khi có bằng chứng hiện tại

B. Probability of evidence under a hypothesis — Xác suất bằng chứng khi giả thuyết đúng

C. Updated hypothesis probability after evidence — Xác suất giả thuyết được cập nhật sau bằng chứng

D. Probability of positive signal when no defect exists — Xác suất báo dương khi thực tế không có lỗi

Đáp án: B

Giải thích: Likelihood xét bằng chứng khi giả thuyết đúng.

Q11. What does posterior mean? — Posterior là gì?

A. Probability before current evidence — Xác suất trước khi có bằng chứng hiện tại

B. Probability of evidence under a hypothesis — Xác suất bằng chứng khi giả thuyết đúng

C. Updated hypothesis probability after evidence — Xác suất giả thuyết được cập nhật sau bằng chứng

D. Probability of positive signal when no defect exists — Xác suất báo dương khi thực tế không có lỗi

Đáp án: C

Giải thích: Posterior đã tính đến bằng chứng quan sát.

Q12. What is a false-positive rate? — Tỷ lệ dương giả là gì?

A. Probability before current evidence — Xác suất trước khi có bằng chứng hiện tại

B. Probability of evidence under a hypothesis — Xác suất bằng chứng khi giả thuyết đúng

C. Updated hypothesis probability after evidence — Xác suất giả thuyết được cập nhật sau bằng chứng

D. Probability of positive signal when no defect exists — Xác suất báo dương khi thực tế không có lỗi

Đáp án: D

Giải thích: Dương giả là cảnh báo dương ở trường hợp không lỗi.

Q13. What is P(A given B) if 20 of 100 B cases have A? — P(A khi B) là gì nếu 20 trên 100 ca B có A?

A. 0.2 — 0.2

B. 0.56 — 0.56

C. 0.3 — 0.3

D. No; base rate and false positives also matter — Không; còn tỷ lệ nền và dương giả

Đáp án: A

Giải thích: 20 chia 100 bằng 0.2.

Q14. What is the total with group shares 0.6,0.4 and rates 0.8,0.2? — Tỷ lệ chung từ nhóm 0.6/0.4 với xác suất 0.8/0.2 là gì?

A. 0.2 — 0.2

B. 0.56 — 0.56

C. 0.3 — 0.3

D. No; base rate and false positives also matter — Không; còn tỷ lệ nền và dương giả

Đáp án: B

Giải thích: 0.6×0.8+0.4×0.2 bằng 0.56.

Q15. If P(A)=0.5 and P(B)=0.6 and they are independent, what is P(A and B)? — Nếu A, B độc lập có xác suất 0.5 và 0.6 thì P(A và B) là gì?

A. 0.2 — 0.2

B. 0.56 — 0.56

C. 0.3 — 0.3

D. No; base rate and false positives also matter — Không; còn tỷ lệ nền và dương giả

Đáp án: C

Giải thích: Với biến cố độc lập, 0.5×0.6=0.3.

Q16. If an alert has 99% sensitivity, does that mean 99% of alerts are true defects? — Độ nhạy 99% có nghĩa 99% cảnh báo là lỗi thật không?

A. 0.2 — 0.2

B. 0.56 — 0.56

C. 0.3 — 0.3

D. No; base rate and false positives also matter — Không; còn tỷ lệ nền và dương giả

Đáp án: D

Giải thích: Độ nhạy và posterior xét hai nhóm mẫu nền khác nhau.

Q17. Why does Bayes need the base rate? — Vì sao Bayes cần tỷ lệ nền?

A. Rare cases can yield many false alarms among positives — Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

B. The formula divides by P(B) — Vì công thức chia cho P(B)

C. That viewing ads caused purchases — Việc xem quảng cáo gây ra hành vi mua

D. They condition on different reference groups — Chúng xét trên hai nhóm điều kiện khác nhau

Đáp án: A

Giải thích: Sự kiện hiếm có thể khiến phần lớn cảnh báo là dương giả.

Q18. Why must P(B)>0 for P(A given B)? — Vì sao cần P(B)>0 khi tính P(A khi B)?

A. Rare cases can yield many false alarms among positives — Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

B. The formula divides by P(B) — Vì công thức chia cho P(B)

C. That viewing ads caused purchases — Việc xem quảng cáo gây ra hành vi mua

D. They condition on different reference groups — Chúng xét trên hai nhóm điều kiện khác nhau

Đáp án: B

Giải thích: Chia cho 0 không xác định.

Q19. What does an observational conditional purchase rate not prove? — Tỷ lệ mua có điều kiện trong quan sát không chứng minh gì?

A. Rare cases can yield many false alarms among positives — Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

B. The formula divides by P(B) — Vì công thức chia cho P(B)

C. That viewing ads caused purchases — Việc xem quảng cáo gây ra hành vi mua

D. They condition on different reference groups — Chúng xét trên hai nhóm điều kiện khác nhau

Đáp án: C

Giải thích: Liên hệ quan sát không tự chứng minh nhân quả.

Q20. What distinguishes P(A given B) from P(B given A)? — P(A khi B) khác P(B khi A) ở đâu?

A. Rare cases can yield many false alarms among positives — Sự kiện hiếm có thể làm dương giả chiếm nhiều trong cảnh báo

B. The formula divides by P(B) — Vì công thức chia cho P(B)

C. That viewing ads caused purchases — Việc xem quảng cáo gây ra hành vi mua

D. They condition on different reference groups — Chúng xét trên hai nhóm điều kiện khác nhau

Đáp án: D

Giải thích: Đảo điều kiện làm thay đổi mẫu số.

8. Liên hệ bài tiếp theo

Biến cố → Điều kiện → Bayes → Đánh giá mô hình AI.

PY-36 — Phân phối nhị thức và phân phối chuẩn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Random variable — biến ngẫu nhiên; **Binomial** — phân phối nhị thức.

`Xác suất → Biến ngẫu nhiên → Phân phối → Thống kê`

Học xong dùng để làm gì? Mô tả xác suất của các mức số đếm hay kết quả đo và xem độ tập trung/phân tán của biến ngẫu nhiên.

Kết nối kiến thức tiếp theo: PY-37 tính trung bình và độ lệch chuẩn trực tiếp từ dữ liệu quan sát.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Random variable	biến ngẫu nhiên	Đại lượng có kết quả tùy phép thử
Binomial	phân phối nhị thức	Số thành công của n thử Bernoulli độc lập
Normal	phân phối chuẩn	Phân phối liên tục hình chuông với mean/std
PMF/PDF	hàm khối/mật độ	PMF cho biến rời rạc, PDF là mật độ cho biến liên tục

3. Các hàm Python cần nhớ

Hàm	Ý nghĩa
`binom.pmf(k, n, p)`	Xác suất **đúng** k thành công
`binom.cdf(k, n, p)`	Xác suất **không quá** k thành công
`norm.pdf(x, loc, scale)`	Mật độ chuẩn tại x, không phải xác suất tại điểm
`norm.cdf(x, loc, scale)`	Xác suất $X \leq x$
`rng.normal(loc, scale, size)`	Lấy số mẫu giả ngẫu nhiên theo phân phối chuẩn

4. Khái niệm chính và liên quan

Mạch kiến thức: Phép thử tạo kết quả; biến ngẫu nhiên biến kết quả thành số; phân phối cho khả năng từng giá trị. Nhị thức dành cho số lần thành công rời rạc, chuẩn cho biến liên tục; PMF, PDF và CDF trả lời những câu hỏi xác suất khác nhau.

4.1. Random variable — Chuyển kết quả thử thành con số

Từ vựng: Random variable — biến ngẫu nhiên; **Outcome** — kết quả thử.

Biến ngẫu nhiên X gán một giá trị số cho mỗi kết quả của phép thử. Ví dụ trong ba lần giao hàng, X là **số đơn đúng giờ** nên X có thể bằng 0, 1, 2 hoặc 3, không phải tên từng đơn. Một phép thử cụ thể chỉ tạo một giá trị X; **phân phối xác suất** mô tả khả năng của toàn bộ các giá trị đó. Mô phỏng sử dụng `seed` chỉ để có thể lặp lại kết quả.

```
import numpy as np

rng = np.random.default_rng(42)
print(rng.binomial(n=3, p=0.8, size=5).tolist())
```

Kết quả khi chạy:

```
[2, 3, 2, 2, 3]
```

Mỗi phần tử là số đơn đúng giờ trong một nhóm ba đơn; đây là năm lần thử giả lập.

4.2. Binomial — Đếm thành công từ các lần thử độc lập

Từ vựng: Binomial — nhị thức; **Bernoulli** — phép thử thành công/thất bại.

Phân phối nhị thức tính số lần **thành công trong n phép thử độc lập**, mỗi lần có cùng xác suất thành công p. Với 10 đơn hàng, xác suất giao đúng giờ từng đơn bằng 0,8, biến X đếm số đơn đúng giờ. `binom.pmf(8,10,0.8)` tính **đúng 8 đơn** đúng giờ; `binom.cdf(8,10,0.8)` lại tính **tối đa 8 đơn**. Không dùng nhị thức khi xác suất từng lần khác nhau hoặc các lần phụ thuộc mạnh mà không điều chỉnh mô hình.

```
from scipy.stats import binom

print(round(float(binom.pmf(8, 10, 0.8)), 4))
```

Kết quả khi chạy:

```
0.302
```

Xác suất đúng 8 trong 10 đơn đúng giờ khoảng 30,2% theo mô hình độc lập.

4.3. Normal — Phân phối liên tục quanh trung bình

Từ vựng: Normal — chuẩn; **Mean/standard deviation** — trung bình/độ lệch chuẩn.

Phân phối chuẩn có dạng chuông đối xứng quanh trung bình μ, với độ phân tán đo bằng σ (độ lệch chuẩn, σ>0). Ví dụ điểm cảm biến được **giả định** chuẩn với μ=100 và σ=15 (cùng đơn vị đo). Ngưỡng 115 cao hơn trung bình đúng một σ. `norm.cdf(115,loc=100,scale=15)` tính khả năng X **không vượt quá 115**, xấp xỉ 0,8413. Không nhầm giá trị CDF với mật độ tại điểm.

```
from scipy.stats import norm

print(round(float(norm.cdf(115, loc=100, scale=15)), 4))
```

Kết quả khi chạy:

```
0.8413
```

Khoảng 84,13% giá trị nằm từ phía âm vô hạn tới ngưỡng 115 theo giả định chuẩn.

4.4. PMF, PDF và CDF — Ba câu hỏi khác nhau

Từ vựng: PMF — khối xác suất; **PDF** — mật độ xác suất; **CDF** — xác suất tích lũy.

PMF cho biến rời rạc trả xác suất **bằng đúng** một giá trị, chẳng hạn P(X=1) khi X là số lần thành công. **PDF** cho biến liên tục trả *mật độ* tại x, **không phải P(X=x)** (với phân phối chuẩn, xác suất tại đúng một điểm là 0). **CDF** ở cả hai loại biến biểu diễn P(X≤x). Để tính xác suất trong một khoảng liên tục `(a,b]`, ta trừ CDF(b) −CDF(a).

```
from scipy.stats import norm

probability = norm.cdf(1) - norm.cdf(-1)
print(round(float(probability), 4))
```

Kết quả khi chạy:

```
0.6827
```

Theo chuẩn tắc N(0,1), khoảng −1 đến +1 chứa khoảng 68,27% xác suất.

5. Năm tình huống ứng dụng

Tình huống 1 — Đếm đúng tám đơn hàng giao đúng giờ

Tình huống thực tế: Một kho vận giả định mỗi đơn có xác suất 0,8 giao đúng giờ, độc lập với các đơn khác. Trong một nhóm 10 đơn, cần biết xác suất **chính xác tám đơn** đạt yêu cầu.

Vì sao chọn cách giải này? PMF nhị thức tính xác suất đúng k lần thành công trên n phép thử độc lập cùng xác suất p.

Quy trình giải:

1. Xác định thành công là đơn đúng giờ, n=10, p=0.8 và k=8.
2. Dùng `binom.pmf(k,n,p)`.
3. In xác suất và ghi rõ đây là đúng 8, không phải tối đa 8.

```
from scipy.stats import binom

print(round(float(binom.pmf(8, 10, 0.8)), 4))
```

Kết quả khi chạy:

```
0.302
```

Xác suất đúng tám đơn đúng giờ là khoảng 0,302 theo các giả định.

Tình huống 2 — Xác suất có không quá ba lỗi

Tình huống thực tế: Nhà máy xét 10 linh kiện độc lập, xác suất lỗi mỗi chiếc 0,1. Bộ phận kiểm định muốn xác suất có từ 0 đến 3 chiếc lỗi, **bao gồm cả 3**.

Vì sao chọn cách giải này? CDF nhị thức cộng các xác suất P(X=0), P(X=1), P(X=2), P(X=3).

Quy trình giải:

1. Đặt X là số linh kiện lỗi, n=10 và p=0.1.
2. Tính `binom.cdf(3,10,0.1)`.
3. Làm tròn bốn chữ số và in kết quả.

```
from scipy.stats import binom

print(round(float(binom.cdf(3, 10, 0.1)), 4))
```

Kết quả khi chạy:

```
0.9872
```

Theo mô hình, xác suất có tối đa ba linh kiện lỗi khoảng 98,72%.

Tình huống 3 — Tỷ lệ đo cảm biến dưới ngưỡng

Tình huống thực tế: Một cảm biến có phép đo được giả định chuẩn với trung bình 100, độ lệch chuẩn 15 đơn vị đo. Ngưỡng theo dõi là 115; cần tính tỷ lệ phép đo không vượt ngưỡng.

Vì sao chọn cách giải này? CDF chuẩn cộng xác suất của mọi giá trị từ dưới ngưỡng đến ngưỡng.

Quy trình giải:

1. Gán μ=100, σ=15 và ngưỡng 115 cùng đơn vị.
2. Gọi `norm.cdf` với `loc=100`, `scale=15`.
3. In xác suất đến bốn chữ số thập phân.

```
from scipy.stats import norm

print(round(float(norm.cdf(115, loc=100, scale=15)), 4))
```

Kết quả khi chạy:

```
0.8413
```

Khoảng 84,13% phép đo không vượt 115 nếu giả định phân phối chuẩn phù hợp.

Tình huống 4 — Phân biệt mật độ tại điểm và xác suất

Tình huống thực tế: Mô hình độ lệch chuẩn tắc X~N(0,1). Nhà phân tích cần xem **mật độ** ở x=0, không phải tỷ lệ quan sát bằng chính xác 0.

Vì sao chọn cách giải này? PDF chuẩn cho chiều cao đường cong tại điểm; xác suất của điểm đơn lẻ trong biến liên tục bằng 0.

Quy trình giải:

1. Chọn phân phối chuẩn tắc μ=0, σ=1.
2. Tính `norm.pdf(0)`.
3. In mật độ và tránh diễn giải như tỷ lệ sự kiện X=0.

```
from scipy.stats import norm

print(round(float(norm.pdf(0)), 4))
```

Kết quả khi chạy:

```
0.3989
```

0,3989 là mật độ tại 0, **không phải 39,89% xác suất có giá trị đúng bằng 0**.

Tình huống 5 — Tạo năm mẫu cảm biến chuẩn để thử code

Tình huống thực tế: Lập trình viên muốn năm số thử để kiểm tra luồng dữ liệu cảm biến giả lập N(0,1), có thể tạo lại khi học viên chạy đúng môi trường.

Vì sao chọn cách giải này? Bộ sinh NumPy với seed giúp kiểm tra cùng chuỗi dữ liệu trong phiên bản tương thích.

Quy trình giải:

1. Tạo `default_rng(42)`.
2. Sinh năm mẫu chuẩn trung bình 0, độ lệch chuẩn 1.
3. Làm tròn ba chữ số và chuyển List để in ổn định.

```
import numpy as np

rng = np.random.default_rng(42)
samples = rng.normal(loc=0, scale=1, size=5)
print(np.round(samples, 3).tolist())
```

Kết quả khi chạy:

```
[0.305, -1.04, 0.75, 0.941, -1.951]
```

Đây là năm giá trị giả lập, không phải minh chứng rằng dữ liệu cảm biến thực tế tuân phân phối chuẩn.

6. Sai lầm thường gặp

- PMF tính xác suất đúng một giá trị **rời rạc**; PDF là mật độ cho biến liên tục, không phải P(X=x).
- CDF là P(X≤x), khác PMF tại k và PDF tại x.
- Mô hình nhị thức cần n lần thử độc lập với p giống nhau; thay điều kiện có thể làm kết quả sai.
- `scale` là độ lệch chuẩn dương, không phải phương sai.
- Seed cố định cho mô phỏng có thể tái lập trong môi trường tương thích; không bảo đảm thống kê mẫu bằng lý thuyết.
- Xác suất dưới mô hình không chứng minh quy trình thực tế đúng giả định.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a random variable? — Biến ngẫu nhiên là gì?

A. A numerical quantity assigned to random outcomes — Đại lượng số gắn với kết quả ngẫu nhiên

B. Successes in independent same-p trials — Số thành công trong các lần thử độc lập cùng p

C. A continuous symmetric bell-shaped density — Mật độ liên tục dạng chuông đối xứng

D. Probability mass at a discrete value — Xác suất ở một giá trị rời rạc

Đáp án: A

Giải thích: Nó biến kết quả phép thử thành số. Biến ngẫu nhiên không phải toàn bộ kết quả thử; nó gán một con số, chẳng hạn số đơn đúng giờ, cho mỗi kết quả có thể.

Q2. What does binomial X count? — Biến nhị thức X đếm gì?

A. A numerical quantity assigned to random outcomes — Đại lượng số gắn với kết quả ngẫu nhiên

B. Successes in independent same-p trials — Số thành công trong các lần thử độc lập cùng p

C. A continuous symmetric bell-shaped density — Mật độ liên tục dạng chuông đối xứng

D. Probability mass at a discrete value — Xác suất ở một giá trị rời rạc

Đáp án: B

Giải thích: Mỗi lần thử có hai kết quả thành công hoặc thất bại. Biến nhị thức X nhận từ 0 đến n; nó đếm số lần đạt, không đếm số loại kết quả của một phép thử.

Q3. What characterizes a normal distribution? — Phân phối chuẩn có đặc điểm gì?

A. A numerical quantity assigned to random outcomes — Đại lượng số gắn với kết quả ngẫu nhiên

B. Successes in independent same-p trials — Số thành công trong các lần thử độc lập cùng p

C. A continuous symmetric bell-shaped density — Mật độ liên tục dạng chuông đối xứng

D. Probability mass at a discrete value — Xác suất ở một giá trị rời rạc

Đáp án: C

Giải thích: Được xác định bởi trung bình và độ lệch chuẩn. Hai tham số có ý nghĩa khác nhau: mean xác định tâm và độ lệch chuẩn dương quyết định độ phân tán.

Q4. What is a PMF? — PMF là gì?

A. A numerical quantity assigned to random outcomes — Đại lượng số gắn với kết quả ngẫu nhiên

B. Successes in independent same-p trials — Số thành công trong các lần thử độc lập cùng p

C. A continuous symmetric bell-shaped density — Mật độ liên tục dạng chuông đối xứng

D. Probability mass at a discrete value — Xác suất ở một giá trị rời rạc

Đáp án: D

Giải thích: PMF tính xác suất đúng một kết quả rời rạc. Với X nhị thức, PMF tại k trả đúng k lần thành công; CDF tại k tính cả mọi số lần ít hơn k.

Q5. What is a PDF? — PDF là gì?

A. Density, not point probability, for continuous data — Mật độ, không phải xác suất tại điểm liên tục

B. Probability that X is at most x — Xác suất X nhỏ hơn hoặc bằng x

C. Probability of exactly k successes — Xác suất đúng k lần thành công

D. Probability of k or fewer successes — Xác suất tối đa k lần thành công

Đáp án: A

Giải thích: Xác suất khoảng liên tục tính từ diện tích dưới mật độ. Chẳng hạn giá trị PDF bằng 0,4 không nghĩa là có 40% xác suất nhận đúng số thực ấy.

Q6. What is a CDF? — CDF là gì?

A. Density, not point probability, for continuous data — Mật độ, không phải xác suất tại điểm liên tục

B. Probability that X is at most x — Xác suất X nhỏ hơn hoặc bằng x

C. Probability of exactly k successes — Xác suất đúng k lần thành công

D. Probability of k or fewer successes — Xác suất tối đa k lần thành công

Đáp án: B

Giải thích: CDF cộng xác suất đến ngưỡng đã cho. CDF cộng xác suất tới ngưỡng; với biến liên tục, xác suất một khoảng được tính bằng hiệu hai CDF.

Q7. What does binom.pmf(k,n,p) compute? — binom.pmf(k,n,p) tính gì?

A. Density, not point probability, for continuous data — Mật độ, không phải xác suất tại điểm liên tục

B. Probability that X is at most x — Xác suất X nhỏ hơn hoặc bằng x

C. Probability of exactly k successes — Xác suất đúng k lần thành công

D. Probability of k or fewer successes — Xác suất tối đa k lần thành công

Đáp án: C

Giải thích: PMF cho xác suất đúng số lần thành công. Đối số n là số lần thử, p là xác suất mỗi lần; pmf trả xác suất của đúng một số lần thành công.

Q8. What does binom.cdf(k,n,p) compute? — binom.cdf(k,n,p) tính gì?

A. Density, not point probability, for continuous data — Mật độ, không phải xác suất tại điểm liên tục

B. Probability that X is at most x — Xác suất X nhỏ hơn hoặc bằng x

C. Probability of exactly k successes — Xác suất đúng k lần thành công

D. Probability of k or fewer successes — Xác suất tối đa k lần thành công

Đáp án: D

Giải thích: CDF cộng từ 0 tới k. Ví dụ $P(X \le 2)$ cộng $P(X=0)$, $P(X=1)$, $P(X=2)$, chứ không chỉ $P(X=2)$.

Q9. What condition is essential for a simple binomial model? — Điều kiện quan trọng của nhị thức đơn giản là gì?

A. Independent trials with constant success probability — Các lần thử độc lập cùng xác suất thành công

B. Center and spread — Trung tâm và độ phân tán

C. Zero — Bằng 0

D. Strictly positive standard deviation — Độ lệch chuẩn phải lớn hơn 0

Đáp án: A

Giải thích: Phụ thuộc giữa lần thử làm sai giả định mô hình. Nếu đơn giao cùng chịu một sự cố hoặc xác suất đúng giờ khác nhau, mô hình nhị thức đơn giản có thể không phù hợp.

Q10. What do mean and standard deviation determine for normal? — Mean và std xác định gì trong chuẩn?

A. Independent trials with constant success probability — Các lần thử độc lập cùng xác suất thành công

B. Center and spread — Trung tâm và độ phân tán

C. Zero — Bằng 0

D. Strictly positive standard deviation — Độ lệch chuẩn phải lớn hơn 0

Đáp án: B

Giải thích: Trung bình định vị và độ lệch chuẩn xác định độ rộng. Đổi độ lệch chuẩn làm chuông rộng/hẹp nhưng không nhất thiết thay tâm của phân phối.

Q11. What is P(X=x) for a continuous normal variable? — P(X=x) với biến chuẩn liên tục bằng bao nhiêu?

A. Independent trials with constant success probability — Các lần thử độc lập cùng xác suất thành công

B. Center and spread — Trung tâm và độ phân tán

C. Zero — Bằng 0

D. Strictly positive standard deviation — Độ lệch chuẩn phải lớn hơn 0

Đáp án: C

Giải thích: Một điểm đơn có xác suất bằng 0. Xác suất trong khoảng là diện tích dưới PDF; một điểm riêng lẻ không có diện tích.

Q12. Which quantity must be positive for norm.pdf scale? — scale của norm.pdf cần điều kiện nào?

A. Independent trials with constant success probability — Các lần thử độc lập cùng xác suất thành công

B. Center and spread — Trung tâm và độ phân tán

C. Zero — Bằng 0

D. Strictly positive standard deviation — Độ lệch chuẩn phải lớn hơn 0

Đáp án: D

Giải thích: Độ lệch chuẩn không dương không hợp lệ. Trong SciPy, `scale` là sigma chứ không phải sigma bình phương; truyền phương sai sẽ làm sai toàn bộ phân phối.

Q13. What does norm.cdf(0) equal for N(0,1)? — norm.cdf(0) với N(0,1) bằng gì?

A. 0.5 — 0.5

B. CDF(b) minus CDF(a) — CDF(b) trừ CDF(a)

C. No, sampling noise remains — Không, vẫn có dao động lấy mẫu

D. From 0 to 1 inclusive — Từ 0 đến 1 gồm hai đầu

Đáp án: A

Giải thích: Chuẩn tắc đối xứng quanh 0. Chuẩn tắc đối xứng quanh trung bình 0 nên mỗi phía chứa một nửa xác suất.

Q14. How do you calculate normal P(a<X≤b)? — Tính P(a<X≤b) cho chuẩn bằng gì?

A. 0.5 — 0.5

B. CDF(b) minus CDF(a) — CDF(b) trừ CDF(a)

C. No, sampling noise remains — Không, vẫn có dao động lấy mẫu

D. From 0 to 1 inclusive — Từ 0 đến 1 gồm hai đầu

Đáp án: B

Giải thích: Hiệu xác suất tích lũy cho xác suất đoạn. Ví dụ P(−1<X≤1) bằng `norm.cdf(1)-norm.cdf(-1)`, xấp xỉ 0,6827.

Q15. Does fixed RNG seed force sample mean to equal μ? — Seed cố định có buộc mean mẫu bằng μ?

A. 0.5 — 0.5

B. CDF(b) minus CDF(a) — CDF(b) trừ CDF(a)

C. No, sampling noise remains — Không, vẫn có dao động lấy mẫu

D. From 0 to 1 inclusive — Từ 0 đến 1 gồm hai đầu

Đáp án: C

Giải thích: Seed lặp được mẫu, không đảm bảo đúng moment lý thuyết. Mô phỏng cố định seed có thể lặp lại nhưng trung bình mẫu vẫn lệch so với kỳ vọng lý thuyết.

Q16. What is the range of probabilities? — Xác suất nằm trong đoạn nào?

A. 0.5 — 0.5

B. CDF(b) minus CDF(a) — CDF(b) trừ CDF(a)

C. No, sampling noise remains — Không, vẫn có dao động lấy mẫu

D. From 0 to 1 inclusive — Từ 0 đến 1 gồm hai đầu

Đáp án: D

Giải thích: Xác suất không âm và không vượt một. Hai đầu 0 và 1 tương ứng biến cố không thể và chắc chắn.

Q17. Can a continuous PDF exceed 1 at a point? — PDF liên tục có thể lớn hơn 1 tại một điểm không?

A. Yes; only its total integral must be one — Có; chỉ tích phân toàn miền phải bằng một

B. The distribution becomes wider — Phân phối trải rộng hơn

C. Point probabilities behave differently — Xác suất tại một điểm có ý nghĩa khác nhau

D. Cumulative probability up to x — Xác suất tích lũy tới x

Đáp án: A

Giải thích: Chiều cao mật độ không phải xác suất. Đường chuẩn có độ lệch chuẩn nhỏ có thể có đỉnh PDF cao hơn 1 nhưng tổng diện tích vẫn bằng 1.

Q18. What happens when standard deviation of normal increases? — Std phân phối chuẩn tăng sẽ thế nào?

A. Yes; only its total integral must be one — Có; chỉ tích phân toàn miền phải bằng một

B. The distribution becomes wider — Phân phối trải rộng hơn

C. Point probabilities behave differently — Xác suất tại một điểm có ý nghĩa khác nhau

D. Cumulative probability up to x — Xác suất tích lũy tới x

Đáp án: B

Giải thích: Cùng tổng diện tích được trải ra trên miền rộng hơn. Sigma lớn hơn không tự làm dịch mean; nó khiến số đo phân tán xa hơn quanh cùng tâm.

Q19. Why distinguish discrete and continuous variables? — Vì sao phân biệt biến rời rạc và liên tục?

A. Yes; only its total integral must be one — Có; chỉ tích phân toàn miền phải bằng một

B. The distribution becomes wider — Phân phối trải rộng hơn

C. Point probabilities behave differently — Xác suất tại một điểm có ý nghĩa khác nhau

D. Cumulative probability up to x — Xác suất tích lũy tới x

Đáp án: C

Giải thích: PMF là xác suất còn PDF là mật độ. Số đếm như X=3 có xác suất tại điểm dương, còn biến đo liên tục có xác suất bằng đúng một giá trị là 0.

Q20. What does norm.cdf(x,loc,scale) return? — norm.cdf(x,loc,scale) trả gì?

A. Yes; only its total integral must be one — Có; chỉ tích phân toàn miền phải bằng một

B. The distribution becomes wider — Phân phối trải rộng hơn

C. Point probabilities behave differently — Xác suất tại một điểm có ý nghĩa khác nhau

D. Cumulative probability up to x — Xác suất tích lũy tới x

Đáp án: D

Giải thích: Nó là P(X≤x) theo phân phối chuẩn. Không lấy chiều cao PDF nếu câu hỏi là P(X≤x); cần dùng `cdf` để tính xác suất tích lũy.

8. Liên hệ bài tiếp theo

`Xác suất → Biến ngẫu nhiên → Phân phối → Thống kê.`

PY-37 — Thống kê mô tả: mean, median, mode, variance và std

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Mean — trung bình; **Median** — trung vị.

`NumPy/Pandas → Thống kê mô tả → Outlier → ML`

Học xong dùng để làm gì? Tóm tắt bảng số bằng trung bình, trung vị, phương sai và độ lệch chuẩn để so sánh các nhóm.

Kết nối kiến thức tiếp theo: PY-38 dùng phân vị/IQR để tìm giá trị khác thường mà trung bình có thể che khuất.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Mean	trung bình	Tổng giá trị chia số quan sát
Median	trung vị	Giá trị giữa sau khi sắp xếp
Variance	phương sai	Mức phân tán bình phương quanh trung bình
Standard deviation	độ lệch chuẩn	Căn bậc hai của phương sai, cùng đơn vị dữ liệu

3. Hàm Python và tham số quan trọng

Từ vựng trong mục: Function — hàm; **Parameter** — tham số.

Hàm / cú pháp	Cách sử dụng
`np.mean()/np.median()`	Trung bình và trung vị, median bớt nhạy với giá trị cực lớn
`np.var(x,ddof=0/1)`	ddof=0 cho quần thể, ddof=1 cho mẫu
`np.std(x,ddof=0/1)`	Độ lệch chuẩn nhất quán quy ước ddof
`pd.Series.mode()/value_counts()`	Giá trị phổ biến và tần suất

4. Khái niệm chính và liên quan

Mạch kiến thức: Khi có nhiều số đo, mean và median trả lời câu hỏi giá trị ở giữa; variance và std đo mức phân tán. Mode trả lời câu hỏi nhãn nào xuất hiện nhiều nhất. Cần biết đơn vị và đang tính theo quần thể hay mẫu.

4.1. Mean — Điểm cân bằng của các quan sát

Từ vựng: Mean — trung bình; **Observation** — quan sát.

Mean cộng các giá trị rồi chia cho số quan sát. Ba đơn giao trong 2, 3, 4 ngày có trung bình `(2+3+4)/3=3` ngày. Mean sử dụng **mọi giá trị**, nên có thể bị kéo lệch nếu có một đơn rất chậm. Phải đảm bảo các thời gian dùng cùng đơn vị; mean của các mã sản phẩm không có ý nghĩa.

```
import numpy as np

days = np.array([2, 3, 4])
print(float(np.mean(days)))
```

Kết quả khi chạy:

```
3.0
```

Thời gian trung bình ba đơn là 3 ngày, không có nghĩa đơn nào cũng giao đúng 3 ngày.

4.2. Median — Vị trí giữa sau khi xếp thứ tự

Từ vựng: Median — trung vị; **Outlier** — giá trị khác biệt lớn.

Median là số ở giữa khi dữ liệu được **sắp tăng dần**; nếu có số quan sát chẵn thì là trung bình hai số giữa. Với giá trị đơn hàng `[100,120,10000]` nghìn đồng, trung vị 120 nghìn, còn mean vượt 3.400 nghìn vì đơn 10.000 quá lớn. Median thường giúp mô tả giá trị trung tâm khi dữ liệu lệch, nhưng không thay thế việc kiểm tra liệu giá trị bất thường có hợp lệ.

```
import numpy as np

amounts = [100, 120, 10000]
print(float(np.median(amounts)))
```

Kết quả khi chạy:

```
120.0
```

Trung vị 120 nghìn đồng ít bị ảnh hưởng bởi đơn 10.000 nghìn đồng.

4.3. Variance — Sai khác bình phương quanh mean

Từ vựng: Variance — phương sai; **ddof** — số bậc tự do trừ đi.

Phương sai đo độ phân tán bằng **trung bình bình phương sai lệch so với mean**. Điểm `[6,8,10]` có mean 8, các lệch `[-2,0,2]`, bình phương `[4,0,4]`. Phương sai **quần thể** `ddof=0` bằng `8/3≈2.667` điểm². Nếu dùng để **ước lượng từ mẫu**, phương sai mẫu với `ddof=1` là `8/2=4` điểm². Cần nêu rõ đang dùng mẫu hay toàn bộ quần thể.

```
import numpy as np

scores = [6, 8, 10]
print(round(float(np.var(scores, ddof=0)), 3))
```

Kết quả khi chạy:

```
2.667
```

Phương sai quần thể 2,667 điểm bình phương; không phải số điểm trung bình.

4.4. Standard deviation — Độ phân tán cùng đơn vị dữ liệu

Từ vựng: Standard deviation — độ lệch chuẩn; **Mode** — yếu vị.

Standard deviation (std) là căn bậc hai của phương sai, có **cùng đơn vị** với số đo. Cảm biến nhiệt độ đo `[20,21,19,20]` °C: mean=20 °C, std quần thể bằng `sqrt(0.5)≈0.707` °C. Trong khi đó **mode** là giá trị xuất hiện nhiều nhất, ở đây 20 °C hai lần; đây là câu hỏi về tần suất, không phải độ phân tán.

```
import numpy as np

celsius = [20, 21, 19, 20]
print(round(float(np.std(celsius, ddof=0)), 3))
```

Kết quả khi chạy:

```
0.707
```

Độ lệch chuẩn 0,707 °C mô tả dao động nhiệt quanh mean 20 °C.

5. Năm tình huống ứng dụng

Tình huống 1 — Trung bình thời gian giao ba đơn

Tình huống thực tế: Ba đơn hàng có thời gian giao lần lượt 2,3,4 ngày, theo cùng quy định đo. Quản lý muốn một chỉ số đại diện mức thời gian giao trung bình.

Vì sao chọn cách giải này? Mean phù hợp khi cần tổng thời gian trên mỗi đơn và dữ liệu không có ngoại lệ lớn.

Quy trình giải:

1. Đưa ba thời gian theo ngày vào một mảng.
2. Tính trung bình bằng `np.mean`.
3. In kết quả, chú ý không phải mọi đơn đều bằng mean.

```
import numpy as np

days = np.array([2, 3, 4])
print(float(np.mean(days)))
```

Kết quả khi chạy:

```
3.0
```

Trung bình ba đơn là 3 ngày.

Tình huống 2 — So mean và median hóa đơn

Tình huống thực tế: Cửa hàng có ba hóa đơn 100, 120 và 10.000 nghìn đồng. Cần trình bày một giá trị trung tâm ít bị hóa đơn cực lớn kéo lên.

Vì sao chọn cách giải này? Median bền hơn với giá trị cực đoan so với mean; in cả hai để thấy sự khác biệt.

Quy trình giải:

1. Lưu ba hóa đơn theo nghìn đồng.
2. Tính mean và median cùng bộ dữ liệu.
3. Làm tròn mean ba chữ số và in trung vị.

```
import numpy as np

amounts = [100, 120, 10000]
print(round(float(np.mean(amounts)), 3), float(np.median(amounts)))
```

Kết quả khi chạy:

```
3406.667 120.0
```

Mean cao bất thường nhưng trung vị 120 nghìn gần hai hóa đơn thông thường.

Tình huống 3 — Đo độ phân tán điểm thi

Tình huống thực tế: Ba điểm kiểm tra `[6,8,10]` (điểm trên thang 10) tạo toàn bộ nhóm cần mô tả. Cần tính phương sai **quần thể** của nhóm này.

Vì sao chọn cách giải này? Bình phương sai khác làm mất dấu âm, sau đó lấy trung bình trên cả ba điểm `ddof=0`.

Quy trình giải:

1. Tạo mảng điểm [6,8,10].
2. Tính `np.var(...,ddof=0)` vì đang mô tả toàn bộ nhóm.
3. In kết quả ba chữ số.

```
import numpy as np

scores = [6, 8, 10]
print(round(float(np.var(scores, ddof=0)), 3))
```

Kết quả khi chạy:

```
2.667
```

Phương sai quần thể khoảng 2,667 điểm².

Tình huống 4 — Kiểm tra độ dao động nhiệt độ

Tình huống thực tế: Cảm biến ghi 20,21,19,20 °C tại bốn thời điểm. Nhóm kiểm soát muốn đo mức dao động, giữ đơn vị °C để dễ so ngưỡng vận hành.

Vì sao chọn cách giải này? Std là căn phương sai, giữ cùng đơn vị nhiệt độ để có thể đọc trực tiếp.

Quy trình giải:

1. Lưu bốn nhiệt độ theo °C.
2. Dùng std quần thể `ddof=0`.
3. Làm tròn ba chữ số và in.

```
import numpy as np

celsius = [20, 21, 19, 20]
print(round(float(np.std(celsius, ddof=0)), 3))
```

Kết quả khi chạy:

```
0.707
```

Std khoảng 0,707 °C quanh mức trung bình 20 °C.

Tình huống 5 — Tìm nhãn sản phẩm xuất hiện nhiều nhất

Tình huống thực tế: Danh sách năm lượt mua mang mã `A,A,B,C,A`. Mỗi phần tử là một lượt, không phải số lượng tồn kho. Cần tìm mã được mua nhiều lượt nhất.

Vì sao chọn cách giải này? Mode hoặc `value_counts` phù hợp với nhãn phân loại; không lấy mean của chữ.

Quy trình giải:

1. Tạo Series nhãn từng lượt mua.
2. Dùng `value_counts` đếm tần suất.
3. Lấy nhãn đầu và số lượt mua.

```
import pandas as pd

products = pd.Series(['A', 'A', 'B', 'C', 'A'])
counts = products.value_counts()
print(counts.index[0], int(counts.iloc[0]))
```

Kết quả khi chạy:

```
A 3
```

Mã A xuất hiện ba lượt, nhiều nhất trong danh sách.

6. Sai lầm thường gặp

- Mean bị ảnh hưởng bởi ngoại lệ cực lớn; median thường bền hơn nhưng không phát hiện lỗi thay cho việc kiểm tra dữ liệu.
- `ddof=0` chia cho n; `ddof=1` chia cho n−1 và cần ít nhất hai quan sát để có ước lượng mẫu.
- Variance mang đơn vị bình phương; std mang đơn vị gốc.
- Mode/value_counts dành cho nhãn phân loại; không lấy trung bình của mã chữ.
- Đừng diễn giải std của quan sát thành sai số chuẩn của trung bình mẫu.
- Đổi dữ liệu về cùng đơn vị trước khi tính thống kê.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What does mean calculate? — Mean tính gì?

A. Sum divided by observation count — Tổng chia số quan sát

B. Middle ranked value after sorting — Giá trị ở giữa sau khi sắp

C. Squared dispersion around the mean — Độ phân tán bình phương quanh mean

D. Spread in original measurement units — Độ phân tán cùng đơn vị gốc

Đáp án: A

Giải thích: Mọi quan sát góp vào trung bình.

Q2. What is median? — Median là gì?

A. Sum divided by observation count — Tổng chia số quan sát

B. Middle ranked value after sorting — Giá trị ở giữa sau khi sắp

C. Squared dispersion around the mean — Độ phân tán bình phương quanh mean

D. Spread in original measurement units — Độ phân tán cùng đơn vị gốc

Đáp án: B

Giải thích: Trung vị dựa trên thứ tự, không dựa trên tổng độ lớn.

Q3. What does variance measure? — Variance đo gì?

A. Sum divided by observation count — Tổng chia số quan sát

B. Middle ranked value after sorting — Giá trị ở giữa sau khi sắp

C. Squared dispersion around the mean — Độ phân tán bình phương quanh mean

D. Spread in original measurement units — Độ phân tán cùng đơn vị gốc

Đáp án: C

Giải thích: Nó lấy trung bình bình phương độ lệch.

Q4. What does standard deviation measure? — Std đo gì?

A. Sum divided by observation count — Tổng chia số quan sát

B. Middle ranked value after sorting — Giá trị ở giữa sau khi sắp

C. Squared dispersion around the mean — Độ phân tán bình phương quanh mean

D. Spread in original measurement units — Độ phân tán cùng đơn vị gốc

Đáp án: D

Giải thích: Std là căn bậc hai phương sai.

Q5. What is mode? — Mode là gì?

A. Most frequent observed value — Giá trị xuất hiện nhiều nhất

B. Divide sum of squares by n — Chia tổng bình phương cho n

C. Divide by n−1 — Chia cho n−1

D. Frequency per distinct label — Tần suất từng nhãn khác nhau

Đáp án: A

Giải thích: Yếu vị dùng được cho nhãn lẫn số.

Q6. What is ddof=0 in np.var? — ddof=0 trong np.var là gì?

A. Most frequent observed value — Giá trị xuất hiện nhiều nhất

B. Divide sum of squares by n — Chia tổng bình phương cho n

C. Divide by n−1 — Chia cho n−1

D. Frequency per distinct label — Tần suất từng nhãn khác nhau

Đáp án: B

Giải thích: Nó tính phương sai theo quy ước quần thể.

Q7. What is ddof=1 for sample variance? — ddof=1 dùng cho gì?

A. Most frequent observed value — Giá trị xuất hiện nhiều nhất

B. Divide sum of squares by n — Chia tổng bình phương cho n

C. Divide by n−1 — Chia cho n−1

D. Frequency per distinct label — Tần suất từng nhãn khác nhau

Đáp án: C

Giải thích: Đó là ước lượng phương sai mẫu thông dụng.

Q8. What does value_counts() produce? — value_counts() tạo gì?

A. Most frequent observed value — Giá trị xuất hiện nhiều nhất

B. Divide sum of squares by n — Chia tổng bình phương cho n

C. Divide by n−1 — Chia cho n−1

D. Frequency per distinct label — Tần suất từng nhãn khác nhau

Đáp án: D

Giải thích: Nó đếm lượt xuất hiện của từng nhãn.

Q9. What is mean of 2,3,4? — Mean của 2,3,4 là bao nhiêu?

A. 3 — 3

B. 4 — 4

C. 2/3 — 2/3

D. 1 — 1

Đáp án: A

Giải thích: Tổng 9 chia 3 bằng 3.

Q10. What is median of 2,4,100? — Median của 2,4,100 là bao nhiêu?

A. 3 — 3

B. 4 — 4

C. 2/3 — 2/3

D. 1 — 1

Đáp án: B

Giải thích: Bốn nằm giữa dãy đã sắp.

Q11. What is population variance of 1,2,3? — Phương sai quần thể của 1,2,3 là gì?

A. 3 — 3

B. 4 — 4

C. 2/3 — 2/3

D. 1 — 1

Đáp án: C

Giải thích: Bình phương lệch là 1,0,1 rồi chia ba.

Q12. What is sample variance of 1,2,3? — Phương sai mẫu của 1,2,3 là gì?

A. 3 — 3

B. 4 — 4

C. 2/3 — 2/3

D. 1 — 1

Đáp án: D

Giải thích: Tổng bình phương 2 chia $n-1=2$.

Q13. What unit does variance have? — Variance có đơn vị gì?

A. Squared input units — Bình phương đơn vị dữ liệu

B. Original input units — Đơn vị gốc của dữ liệu

C. Median — Trung vị

D. Mode — Yếu vị

Đáp án: A

Giải thích: Bình phương lệch làm bình phương đơn vị.

Q14. What unit does std have? — Std có đơn vị gì?

A. Squared input units — Bình phương đơn vị dữ liệu

B. Original input units — Đơn vị gốc của dữ liệu

C. Median — Trung vị

D. Mode — Yếu vị

Đáp án: B

Giải thích: Lấy căn đưa về đơn vị ban đầu.

Q15. Which statistic resists large outliers better? — Thống kê nào ít nhạy với ngoại lệ cực lớn hơn?

A. Squared input units — Bình phương đơn vị dữ liệu

B. Original input units — Đơn vị gốc của dữ liệu

C. Median — Trung vị

D. Mode — Yếu vị

Đáp án: C

Giải thích: Giá trị cực lớn ít tác động vị trí giữa.

Q16. Which statistic is useful for categorical labels? — Thống kê nào phù hợp nhãn phân loại?

A. Squared input units — Bình phương đơn vị dữ liệu

B. Original input units — Đơn vị gốc của dữ liệu

C. Median — Trung vị

D. Mode — Yếu vị

Đáp án: D

Giải thích: Lấy trung bình mã chữ không có ý nghĩa số học.

Q17. Does std equal standard error? — Std có giống sai số chuẩn không?

A. No, std measures spread of individual values — Không, std đo độ phân tán quan sát

B. Yes, large outliers can pull it up — Có, ngoại lệ lớn kéo mean lên

C. Different denominators give different results — Mẫu số khác nhau cho kết quả khác

D. Summary statistics can lose meaning — Thống kê tổng hợp có thể mất ý nghĩa

Đáp án: A

Giải thích: SE đo độ biến thiên của thống kê ước lượng.

Q18. Can mean lie above most observations? — Mean có thể lớn hơn phần lớn quan sát không?

A. No, std measures spread of individual values — Không, std đo độ phân tán quan sát

B. Yes, large outliers can pull it up — Có, ngoại lệ lớn kéo mean lên

C. Different denominators give different results — Mẫu số khác nhau cho kết quả khác

D. Summary statistics can lose meaning — Thống kê tổng hợp có thể mất ý nghĩa

Đáp án: B

Giải thích: Mean phụ thuộc tổng các giá trị.

Q19. Why state ddof when reporting variance? — Vì sao báo ddof khi trình bày phương sai?

A. No, std measures spread of individual values — Không, std đo độ phân tán quan sát

B. Yes, large outliers can pull it up — Có, ngoại lệ lớn kéo mean lên

C. Different denominators give different results — Mẫu số khác nhau cho kết quả khác

D. Summary statistics can lose meaning — Thống kê tổng hợp có thể mất ý nghĩa

Đáp án: C

Giải thích: Quy ước mẫu và quần thể khác nhau.

Q20. What happens if values use mismatched units? — Nếu số liệu dùng đơn vị không thống nhất thì sao?

A. No, std measures spread of individual values — Không, std đo độ phân tán quan sát

B. Yes, large outliers can pull it up — Có, ngoại lệ lớn kéo mean lên

C. Different denominators give different results — Mẫu số khác nhau cho kết quả khác

D. Summary statistics can lose meaning — Thống kê tổng hợp có thể mất ý nghĩa

Đáp án: D

Giải thích: Cần thống nhất đơn vị trước khi tính.

8. Liên hệ bài tiếp theo

`NumPy/Pandas → Thống kê mô tả → Outlier → ML.`

PY-38 — Quantile, IQR, outlier, skewness và kurtosis

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Quantile — phân vị; **IQR** — khoảng tứ phân vị.

`Thống kê mô tả → Quantile/IQR → Outlier → Kiểm tra dữ liệu ML`

Học xong dùng để làm gì? Dùng Q1/Q3/IQR phát hiện quan sát nằm xa phân bố, phân biệt ngoại lệ thật và nhập liệu sai.

Kết nối kiến thức tiếp theo: PY-39 xem kết quả thống kê thay đổi như thế nào khi lấy mẫu nhiều lần.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Quantile	phân vị	Giá trị chia phân phối theo tỷ lệ tích lũy
IQR	khoảng tứ phân vị	Khoảng cách Q3-Q1, mô tả 50% dữ liệu giữa
Skewness	độ lệch phân phối	Mức bất đối xứng của phân phối quanh tâm
Kurtosis	độ nhọn/đuôi phân phối	Chỉ số mô tả đặc điểm đuôi so với chuẩn theo quy ước

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`np.quantile(x,[.25,.5,.75])`	Tìm các phân vị 25%, 50%, 75%
`Q1 - 1.5*IQR` và `Q3 + 1.5*IQR`	Ngưỡng dưới và trên để gắn cờ giá trị cần rà soát
`pd.Series.skew()`	Đánh giá bất đối xứng, nhạy với cỡ mẫu
`pd.Series.kurt()`	Excess kurtosis theo quy ước Pandas

4. Khái niệm chính và liên quan

Mạch kiến thức: Phân vị cho biết vị trí của một giá trị trong dãy đã sắp; IQR đo độ rộng phần giữa để sàng lọc ngoại lệ; skewness nói về bất đối xứng và kurtosis liên quan đặc điểm đuôi. Bất thường không đồng nghĩa sai dữ liệu.

4.1. Quantile — Mốc chia thứ tự dữ liệu

Từ vựng: Quantile — phân vị; **Q1/Q2/Q3** — tứ phân vị.

Phân vị q là mốc trên dãy số đã sắp, liên quan đến tỷ lệ giá trị ở phía dưới. **Q1** ứng với mức 25%, **Q2** là trung vị 50%, **Q3** ứng với 75%. Ví dụ năm hóa đơn 10,20,30,40,50 nghìn đồng: `np.quantile(x,0.75)` trả 40. Với bộ dữ liệu nhỏ, phương pháp **nội suy** giữa vị trí của các số làm cho kết quả phụ thuộc quy ước từng thư viện; không phải lúc nào phân vị cũng là số gốc trong dữ liệu.

```
import numpy as np

amounts = [10, 20, 30, 40, 50]
print(float(np.quantile(amounts, 0.75)))
```

Kết quả khi chạy:

```
40.0
```

Mốc phân vị 75% bằng 40 nghìn đồng theo phương pháp nội suy mặc định NumPy.

4.2. IQR — Độ rộng nửa giữa bộ dữ liệu

Từ vựng: IQR — khoảng tứ phân vị; **Q1/Q3** — phân vị 25%/75%.

IQR = Q3−Q1 đo độ rộng của khoảng chứa phần giữa dữ liệu. Với `[1,2,3,4,5]`, Q1=2 và Q3=4, nên IQR=2 đơn vị. Một cách **gắn cờ** ngoại lệ là ngoài các ngưỡng `Q1−1.5×IQR` hoặc `Q3+1.5×IQR`. Đây chỉ là quy tắc rà soát, **không chứng minh dữ liệu sai**; chẳng hạn một hóa đơn lớn có thể hoàn toàn hợp lệ.

```
import numpy as np

x = [1, 2, 3, 4, 5]
q1, q3 = np.quantile(x, [0.25, 0.75])
print(float(q3 - q1))
```

Kết quả khi chạy:

```
2.0
```

Nửa giữa của dữ liệu trải rộng 2 đơn vị theo quy tắc phân vị đã chọn.

4.3. Skewness — Độ bất đối xứng của phân phối

Từ vựng: Skewness — độ lệch; **Right tail** — đuôi phải.

Skewness đo mức bất đối xứng. Một dãy như `[1,1,2,8]` có ba số nhỏ và một số lớn, khiến **đuôi phải kéo dài** và độ lệch dương. `pd.Series.skew()` trả hệ số skew đã được hiệu chỉnh theo công thức Pandas; tính trên mẫu ít quan sát rất kém ổn định. Không suy ra mọi giá trị lớn đều là lỗi từ skew dương.

```
import pandas as pd

x = pd.Series([1, 1, 2, 8])
print(round(float(x.skew()), 3))
```

Kết quả khi chạy:

```
1.887
```

Độ lệch dương khoảng 1,887 gợi ý đuôi phải dài hơn đuôi trái.

4.4. Kurtosis — Đặc điểm đuôi theo quy ước

Từ vựng: Kurtosis — độ nhọn/hệ số liên quan đến đuôi; **Excess kurtosis** — kurtosis dư.

Kurtosis liên quan đến các moment bậc bốn và độ nặng của đuôi phân phối; không nên hiểu đơn giản chỉ là **độ cao của đỉnh**. Pandas `Series.kurt()` trả **excess kurtosis**, lấy phân phối chuẩn làm mốc 0. Với dãy `[2,3,3,4,4,5,5,6,7,20]`, số 20 rất lớn có thể làm kurtosis dư dương. Trên mẫu nhỏ, chỉ số này biến động mạnh.

```
import pandas as pd

x = pd.Series([2, 3, 3, 4, 4, 5, 5, 6, 7, 20])
print(round(float(x.kurt()), 3))
```

Kết quả khi chạy:

```
7.824
```

Excess kurtosis dương lớn cho thấy mẫu này có hành vi đuôi bất thường so với dạng chuẩn.

5. Năm tình huống ứng dụng

Tình huống 1 — Mốc 75% của giá trị đơn hàng

Tình huống thực tế: Năm hóa đơn có giá trị 10,20,30,40,50 nghìn đồng. Bộ phận kinh doanh muốn mốc phân vị 75% để mô tả vị trí của hóa đơn cao trong nhóm.

Vì sao chọn cách giải này? Phân vị xét vị trí theo thứ tự đã sắp, không dựa trên trung bình mọi hóa đơn.

Quy trình giải:

1. Lưu năm giá trị theo nghìn đồng.
2. Tính phân vị 0.75 bằng `np.quantile`.
3. In mốc theo nghìn đồng và nêu quy ước nội suy.

```
import numpy as np

amounts = [10, 20, 30, 40, 50]
print(float(np.quantile(amounts, 0.75)))
```

Kết quả khi chạy:

```
40.0
```

Mốc 75% bằng 40 nghìn đồng theo quy ước NumPy mặc định.

Tình huống 2 — Gắn cờ hóa đơn lớn cần kiểm tra

Tình huống thực tế: Tám hóa đơn có số tiền `[10,11,12,12,13,15,16,90]` nghìn đồng. Kế toán muốn tìm những hóa đơn vượt ngưỡng IQR để xem lại chứng từ, không xóa ngay.

Vì sao chọn cách giải này? IQR đo phần giữa và ngưỡng 1.5×IQR tạo danh sách điểm cần xác minh.

Quy trình giải:

1. Tính Q1, Q3 và IQR trên toàn bộ tám hóa đơn.
2. Lập ngưỡng dưới và trên bằng 1.5×IQR.
3. Lọc số tiền ngoài hai ngưỡng và in List.

```
import numpy as np

amounts = np.array([10, 11, 12, 12, 13, 15, 16, 90])
q1, q3 = np.quantile(amounts, [0.25, 0.75])
iqr = q3 - q1
outliers = amounts[(amounts < q1 - 1.5 * iqr) | (amounts > q3 + 1.5 * iqr)]
print(outliers.tolist())
```

Kết quả khi chạy:

```
[90]
```

Hóa đơn 90 nghìn đồng bị gắn cờ để xác minh; không kết luận đây là lỗi ghi chép.

Tình huống 3 — Kiểm tra độ lệch phải của dữ liệu

Tình huống thực tế: Bốn lần đo `[1,1,2,8]` cùng đơn vị có một giá trị lớn hơn nhiều. Nhóm phân tích muốn biết phân phối mẫu lệch về phía giá trị lớn hay nhỏ.

Vì sao chọn cách giải này? Skewness dương thể hiện đuôi phải dài hơn; `Series.skew` tính hệ số phù hợp cho dữ liệu một chiều.

Quy trình giải:

1. Tạo Series bốn số đo.
2. Tính hệ số skew theo quy ước Pandas.
3. Làm tròn ba chữ số rồi đánh giá dấu, không xem là bằng chứng dữ liệu sai.

```
import pandas as pd

measurements = pd.Series([1, 1, 2, 8])
print(round(float(measurements.skew()), 3))
```

Kết quả khi chạy:

```
1.887
```

Skew dương gợi ý đuôi phải dài; cỡ mẫu bốn quan sát còn rất nhỏ.

Tình huống 4 — Đánh giá đặc điểm đuôi của một mẫu

Tình huống thực tế: Mười số đo cùng đơn vị `[2,3,3,4,4,5,5,6,7,20]` có một số lớn. Nhóm phân tích cần xem hệ số kurtosis dư có đáng chú ý không.

Vì sao chọn cách giải này? Kurtosis dư nhạy với chênh lệch lớn bậc bốn, giúp so đặc điểm đuôi theo quy ước Pandas.

Quy trình giải:

1. Tạo Series mười số đo.
2. Dùng `kurt()` tính excess kurtosis.
3. Làm tròn ba chữ số và nêu giới hạn mẫu nhỏ.

```
import pandas as pd

measurements = pd.Series([2, 3, 3, 4, 4, 5, 5, 6, 7, 20])
print(round(float(measurements.kurt()), 3))
```

Kết quả khi chạy:

```
7.824
```

Mẫu có kurtosis dư dương cao, nhưng chỉ là dấu hiệu phân bố, không phải phép kết luận chất lượng dữ liệu.

Tình huống 5 — Điền giá trị thiếu trong nhóm có số cực lớn

Tình huống thực tế: Ba hóa đơn là 100,120,10.000 nghìn đồng và có một ô chưa biết. Muốn điền tạm ô thiếu bằng trung vị của **các số quan sát được** trước khi dùng thử mô hình.

Vì sao chọn cách giải này? Median bền hơn mean khi có giá trị cực lớn; số điền chỉ là ước lượng, không phải dữ liệu thật.

Quy trình giải:

1. Tạo Series gồm ba giá trị và một None.
2. Tính median từ ba số không thiếu.
3. Điền None bằng median và in danh sách để kiểm tra.

```
import pandas as pd

amounts = pd.Series([100, 120, 10000, None])
print(amounts.fillna(amounts.median()).tolist())
```

Kết quả khi chạy:

```
[100.0, 120.0, 10000.0, 120.0]
```

Ô thiếu được điền tạm 120 nghìn đồng; với ML cần tính median trên train để tránh rò rỉ.

6. Sai lầm thường gặp

- Q1 và Q3 phụ thuộc phương pháp nội suy, nhất là với mẫu nhỏ; cần giữ nhất quán giữa các thư viện.
- IQR dùng ngưỡng **gợi ý** ngoại lệ, không chứng minh giá trị là lỗi hay phải xóa.
- Skew dương nói về đuôi phải; skew âm nói về đuôi trái, không đồng nghĩa mean luôn lớn/nhỏ trong mọi dữ liệu.
- Kurtosis liên quan moment bậc bốn/đuôi, không chỉ độ cao đỉnh; Pandas trả **excess kurtosis**.
- Skew và kurt mẫu nhỏ không ổn định; cần xem đồ thị và nguồn dữ liệu.
- Khi điền khuyết cho ML, tính median trên train, không lấy thông tin test.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a quantile? — Phân vị là gì?

A. A value associated with a cumulative rank — Giá trị gắn với một mức thứ hạng tích lũy

B. The difference Q3−Q1 — Hiệu Q3−Q1

C. Distribution asymmetry — Mức bất đối xứng phân phối

D. Fourth-moment tail behavior relative to normal — Đặc trưng moment bậc bốn/đuôi so với chuẩn

Đáp án: A

Giải thích: Phân vị chia dữ liệu sắp thứ tự theo tỷ lệ.

Q2. What is IQR? — IQR là gì?

A. A value associated with a cumulative rank — Giá trị gắn với một mức thứ hạng tích lũy

B. The difference Q3−Q1 — Hiệu Q3−Q1

C. Distribution asymmetry — Mức bất đối xứng phân phối

D. Fourth-moment tail behavior relative to normal — Đặc trưng moment bậc bốn/đuôi so với chuẩn

Đáp án: B

Giải thích: Nó đo độ rộng nửa giữa dữ liệu.

Q3. What does skewness measure? — Skewness đo gì?

A. A value associated with a cumulative rank — Giá trị gắn với một mức thứ hạng tích lũy

B. The difference Q3−Q1 — Hiệu Q3−Q1

C. Distribution asymmetry — Mức bất đối xứng phân phối

D. Fourth-moment tail behavior relative to normal — Đặc trưng moment bậc bốn/đuôi so với chuẩn

Đáp án: C

Giải thích: Dấu gợi ý phía đuôi dài.

Q4. What does excess kurtosis measure? — Excess kurtosis mô tả gì?

A. A value associated with a cumulative rank — Giá trị gắn với một mức thứ hạng tích lũy

B. The difference Q3−Q1 — Hiệu Q3−Q1

C. Distribution asymmetry — Mức bất đối xứng phân phối

D. Fourth-moment tail behavior relative to normal — Đặc trưng moment bậc bốn/đuôi so với chuẩn

Đáp án: D

Giải thích: Pandas dùng mốc 0 cho chuẩn.

Q5. What is the Q2 quantile? — Q2 là phân vị nào?

A. The median — Trung vị

B. The 75th-percentile value — Giá trị phân vị 75%

C. Q3+1.5×IQR — Q3+1.5×IQR

D. Q1−1.5×IQR — Q1−1.5×IQR

Đáp án: A

Giải thích: Q2 tương ứng phân vị 50%.

Q6. What does np.quantile(x,0.75) return? — np.quantile(x,0.75) trả gì?

A. The median — Trung vị

B. The 75th-percentile value — Giá trị phân vị 75%

C. Q3+1.5×IQR — Q3+1.5×IQR

D. Q1−1.5×IQR — Q1−1.5×IQR

Đáp án: B

Giải thích: Đây là Q3 theo quy ước nội suy đã chọn.

Q7. How is an upper IQR fence computed? — Ngưỡng trên IQR được tính sao?

A. The median — Trung vị

B. The 75th-percentile value — Giá trị phân vị 75%

C. Q3+1.5×IQR — Q3+1.5×IQR

D. Q1−1.5×IQR — Q1−1.5×IQR

Đáp án: C

Giải thích: Giá trị vượt được gắn cờ rà soát.

Q8. How is a lower IQR fence computed? — Ngưỡng dưới IQR được tính sao?

A. The median — Trung vị

B. The 75th-percentile value — Giá trị phân vị 75%

C. Q3+1.5×IQR — Q3+1.5×IQR

D. Q1−1.5×IQR — Q1−1.5×IQR

Đáp án: D

Giải thích: Giá trị dưới là ứng viên ngoại lệ.

Q9. What does positive skew suggest? — Skew dương gợi ý gì?

A. A longer right tail — Đuôi phải dài hơn

B. A longer left tail — Đuôi trái dài hơn

C. Excess kurtosis — Kurtosis dư

D. A skewness statistic — Hệ số độ lệch phân phối

Đáp án: A

Giải thích: Giá trị lớn kéo đuôi sang phải.

Q10. What does negative skew suggest? — Skew âm gợi ý gì?

A. A longer right tail — Đuôi phải dài hơn

B. A longer left tail — Đuôi trái dài hơn

C. Excess kurtosis — Kurtosis dư

D. A skewness statistic — Hệ số độ lệch phân phối

Đáp án: B

Giải thích: Giá trị nhỏ kéo đuôi về trái.

Q11. What does Series.kurt() return? — Series.kurt() trả gì?

A. A longer right tail — Đuôi phải dài hơn

B. A longer left tail — Đuôi trái dài hơn

C. Excess kurtosis — Kurtosis dư

D. A skewness statistic — Hệ số độ lệch phân phối

Đáp án: C

Giải thích: Pandas dùng quy ước kurtosis dư.

Q12. What does Series.skew() return? — Series.skew() trả gì?

A. A longer right tail — Đuôi phải dài hơn

B. A longer left tail — Đuôi trái dài hơn

C. Excess kurtosis — Kurtosis dư

D. A skewness statistic — Hệ số độ lệch phân phối

Đáp án: D

Giải thích: Nó ước lượng bất đối xứng từ mẫu.

Q13. What is Q3−Q1 called? — Q3−Q1 gọi là gì?

A. Interquartile range — Khoảng tứ phân vị

B. 3 — 3

C. 5 — 5

D. No; legitimate extremes may be flagged — Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

Đáp án: A

Giải thích: Hiệu này chính là IQR.

Q14. What is median of 1,3,5? — Median của 1,3,5 bằng gì?

A. Interquartile range — Khoảng tứ phân vị

B. 3 — 3

C. 5 — 5

D. No; legitimate extremes may be flagged — Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

Đáp án: B

Giải thích: Giá trị đứng giữa là ba.

Q15. What is IQR if Q1=2 and Q3=7? — IQR nếu Q1=2, Q3=7 bằng bao nhiêu?

A. Interquartile range — Khoảng tứ phân vị

B. 3 — 3

C. 5 — 5

D. No; legitimate extremes may be flagged — Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

Đáp án: C

Giải thích: Bảy trừ hai bằng năm.

Q16. Does an IQR outlier flag prove a measurement error? — Gắn cờ outlier theo IQR có chứng minh số đo sai không?

A. Interquartile range — Khoảng tứ phân vị

B. 3 — 3

C. 5 — 5

D. No; legitimate extremes may be flagged — Không; giá trị cực lớn hợp lệ vẫn có thể bị gắn cờ

Đáp án: D

Giải thích: Ngưỡng chỉ là quy tắc sàng lọc.

Q17. Why can quantile values differ between software packages? — Vì sao phân vị có thể khác giữa các phần mềm?

A. Different interpolation conventions — Quy ước nội suy khác nhau

B. It concerns fourth moments and tails — Nó liên quan moment bậc bốn và đuôi

C. Median — Trung vị

D. Fit the imputation value on train only — Tính giá trị điền chỉ từ train

Đáp án: A

Giải thích: Mẫu nhỏ dễ cho thấy sự khác biệt quy ước.

Q18. Why should you avoid interpreting high kurtosis only as a tall peak? — Vì sao không nên hiểu kurtosis cao chỉ là đỉnh cao?

A. Different interpolation conventions — Quy ước nội suy khác nhau

B. It concerns fourth moments and tails — Nó liên quan moment bậc bốn và đuôi

C. Median — Trung vị

D. Fit the imputation value on train only — Tính giá trị điền chỉ từ train

Đáp án: B

Giải thích: Chỉ độ cao đỉnh không đủ diễn giải.

Q19. What statistic is often robust for imputation with extreme values? — Thống kê nào thường phù hợp điền thiếu khi có ngoại lệ lớn?

A. Different interpolation conventions — Quy ước nội suy khác nhau

B. It concerns fourth moments and tails — Nó liên quan moment bậc bốn và đuôi

C. Median — Trung vị

D. Fit the imputation value on train only — Tính giá trị điền chỉ từ train

Đáp án: C

Giải thích: Thứ tự khiến trung vị ít nhạy với cực trị.

Q20. What is a crucial train/test rule when imputing missing values? — Quy tắc train/test quan trọng khi điền khuyết là gì?

A. Different interpolation conventions — Quy ước nội suy khác nhau

B. It concerns fourth moments and tails — Nó liên quan moment bậc bốn và đuôi

C. Median — Trung vị

D. Fit the imputation value on train only — Tính giá trị điền chỉ từ train

Đáp án: D

Giải thích: Dữ liệu test không tham gia học tham số tiền xử lý.

8. Liên hệ bài kế tiếp

`Thống kê mô tả → Quantile/IQR → Outlier → Kiểm tra dữ liệu ML.`

PY-39 — Lấy mẫu, CLT và sai số chuẩn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Population — tổng thể; **Sample** — mẫu.

`Thống kê → Sampling → Central Limit Theorem → Confidence Interval`

Học xong dùng để làm gì? Ước lượng đặc trưng tổng thể từ mẫu, nhận biết sai số lấy mẫu và quy luật của trung bình mẫu.

Kết nối kiến thức tiếp theo: PY-40 dùng sai số đó để xây khoảng tin cậy và kiểm định giả thuyết.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Population	tổng thể	Toàn bộ đối tượng cần nghiên cứu
Sample	mẫu	Nhóm con lấy từ tổng thể theo cách rõ ràng
CLT	định lý giới hạn trung tâm	Phân phối trung bình mẫu tiến gần chuẩn theo điều kiện thích hợp
Standard error	sai số chuẩn	Độ biến thiên của một thống kê giữa các mẫu

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`df.sample(n=,random_state=)`	Lấy mẫu có thể lặp, chú ý phương pháp lấy mẫu
`rng.choice(values,size=,replace=)`	Lấy mẫu thay thế hoặc không thay thế
`np.mean(samples,axis=1)`	Tính trung bình từng lần lấy mẫu
`s / np.sqrt(n)`	Ước lượng SE của mean từ std mẫu s

4. Khái niệm chính và liên quan

Mạch kiến thức: Tổng thể là nhóm cần kết luận, mẫu là những quan sát thực sự lấy. Lặp lấy mẫu sinh ra nhiều trung bình mẫu; CLT giải thích xu hướng phân phối của chúng, còn sai số chuẩn đo mức dao động của thống kê giữa các mẫu.

4.1. Population — Toàn bộ nhóm cần nghiên cứu

Từ vựng: Population — tổng thể; **Unit** — đơn vị quan sát.

Tổng thể là toàn bộ các đối tượng mà ta muốn kết luận, như 100 hóa đơn của một cửa hàng trong một ngày. Mỗi hóa đơn là **một đơn vị quan sát**; các cột có thể là giá trị, thời gian, trạng thái giao. Nếu chỉ đo 5 hóa đơn, kết luận không tự mở rộng thành 100 hóa đơn mà không xem cách chọn mẫu. Trong ví dụ, tổng thể gồm đúng 100 mã 0–99, nên `len` bằng 100.

```
population = list(range(100))
print(len(population))
```

Kết quả khi chạy:

```
100
```

Tổng thể có 100 mã hóa đơn; số lượng tổng thể là N=100.

4.2. Sample — Nhóm con lấy theo cách xác định

Từ vựng: Sample — mẫu; **Sampling** — lấy mẫu.

Mẫu là một phần các đối tượng được chọn từ tổng thể. Ví dụ từ 5 mã hóa đơn `[1,2,3,4,5]`, chọn 3 mã **không hoàn lại** nghĩa là mỗi mã xuất hiện tối đa một lần trong mẫu. `rng.choice(...,replace=False)` cho chọn mẫu giả ngẫu nhiên, seed giúp tái lập. Chọn ngẫu nhiên hợp lý hỗ trợ giảm thiên lệch chọn mẫu, nhưng nếu dữ liệu nguồn đã thiếu nhóm quan trọng thì seed không thể khắc phục.

```
import numpy as np

rng = np.random.default_rng(1)
sample = rng.choice([1, 2, 3, 4, 5], size=3, replace=False)
print(sample.tolist())
```

Kết quả khi chạy:

```
[3, 2, 4]
```

Mẫu có ba mã khác nhau vì lấy không hoàn lại; đây là một kết quả ngẫu nhiên có thể tái lập.

4.3. CLT — Phân phối trung bình của các mẫu

Từ vựng: CLT — định lý giới hạn trung tâm; **Sampling distribution** — phân phối lấy mẫu.

CLT nói rằng dưới các điều kiện phù hợp như quan sát độc lập, cùng phân phối, phương sai hữu hạn, khi **cỡ mẫu n lớn**, phân phối của **trung bình mẫu** sau chuẩn hóa thường tiến gần dạng chuẩn, ngay cả khi dữ liệu gốc không chuẩn. Nó **không** nói dữ liệu gốc trở thành chuẩn hay từng mẫu đều có mean giống nhau. Mô phỏng 200 nhóm, mỗi nhóm 40 giá trị từ phân phối mũ mean lý thuyết 2, cho trung bình các mean gần 2.

```
import numpy as np

rng = np.random.default_rng(1)
```

```
groups = rng.exponential(scale=2, size=(200, 40))
means = groups.mean(axis=1)
print(round(float(means.mean()), 3))
```

Kết quả khi chạy:

```
2.011
```

200 trung bình mẫu xoay quanh giá trị kỳ vọng 2; một số mô phỏng không chứng minh CLT nhưng minh họa cách lấy mẫu lặp.

4.4. Standard error — Độ biến thiên của một ước lượng

Từ vựng: Standard error (SE) — sai số chuẩn; **Standard deviation (SD)** — độ lệch chuẩn.

SD mô tả độ phân tán của **quan sát riêng lẻ**, còn **SE của mean** mô tả mức dao động **giữa các trung bình mẫu** nếu lặp lấy mẫu cùng cách. Với n quan sát độc lập và ước lượng SD mẫu s, SE(mean) thường ước tính bằng `s/sqrt(n)` (trường hợp thích hợp). Năm điểm `[6,7,8,9,10]` có s≈1,581 điểm, nên SE≈0,707 điểm. Cỡ mẫu gấp 4 làm SE còn một nửa nếu độ phân tán giữ nguyên.

```
import numpy as np

scores = np.array([6, 7, 8, 9, 10], dtype=float)
se = scores.std(ddof=1) / np.sqrt(len(scores))
print(round(float(se), 3))
```

Kết quả khi chạy:

```
0.707
```

SE của trung bình mẫu xấp xỉ 0,707 điểm; nó không phải SD của từng điểm.

5. Năm tình huống ứng dụng

Tình huống 1 — Rút năm đơn trong hai mươi đơn để kiểm kê

Tình huống thực tế: Kho có đúng 20 đơn được gắn mã 1 đến 20; người kiểm kê chỉ đủ thời gian xem 5 đơn. Cần lấy mẫu không hoàn lại, mọi mã có cơ hội như nhau.

Vì sao chọn cách giải này? Lấy mẫu ngẫu nhiên không hoàn lại giúp chọn năm mã phân biệt; seed cố định chỉ phục vụ tái lập thử nghiệm.

Quy trình giải:

1. Tạo danh sách ID từ 1 đến 20.
2. Dùng `random.Random(42).sample` lấy năm ID không trùng.
3. In danh sách mã được chọn.

```
import random
```

```
ids = list(range(1, 21))
print(random.Random(42).sample(ids, 5))
```

Kết quả khi chạy:

```
[4, 1, 9, 8, 17]
```

Năm đơn được chọn không trùng nhau, nhưng kết quả không thay thế quy trình lấy mẫu thực tế của kho.

Tình huống 2 — Thử biến thiên trung bình với nguồn lệch phải

Tình huống thực tế: Một quá trình giả lập có thời gian chờ tuân phân phối mũ với trung bình 2 phút. Nhóm phân tích lấy 200 mẫu độc lập, mỗi mẫu gồm 40 thời gian chờ, để quan sát phân phối các mean.

Vì sao chọn cách giải này? Lấy nhiều nhóm rồi tính mean từng nhóm giúp minh họa biến thiên của trung bình mẫu trong CLT.

Quy trình giải:

1. Sinh bảng 200×40 số từ phân phối mũ có mean lý thuyết 2 phút.
2. Lấy mean theo từng hàng, được 200 trung bình mẫu.
3. In trung bình và SD giữa các mean sau làm tròn.

```
import numpy as np

rng = np.random.default_rng(1)
groups = rng.exponential(scale=2, size=(200, 40))
means = groups.mean(axis=1)
print(round(float(means.mean()), 3), round(float(means.std(ddof=1)), 3))
```

Kết quả khi chạy:

```
2.011 0.333
```

Các mean dao động quanh 2 phút; độ phân tán thực nghiệm giữa chúng khoảng 0,333 phút.

Tình huống 3 — Ước lượng sai số chuẩn của điểm trung bình

Tình huống thực tế: Trung tâm có năm điểm `[6,7,8,9,10]` cùng thang điểm. Muốn báo SE cho **trung bình mẫu**, không phải độ phân tán của từng học viên.

Vì sao chọn cách giải này? SE ước lượng bằng SD mẫu `ddof=1` chia căn cỡ mẫu khi giả định lấy mẫu độc lập phù hợp.

Quy trình giải:

1. Tạo mảng năm điểm.
2. Tính SD mẫu và chia cho căn của 5.
3. In SE sau làm tròn ba chữ số.

```
import numpy as np
```

```
scores = np.array([6, 7, 8, 9, 10], dtype=float)
print(round(float(scores.std(ddof=1) / np.sqrt(len(scores))), 3))
```

Kết quả khi chạy:

```
0.707
```

SE ước lượng khoảng 0,707 điểm. Mẫu chỉ có năm người nên còn bất định cao.

Tình huống 4 — Xem tác động của số người đo lên SE

Tình huống thực tế: Độ lệch chuẩn quần thể được giả định cố định σ=10 điểm. So hai nghiên cứu dùng n=25 và n=100 người, trong điều kiện lấy mẫu độc lập thích hợp.

Vì sao chọn cách giải này? SE của trung bình theo công thức σ/sqrt(n), nên n tăng bốn lần khiến SE giảm một nửa.

Quy trình giải:

1. Đặt sigma=10.
2. Với n=25 và n=100, chia sigma cho căn n.
3. In từng cỡ mẫu cùng SE, giữ đơn vị điểm.

```
import numpy as np

sigma = 10
for n in [25, 100]:
    print(n, sigma / np.sqrt(n))
```

Kết quả khi chạy:

```
25 2.0
100 1.0
```

Cỡ mẫu 100 cho SE 1 điểm, một nửa của SE khi n=25.

Tình huống 5 — Chọn khách từ bảng dữ liệu để khảo sát

Tình huống thực tế: Cửa hàng có bảng 10 khách, ID từ 0 tới 9. Nhóm chăm sóc khách muốn rút 3 ID không trùng từ DataFrame để mời khảo sát.

Vì sao chọn cách giải này? `DataFrame.sample` chọn hàng; `random_state` giữ ví dụ có kết quả xác định và có thể sắp ID trước khi in.

Quy trình giải:

1. Tạo bảng một cột `id` với 10 khách.
2. Lấy ba hàng bằng `.sample(n=3,random_state=42)`.
3. Chỉ lấy ID, sắp tăng để đầu ra dễ đối chiếu.

```
import pandas as pd

customers = pd.DataFrame({'id': list(range(10))})
```

```
sampled = customers.sample(n=3, random_state=42)
print(sorted(sampled['id'].tolist()))
```

Kết quả khi chạy:

```
[1, 5, 8]
```

Mẫu gồm ba ID khác nhau; cố định seed làm ví dụ tái lập, không bảo đảm tính đại diện của mọi nhóm khách.

6. Sai lầm thường gặp

- CLT nói về phân phối của **trung bình mẫu** dưới các điều kiện thích hợp, không biến dữ liệu gốc thành chuẩn.
- SD mô tả số đo cá thể, SE mô tả mức biến thiên ước lượng giữa các mẫu.
- Công thức SE≈s/√n cần giả định lấy mẫu phù hợp; phụ thuộc, chọn mẫu không ngẫu nhiên và quần thể hữu hạn có thể cần điều chỉnh.
- Seed giúp tái lập phép mô phỏng, **không** sửa thiên lệch chọn mẫu hoặc bảo đảm mẫu đại diện.
- Cỡ mẫu lớn thường giảm SE, nhưng không sửa được sai lệch có hệ thống.
- Không kết luận mọi mẫu sẽ gần trung bình thật hay một ngưỡng n cố định đúng cho mọi phân phối.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a population? — Tổng thể là gì?

A. All units in the target group — Toàn bộ đơn vị của nhóm cần nghiên cứu

B. A selected subset of the population — Một nhóm con được chọn từ tổng thể

C. Approximate sampling distribution of sample means — Phân phối gần đúng của các trung bình mẫu

D. Sampling variability of an estimator — Độ biến thiên của thống kê ước lượng qua các mẫu

Đáp án: A

Giải thích: Kết luận dự kiến hướng đến tổng thể.

Q2. What is a sample? — Mẫu là gì?

A. All units in the target group — Toàn bộ đơn vị của nhóm cần nghiên cứu

B. A selected subset of the population — Một nhóm con được chọn từ tổng thể

C. Approximate sampling distribution of sample means — Phân phối gần đúng của các trung bình mẫu

D. Sampling variability of an estimator — Độ biến thiên của thống kê ước lượng qua các mẫu

Đáp án: B

Giải thích: Mẫu chứa một phần quan sát.

Q3. What does CLT describe? — CLT mô tả gì?

A. All units in the target group — Toàn bộ đơn vị của nhóm cần nghiên cứu

B. A selected subset of the population — Một nhóm con được chọn từ tổng thể

C. Approximate sampling distribution of sample means — Phân phối gần đúng của các trung bình mẫu

D. Sampling variability of an estimator — Độ biến thiên của thống kê ước lượng qua các mẫu

Đáp án: C

Giải thích: CLT nói về mean qua nhiều mẫu, không nói mọi giá trị gốc thành chuẩn.

Q4. What is standard error? — SE là gì?

A. All units in the target group — Toàn bộ đơn vị của nhóm cần nghiên cứu

B. A selected subset of the population — Một nhóm con được chọn từ tổng thể

C. Approximate sampling distribution of sample means — Phân phối gần đúng của các trung bình mẫu

D. Sampling variability of an estimator — Độ biến thiên của thống kê ước lượng qua các mẫu

Đáp án: D

Giải thích: SE mô tả độ bất định của thống kê.

Q5. What does `replace=False` mean when sampling? — `replace=False` khi lấy mẫu nghĩa là gì?

A. Items cannot be selected twice in one sample — Không chọn lặp một phần tử trong cùng mẫu

B. Reproducible pseudorandom selection — Chọn giả ngẫu nhiên có thể lặp

C. Computes one mean per row — Tính một trung bình cho mỗi hàng

D. Three randomly sampled rows — Ba hàng được chọn ngẫu nhiên

Đáp án: A

Giải thích: Lấy mẫu không hoàn lại.

Q6. What does a random seed enable? — Seed ngẫu nhiên cho phép gì?

A. Items cannot be selected twice in one sample — Không chọn lặp một phần tử trong cùng mẫu

B. Reproducible pseudorandom selection — Chọn giả ngẫu nhiên có thể lặp

C. Computes one mean per row — Tính một trung bình cho mỗi hàng

D. Three randomly sampled rows — Ba hàng được chọn ngẫu nhiên

Đáp án: B

Giải thích: Nó kiểm soát chuỗi có thể tái tạo.

Q7. What does `mean(axis=1)` do on a 2-D sample table? — `mean(axis=1)` trên bảng mẫu 2-D làm gì?

A. Items cannot be selected twice in one sample — Không chọn lặp một phần tử trong cùng mẫu

B. Reproducible pseudorandom selection — Chọn giả ngẫu nhiên có thể lặp

C. Computes one mean per row — Tính một trung bình cho mỗi hàng

D. Three randomly sampled rows — Ba hàng được chọn ngẫu nhiên

Đáp án: C

Giải thích: Mỗi hàng có thể là một lần lấy mẫu.

Q8. What does `df.sample(n=3)` select? — `df.sample(n=3)` chọn gì?

A. Items cannot be selected twice in one sample — Không chọn lặp một phần tử trong cùng mẫu

B. Reproducible pseudorandom selection — Chọn giả ngẫu nhiên có thể lặp

C. Computes one mean per row — Tính một trung bình cho mỗi hàng

D. Three randomly sampled rows — Ba hàng được chọn ngẫu nhiên

Đáp án: D

Giải thích: Mặc định hàm lấy hàng của DataFrame.

Q9. What does sample SD measure? — SD mẫu đo gì?

A. Spread of individual values — Độ phân tán giá trị từng quan sát

B. Spread among hypothetical sample means — Độ phân tán giữa các mean mẫu giả định

C. s/sqrt(n) — s/sqrt(n)

D. It halves — Giảm một nửa

Đáp án: A

Giải thích: SD khác bất định của mean mẫu.

Q10. What does the SE of a mean measure? — SE của mean đo gì?

A. Spread of individual values — Độ phân tán giá trị từng quan sát

B. Spread among hypothetical sample means — Độ phân tán giữa các mean mẫu giả định

C. s/sqrt(n) — s/sqrt(n)

D. It halves — Giảm một nửa

Đáp án: B

Giải thích: Lấy mẫu lặp gây biến thiên trung bình.

Q11. What is a common estimated SE formula? — Công thức ước lượng SE thường dùng?

A. Spread of individual values — Độ phân tán giá trị từng quan sát

B. Spread among hypothetical sample means — Độ phân tán giữa các mean mẫu giả định

C. s/sqrt(n) — s/sqrt(n)

D. It halves — Giảm một nửa

Đáp án: C

Giải thích: Công thức phù hợp trong các giả định lấy mẫu độc lập thường gặp.

Q12. How does SE change when n quadruples and sigma is fixed? — SE đổi sao khi n gấp bốn, sigma giữ nguyên?

A. Spread of individual values — Độ phân tán giá trị từng quan sát

B. Spread among hypothetical sample means — Độ phân tán giữa các mean mẫu giả định

C. s/sqrt(n) — s/sqrt(n)

D. It halves — Giảm một nửa

Đáp án: D

Giải thích: Căn bậc hai của bốn là hai.

Q13. Does CLT assert that raw data become normally distributed? — CLT có nói dữ liệu gốc thành phân phối chuẩn không?

A. No, it concerns standardized sample means — Không, nói về mean mẫu được chuẩn hóa

B. No; sampling design matters — Không; thiết kế lấy mẫu mới quan trọng

C. It decreases for the sample mean under assumptions — Giảm đối với mean mẫu nếu giả định phù hợp

D. A systematically unrepresentative sample — Mẫu không đại diện một cách có hệ thống

Đáp án: A

Giải thích: Tổng thể lệch vẫn có thể lệch.

Q14. Is a sample necessarily representative because a seed is fixed? — Có seed cố định thì mẫu chắc chắn đại diện không?

A. No, it concerns standardized sample means — Không, nói về mean mẫu được chuẩn hóa

B. No; sampling design matters — Không; thiết kế lấy mẫu mới quan trọng

C. It decreases for the sample mean under assumptions — Giảm đối với mean mẫu nếu giả định phù hợp

D. A systematically unrepresentative sample — Mẫu không đại diện một cách có hệ thống

Đáp án: B

Giải thích: Seed chỉ lặp cách chọn.

Q15. What often happens to sampling variability as n grows? — n tăng thì dao động lấy mẫu thường thế nào?

A. No, it concerns standardized sample means — Không, nói về mean mẫu được chuẩn hóa

B. No; sampling design matters — Không; thiết kế lấy mẫu mới quan trọng

C. It decreases for the sample mean under assumptions — Giảm đối với mean mẫu nếu giả định phù hợp

D. A systematically unrepresentative sample — Mẫu không đại diện một cách có hệ thống

Đáp án: C

Giải thích: SE thường tỷ lệ nghịch căn n.

Q16. What is the impact of selection bias? — Thiên lệch chọn mẫu gây gì?

A. No, it concerns standardized sample means — Không, nói về mean mẫu được chuẩn hóa

B. No; sampling design matters — Không; thiết kế lấy mẫu mới quan trọng

C. It decreases for the sample mean under assumptions — Giảm đối với mean mẫu nếu giả định phù hợp

D. A systematically unrepresentative sample — Mẫu không đại diện một cách có hệ thống

Đáp án: D

Giải thích: Tăng mẫu không tự sửa cách chọn sai.

Q17. Does n=40 guarantee a good normal approximation in every problem? — n=40 có bảo đảm xấp xỉ chuẩn tốt ở mọi bài toán không?

A. No, underlying assumptions and tails matter — Không, còn phụ thuộc giả định và đuôi phân phối

B. Uses n−1 in variance denominator — Dùng n−1 ở mẫu số phương sai

C. Standard deviation — Độ lệch chuẩn

D. Clearly defined sampling assumptions — Giả định lấy mẫu được xác định rõ

Đáp án: A

Giải thích: Không có ngưỡng cỡ mẫu đúng cho mọi trường hợp.

Q18. What does `ddof=1` change in SD? — `ddof=1` đổi gì trong SD?

A. No, underlying assumptions and tails matter — Không, còn phụ thuộc giả định và đuôi phân phối

B. Uses n−1 in variance denominator — Dùng n−1 ở mẫu số phương sai

C. Standard deviation — Độ lệch chuẩn

D. Clearly defined sampling assumptions — Giả định lấy mẫu được xác định rõ

Đáp án: B

Giải thích: Đây là quy ước cho ước lượng phương sai mẫu.

Q19. Which quantity stays in original units, SD or variance? — Đại lượng nào giữ đơn vị gốc, SD hay variance?

A. No, underlying assumptions and tails matter — Không, còn phụ thuộc giả định và đuôi phân phối

B. Uses n−1 in variance denominator — Dùng n−1 ở mẫu số phương sai

C. Standard deviation — Độ lệch chuẩn

D. Clearly defined sampling assumptions — Giả định lấy mẫu được xác định rõ

Đáp án: C

Giải thích: Phương sai có đơn vị bình phương.

Q20. What is a prerequisite for repeated-sampling inference? — Điều kiện quan trọng của suy luận từ lấy mẫu lặp?

A. No, underlying assumptions and tails matter — Không, còn phụ thuộc giả định và đuôi phân phối

B. Uses n−1 in variance denominator — Dùng n−1 ở mẫu số phương sai

C. Standard deviation — Độ lệch chuẩn

D. Clearly defined sampling assumptions — Giả định lấy mẫu được xác định rõ

Đáp án: D

Giải thích: Sự phụ thuộc và cách chọn mẫu ảnh hưởng SE.

8. Liên hệ bài kế tiếp

`Thống kê → Sampling → Central Limit Theorem → Confidence Interval.`

PY-40 — Kiểm định giả thuyết, p-value và khoảng tin cậy

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Null hypothesis — giả thuyết không; **p-value** — giá trị p.

`CLT/SE → Hypothesis Testing → Confidence Interval → A/B Test`

Học xong dùng để làm gì? Đánh giá bằng chứng mẫu bằng p-value và khoảng tin cậy mà không kết luận chắc chắn quá mức.

Kết nối kiến thức tiếp theo: PY-41 chia dữ liệu train/test và đo chất lượng mô hình trên tập chưa dùng để học.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Null hypothesis	giả thuyết không	Phát biểu gốc đem kiểm định, ký hiệu H0
p-value	giá trị p	Độ bất thường của dữ liệu nếu giả định H0 đúng
T-test	kiểm định t	Kiểm định trung bình khi phương sai tổng thể chưa biết
Confidence interval	khoảng tin cậy	Khoảng tính theo quy trình có mức bao phủ dài hạn xác định

3. Hàm và công thức áp dụng

Công cụ	Ý nghĩa
`ttest_1samp(values, popmean)`	Kiểm định t một mẫu so với trung bình giả thuyết
`sem(values)`	Sai số chuẩn của mean mẫu, mặc định theo SD mẫu
`t.interval(confidence, df, loc, scale)`	Khoảng t với mức tin cậy, bậc tự do, trung bình và SE
`pvalue < alpha`	Quy tắc bác bỏ H_0 nếu p nhỏ hơn mức ý nghĩa đã chọn

4. Khái niệm chính và liên quan

Mạch kiến thức: Đầu tiên đặt giả thuyết H_0 và H_1. Kiểm định t tạo thống kê kiểm định và p-value dưới giả định H_0; so p với ngưỡng α để quyết định có bác bỏ hay không. Khoảng tin cậy dùng thống kê mẫu và sai số chuẩn để biểu diễn mức bất định của ước lượng theo quy trình bao phủ dài hạn.

4.1. Null hypothesis — Giả thuyết gốc cần kiểm định

Từ vựng: Null hypothesis (H_0) — giả thuyết không; **Alternative (H_1)** — giả thuyết đối.

H_0 là phát biểu gốc để đối chiếu dữ liệu, thường là "trung bình μ=50 điểm", còn H_1 có thể là "μ khác 50 điểm". Không biết trước giả thuyết nào đúng; ta xem dữ liệu mẫu có **quá bất thường nếu H_0 đúng** hay không. H_0 là một phát biểu về quần thể, không phải câu hỏi liệu mean của bảy điểm đo có đúng bằng 50. Ví dụ in cặp giả thuyết rõ ràng trước khi tính.

```
print('H0: population mean = 50')
print('H1: population mean != 50')
```

Kết quả khi chạy:

```
H0: population mean = 50
H1: population mean != 50
```

Hai phát biểu về trung bình quần thể khác với giá trị trung bình của riêng một mẫu.

4.2. p-value — Độ bất thường dưới giả định H_0

Từ vựng: p-value — giá trị p; **Significance level (α)** — mức ý nghĩa.

p-value là xác suất (nếu H_0 đúng và mô hình kiểm định phù hợp) thấy một thống kê kiểm định **ít nhất cực đoan như đã quan sát**. Nó **không** phải xác suất H_0 đúng hoặc xác suất kết quả do ngẫu nhiên. Ví dụ bảy điểm `[48,52,55,49,51,47,53]`, kiểm định hai phía so μ=50 bằng t-test cho p≈0.5347. Với α=0.05, chưa đủ bằng chứng để bác bỏ H_0; không có nghĩa chứng minh μ=50.

```
from scipy.stats import ttest_1samp

scores = [48, 52, 55, 49, 51, 47, 53]
p = ttest_1samp(scores, popmean=50).pvalue
print(round(float(p), 4))
```

Kết quả khi chạy:

```
0.5347
```

p≈0,5347 lớn hơn 0,05; chưa đủ bằng chứng bác bỏ H_0 với kiểm định hai phía.

4.3. T-test — So trung bình mẫu với mốc cho trước

Từ vựng: T-test — kiểm định t; **Standard error** — sai số chuẩn.

T-test một mẫu xét liệu trung bình của quần thể có thể khác một mốc μ_0 khi chưa biết phương sai quần thể. Thống kê t bằng $(\text{mean_mẫu} - \mu_0)/SE$, với $SE = s/\sqrt{n}$ và s là độ lệch chuẩn mẫu. Nếu mẫu `[8,9,10,11,12]` và μ_0=10, mean bằng mốc nên t=0 và p hai phía=1. Cần xét tính độc lập và mức phù hợp của phân phối/độ nhạy với ngoại lệ, nhất là mẫu nhỏ.

```
from scipy.stats import ttest_1samp

values = [8, 9, 10, 11, 12]
```

```
result = ttest_1samp(values, popmean=10)
print(round(float(result.statistic), 3))
```

Kết quả khi chạy:

```
0.0
```

Vì mean mẫu đúng bằng 10, thống kê kiểm định t=0.

4.4. Confidence interval — Khoảng tạo bởi quy trình suy luận

Từ vựng: Confidence interval (CI) — khoảng tin cậy; **Coverage** — tỷ lệ bao phủ.

Khoảng tin cậy 95% là khoảng được tạo bởi **một quy trình** sao cho trong các lần lấy mẫu lặp phù hợp, khoảng 95% các khoảng chứa tham số thật. Với một khoảng **đã tính**, μ thật là cố định; không diễn giải "xác suất 95% μ ở trong khoảng này" theo nghĩa tần suất. Ví dụ năm thời gian giao `[2,3,3,4,2]` ngày cho mean=2,8 và CI t 95% xấp xỉ `[1.761,3.839]` ngày. Mẫu nhỏ nên khoảng rộng và dựa vào giả định dữ liệu độc lập, phân phối phù hợp.

```
import numpy as np
from scipy.stats import t, sem

days = np.array([2.0, 3.0, 3.0, 4.0, 2.0])
ci = t.interval(0.95, df=len(days) - 1, loc=days.mean(), scale=sem(days))
print(np.round(ci, 3).tolist())
```

Kết quả khi chạy:

```
[1.761, 3.839]
```

Khoảng ước lượng mean theo t từ khoảng 1,761 đến 3,839 ngày; đây là khoảng cho trung bình quần thể.

5. Năm tình huống ứng dụng

Tình huống 1 — Kiểm định điểm trung bình so với chuẩn 50

Tình huống thực tế: Trung tâm có bảy điểm `[48,52,55,49,51,47,53]`, cùng thang 100 và giả định độc lập. Cần kiểm tra trung bình quần thể có khác chuẩn 50 theo kiểm định hai phía ở mức α=0.05 không.

Vì sao chọn cách giải này? T-test một mẫu so mean với mốc khi phương sai quần thể chưa biết.

Quy trình giải:

1. Đặt H_0: μ=50, H_1: μ≠50 và α=0.05 trước khi xem kết quả.
2. Tính `ttest_1samp` trên bảy điểm.
3. In p-value làm tròn và so với α.

```
from scipy.stats import ttest_1samp

scores = [48, 52, 55, 49, 51, 47, 53]
p = ttest_1samp(scores, popmean=50).pvalue
print(round(float(p), 4))
```

Kết quả khi chạy:

```
0.5347
```

Vì p lớn hơn 0,05 nên chưa đủ bằng chứng bác bỏ H_0; không khẳng định H_0 đúng.

Tình huống 2 — Tính khoảng tin cậy thời gian giao

Tình huống thực tế: Năm đơn giao mất `[2,3,3,4,2]` ngày, giả định độc lập và mô hình suy luận t phù hợp. Doanh nghiệp muốn khoảng tin cậy 95% cho **trung bình số ngày** của tổng thể.

Vì sao chọn cách giải này? Khi chưa biết phương sai tổng thể và mẫu nhỏ, khoảng t dùng độ lệch chuẩn mẫu và n−1 bậc tự do.

Quy trình giải:

1. Tạo mảng năm số ngày và tính mean mẫu.
2. Tính SE qua `sem` và khoảng t với df=4.
3. In hai đầu mút làm tròn ba chữ số.

```
import numpy as np
from scipy.stats import t, sem

days = np.array([2.0, 3.0, 3.0, 4.0, 2.0])
ci = t.interval(0.95, df=len(days) - 1, loc=days.mean(), scale=sem(days))
print(np.round(ci, 3).tolist())
```

Kết quả khi chạy:

```
[1.761, 3.839]
```

Khoảng t 95% ước lượng trung bình quần thể từ 1,761 đến 3,839 ngày.

Tình huống 3 — Tính SE của thời gian giao trung bình

Tình huống thực tế: Với đúng năm thời gian giao `[2,3,3,4,2]` ngày, quản lý muốn báo sai số chuẩn của **ước lượng mean**, không phải mức phân tán từng đơn.

Vì sao chọn cách giải này? `sem` ước lượng độ biến thiên mean bằng SD mẫu chia căn cỡ mẫu.

Quy trình giải:

1. Dùng mảng năm số ngày.
2. Gọi `scipy.stats.sem` với ddof mẫu mặc định.
3. Làm tròn ba chữ số, ghi đơn vị ngày.

```
from scipy.stats import sem

days = [2, 3, 3, 4, 2]
print(round(float(sem(days)), 3))
```

Kết quả khi chạy:

```
0.374
```

SE ước lượng khoảng 0,374 ngày của mean, không phải std của thời gian từng đơn.

Tình huống 4 — Ra quyết định với kiểm định hai phía

Tình huống thực tế: Dữ liệu năm số đo `[5,6,7,8,9]` được dùng kiểm định trung bình với mốc μ_0=6, α=0.05. Nhóm phân tích muốn chương trình đưa ra quyết định theo ngưỡng này.

Vì sao chọn cách giải này? So p-value với α là một quy tắc ra quyết định đã đặt trước, không coi p là xác suất H_0 đúng.

Quy trình giải:

1. Đặt H_0: μ=6 và α=0.05.
2. Tính p-value của t-test hai phía.
3. In quyết định bằng câu 'Reject H0' hoặc 'Fail to reject H0'.

```
from scipy.stats import ttest_1samp

values = [5, 6, 7, 8, 9]
p = ttest_1samp(values, popmean=6).pvalue
print('Reject H0' if p < 0.05 else 'Fail to reject H0')
```

Kết quả khi chạy:

```
Fail to reject H0
```

Không bác bỏ H_0 tại α=0.05, chứ không có nghĩa đã chứng minh μ=6.

Tình huống 5 — Thấy vai trò của mức ý nghĩa alpha

Tình huống thực tế: Một phép kiểm định đã tính được p-value=0.04. Đội kiểm thử muốn so hai mức ý nghĩa **0.01 và 0.05** trên cùng bằng chứng, không thay đổi dữ liệu.

Vì sao chọn cách giải này? α là ngưỡng lựa chọn để quyết định bác bỏ H_0; thay α có thể đổi quyết định dù p giữ nguyên.

Quy trình giải:

1. Đặt p=0.04 đã tính sẵn.
2. So p với hai ngưỡng 0.01 và 0.05.
3. In ngưỡng và Boolean cho việc bác bỏ H_0.

```
p = 0.04
for alpha in [0.01, 0.05]:
    print(alpha, p < alpha)
```

Kết quả khi chạy:

```
0.01 False
0.05 True
```

Với α=0.01 chưa bác bỏ, với α=0.05 thì bác bỏ; không suy ra xác suất H_0 đúng là 4%.

6. Sai lầm thường gặp

- p-value không phải xác suất H_0 đúng; nó là xác suất của kết quả cực đoan dưới giả định H_0.
- Không bác bỏ H_0 **không** đồng nghĩa chấp nhận hay chứng minh H_0 đúng.
- α=0.05 là ngưỡng kiểm định lựa chọn trước, không phải ngưỡng hiệu quả thực tế.
- CI 95% có **tỷ lệ bao phủ dài hạn 95%** khi lặp quy trình, không phải xác suất 95% tham số cố định trong một khoảng đã quan sát.
- SE khác SD: SE nói về bất định của mean, SD nói về số đo riêng lẻ.
- T-test trên mẫu nhỏ nhạy với ngoại lệ/phụ thuộc; cần xem giả định và thiết kế lấy mẫu.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is H0? — H_0 là gì?

A. A null hypothesis subjected to statistical testing — Giả thuyết không được đem kiểm định

B. Probability of equally or more extreme test results under H0 — Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

C. Test a population mean against a specified value — Kiểm định trung bình quần thể với một mốc

D. An interval from a procedure with 95% long-run coverage — Khoảng từ quy trình có mức bao phủ dài hạn 95%

Đáp án: A

Giải thích: Đó là giả định gốc của kiểm định. Ví dụ H_0: μ=50 nói về mean quần thể; không đòi mean mẫu quan sát phải đúng 50.

Q2. What does p-value mean? — p-value nghĩa là gì?

A. A null hypothesis subjected to statistical testing — Giả thuyết không được đem kiểm định

B. Probability of equally or more extreme test results under H0 — Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

C. Test a population mean against a specified value — Kiểm định trung bình quần thể với một mốc

D. An interval from a procedure with 95% long-run coverage — Khoảng từ quy trình có mức bao phủ dài hạn 95%

Đáp án: B

Giải thích: Nó được tính với điều kiện mô hình H_0 đúng. p-value được tính từ thống kê kiểm định và phân phối dưới H_0, chứ không trực tiếp gán xác suất cho H_0.

Q3. What is a one-sample t-test for? — T-test một mẫu dùng để làm gì?

A. A null hypothesis subjected to statistical testing — Giả thuyết không được đem kiểm định

B. Probability of equally or more extreme test results under H0 — Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

C. Test a population mean against a specified value — Kiểm định trung bình quần thể với một mốc

D. An interval from a procedure with 95% long-run coverage — Khoảng từ quy trình có mức bao phủ dài hạn 95%

Đáp án: C

Giải thích: Nó dùng trung bình và ước lượng phương sai mẫu. Khác kiểm định hai mẫu, t-test một mẫu so trung bình một mẫu với mốc cố định.

Q4. What is a 95% confidence interval? — Khoảng tin cậy 95% là gì?

A. A null hypothesis subjected to statistical testing — Giả thuyết không được đem kiểm định

B. Probability of equally or more extreme test results under H0 — Xác suất thống kê cực đoan bằng hoặc hơn khi H_0 đúng

C. Test a population mean against a specified value — Kiểm định trung bình quần thể với một mốc

D. An interval from a procedure with 95% long-run coverage — Khoảng từ quy trình có mức bao phủ dài hạn 95%

Đáp án: D

Giải thích: Bao phủ được hiểu qua những lần lấy mẫu lặp. Mức bao phủ 95% thuộc quy trình qua nhiều mẫu; một CI đã tính cần báo hai đầu mút và giả định đi kèm.

Q5. What does alpha represent? — α đại diện cho gì?

A. A preselected significance threshold — Ngưỡng mức ý nghĩa được chọn trước

B. Reject H0 at that significance level — Bác bỏ H_0 ở mức ý nghĩa đó

C. Fail to reject H0 — Chưa đủ bằng chứng bác bỏ H_0

D. Compare one sample mean with mu0 — So mean một mẫu với mốc mu0

Đáp án: A

Giải thích: α điều khiển quy tắc bác bỏ. Chọn alpha sau khi nhìn p-value có thể làm sai ý nghĩa kiểm soát báo động nhầm dự kiến.

Q6. What does p<alpha typically imply? — p<α thường dẫn tới gì?

A. A preselected significance threshold — Ngưỡng mức ý nghĩa được chọn trước

B. Reject H0 at that significance level — Bác bỏ H_0 ở mức ý nghĩa đó

C. Fail to reject H0 — Chưa đủ bằng chứng bác bỏ H_0

D. Compare one sample mean with mu0 — So mean một mẫu với mốc mu0

Đáp án: B

Giải thích: Kết quả bất thường dưới giả thuyết gốc. Bác bỏ cung cấp bằng chứng chống H_0 dưới mô hình, không đo xác suất H_0 sai.

Q7. What does p≥alpha imply? — p≥α thường kết luận gì?

A. A preselected significance threshold — Ngưỡng mức ý nghĩa được chọn trước

B. Reject H0 at that significance level — Bác bỏ H_0 ở mức ý nghĩa đó

C. Fail to reject H0 — Chưa đủ bằng chứng bác bỏ H_0

D. Compare one sample mean with mu0 — So mean một mẫu với mốc mu0

Đáp án: C

Giải thích: Không bác bỏ không phải chứng minh H_0. Mẫu nhỏ có thể bỏ sót tác động thật; 'không bác bỏ' khác với 'đã xác nhận bằng nhau'.

Q8. What is `ttest_1samp(x,popmean=mu0)` for? — `ttest_1samp(x,popmean=mu0)` dùng làm gì?

A. A preselected significance threshold — Ngưỡng mức ý nghĩa được chọn trước

B. Reject H0 at that significance level — Bác bỏ H_0 ở mức ý nghĩa đó

C. Fail to reject H0 — Chưa đủ bằng chứng bác bỏ H_0

D. Compare one sample mean with mu0 — So mean một mẫu với mốc mu0

Đáp án: D

Giải thích: Nó chạy kiểm định t một mẫu. Đối tượng trả về có thống kê kiểm định và p-value; lấy `.pvalue` khi cần quyết định.

Q9. How is a one-sample t statistic formed? — Thống kê t một mẫu có dạng gì?

A. (sample mean−mu0)/SE — (mean mẫu−mu0)/SE

B. Standard error of the mean — Sai số chuẩn của trung bình

C. Lower and upper endpoints of a t interval — Hai đầu dưới/trên của khoảng t

D. n−1 — n−1

Đáp án: A

Giải thích: Nó chuẩn hóa độ lệch của mean so với mốc. Chênh hai điểm chỉ có ý nghĩa kiểm định khi so với độ bất định lấy mẫu của mean.

Q10. What does `scipy.stats.sem` estimate? — `scipy.stats.sem` ước lượng gì?

A. (sample mean−mu0)/SE — (mean mẫu−mu0)/SE

B. Standard error of the mean — Sai số chuẩn của trung bình

C. Lower and upper endpoints of a t interval — Hai đầu dưới/trên của khoảng t

D. n−1 — n−1

Đáp án: B

Giải thích: Nó ước lượng độ dao động của mean mẫu. SE có xu hướng giảm khi n tăng dưới giả định phù hợp, còn SD của từng điểm không nhất thiết giảm.

Q11. What does t.interval return? — t.interval trả gì?

A. (sample mean−mu0)/SE — (mean mẫu−mu0)/SE

B. Standard error of the mean — Sai số chuẩn của trung bình

C. Lower and upper endpoints of a t interval — Hai đầu dưới/trên của khoảng t

D. n−1 — n−1

Đáp án: C

Giải thích: Khoảng phụ thuộc mức tin cậy, df, vị trí và độ rộng. Scale lớn làm khoảng rộng; loc là ước lượng trung tâm chứ không phải phần trăm tin cậy.

Q12. What degrees of freedom are usual for one-sample t inference? — Bậc tự do thông dụng của t một mẫu là gì?

A. (sample mean−mu0)/SE — (mean mẫu−mu0)/SE

B. Standard error of the mean — Sai số chuẩn của trung bình

C. Lower and upper endpoints of a t interval — Hai đầu dưới/trên của khoảng t

D. n−1 — n−1

Đáp án: D

Giải thích: Ước lượng phương sai mẫu làm mất một bậc tự do. Thống kê t ước lượng phương sai từ mẫu nên thường còn n−1 bậc tự do.

Q13. Is p-value equal to P(H0 is true)? — p-value có bằng P(H_0 đúng) không?

A. No; p is conditional on H0 — Không; p được tính khi giả định H_0 đúng

B. It does not reject at the matching alpha — Không bác bỏ với alpha tương ứng

C. Rejecting a true null hypothesis — Bác bỏ H_0 khi H_0 thực sự đúng

D. Failing to reject a false H0 — Không bác bỏ H_0 khi H_0 sai

Đáp án: A

Giải thích: p theo tần suất không phải posterior. Nhầm p với P(H_0|dữ liệu) là đảo chiều điều kiện; kiểm định tần suất xét mức cực đoan của dữ liệu khi H_0 đúng.

Q14. If a confidence interval covers a null value, what often happens in a matching two-sided test? — Nếu CI chứa mốc H_0 thì kiểm định hai phía tương ứng thường thế nào?

A. No; p is conditional on H0 — Không; p được tính khi giả định H_0 đúng

B. It does not reject at the matching alpha — Không bác bỏ với alpha tương ứng

C. Rejecting a true null hypothesis — Bác bỏ H_0 khi H_0 thực sự đúng

D. Failing to reject a false H0 — Không bác bỏ H_0 khi H_0 sai

Đáp án: B

Giải thích: Các quy trình tương ứng thường thống nhất dưới cùng giả định. Sự tương ứng cần kiểm định hai phía, mức tin cậy và giả định phù hợp; không đúng với mọi loại kiểm định và khoảng.

Q15. What is a type I error? — Sai lầm loại I là gì?

A. No; p is conditional on H0 — Không; p được tính khi giả định H_0 đúng

B. It does not reject at the matching alpha — Không bác bỏ với alpha tương ứng

C. Rejecting a true null hypothesis — Bác bỏ H_0 khi H_0 thực sự đúng

D. Failing to reject a false H0 — Không bác bỏ H_0 khi H_0 sai

Đáp án: C

Giải thích: Alpha kiểm soát lỗi loại I dưới giả định kiểm định. Sai lầm loại I là báo dương sai; alpha được chọn trước để kiểm soát xác suất lỗi này dưới giả định H_0.

Q16. What is a type II error? — Sai lầm loại II là gì?

A. No; p is conditional on H0 — Không; p được tính khi giả định H_0 đúng

B. It does not reject at the matching alpha — Không bác bỏ với alpha tương ứng

C. Rejecting a true null hypothesis — Bác bỏ H_0 khi H_0 thực sự đúng

D. Failing to reject a false H0 — Không bác bỏ H_0 khi H_0 sai

Đáp án: D

Giải thích: Nó khác lỗi bác bỏ H_0 đúng. Power bằng một trừ xác suất sai lầm loại II dưới giả thuyết đối cụ thể, liên quan đến thiết kế cỡ mẫu.

Q17. Does a 95% CI say the fixed parameter has 95% probability of lying in this realized interval? — CI 95% có nghĩa xác suất tham số cố định trong khoảng đã tính là 95%?

A. No; 95% describes repeated-interval coverage — Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. Strong dependence or problematic extreme data in small samples — Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

C. It becomes wider — Trở nên rộng hơn

D. No; effect size and context still matter — Không; còn kích thước tác động và bối cảnh

Đáp án: A

Giải thích: Trong diễn giải tần suất, tham số cố định và khoảng thay đổi theo mẫu. Phát biểu xác suất áp dụng cho quy trình tạo khoảng ngẫu nhiên trước lấy mẫu, không phải μ cố định sau khi quan sát một CI.

Q18. What can violate simple t-test assumptions? — Điều gì có thể vi phạm giả định t-test đơn giản?

A. No; 95% describes repeated-interval coverage — Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. Strong dependence or problematic extreme data in small samples — Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

C. It becomes wider — Trở nên rộng hơn

D. No; effect size and context still matter — Không; còn kích thước tác động và bối cảnh

Đáp án: B

Giải thích: Suy luận cần mô hình lấy mẫu phù hợp. Mẫu rất nhỏ, ngoại lệ nặng hoặc phụ thuộc có thể khiến p-value và CI báo ra thiếu tin cậy.

Q19. What happens to CI width if SE grows with other inputs fixed? — SE tăng khi yếu tố khác giữ nguyên thì CI rộng ra sao?

A. No; 95% describes repeated-interval coverage — Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. Strong dependence or problematic extreme data in small samples — Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

C. It becomes wider — Trở nên rộng hơn

D. No; effect size and context still matter — Không; còn kích thước tác động và bối cảnh

Đáp án: C

Giải thích: Bất định cao làm khoảng rộng hơn. Trong khoảng t, SE tăng làm biên `t_critical × SE` tăng, khiến hai đầu mút cách mean xa hơn.

Q20. Is statistical significance identical to practical importance? — Ý nghĩa thống kê có giống giá trị thực tiễn không?

A. No; 95% describes repeated-interval coverage — Không; 95% mô tả tỷ lệ bao phủ khi lặp

B. Strong dependence or problematic extreme data in small samples — Phụ thuộc mạnh hoặc dữ liệu cực đoan trong mẫu nhỏ

C. It becomes wider — Trở nên rộng hơn

D. No; effect size and context still matter — Không; còn kích thước tác động và bối cảnh

Đáp án: D

Giải thích: p nhỏ không chứng minh hiệu quả thực tế lớn. Ngoài p-value cần báo kích thước tác động và độ bất định; tác động rất nhỏ có thể có ý nghĩa thống kê nhưng không hữu ích thực tế.

8. Liên hệ bài kế tiếp

`CLT/SE → Hypothesis Testing → Confidence Interval → A/B Test.`

PY-41 — Phân hoạch dữ liệu, classification và clustering

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Train set — tập huấn luyện; **Test set** — tập kiểm tra.

`Set/NumPy → Train/Test → Preprocessing → Classification/Clustering`

Học xong dùng để làm gì? Tách tập dữ liệu đúng cách, huấn luyện mô hình và đánh giá kết quả trên dữ liệu độc lập.

Kết nối kiến thức tiếp theo: PY-42 chuẩn bị dữ liệu chữ thành token và đặc trưng phục vụ mô hình.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Train set	tập huấn luyện	Mẫu dùng để học tham số mô hình
Test set	tập kiểm tra	Mẫu giữ lại chỉ để đánh giá mô hình
Classification	phân loại	Dự đoán nhãn rời rạc từ feature
Clustering	phân cụm	Thử nhóm mẫu giống nhau mà không cần nhãn sẵn

3. Thư viện và hàm Python

Từ vựng trong mục: Function — hàm; **Argument** — đối số.

Hàm / cách viết	Khi nào dùng, ý nghĩa tham số
`train_test_split(...,test_size=.25,stratify=y)`	Chia train/test có xét tỷ lệ nhãn
`Pipeline([('scale',...),('model',...)])`	Fit chỉ trên train để giảm leakage
`LogisticRegression(max_iter=1000)`	Mô hình phân loại có giám sát
`KMeans(n_clusters=2,random_state=42,n_init=10)`	Phân nhóm không giám sát, số nhóm là giả định

4. Khái niệm chính và liên quan

Mạch kiến thức: Để đánh giá mô hình đáng tin, trước hết chia dữ liệu thành train và test. Classification học cách đoán nhãn từ ví dụ đã có nhãn; clustering nhóm các điểm tương tự khi chưa có nhãn. Một test độc lập giúp tránh tự đánh giá trên dữ liệu đã học.

4.1. Train set — Tập để học tham số

Từ vựng: Train set.

Dữ liệu train chứa các hàng quan sát dùng để mô hình **học** tham số. Trong bảng Iris, mỗi hàng là một hoa, bốn cột là kích thước tính bằng cm và nhãn là loài. Nếu lấy 112 hoa để train thì các đặc trưng của chúng được đưa vào `fit`; không đưa test vào học scaler hoặc chỉnh siêu tham số.

```
from sklearn.datasets import load_iris

data = load_iris()
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

150 hàng hoa, bốn cột đặc trưng và một nhãn cho mỗi hàng.

4.2. Test set — Tập chỉ dùng để đánh giá cuối

Từ vựng: Test set.

Test set là các mẫu giữ lại, không dùng để fit hay chọn cách xử lý. Một cách kiểm tra tối thiểu là **ID train không trùng ID test**. Điều đó chưa đủ loại rò rỉ qua các mẫu liên quan, đặc trưng sinh từ tương lai, hoặc bản ghi trùng nội dung nhưng khác ID.

```
train_ids = {1, 2, 3}
test_ids = {4, 5}
print(train_ids.isdisjoint(test_ids))
```

Kết quả khi chạy:

```
True
```

Không có ID xuất hiện ở cả train và test; vẫn cần kiểm tra rò rỉ gián tiếp.

4.3. Classification — Dự đoán nhãn đã biết

Từ vựng: Classification.

Classification là học có giám sát: từ **X là các đặc trưng**, mô hình dự đoán **y là nhãn**. Iris có ba nhãn loài là 0,1,2. Một classifier học từ những hàng đã có nhãn rồi dự đoán loài của hoa mới; khác regression dự đoán giá trị liên tục và không thể thay độ chính xác bằng khoảng cách giữa nhãn số.

```
from sklearn.datasets import load_iris

data = load_iris()
print(sorted(set(data.target.tolist())))
```

Kết quả khi chạy:

```
[0, 1, 2]
```

Ba số là mã phân loại ba loài, không phải các đại lượng đo theo cm.

4.4. Clustering — Nhóm mẫu khi chưa có nhãn

Từ vựng: Clustering.

Clustering là học không giám sát: tìm các nhóm điểm gần nhau trong không gian đặc trưng, **không dùng nhãn thật để huấn luyện**. Ví dụ hai cụm điểm quanh `[0,0]` và `[10,10]`; KMeans tìm hai tâm nhóm. Mã cụm `0`/`1` chỉ là định danh tùy cách khởi tạo, không đồng nghĩa nhãn lớp 0/1 của bài phân loại.

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[0, 0], [0, 1], [10, 10], [10, 11]])
labels = KMeans(n_clusters=2, random_state=42, n_init=10).fit_predict(X)
print(len(set(labels.tolist())))
```

Kết quả khi chạy:

```
2
```

KMeans tìm được hai cụm; mã số gắn cho từng cụm có thể đổi.

5. Năm tình huống ứng dụng

Tình huống 1 — Kiểm tra ID train/test

Tình huống thực tế: Một nhóm thử mô hình với train `{1,2,3}` và test `{4,5}` (mỗi số là ID hồ sơ). Cần xác nhận không có ID nào xuất hiện trong cả hai tập.

Vì sao chọn cách giải này? Dùng `isdisjoint` cho đúng câu hỏi về phần tử trùng nhau; không phải đo độ chính xác.

Quy trình giải:

1. Tạo hai Set ID train và test.
2. Kiểm tra phần giao có rỗng.
3. In Boolean và nêu rằng đây chỉ là một bước chống rò rỉ.

```
train = {1, 2, 3}
test = {4, 5}
print(train.isdisjoint(test))
```

Kết quả khi chạy:

```
True
```

Hai tập không trùng ID, chưa chứng minh mọi dữ liệu đều độc lập.

Tình huống 2 — Phân chia Iris có phân tầng

Tình huống thực tế: Tập Iris có 150 hoa, bốn số đo cm/hoa và ba loài với 50 hoa mỗi loài. Cần dành 25% làm test nhưng duy trì tỷ lệ loài gần tương đương.

Vì sao chọn cách giải này? `stratify=y` chia theo nhãn, giúp test không tình cờ thiếu lớp trong bộ mẫu này.

Quy trình giải:

1. Nạp bảng 150×4 và nhãn 150 giá trị.
2. Dùng `train_test_split` với test_size=0.25, stratify và seed 42.
3. In shape train/test.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42,
)
print(X_train.shape, X_test.shape)
```

Kết quả khi chạy:

```
(112, 4) (38, 4)
```

112 hoa dùng train, 38 hoa giữ test; mỗi hàng vẫn có bốn đặc trưng.

Tình huống 3 — Fit scaler và mô hình trên train

Tình huống thực tế: Nhóm học máy dùng bốn kích thước của Iris để đoán một trong ba loài. Cần chuẩn hóa số đo mà không dùng trung bình/độ lệch chuẩn của test trong lúc fit.

Vì sao chọn cách giải này? Pipeline tự fit scaler và Logistic Regression trên train, rồi áp cùng biến đổi lên test.

Quy trình giải:

1. Chia Iris trước khi fit bất kỳ phép chuẩn hóa nào.
2. Tạo pipeline StandardScaler → LogisticRegression và fit train.
3. In số mẫu dự đoán trên test.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42,
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
```

```
model.fit(X_train, y_train)
print(len(model.predict(X_test)))
```

Kết quả khi chạy:

```
38
```

Model trả một dự đoán nhãn cho mỗi hoa trong 38 hoa test.

Tình huống 4 — Nhóm bốn tọa độ 2D

Tình huống thực tế: Bốn điểm trong mặt phẳng, hai điểm quanh `[0,0]`, hai điểm quanh `[10,10]`, chưa có nhãn. Cần phát hiện số cụm khi yêu cầu trước `k=2`.

Vì sao chọn cách giải này? KMeans phân nhóm từ khoảng cách giữa các đặc trưng, không dùng y thật.

Quy trình giải:

1. Tạo ma trận X shape (4,2), mỗi hàng là một điểm (x,y) cùng đơn vị.
2. Fit KMeans với k=2, seed cố định.
3. Đếm số mã cụm khác nhau, tránh phụ thuộc hoán đổi mã cụm.

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[0, 0], [0, 1], [10, 10], [10, 11]])
cluster_ids = KMeans(n_clusters=2, n_init=10, random_state=42).fit_predict(X)
print(len(set(cluster_ids.tolist())))
```

Kết quả khi chạy:

```
2
```

Bốn điểm được chia thành hai nhóm, không tự biết nhóm nào là nhãn thật.

Tình huống 5 — So classification và clustering

Tình huống thực tế: Dữ liệu Iris có bốn đặc trưng số (cm) và nhãn loài. Nhóm thực hành cần xác định thông tin đầu vào nào được phép dùng ở mô hình có giám sát và không giám sát.

Vì sao chọn cách giải này? Kiểm tra shape của X và y làm rõ classifier dùng cả nhãn lúc fit, trong khi clustering fit từ X.

Quy trình giải:

1. Nạp Iris, lấy X chứa kích thước hoa và y chứa nhãn.
2. In shape X và y.
3. Phân biệt dùng `(X,y)` để train classification với dùng `X` cho clustering.

```
from sklearn.datasets import load_iris

data = load_iris()
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

Mỗi hoa có bốn đặc trưng và một nhãn; KMeans không sử dụng nhãn để fit.

6. Sai lầm thường gặp

- Dữ liệu test không được dùng để fit scaler hoặc chọn siêu tham số; cần tách trước khi tiền xử lý học tham số.
- ID không trùng vẫn có thể rò rỉ qua bản ghi liên quan hoặc giá trị tương lai.
- Classification học từ nhãn y; clustering nhóm X không có nhãn. Không gọi mã cụm là nhãn lớp thật.
- `stratify=y` giúp giữ tỷ lệ lớp nhưng không chứng minh tập test đại diện toàn bộ dân số.
- Accuracy test khác loss train và không phản ánh đầy đủ hiệu quả trên dữ liệu mất cân bằng.
- `scikit-learn` là thư viện ngoài; phiên bản khác nhau có thể thay đổi kết quả học tối ưu.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a train set? — Train set là gì?

A. Data used for fitting model parameters — Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation — Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels — Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity — Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: A

Giải thích: Mô hình học tham số từ mẫu train, không cập nhật bằng bản ghi test. Đơn vị học là hàng train với đặc trưng và nhãn, không phải hàng test để chấm điểm.

Q2. What is a test set? — Test set là gì?

A. Data used for fitting model parameters — Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation — Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels — Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity — Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: B

Giải thích: Test đo hiệu quả trên dữ liệu chưa học; chỉnh đi chỉnh lại theo test làm đánh giá thiên lệch. Điểm cốt lõi là test độc lập với quá trình học; điều chỉnh lặp dựa trên test sẽ làm mất tính độc lập ấy.

Q3. What is classification? — Classification là gì?

A. Data used for fitting model parameters — Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation — Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels — Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity — Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: C

Giải thích: Lúc train phải có nhãn; đầu ra dự đoán là loại/phân lớp. Khác phân cụm, bài phân loại có nhãn mục tiêu rõ ràng khi học.

Q4. What is clustering? — Clustering là gì?

A. Data used for fitting model parameters — Dữ liệu để fit tham số mô hình

B. Held-out data for final evaluation — Dữ liệu giữ lại để đánh giá cuối

C. Supervised prediction of discrete labels — Dự đoán nhãn rời rạc có giám sát

D. Grouping unlabeled observations by similarity — Nhóm quan sát chưa có nhãn theo độ tương đồng

Đáp án: D

Giải thích: Phân cụm dựa vào X mà không fit nhãn lớp có sẵn. Khác classifier, KMeans không tự hiểu ý nghĩa đời thực của số hiệu cụm.

Q5. What does stratify=y help preserve? — stratify=y giúp giữ gì?

A. Approximate label proportions in train and test — Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only — Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction — Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers — Định danh cụm tùy ý

Đáp án: A

Giải thích: Nó chia các lớp theo tỷ lệ gần tương ứng khi số mẫu cho phép. Nó chỉ giữ tỷ lệ lớp xấp xỉ, không bảo đảm mọi nhóm nhỏ đều đại diện.

Q6. What does StandardScaler fit on? — StandardScaler phải fit trên dữ liệu nào?

A. Approximate label proportions in train and test — Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only — Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction — Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers — Định danh cụm tùy ý

Đáp án: B

Giải thích: Fit từ test làm rò rỉ phân phối đánh giá vào tiền xử lý. Được áp thống kê đã học từ train lên test, nhưng không được học lại thống kê từ test.

Q7. What does Pipeline combine? — Pipeline gộp gì?

A. Approximate label proportions in train and test — Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only — Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction — Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers — Định danh cụm tùy ý

Đáp án: C

Giải thích: Các phép đổi đã học được áp nhất quán khi dự đoán. Vì thế tiền xử lý phải nằm trong quy trình train, không fit trước trên toàn bộ dữ liệu.

Q8. What do KMeans cluster IDs mean? — Mã cụm KMeans có nghĩa gì?

A. Approximate label proportions in train and test — Tỷ lệ nhãn xấp xỉ giữa train và test

B. Training features only — Chỉ các đặc trưng train

C. Preprocessing and estimator fitting/prediction — Tiền xử lý và các bước fit/dự đoán

D. Arbitrary cluster identifiers — Định danh cụm tùy ý

Đáp án: D

Giải thích: Mã có thể đổi 0↔1 dù cách chia hình học không đổi. Hãy đánh giá theo thành viên nhóm hoặc ánh xạ có căn cứ, không mặc định mã 0 là một loài cụ thể.

Q9. What is a feature? — Feature là gì?

A. Input measurement used to predict — Số đo đầu vào dùng để dự đoán

B. The desired discrete output class — Lớp đầu ra muốn dự đoán

C. One predicted label per input row — Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels — Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: A

Giải thích: Đặc trưng nằm ở cột X, không phải nhãn đích. Mỗi cột cần có ý nghĩa và đơn vị đo rõ ràng trước khi đưa vào classifier.

Q10. What is a target label? — Target label là gì?

A. Input measurement used to predict — Số đo đầu vào dùng để dự đoán

B. The desired discrete output class — Lớp đầu ra muốn dự đoán

C. One predicted label per input row — Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels — Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: B

Giải thích: Classifier học ánh xạ từ X sang y. Nhãn là mục tiêu dự đoán, không phải đặc trưng được đưa sẵn khi dự đoán.

Q11. What does `model.predict(X_test)` return? — `model.predict(X_test)` trả gì?

A. Input measurement used to predict — Số đo đầu vào dùng để dự đoán

B. The desired discrete output class — Lớp đầu ra muốn dự đoán

C. One predicted label per input row — Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels — Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: C

Giải thích: Mỗi hàng test có một dự đoán lớp. Số nhãn dự đoán phải bằng số hàng đầu vào, nếu không sẽ thiếu kết quả cho một số mẫu.

Q12. What does `model.score(X_test,y_test)` mean for a standard classifier? — `model.score` cho classifier thông dụng đo gì?

A. Input measurement used to predict — Số đo đầu vào dùng để dự đoán

B. The desired discrete output class — Lớp đầu ra muốn dự đoán

C. One predicted label per input row — Một nhãn dự đoán cho mỗi hàng đầu vào

D. Prediction accuracy on test labels — Tỷ lệ dự đoán đúng trên nhãn test

Đáp án: D

Giải thích: Score so dự đoán với nhãn thật; không phải loss train. Nếu nhãn mất cân bằng, cần đọc thêm precision/recall theo lớp hoặc confusion matrix.

Q13. Which check detects overlapping IDs across train and test? — Phép kiểm tra nào phát hiện ID xuất hiện ở cả train và test?

A. `train_ids.isdisjoint(test_ids)` — `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak — Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X — Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations — Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: A

Giải thích: isdisjoint xác nhận không có cùng mã ở cả hai tập. Kiểm tra này bắt trùng mã đơn giản, không phát hiện phép đo liên quan mang mã khác nhau.

Q14. Does disjoint ID imply zero leakage? — Không trùng ID có chắc không rò rỉ?

A. `train_ids.isdisjoint(test_ids)` — `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak — Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X — Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations — Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: B

Giải thích: Rò rỉ có thể qua dữ liệu liên quan, phép đổi hoặc thông tin thời gian. Rò rỉ là thông tin đáng lẽ chưa biết lọt vào train, không chỉ là vấn đề trùng mã.

Q15. Does KMeans need y labels to fit? — KMeans cần nhãn y khi fit không?

A. `train_ids.isdisjoint(test_ids)` — `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak — Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X — Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations — Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: C

Giải thích: Phân cụm không giám sát và không tối ưu accuracy theo nhãn thật. KMeans chỉ cần tọa độ đặc trưng và số cụm muốn tìm.

Q16. Why not directly compare KMeans IDs to true classes for accuracy? — Vì sao không so trực tiếp mã KMeans với lớp thật để tính accuracy?

A. `train_ids.isdisjoint(test_ids)` — `train_ids.isdisjoint(test_ids)`

B. No, correlated records or future information may leak — Không, bản ghi liên quan hoặc thông tin tương lai vẫn rò rỉ

C. No, it fits features X — Không, nó fit đặc trưng X

D. Cluster numbers have arbitrary permutations — Số hiệu cụm có thể hoán đổi tùy ý

Đáp án: D

Giải thích: Cùng nhóm có thể gắn mã đảo ngược mà chất lượng không đổi. Nếu đổi nhãn số của hai cụm, accuracy kiểu phân loại có thể đổi dù cách chia nhóm vẫn y hệt.

Q17. What shape does a 150-row four-feature table have? — Bảng 150 hàng, bốn đặc trưng có shape nào?

A. (150,4) — (150,4)

B. Allocate about a quarter to test — Dành khoảng một phần tư cho test

C. Held-out final test set — Tập test cuối cùng đã giữ lại

D. Predicted known class names — Tên lớp đã biết từ dữ liệu nhãn

Đáp án: A

Giải thích: Hàng là hoa, cột là bốn đặc trưng. Chiều thứ nhất đếm số hoa, chiều thứ hai đếm số đo trên mỗi hoa.

Q18. What does test_size=0.25 mean? — test_size=0.25 nghĩa là gì?

A. (150,4) — (150,4)

B. Allocate about a quarter to test — Dành khoảng một phần tư cho test

C. Held-out final test set — Tập test cuối cùng đã giữ lại

D. Predicted known class names — Tên lớp đã biết từ dữ liệu nhãn

Đáp án: B

Giải thích: Làm tròn quyết định số nguyên mẫu chính xác. Với số mẫu hữu hạn, thư viện phải làm tròn tỷ lệ yêu cầu thành số lượng nguyên.

Q19. Which dataset must be excluded from hyperparameter tuning? — Không được dùng tập nào để điều chỉnh siêu tham số?

A. (150,4) — (150,4)

B. Allocate about a quarter to test — Dành khoảng một phần tư cho test

C. Held-out final test set — Tập test cuối cùng đã giữ lại

D. Predicted known class names — Tên lớp đã biết từ dữ liệu nhãn

Đáp án: C

Giải thích: Chọn siêu tham số bằng validation/CV thuộc phần train. Hãy chọn siêu tham số bằng validation hoặc cross-validation trên phần train, sau đó đánh giá test cuối.

Q20. What can a classification model output that KMeans does not inherently provide? — Classifier có đầu ra gì mà KMeans không tự bảo đảm?

A. (150,4) — (150,4)

B. Allocate about a quarter to test — Dành khoảng một phần tư cho test

C. Held-out final test set — Tập test cuối cùng đã giữ lại

D. Predicted known class names — Tên lớp đã biết từ dữ liệu nhãn

Đáp án: D

Giải thích: KMeans tạo cụm chứ không tự gán ý nghĩa lớp thật. Mã lớp là nhãn đã xác định; mã cụm chỉ đánh dấu một nhóm vừa phát hiện.

8. Liên hệ bài kế tiếp

`Set/NumPy → Train/Test → Preprocessing → Classification/Clustering`.

PY-42 — Tiền xử lý văn bản, token và Jaccard trong NLP

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Token — đơn vị từ; **Vocabulary** — từ vựng.

`String → Set → Tokenization → Vector hóa văn bản`

Học xong dùng để làm gì? Làm sạch văn bản, tách token và tạo tập từ để so sánh hoặc phân loại nội dung.

Kết nối kiến thức tiếp theo: PY-43 quản lý các phiên bản mã/tiền xử lý bằng Git để thí nghiệm tái tạo được.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Khái niệm
Token	đơn vị từ	Một phần văn bản sau khi tách theo quy tắc
Vocabulary	từ vựng	Tập token khác nhau trong dữ liệu
Normalization	chuẩn hóa văn bản	Đưa chữ về dạng phù hợp để so khớp
Jaccard	tương đồng Jaccard	Tỷ lệ giao trên hợp của hai tập token

3. Hàm và công thức quan trọng

| Cú pháp | Cách dùng | |---|---| | `text.strip().lower()` | Bỏ khoảng trắng hai đầu rồi chuyển chữ thường | | `text.split()` | Tách theo khoảng trắng; không đủ cho mọi ngôn ngữ | | `set(tokens)` | Lấy token khác nhau, bỏ lượt lặp | | `len(a & b) / len(a \| b)` | Jaccard khi hợp khác rỗng; kiểm tra mẫu số trước khi chia |

4. Khái niệm chính và liên quan

Mạch kiến thức: Một câu văn trước tiên được chuẩn hóa rồi tách thành token. Các token khác nhau tạo vocabulary; từ hai vocabulary, Jaccard đo mức trùng nhau theo mặt chữ. Cần hiểu sự khác nhau giữa tách từ đơn giản và hiểu ngữ nghĩa.

4.1. Token — Đơn vị nhỏ sau quy tắc tách

Từ vựng: Token.

Trong xử lý ngôn ngữ tự nhiên (NLP), **token** là đơn vị văn bản sau một bước tách: có thể là từ, dấu câu, hoặc phần nhỏ của từ. Với câu tiếng Anh đơn giản `AI helps Python`, `split()` theo khoảng trắng tạo ba

token. Đây **không phải** bộ tách từ tiếng Việt chuẩn: một từ ghép tiếng Việt có thể gồm nhiều tiếng và cần tokenizer chuyên biệt.

```
sentence = 'AI helps Python'
print(sentence.lower().split())
```

Kết quả khi chạy:

```
['ai', 'helps', 'python']
```

Ba token chữ thường sau tách theo khoảng trắng; dấu câu chưa được xử lý.

4.2. Vocabulary — Tập token duy nhất

Từ vựng: Vocabulary.

Vocabulary là tập các token **khác nhau** quan sát được trong kho văn bản. Hai câu `AI uses data` và `AI uses Python` có token chung `AI`, `uses`; dùng Set chỉ giữ mỗi từ một lần. Set giúp đếm từ duy nhất nhưng không giữ số lần xuất hiện, khác Bag-of-Words. In qua `sorted()` để không phụ thuộc thứ tự Set.

```
sentences = ['AI uses data', 'AI uses Python']
tokens = ' '.join(sentences).lower().split()
print(sorted(set(tokens)))
```

Kết quả khi chạy:

```
['ai', 'data', 'python', 'uses']
```

Kho từ vựng có bốn token khác nhau; `ai` và `uses` không bị đếm hai lần.

4.3. Normalization — Chuẩn hóa trước khi so từ

Từ vựng: Normalization.

Normalization đưa văn bản về một quy ước nhất quán: bỏ khoảng trắng hai đầu, chuyển chữ về thường, đôi khi xử lý Unicode và dấu câu. Ví dụ `Machine LEARNING` chuẩn hóa thành `machine learning`. Nếu không chuẩn hóa, `AI` và `ai` có thể bị xem là hai token; tuy nhiên xóa dấu tiếng Việt tùy tiện có thể làm mất nghĩa.

```
text = '  Machine LEARNING '
print(text.strip().lower())
```

Kết quả khi chạy:

```
machine learning
```

Khoảng trắng ở đầu/cuối bị bỏ, chữ hoa chuyển thành chữ thường.

4.4. Jaccard — Tỷ lệ token chung so với tất cả token

Từ vựng: Jaccard similarity.

Jaccard đo độ giống của hai **tập token**: `J(A,B)=|A∩B|/|A∪B|` nếu hợp khác rỗng. Câu A `python for ai` có ba token; câu B `python and sql` cũng ba token. Giao chỉ có `python` (1 phần tử), hợp có năm từ, nên J=0.2. Jaccard Set không tính tần suất và không hiểu nghĩa tương đương của từ; với hai tập rỗng cần chọn quy ước riêng để tránh chia 0.

```
a = set('python for ai'.split())
b = set('python and sql'.split())
print(round(len(a & b) / len(a | b), 2))
```

Kết quả khi chạy:

```
0.2
```

Hai tài liệu chung một trên năm token khác nhau; độ giống tập hợp 20%.

5. Năm tình huống ứng dụng

Tình huống 1 — Chuẩn hóa từ khóa tìm kiếm

Tình huống thực tế: Thanh tìm kiếm nhận chuỗi `Machine LEARNING` từ người dùng. Cần bỏ khoảng trắng hai đầu và đồng nhất chữ thường trước khi so khớp đơn giản.

Vì sao chọn cách giải này? `strip().lower()` xử lý đúng hai thay đổi đang cần, không tự xóa dấu hoặc ký tự khác.

Quy trình giải:

1. Lưu chuỗi gốc.
2. Gọi `strip()` rồi `lower()` theo thứ tự.
3. In chuỗi chuẩn hóa.

```
text = '  Machine LEARNING '
print(text.strip().lower())
```

Kết quả khi chạy:

```
machine learning
```

Chuỗi được chuẩn hóa để so với từ khóa cùng quy ước.

Tình huống 2 — Tách token của câu tiếng Anh

Tình huống thực tế: Một ứng dụng tìm kiếm thử nghiệm nhận câu `AI helps Python`, chưa có dấu câu. Cần tách thành các token theo khoảng trắng và đưa về chữ thường.

Vì sao chọn cách giải này? `lower().split()` đủ cho câu thử đơn giản; công việc NLP thực cần tokenizer theo ngôn ngữ.

Quy trình giải:

1. Lưu câu đầu vào.

2. Chuyển chữ thường rồi tách theo khoảng trắng.
3. In danh sách token theo thứ tự xuất hiện.

```
sentence = 'AI helps Python'
print(sentence.lower().split())
```

Kết quả khi chạy:

```
['ai', 'helps', 'python']
```

Ba token được tạo; cách làm chưa giải quyết từ ghép tiếng Việt hay dấu câu.

Tình huống 3 — Tạo kho từ vựng từ hai tài liệu

Tình huống thực tế: Hai đoạn `AI uses data`, `AI uses Python` có token lặp. Bộ phận tìm kiếm cần tập các từ khác nhau để xây dựng danh sách tra cứu.

Vì sao chọn cách giải này? Set bỏ trùng; sắp xếp khi in giúp học viên kiểm tra kết quả dễ.

Quy trình giải:

1. Ghép hai câu bằng khoảng trắng.
2. Chuyển chữ thường, tách và đưa vào Set.
3. Sắp thứ tự token rồi in.

```
sentences = ['AI uses data', 'AI uses Python']
words = set(' '.join(sentences).lower().split())
print(sorted(words))
```

Kết quả khi chạy:

```
['ai', 'data', 'python', 'uses']
```

Vocabulary có bốn từ khác nhau, không đếm số lần xuất hiện.

Tình huống 4 — So hai tài liệu bằng Jaccard

Tình huống thực tế: Hai câu có từ chung `python`: A=`python for ai`, B=`python and sql`. Đội tìm kiếm muốn tỷ lệ trùng token khác nhau, chưa cần so ngữ nghĩa.

Vì sao chọn cách giải này? Jaccard của hai Set cho tỷ lệ giao/hợp; phù hợp so trùng từ không xét tần suất.

Quy trình giải:

1. Biến mỗi câu thành Set token.
2. Tính kích thước giao và hợp; kiểm tra hợp không rỗng.
3. Chia, làm tròn hai chữ số và in.

```
a = set('python for ai'.split())
b = set('python and sql'.split())
print(round(len(a & b) / len(a | b), 2))
```

Kết quả khi chạy:

```
0.2
```

Tập từ trùng chiếm một phần năm tất cả từ, Jaccard bằng 0,2.

Tình huống 5 — Tránh chia cho 0 ở hai văn bản rỗng

Tình huống thực tế: Hệ thống có thể nhận hai chuỗi chỉ gồm khoảng trắng. Sau `split()`, cả hai tập token rỗng. Cần quy ước đánh giá giống nhau 100% trong **ứng dụng này** và tránh chia cho 0.

Vì sao chọn cách giải này? Kiểm tra hợp rỗng trước khi chia; kết quả 1 cho hai tập rỗng là **quy ước**, không phải công thức Jaccard thông thường.

Quy trình giải:

1. Tách hai chuỗi rỗng thành Set.
2. Nếu hợp rỗng, trả 1.0 theo chính sách này.
3. Ngược lại chia giao cho hợp và in.

```
a = set(' '.split())
b = set(''.split())
union = a | b
score = 1.0 if not union else len(a & b) / len(union)
print(score)
```

Kết quả khi chạy:

```
1.0
```

Hai tập rỗng được gán độ giống 1.0 theo quy ước đã công bố.

6. Sai lầm thường gặp

- Tách theo khoảng trắng không luôn phân đoạn từ tiếng Việt chính xác; tokenizer phụ thuộc ngôn ngữ và mục tiêu.
- `lower()` không xóa dấu câu, `strip()` không xóa mọi khoảng trắng giữa câu.
- Set bỏ trùng nhưng đánh mất thứ tự và tần suất; Jaccard trên Set không hiểu từ đồng nghĩa.
- Mẫu số của Jaccard là kích thước **hợp**, không phải độ dài một tài liệu.
- Hai tập rỗng gây phép chia 0/0, cần nêu rõ quy ước ứng dụng.
- Không so sánh tài liệu khác cách chuẩn hóa, vì khác tiền xử lý sẽ tạo token khác nhau.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a token? — Token là gì?

A. A unit produced by a text-splitting rule — Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens — Tập các token khác nhau

C. Applying consistent text representations — Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

Đáp án: A

Giải thích: Token có thể là từ, mảnh từ hoặc dấu câu tùy bộ tách. Đơn vị cụ thể phụ thuộc quy tắc tokenizer; token không nhất thiết là một từ hoàn chỉnh.

Q2. What is a vocabulary? — Vocabulary là gì?

A. A unit produced by a text-splitting rule — Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens — Tập các token khác nhau

C. Applying consistent text representations — Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

Đáp án: B

Giải thích: Tập từ vựng bỏ trùng danh tính token, khác bảng tần suất. Nó trả lời có những token nào, chứ không trả lời mỗi token xuất hiện bao nhiêu lần.

Q3. What is text normalization? — Chuẩn hóa văn bản là gì?

A. A unit produced by a text-splitting rule — Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens — Tập các token khác nhau

C. Applying consistent text representations — Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

Đáp án: C

Giải thích: Chữ thường và cắt khoảng trắng giúp khớp dạng văn bản. Chuẩn hóa đổi cách biểu diễn; quy tắc phải nhất quán giữa các văn bản đem so.

Q4. What is set-based Jaccard similarity? — Jaccard trên Set là gì?

A. A unit produced by a text-splitting rule — Đơn vị do quy tắc tách văn bản tạo

B. A collection of distinct tokens — Tập các token khác nhau

C. Applying consistent text representations — Đưa văn bản về dạng biểu diễn nhất quán

D. Intersection size divided by union size — Số phần tử giao chia số phần tử hợp

Đáp án: D

Giải thích: Nó đo từ khác nhau chung so với tổng số từ khác nhau. Một từ chung trong năm từ khác nhau cho 1/5, không phải lấy độ dài một câu làm mẫu số.

Q5. What does `str.split()` do without arguments? — `str.split()` không đối số làm gì?

A. Splits on runs of whitespace — Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase — Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace — Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently — Để bỏ token trùng và hiển thị ổn định

Đáp án: A

Giải thích: Tách bằng khoảng trắng không hiểu mọi ranh giới từ. Đây là quy tắc tách chuỗi, không phải thuật toán phân đoạn từ tiếng Việt về mặt ngôn ngữ.

Q6. What does `lower()` do? — `lower()` làm gì?

A. Splits on runs of whitespace — Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase — Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace — Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently — Để bỏ token trùng và hiển thị ổn định

Đáp án: B

Giải thích: Giúp so biến thể chữ hoa/thường nhưng không bỏ dấu câu. `lower` chỉ đổi chữ hoa/thường, không tự xóa khoảng trắng hoặc tách token.

Q7. What does `strip()` do? — `strip()` làm gì?

A. Splits on runs of whitespace — Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase — Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace — Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently — Để bỏ token trùng và hiển thị ổn định

Đáp án: C

Giải thích: strip không xóa khoảng trắng giữa câu. Muốn xử lý khoảng trắng giữa câu cần `split` hoặc bước khác, không thể chỉ dùng `strip`.

Q8. Why use `sorted(set(tokens))` for examples? — Vì sao dùng `sorted(set(tokens))`?

A. Splits on runs of whitespace — Tách theo chuỗi khoảng trắng liên tiếp

B. Converts letters to lowercase — Chuyển chữ thành chữ thường

C. Removes leading and trailing whitespace — Bỏ khoảng trắng hai đầu

D. To deduplicate and display consistently — Để bỏ token trùng và hiển thị ổn định

Đáp án: D

Giải thích: Set không bảo đảm thứ tự; sorted tạo List có thứ tự. Set loại phần tử trùng, còn `sorted` tạo List có thứ tự để in ổn định.

Q9. What does A & B produce for sets? — A & B trên Set tạo gì?

A. Common tokens — Các token chung

B. All distinct tokens from either set — Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences — Bỏ qua số lần lặp

D. 1.0 — 1.0

Đáp án: A

Giải thích: Giao chỉ giữ giá trị có ở cả hai tập. Với A={'ai','python'}, B={'ai'}, giao chỉ có 'ai', không chứa toàn bộ A.

Q10. What does A | B produce for sets? — A | B trên Set tạo gì?

A. Common tokens — Các token chung

B. All distinct tokens from either set — Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences — Bỏ qua số lần lặp

D. 1.0 — 1.0

Đáp án: B

Giải thích: Hợp loại token trùng ở cả hai bên. Hợp gồm từ riêng ở mỗi bên lẫn các từ chung giữa hai bên.

Q11. How does Set Jaccard treat token frequency? — Jaccard Set xử lý tần suất token ra sao?

A. Common tokens — Các token chung

B. All distinct tokens from either set — Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences — Bỏ qua số lần lặp

D. 1.0 — 1.0

Đáp án: C

Giải thích: Chuyển sang Set mất số lần và thứ tự từ. Vì vậy Jaccard trên Set không phân biệt từ xuất hiện một lần với mười lần.

Q12. What happens for identical nonempty token sets? — Hai tập token giống hệt và không rỗng cho Jaccard gì?

A. Common tokens — Các token chung

B. All distinct tokens from either set — Mọi token khác nhau thuộc ít nhất một tập

C. It ignores repeated occurrences — Bỏ qua số lần lặp

D. 1.0 — 1.0

Đáp án: D

Giải thích: Giao bằng hợp nên tỷ số bằng một. Kết quả này áp dụng khi hợp không rỗng, đúng điều kiện đã nêu trong câu hỏi.

Q13. What happens for disjoint nonempty token sets? — Hai tập token rời không rỗng cho Jaccard gì?

A. 0.0 — 0.0

B. The formula has zero denominator; choose a convention — Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care — Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains — Không, dấu câu vẫn còn

Đáp án: A

Giải thích: Không có token chung nên tử số bằng không. Tử số bằng không nhưng mẫu số vẫn dương.

Q14. What happens when both sets are empty? — Nếu hai Set đều rỗng thì Jaccard sao?

A. 0.0 — 0.0

B. The formula has zero denominator; choose a convention — Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care — Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains — Không, dấu câu vẫn còn

Đáp án: B

Giải thích: Phép chia 0/0 không xác định; ứng dụng phải tự quy định. Ứng dụng có thể quy ước hai văn bản rỗng giống nhau 1, nhưng hệ khác có thể chọn 0 hoặc không xác định.

Q15. Does whitespace splitting correctly segment all Vietnamese words? — split theo khoảng trắng có tách đúng mọi từ tiếng Việt không?

A. 0.0 — 0.0

B. The formula has zero denominator; choose a convention — Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care — Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains — Không, dấu câu vẫn còn

Đáp án: C

Giải thích: Các tiếng ngăn bởi khoảng trắng chưa chắc là từ độc lập. Một từ tiếng Việt có thể gồm nhiều tiếng được ngăn bằng khoảng trắng.

Q16. Can lowercasing alone remove punctuation? — Chỉ lower() có xóa dấu câu không?

A. 0.0 — 0.0

B. The formula has zero denominator; choose a convention — Công thức có mẫu số 0; cần quy ước riêng

C. No, multi-syllable words need more care — Không, từ nhiều tiếng cần xử lý thêm

D. No; punctuation remains — Không, dấu câu vẫn còn

Đáp án: D

Giải thích: Đổi chữ thường và xử lý dấu câu là hai bước khác nhau. Dấu câu có thể còn dính vào token cho đến khi có bước làm sạch/tách riêng.

Q17. Does Jaccard understand semantic synonyms? — Jaccard theo Set có hiểu từ đồng nghĩa không?

A. No, it compares literal token identities — Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document — Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies — Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation — Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: A

Giải thích: Hai từ khác chữ dù đồng nghĩa vẫn có thể không chung token. Jaccard đo trùng mặt chữ; muốn so ý nghĩa cần biểu diễn ngôn ngữ phong phú hơn.

Q18. What does len(A|B) represent? — len(A|B) biểu thị gì?

A. No, it compares literal token identities — Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document — Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies — Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation — Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: B

Giải thích: Hợp là mẫu số của Jaccard trên Set. Khi hợp khác rỗng, đây là mẫu số; chỉ dùng len(A) có thể khiến điểm sai.

Q19. What is lost when building a vocabulary as a Set? — Tạo vocabulary bằng Set mất thông tin nào?

A. No, it compares literal token identities — Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document — Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies — Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation — Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: C

Giải thích: Set chỉ lưu sự có mặt, không lưu vị trí hay lượt xuất hiện. Khi cần đếm số lần hoặc giữ thứ tự, hãy dùng Counter hoặc List token.

Q20. When should a specialized tokenizer be considered? — Khi nào nên xem bộ tách từ chuyên biệt?

A. No, it compares literal token identities — Không, nó so token theo mặt chữ

B. Number of distinct tokens in either document — Số token khác nhau ở ít nhất một tài liệu

C. Word order and term frequencies — Thứ tự và tần suất từ

D. For complex punctuation or multilingual segmentation — Khi có dấu câu phức tạp hoặc phân đoạn đa ngôn ngữ

Đáp án: D

Giải thích: Tokenizer theo ngôn ngữ xử lý ranh giới vượt quá khoảng trắng. Chọn bộ tách phù hợp ngôn ngữ và dùng thống nhất lúc lập chỉ mục lẫn lúc truy vấn.

8. Liên hệ bài kế tiếp

`String → Set → Tokenization → Vector hóa văn bản.`

PY-43 — Git: quản lý phiên bản và hợp tác trên mã nguồn

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Repository — kho mã nguồn; **Commit** — bản ghi thay đổi.

`File/Project → git init/add/commit → branch → pull/push → Review`

Học xong dùng để làm gì? Lưu lịch sử thay đổi code bằng commit, tạo nhánh thử nghiệm và khôi phục phiên bản chạy tốt.

Kết nối kiến thức tiếp theo: PY-44 đóng gói chức năng thao tác dữ liệu thành CLI có kiểm tra đầu vào.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Repository	kho mã nguồn	Nơi lưu lịch sử phiên bản
Commit	bản ghi thay đổi	Điểm lưu thay đổi có thông điệp
Branch	nhánh	Hướng phát triển độc lập
Remote	kho từ xa	Đường dẫn đến repository cộng tác

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`git status`	Xem file được sửa và đã stage hay chưa
`git add <file> / git commit -m`	Chọn thay đổi và tạo lịch sử có thông điệp
`git switch -c <branch>`	Tách nhánh thử nghiệm để không sửa nhánh chính
`git pull --ff-only / git push`	Đồng bộ an toàn khi lịch sử cho phép, kiểm tra remote trước

4. Khái niệm chính và liên quan

Mạch kiến thức: Git theo dõi lịch sử của một repository. Ta chọn thay đổi để stage, ghi chúng vào commit, phát triển độc lập trên branch và dùng remote để đồng bộ với người khác. Bốn khái niệm này tạo thành một quy trình liền mạch.

4.1. Repository — Nơi lưu mã nguồn và lịch sử

Từ vựng trong mục: Repository.

Repository là thư mục dự án được Git theo dõi; bên trong `.git` lưu thông tin commit, nhánh và tham chiếu. Ví dụ bài tập có `lesson.py`. Sau khi chạy `git init`, Git chưa tự đưa mọi file vào commit; `git status` chỉ cho biết trạng thái so với commit gần nhất và khu vực stage. Lệnh dưới đây dành cho terminal trong thư mục dự án đã khởi tạo Git.

```
git status
```

Kết quả có thể quan sát: Trạng thái hiển thị phụ thuộc file đang sửa và các commit hiện có; xem phần Changes not staged, Staged, Untracked.

4.2. Commit — Bản chụp các thay đổi đã stage

Từ vựng trong mục: Commit.

Commit ghi lại một trạng thái của những file đã **stage** vào lịch sử cùng tác giả, thời gian và thông điệp. Quy trình `git add lesson.py` đưa bản đang chọn vào staging area, sau đó `git commit -m ...` tạo commit. Nếu sửa file thêm **sau khi stage**, phần sửa mới chưa tự nằm trong commit. `git log -1 --oneline` xem commit gần nhất; mã hash phụ thuộc repository.

```
git log -1 --oneline
```

Kết quả có thể quan sát: Lệnh in mã commit rút gọn và thông điệp của bản gần nhất nếu dự án có commit; không có một mã cố định.

4.3. Branch — Nhánh để phát triển song song

Từ vựng trong mục: Branch.

Branch là một **tham chiếu đến commit**, giúp làm tính năng riêng mà không làm thay đổi ngay `main`. `git switch -c feature/score` tạo nhánh mới và chuyển sang nó từ commit hiện tại; code mới chỉ có trên nhánh đó cho tới khi được merge. Để tránh nhầm, kiểm tra `git branch --show-current` trước khi làm; tên nhánh này là ví dụ, không phải chính sách của mọi nhóm.

```
git switch -c feature/score
git branch --show-current
```

Kết quả có thể quan sát: Sau khi tạo thành công, lệnh sau in `feature/score`; lần chạy tiếp sẽ báo nhánh đã tồn tại.

4.4. Remote — Liên kết đến kho Git ở nơi khác

Từ vựng trong mục: Remote.

Remote là một tên tham chiếu đến địa chỉ repository khác, thường tên `origin` dùng cho kho trên GitHub. `git remote -v` liệt kê địa chỉ fetch/push đang cấu hình; nó không tự chứng minh đã đăng nhập hay có quyền push. `git fetch` tải tham chiếu từ remote, `git pull --ff-only` chỉ nhận thay đổi khi có thể đi thẳng không tạo merge commit, còn `git push` gửi commit nếu có quyền.

```
git remote -v
```

Kết quả có thể quan sát: Lệnh hiển thị tên và URL remote đã cấu hình; repository không có remote sẽ không in dòng nào.

5. Năm tình huống ứng dụng

Tình huống 1 — Kiểm file trước khi commit

Từ vựng trong mục: Git — quản lý phiên bản; **Terminal** — cửa sổ lệnh.

Tình huống thực tế: Học viên vừa chỉnh `lesson.py` trong repository Git của bài tập và muốn biết file đang thay đổi hay đã được stage. Thao tác phải không sửa file và không tạo commit.

Vì sao chọn cách giải này? `git status` chỉ đọc trạng thái làm việc và staging area, an toàn để kiểm trước khi lưu.

Quy trình giải:

1. Mở terminal tại root repository.
2. Chạy `git status` và đọc phần Changes not staged / Changes to be committed.
3. Nếu thấy file ngoài dự kiến, kiểm tra trước khi stage.

```
git status
```

Kết quả có thể quan sát: Thông báo trạng thái phụ thuộc repository; lệnh không tự lưu thay đổi.

Tình huống 2 — Chỉ lưu script đã sửa

Từ vựng trong mục: Git — quản lý phiên bản; **Terminal** — cửa sổ lệnh.

Tình huống thực tế: Dự án có `lesson.py` mới được chỉnh và các file khác chưa sẵn sàng. Học viên chỉ muốn tạo commit chứa thay đổi của `lesson.py`.

Vì sao chọn cách giải này? Stage cụ thể file trước để tránh commit nhầm thay đổi ngoài phạm vi.

Quy trình giải:

1. Chạy `git diff -- lesson.py` xem nội dung thay đổi.
2. Stage `lesson.py`, kiểm `git diff --cached -- lesson.py`.
3. Commit với thông điệp ngắn rồi kiểm lịch sử.

```
git add lesson.py
git diff --cached -- lesson.py
git commit -m "lesson: update practice"
```

Kết quả có thể quan sát: Nếu file có thay đổi staged, commit được tạo; mã hash và thống kê file phụ thuộc repository.

Tình huống 3 — Tạo nhánh thử thuật toán mới

Từ vựng trong mục: Git — quản lý phiên bản; **Terminal** — cửa sổ lệnh.

Tình huống thực tế: Nhóm thực hành muốn làm bản thử PCA mà vẫn giữ `main` ổn định; dự án hiện đang ở nhánh main và tên `experiment/pca` chưa tồn tại.

Vì sao chọn cách giải này? `git switch -c` tạo nhánh riêng từ vị trí hiện tại và chuyển sang nhánh đó.

Quy trình giải:

1. Kiểm tra nhánh đang đứng.
2. Tạo và chuyển nhánh thử với tên chưa tồn tại.
3. Kiểm tên nhánh hiện tại trước khi sửa code.

```
git switch -c experiment/pca
git branch --show-current
```

Kết quả có thể quan sát: Lệnh thứ hai in `experiment/pca` nếu tạo nhánh thành công.

Tình huống 4 — Đọc năm commit gần nhất

Từ vựng trong mục: Git — quản lý phiên bản; **Terminal** — cửa sổ lệnh.

Tình huống thực tế: Repository bài học đã có ít nhất năm commit. Người phụ trách muốn xem những thay đổi gần nhất để tìm mã commit cần đối chiếu.

Vì sao chọn cách giải này? `git log --oneline -5` hiển thị commit rút gọn theo thời gian, không thay đổi code.

Quy trình giải:

1. Vào đúng repository.
2. Chạy log tối đa năm commit.
3. Đọc hash và thông điệp để chọn bản cần kiểm tra.

```
git log --oneline -5
```

Kết quả có thể quan sát: In tối đa năm dòng hash và thông điệp; giá trị thay đổi theo lịch sử thật.

Tình huống 5 — So nội dung chưa stage

Từ vựng trong mục: Git — quản lý phiên bản; **Terminal** — cửa sổ lệnh.

Tình huống thực tế: Một sinh viên mới sửa `lesson.py` nhưng chưa stage. Trước khi commit cần xem chính xác dòng nào bị thêm hoặc xóa.

Vì sao chọn cách giải này? `git diff -- lesson.py` so working tree với phần đã stage, tránh đưa nhầm sửa đổi vào lịch sử.

Quy trình giải:

1. Đảm bảo file đang có thay đổi chưa stage.
2. Chạy `git diff` giới hạn theo đường dẫn.
3. Đọc các dòng dấu +/− rồi quyết định có stage không.

```
git diff -- lesson.py
```

Kết quả có thể quan sát: Hiển thị diff nếu có phần chưa stage; không in gì nếu không có khác biệt tương ứng.

6. Sai lầm thường gặp

- `git add` chỉ stage, không tạo commit; `git commit` chỉ ghi nội dung đã stage.
- `git diff` xem phần chưa stage, `git diff --cached` xem phần đã stage.
- Tạo nhánh khác không có nghĩa đã đẩy nhánh lên remote; push cần có quyền truy cập.
- Đường dẫn remote có thể tồn tại dù chưa có quyền push hoặc chưa xác thực.
- `git pull --ff-only` từ chối trường hợp phải merge/rebase; không nên `reset --hard` chỉ để bỏ thông báo lệch lịch sử.
- Đầu ra các lệnh Git phụ thuộc repository và trạng thái; không được coi hash ví dụ là kết quả của dự án khác.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What does a Git repository contain? — Repository Git chứa gì?

A. Project content and version history metadata — Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content — Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit — Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL — Tên tham chiếu đến URL repository khác

Đáp án: A

Giải thích: Git lưu trạng thái và tham chiếu trong `.git`, không chỉ gom file vào thư mục. Nó là phạm vi quản lý lịch sử dự án, không phải một commit hay nhánh riêng lẻ.

Q2. What is a commit? — Commit là gì?

A. Project content and version history metadata — Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content — Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit — Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL — Tên tham chiếu đến URL repository khác

Đáp án: B

Giải thích: Commit lưu nội dung đã stage cùng mã định danh và thông điệp. Chỉnh file mà chưa stage không tự làm thay đổi nội dung của commit kế.

Q3. What is a branch? — Nhánh Git là gì?

A. Project content and version history metadata — Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content — Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit — Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL — Tên tham chiếu đến URL repository khác

Đáp án: C

Giải thích: Tạo nhánh không sao chép vật lý toàn bộ file sang thư mục khác. Nhánh dịch chuyển khi có commit mới, còn commit đã tạo là một bản ghi bất biến.

Q4. What is a remote? — Remote là gì?

A. Project content and version history metadata — Nội dung dự án và siêu dữ liệu lịch sử phiên bản

B. A recorded snapshot of staged content — Bản ghi trạng thái từ nội dung đã stage

C. A movable reference to a commit — Tham chiếu di chuyển trỏ đến commit

D. A named reference to another repository URL — Tên tham chiếu đến URL repository khác

Đáp án: D

Giải thích: Remote chỉ định địa chỉ đồng bộ; quyền truy cập được kiểm riêng. Đổi URL origin không tự xác thực người dùng hay tạo commit ở remote.

Q5. What does `git status` show? — `git status` hiển thị gì?

A. Untracked, staged and working changes — File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path — Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message — Lưu phần đã stage thành commit có thông điệp

D. Compact commit history — Lịch sử commit rút gọn

Đáp án: A

Giải thích: Lệnh chỉ báo trạng thái, không tạo commit. Hãy xem status trước add/commit để phát hiện file chưa theo dõi hoặc chưa stage.

Q6. What does `git add file.py` do? — `git add file.py` làm gì?

A. Untracked, staged and working changes — File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path — Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message — Lưu phần đã stage thành commit có thông điệp

D. Compact commit history — Lịch sử commit rút gọn

Đáp án: B

Giải thích: Stage chuẩn bị cho commit, không phải tải lên remote. Sau `git add`, xem `git diff --cached` để chắc chỉ những phần mong muốn được đưa vào stage.

Q7. What does `git commit -m ...` do? — `git commit -m ...` làm gì?

A. Untracked, staged and working changes — File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path — Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message — Lưu phần đã stage thành commit có thông điệp

D. Compact commit history — Lịch sử commit rút gọn

Đáp án: C

Giải thích: Sau commit vẫn có thể còn sửa đổi chưa stage. Sửa đổi không nằm trong index sẽ không thuộc commit cho tới khi stage lại.

Q8. What does `git log --oneline` show? — `git log --oneline` hiển thị gì?

A. Untracked, staged and working changes — File chưa theo dõi, đã stage và đang sửa

B. Stages current changes to that path — Stage thay đổi hiện tại của đường dẫn đó

C. Records staged state with a message — Lưu phần đã stage thành commit có thông điệp

D. Compact commit history — Lịch sử commit rút gọn

Đáp án: D

Giải thích: Hiển thị hash và tiêu đề, không phải diff chi tiết. Muốn xem thay đổi từng dòng phải dùng diff hoặc show, không chỉ đọc tiêu đề log.

Q9. What does `git switch -c name` do? — `git switch -c name` làm gì?

A. Creates and checks out a new branch — Tạo và chuyển sang nhánh mới

B. Name of current branch — Tên nhánh hiện tại

C. Unstaged differences for that path — Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit — Khác biệt đã stage so với commit gần nhất

Đáp án: A

Giải thích: Nhánh bắt đầu từ commit hiện tại nếu không chỉ định mốc khác. Nếu tên đã tồn tại thì tạo nhánh mới sẽ báo lỗi; khi đó cần chuyển nhánh thay vì tạo tiếp.

Q10. What does `git branch --show-current` return? — `git branch --show-current` trả gì?

A. Creates and checks out a new branch — Tạo và chuyển sang nhánh mới

B. Name of current branch — Tên nhánh hiện tại

C. Unstaged differences for that path — Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit — Khác biệt đã stage so với commit gần nhất

Đáp án: B

Giải thích: Giúp tránh commit trên nhánh không mong muốn. Nếu đang ở detached HEAD, có thể không có tên nhánh hiện tại để in.

Q11. What does `git diff -- file` show? — `git diff -- file` hiển thị gì?

A. Creates and checks out a new branch — Tạo và chuyển sang nhánh mới

B. Name of current branch — Tên nhánh hiện tại

C. Unstaged differences for that path — Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit — Khác biệt đã stage so với commit gần nhất

Đáp án: C

Giải thích: diff thường so working tree với staging area. Với nội dung đã stage, `git diff` thường không hiện; phải xem diff cached.

Q12. What does `git diff --cached` show? — `git diff --cached` hiển thị gì?

A. Creates and checks out a new branch — Tạo và chuyển sang nhánh mới

B. Name of current branch — Tên nhánh hiện tại

C. Unstaged differences for that path — Khác biệt chưa stage của đường dẫn đó

D. Staged differences against last commit — Khác biệt đã stage so với commit gần nhất

Đáp án: D

Giải thích: Giúp xem nội dung đã stage trước khi commit. Nó cho biết nội dung sẽ vào commit, khác phần mới chỉ đang sửa ngoài working tree.

Q13. What does `git fetch` do? — `git fetch` làm gì?

A. Downloads remote refs and objects without merging — Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted — Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration — Để tách công việc trước khi tích hợp

D. The index of content selected for next commit — Chỉ mục nội dung được chọn cho commit kế

Đáp án: A

Giải thích: Fetch cập nhật thông tin remote nhưng chưa nhập vào nhánh đang làm. Fetch có thể cập nhật nhánh theo dõi remote nhưng không nhập chúng vào nhánh local đang đứng.

Q14. What does `git push` do? — `git push` làm gì?

A. Downloads remote refs and objects without merging — Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted — Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration — Để tách công việc trước khi tích hợp

D. The index of content selected for next commit — Chỉ mục nội dung được chọn cho commit kế

Đáp án: B

Giải thích: Push không tự stage hoặc commit thay đổi chưa lưu. Các sửa đổi chưa commit phải được ghi vào lịch sử trước khi push chia sẻ bình thường.

Q15. Why use a feature branch? — Vì sao dùng nhánh tính năng?

A. Downloads remote refs and objects without merging — Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted — Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration — Để tách công việc trước khi tích hợp

D. The index of content selected for next commit — Chỉ mục nội dung được chọn cho commit kế

Đáp án: C

Giải thích: Tách nhánh cho phép xem thử trước khi ảnh hưởng main. Chỉ nên merge nhánh tính năng sau khi kiểm tra đúng phần thay đổi dự định.

Q16. What is staging area? — Staging area là gì?

A. Downloads remote refs and objects without merging — Tải tham chiếu/dữ liệu remote nhưng chưa merge

B. Sends local refs and commits to remote if permitted — Gửi tham chiếu/commit lên remote nếu có quyền

C. To isolate work before integration — Để tách công việc trước khi tích hợp

D. The index of content selected for next commit — Chỉ mục nội dung được chọn cho commit kế

Đáp án: D

Giải thích: Nội dung đã stage có thể khác sửa đổi mới trong working tree. Index tách phần được ghi vào commit với những sửa đổi khác vẫn còn ở máy.

Q17. What does `git pull --ff-only` require? — `git pull --ff-only` đòi hỏi gì?

A. A fast-forward update without a merge commit — Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate — Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again — Không, cần stage lại phần sửa mới

D. Review status and staged diff — Xem status và diff đã stage

Đáp án: A

Giải thích: Nó từ chối lịch sử cần tích hợp kiểu không fast-forward. Lịch sử đã phân nhánh cần quyết định tích hợp riêng; ff-only không tự xóa commit.

Q18. Does setting origin grant push permissions? — Đặt origin có tự cấp quyền push không?

A. A fast-forward update without a merge commit — Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate — Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again — Không, cần stage lại phần sửa mới

D. Review status and staged diff — Xem status và diff đã stage

Đáp án: B

Giải thích: Có thể cấu hình URL dù không có quyền ghi. Quyền do máy chủ và thông tin đăng nhập quyết định, không phải do tên remote trên máy.

Q19. Can git commit include edits made after the last git add automatically? — git commit có tự chứa sửa đổi sau lần git add cuối không?

A. A fast-forward update without a merge commit — Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate — Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again — Không, cần stage lại phần sửa mới

D. Review status and staged diff — Xem status và diff đã stage

Đáp án: C

Giải thích: Commit lưu trạng thái index, có thể cũ hơn nội dung đang sửa. Stage lại sẽ chụp phiên bản mới; nếu không commit dùng bản đã stage cũ.

Q20. How should a developer check changes before committing? — Nên kiểm thay đổi trước commit bằng cách nào?

A. A fast-forward update without a merge commit — Có thể tiến thẳng lịch sử mà không tạo merge commit

B. No; authentication and permissions are separate — Không; xác thực và quyền là chuyện khác

C. No, they need to be staged again — Không, cần stage lại phần sửa mới

D. Review status and staged diff — Xem status và diff đã stage

Đáp án: D

Giải thích: Kiểm tra giúp phát hiện file hoặc dòng thay đổi ngoài ý muốn. Nếu diff đã stage có bí mật hoặc file ngoài phạm vi, hãy bỏ stage hoặc sửa trước khi commit.

8. Nối sang bài tiếp

`File/Project → git init/add/commit → branch → pull/push → Review`.

PY-44 — Ứng dụng CLI: CRUD, kiểm tra dữ liệu và thống kê hồ sơ

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: CRUD — tạo/đọc/sửa/xóa; **Validation** — xác minh dữ liệu.

`Dictionary/Function → CRUD → CLI → File/CSV → Data App`

Học xong dùng để làm gì? Tạo công cụ thêm/đọc/sửa/xóa dữ liệu từ terminal, từ chối đầu vào sai và báo lỗi rõ ràng.

Kết nối kiến thức tiếp theo: PY-45 trực quan hóa dữ liệu đầu vào và kết quả để người dùng kiểm tra.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
CRUD	tạo/đọc/sửa/xóa	Bốn thao tác quản lý bản ghi
Validation	xác minh dữ liệu	Đảm bảo giá trị đúng giới hạn
Lookup	tra cứu	Tìm bản ghi qua khóa
Aggregate	tổng hợp	Tính chỉ số từ nhiều hồ sơ

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`dict.get(id)`	Đọc bản ghi và trả mặc định nếu thiếu
`dict[id] = record`	Thêm/sửa bản ghi khi đã kiểm tra khóa
`dict.pop(id,default)`	Xóa có xử lý mã chưa tồn tại
`sum()/max()/min()/sorted()`	Tính thống kê và sắp xếp hồ sơ

4. Khái niệm chính và liên quan

Mạch kiến thức: Một CLI nhận yêu cầu người dùng để tạo, đọc, sửa, xóa hồ sơ. Các thao tác CRUD cần Validation trước khi ghi, Lookup để tìm đúng mã và Aggregate để tạo báo cáo. Ví dụ xuyên suốt dùng mã sinh viên làm khóa và điểm trên thang 10 làm trường số.

4.1. CRUD — Bốn thao tác trên hồ sơ

Từ vựng trong mục: CRUD — Create/Read/Update/Delete.

Ứng dụng quản lý sinh viên cần tạo, đọc, sửa và xóa hồ sơ. Với Dictionary, **khóa là mã sinh viên duy nhất**, giá trị là Dictionary trường `name`, `score`. `Create` cần kiểm trùng mã; `Read` tra cứu; `Update` chỉ sửa mã tồn tại; `Delete` xử lý trường hợp mã không tìm thấy. Một CLI dùng lệnh nhập văn bản để thực hiện các thao tác này; đây mới là dữ liệu trong RAM, chưa được lưu bền xuống tệp.

```
db = {}
db['S1'] = {'name': 'An', 'score': 8}
print(db['S1']['score'])
```

Kết quả khi chạy:

```
8
```

Bản ghi S1 được tạo trong RAM; lấy `score` của S1 cho kết quả 8/10.

4.2. Validation — Chặn dữ liệu sai trước khi ghi

Từ vựng trong mục: Validation — kiểm tra hợp lệ.

Validation phải dựa trên **quy tắc dữ liệu**, ví dụ điểm là số nằm trong `[0,10]`, mã không rỗng, mã sinh viên chưa tồn tại khi thêm mới. Nếu chỉ thử `0 <= score <= 10`, một chuỗi nhập từ CLI phải được đổi sang số có xử lý lỗi trước; không so chuỗi với số. Kiểm tra ràng buộc **trước** khi ghi để tránh ghi sai rồi mới phát hiện.

```
score = 8
print(isinstance(score, (int, float)) and 0 <= score <= 10)
```

Kết quả khi chạy:

```
True
```

Điểm 8 là kiểu số và nằm trong thang điểm 0 đến 10.

4.3. Lookup — Tìm bản ghi theo mã

Từ vựng trong mục: Lookup — tra cứu theo khóa.

Với Dictionary `db`, `db.get('S1')` trả hồ sơ S1 nếu tồn tại, còn `db.get('S3')` trả `None` theo mặc định. Lệnh `db['S3']` sẽ ném `KeyError` khi thiếu mã. Trong CLI, việc không có hồ sơ cần được phản hồi rõ, không đánh đồng với một hồ sơ thật có điểm 0. Nếu cần thông báo thay vì `None`, truyền đối số mặc định cho `get`.

```
db = {'S1': {'score': 8}}
print(db.get('S3', 'Not found'))
```

Kết quả khi chạy:

```
Not found
```

Mã S3 không có trong Dictionary nên in thông báo thay vì gây KeyError.

4.4. Aggregate — Tóm tắt nhiều bản ghi

Từ vựng trong mục: Aggregate — tổng hợp.

Tổng hợp biến nhiều hồ sơ thành một con số hay báo cáo: ví dụ mean của ba điểm `[7,8,9]` bằng 8 và điểm cao nhất bằng 9. Trước khi tính cần lấy giá trị số từ trường `score`, không phải mã ID. Với danh sách rỗng, phép `sum(scores)/len(scores)` sẽ chia cho 0; chương trình phải xử lý trường hợp không có hồ sơ.

```
scores = [7, 8, 9]
print(sum(scores) / len(scores), max(scores))
```

Kết quả khi chạy:

```
8.0 9
```

Điểm trung bình bằng 8/10, điểm cao nhất bằng 9/10 trên ba hồ sơ.

5. Năm tình huống ứng dụng

Tình huống 1 — Thêm sinh viên không ghi đè mã cũ

Từ vựng trong mục: Record — bản ghi; **ID** — mã định danh.

Tình huống thực tế: Trung tâm muốn tạo hồ sơ cho sinh viên S1 tên An, điểm 8/10. Kho hiện rỗng. Chỉ thêm nếu mã chưa tồn tại và điểm thuộc 0–10.

Vì sao chọn cách giải này? Kiểm tra cả tính duy nhất của khóa và phạm vi điểm trước khi tạo bản ghi.

Quy trình giải:

1. Tạo Dictionary rỗng, mã S1, điểm 8.
2. Kiểm mã chưa có và điểm hợp lệ.
3. Thêm hồ sơ rồi in dữ liệu đã lưu.

```
db = {}
student_id = 'S1'
score = 8
if student_id not in db and 0 <= score <= 10:
    db[student_id] = {'name': 'An', 'score': score}
print(db)
```

Kết quả khi chạy:

```
{'S1': {'name': 'An', 'score': 8}}
```

Đã thêm một hồ sơ S1, giữ điểm theo thang 10.

Tình huống 2 — Thông báo khi mã không tồn tại

Từ vựng trong mục: Record — bản ghi; **ID** — mã định danh.

Tình huống thực tế: Danh sách chỉ có S1 điểm 8 và S2 điểm 9; nhân viên tìm mã S3. Chương trình cần báo không tìm thấy thay vì dừng vì lỗi KeyError.

Vì sao chọn cách giải này? `dict.get` cho phép định nghĩa giá trị mặc định khi thiếu mã.

Quy trình giải:

1. Tạo kho hai sinh viên.
2. Tra khóa S3 bằng `get` kèm thông báo mặc định.
3. In kết quả tra cứu.

```
db = {'S1': {'score': 8}, 'S2': {'score': 9}}
print(db.get('S3', 'Not found'))
```

Kết quả khi chạy:

```
Not found
```

Hồ sơ S3 không tồn tại; không có bản ghi nào bị thay đổi.

Tình huống 3 — Sửa điểm sau khi kiểm thang

Từ vựng trong mục: Record — bản ghi; **ID** — mã định danh.

Tình huống thực tế: Hồ sơ S1 có điểm 7/10, người quản lý duyệt sửa lên 9/10. Chỉ cập nhật nếu mã tồn tại và điểm mới hợp lệ.

Vì sao chọn cách giải này? Kiểm tra trước khi Update tránh vô tình tạo hồ sơ mới hoặc lưu điểm ngoài phạm vi.

Quy trình giải:

1. Tạo bản ghi ban đầu S1 điểm 7.
2. Kiểm S1 đang tồn tại và 9 nằm trong thang.
3. Cập nhật điểm, in bản ghi sau sửa.

```
db = {'S1': {'score': 7}}
new_score = 9
if 'S1' in db and 0 <= new_score <= 10:
    db['S1']['score'] = new_score
print(db)
```

Kết quả khi chạy:

```
{'S1': {'score': 9}}
```

Điểm đã đổi 7→9/10; đây là Update, không tạo sinh viên thứ hai.

Tình huống 4 — Xóa mã sinh viên thiếu an toàn

Từ vựng trong mục: Record — bản ghi; **ID** — mã định danh.

Tình huống thực tế: Kho chỉ có S1, nhân viên yêu cầu xóa S2 vốn không tồn tại. Chương trình phải báo không có bản ghi, không ném lỗi hoặc xóa S1.

Vì sao chọn cách giải này? `pop(key,default)` xử lý được nhánh không tìm thấy và không xóa khóa khác.

Quy trình giải:

1. Khởi tạo kho chỉ có S1.
2. Gọi `pop('S2',None)`.
3. In kết quả và kiểm kho không thay đổi.

```
db = {'S1': {'score': 8}}
print(db.pop('S2', None))
print(db)
```

Kết quả khi chạy:

```
None
{'S1': {'score': 8}}
```

Không có S2 để xóa nên trả None; S1 được giữ nguyên.

Tình huống 5 — Tổng hợp kết quả của một lớp

Từ vựng trong mục: Record — bản ghi; **ID** — mã định danh.

Tình huống thực tế: Lớp có ba điểm 7,8,9 theo thang 10; giáo viên muốn in mean và điểm tối đa để báo cáo nội bộ.

Vì sao chọn cách giải này? Tính các chỉ số trực tiếp từ điểm số, thay vì lấy trung bình mã sinh viên hoặc chuỗi tên.

Quy trình giải:

1. Tạo List gồm ba điểm.
2. Tính mean bằng tổng chia cỡ nhóm và max.
3. In Dictionary báo cáo có hai chỉ số.

```
scores = [7, 8, 9]
report = {'mean': sum(scores) / len(scores), 'max': max(scores)}
print(report)
```

Kết quả khi chạy:

```
{'mean': 8.0, 'max': 9}
```

Trung bình lớp 8 điểm; cao nhất là 9 điểm.

6. Sai lầm thường gặp

- Thêm mã đã có bằng `dict[id]=...` sẽ ghi đè dữ liệu cũ nếu không kiểm trùng.
- Điểm từ CLI có thể là chuỗi; phải parse với xử lý ValueError trước khi so với số.

- `get()` khác `dict[key]` khi thiếu khóa; `pop` có default khác xóa trực tiếp.
- Update cần kiểm mã tồn tại và giới hạn điểm; xóa mã thiếu không được làm mất dữ liệu khác.
- Tính mean cần kiểm List không rỗng và chỉ tính trên trường số thực sự.
- Dictionary tồn tại trong RAM; chương trình thực tế cần lưu file/database và cân nhắc sao lưu/quyền truy cập.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What are the four CRUD operations? — Bốn thao tác CRUD là gì?

A. Create, Read, Update, Delete — Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing — Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record — Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records — Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: A

Giải thích: CRUD mô tả vòng đời bản ghi, không phải một lệnh cơ sở dữ liệu. Nếu coi thêm mới như ghi đè mà không có chính sách, dữ liệu sinh viên cũ có thể bị mất.

Q2. What is input validation? — Kiểm tra dữ liệu đầu vào là gì?

A. Create, Read, Update, Delete — Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing — Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record — Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records — Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: B

Giải thích: Validation chặn điểm sai hoặc mã trùng trước khi ghi. Validation kiểm cả kiểu dữ liệu và giới hạn nghiệp vụ, không chỉ kiểm code chạy được.

Q3. What is a lookup key? — Khóa tra cứu là gì?

A. Create, Read, Update, Delete — Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing — Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record — Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records — Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: C

Giải thích: Mã sinh viên dùng làm khóa duy nhất, còn điểm là giá trị bản ghi. Nên dùng mã sinh viên duy nhất và ổn định làm khóa, không dùng điểm có thể thay đổi.

Q4. What is aggregation? — Aggregate là gì?

A. Create, Read, Update, Delete — Tạo, đọc, sửa, xóa

B. Checking constraints before processing or storing — Kiểm ràng buộc trước khi xử lý hoặc lưu

C. A field identifying the desired record — Trường xác định bản ghi cần tìm

D. Computing summaries from multiple records — Tính chỉ số tổng hợp từ nhiều bản ghi

Đáp án: D

Giải thích: Trung bình và cực đại tổng hợp trường số qua nhiều học viên. Với lớp rỗng, cần quy định kết quả 'không có dữ liệu' trước khi tính thống kê số.

Q5. What does `db.get('S1')` do? — `db.get('S1')` làm gì?

A. Returns S1 value or None if missing — Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value — Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent — Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list — Số điểm trong danh sách

Đáp án: A

Giải thích: get cho phép tra thiếu khóa mà không ném KeyError. `get` chỉ đọc; việc nhận giá trị mặc định không làm tạo hồ sơ còn thiếu.

Q6. What does `db['S1']=record` do? — `db['S1']=record` làm gì?

A. Returns S1 value or None if missing — Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value — Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent — Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list — Số điểm trong danh sách

Đáp án: B

Giải thích: Không kiểm khóa tồn tại, phép gán có thể ghi đè hồ sơ. Hàm thêm mới nên từ chối mã đã tồn tại, không chỉ dùng phép gán.

Q7. What does `db.pop('S2',None)` do? — `db.pop('S2',None)` làm gì?

A. Returns S1 value or None if missing — Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value — Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent — Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list — Số điểm trong danh sách

Đáp án: C

Giải thích: Giá trị mặc định tránh lỗi KeyError khi khóa vắng mặt. Nếu không có giá trị mặc định, pop khóa S2 còn thiếu sẽ ném KeyError.

Q8. What does `len(scores)` provide? — `len(scores)` cung cấp gì?

A. Returns S1 value or None if missing — Trả giá trị S1 hoặc None nếu thiếu

B. Inserts or overwrites the key's value — Thêm hoặc ghi đè giá trị của khóa

C. Removes S2 or returns None if absent — Xóa S2 hoặc trả None nếu thiếu

D. Number of scores in the list — Số điểm trong danh sách

Đáp án: D

Giải thích: Mean chia tổng cho số phần tử khi số lượng lớn hơn 0. Số lượng bằng 0 là ca biên, cần kiểm trước khi chia tổng.

Q9. What is a safe score validation for integer score 8? — Kiểm tra điểm 8 hợp lệ trên thang 0–10 thế nào?

A. `0 <= score <= 10` — `0 <= score <= 10`

B. Confirm the key does not exist — Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid — Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError — Lỗi KeyError

Đáp án: A

Giải thích: Cần kiểm cả cận dưới và cận trên. Nếu điểm đến từ chuỗi CLI bất kỳ, cần xác minh/đổi kiểu trước khi so sánh.

Q10. What is required before adding a unique student ID? — Trước khi thêm mã sinh viên duy nhất cần gì?

A. `0 <= score <= 10` — `0 <= score <= 10`

B. Confirm the key does not exist — Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid — Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError — Lỗi KeyError

Đáp án: B

Giải thích: Nhờ đó tránh ghi đè âm thầm hồ sơ khác. Kiểm khóa trước khi ghi; ghi đè rồi mới phát hiện có thể không khôi phục tự động được.

Q11. What is required before updating a student record? — Trước khi sửa hồ sơ cần gì?

A. `0 <= score <= 10` — `0 <= score <= 10`

B. Confirm the key does not exist — Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid — Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError — Lỗi KeyError

Đáp án: C

Giải thích: Sửa không nên vô tình tạo khóa mới. Nhờ đó vừa cập nhật đúng người vừa giữ điểm trong thang hợp lệ.

Q12. What happens with `db['missing']` for an absent key? — Truy cập `db['missing']` khi thiếu khóa sẽ sao?

A. `0 <= score <= 10` — `0 <= score <= 10`

B. Confirm the key does not exist — Xác nhận khóa chưa tồn tại

C. Confirm the key exists and new values are valid — Xác nhận khóa có và giá trị mới hợp lệ

D. KeyError — Lỗi KeyError

Đáp án: D

Giải thích: Truy cập trực tiếp gây lỗi khi khóa không tồn tại. Cần phân biệt khóa không tồn tại với học viên tồn tại nhưng điểm bằng 0.

Q13. What does `sum([7,8,9])/3` equal? — `sum([7,8,9])/3` bằng gì?

A. 8.0 — 8.0

B. 9 — 9

C. Check length before dividing — Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself — Không, tự nó không giữ qua lúc khởi động lại

Đáp án: A

Giải thích: Tổng bằng 24 chia ba bằng 8. Cả ba điểm có trọng số bằng nhau; kết quả không phải điểm cao nhất hay tổng điểm.

Q14. What does `max([7,8,9])` return? — `max([7,8,9])` trả gì?

A. 8.0 — 8.0

B. 9 — 9

C. Check length before dividing — Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself — Không, tự nó không giữ qua lúc khởi động lại

Đáp án: B

Giải thích: max chọn điểm lớn nhất. Phép này chọn điểm cao nhất, chưa tự xác định mã sinh viên nếu không giữ liên hệ bản ghi.

Q15. How should an empty score list be handled before mean? — Trước khi tính mean với List điểm rỗng cần gì?

A. 8.0 — 8.0

B. 9 — 9

C. Check length before dividing — Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself — Không, tự nó không giữ qua lúc khởi động lại

Đáp án: C

Giải thích: Nếu không kiểm, chia cho len=0 sẽ gây lỗi. Chương trình cần nêu cách xử lý lớp rỗng, không bịa ra một điểm trung bình.

Q16. What does a Python Dictionary persist automatically? — Dictionary Python tự lưu bền dữ liệu không?

A. 8.0 — 8.0

B. 9 — 9

C. Check length before dividing — Kiểm số phần tử trước khi chia

D. No, not across process restarts by itself — Không, tự nó không giữ qua lúc khởi động lại

Đáp án: D

Giải thích: Hồ sơ trong RAM cần lưu ra file hoặc database để tồn tại lâu dài. Muốn dữ liệu còn sau khi mở lại, phải lưu bằng JSON, SQLite hoặc cơ chế lưu bền khác.

Q17. What is a duplicate ID risk in CRUD? — Nguy cơ mã trùng trong CRUD là gì?

A. Existing data may be overwritten — Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling — Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled — Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values — Các giá trị điểm dạng số

Đáp án: A

Giải thích: Kiểm tra khóa duy nhất tránh thay nhầm. Nên từ chối mã trùng hoặc dùng lệnh Update riêng nếu chủ ý sửa.

Q18. What should CLI do with a non-numeric score input? — CLI nên làm gì với điểm nhập chữ thay vì số?

A. Existing data may be overwritten — Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling — Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled — Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values — Các giá trị điểm dạng số

Đáp án: B

Giải thích: Chuỗi nhập từ terminal cần đổi số trước khi so ngưỡng. Hãy đổi kiểu an toàn, xử lý ValueError rồi mới kiểm tra thang điểm.

Q19. When is `dict.get` preferable to square indexing? — Khi nào dùng `dict.get` thay `dict[key]`?

A. Existing data may be overwritten — Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling — Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled — Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values — Các giá trị điểm dạng số

Đáp án: C

Giải thích: get trả mặc định thay cho ném KeyError. Khi chắc chắn mọi khóa đều có, truy cập [] phù hợp; nếu có thể thiếu thì `get` an toàn hơn.

Q20. Which fields should an average score computation use? — Tính điểm trung bình nên dùng trường nào?

A. Existing data may be overwritten — Dữ liệu cũ có thể bị ghi đè

B. Reject or safely parse with error handling — Từ chối hoặc đổi kiểu có xử lý lỗi

C. When missing keys are expected and handled — Khi dự kiến thiếu khóa và cần xử lý

D. Numeric score values — Các giá trị điểm dạng số

Đáp án: D

Giải thích: ID và tên để nhận diện, không đưa vào phép trung bình. Chỉ tổng hợp trường điểm dạng số, thống nhất thang đo và cách xử lý ô thiếu.

8. Nối sang bài tiếp

`Dictionary/Function → CRUD → CLI → File/CSV → Data App.`

PY-45 — Matplotlib: biểu đồ line, scatter và histogram

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Line chart — biểu đồ đường; **Scatter plot** — biểu đồ phân tán.

`NumPy/Pandas → Visualization → EDA → PCA/ML`

Học xong dùng để làm gì? Dùng biểu đồ thể hiện phân phối, xu hướng và so sánh, ghi nhãn trục/đơn vị để đọc đúng kết quả.

Kết nối kiến thức tiếp theo: PY-46 tích hợp dữ liệu, xử lý, mô hình và trực quan hóa trong một dự án nhỏ.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Line chart	biểu đồ đường	Biểu diễn thay đổi theo thứ tự thời gian
Scatter plot	biểu đồ phân tán	Mỗi cặp x,y tạo thành một điểm quan sát
Histogram	biểu đồ tần suất	Đếm số mẫu trong từng khoảng giá trị
Axis label	nhãn trục	Mô tả đại lượng và đơn vị trên trục

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`plt.plot(x,y)`	Đường thay đổi theo thời gian; x,y có cùng số phần tử
`plt.scatter(x,y)`	Vẽ điểm từng mẫu từ hai đại lượng
`plt.hist(values,bins=...)`	Gom quan sát vào các khoảng (bins)
`plt.xlabel()/ylabel()/savefig()`	Ghi nhãn kèm đơn vị và lưu hình

4. Khái niệm chính và liên quan

Mạch kiến thức: Khi đã có bảng số liệu, chọn biểu đồ phù hợp câu hỏi: line cho mốc có thứ tự, scatter cho cặp biến trên mỗi mẫu, histogram cho phân bố một biến số. Nhãn trục nối dữ liệu với ý nghĩa và đơn vị để người đọc không suy luận sai.

4.1. Line chart — Nối các mốc có thứ tự

Từ vựng trong mục: Line chart — biểu đồ đường.

Biểu đồ đường nối những điểm theo **thứ tự x**, thích hợp quan sát biến thiên doanh thu theo ngày. Ví dụ ba ngày 1,2,3 có doanh thu 10,15,12 nghìn đồng: trục x là ngày, trục y là nghìn đồng. Nối điểm cho thấy tăng rồi giảm; không có nghĩa tồn tại dữ liệu ở mọi thời điểm giữa hai ngày. `plt.plot(x,y)` yêu cầu hai dãy có cùng số phần tử.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [10, 15, 12])
print(len(plt.gca().lines[0].get_xdata()))
plt.close()
```

Kết quả khi chạy:

```
3
```

Biểu đồ đã nhận ba điểm, rồi đóng Figure; số 3 là số mốc, không phải doanh thu.

4.2. Scatter — Mỗi hàng là một cặp quan sát

Từ vựng trong mục: Scatter plot — biểu đồ phân tán.

Biểu đồ scatter đặt **mỗi mẫu thành một điểm (x,y)**. Ví dụ ba sinh viên học 2,4,6 giờ/tuần, có điểm 5,7,9: mỗi vị trí x ghép đúng điểm y của cùng học viên. Scatter giúp thấy liên hệ, nhưng không tự chứng minh học nhiều gây điểm cao. `plt.scatter` tạo các điểm riêng không nối đường như line chart.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.scatter([2, 4, 6], [5, 7, 9])
print(len(plt.gca().collections[0].get_offsets()))
plt.close()
```

Kết quả khi chạy:

```
3
```

Đã vẽ ba điểm đại diện ba sinh viên, giữ đúng cặp giờ học–điểm.

4.3. Histogram — Đếm số đo rơi vào các khoảng

Từ vựng trong mục: Histogram — biểu đồ tần suất; Bin — khoảng.

Histogram dùng **một dãy số** và chia trục giá trị thành các **bins**, rồi đếm số quan sát trong mỗi khoảng. Sáu điểm `[5,6,6,7,8,9]` chia bốn bins đều từ 5 đến 9 tạo tần suất `[1,2,1,2]`. Khác biểu đồ cột cho các loại (A,B,C), histogram phù hợp biến số liên tục hoặc có thứ tự giá trị và phụ thuộc độ rộng bins.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

counts, edges, bars = plt.hist([5, 6, 6, 7, 8, 9], bins=[5, 6, 7, 8, 10])
print(counts.astype(int).tolist())
plt.close()
```

Kết quả khi chạy:

```
[1, 2, 1, 2]
```

Bốn khoảng chứa lần lượt 1, 2, 1, 2 quan sát; khoảng cuối bao gồm cận phải.

4.4. Axis labels — Ghi rõ biến và đơn vị

Từ vựng trong mục: Axis label — nhãn trục; Figure — hình.

Nhãn trục cho biết đang đo gì: `Time (hours)` ở trục ngang và `Temperature (°C)` ở trục dọc. Không ghi đơn vị, người xem dễ nhầm giờ với phút hoặc độ C với độ F. `xlabel`/`ylabel` chỉ đổi chữ trên Figure, không tự đổi dữ liệu. Có thể lưu hình bằng `savefig` rồi đóng `close` để không giữ Figure trong bộ nhớ.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.plot([1, 2], [22, 24])
plt.xlabel('Time (hours)')
plt.ylabel('Temperature (C)')
print(plt.gca().get_xlabel())
plt.close()
```

Kết quả khi chạy:

```
Time (hours)
```

Trục x được gắn đúng ý nghĩa thời gian; phải đặt cả y và đơn vị trước khi gửi biểu đồ.

5. Năm tình huống ứng dụng

Tình huống 1 — Theo dõi doanh thu ba ngày

Từ vựng trong mục: Figure — hình biểu đồ; **Axis** — trục.

Tình huống thực tế: Cửa hàng ghi ngày 1,2,3 và doanh thu 10,15,12 nghìn đồng, một giá trị mỗi ngày. Muốn tạo biểu đồ đường để thấy chiều biến động và kiểm số điểm vẽ.

Vì sao chọn cách giải này? Biểu đồ đường phù hợp chuỗi có thứ tự thời gian; nhãn y nêu nghìn đồng, không chỉ số thô.

Quy trình giải:

1. Tạo hai List ngày và doanh thu cùng thứ tự.

2. Vẽ line chart, đặt tên hai trục.
3. In số điểm dữ liệu rồi đóng Figure.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

days = [1, 2, 3]
revenue = [10, 15, 12]
plt.plot(days, revenue)
plt.xlabel('Day')
plt.ylabel('Revenue (thousand VND)')
print(len(revenue))
plt.close()
```

Kết quả khi chạy:

```
3
```

Biểu đồ có ba mốc; doanh thu tăng ngày 2 rồi giảm ngày 3.

Tình huống 2 — So số giờ học và điểm kiểm tra

Từ vựng trong mục: Figure — hình biểu đồ; **Axis** — trục.

Tình huống thực tế: Ba học viên học 2,4,6 giờ/tuần và đạt điểm 5,7,9/10 tương ứng. Trung tâm muốn xem mối liên hệ qua ba điểm độc lập.

Vì sao chọn cách giải này? Scatter ghép mỗi giờ học với điểm của cùng người mà không giả định đường nối theo thời gian.

Quy trình giải:

1. Tạo ba cặp x=giờ/tuần và y=điểm/10.
2. Vẽ scatter, ghi rõ trục.
3. In số điểm, đóng hình để giải phóng bộ nhớ.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

hours = [2, 4, 6]
scores = [5, 7, 9]
plt.scatter(hours, scores)
plt.xlabel('Hours per week')
plt.ylabel('Score (out of 10)')
print(len(hours))
plt.close()
```

Kết quả khi chạy:

```
3
```

Ba điểm mô tả ba học viên; không thể suy ra quan hệ nhân quả từ ba điểm.

Tình huống 3 — Đếm phân bố điểm theo khoảng

Từ vựng trong mục: Figure — hình biểu đồ; **Axis** — trục.

Tình huống thực tế: Giáo viên có sáu điểm `[5,6,6,7,8,9]` trên thang 10. Cần đếm số học viên rơi vào bốn khoảng điểm định trước.

Vì sao chọn cách giải này? Histogram đếm theo bins; cần công bố biên khoảng để người đọc hiểu tần suất.

Quy trình giải:

1. Xác định bốn khoảng [5,6), [6,7), [7,8), [8,10].
2. Vẽ histogram và nhận số mẫu mỗi bin.
3. In tần suất rồi đóng Figure.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

scores = [5, 6, 6, 7, 8, 9]
counts, edges, bars = plt.hist(scores, bins=[5, 6, 7, 8, 10])
print(counts.astype(int).tolist())
plt.close()
```

Kết quả khi chạy:

```
[1, 2, 1, 2]
```

Bốn dải điểm lần lượt có 1, 2, 1, 2 học viên.

Tình huống 4 — Lưu biểu đồ thành PNG

Từ vựng trong mục: Figure — hình biểu đồ; **Axis** — trục.

Tình huống thực tế: Nhân viên lập báo cáo cần tạo đồ thị hai tháng doanh thu 10 và 15 nghìn đồng và lưu hình trong một thư mục tạm, không bật cửa sổ màn hình.

Vì sao chọn cách giải này? Backend Agg và `savefig` cho phép tạo PNG không cần tương tác; kiểm tệp trước khi thoát thư mục tạm.

Quy trình giải:

1. Tạo Figure đường cho hai tháng.
2. Lưu PNG trong TemporaryDirectory.
3. Kiểm tệp tồn tại và đóng hình.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from pathlib import Path
from tempfile import TemporaryDirectory

with TemporaryDirectory() as folder:
    plt.plot([1, 2], [10, 15])
    path = Path(folder) / 'chart.png'
```

```
    plt.savefig(path)
    print(path.is_file())
    plt.close()
```

Kết quả khi chạy:

```
True
```

File chart.png đã được tạo trong thư mục tạm và sẽ được xóa khi rời khối with.

Tình huống 5 — Kiểm nhãn trên biểu đồ nhiệt độ

Từ vựng trong mục: Figure — hình biểu đồ; **Axis** — trục.

Tình huống thực tế: Ba thời điểm 1,2,3 giờ tương ứng 22,24,23 °C. Bộ phận kỹ thuật muốn người xem hiểu trục x là giờ và trục y là nhiệt độ Celsius.

Vì sao chọn cách giải này? `xlabel` và `ylabel` làm rõ biến/đơn vị thay vì để trục chỉ ghi số không có ngữ cảnh.

Quy trình giải:

1. Vẽ ba điểm nhiệt độ theo thời gian.
2. Gắn nhãn Time (hours), Temperature (C).
3. In hai nhãn để đối chiếu rồi đóng hình.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [22, 24, 23])
plt.xlabel('Time (hours)')
plt.ylabel('Temperature (C)')
print(plt.gca().get_xlabel(), plt.gca().get_ylabel())
plt.close()
```

Kết quả khi chạy:

```
Time (hours) Temperature (C)
```

Hai trục có tên và đơn vị nhất quán với dữ liệu đầu vào.

6. Sai lầm thường gặp

- Dùng line chart để nối dữ liệu không có thứ tự có thể tạo cảm giác xu hướng giả.
- Scatter phải ghép đúng x và y của **cùng hàng mẫu**; không suy ra nhân quả chỉ từ mối tương quan.
- Histogram phụ thuộc `bins`; ghi biên khoảng khi báo cáo.
- Biểu đồ thiếu đơn vị dễ khiến người xem nhầm thời gian, tiền hoặc nhiệt độ.
- Lưu PNG không thay thế việc lưu bảng dữ liệu gốc và script tạo hình.
- `matplotlib` là thư viện ngoài; với máy không có màn hình có thể dùng backend Agg và `close()` sau khi vẽ.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a line chart best suited for? — Biểu đồ đường phù hợp gì?

A. Showing changes across ordered positions — Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations — Từng quan sát theo cặp x-y

C. Counts of numeric values within bins — Số quan sát số học trong từng khoảng

D. Naming variables and their units — Đặt tên đại lượng và đơn vị đo

Đáp án: A

Giải thích: Nối các mốc cho xu hướng, không đảm bảo có số đo giữa các mốc. Nếu dữ liệu là các loại không có thứ tự, biểu đồ cột phù hợp hơn việc nối đường.

Q2. What does a scatter plot represent? — Scatter plot biểu diễn gì?

A. Showing changes across ordered positions — Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations — Từng quan sát theo cặp x-y

C. Counts of numeric values within bins — Số quan sát số học trong từng khoảng

D. Naming variables and their units — Đặt tên đại lượng và đơn vị đo

Đáp án: B

Giải thích: Mỗi điểm là x và y của cùng một bản ghi. Mảng x và y phải cùng thứ tự bản ghi, nếu không các điểm ghép sẽ sai.

Q3. What does a histogram represent? — Histogram biểu diễn gì?

A. Showing changes across ordered positions — Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations — Từng quan sát theo cặp x-y

C. Counts of numeric values within bins — Số quan sát số học trong từng khoảng

D. Naming variables and their units — Đặt tên đại lượng và đơn vị đo

Đáp án: C

Giải thích: Histogram chia khoảng giá trị số, không giống biểu đồ cột loại. Đổi biên bins sẽ đổi số đếm; khi so histogram cần nêu rõ khoảng chia.

Q4. What are axis labels for? — Nhãn trục để làm gì?

A. Showing changes across ordered positions — Thể hiện thay đổi theo các mốc có thứ tự

B. Individual paired x-y observations — Từng quan sát theo cặp x-y

C. Counts of numeric values within bins — Số quan sát số học trong từng khoảng

D. Naming variables and their units — Đặt tên đại lượng và đơn vị đo

Đáp án: D

Giải thích: Nhãn tránh nhầm đơn vị thời gian, nhiệt độ hay tiền. Con số trên trục thiếu đơn vị khiến người xem không rõ ý nghĩa đo lường.

Q5. Which function draws a line? — Hàm nào vẽ đường?

A. `plt.plot(x,y)` — `plt.plot(x,y)`

B. `plt.scatter(x,y)` — `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` — `plt.hist(values,bins=...)`

D. `plt.savefig(path)` — `plt.savefig(path)`

Đáp án: A

Giải thích: plot nối điểm theo thứ tự đầu vào mặc định. Hàm vẽ đòi x,y tương ứng nhưng không tự kiểm dữ liệu có đúng ý nghĩa thực tế.

Q6. Which function draws separate x-y points? — Hàm nào vẽ các điểm x-y riêng?

A. `plt.plot(x,y)` — `plt.plot(x,y)`

B. `plt.scatter(x,y)` — `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` — `plt.hist(values,bins=...)`

D. `plt.savefig(path)` — `plt.savefig(path)`

Đáp án: B

Giải thích: scatter tạo dấu chấm riêng cho từng cặp quan sát. Đây là lựa chọn phù hợp khi các điểm là quan sát rời, không phải chuỗi thời gian cần nối.

Q7. Which function draws histograms? — Hàm nào vẽ histogram?

A. `plt.plot(x,y)` — `plt.plot(x,y)`

B. `plt.scatter(x,y)` — `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` — `plt.hist(values,bins=...)`

D. `plt.savefig(path)` — `plt.savefig(path)`

Đáp án: C

Giải thích: hist đếm trong từng khoảng rồi vẽ cột các khoảng số. `bins` xác định biên hoặc số khoảng; thay đổi tham số này làm thay đổi bản tóm tắt phân bố.

Q8. Which function saves a figure? — Hàm nào lưu hình?

A. `plt.plot(x,y)` — `plt.plot(x,y)`

B. `plt.scatter(x,y)` — `plt.scatter(x,y)`

C. `plt.hist(values,bins=...)` — `plt.hist(values,bins=...)`

D. `plt.savefig(path)` — `plt.savefig(path)`

Đáp án: D

Giải thích: savefig ghi hình đã vẽ ra tệp PNG. Phải lưu trước khi đóng Figure và kiểm đường dẫn cùng quyền ghi.

Q9. Why should x and y have matching observation counts? — Vì sao x và y phải có số quan sát tương ứng?

A. Each x needs its matching y — Mỗi x cần y tương ứng

B. Closes the current Figure — Đóng Figure hiện tại

C. Horizontal-axis label — Nhãn trục ngang

D. Vertical-axis label — Nhãn trục dọc

Đáp án: A

Giải thích: Không ghép được cặp khi thiếu phần tử tương ứng. Dãy lệch chiều có thể gây lỗi hoặc ghép sai đối tượng quan sát.

Q10. What does `plt.close()` do? — `plt.close()` làm gì?

A. Each x needs its matching y — Mỗi x cần y tương ứng

B. Closes the current Figure — Đóng Figure hiện tại

C. Horizontal-axis label — Nhãn trục ngang

D. Vertical-axis label — Nhãn trục dọc

Đáp án: B

Giải thích: Đóng giúp giải phóng tài nguyên và tránh mở nhiều hình. Nên đóng Figure sau khi xuất, nhất là trong vòng lặp tạo nhiều tệp.

Q11. What does `plt.xlabel()` set? — `plt.xlabel()` đặt gì?

A. Each x needs its matching y — Mỗi x cần y tương ứng

B. Closes the current Figure — Đóng Figure hiện tại

C. Horizontal-axis label — Nhãn trục ngang

D. Vertical-axis label — Nhãn trục dọc

Đáp án: C

Giải thích: Nó chỉ đổi chữ, không tự đổi dữ liệu hoặc đơn vị. Nhãn có ích nên ghi cả đại lượng và đơn vị như ngày hay giờ.

Q12. What does `plt.ylabel()` set? — `plt.ylabel()` đặt gì?

A. Each x needs its matching y — Mỗi x cần y tương ứng

B. Closes the current Figure — Đóng Figure hiện tại

C. Horizontal-axis label — Nhãn trục ngang

D. Vertical-axis label — Nhãn trục dọc

Đáp án: D

Giải thích: Nó mô tả y và nên kèm đơn vị. Trục nhiệt độ nên ghi rõ Celsius hoặc Fahrenheit, không chỉ ghi 'value'.

Q13. How many markers represent three scatter records? — Ba bản ghi scatter tạo mấy điểm?

A. Three — Ba

B. Yes, changing bins changes counts per interval — Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer — Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient — Không, chỉ liên hệ chưa đủ

Đáp án: A

Giải thích: Mỗi hàng tạo một điểm. Mỗi điểm tương ứng một sinh viên, không phải mỗi cột đặc trưng tạo một điểm.

Q14. Do histogram counts depend on bin boundaries? — Số đếm histogram có phụ thuộc biên bins không?

A. Three — Ba

B. Yes, changing bins changes counts per interval — Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer — Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient — Không, chỉ liên hệ chưa đủ

Đáp án: B

Giải thích: Cùng dữ liệu có thể được nhóm khác nhau. Chiều cao histogram thường là số lượng mẫu, không mặc nhiên là xác suất nếu chưa chuẩn hóa.

Q15. Is a histogram appropriate for names A, B, C as categories? — Histogram có thích hợp trực tiếp cho nhãn A,B,C không?

A. Three — Ba

B. Yes, changing bins changes counts per interval — Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer — Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient — Không, chỉ liên hệ chưa đủ

Đáp án: C

Giải thích: Histogram cần khoảng số thay vì tên loại. Cột phân loại so các nhóm tên; histogram dùng khoảng liên tục trên trục số.

Q16. Can scatter points establish causation? — Scatter có chứng minh quan hệ nhân quả không?

A. Three — Ba

B. Yes, changing bins changes counts per interval — Có, đổi bins sẽ đổi số đếm theo khoảng

C. No, a categorical bar chart is clearer — Không, biểu đồ cột theo nhãn rõ hơn

D. No, association alone is insufficient — Không, chỉ liên hệ chưa đủ

Đáp án: D

Giải thích: Biến khác hoặc cách lấy mẫu có thể tạo cùng xu hướng. Cần xem cách thiết kế nghiên cứu và yếu tố gây nhiễu trước khi nói một biến gây ra biến kia.

Q17. What does backend Agg help with? — Backend Agg giúp gì?

A. Rendering images without a GUI window — Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis — Thứ tự và đơn vị trục thời gian

C. Array of bin counts — Mảng số lượng theo mỗi bin

D. No, it is a rendered image — Không, nó là ảnh được kết xuất

Đáp án: A

Giải thích: Agg phù hợp chạy script hàng loạt hoặc lưu hình trên server. Agg đặc biệt hữu ích cho kiểm thử tự động không cần màn hình đồ họa.

Q18. What should be checked before interpreting a time plot? — Trước khi đọc biểu đồ thời gian cần kiểm gì?

A. Rendering images without a GUI window — Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis — Thứ tự và đơn vị trục thời gian

C. Array of bin counts — Mảng số lượng theo mỗi bin

D. No, it is a rendered image — Không, nó là ảnh được kết xuất

Đáp án: B

Giải thích: Sai thứ tự hoặc đơn vị khiến xu hướng bị hiểu sai. Nếu ngày bị xếp sai, đường nối có thể khiến người xem tưởng thời gian đi ngược.

Q19. What does `plt.hist` return first? — `plt.hist` trả gì ở vị trí đầu?

A. Rendering images without a GUI window — Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis — Thứ tự và đơn vị trục thời gian

C. Array of bin counts — Mảng số lượng theo mỗi bin

D. No, it is a rendered image — Không, nó là ảnh được kết xuất

Đáp án: C

Giải thích: Giá trị trả về đầu là tần suất, phần sau là biên và đối tượng vẽ. Mảng biên trả về giúp xác định giá trị nào đóng góp vào từng khoảng đếm.

Q20. Does a saved PNG retain an editable dataset automatically? — PNG lưu hình có tự chứa bảng dữ liệu có thể sửa không?

A. Rendering images without a GUI window — Kết xuất hình không cần cửa sổ GUI

B. Order and units of the time axis — Thứ tự và đơn vị trục thời gian

C. Array of bin counts — Mảng số lượng theo mỗi bin

D. No, it is a rendered image — Không, nó là ảnh được kết xuất

Đáp án: D

Giải thích: Muốn tính lại và sửa biểu đồ cần lưu dữ liệu gốc riêng. Cần lưu script, nhãn và bảng số gốc để tái tạo lại biểu đồ khi cần.

8. Nối sang bài tiếp

`NumPy/Pandas → Visualization → EDA → PCA/ML.`

PY-46 — Mini project: dữ liệu → làm sạch → mô hình → đánh giá

1. Vị trí trong chuỗi kiến thức

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn.

`Python → NumPy/Pandas → Train/Test → Pipeline → Report`

Học xong dùng để làm gì? Kết hợp chuẩn bị dữ liệu, đặc trưng, huấn luyện mô hình, đánh giá và báo cáo thành một quy trình AI có thể chạy lại.

Kết nối kiến thức tiếp theo: Sau bài cuối, áp dụng toàn bộ chuỗi vào bài toán thực tế tự chọn: xác định dữ liệu, đặt tiêu chí đánh giá và lưu kết quả có thể tái tạo.

2. English vocabulary — Từ vựng

English	Tiếng Việt	Ý nghĩa
Feature	đặc trưng	Cột đầu vào cho mô hình
Label	nhãn	Giá trị mục tiêu cần dự đoán
Pipeline	chuỗi xử lý	Gom tiền xử lý và mô hình thành các bước
Evaluation	đánh giá	Đo kết quả trên dữ liệu chưa dùng để fit

3. Hàm và cách sử dụng

Từ vựng trong mục: Function — hàm; **Option** — tùy chọn.

Cú pháp	Mục đích và lựa chọn
`pd.DataFrame()/dropna()`	Tạo bảng và đánh giá xử lý giá trị thiếu
`train_test_split(...,stratify=y)`	Chia dữ liệu theo nhãn, tránh leakage
`make_pipeline(StandardScaler(),LogisticRegression())`	Fit scaler/model trên train trong một chuỗi
`model.score(X_test,y_test)`	Đánh giá tập test, không suy từ điểm train

4. Khái niệm chính và liên quan

Mạch kiến thức: Một dự án AI bắt đầu từ việc hiểu mỗi hàng mẫu, cột đặc trưng và nhãn mục tiêu. Dữ liệu được làm sạch theo đúng quy tắc, chia train/test **trước khi học tham số xử lý**, huấn luyện bằng Pipeline rồi đánh giá trên nhãn test. Kết quả mô hình cần đọc cùng giới hạn, không chỉ nhìn một con số accuracy.

4.1. Feature và label — Đầu vào khác mục tiêu

Từ vựng trong mục: Feature — đặc trưng; Label — nhãn.

Trong mini-project phân loại hoa Iris, một **hàng** ứng với một bông hoa; bốn **cột feature** là chiều dài/rộng đài hoa và cánh hoa, đơn vị cm. **Label** (y) là mã loài 0,1,2 cần dự đoán. X có shape `(150,4)`, y có 150 nhãn. Không đưa cột chứa nhãn hoặc thông tin được biết sau kết quả vào X, vì đó là rò rỉ mục tiêu.

```
from sklearn.datasets import load_iris

data = load_iris()
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

Mỗi trong 150 hoa có bốn số đo dùng dự đoán một trong ba loài.

4.2. Cleaning — Xử lý ô khuyết đúng thứ tự

Từ vựng trong mục: Cleaning — làm sạch; Imputation — điền thiếu.

Trước khi huấn luyện cần kiểm kiểu dữ liệu, giá trị thiếu, lỗi nhập và đơn vị. Nếu thiếu số đo, có thể điền bằng **median của tập train**, nhưng **không** tính median từ cả train và test. Ví dụ ba chiều dài (cm) `[5,None,7]`, median từ hai giá trị có thực là 6; điền None bằng 6. Đây là giá trị ước lượng, không phải số đo gốc.

```
import pandas as pd

values = pd.Series([5.0, None, 7.0])
print(values.fillna(values.median()).tolist())
```

Kết quả khi chạy:

```
[5.0, 6.0, 7.0]
```

Giá trị thiếu được thay bằng trung vị 6 cm của hai số đã quan sát.

4.3. Pipeline — Gắn tiền xử lý với mô hình

Từ vựng trong mục: Pipeline — chuỗi xử lý; Fit — học tham số.

Pipeline ghép các bước `SimpleImputer` → `StandardScaler` → `LogisticRegression`. Khi `.fit(X_train,y_train)`, bước điền thiếu và chuẩn hóa **học tham số từ train**, mô hình sau đó học nhãn. Khi `.predict(X_test)`, các phép biến đổi dùng lại tham số train. Điều này hạn chế rò rỉ test do tính mean/median hay scale trên toàn bảng trước khi chia.

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
model = make_pipeline(
    SimpleImputer(strategy='median'),
    StandardScaler(),
    LogisticRegression(max_iter=1000),
)
print(len(model.steps))
```

Kết quả khi chạy:

```
3
```

Pipeline gồm ba bước nối tiếp: điền thiếu, chuẩn hóa, phân loại; chưa gọi fit nên chưa có mô hình đã học.

4.4. Evaluation — Đánh giá trên dữ liệu chưa fit

Từ vựng trong mục: Evaluation — đánh giá; Confusion matrix — ma trận nhầm lẫn.

Đánh giá phân loại phải so nhãn dự đoán với nhãn thật của **test giữ riêng**. **Accuracy** là số dự đoán đúng chia tổng số mẫu, nhưng có thể gây hiểu sai nếu một lớp rất phổ biến. **Confusion matrix** ghi số mẫu theo **hàng = nhãn thật**, **cột = nhãn dự đoán**; ô ngoài đường chéo cho biết những lỗi nhầm lớp cụ thể.

```
from sklearn.metrics import confusion_matrix

actual = [0, 1, 1, 0]
predicted = [0, 1, 0, 0]
print(confusion_matrix(actual, predicted).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [1, 1]]
```

Lớp thật 0 có hai mẫu dự đoán đúng; lớp thật 1 có một mẫu nhầm sang 0 và một mẫu đúng.

5. Năm tình huống ứng dụng

Tình huống 1 — Kiểm cấu trúc dữ liệu trước khi mô hình hóa

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Test set** — tập đánh giá.

Tình huống thực tế: Nhóm đào tạo dùng dữ liệu Iris công khai có 150 hoa, bốn số đo cm mỗi hoa và nhãn thuộc ba loài. Trước khi viết mô hình, cần xác nhận mỗi hoa có đủ cột đặc trưng và một nhãn.

Vì sao chọn cách giải này? Kiểm `shape` ngay từ đầu để tránh ghép sai chiều dữ liệu với vector nhãn.

Quy trình giải:

1. Nạp Iris và lấy X, y.
2. In shape của X và y.
3. Xác nhận 150 hàng X khớp 150 phần tử y.

```
from sklearn.datasets import load_iris
```

```
data = load_iris()
print(data.data.shape, data.target.shape)
```

Kết quả khi chạy:

```
(150, 4) (150,)
```

Có 150 mẫu và bốn cột đặc trưng mỗi mẫu, tương ứng 150 nhãn.

Tình huống 2 — Giữ tỷ lệ lớp khi chia train/test

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Test set** — tập đánh giá.

Tình huống thực tế: Iris có ba loài, mỗi loài 50 hoa. Cần chia 25% test, đảm bảo mô hình không thấy dữ liệu test trong lúc fit và tỷ lệ nhãn gần được giữ.

Vì sao chọn cách giải này? `train_test_split` với `stratify` giảm nguy cơ tỷ lệ lớp thay đổi ngẫu nhiên ở tập test.

Quy trình giải:

1. Nạp Iris và chia theo y với test_size=0.25, seed 42.
2. Giữ riêng X_train, X_test, y_train, y_test.
3. In shape hai ma trận đặc trưng.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42,
)
print(X_train.shape, X_test.shape)
```

Kết quả khi chạy:

```
(112, 4) (38, 4)
```

112 hoa train và 38 hoa test, vẫn giữ bốn cột cm.

Tình huống 3 — Huấn luyện pipeline chống rò rỉ

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Test set** — tập đánh giá.

Tình huống thực tế: Nhóm kỹ thuật muốn học phân loại loài Iris từ bốn số đo cm, có quy tắc điền giá trị thiếu, chuẩn hóa và Logistic Regression. Tập test 25% không được dùng để fit bước tiền xử lý.

Vì sao chọn cách giải này? Pipeline áp cùng các biến đổi đã học từ train cho test, không chuẩn hóa cả 150 mẫu trước khi chia.

Quy trình giải:

1. Chia Iris thành train/test theo nhãn.

2. Tạo pipeline imputer → scaler → classifier và fit trên train.
3. Dự đoán X_test và in số nhãn được trả về.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42,
)
model = make_pipeline(
    SimpleImputer(strategy='median'), StandardScaler(),
    LogisticRegression(max_iter=1000),
)
model.fit(X_train, y_train)
print(len(model.predict(X_test)))
```

Kết quả khi chạy:

```
38
```

Pipeline đưa ra một dự đoán lớp cho mỗi mẫu test, nhưng chưa đủ để nói độ chính xác.

Tình huống 4 — Đọc nhầm lẫn giữa các lớp

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Test set** — tập đánh giá.

Tình huống thực tế: Một mô hình dự đoán bốn trường hợp có nhãn thật `[0,1,1,0]` và dự đoán `[0,1,0,0]`. Cần biết lớp nào bị nhầm chứ không chỉ accuracy 3/4.

Vì sao chọn cách giải này? Ma trận nhầm lẫn đặt nhãn thật theo hàng, dự đoán theo cột; ô ngoài đường chéo biểu thị lỗi.

Quy trình giải:

1. Lưu hai danh sách nhãn cùng thứ tự quan sát.
2. Tạo `confusion_matrix` rồi đổi sang List.
3. In bảng 2×2, đọc từng ô theo hàng thật–cột dự đoán.

```
from sklearn.metrics import confusion_matrix

actual = [0, 1, 1, 0]
predicted = [0, 1, 0, 0]
print(confusion_matrix(actual, predicted).tolist())
```

Kết quả khi chạy:

```
[[2, 0], [1, 1]]
```

Một mẫu lớp 1 bị đoán nhầm thành 0; ba trong bốn mẫu dự đoán đúng.

Tình huống 5 — So baseline và PCA đúng nguyên tắc

Từ vựng trong mục: Feature — đặc trưng; **Label** — nhãn; **Test set** — tập đánh giá.

Tình huống thực tế: Trên Iris, nhóm thực hành muốn so một baseline dùng bốn đặc trưng với mô hình PCA giảm còn hai thành phần. Cần xác định shape trước/sau trên **train**, tránh fit PCA lên toàn dữ liệu test.

Vì sao chọn cách giải này? PCA phải fit trên train trong pipeline; hai thành phần là đặc trưng được rút ra, không phải nhãn loài.

Quy trình giải:

1. Chia Iris trước, rồi fit scaler và PCA trên X_train.
2. Biến đổi X_test bằng đúng hai bộ biến đổi đã fit.
3. In shape của train/test sau PCA để thấy số cột giảm còn 2.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25,
    stratify=data.target, random_state=42,
)
features = make_pipeline(StandardScaler(), PCA(n_components=2))
train_2d = features.fit_transform(X_train)
test_2d = features.transform(X_test)
print(train_2d.shape, test_2d.shape)
```

Kết quả khi chạy:

```
(112, 2) (38, 2)
```

PCA giảm bốn cột cm thành hai thành phần tổng hợp, giữ số hoa. Muốn so hiệu quả phải huấn luyện và đánh giá trên cùng split.

6. Sai lầm thường gặp

- X là cột đặc trưng, y là nhãn; không đưa nhãn hoặc kết quả biết sau sự kiện vào feature.
- Phải chia train/test trước khi học median, mean/std hoặc PCA để tránh rò rỉ.
- Không `fit_transform` PCA trên X_test; chỉ `transform` bằng PCA đã fit từ train.
- Accuracy cao vẫn có thể che lỗi lớp hiếm; đọc thêm confusion matrix theo hàng thật–cột dự đoán.
- Thay đổi split hoặc metric khi so hai phương pháp sẽ gây so sánh thiếu công bằng.
- Đánh giá trên test không phải huấn luyện; chỉnh siêu tham số dựa mãi vào test làm rò rỉ đánh giá.

7. English–Vietnamese multiple-choice questions — Trắc nghiệm song ngữ

Q1. What is a feature in supervised learning? — Feature trong học có giám sát là gì?

A. An input measurement used by the predictor — Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict — Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps — Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets — So dự đoán với nhãn thật giữ riêng

Đáp án: A

Giải thích: Trong Iris, bốn số đo cm nằm ở X, không phải nhãn y. Nếu đưa kết quả tương lai hoặc mã lớp vào X, đánh giá sẽ dễ giả tạo do rò rỉ.

Q2. What is a label? — Label là gì?

A. An input measurement used by the predictor — Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict — Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps — Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets — So dự đoán với nhãn thật giữ riêng

Đáp án: B

Giải thích: Classifier có giám sát học từ nhãn thật ghép với từng hàng train. Mỗi trong 150 hàng đặc trưng cần có nhãn tương ứng khi train.

Q3. What is a pipeline? — Pipeline là gì?

A. An input measurement used by the predictor — Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict — Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps — Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets — So dự đoán với nhãn thật giữ riêng

Đáp án: C

Giải thích: Nó fit bộ biến đổi trên train rồi tái sử dụng cho dữ liệu sau. Thứ tự rất quan trọng: điền thiếu và chuẩn hóa trước dự đoán, nhưng tham số chỉ học từ train.

Q4. What is evaluation? — Evaluation là gì?

A. An input measurement used by the predictor — Đại lượng đo đầu vào mô hình sử dụng

B. A target value the model should predict — Giá trị đích mô hình cần dự đoán

C. An ordered chain of preprocessing and model steps — Chuỗi tiền xử lý và mô hình có thứ tự

D. Checking predictions against held-out targets — So dự đoán với nhãn thật giữ riêng

Đáp án: D

Giải thích: Test cuối đánh giá dữ liệu chưa fit, không dùng lại mẫu học. Hãy dùng test giữ riêng không thay đổi; điểm train cao có thể chỉ là ghi nhớ dữ liệu.

Q5. What does an Iris feature row represent? — Một hàng feature Iris biểu diễn gì?

A. Four measurements for one flower — Bốn số đo của một bông hoa

B. The target class per sample — Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting — Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment — Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: A

Giải thích: Hàng là một mẫu hoa, cột là số đo như chiều dài cánh. Với shape (150,4), 150 là mẫu, bốn là biến đo theo cm.

Q6. What does y represent in classification? — y biểu diễn gì khi phân loại?

A. Four measurements for one flower — Bốn số đo của một bông hoa

B. The target class per sample — Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting — Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment — Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: B

Giải thích: Nhãn y phải ghép một-một với các hàng X. Nếu X và y sai thứ tự, thuật toán đúng vẫn học sai mối quan hệ dữ liệu–nhãn.

Q7. What is data leakage? — Rò rỉ dữ liệu là gì?

A. Four measurements for one flower — Bốn số đo của một bông hoa

B. The target class per sample — Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting — Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment — Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: C

Giải thích: Fit tiền xử lý trên test hoặc dùng kết quả tương lai có thể làm chỉ số ảo. Rò rỉ còn xảy ra nếu chọn đặc trưng từ toàn bộ dữ liệu trước khi chia.

Q8. Why inspect X.shape and y.shape first? — Vì sao kiểm X.shape và y.shape đầu tiên?

A. Four measurements for one flower — Bốn số đo của một bông hoa

B. The target class per sample — Lớp mục tiêu cho từng mẫu

C. Unavailable evaluation information entering fitting — Thông tin đánh giá đáng lẽ chưa biết lọt vào fit

D. To check sample/feature dimensions and alignment — Để kiểm chiều mẫu/đặc trưng và sự khớp nhãn

Đáp án: D

Giải thích: Số hàng X lệch số nhãn y khiến việc fit sai. Hàng và nhãn lệch nhau phải được sửa trước khi fit hoặc tính điểm.

Q9. What does train_test_split do? — train_test_split làm gì?

A. Separates data into fitting and held-out portions — Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts — Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features — Các đặc trưng train

D. Training features only — Chỉ đặc trưng train

Đáp án: A

Giải thích: Hàm chia dữ liệu, không tự học mô hình hay đo accuracy. Phải giữ cả X và y train/test; chia dữ liệu không tự tương đương huấn luyện.

Q10. Why use stratify=y? — Vì sao dùng stratify=y?

A. Separates data into fitting and held-out portions — Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts — Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features — Các đặc trưng train

D. Training features only — Chỉ đặc trưng train

Đáp án: B

Giải thích: Giúp giảm lệch lớp khi mỗi lớp có đủ mẫu. Phân tầng hữu ích khi lớp ít mẫu có nguy cơ vắng khỏi tập test.

Q11. What should SimpleImputer learn from? — SimpleImputer nên học từ đâu?

A. Separates data into fitting and held-out portions — Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts — Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features — Các đặc trưng train

D. Training features only — Chỉ đặc trưng train

Đáp án: C

Giải thích: Median tính từ toàn bộ dữ liệu làm lộ thống kê test. Hàng test không được tác động đến giá trị điền học được, kể cả khi hàng ấy thiếu dữ liệu.

Q12. What should StandardScaler learn from? — StandardScaler nên fit từ đâu?

A. Separates data into fitting and held-out portions — Chia dữ liệu thành phần học và phần giữ riêng

B. To preserve approximate class ratios in split parts — Để giữ tương đối tỷ lệ lớp trong các tập

C. Training features — Các đặc trưng train

D. Training features only — Chỉ đặc trưng train

Đáp án: D

Giải thích: Biến đổi test bằng mean/std train, không fit từ test. Mean và variance từ train phải được áp nhất quán lên mọi mẫu mới.

Q13. What does LogisticRegression produce in Iris? — LogisticRegression trên Iris dự đoán gì?

A. One of three discrete flower classes — Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions — Số dự đoán đúng chia tổng số dự đoán

C. Actual labels — Các nhãn thật

D. Predicted labels — Các nhãn dự đoán

Đáp án: A

Giải thích: Dù có chữ Regression, LogisticRegression dùng để phân loại. Mô hình có thể xuất xác suất lớp, nhưng nhiệm vụ đang xét chọn một nhãn rời rạc.

Q14. What does accuracy count? — Accuracy đếm gì?

A. One of three discrete flower classes — Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions — Số dự đoán đúng chia tổng số dự đoán

C. Actual labels — Các nhãn thật

D. Predicted labels — Các nhãn dự đoán

Đáp án: B

Giải thích: Accuracy hữu ích nhưng có thể che lỗi lớp hiếm. Với tập lệch lớp, accuracy cao có thể đến từ việc chỉ đoán lớp đông nhất.

Q15. What do confusion matrix rows mean? — Hàng của confusion matrix nghĩa là gì?

A. One of three discrete flower classes — Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions — Số dự đoán đúng chia tổng số dự đoán

C. Actual labels — Các nhãn thật

D. Predicted labels — Các nhãn dự đoán

Đáp án: C

Giải thích: Mỗi hàng đếm mẫu thuộc lớp thật tương ứng. Hàng đại diện nhãn thật; ô ngoài đường chéo là lỗi xuất phát từ lớp thật ở hàng đó.

Q16. What do confusion matrix columns mean? — Cột của confusion matrix nghĩa là gì?

A. One of three discrete flower classes — Một trong ba loài hoa rời rạc

B. Correct predictions divided by total predictions — Số dự đoán đúng chia tổng số dự đoán

C. Actual labels — Các nhãn thật

D. Predicted labels — Các nhãn dự đoán

Đáp án: D

Giải thích: Ô ngoài đường chéo là trường hợp dự đoán sang lớp khác. Cột cho biết lớp dự đoán nào nhận các mẫu bị gán nhầm.

Q17. What does PCA reduce? — PCA giảm gì?

A. Feature dimensionality while retaining sample rows — Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes — Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric — Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed — Không, phải đánh giá dữ liệu giữ riêng

Đáp án: A

Giải thích: Thành phần PCA là trục đặc trưng mới, không phải nhãn. Trục PCA là tổ hợp đặc trưng ban đầu được học, không làm đổi số mẫu.

Q18. Why fit PCA on train only? — Vì sao chỉ fit PCA trên train?

A. Feature dimensionality while retaining sample rows — Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes — Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric — Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed — Không, phải đánh giá dữ liệu giữ riêng

Đáp án: B

Giải thích: Test chỉ được transform bằng PCA đã fit từ train. Nếu không, thông tin phân phối test đã lọt vào biểu diễn đặc trưng.

Q19. How should baseline and PCA models be compared? — So baseline và mô hình PCA thế nào?

A. Feature dimensionality while retaining sample rows — Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes — Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric — Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed — Không, phải đánh giá dữ liệu giữ riêng

Đáp án: C

Giải thích: Khác split hoặc metric không tách được ảnh hưởng của PCA. Train/test phải giống nhau giữa hai cách để chênh lệch metric phản ánh thay đổi mô hình.

Q20. Does a high train score prove model generalization? — Train score cao có chứng minh tổng quát hóa?

A. Feature dimensionality while retaining sample rows — Số chiều đặc trưng nhưng giữ số hàng mẫu

B. To avoid using test distribution to learn axes — Tránh lấy phân phối test để học các trục

C. On the same held-out split and metric — Trên cùng tập test và chỉ số đánh giá

D. No, held-out evaluation is needed — Không, phải đánh giá dữ liệu giữ riêng

Đáp án: D

Giải thích: Mô hình có thể nhớ train nhưng dự đoán kém trên mẫu mới. Cần báo metric test và lỗi theo lớp trước khi đánh giá khả năng ứng dụng thực tế.

8. Nối sang bài tiếp

`Python → NumPy/Pandas → Train/Test → Pipeline → Report.`